



一个人，怎么假装自己有一支投资团队

未来已来

AI 驱动的量化交易

从本系统的智能体驱动、量化交易原理
到实际操作

QuantBot Lab 著

一份围绕真实运行系统写成的技术专著

开源仓库

github.com/jeokeo011222/quantbot

读者交流群（QQ） 179590131



本书仅供学习与研究使用，不构成任何投资建议

前言

一个很朴素的问题

这本书的开头，是一个人在电脑前发呆了很久之后问自己的一句话：

能不能让 AI 帮我做量化投资？

这句话听起来像是“能不能让 AI 帮我写作业”，但真做起来，问题完全不是“AI 聪明不聪明”。做了大半年之后我才慢慢明白，真正难的是下面这一串：

1. 我白天要上班，我看不到盘，我必须在开盘前就把今天该做的事安排好；
2. 我只有 10 万块钱，任何一个错误都可能让我几个月白干，所以“宁可不动”必须是一个合法选项；
3. 我不懂财报，也不懂盘口，我需要系统告诉我“它凭什么说这只股票好”；
4. 我最怕的不是亏钱，是它亏了钱，我还不知道它当时到底看到了什么；
5. 而最要紧的是：我不能接受它编数据。它要是不知道，它就说不知道。

于是我发现，我要造的不是一个“更聪明的模型”，而是一个会分工、有纪律、能留痕、会认输的系统。

这本书讲的就是这个系统：QuantBot——一个每天真的会自己去上班的 A 股量化交易系统。

为什么值得写成一本书

市面上讲 AI 投资的书，多半在两个极端之间摆动。一种是“AI 很神”，把大模型写成股神，读完你还是不知道该在哪儿按下第一个按钮；另一种是“AI 很危险”，把量化写成黑箱，读完你只剩下恐惧。

这两种写法都漏掉了一件事：**中间那一段工程。**

真正的难点，全都是中间那一段：

- 为什么必须分成五个角色，而不是一个？（第 1、2 章）
- 怎么给 AI 派活，它才不会胡编一个数字？（第 3 章）
- 模型说“买”，市场说“别动”，最后听谁的？（第 7、21 章）
- 一个因子回测好看得离谱，怎么识破它？（第 15 章）
- 数据源挂了，是编一个数顶上，还是让它停下来？（第 20 章）

这些问题，光靠“讲道理”是讲不清楚的。所以这本书的写法是：**每一章都对着一个真实运行的系统讲。**系统里长什么样、参数是多少、阈值定在哪里、哪一行代码做的判断，全都写清楚。读者可以自己去看代码核对。

关于“严禁模拟数据”的承诺

这本书有一条自定的规矩，写在书末的《版本与说明》里，也写在附录 I：

本书的写作纪律

全书出现的每一个数字，要么直接取自系统运行库（`data/stock.duckdb` 与 `quantpilot.db`），要么是本书按系统实现中的同一公式、在系统的真实数据上重新算出来的。两种情形都会在正文中当场标注。凡是系统库里没有的数据，本书留白并注明“数据缺失”，绝不填一个“看起来合理”的估计值。

这条规矩不是为了显得严谨，而是因为它是这个系统本身的第一原则——数据不可用就报错，不降级、不伪造。第 20 章会详细讲为什么这条原则比“模型能力”重要得多。如果这本书自己都做不到，那它就没资格讲那套原则。

这本书不做什么

- 不教你怎么赚钱。本书不给任何操作建议，也不承诺任何收益。系统里的每一个回测数字都是历史，历史不保证未来。
- 不吹“AI 会取代人”。恰恰相反，全书反复强调：AI 的权限必须被关在笼子里，也只能让仓位更保守，不能更激进。
- 不假设你懂编程。公式、代码片段、参数表都会出现，但每一处都先给出生活化的直觉，看不懂代码可以跳过代码，只看结论和出处。
- 不假装这是一本完结的书。系统还在改，书里也会如实写出那些曾经写错、后来又改对的地方（见附录 H）。错误比正确更有信息量。

致谢与免责

本书写作过程中反复用到一个词：*honest*（诚实）。一个量化系统可以笨，可以慢，可以错过机会，但不能骗自己的主人。这一点也是我对读者的最大期望。

最后，请允许我把免责声明放在最显眼的位置，而不是藏在书末：

本书与本书所描述的系统，均仅供学习、研究与量化投资实验使用。程序产生的信号、评分、策略、分析、决策与回测结果，均基于历史数据与模型计算，不保证未来收益，不构成任何投资建议。金融市场存在风险，股市有风险，投资须谨慎。

QuantBot Lab

2026 年秋，于一台一直开着的电脑旁

目录

前言 ii

插图目录 xvii

表格目录 xix

代码清单目录 xxiii

怎么读这本书 xxv

记号表 xxviii

第一部分 智能体驱动：一个人怎么假装自己有一支投资团队 1

第一章 一个人，怎么同时当研究员、风控和操盘手？ 2

1.1 早上九点二十四分，我在开会 2

1.2 我的第一个答案，错得很彻底 2

1.3 不是找人，是开一家公司 3

1.4 五个人，五份活 4

1.5 它在系统里长什么样 5

1.6 它还没有被约束住 7

第二章 五个岗位，五条不能越过的线 9

2.1 同一条线，写在三个地方，三个数字 9

2.2 两个生活里的“不” 9

2.3 如果不这么做，会怎样 10

2.4 从“职责描述”到“权限矩阵” 10

2.5 五条线：它在系统里长什么样 11

2.5.1 第一条线：流水线 11

2.5.2 第二条线：权限矩阵 12

2.5.3 第三条线：三句写死的硬约束 13

2.6 常见误解与纠偏 14

2.7 真实出处、动手试一试与下一章 15

第三章 怎么给 AI 派活，它才不会乱来？ 16

3.1 “帮我看看今天的市场，然后决定买什么” 16

3.2 两个合同里的常识 17

3.3	如果不给合同，会怎样	17
3.4	从“说清楚”到“可校验”	18
3.5	它在系统里长什么样	18
3.5.1	第一件东西：12 个约束字段	18
3.5.2	第二件东西：每个任务都有自己的“什么都不做”	19
3.5.3	第三件东西：输出守卫的三层检查	20
3.6	真实出处、动手试一试与下一章	21
第四章	AI 的手：它能动手，但只能动你允许的那几样东西	23
4.1	为什么不让它自己算收益率	23
4.2	给新员工开账号	23
4.3	不给工具，会怎样	24
4.4	从“给模型工具”到“工具契约”	24
4.5	它在系统里长什么样	24
4.5.1	一个工具的解剖	24
4.5.2	取不到数据，工具会拒绝	25
4.5.3	一张表决定谁能碰什么	27
4.5.4	每个角色手里有几把工具	27
4.5.5	运行时还有一次校验	29
4.6	常见误解与纠偏	30
4.7	真实出处、动手试一试与下一章	31
第五章	一天是怎么过完的？——盘前、盘中、盘后、深夜	32
5.1	九点我在开会，十五点我在地铁上	32
5.2	先别急着讲程序，先看一张值班表	32
5.3	如果不排这张表，会发生什么	33
5.4	一个差不多对的模型：把一天切成四段	33
5.5	它在系统里长什么样	34
第六章	让 AI 自己吵一架：多空辩论	39
6.1	两个 AI，你信哪个	39
6.2	让同一个模型“再想一遍”，为什么没有用	39
6.3	一个差不多对的模型：吵一架	40
6.4	它在系统里长什么样	41
第七章	意见不一致时，谁来拍板？——Kelly 缩放与“只降不升”	45
7.1	信心六十分，共识七十分，这算看好吗	45
7.2	先看一场预算审批	45
7.3	不这么设计，会怎样	46
7.4	一个差不多对的模型，和一个严谨的版本	47
7.5	它在系统里长什么样	50

第八章 投票能不能比专家更靠谱？——多提案集成	54
8.1 四个人，四个说法	54
8.2 直觉：陪审团不改变上限，它只防止一个人说了算	54
8.3 不引入投票，会怎样	55
8.4 一个差不多对的模型：数人头	55
8.5 严谨版本：把四个说法折成一个数	55
8.6 它在系统里长什么样	59
8.7 常见误解与纠偏	62
第九章 如果 AI 突然失联，交易还继续吗？	64
9.1 窗口没了	64
9.2 不给它方向盘，会付出三种代价	64
9.3 一个差不多对的模型：挂了就什么都不做	65
9.4 第一层：单次调用失败，退回规则	65
9.5 第二层：一个阶段失败了，其余阶段照跑	66
9.6 第三层：后台任务崩了，进程还得活着	67
9.6.1 那次闪退的根因：DLL 被拆走了	68
9.7 真实出处与动手试一试	71
第十章 黑匣子：怎么证明这个决策不是瞎猜的？	72
10.1 亏钱那天，你能查到什么	72
10.2 没有它，会坏在哪	73
10.3 一个差不多对的模型：写日志	73
10.4 它在系统里长什么样	74
10.5 常见误解与纠偏	76
10.6 真实出处与动手试一试	77
10.7 这一部分的收尾：他们到底在算什么	77
 第二部分 量化交易原理：把“感觉”变成“可以计算的东西”	 79
第十一章 什么叫“这只股票不错”？——从感觉到因子	80
11.1 我的备注栏里，写着三十七个“不错”	80
11.2 如果你说“不错”，我问你三个问题	80
11.3 不把它变成数字，会发生什么	81
11.4 因子，是一种可以被检验的偏见	81
11.5 它在系统里长什么样	83
11.6 本章的真实出处	85
第十二章 给股票做一次体检：六大因子与盘口	87
12.1 我的第一版选股，只量了一件事	87
12.2 六个科室，各防一种病	87

12.3 盘口：唯一一个“今天几点几分”的因子	88
12.4 从原始值到分数：一个近似模型和它的三个结论	89
12.5 它在系统里长什么样	94
12.6 常见误解与纠偏	95
12.7 本章的真实出处	96
第十三章 每个因子该占多少分？——权重从哪里来	97
13.1 我盯着那个 0.20，越看越不顺眼	97
13.2 不调整，或者放开调	97
13.3 一条“差不多对”的规则，和它的两个漏口	98
13.4 把它写严谨：一个假设、两条命题	99
13.5 它在系统里长什么样	102
13.6 本章的真实出处	105
第十四章 因子也要写体检报告：IC、RankIC、分组与衰减	107
14.1 九月十一日，盘后那行正数	107
14.2 跟着一个天气预报员上班	107
14.3 不做体检，要付三种代价	108
14.4 先给一个“差不多对”的模型	108
14.4.1 IC 与 RankIC：同一张散点图，两种读法	109
14.4.2 分组收益差：把一个相关系数翻译成钱	111
14.4.3 这个等价性在哪里失效	114
14.4.4 剩下四项：覆盖率、准确率、稳定性、衰减	114
14.5 它在系统里长什么样	115
14.6 常见误解与纠偏	117
14.7 真实出处	118
第十五章 怎样识破一个“看起来很厉害”的因子？	119
15.1 一个 0.08 的因子，第二天全崩了	119
15.2 把全班分成两半	119
15.3 不留“反对席”，检验就变成表彰	120
15.4 分半检验：从朴素版本到严谨版本	120
15.4.1 怎么分：固定种子的交替分配	120
15.4.2 判据，与它判错的概率	122
15.4.3 诚实下线：同一个判断，两把尺子	125
15.5 它在系统里长什么样	126
15.6 常见误解与纠偏	128
15.7 真实出处	128
第十六章 十七种策略：它们其实只做一件事	130
16.1 同一天，十七个策略吵起来了	130

16.2	工具箱里的十七把螺丝刀	130
16.3	没有统一接口，要付三种代价	130
16.4	一个接口，三种输出	131
16.4.1	十七个函数体，其实只有两种算法	132
16.4.2	均线信号的可分解性与水平位移不变性	133
16.4.3	超级趋势：让通道跟着波动呼吸	135
16.5	它在系统里长什么样	137
16.5.1	新增一个策略，要同时动三处	138
16.5.2	指标为什么自己写	138
16.6	常见误解与纠偏	139
16.7	真实出处	140
第十七章	回测到底在验证什么？——成本、前视与样本池	141
17.1	年化 60% 的策略，实盘第一周亏了 3%	141
17.2	台架上的发动机，和装在车上的发动机	141
17.3	把摩擦写成公式：三因子成本模型	142
17.3.1	最低佣金：一条给小额交易设的税率	142
17.3.2	平方根冲击律：为什么大单必须拆	143
17.3.3	把上面两笔账合起来：一次往返要付多少	145
17.4	比成本更贵的错：偷看未来	145
17.5	胜率是怎么被一只股票骗出来的	147
17.6	把参数选好看的代价：walk-forward	148
17.7	它在系统里长什么样	151
第十八章	机器学习的陷阱：五个模型为什么常常输给三条均线	153
18.1	五个模型，打不过一条均线	153
18.2	用历年真题训练出来的学生	153
18.3	泄漏是从哪儿漏进去的	153
18.4	五个模型合起来，为什么不一定更稳	155
18.5	它在系统里长什么样	157
18.6	最后一句：它失败了会怎样	158
第十九章	今天能不能下注？（上）——市场六维判势怎么打分	160
19.1	同一只股票，两只手	160
19.2	出门前为什么要看天气	160
19.3	六个维度，六种问法	161
19.3.1	技术趋势：三件事，加减各一档	161
19.3.2	情绪赚钱效应：涨停加分，炸板扣分	161
19.4	六个分数怎么合成一个总分	162
19.4.1	冲突修正：当市场自己都不一致	162
19.5	用五条真实记录把它算一遍	164

19.6 从分数到仓位：一张分段映射表	166
19.7 它在系统里长什么样	168
第二十章 今天能不能下注？（下）——数据从哪来，缺数怎么办	170
20.1 一个“总是很淡”的市场广度	170
20.2 体检设备的精度说明	170
20.3 先讲最贵的那个错：抽样抽歪了	171
20.4 第二个安静的错误：差一根 K 线	174
20.5 六维的来源与降级链	175
20.5.1 缓存与限流：不是性能优化，是纪律	176
20.6 它在系统里长什么样	176
第二十一章 决定“下多少注”：仓位系数是怎么算出来的	179
21.1 同一条决策里，出现了两个数字	179
21.2 发动机和限速牌，从来不吵架	179
21.3 不装限速牌，会付出什么代价	180
21.4 近似模型：乘一个系数；严谨版本：取一个最小值	180
21.4.1 缩放本身：保方向、不放大敞口	183
21.5 它在系统里长什么样	184
21.5.1 硬闸门：该不该建仓，是另一个问题	184
21.5.2 盘中：同一把标尺，只量不记账	185
21.5.3 把 5 条真实记录摆出来	185
21.5.4 前端怎么把这个数露出来	186
21.6 常见误解与纠偏	186
21.7 真实出处	188
第二十二章 风控不是一堵墙，是三层走廊	190
22.1 「最多亏 10%」这句话，能兑现吗	190
22.2 飞机不靠「更小心」	190
22.3 为什么只有一堵墙会漏	191
22.4 它在系统里长什么样：三层走廊	191
22.4.1 第一层：规则风控门——把数写死，并且只写一遍	191
22.4.2 第二层：AI 评审——只能更保守	192
22.4.3 第三层：熔断与阻断——把开关按下去	192
22.4.4 执行端的最后一厘米：buyRiskGate	193
22.5 把「最多亏多少」变成可算的数	193
22.5.1 VaR：一个分位数	194
22.5.2 VaR 的致命缺陷：它不是次可加的	195
22.5.3 CVaR：把分位数换成尾部均值	196
22.5.4 CVaR 的一致性	197
22.5.5 CVaR 至少和 VaR 一样大	200

22.5.6 系统里的历史法：指标是怎么算出来的	200
22.6 风控的最后一步：盘口五态	201
22.7 跑不掉的仓位等于负债：流动性锁定指标	202
22.8 常见误解与纠偏	202
22.9 真实出处	203
第二十三章 A 股的几个硬规矩：T+1、整手与涨跌停	205
23.1 第一天的废单	205
23.2 你的车再好，红灯也得停	205
23.3 不把这些算进去，会付出四种代价	206
23.4 近似模型：现金只够买整手	206
23.4.1 整手取整	206
23.4.2 现金约束下的最大可行购买量	207
23.4.3 回退是幂等的	209
23.4.4 费用项与“低价股”过滤	210
23.5 涨跌停：想买买不进，想卖卖不掉	210
23.6 它在系统里长什么样	212
23.7 两道硬筛选：流通市值与流动性	213
23.8 常见误解与纠偏	214
23.9 真实出处	214
第三部分 实际操作：让它在今天就开始工作	216
第二十四章 第一天：安装、接模型、连数据	217
24.1 下载、解压、双击——然后呢？	217
24.2 装一台新机器：先通电，再通水，最后开机自检	217
24.3 为什么“带病开机”比“开不了机”更糟	217
24.4 把“开机”写成一个状态机	218
24.5 它在系统里长什么样	222
24.5.1 第一步：一个不用安装的文件夹	222
24.5.2 第二步：四步向导，一次把三件事问完	222
24.5.3 第三步：开机自检——六个组件与十步初始化	223
24.5.4 第四步：点 × 不是退出	225
24.6 常见误解与纠偏	226
24.7 真实出处	227
第二十五章 先把米下锅：数据维护的正确顺序	228
25.1 我急着选股，系统却问我“米呢”	228
25.2 下厨之前，先想清楚备料的次序	228
25.3 为什么顺序不能颠倒：六步备料，各自的“跳过会怎样”	228

25.3.1	第一步：先把米缸接上——通达信 vipdoc 目录	229
25.3.2	第二步：全量导入（近 3 年）	229
25.3.3	第三步：一键同步全部（复权因子 + 交易日历）	230
25.3.4	第四步：财务数据维护	230
25.3.5	第五步：每天 16:00，让它自己下锅	231
25.3.6	第六步：收盘后的盘口摘要	231
25.4	一条容易被忽略的硬约束：同一天的多条复权事件	232
25.4.1	先给一个“差不多对”的近似，再看它在哪失效	232
25.4.2	命题与证明	233
25.5	它在系统里长什么样：真实库容	235
25.6	常见误解与纠偏	235
25.7	真实出处	236
第二十六章	生成属于你的第一份投资方案	238
26.1	十二个问题，一个都不能少	238
26.2	先量身体，再定训练量	238
26.3	不做画像，要付什么代价	238
26.4	十二个维度，量的到底是什么	239
26.4.1	问卷的答案，先变成一份投资风格	239
26.4.2	同一份画像，三套候选方案	240
26.5	从候选到股票池：选股引擎接棒	240
26.5.1	买几只，由本金决定	241
26.6	单票上限：保守 15%，均衡 20%，成长 30%	241
26.7	小额成长方案：弱市也要把钱用起来	242
26.8	归一化之后，必须再收口一次	242
26.8.1	先说一个“差不多对”的模型	242
26.8.2	这个模型在哪里失效	242
26.8.3	把两步写成算法，再证它对	243
26.8.4	定理 26.4 在代码里长什么样	245
26.9	方案每天会自动换股	245
26.9.1	一个必须分清的细节：两个动作，两种行为	246
26.10	常见误解与纠偏	247
26.11	真实出处	248
第二十七章	读懂仪表盘：控制中心、AI 团队与实时活动	250
27.1	屏幕上全是数字，先看哪一个	250
27.2	先看故障灯，再看速度，最后看油量	250
27.3	三个页面，三个问题	251
27.3.1	控制中心：钱还剩多少	251
27.3.2	AI 团队：谁在干活、谁有权干什么	253

27.3.3 实时活动：现在跑到哪一步	254
27.4 近似模型 → 严谨版本：刷新闻间隔决定陈旧度	255
27.5 它在系统里长什么样	259
27.6 常见误解与纠偏	259
27.7 真实出处	260
第二十八章 读懂决策：CIO 决策中心里的每一行字	262
28.1 一行字，十二个信息	262
28.2 先读“病历首页”，不读结论	262
28.3 第一块：这张页面的两个 Tab	263
28.4 第二块：市场六维判势卡片	263
28.5 第三块：今日决策卡片	264
28.6 第四块：证据链——决策日志里的五行	265
28.7 第五块：每日复盘 Tab	266
28.8 这条决策从哪儿来	266
第二十九章 我的投资：持仓、成交与“以数据库为准”的盈亏	268
29.1 差了三块五	268
29.2 记账本的三样东西	268
29.3 持仓明细：八列	269
29.4 交易记录：六列，和一个容易看漏的“净额”	270
29.5 盈亏分析：为什么“今日盈亏”以交易日为界	271
29.6 “以数据库为准”到底是什么意思	272
29.7 手动交易与“连续交易”	273
第三十章 研究与调参：选股、策略、回测、每日策略	275
30.1 我想自己动手改点东西	275
30.2 实验室里的旋钮	275
30.3 第一档：选股引擎（立刻出读数）	275
30.4 第二档：策略管理与指标刷新（几分钟）	276
30.5 第三档：回测与网格扫描（几十分钟）	277
30.6 第四档：每日策略（每天一次）	279
第三十一章 安全开关：模拟、实盘、暂停建仓与紧急停止	282
31.1 三个长得一样的开关	282
31.2 手刹、熄火键、空挡	282
31.3 第一件事：模拟和实盘不是“参数区别”	283
31.4 第二件事：暂停建仓，只拦买、放行卖	284
31.5 第三件事：紧急停止	284
31.6 第四件事：还有多少道门	285
31.7 第五件事：合规的两道防线	286

第三十二章 每天进步一点的机制：复盘 → 改权重 → 换策略	288
32.1 一本写了三周没人看的日记	288
32.2 看比赛录像的目的	288
32.3 出口一：复盘 → 权重	289
32.4 出口二：复盘 → 策略	290
32.5 出口三：复盘 → 股票池	291
32.6 那条纪律：幂等标记	291
第三十三章 出问题怎么办：日志、黑匣子与常见故障	294
33.1 一次日志里什么都没写的闪退	294
33.2 黑匣子的读法	294
33.3 为什么致命错误会“静默消失”	294
33.4 那个段错误的完整故事	295
33.5 故障的三种类型，和三种查法	297
33.6 现场清单：出事了先去哪几个地方	297
33.6.1 常见故障速查	297
33.7 最后一件事：回到第 1 章那个问题	298
 第四部分 附录	 301
附录 A 系统架构与包结构速查	302
A.1 系统身份	302
A.2 四层结构：一张图看清谁调用谁	302
A.3 启动十步	303
A.4 53 个包的职责一览	303
A.5 关键路径对照表	304
附录 B 数据表速查	306
B.1 两个库，各管什么	306
B.2 SQLite：58 张业务表	306
B.3 DuckDB：行情与基本面	308
B.4 「以数据库为准」的三条口径	309
附录 C 工具清单与可用角色	310
C.1 注册表：55 个工具	310
C.2 两层注册：声明与装配	312
C.3 七条只有一个人能做的事	312
附录 D 十七种策略的信号逻辑与默认参数	314
D.1 十七个策略一览（完整版）	314
D.2 后六个指标的公式	314
D.2.1 ATR：平均真实波幅	314

D.2.2	WMA 与 HMA：把滞后压下去	316
D.2.3	Aroon：离新高有多远	316
D.2.4	CMF：资金往哪边流	316
D.2.5	SuperTrend：ATR 画出的动态止损线	316
D.3	可调参数的扫描范围	317
D.4	成本模型：回测里那笔被扣掉的钱	318
D.5	回测口径	318
附录 E	六维判势的数据源与降级矩阵	320
E.1	权重与基准	320
E.2	六个维度的数据源与降级链	320
E.3	证据：同一维度的两版来源标注	320
E.4	配置与接口一览	322
附录 F	风险指标的数学定义与推导	323
F.1	记号、假设与分位点的构造	323
F.1.1	记号	323
F.1.2	下分位点与上分位点	323
F.2	VaR：三条性质，与它为什么不是一致风险度量	324
F.2.1	VaR 就是下分位点，三条性质一次证完	324
F.2.2	VaR 次可加性失效的一般判据	325
F.3	CVaR：两种定义何时分家，以及一致性的完整证明	327
F.3.1	一个精确恒等式（本附录的核心）	327
F.3.2	两种定义在什么时候分家	328
F.3.3	一致性的完整证明	329
F.4	Rockafellar–Uryasev 形式与共轭表示	330
F.4.1	RU 形式：凸性、最优解集、取值	330
F.4.2	共轭（上确界）表示	330
F.5	谱风险度量：CVaR 在度量族中的位置	332
F.6	有限样本：系统的历史法算的到底是什么	333
F.6.1	经验分布与顺序统计量	333
F.6.2	120 天窗口意味着什么：把条数算出来	334
F.6.3	一处必须点名的口径缺陷：序列被倒着喂进去了	335
F.7	椭圆分布下的闭式解	336
F.8	Kelly 缩放系数的数学	337
F.8.1	代码守卫在数学上意味着什么	339
F.9	理论口径与系统实现的对照表	339
F.10	真实出处	339
附录 G	术语表（中英对照）	342
G.1	怎么用这张表	342

G.2	一、组织与治理	342
G.3	二、因子、回测与统计	343
G.4	三、市场与行情	344
G.5	四、仓位、成本与风险	344
G.6	五、软件、数据与智能体	345
G.7	六、系统标识符速查	346
G.7.1	六个角色	346
G.7.2	五条权限	346
G.7.3	任务类型	347
G.7.4	决策类型与状态	347
G.7.5	两个库里的关键表	347
G.8	附：容易混淆的三对词	349
附录 H	真实事故索引	350
H.1	怎么用这份索引	350
H.2	一、进程稳定性：程序自己把自己弄死了	350
H.3	二、数据口径不一致：两个地方对同一个东西叫法不同	351
H.4	三、样本与窗口：差一根 K 线，结论就反过来	351
H.5	四、风控与配置：同一个数字被写了三遍	352
H.6	五、性能与调度：慢，或者空转	352
H.7	六、因子与模型：看起来很棒，实盘很糟	353
H.8	七、静默失效：错了，但没人知道	353
H.9	结语：29 条事故的共同形状	354
附录 I	数据诚信原则与免责声明	355
I.1	一、系统自己的态度：宁可不做，也不假装	355
I.2	二、本书的三类标注	355
I.3	三、本书的数据底座（实测）	356
I.4	四、系统里没有的东西	357
I.5	五、如实记录的两处系统缺陷	357
I.5.1	第一处：风险指标的历史法取数存在顺序缺陷	357
I.5.2	第二处：风险引擎把「数据缺失」和「风险为零」输出成同一个值	358
I.6	六、你自己怎么复现	358
I.7	七、免责声明	358
附录 J	系统标识符索引	360
J.1	怎么用这份索引	360
J.2	收录范围与规模	361
J.3	工具名（智能体可调用的函数）	361
J.4	数据表名	362
J.5	字段名	363

J.6 源码文件与包	364
版本与说明	367

插图目录

1.1	五个智能体的流水线。第一个岗位向内问“你要什么”，其余四个向外回答“能做什么”。	4
2.1	五个岗位在权限链上的位置。每一格下面的动词是这个角色在这条链上唯一被允许推进的那一步。	12
5.1	一天的四幕。横轴是时间，四行是四个任务阶段。盘中那一段里颜色变浅的两小时是午休——按钟点排班最容易漏掉的，就是它。	35
6.1	决策主张层的五个阶段。前两段是“把话和证据说清楚”，中两段是“吵并且折算”，最后一段是“多方对表”。	40
9.1	五阶段流水线的容错方向：错误向上汇聚成一张清单，而不是向下传导成一次中断。 . . .	70
10.1	三条落库通道：决策证据链挂在决策记录上，交付物同时进文件与数据库，操作级动作另进审计日志。	75
11.1	从“感觉”到“可排序”的五步。前三步在横向截面上完成，后两步才产生名次。	84
13.1	乘数 mult 关于相对质量 q/q_{med} 的形状（式 eq. (13.2)）：封底 0.5、线性、封顶 1.5。 . . .	100
14.1	eq. (14.7) 的五个点。四个点（除 $x = 1$ 那一个）次序与收益一致，名次相关为正；但 (1, 100) 这一个点的杠杆把 Pearson 相关拉成负数。	111
15.1	判据 eq. (15.7) 的四个象限。深蓝虚线是 ± 0.02 阈值，绿色两块是“同号且都超阈”。原假设下四个象限各占一份，同号条件只留两块，假阳性正好减半。	125
17.1	回测链路的五个环节。前三环决定“数字真不真”，后两环决定“数字算不算数”。	151
20.1	六维判势的取数链。左边是首选，越往右越是兜底；无论走到哪一站，来源都必须写进 <code>sources</code> 。	177
22.1	三层风控走廊：买单自上而下依次通过，任何一层都可以单独拦下它。拦截不是「静默丢单」，而是带结构化拒绝原因的留痕。	191

24.1	fail-closed 启动状态机：只有关键组件全绿才进入工作状态；否则停在 ABORT 分支，修好后重新走一遍自检。	219
24.2	设置 • 关于：版本信息、组件状态与数据基准日	226
25.1	设置 • 数据源：五个可选数据源，以及“腾讯财经只覆盖实时行情，研究中心仍读本地 DuckDB”的口径说明	236
26.1	投资方案详情：风险等级与四项指标、投资任务书，以及资产配置后的具体持仓权重 . . .	247
27.1	看板信息的三层结构。每一层回答一个不同的问题，而且只有上层的答案是“好”的时候，下层的数字才值得读。	251
27.2	控制中心（驾驶舱）：顶部五张卡片、CIO 今日状态、风险监控、持仓分布与 AI 今日决策	252
27.3	AI 团队页：五智能体架构（PROPOSE → EVIDENCE → VETO → DECIDE → EXECUTE）与 Agent 任务规格清单	254
27.4	实时活动页：投资流水线五格（提出方案 → 提供证据 → 风险否决 → 最终决策 → 执行交易）、AI 实际读取到的市场指数，与可按角色筛选的活动日志	256
28.1	投资管理 • 投资决策中心：今日决策卡片、风险评价（VaR / ES）与 CIO 决策日志 . . .	263
29.1	组合交易中心：顶部四项指标卡与持仓明细表（图中的 5 只持仓与权重均为库中真实记录）	271
30.1	选股引擎：三步流程与三个策略模板（每个模板下方写着它真实的加权公式）	277
30.2	回测页：回测配置与最近结果列表（累计 73 次，表中数值为系统真实回测输出）	279

表格目录

1.1	五个智能体的岗位与任务类型	4
1.2	工具的角色可见性（节选）	6
2.1	五条线的完整定义	12
3.1	12 个约束字段对应的人话	19
3.2	系统里 16 个任务的无动作状态码（NOActionCode）	19
4.1	market_sixdim_detect 最近的真实落库记录（节选两日）	26
4.2	工具的角色可见性	28
4.3	装配完成后，每个角色实际挂载的工具数量	28
5.1	七段市场时段与它们的时间窗	35
5.2	盘前阶段的七个任务（真实执行顺序）	36
6.1	系统真实记录：同一天里，两个维度自己就吵起来了	43
7.1	引理 7.5 在真实记录上的表现：生效仓位的上界就是 ruleRate	51
7.2	κ 的取值（按式 (7.2) 计算）	52
8.1	四路来源的表态分与权重	60
8.2	三个可复算的算例：从四路表态到净多分与方向	61
9.1	五级失效点与各自的兜底方式	70
10.1	决策证据链的字段，以及每个字段负责回答的问题	74
11.1	系统里的八个因子（固定顺序）	84
12.1	六个体检科：每个因子试图捕捉哪种人性偏差、防哪种病	88
12.2	盘口状态罚分 p	89

13.1	三套模板的默认因子权重（写死的常量，不是从数据里推出来的）	104
14.1	因子体检的六栏，与它们在质量分里的权重	116
15.1	无效因子被判“稳定”的假阳性概率上界（阈值 0.02）	124
16.1	17 个内置策略：族、类型、默认参数与信号方向	137
17.1	系统的默认成本模型参数（A 股口径）	142
17.2	最低佣金下的有效佣金率（ $a = 0.0003$, $M = 5$ ）	143
17.3	网格扫描的输出字段（GridScanResult）	150
18.1	ML 策略在系统里的四个工程要点	158
19.1	市场六维判势的六个维度与权重	161
19.2	系统运行库中的市场六维判势真实记录（全部 5 条）	164
19.3	修正后总分 $\rightarrow$ 仓位系数 r_{pos} $\rightarrow$ 市场标签	166
19.4	四条真实记录从 adjusted_total_score 到 position_rate 的验算	167
20.1	字典序截断 300 只 vs. 分层等比例配额 300 只（真实板块规模）	173
20.2	六个维度的首选源、备选源与兜底代理	175
21.1	5 条真实六维判势记录：总分 $\rightarrow$ 仓位系数 $\rightarrow$ 建仓闸门	186
22.1	九项硬限制的默认值，以及它在拦什么	191
22.2	盘口五态：判定条件与后果	201
22.3	流动性锁定指标的字段与含义	202
23.1	A 股三条硬规矩与它们在系统里的落点	212
23.2	总资产规模与最大候选只数、单标的预算	213
24.1	启动加载界面的六个组件，以及失败意味着什么	224
25.1	行情库的表与视图及其真实规模	235
26.1	画像问卷的十二个维度及其对方案的影响	239
26.2	三档风格的预期收益、回撤与资产配置	240
26.3	A/B/C 三套候选策略的核心指标	240
26.4	本金区间与候选只数、单标的预算	241

26.5	各风险等级的单票上限与基础现金缓冲	241
27.1	8 个操作级权限 × 5 个角色（默认矩阵中“恰好只有一个角色可执行”的那些条目）	254
27.2	三个页面的路由、回答的问题与刷新节奏	259
28.1	市场六维判势卡片上四个格子的含义	264
28.2	决策证据链的五块标签	265
29.1	账本的三样东西在系统里的位置	268
29.2	持仓明细的八列	269
30.1	三种选股模板	275
31.1	三个安全开关的精确含义	282
31.2	一笔买入要穿过的门（自上而下）	285
32.1	复盘闭环的三个出口	288
33.1	三类故障的判别与查法	297
33.2	排障现场清单（按查看顺序）	297
33.3	四个最常见的故障与处理	298
A.1	QuantBot 的身份卡	302
A.2	启动十步（App.go 的 startup）	303
A.3	internal/ 的 53 个包，按职责分组	303
A.4	正文主题到源码路径的对照	304
B.1	两个数据库的分工	306
B.2	SQLite 业务表清单与行数（database/quantpilot.db）	306
B.3	DuckDB 的表与视图及其真实规模（data/stock.duckdb）	308
C.1	中央工具注册表：55 个工具、类别、风险级别与声明可用角色	310
C.2	各角色实际装配的工具数	312
C.3	「仅此一人」的权限条目（默认矩阵）	313
D.1	17 个内置策略：类型、私有默认参数、止损/止盈与信号逻辑	315
D.2	17 个策略的默认调参范围（最小值 → 最大值，步长）	317

E.1	六维的权重、评分基准与它看什么	320
E.2	总分修正与分数 $\rightarrow$ 仓位 $\rightarrow$ 标签映射	320
E.3	六维的数据源、降级链与缓存策略	321
E.4	六维的配置项与对外接口	322
F.1	名义置信水平、实际尾部条数与等价水平 ($n = 120$)	334
F.2	被倒序喂入后,「尾部」取到的是涨幅而非跌幅——沪深 300, $n = 120$	335
F.3	哪些对外指标被这一处顺序影响(沪深 300 与上证指数, $n = 120$, 2026-04-02 至 2026-09-23)	336
F.4	正态假设下的风险倍数(以标准差 σ 为单位)	337
F.5	风险指标: 理论对象、系统实现、以及两者的差异	340
G.1	组织与治理术语	342
G.2	因子、回测与统计术语	343
G.3	市场与行情术语	344
G.4	仓位、成本与风险术语	344
G.5	软件、数据与智能体术语	345
H.1	进程稳定性事故	350
H.2	数据口径不一致事故	351
H.3	样本与窗口事故	351
H.4	风控与配置一致性事故	352
H.5	性能与调度事故	352
H.6	因子与模型事故	353
H.7	静默失效与告警缺失事故	354

代码清单目录

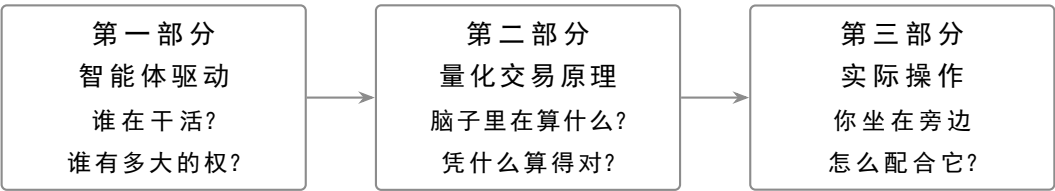
1.1	五个角色的常量定义（节选）	6
2.1	权限矩阵里 TRADER 与 CIO 两行（节选）	12
2.2	三条硬约束（ <code>internal/agentworkflow/permissions.go</code> ，原文）	13
3.1	任务契约 v1.0 的完整定义（ <code>internal/scheduler/task_scheduler.go</code> ，原文）	18
4.1	工具的执行接口（ <code>internal/tools/tools.go</code> ，原文）	25
4.2	<code>market_sixdim_detect</code> 的参数声明（ <code>internal/tools/sixdim_tool.go</code> ，原文节选）	25
4.3	工具在动手前先检查数据是否可用（ <code>internal/tools/sixdim_tool.go</code> ，原文）	25
4.4	同一个工具的返回结构（ <code>internal/tools/sixdim_tool.go</code> ，原文节选）	26
4.5	25 个工具注册入口（ <code>internal/brainhost/team_tools.go</code> ）	27
4.6	中央工具注册表与注册闭包（ <code>internal/brainhost/team_tools.go</code> ，第 223–238 行，原文）	29
4.7	三条注册声明（ <code>internal/brainhost/team_tools.go</code> ，第 244 行、第 250 行、第 270 行，原文节选）	29
5.1	市场时段常量（节选， <code>internal/util/trading_hours.go</code> ）	34
5.2	主调度循环（节选， <code>internal/scheduler/auto_scheduler.go</code> ）	34
5.3	盘前任务的顺序与依赖（节选， <code>internal/agentworkflow/workflow_engine.go</code> ）	36
6.1	决策主张层入口（节选， <code>internal/cio/proposal.go</code> ）	41
6.2	主张官输出的字段（节选， <code>internal/cio/proposal.go</code> ）	41
6.3	多空辩论与解析（节选， <code>internal/cio/proposal.go</code> ）	42
7.1	Kelly 仓位缩放（ <code>internal/cio/proposal.go</code> ）	50
7.2	Kelly 缩放进入门控（节选， <code>internal/cio/proposal.go</code> ）	50
7.3	证据链负载（节选， <code>internal/cio/proposal.go</code> ）	51
8.1	多提案集成投票（节选， <code>internal/cio/ensemble.go</code> ；为便于阅读合并了分支赋值）	59
8.2	集成结果如何收敛仓位（节选， <code>internal/cio/proposal.go</code> ）	61
9.1	硬回退（节选， <code>internal/cio/proposal.go</code> ）	66
9.2	单步失败不中断（节选， <code>internal/agentworkflow/workflow_engine.go</code> ）	66
9.3	后台 goroutine 的兜底（节选， <code>internal/util/goroutine.go</code> ）	67
9.4	把致命崩溃接出来（节选， <code>main.go</code> ）	69
10.1	白名单的失败即拒绝（节选， <code>internal/cio/cio.go</code> ）	74
11.1	两把尺子：正向与反向（节选自 <code>internal/screener/factor_engine.go</code> ）	83
11.2	加权总分（节选自 <code>internal/screener/pro_factor_engine.go</code> ）	83
11.3	固定因子顺序（节选自 <code>internal/screener/factor_engine.go</code> ）	84
13.1	因子质量分的稳健基准（节选自 <code>internal/cio/cio.go</code> ）	102
13.2	自适应加权主流程（节选自 <code>internal/cio/cio.go</code> ）	103
14.1	质量分的五项加权（节选，去掉了注释）	116

15.1	权重侧的诚实下线（节选）	127
16.1	策略接口与信号常量（节选）	131
16.2	新策略的三处注册（以超级趋势为例，均为节选）	138
17.1	walk-forward 切分与样本外验证（节选自 <code>internal/backtest/grid.go</code> ）	149
18.1	ML 策略的默认配置（节选自 <code>internal/backtest/strategies.go</code> ）	157
20.1	分层等比例配额抽样（节选自 <code>internal/data/duckdb.go</code> ）	172
20.2	<code>sources</code> 字段的两个真实版本（原文摘录，仅截取关键片段）	175
20.3	实时快照的 TTL 缓存与非交易时段短路（节选自 <code>internal/marketsixdim/fetcher.go</code> ）	176
21.1	按仓位系数缩放买入订单（节选， <code>internal/cio/cio.go</code> ）	184
21.2	六维判势硬阻断建仓（节选， <code>internal/cio/cio.go</code> ）	184
21.3	盘中复用判势与告警（节选， <code>internal/cio/cio.go</code> ）	185
22.1	<code>buyRiskGate</code> ：执行端硬风控（节选）	193
23.1	一次真实的拒单（来自执行层校验）	205
23.2	整手校验（节选）	206
23.3	低价股筛选参数（节选）	210
23.4	流动性硬下限（节选）	213
24.1	后端的启动看门狗与 <code>panic</code> 兜底（节选）	223
26.1	每日自动重建方案的调用链（节选）	245
28.1	多提案集成区块里“来源表态”的真实拼法（节选自 <code>ui/src/pages/CIO.tsx</code> ）	265
30.1	策略指标刷新按钮的真实行为（节选自 <code>ui/src/pages/Strategies.tsx</code> ）	276
31.1	<code>buyGate</code> 的实现意图（节选自 <code>internal/portfolio/engine.go</code> ）	284
33.1	把致命错误的输出重定向到文件（节选自 <code>main.go</code> 的 <code>setupLogging()</code> ）	295
F.1	另一个消费方对 <code>GetRecentPrices</code> 返回值的处理（ <code>internal/portfolio/dataprep.go</code> ）	335
I.1	复现附录 F 全部风险指标数字的一条命令	358
J.1	复现本书全部数据的命令	368

怎么读这本书

一、三条主线，其实是同一个东西

本书分三个部分，它们不是三本书拼在一起，而是同一个系统的三种切法：



- **第一部分（第 1–10 章）**回答“谁在干活”。读完你应该能画出这个系统的组织架构图，并说清楚每个 AI 员工的权力边界。
- **第二部分（第 11–23 章）**回答“它在算什么”。读完你应该能对一个策略提出三个致命问题：因子站得住吗？回测说谎了吗？今天该下多少注？
- **第三部分（第 24–33 章）**回答“你今天该做什么”。读完你应该能在两小时内完成安装、数据、方案、监控的完整链路，并且在出事时知道去哪儿找原因。

二、按你的背景选一条路

如果你是	建议这样读
会用电脑、懂一点股票、不会写代码	第 1、5、11、19、21、22 章 + 第 24–33 章全部 + 附录 G、I。遇到公式先跳过，只看“为什么”和“结论”。
会写代码、想做量化	第一部分全部 + 第二部分全部 + 第 25、30、31 章。重点看第 15、17、18 章（过拟合与回测的坑），手边放着附录 J 的标识符索引，读到哪个名字都能查到它还在哪儿出现过。
已经在做策略或已经在搭系统	直接从第 14–18、20、22 章切入，把重点放在“约束条件怎么写进代码”，而不是概念本身；再看附录 H 的真实事故索引做自查，附录 G 的术语表与附录 J 的标识符索引可以当工具书用。

三、每一章都按同一套七步写

费曼讲物理时有一个习惯：他从不先给定义。他先给你一个现象，让你觉得“这事有点怪”，然后才慢慢告诉你这东西叫什么。本书照这个顺序组织每一章：

1. **一个具体场景**——开头必须是一个让人想听下去的疑问，或者一次真实的翻车；
2. **直觉先行**——用一个生活类比先搭出“图像”，让你先看见，再算；
3. **为什么需要它**——先把“为什么”讲透，才说“是什么”；
4. **近似模型 → 严谨版本**——先给一个“差不多对”的模型讲清本质，再补严谨与量化；

- 5. 它在系统里长什么样——真实参数、真实阈值、真实文件位置；
- 6. 常见误解与纠偏——每一章至少两条，专治“我以为我懂了”；
- 7. 真实出处 + 动手试一试——这一章的结论对应系统里的哪个包、哪张表，以及你现在就能做的那个小实验。

本书把长推导、长清单、长表格全部放进附录，正文只留下理解所必需的部分。第 22 章讲风险时会用到 VaR 与 CVaR，正文只给直觉与定义，完整的数学推导在附录 F；第 7 章讲 Kelly 缩放时同理。

四、全书只用一套场景，不换例子

为了让读者不迷路，全书自始至终只有一套设定：

- 人物：“我”——一个普通上班族，本金 10 万元，白天上班，没时间盯盘；
- 系统：QuantBot——一台装在 Windows 电脑上的软件，雇了 5 个 AI 员工；
- 时刻：反复回到同一天。9:00 盘前 → 9:30-15:00 盘中 → 15:00 盘后 → 20:00 夜里复盘。每一章都在这条时间线上找自己的位置；
- 样本：需要个股示例时，使用系统真实行情库中的真实标的，代码照原样写出，读者可以自己到库里核对。

五、数据从哪里来，怎么自己复现

本书的数据底座是系统的运行库：

库	文件	本书写作时规模
行情与基本面	data/stock.duckdb	1,499,738,112 字节（约 1.4 GB）
系统状态与交易	database/quantpilot.db	7,217,152 字节，58 张业务表

本书所有导出与复算脚本都在 book/data/scripts/ 目录下，其中：

- export_real_data.py——只读连接行情库，把全书正文引用的真实数据导出为 data/raw/ 下的 JSON；
- export_for_rest.py——第二批导出，供后半部章节与附录使用，口径与上一个完全相同；
- recompute_risk.py——复算附录 F 的全部风险指标数字（尾部条数、VaR/CVaR、以及一处顺序口径的对照），逐字照抄系统实现的取数与算法；
- make_stubs.py——只用来生成章节骨架文件，不产出任何正文数字。

另外，book/ 根目录下还有四份校验脚本，都不产出数据，只用来在定稿前挡住四类问题：check_tex_escapes.py 检查等宽宏参数里有没有多写转义，gen_identifier_index.py 生成附录 J 的标识符索引并顺带核对全书标识符的用法是否自相矛盾，fix_cjk_quotes.py 清理正文里用 ASCII 直引号写的中文引号，check_book_stats.py 核对书中自报的各类计数与实际是否一致。它们的详

细说明写在《版本与说明》页。

这些脚本一律以只读方式连接数据库（`duckdb.connect(DB, read_only=True)`），不会修改系统数据。读者可以自己跑一遍，得到与本书一致的数字；附录 F 里的每一个数都可由 `recompute_risk.py` 一条命令重现。

六、记号约定

- 等宽字体（如 `market_sixdim_detect`）表示系统里的真实标识符：工具名、字段名、表名、文件路径；
- 第 X 章、第 X 节指本书；`internal/xxx` 指系统源码中的包；
- 全书脚注中会出现三类醒目标记：**真实数据**（直接取自运行库）、**复算数据**（按系统同一公式在真实数据上重算）、**数据缺失**（系统库中无有效记录，本书留白）。

记号表

一、数学记号

记号	含义
N	横截面上的标的数量（同一时点、同一股票池）
T	时间序列上的期数（日频时即交易日数）
$r_{i,t}$	标的 i 在第 t 期的收益率，本文一律使用对数收益率 $\ln(P_{i,t}/P_{i,t-1})$ ，除非另有说明
$f_{i,t}^{(k)}$	第 k 个因子在标的 i 、时点 t 上的原始取值
$z_{i,t}^{(k)}$	第 k 个因子的横截面标准化取值（ z -score）
$s_{i,t}^{(k)}$	第 k 个因子的分数，取值于 $[0, 100]$
IC_k	第 k 个因子与下一期收益率的 Pearson 相关系数
RankIC_k	第 k 个因子排名与下一期收益率排名的 Spearman 相关系数
w_k	第 k 个因子在综合评分中的权重，满足 $\sum_k w_k = 1$ 、 $w_k \geq 0$
q_k	第 k 个因子的质量分，取值于 $[0, 1]$
r_{pos}	市场六维判势输出的仓位系数， $r_{\text{pos}} \in [0, 1]$
c	主张置信度（ <i>confidence</i> ），取值于 $[0, 100]$
a	多空共识度（ <i>agreement</i> ），取值于 $[0, 100]$
κ	Kelly 缩放系数， $\kappa \in [0.4, 1]$
u	决策前的规则仓位（未经 AI 调整）
u^*	生效仓位， $u^* = \min\{\text{各来源的上界}\}$
σ	波动率（日频收益率标准差，除非注明年化）
DD	回撤（ <i>drawdown</i> ）
MDD	最大回撤（ <i>maximum drawdown</i> ）
VaR_α	置信水平 α 下的在险价值
CVaR_α	置信水平 α 下的条件在险价值（期望损失）
Π	组合权重向量 $(\pi_1, \dots, \pi_N)^\top$ ， $\sum_i \pi_i = 1$ 、 $\pi_i \geq 0$
λ	风险厌恶系数
$c_{\text{buy}}, c_{\text{sell}}$	单边交易成本率（买入、卖出）
$\mathbf{1}\{\cdot\}$	示性函数，条件成立取 1，否则取 0
$\text{clamp}(x, l, h)$	截断算子： $\min\{\max\{x, l\}, h\}$

二、系统标识符约定

书中出现的系统标识符一律保持原样、不加翻译，以便读者直接检索：

- 工具名：智能体可以调用的函数，如 `market_sixdim_detect`、`select_next_day_strategy`；
- 表名：如 `stock.ohlc`、`market_sixdim_daily`、`daily_strategy_plans`；
- 字段名：如 `position_rate`、`raw_total_score`、`conflict_count`；
- 包与文件：如 `internal/marketsixdim/sixdim.go`。

三、单位与量纲约定

- 金额单位一律为人民币元；表格中若使用“亿元”会明确写出；
- 收益率一律为小数（0.0123 表示 1.23%），仅在正文行文中折成百分数；
- 波动率默认年化时，用 $\sqrt{244}$ 折算（A 股年均交易日按 244 天计）；
- 所有“分数”类指标（因子分、六维分、质量分）都明确写出取值区间，不做隐含归一化。

第一部分

智能体驱动：一个人怎么假装自己有一支投资团队

第一章

一个人，怎么同时当研究员、风控和操盘手？

1.1 早上九点二十四分，我在开会

二零二六年九月十日，星期四，上午九点二十四分。

我坐在会议室里，投影仪上放着第三季度的项目复盘，主讲人正在解释一张我早就看过的图。我的手机在桌面底下亮了一下——不是消息，是我自己设的闹钟。

闹钟备注写着六个字：**看今天该买什么。**

九点三十分，A 股开盘。而我要在九点五十分之前，把那份复盘的第一版意见交出去。两件事撞在一起，一件都不能推迟，一件都不能敷衍。

这不是那一天的意外。这是我过去两年的常态。

我把“一个业余投资者每天该做的事”写在一张纸上，写完发现是九件：

1. 看市场——今天大盘是什么状态，是暖的还是冷的；
2. 找机会——哪个行业在走强，哪条消息值得重视；
3. 做研究——把感兴趣的几家公司翻一遍报表；
4. 选股票——在几千只股票里排出前几名；
5. 做回测——把想法放到历史上跑一遍，看看是不是自欺欺人；
6. 控风险——算清楚最坏情况会亏多少，仓位该怎么收；
7. 管组合——手里已经有五只，新来的这只跟它们冲不冲突；
8. 交易——真的下单，还要盯着成交；
9. 复盘——收盘后回头看看，今天哪一步想错了。

麻烦在于：这九件事，没有一件可以不干；而这九件事，没有一件可以和另外八件同时干。

看市场要盯盘，做研究要安静，下单要专注，复盘要等收盘。它们的时间窗口互相打架，需要的心理状态也完全不同。第 2 件要贪心，第 6 件要胆小；第 8 件要果断，第 9 件要诚实。

我把这张纸贴在了显示器边上，贴了很久。贴纸没有解决问题，它只是让我每天更清楚地知道自己做不到。

1.2 我的第一个答案，错得很彻底

我一开始的答案是：找一个更聪明的模型，把这些事全交给它。

逻辑听起来很顺：既然这九件事都需要判断力，那就找一个判断力最强的家伙，让它从头做到尾。中间不交接，就不会掉信息；不分工，就不会互相推诿。

我甚至已经想好了提示词的开头：你是一位经验丰富的量化投资专家，请分析今天的市场

并决定买什么。

第一次跑，它给了我一份相当漂亮的回答。市场判断、行业排序、三只候选、买入理由、风险提示，一样不缺。文笔很好，逻辑也很顺。

第二次跑，我发现它推荐的第一只股票，和第一次完全不同。

第三次跑，我又发现它说的“当前市盈率 18.3 倍”这个数字，在前两次的回答里根本没有出现过，而且我在行情库里查不到。

那一刻我意识到问题不在“它聪不聪明”，而在 ** 我把一个需要被拆开的过程，塞进了一个没有接缝的黑盒子 **。

黑盒子有三个致命的毛病：

第一，它的正确性无法局部检查。输出一份完整的投资建议，你只能选择全信或者全不信。你没法说“行业判断我认可，风险那一块我不放心”。因为它是同一个模型在同一次生成的，你拆不开。

第二，它没有内部制衡。一个人既负责“推荐买什么”，又负责“评估这个推荐的风险”，那么在生成文字的那一刻，这两个目标是在同一段计算里被同时优化的。它天然会选择自洽的答案，而不是暴露矛盾的答案。

第三，“什么都做”的角色，最后一定只做其中一件事。这不是模型的缺陷，是人性的规律，也是任何被过度授权的系统的规律。我见过太多次：被要求“既要收益又要安全”的策略，最后一定选择其中一个，然后给另一个找一套说辞。

直觉·一间只有一个人的公司 你注册了一家公司，法人、财务、销售、法务、出纳，全是你自己。第一天，你签了一份合同。第二天，你发现合同里的付款条件是“货到三个月后付”，而公司账上只够撑四十天。

问题不在于你不够聪明。问题在于：** 在同一个脑子里，“拿下这单”和“这笔钱什么时候回来”，会被合并成一个念头。** 你不会先写完销售方案，再换一顶帽子去审它——你只会一边写一边替自己圆场。

要解决它，不需要一个更聪明的脑子，需要 ** 第二个脑子，并且给它一个不同的考核指标 **。

所以我推翻了自己的第一个答案。正确的方向不是“找一个人做完九件事”，而是——

1.3 不是找人，是开一家公司

一间五个人的小公司，是怎么解决“又要进攻又要防守”这个矛盾的？

它不靠让每个人都变成完人。它靠三件事：** 分岗、定责、留痕 **。

分岗，是说每个人只盯一小块；定责，是说每块工作有一个明确的、可被追问的产出；留痕，是说每个判断都要写下来，写下来的东西要能被别人查。

这三件事，恰好可以把那九件事切成五份，并且切得干干净净。

我们先把公司需要的“工种”想清楚。不要去查任何资料，就凭常识问三个问题：

谁看？——谁负责把外部世界的信息变成事实。

谁想？——谁负责基于事实形成主张。

谁动手？——谁负责把主张变成动作。

三个问题，三个工种。这是一个“差不多对”的模型，它抓住了分工的本质。但它有一个明显的裂缝：在“谁想”里面，“提出主张”和“否决主张”被塞进了同一个人。

我们刚刚才用了整整一节论证为什么这件事不能干。所以“谁想”必须再拆一刀。

拆完还有第二道裂缝。“谁看”里面，“搞清楚主人想要什么”和“研究市场发生了什么”也不是一回事。前者是问客户的预算和风险偏好，后者是问市场。一个向内，一个向外，混在一起就会出现那种最经典的错误：因为我想赚大钱，所以我看什么都觉得能赚大钱。

直觉·手术台上几个人 一台心脏手术，台上至少站着三类人：主刀、助手、麻醉。助手不是“水平差一点的主刀”，麻醉也不是“管睡觉的助手”。他们站在同一个台子边上，但 ** 看的是不同的仪表，担的是不同的责任，犯的是不同性质的错 **。

主刀可以决定在哪一刀切下去；麻醉可以决定这台手术现在能不能继续——即使主刀认为可以。没有人会说“让主刀一个人干完算了，反正他最懂”。因为所有人都知道：** 手术失败的代价，主要不是因为主刀不够熟练，而是因为当主刀判断失误时，没有人有资格喊停。 **

于是那三个工种，被拆成了五个岗位。这就是本书从头到尾要讲的那个团队。

1.4 五个人，五份活

表 1.1 是这支队伍的编制。注意第四列——每个岗位在系统里都对应一组 ** 有名字的任务类型 **，而不是一句模糊的“负责相关分析”。任务类型是这支队伍最关键的工程细节之一，我们在第 3 章会专门拆开它，这里先看个大概。

表 1.1 五个智能体的岗位与任务类型

角色	中文	一句话职责	任务类型数
AgentPlanner	投资规划师	弄明白你要什么、你能承受什么	3
AgentQuant	量化分析师	市场发生了什么、哪些股票值得排前面	7
AgentRisk	风控师	这件事最坏会坏到什么程度	6
AgentCIO	首席投资官	综合所有意见，拍板，并对结果负责	6
AgentTrader	操盘手	把拍板的结果变成订单，并盯着它成交	5

任务类型总数 27 个，定义于 `internal/agentworkflow/models.go`。
数据来源：系统源码 `internal/agentworkflow/models.go`；核对时间 2026-09-24。

这五个岗位之间的关系，不是上下级汇报，而是一条流水线。信息从左边流进，动作从右边流出：



图 1.1 五个智能体的流水线。第一个岗位向内问“你要什么”，其余四个向外回答“能做什么”。

现在逐个看这五份活到底在干什么。

投资规划师（Planner）。它是队伍里唯一“向内看”的人。它的任务只有三类：更新用户画像（UPDATE_PROFILE）、检查投资委托书是否还成立（CHECK_MANDATE）、出合规报告

(COMPLIANCE_REPORT)。

它不管市场涨跌，它管的是那把尺子。尺子要是歪的，后面所有人量出来的都是错的。第 26 章我们会看到，这把尺子最终会变成一个“单票权重上限”——风险等级保守的人，上限是 15%；均衡是 20%；成长是 30%。

量化分析师 (Quant)。它是队伍里最像“研究员”的人，任务最多，七项：市场扫描 (MARKET_SCAN)、因子健康度检查 (FACTOR_HEALTH)、选股 (STOCK_SCREENING)、组合模拟 (PORTFOLIO_SIMULATION)，再加上寻找新因子 (ALPHA_DISCOVERY)、判断市场状态 (REGIME_DETECTION)、因子失效分析 (FACTOR_BREAK_ANALYSIS)。

它的产出必须是**可复算的数字**，不是形容词。“动量因子最近 20 天有效”这句话在它这里是不合格的，合格的写法是“动量因子最近 20 天的 RankIC 是某某值，覆盖了百分之多少的样本”。

风控师 (Risk)。它是队伍里唯一有否决权的人，六项任务全部围绕“最坏情况”：隔夜风险 (OVERNIGHT_RISK)、日终风险 (END_OF_DAY_RISK)、交易风险检查 (TRADE_RISK_CHECK)、压力测试 (STRESS_TEST)、风险评估 (RISK_ASSESSMENT)、风险预警 (RISK_ALERT)。

请特别注意“否决”这两个字。风控师不负责提出更好的方案，它只负责说“这个方案不行”。一个只能说不、不能说是的角色，看起来很弱——这恰恰是它的力量来源。第 22 章会证明，一条只能收紧、不能放松的约束，是整个系统里最难被绕过的那一条。

首席投资官 (CIO)。它是唯一能拍板的人，六项任务：晨间简报 (MORNING_BRIEF)、投资决策 (INVESTMENT_DECISION)、组合回顾 (PORTFOLIO_REVIEW)、紧急决策 (EMERGENCY_DECISION)、每日复盘 (DAILY_REVIEW)、T+1 计划 (T1_PLANNING)。

它的位置很微妙。它既不是最懂市场的（那是 Quant），也不是最懂风险的（那是 Risk），它是那个要在两边都不服气的时候给出一个数的人。第 6 到第 8 章会花整整三章讲它怎么拍板，因为这是全书技术含量最高的一段。

操盘手 (Trader)。它唯一的工作是把决定变成订单，五项任务：生成订单 (GENERATE_ORDER)、执行订单 (EXECUTE_ORDER)、监控订单 (MONITOR_ORDER)、执行报告 (EXECUTION_REPORT)、对账 (RECONCILIATION)。

它是队伍里**权限最小**的角色。在系统的权限定义里，它只有一个权限：读用户画像。它不能选股、不能改风险上限、不能否决任何人。它甚至不能“觉得今天这只票不错所以多买一点”——它的买入口被一道硬闸门卡着，第 31 章会讲那道闸门。

这一节的骨架

五个岗位 = 三个问题的两次细分：

谁看 (Planner 向内 + Quant 向外) → 谁想 (Risk 否决 + CIO 拍板) → 谁动手 (Trader)。

每个岗位的产出必须能被下一个岗位直接使用，也必须能被第一个人复核。

1.5 它在系统里长什么样

上面讲的全是“应该怎样”。一个真的跑起来的系统，必须把“应该”写成代码。

第一件事，是把五个角色变成五个常量。在 `internal/agentworkflow/models.go` 里，它们长这样：

```
1 const (
2     AgentCIO      AgentRole = "CIO"      // 首席投资官
3     AgentPlanner AgentRole = "PLANNER"   // 投资规划师
4     AgentQuant    AgentRole = "QUANT"    // 量化分析师
5     AgentRisk     AgentRole = "RISK"     // 风控师
6     AgentTrader   AgentRole = "TRADER"   // 操盘手
7 )
```

代码 1.1 五个角色的常量定义（节选）

这段话在说什么：系统里不存在“一个通用 AI”这种东西。任何一次智能体调用，都必须先声明它是五个角色中的哪一个。角色不是给模型看的人格设定，而是给代码看的身份标签——后面所有的权限判断、工具可见性、任务类型校验，都要拿这个标签去查表。

第二件事，是给每个角色装配它自己能用的工具。在 `internal/brainhost/team_tools.go` 里，系统逐个调用 `RegisterTool` 把工具挂到对应角色身上。当前版本一共装配了 25 个工具入口¹，其中一部分的可见范围如表 1.2 所示。

表 1.2 工具的角色可见性（节选）

工具名	可用角色	它回答的问题
<code>get_tdx_stock_data</code>	QUANT, RISK	这只股票的历史行情长什么样
<code>search_tdx_stocks</code>	QUANT, CIO	在几千只股票里找出符合条件的那几只
<code>run_backtest</code>	QUANT	这个策略放到历史上跑一遍，结果如何
<code>market_sixdim_detect</code>	CIO, PLANNER, RISK, QUANT	今天这个市场，适合下多少注
<code>select_next_day_strategy</code>	QUANT, CIO	明天用哪条策略来管持仓
<code>optimize_portfolio</code>	CIO, QUANT	这五只股票各买多少，组合波动最小

数据来源：系统源码 `internal/brainhost/team_tools.go`；核对时间 2026-09-24。

这张表里藏着一个很容易被忽略的设计。`run_backtest`（跑回测）只有 QUANT 能用——因为回测是“研究”，研究的结果必须经过风控和 CIO 才能变成动作。而 `market_sixdim_detect`（市场判势）四个角色都能用，因为“今天市场好不好”是共享上下文，不是任何人的私产。

常见误解·把多智能体理解成“多开几个聊天窗口”

很多介绍多智能体的文章，画面上是五个并排的对话框，每个对话框里贴一段不同的人格设定：“你是一个谨慎的风控师”“你是一个激进的交易员”。

这不是多智能体，这是多开窗口。它们的区别只有一个问题就能测出来：如果那个“激进交易员”窗口直接说“买入 603xxx 一万股”，系统会真的下单吗？

在这个系统里不会——因为下单权限只给了 `AgentTrader`，而 `AgentTrader` 的工具集里根本没有“自己决定买什么”这个能力。人格设定挡不住越权，只有代码里的权限表挡得住。所以判断一个多智能体系统是真货还是演示品，不看它有几个角色，看它有几个会拒绝的角色。

¹数据来源：系统源码 `internal/brainhost/team_tools.go`，注册调用点位于第 338-452 行区间；核对时间 2026-09-24。

常见误解·AI 是“股神”，所以应该让它自由发挥 我在最初那版提示词里，用的词是“你是一位经验丰富的量化投资专家”。

这句话听起来无害，其实是在给模型发一张空白支票：它被授权“凭本事判断”。而语言模型的本事，恰恰是生成看起来合理的文本——这两件事在投资里几乎是反义词。

后来的做法是彻底反过来：不再问它“你懂不懂”，而是给它一份合同，告诉它该做哪几步、能做哪些事、产出必须长什么样、数据取不到时必须说什么。

一个诚实的失败比一个漂亮的编造有价值一万倍。系统里为此专门留了两个“拒答码”——`DATA_UNAVAILABLE` 和 `DATA_INVALID`。它们的存在本身就是设计：如果不给模型留一个“我不知道”的出口，它就只能编。

1.6 它还没有被约束住

现在你大概明白这支队伍长什么样了。但有三个问题还没回答，它们正好是接下来三章的题目。

问题一：划分了岗位，谁听谁的？我们说风控师有否决权，说操盘手权限最小。这些说法在系统的哪个文件里？如果一个角色被删了权限记录，系统会放行还是拒绝？这是第 2 章的内容。

问题二：凭什么相信它真的照做了？“任务类型”这四个字听起来像文档里的分类，但在系统里它是可执行的约束。一个模糊的指令会被拆成什么？谁检查它有没有跑偏？这是第 3 章的内容。

问题三：它拿什么去做？我们反复说“模型的产出必须是可复算的数字”，可是模型自己算不算数？谁来算？这是第 4 章的内容。

回到开头那张贴在显示器边上的纸。它现在还在，只是内容变了。原来写的是九件事，现在写的是五个岗位——以及每个岗位不许做的事。

一个人当然没办法同时当研究员、风控和操盘手。但他可以当那个设岗、定规、看记录的人。这大概是这本书里最重要的一次身份转换：你从“操盘的人”，变成了“开公司的人”。

本章的真实出处

- 五个角色的常量与中英映射：`internal/agentworkflow/models.go`
- 27 个任务类型的定义：`internal/agentworkflow/models.go`
- 权限矩阵与三条硬约束：`internal/agentworkflow/permissions.go`
- 团队装配与工具的按角色注册：`internal/brainhost/team_tools.go`
- 前端“AI 团队”页（5 个状态卡片、市场时段指示、8 权限 × 5 角色矩阵）：`ui/src/pages/` 下的团队页面，路由表在 `ui/src/App.tsx`

动手试一试（五分钟）

1. 打开系统，进入左侧菜单的 **AI 团队** 页。
2. 找到“权限矩阵”表格。它是 8 行（操作权限）乘 5 列（角色）。逐行看：哪些操作同时被两个以上角色共享？哪些操作只属于一个角色？
3. 找到 `market_sixdim_detect` 对应的那一行“市场判势”，看它被授予了哪几个角色。然后再

找“执行订单”，看它被授予了谁。

4. 自己回答一个问题：如果让你给这五个角色各删掉一个权限，你会删谁的那个？为什么？

这个页面的数据每 30 秒自动刷新一次，所以你可以一边看一边等它变化。

下章我们就去看那张权限表——看它怎么写进代码，以及“让能改风险上限的人同时执行下单”这句话为什么是一起真实的、代价很高的设计错误。

第二章

五个岗位，五条不能越过的线

2.1 同一条线，写在三个地方，三个数字

二零二六年九月，我做了一次很普通的核对：把几处定义“风控硬限制”的代码并排打开。

系统里有两条硬限制：一只股票最多能占组合多大比例，以及组合最多能容忍多深的回撤。核对结果让我不太舒服——**同一条限制，在不同的地方写着不同的数字。**

配置默认值里，回撤容忍写的是 20%¹；策略引擎的硬限制结构里，同一个回撤容忍写的是 10%²；到了风险中心的合规校验里，30% 与 10% 又被当作文案参数各自写死了一遍³。

我的第一反应是抄写错误。抄错了，改回来就行。

第二反应让我坐了两个小时没动：**一笔买入在下单之前，只会走过其中一条路径。**它被哪一处检查，取决于它是从哪个入口发起的。也就是说，同一笔交易换个入口，面对的就是另一条线。

这几处数字里，最多只有一处是当前的真实口径。项目后来把口径统一成单票 30%、回撤 10%。

这件事在项目里被记成“风控硬限制形同虚设”⁴。我一开始把它当作数值一致性的问题，后来发现完全不是——它是一个**权限问题**。

因为那条线之所以会被写成三个数，根本原因是：**这条线没有主人。**谁都能定义它，谁都不用为它负责。而一条没有主人的线，最后一定会被走它的人改掉。

我在这里第一次想清楚一件事，它后来变成了整本书反复出现的一条原则：

如果一个人既能修改这条线，又能执行下单，那么这条线存不存在，只取决于他的自觉。

用财务的话说：让管账的人自己去提钱——账本是平的，钱也是没的。

2.2 两个生活里的“不”

讲权限很容易写成条文。我们先绕开条文，去看两个真实存在的地方：它们都不允许“一个人说了算”，而且理由不是道德，是结构。

直觉·一个航班上有三种“不” 一次航班起飞前，至少有三个人可以拦下它。

机长可以决定今天飞不飞；签派员可以因为航路天气说“今天不放行”；加油没加够、配载没算好，机长再想飞也飞不起来。

¹数据来源：默认配置 `internal/config/config.go: MaxPositionSize=0.30、MaxDrawdownPct=0.20`。核对时间 2026-09-24。

²数据来源：策略引擎 `internal/policy/engine.go` 的 `HardLimits: MaxSinglePosition=0.30、MaxDrawdown=0.10`。核对时间 2026-09-24。

³数据来源：风险中心 `internal/riskcenter/riskcenter.go` 的校验项把 0.30 与 0.10 硬编码进提示文案（第 233、247 行）。核对时间 2026-09-24。

⁴数据来源：同上，记为项目真实修复记录第 7 条。

三个“不”性质完全不同。机长判断的是“能不能飞得成”，签派判断的是“今天允不允许飞”，配载判断的是“这架飞机现在的状态对不对”。

关键不在谁官大，而在于：说“不”的权力，被分发给了那些不靠“这趟飞成功”来领奖金的人。如果签派员和机长是同一个人、拿同一份奖金，那么恶劣天气下“能不能飞”这个判断，一定会被“这趟必须飞”的动机污染。

直觉·会计和出纳为什么必须分开 一家公司管钱，其实只有两个动作：记账，和付款。

如果这两个动作由同一个人做，“账上还剩多少”和“我现在能付多少”就变成了同一个脑子里的两个念头。他不会先记完账再决定付不付——他会一边付，一边给账本找一个说法。

分开之后，两个人各自都不完整：会计不能说“钱够”，出纳不能说“账对”。谁都不能单独把一件事办成，但谁都能单独把一件事拦下。

这就是分工真正的作用。它不产生能力，它产生否决。

第一个类比回答“为什么要说不能的权力分出去”，第二个回答“分出去之后会变成什么样”。两句话合起来，就是本章要落到代码里的东西。

2.3 如果不这么做，会怎样

有人会说：我知道不能让同一个人既管账又提钱，那我就在提示词里写一句“你不许同时做这两件事”，不就完了？

我在最早期那版设计里，权限就是“一张清单”：每个角色下面列几条“它能做什么”。清单写在系统提示词里，措辞也算清楚。

它失效的方式有三种，而且都不报错。

第一种，模型会改口。同一个角色，上下文里被塞进更多信息之后，“我能不能做这件事”的判断会漂移。今天它认为“我只是建议一下”，明天它认为“我这是在帮 CIO 拍板”。清单是一句形容词，形容词没有边界。

第二种，多轮累积。一次越权不致命。致命的是越权的结果变成了下一轮的输入。“我已经定下这只股票”这句话一旦出现，后面的风控环节就会围着它打转。

第三种，没人能验证。提示词写在代码里，本质是一次字符串拼接。跑完之后，日志里只有它说的话，没有“它当时有没有资格说这句话”的记录。你想查它有没有越权，只能把那段文字重读一遍，然后自己判断。

第三种最要命。它意味着：权限这件事在系统里没有任何一个地方被“计算”过，全都靠人读。

2.4 从“职责描述”到“权限矩阵”

先给一个“差不多对”的模型：分工就是谁负责什么。

它对了一半。它的问题是：“负责什么”是名词，而权限是动词。

“风控师负责风险”是名词。它能被解释成一百件事：评估风险、决定不买、建议减仓、直接卖掉一半。“能不能提交一张卖单”是动词，只有两个答案。

近似模型在这里失效了——**颗粒度不同**。名词回答不了“具体哪个动作可以、哪个不行”。所以权限不能写成职责，只能写成“角色 × 动作”的格子。

严谨版本长这样。

定义 2.1 (权限矩阵). 设 R 为角色集合、 P 为动作集合。权限矩阵 (*permission matrix*) 是一个映射

$$A: R \times P \rightarrow \{\text{允许}, \text{拒绝}\},$$

只有当 $A(r, p) = \text{允许}$ 时，角色 r 才能执行动作 p ；未在表中显式列出的 (r, p) ，一律按拒绝处理。

一句话解释：这张表不是“每个角色能做什么”的说明书，而是“某个角色做某个动作时，系统放不放行”的开关。表里没有的格子，默认是关着的。

命题 2.2 (默认拒绝). 若权限表里既没有角色 r 与动作 p 的记录，默认矩阵的 r 行里也没有 p ，则系统判定的结果是不允许。

这个命题在说什么：权限系统的默认答案必须是“不”，做不到这一点的权限系统等于没有权限。

证明. 系统的权限查询按固定顺序进行 (`internal/agentworkflow/permissions.go`):

(1) 先在权限表中按 (r, p) 查询。若命中一条记录，就直接返回该记录里的布尔值，查询结束。

(2) 若查询结果为“没有这条记录”，进入回退分支：取出默认矩阵中 r 对应的那一行；若该行不存在，或该行里没有键 p ，返回 `false`。

(3) 只有当“行存在且键 p 存在”时，才返回该键的布尔值。

于是“既无记录、也未列出”这一情形必然走到返回 `false` 的那一条语句上。命题成立。 ■

近似模型的第二处失效：**它管不了新来的角色**。如果我明天加第六个岗位，一个只写职责的模型不会告诉我它默认该有哪些权限；矩阵模型的答案却是确定的——一行都不加，等于什么都没有。

第三处失效最深：近似模型看不见“谁能否决谁”。这是名词清单天生缺的东西，下面单独说。

2.5 五条线：它在系统里长什么样

现在把话说准。这五条线，第一条是流水线，第二条是权限矩阵，第三条是三句写死在代码里的硬约束。

2.5.1 第一条线：流水线

和第 chapter 一章那张流水线图相比，这张图每一格下面多了一个动词。这不是修辞：这五个动词在系统里是有名字的，它们决定每个角色在这条链上**只能推进哪一步**。



图 2.1 五个岗位在权限链上的位置。每一格下面的动词是这个角色在这条链上唯一被允许推进的那一步。

表 2.1 五条线的完整定义

角色	链上动作	系统里对应的一条“不能”
AgentPlanner	PROPOSE	在默认矩阵里挂着“禁止批准投资”（PermForbidApproveInvestment）与“禁止执行订单”（PermForbidExecuteOrder）
AgentQuant	EVIDENCE	同上两条；它只有推荐类权限（推荐资产 / 推荐因子 / 推荐组合 / 生成候选），没有一条决策类权限
AgentRisk	VETO	唯一允许否决的角色；它的权限里包含“风险否决”（PermRiskReject）与“紧急冻结”（PermEmergencyFreeze）
AgentCIO	DECIDE	它的权限行里没有任何一条订单类权限，也没有“禁止执行订单”这一条——因为它不需要禁止，它本来就不在订单路径上
AgentTrader	EXECUTE	挂着三条禁止：禁止选股（PermForbidSelectStock）、禁止改目标权重（PermForbidChangeTargetWeight）、禁止绕过风控（PermForbidOverrideRisk）

前四列来自默认权限矩阵 `PermissionMatrix`；“紧急冻结”等权限名见同一文件的权限常量区。
 数据来源：系统源码 `internal/agentworkflow/models.go`（`PermissionMatrix` 与权限常量，第 49-143 行）；核对时间 2026-09-24。

看第四列。系统没有只写“能做什么”，它同时写下了“不能做什么”，而且这些“不能”被做成了有名字的权限项。

这是个很实用的工程决定：如果“不能做”只是一句注释，它就会随代码一起烂掉；把它做成一条权限记录，它就变成了一个可以查、可以测、可以被审计的对象。

2.5.2 第二条线：权限矩阵

```

1 AgentCIO: {
2     PermSelectAsset:           true,  // 选择标的
3     PermSetTargetWeight:      true,  // 设定目标权重
4     PermApproveInvestment:    true,  // 批准投资
5     PermChangeInvestmentDecision: true, // 变更投资决策
6     // ... 另有 增/减/清仓、批准/拒绝投资方案 等
7 },
8 AgentTrader: {
9     PermCreateOrder:          true,  // 创建订单
10    PermSubmitOrder:          true,  // 提交订单
11    PermMonitorOrder:         true,  // 监控订单
12    PermCancelOrder:          true,  // 撤单
13    PermReconcile:            true,  // 对账
14    PermForbidSelectStock:     true,  // 禁止选股
15    PermForbidChangeTargetWeight: true, // 禁止修改目标权重
16    PermForbidOverrideRisk:    true,  // 禁止绕过风控
17 },
  
```

代码 2.1 权限矩阵里 TRADER 与 CIO 两行（节选）

这段话在说什么：CIO 那一行全是“决定什么”，Trader 那一行全是“执行什么”，两行的交集是空的。你把两张表并排看一遍，就能回答“谁能下单、谁能选股”——不需要读任何一段自然语言。

这张表最容易被忽略的一行

规划师那一行里挂着一条 `PermRiskApprove`（风险批准）。按它当前的三项职责（创建画像、修改目标、创建方案），这条记录并不使用。

我把它写出来不是为了挑刺，而是想说一件很实在的事：**真实系统的权限表会漂移**。所以判断权限不能只看“设计意图”，必须看“表里到底写了什么”——这也正是第 2.7 节那份“动手试一试”要你做的事。

前端把这件事做成了一张 8 个操作权限 × 5 个角色的矩阵表格，绿色表示允许、灰色表示不允许⁵。代码里的默认矩阵是按角色分行、按权限分列的另一张表，条目比 8 条多——CIO 一行就有 9 条。两张表的用途不同：代码那张用来判权，前端那张用来给人看。

2.5.3 第三条线：三句写死的硬约束

权限表是可以改的：它在数据库里有一张表，能授、能撤。但有三件事不在那张表里。

```
1 // CanExecuteOrder 检查智能体是否可以执行订单
2 func (pm *PermissionManager) CanExecuteOrder(agentID string) bool {
3     // 只有 Trader 可以执行订单
4     return agentID == AgentTrader
5 }
6
7 // CanSelectStock 检查智能体是否可以选股票
8 func (pm *PermissionManager) CanSelectStock(agentID string) bool {
9     // 只有 CIO 可以正式选入股票
10    return agentID == AgentCIO
11 }
12
13 // HasVetoPower 检查智能体是否有否决权
14 func (pm *PermissionManager) HasVetoPower(agentID string) bool {
15     // 只有 Risk 有否决权
16     return agentID == AgentRisk
17 }
```

代码 2.2 三条硬约束（`internal/agentworkflow/permissions.go`，原文）

这段话在说什么：这三个函数里没有数据库查询。它们不读权限表，不看默认矩阵，只做一次身份比较。这意味着——

“谁能下单”“谁能选股”“谁能否决”，不是配置项，是常量。你把数据库里 Trader 的所有权限记录都改成“禁止”，这三条依然成立。这不是偷懒，这是刻意的：这三件事动不得。

⁵数据来源：docs/用户操作手册.md 第 5 节“AI 团队页”；该系统事实清单第 3 节记为“8 个操作权限 × 5 个角色”。核对时间 2026-09-24。

命题 2.3 (三条硬约束互斥). 对任何一个角色 r , “可执行订单” “可选择股票” “有否决权” 这三条硬约束中, 至多只有一条成立。

这个命题在说什么：权力是分给三个不同的人的，不会出现一个人同时握着“下单”和“否决”。

证明. 由上一段源码：判定“可执行订单”成立的充要条件是 $r = \text{AgentTrader}$ ；判定“可选择股票”成立的充要条件是 $r = \text{AgentCIO}$ ；判定“有否决权”成立的充要条件是 $r = \text{AgentRisk}$ 。

在 `internal/agentworkflow/models.go` 中，`AgentTrader`、`AgentCIO`、`AgentRisk` 是三个取不同字符串值的常量（分别对应操盘手、首席投资官、风控师）。

于是三个判定各自成立时， r 必须分别等于三个互不相同的常量，故不可能同时取到两个。命题成立。 ■

把这个命题和第 2.2 条放一起看，本章的主张就完整了：默认拒绝保证了“没写的就是不许”，三条硬约束保证了“最要紧的三件事连改都不许改”。

2.6 常见误解与纠偏

常见误解 · 权限就是提示词里写一句“别乱来” 我在最初那版提示词里写过：“你是风控师，不要越权执行交易。”

这句话的实际效果是零。因为模型的输出空间没有任何一处被裁剪过——它依然可以生成一句“我决定卖出这只股票”，而这句话会被下游当成一段文本读进去。

后来的做法是反过来：不再描述边界，而是把门装上。下单要经过 `CanExecuteOrder`；选股要经过 `CanSelectStock`；否决要走 `HasVetoPower`。判断一个系统有没有权限，不看它提示词写得有多严，看它有没有一个会返回 `false` 的函数。

常见误解 · 决策权最大，否决权只是“提意见” 这是我推翻得最彻底的一个认知。

我一开始觉得，风控师的否决权是一种“建议”：它说了不行，CIO 想想办法还是能过。如果真是这样，那风控就是一块装饰。

但这个系统里的否决权，位置上比决策权更高——因为决策权只能决定“我打算做什么”，否决权能决定“这个打算能不能变成真的”。CIO 拍板一个方案，风控师说不，这个方案就停在原地。反过来不成立：风控师不能说“我要买”，它没有下单权限，也没有选股权限。

一条只能收紧、不能放松的权力，比一条既能收紧也能放松的权力更可靠。第 chapter 二十二 章会把这个结构再讲一遍——那里的主角换成了规则，但骨架完全一样。

常见误解 · 权限存在数据库里，所以它是“配置” “能授能撤”听起来像件好事，但它会让人产生一个错误的安全感：以为把权限放进数据库，就等于权限被管住了。

这个系统里有两条防线，性质完全不同：

数据库里的权限是可以被改的——这是它的用途，也是它的风险点。撤掉某条权限，立刻生效；但如果有人误操作，也同样立刻生效。

那三条硬约束改不动。它们是一句身份比较，不查表。

我后来给自己定的规矩是：凡是“失去了会出人命”的权限，就不放进可配置的那一层。下单、否

决、选股权，都属此类。

2.7 真实出处、动手试一试与下一章

本章的真实出处

- 权限查询顺序与“默认拒绝”回退：`internal/agentworkflow/permissions.go` (`CheckPermission`, 第 21–37 行)
- 三条硬约束的实现：同文件 `CanExecuteOrder` / `CanSelectStock` / `HasVetoPower` (第 89–105 行)
- 默认权限矩阵与全部权限常量：`internal/agentworkflow/models.go` (第 49–143 行)
- 默认数据的落库（把默认矩阵逐条写入权限表）：同文件 `InitDefaultAgentData`
- 前端“AI 团队”页的 8×5 权限矩阵与“可做 / 不可做”清单：`ui/src/pages/AITeam.tsx`, 路由表 `ui/src/App.tsx`

动手试一试（五分钟）

1. 打开系统，进入左侧菜单的 **AI 团队** 页。
2. 展开 PLANNER，把它“不能做”的三条抄下来；再展开 TRADER，把它“不能做”的三条抄下来。
3. 回到权限矩阵那张表，找三行：**执行订单**、**否决决策**、**选择股票**。看清每一行打了几个绿勾。
4. 最后自己回答一句：如果明天系统新增第六个岗位“客服”，而你只写了一句职责描述、没有给它任何一行权限，它能不能下单？（答案在第 2.2 条命题里。）

五条线画完了。但“谁不能做什么”只解决了一半问题：同一句“看看今天的市场”，交到两个人手里，会被做成两份完全不同的东西——一份是一段市场评论，一份是七个字段的证据清单。有了权力边界，还缺一份派活的标准格式。这就是下一章要给每个岗位发的东西：一份写明了“必须做什么、禁止做什么、产出长什么样、什么时候必须停”的合同。

第三章

怎么给 AI 派活，它才不会乱来？

3.1 “帮我看看今天的市场，然后决定买什么”

这是我最早写下的一条指令，也是我这辈子写过的最贵的一句话。

它的语法没有任何问题。它甚至听起来很礼貌——把一件事和另一件事串联起来，像跟人说话。

问题是，我连着跑了三次。

第一次，它给了我一段市场评论：情绪偏暖、量能温和、建议关注科技方向。第二次，它给了我三只股票代码和买入理由。第三次，它给了我一段宏观叙事，末尾加了一句“具体标的建议由专业投顾给出”。

三次回答的差别不是“水平高低的差别”，而是“做的是三件不同的事”。第一次做了市场观察，第二次做了选股，第三次做了免责声明。

我当时以为这是模型的“随机性”。后来我想明白了：不是它随机，是我没说清。“看看市场”这四个字，可以被做成至少十种产出；“决定买什么”这五个字，落在这个系统里根本没有接收方——第 chapter 二 章我们已经看过，选股权属于 CIO，下单权属于 Trader，两个动作都不在这段对话里。

一句模糊的指令，最后必然会变成一种“没人接得住”的产出。

在系统里，“看看今天的市场”这句话对应的不是一段话，而是一个有编号的任务。它叫 pre_planner，我把它完整摘下来给你看：

“看看今天的市场”在系统里的样子

任务名：盘前市场分析 谁做：PLANNER 交付物：pre_market_plan 超时：300 000 毫秒
重试：2 次
必须做的 8 件事：判断隔夜市场状态 → 提取重大市场事件 → 判断事件涉及的资产 → 判断事件重要程度 → 形成今日市场观察事项 → 标记需要 Quant 验证的事项 → 标记需要 Risk 检查的事项 → 生成 pre_market_plan
严禁做的 9 件事：选股、股票排名、生成 Alpha、生成交易信号、计算目标仓位、修改仓位、风险否决、投资决策、交易执行
产出必须包含的 7 个字段：market_status、key_events、affected_assets、expected_impact、items_for_quant、items_for_risk、confidence
交给谁：QUANT 什么都没发生时该说什么：NO_MAJOR_EVENT^a

^a数据来源：以上数字全部来自系统源码 internal/scheduler/task_scheduler.go 中 pre_planner 任务的定义（第 378-419 行）：RequiredActions 8 条、ForbiddenActions 9 条、OutputSchema 7 项、Handoff="Quant"、NOActionCode="NO_MAJOR_EVENT"、TimeoutMs=300000、MaxRetries=2。核对时间 2026-09-24。

同一句话，两个版本：一段有礼貌的祈使句，和一份有交付物、有下游、有超时的任务书。

这一章讲的就是后者——一份给 AI 的合同。

3.2 两个合同里的常识

直觉·给外包公司写合同时，你会写什么 假设你要找一家外包公司做一个网站。你不会只发一句“帮我做个网站”。

你会写清楚：**目的**（给谁看、解决什么问题）、**我提供什么**（素材、账号、品牌规范）、**你必须做的**（首页、内页、后台、适配手机）、**你不许做的**（不许用我的品牌色改我的 Logo、不许把数据传到境外服务器）、**交付格式**（源码、设计稿、可运行的测试地址）、**什么时候停**（验收通过就交付，不要顺手再加功能）、**异常怎么办**（素材缺了要提出来，不许自己编一张图顶上）。

写这些不是为了互相提防，而是因为：一份没有边界的委托，最后一定会以“双方都觉得自己被坑了”结束。

直觉·合同里必须有一条“这笔生意不做” 好的合同都有一个看起来很奇怪的部分：不做成的时候算不算完成。

比如“如果现场勘测发现地基不达标，承包方应当出具书面说明并停止施工”——这句话的意思是：停下来、写一份说明，本身就是一种合格的交付。

如果合同里没有这一条，会发生什么？承包商不能停，因为停下就等于违约。于是他只能想办法让事情“看起来能继续”——比如先按图纸施工，把手上的钱花掉，然后跟你解释“情况比预期复杂”。一套不给“不做”留位置的委托，一定会长出伪造。

第二个类比是这一章真正的重点。我们后面会看到，系统给每一个任务都起了一个“什么都不做”时就该说的话——它叫 `NOActionCode`。

3.3 如果不给合同，会怎样

不给合同的代价有三种，而且都是慢性的：它们不会当场报错，只会让系统在几周后变得不可信。

第一，产出接不住。下游角色要的是一份可核对的东西，模型给的是一段散文。下游没法读散文——它要么放弃使用（于是这一步白干），要么自己从散文里“理解”一个结论（于是这一步的结论，实际上是下游编的）。

第二，无休止扩写。没有停止条件，模型就不知道什么时候该收笔。它会继续补充“进一步的建议”“值得关注的风险”“如果……那么……”。多出来的文字不是免费的：它进了下一轮的上下文，成为下一轮判断的依据。

第三，没有出口，就只能编。这是最要命的一条。任务需要一个数字，而这个数字取不到——如果契约里没有说“取不到就报 `DATA_UNAVAILABLE`”，那么模型唯一能做的，就是生成一个看起来合理的数字。

我在第 chapter 一 章说过一句话，这里再说一遍，因为它是本章的发动机：如果不给模型留一个“我不知道”的出口，它就只能编。

3.4 从“说清楚”到“可校验”

先给一个“差不多对”的模型：派活就是把要求说清楚。

这个模型抓住了本质，但它在两个地方会失效，而且失效得很安静。

失效一：说清楚不等于可校验。我在提示词里写“请给出结构化的分析结果”——这句要求是对的，但它没法被执行。什么叫结构化？几个字段？字段名叫什么？谁来判断它有没有做到？全都停在我脑子里。等到输出回来，只能靠我肉眼判断“这个算不算结构化”。

要让要求可校验，就必须把它变成能和输出逐条比对的清单：输出里必须出现哪些字段、哪些动作必须做过、哪些词不许出现。

失效二：合同只约束“说什么”，不约束“怎么算”。就算我把 12 个字段写得清清楚楚，模型依然可以在 `confidence` 这一栏写一个它自己“感觉”出来的数字。合同管得住格式，管不住真实性。

第二处失效，我们这一章先记下，第 chapter 四 章专门解决——那才是“AI 的手”要管的事。

严谨版本的合同长这样：一份任务契约 + 一道输出守卫。

定义 3.1 (任务契约与输出守卫). 任务契约 (*task contract*) 是对一次任务的完整约束：目的、触发条件、输入、必须做的动作、禁止做的动作、决策边界、可用工具、输出字段、下游交付对象、无动作时的状态码、停止条件、异常策略。

输出守卫 (*output guard*) 是任务结束后的检查器：它把实际输出与契约逐字段比对，不通过就拒绝这次交付。

一句话解释：契约是事前发的活，守卫是事后收的货。

3.5 它在系统里长什么样

3.5.1 第一件东西：12 个约束字段

```

1 // TaskContract Task 契约 v1.0 完整定义
2 type TaskContract struct {
3     Purpose          string    // 任务目的
4     Trigger          string    // 触发条件
5     Inputs           []string // 允许读取的输入
6     RequiredActions  []string // 必须执行的动作
7     ForbiddenActions []string // 严禁执行的动作
8     DecisionBoundary string    // 决策边界（可以决定什么/不能决定什么）
9     AllowedTools     []string // 允许调用的工具
10    OutputSchema     []string // 输出字段
11    Handoff          string    // 下游交付对象
12    NOActionCode     string    // NO_ACTION 状态码
13    StopCondition    string    // 停止条件
14    ExceptionPolicy  string    // 异常处理策略
15 }
```


代码 3.1 任务契约 v1.0 的完整定义（`internal/scheduler/task_scheduler.go`，原文）

这段话在说什么：这 12 个字段不是文档目录，是代码里的结构体。一个任务如果没有被填满，它连编译都过不了——这是这一章最硬的一条保证：**合同的完整性不是靠人检查，是靠类型检查。**

把这 12 个字段翻译成人话，就是外包合同的七个问题：

表 3.1 12 个约束字段对应的人话

字段	它在回答的问题
Purpose / Trigger	为什么做这件事？什么时候做？
Inputs / AllowedTools	你能拿什么？你能用什么工具去拿？
RequiredActions / StopCondition	必须做哪几步？做到哪一步就该收手？
ForbiddenActions / DecisionBoundary	哪些动作碰都不能碰？你能决定什么、不能决定什么？
OutputSchema / Handoff	产出必须长什么样？交给谁？
NOActionCode / ExceptionPolicy	什么都不该做时说什么？出岔子时怎么办？

DecisionBoundary 在 `pre_planner` 里写作“可以决定市场环境判断；不能决定投资标的和交易行为”。

数据来源：系统源码 `internal/scheduler/task_scheduler.go` 的 `TaskContract` 定义（第 264–278 行）与 `pre_planner` 实例（第 393–418 行）；核对时间 2026-09-24。

还有一个字段值得单独说：任务定义里除了契约，还有一个 `Scope`，分 `Allowed` 与 `NotAllowed` 两组。`pre_planner` 的“允许”是 7 项（市场数据、新闻、公告、宏观数据、指数、行业事件、市场状态），“不许”是 6 项（个股打分、因子计算、风险计算、仓位计算、交易决策、交易执行）¹。

契约管“产出”，`Scope` 管“读什么”。两个都要，因为一份干净的产出也可能来自一堆不该读的数据。

3.5.2 第二件东西：每个任务都有自己的“什么都不做”

表 3.2 系统里 16 个任务的无动作状态码（`NOActionCode`）

状态码	用在几个任务上	用在哪些任务上
<code>NO_DECISION_CHANGE</code>	4	<code>pre_cio</code> 、 <code>in_event_cio</code> 、 <code>post_cio</code> 、 <code>night_cio</code>
<code>NO_EXECUTION</code>	3	<code>in_market_monitor</code> 、 <code>in_event_trader</code> 、 <code>post_trader</code>
<code>RISK_NORMAL</code>	3	<code>pre_risk</code> 、 <code>in_risk_monitor</code> 、 <code>post_risk</code>
<code>NO_SIGNAL_CHANGE</code>	3	<code>pre_quant</code> 、 <code>in_signal_monitor</code> 、 <code>night_quant</code>
<code>NO_MAJOR_EVENT</code>	2	<code>pre_planner</code> 、 <code>in_event_risk</code>
<code>NO_TUNE_NEEDED</code>	1	<code>post_quant_tune</code>

状态码共 6 种，覆盖 16 个任务定义；同一状态码被不同角色共用（例如 `RISK_NORMAL` 同时被风控师盘前、盘中、盘后三个任务使用）。

数据来源：系统源码 `internal/scheduler/task_scheduler.go` 的 `getOptimizedTasks()` 全部任务定义；核对时间 2026-09-24。

这张表是全章我最想让你记住的东西。它说明系统给每一种“今天没什么要做的”都起了名字：

¹数据来源：系统源码 `internal/scheduler/task_scheduler.go`，`pre_planner` 的 `Scope`（第 389–392 行）。核对时间 2026-09-24。

风控师看完一圈，一切正常——这不是“没干活”，这是 `RISK_NORMAL`。操盘手盯了一天，没有需要执行的动作——这不是失败，这是 `NO_EXECUTION`。量化分析师复盘完因子，不需要调仓——这是 `NO_TUNE_NEEDED`。

给“不做”起名字有三个立刻见效的好处。

第一，它可统计。“今天有多少个任务是 `NO_EXECUTION`？”这个问题，能直接回答“系统是不是天天在瞎折腾”。

第二，它不可伪造。如果输出必须是这 6 个码之一，那么“编一个数字”这条路就变窄了——你必须先声明“今天本就无事发生”。

第三，它让任务可以正常结束。一个没有 `NOActionCode` 的任务，遇到“确实不该做任何事”的局面时只剩两种选择：硬编一个动作出来，或者卡在那里。两种都不是好结局。

3.5.3 第三件东西：输出守卫的三层检查

任务跑完，输出回来。系统里的 `OutputGuard` 会按顺序做三件事²：

1. **Schema 验证：**输出里的字段名，有没有对得上契约里的 `OutputSchema`；
2. **禁止输出检查：**输出里有没有出现这个角色不该出现的字段（例如风控师的输出里出现 `stock_pick`）；
3. **决策边界验证：**这个角色有没有在自己的输出里“顺手做了决定”。

第一层的写法是“警告”：当输出的字段名一个都对不上契约时，记一条 `OUTPUT_SCHEMA_MISMATCH` 警告。

第二层的写法是“违规”：它会命中就记 `FORBIDDEN_OUTPUT`，并且直接把这次交付判为不通过。

第三层的写法是“越权”：量化分析师输出 `decision`、操盘手输出 `recommendation`，都会被记成 `DECISION_BOUNDARY_VIOLATION`。

常见误解·契约写好就自动生效——检查器也是代码，代码会写错 写这一章的时候，我把守卫的三个函数逐行读了一遍，然后发现了一件很尴尬的事。

第二层检查用的键名写成这样：“Planner”、“Quant”、“Risk”、“Trader”——首字母大写。而任务上记录的角色值是全大写：“PLANNER”、“QUANT”、“RISK”、“TRADER”。这两个字符串不相等，所以查表查不到，函数在第一行就返回了，什么也没检查。

五个角色里只有 `CIO` 恰好两边写法相同，于是它成了唯一真的被检查过的角色。

第三层更彻底：它的 `switch` 只列了 “Risk”、“Quant”、“Trader”、“Planner” 四个分支，同样一个都匹配不上。

我把这件事写出来，不是为了自曝其短，而是因为这正是本章最容易产生的误解：把“检查清单”当成“检查器”。一份写在纸上的三层检查，和一段真的会返回违规的代码，是两回事。任何一条“我们做了校验”的说法，都要能回答一个追问：它在哪个分支里返回 `false`？

²数据来源：系统源码 `internal/scheduler/output_guard.go` 的 `ValidateOutput`（第 28-49 行）依次调用 `validateOutputSchema`、`validateForbiddenOutputs`、`validateDecisionBoundary`。核对时间 2026-09-24。

常见误解·AI 编数据是个道德问题 “模型又在胡编数字了”——这句话把问题归错了地方。模型的输出空间是“所有可能的文本”。只要“编一个数字”和“说我不知道”在产出上一样好用，它就一定会选那个让任务“看起来完成”的。这不是品德，这是目标函数。真正能改的是接口：把“我不知道”做成一个有名字、被接受、能通过校验的产出。系统里给这件事留了两个码——数据取不到时返回 `DATA_UNAVAILABLE`，数据取到了但不合法（口径不对、越界、对不上）时返回 `DATA_INVALID`^a。这两个码的存在本身就是设计。它们不是“错误日志”，它们是一种合法的交付结果。

^a数据来源：两个码在任务定义中作为 `ExceptionPolicy` 的指定返回写入，例如 `pre_planner` 写“数据不可用时返回 `DATA_UNAVAILABLE`，不得猜测或编造”，`pre_quant` 写“数据异常时标记模型异常，返回 `DATA_INVALID`，不得自行修改模型”；出处 `internal/scheduler/task_scheduler.go`。核对时间 2026-09-24。

常见误解·任务失败了才叫异常 我一开始把任务状态想成两态：成功，或者失败。系统里是五态：`SUCCESS`、`RUNNING`、`FAILED`、`PENDING`、`SKIPPED`^a。其中 `SKIPPED` 和 `PENDING` 是我最初完全没有想过的两种状态，而它们恰恰最常见。`PENDING` 是“还没轮到它”——盘前七件事是排队做的，第五件事在前四件没做完之前，本来就应该是 `PENDING`。`SKIPPED` 是“这一轮不需要它”。它不是失败，也不是懒惰，它是一个被明确做出的决定。我后来的体会是：一个只有“成功 / 失败”的系统，会把大量“正常的不作为”误判成故障，然后你会开始忽略告警。而忽略告警，比没有告警更危险。

^a数据来源：五态模型见 `docs/系统功能详细设计书.md` 第 10.4 节“任务状态与阶段”；核对时间 2026-09-24。

3.6 真实出处、动手试一试与下一章

本章的真实出处

- 任务契约 *v1.0* 的 12 个字段与 `AgentTask / TaskScope` 定义：`internal/scheduler/task_scheduler.go`（第 256–299 行）
- 16 个任务的完整契约实例（含 `pre_planner`、`pre_quant`、`pre_risk`）：同文件 `getOptimizedTasks()`
- 输出守卫三层检查与任务前置检查：`internal/scheduler/output_guard.go` (`OutputGuard.ValidateOutput`、`ConstraintChecker.CheckPreExecution`)
- 契约不完整时的阻断：同文件 `CheckPreExecution` 里 `CONTRACT_INCOMPLETE` 违规与 `NO_NO_ACTION_CODE` 警告
- 27 个任务类型常量与 5 态任务状态：`internal/agentworkflow/models.go`

动手试一试（十分钟）

1. 打开系统，进“实时活动”页，等它跑完一次盘前阶段。
2. 在“今日任务日志”里找到 `pre_planner` 这条任务，看它的状态与耗时。
3. 找到同一次运行里风控师的那条风险检查任务，看它的结果状态。如果它今天给的是 `RISK_NORMAL`，那么“风控师今天没干活”这句话是错的——它干完了活，结论是“无事”。
4. 把 `pre_planner` 的 8 条必须动作抄下来，然后自己写一句话派活给一个真人同事，要求他做到同样的 8 步、产出同样 7 个字段。你会发现你写的那句话，比你想象的长得多。

合同签好了，每个角色都清楚自己该做什么、不许做什么、产出长什么样。但还有一个问题被我们刚刚按下没答：报 `RISK_NORMAL` 之前，风控师是拿什么看出来“一切正常”的？它自己算了一遍波动率，还是它调用了某个会算的东西？合同规定了它能做什么，可它拿什么去做？下一章我们去看它的手——它能动，但只能动你允许的那几样东西。

第四章

AI 的手：它能动手，但只能动你允许的那几样东西

4.1 为什么不让它自己算收益率

我做过一个实验，只花了十分钟，但那次之后我改掉了整套设计里的一块砖。

我把一只股票最近 60 个交易日的收盘价粘进对话里，然后问它：“帮我算一下这 60 天的日收益率标准差。”

它给了。数字看起来合理，格式很漂亮，还贴心地把百分比保留了两位小数。

我又换了一个写法问同样的问题：“请用这 60 个数字，按总体标准差（除以 n ）和样本标准差（除以 $n - 1$ ）分别算一遍，两个都给我。”

它给了两个数，而且两个数一模一样。

这就是问题所在。它不是“算得不够准”，它是根本没有在算。它在生成一个“看起来像标准差”的数字——这个数字由价格量级、波动直觉和随机性共同决定，而不是由那 60 个数字决定。

而真正让我后背发凉的，不是这个错误本身，是没有人会发现它。

一个模型在报告里写“年化收益 18.7%”，你怎么核对？你得把那几百行价格重新拉出来自己算一遍。也就是说：凡是让模型“心算”出来的数字，它的成本不是错，而是不可核对。

这一章只有一条原则

LLM 负责理解与决策协作，量化引擎负责计算事实。

翻译成成人话：模型负责“这件事该不该做、怎么做更稳妥”，机器负责“这个数到底是多少”。两者之间不许互相串岗——模型不许算数，引擎不许做决定。

4.2 给新员工开账号

直觉 · 新员工入职，你给他开哪些权限 一个新人来上班，第一件事是开系统账号。

你会给他“查库存”的权限，因为他的工作要用；你不会给他“改价格”的权限，因为他用不着——而且一旦他改错了，损失不会停在他自己那一格。

你还得给他工具：一把能算的计算器，和一本能查的资料册。

但你不会让他“用心算”。心算这件事的麻烦在于：它没有痕迹。他算对了，你没看见；他算错了，你也没看见。计算器不一样——它按了什么、算出什么，都是可以复盘的。

所以给 AI 配工具，其实在回答三个问题：它需要什么能力？哪些能力它不能有？它拿什么去做这些事？前两个问题由第 chapter 二 章和第 chapter 三 章回答过了，这一章回答第三个。

直觉 · 计算器和心算的区别不在速度 很多人以为“工具”是为了让模型更强、更快。

不是。工具真正的价值在于把一次计算变成一个可以被检查的对象。

心算的结果只有一个数字，你只能选择信或者不信。计算器按下去之后，会留下“输入是什么、用

了哪个公式、输出是什么”。有了这三样，一个数字就从“结论”变成了“证据”。

4.3 不给工具，会怎样

三种代价，一种比一种深。

第一，错得没有规律。心算的错误不是随机噪声，它跟数字的“长相”有关：整数好看的数字容易被写对，带小数的数据容易出错。这意味错误不是均匀分布的，你没法用统计方法把它平均掉。

第二，错得无法传播。一个模型算出来的收益率，没法成为下一个模块的输入。因为下一个模块拿到的是一段字符串，不是数。它要么放弃使用，要么自己再“理解”一遍——而“理解”这一步又是一次心算。

第三，错得无法追责。这是最深的一层。如果系统里所有数字都是模型“说”出来的，那么当组合亏钱时，你无法区分三种情况：数据本身错了、计算错了、还是决策错了。三种情况的修复方式完全不同，而你连问题在哪一类都不知道。

所以工具不是“增强”，工具是把计算从语言里搬到代码里。搬过去之后，计算才有痕迹。

4.4 从“给模型工具”到“工具契约”

先给一个“差不多对”的模型：工具就是给模型几个可以调用的函数。

这个模型是对的起点，但它在两处会塌。

塌陷一：如果工具返回的是一句话，等于没有工具。假设有一个工具叫“看看今天的市场”，它返回一段市场评论。那这段评论只能再喂给模型，模型还要再“理解”一次——我们又回到了心算。

要让工具真的有用，它的返回值必须是**结构化的**：字段名固定、类型固定、能被下一个模块直接当输入用。比如不是“今天市场偏弱”，而是 `market_tag=" 退潮风险"、position_rate=0.06`。

塌陷二：如果工具没有“谁能用”的属性，工具越多越危险。一个工具的集合，同时也是一个人能触达的**全部动作**。给风控师装上“下单”工具，就等于给了它下单权——不管你提示词里怎么叮嘱。

所以严格的工具不是函数，是一份**四件套 + 一张可见性表**。

定义 4.1 (工具契约)。一个工具由四部分组成：**名字、说明、参数结构**（JSON Schema 形式的输入声明）、**执行方法**。此外，每个工具还带一组**可见角色**——只有列在其中的角色才拿得到这个工具。

一句话解释：前四样决定“这个工具怎么被调用”，第五样决定“谁有资格调用它”。

4.5 它在系统里长什么样

4.5.1 一个工具的解剖

```

1 type Tool interface {
2     Name() string
3     Description() string
4     GetDefinition() ToolDefinition
5     Execute(ctx context.Context, args map[string]interface{}) (interface{}, error)
6 }

```

代码 4.1 工具的执行接口（internal/tools/tools.go，原文）

这段话在说什么：四个函数，四个问题。Name() 答“我叫什么”；Description() 答“我干什么”，而这段说明就是模型选择工具的唯一依据；GetDefinition() 答“我的参数长什么样”，返回的是给 LLM 看的函数声明；Execute() 才是真的动手。

注意最后一行：它返回的是 (interface{}, error)——一个结构化对象，加一个可能存在的错误。没有第三种返回方式。工具不能说“我觉得大概是”，它只能给出结构，或者报错。

```

1 Parameters: map[string]interface{}{
2     "type": "object",
3     "properties": map[string]interface{}{
4         "save": map[string]interface{}{
5             "type": "boolean",
6             "description": "是否将本次判势结果落库到 market_sixdim_daily 表（默认 false）",
7         },
8         "action": map[string]interface{}{
9             "type": "string",
10            "enum": []string{"detect", "sources"},
11            "description": "默认 detect=执行判势；sources=仅返回数据源配置与运行指标（不判势）"
12        },
13    },
14    "required": []string{},
15 }

```

代码 4.2 market_sixdim_detect 的参数声明（internal/tools/sixdim_tool.go，原文节选）

这段话在说什么：这个工具只有两个参数，而且第二个参数只能是两个值之一。参数写死在 JSON Schema 里，模型不能自己发明一个 action=" 随便试试"。

4.5.2 取不到数据，工具会拒绝

这是本章我最喜欢的一段代码。六维判势工具在动手之前，先检查一件事：

```

1 if t.mkt == nil || !t.mkt.HasStockDB() {
2     return nil, fmt.Errorf("DuckDB 不可用，无法执行市场六维判势")
3 }

```

代码 4.3 工具在动手前先检查数据是否可用（internal/tools/sixdim_tool.go，原文）

这段话在说什么：数据库没挂上，工具不返回“零分”，也不返回“默认中性”，它返回一个错误。工具宁可可不干活，也不编数据。

这就是第 chapter 三 章那两个拒答码在工具层的对应物：DATA_UNAVAILABLE 不是一个礼貌用语，它是链条上每一环都要遵守的行为约定。

拒绝之外的另一半，是它成功时交出来的东西：

```
1 return map[string]interface{}{
2     "as_of":          time.Now().Format("2006-01-02"),
3     "dim_scores":      report.DimScores,
4     "dim_chinese":     sixdim.DimChinese,
5     "raw_total_score": report.RawTotalScore,
6     "conflict_count":  report.ConflictCount,
7     "adjusted_total_score": report.AdjustedTotalScore,
8     "position_rate":   report.PositionRate,
9     "market_tag":      report.MarketTag,
10    "position_advice":  fmt.Sprintf("策略信号仓位 = 原始信号仓位 × %.2f (%s)", ...),
11    "sources":          report.Sources,
12    "note":             "技术/广度/量能来自DuckDB真实行情；...所有实时源失败均自动回退并标注，无伪造。",
13 }, nil
```

代码 4.4 同一个工具的返回结构（internal/tools/sixdim_tool.go，原文节选）

这段话在说什么：它交出的不是一句话，而是一张表。六个维度的分数在一栏，冲突数在一栏，仓位系数在一栏，市场标签在一栏。

最要紧的是 sources 这一栏——每个数字“从哪来”和数字一起返回。第 chapter 二十 章会看到，这一栏里留着的是“这天的数据是用哪个源算的”的原文记录。

还有 note 里那句朴素的话：“所有实时源失败均自动回退并标注，无伪造。”这是工具对自己行为的公开承诺，而且是写在返回值里的，不是写在文档里的。

而工具算出来的东西是能核对的。比如六维判势的结果落库之后，第二天可以逐条查：

表 4.1 market_sixdim_detect 最近的真实落库记录（节选两日）

交易日	修正后总分	仓位系数	冲突数	市场标签
2026-09-10	38.43	0.10	4	退潮风险
2026-09-11	25.71	0.06	5	退潮风险

表中“总分”指冲突修正后的 adjusted_total_score，“仓位系数”指 position_rate，“冲突数”指 conflict_count。

数据来源：库 data/stock.duckdb 表 market_sixdim_daily 的真实记录；核对时间 2026-09-24。

表里的“修正后总分”38.43 与 25.71¹、“仓位系数”0.10 与 0.06²³，都是从库里直接读出来的，不是模型写的。工具的价值就在这里：它把“今天市场怎么样”从一句形容词，变成了一

¹真实数据：本数字直接取自系统运行库 data/stock.duckdb，未做任何模拟或平滑。

²真实数据：本数字直接取自系统运行库 data/stock.duckdb，未做任何模拟或平滑。

³数据来源：数值取自 market_sixdim_daily 表的 2026-09-10 与 2026-09-11 两条真实记录；同一表的原始总分、冲突数、数据来源字段亦一并入库。核对时间 2026-09-24。

行可以复算的记录。

4.5.3 一张表决定谁能碰什么

系统在装配团队时，逐个把工具挂到角色身上。当前版本一共 25 个工具注册入口，注册调用的代码集中在 `internal/brainhost/team_tools.go` 第 338–452 行区间⁴。

1	<code>portfolioOptimizerTool</code>	<code>marketDataTool</code>	<code>screenMarketTool</code>
2	<code>stockPoolTool</code>	<code>getStockPoolTool</code>	<code>portfolioStateTool</code>
3	<code>getPositionsTool</code>	<code>orderCostTool</code>	<code>macroSnapshotTool</code>
4	<code>portfolioRiskViewTool</code>	<code>limitCheckTool</code>	<code>portfolioRiskMetricsTool</code>
5	<code>factorReviewTool</code>	<code>searchMarketTool</code>	<code>marketStatsTool</code>
6	<code>dailySettlementTool</code>	<code>marketRegimeTool</code>	<code>mandateComplianceTool</code>
7	<code>sentimentMonitorTool</code>	<code>stopLossMonitorTool</code>	<code>decisionLogTool</code>
8	<code>sixdimTool</code>	<code>orderBookTool</code>	<code>dailyStrategyTool</code>
9	<code>backtestTool</code>		

代码 4.5 25 个工具注册入口（`internal/brainhost/team_tools.go`）

这些工具的真身分布在 `internal/tools/` 下的 22 个源码文件里，包括 `sixdim_tool.go`、`daily_strategy_tool.go`、`factor_review_tool.go`、`order_book_tool.go`、`backtest_gap_tools.go`、`cninfo_tools.go` 等⁵。

对外的工具名与可用角色，如表 4.2。这张表建议你抄一遍——它是整本书里出现频率最高的一张表。

这张表里藏着一个很容易被忽略的不对称：

`run_backtest` 只有 QUANT 能用。回测是“研究”，研究的结果必须经过风控和 CIO 才能变成动作——所以把它锁在链条的第二环上。

`market_sixdim_detect` 有四个角色能用。“今天市场好不好”是共享上下文，不是谁的私产：规划师要靠它定配置、量化要靠它排候选、风控要靠它设限额、CIO 要靠它缩仓位。

`get_cninfo_info` 是唯一覆盖四个角色（含 PLANNER）的“外部信息”工具。它查的是巨潮的公司公告——这是外部世界的事实，不属于任何岗位的私有财产。

4.5.4 每个角色手里有几把工具

看最后一行和第一行，把它们对起来读：

TRADER 独占下单工具，却没有选股工具；CIO 工具最多，却拿不到下单工具。第 chapter 二 章那条“只有 Trader 能执行订单、只有 CIO 能选择股票”的硬约束，在这里第二次出现——这一次它是用工具清单写出来的。

权限表是一道门，工具可见性是第二道门。两道门用不同的方式表达同一件事，谁改错了一道，另一道还拦得住。

⁴数据来源：该 25 个注册入口见系统事实清单第 5 节“工具清单”，核对时间 2026-09-24。

⁵数据来源：文件清单见系统事实清单第 5 节“工具源码文件”，核对时间 2026-09-24。

表 4.2 工具的角色可见性

工具名	可用角色	它回答的问题
get_tdx_stock_data	QUANT, RISK	这只股票的历史行情长什么样
search_tdx_stocks	QUANT, CIO	在几千只股票里找出符合条件的那几只
get_tdx_market_stats	CIO, RISK, QUANT, PLANNER, TRADER	全市场的基本统计数
get_tdx_common_stocks	CIO, QUANT	常见股票与 ETF 的基础资料
run_backtest	QUANT	这个策略放到历史上跑一遍，结果如何
optimize_portfolio	CIO, QUANT	这几只股票各买多少，组合波动最小
get_market_data	CIO, QUANT, RISK	市场真实数据（来自行情库）
search_market	CIO, QUANT	市场统计（来自行情库）
get_cninfo_info	PLANNER, QUANT, RISK, CIO	这家公司最近发了什么公告
market_sixdim_detect	CIO, PLANNER, RISK, QUANT	今天这个市场，适合下多少注
select_next_day_strategy	QUANT, CIO	明天用哪条策略来管持仓

optimize_portfolio 为 Markowitz 均值-方差模型，支持激进 / 保守 / 平衡三种偏好；market_sixdim_detect 的 save 参数控制是否落库；select_next_day_strategy 写入策略日计划表。

数据来源：系统源码 *internal/brainhost/team_tools.go* 与 *internal/tools/*；同表见系统事实清单第 5 节。
核对时间 2026-09-24。

表 4.3 装配完成后，每个角色实际挂载的工具数量

角色	挂载工具数	值得注意的两项
CIO	24	工具最多；但它的工具集里没有 place_trade——下单不在它手里
QUANT	17	独占回测工具与策略调参工具
RISK	16	有组合风险指标与止损监控，没有交易执行类工具
PLANNER	10	有方案与合规类工具，没有个股打分与回测
TRADER	10	独占下单工具 place_trade；工具集里没有选股工具

同一批工具被多个角色共用（例如市场统计工具 marketStatsTool 五个角色都有），因此五个数字相加远大于 25。
数据来源：系统源码 *internal/brainhost/team_tools.go* 的装配日志（第 403、438、488、538、565 行逐个列出各角色注册的工具名）；核对时间 2026-09-24。

4.5.5 运行时还有一次校验

工具在装配时按角色挂载，只解决了“它有没有”的问题。系统里还有一个工具注册表（`internal/port/agent.go` 的 `ToolRegistry`），每个工具的元数据里除了名字、分类、风险等级，还有一栏 `Roles`：这个工具可以被哪些角色调用。

中央注册表是这样建起来的：

```
1 // 中央工具注册表：用于运行时权限强制校验 (Runtime Enforcement)
2 toolReg := port.NewToolRegistry()
3 registerTeamTool := func(name, desc string, category port.ToolCategory, riskLevel string,
4     roles []port.AgentRole, tool toolpkg.Tool) {
5     if tool == nil {
6         return
7     }
8     _ = toolReg.Register(&port.ToolMetadata{
9         Name:      name,
10        Description: desc,
11        Category:   category,
12        Roles:      roles,
13        RiskLevel:  riskLevel,
14        Version:    "1.0.0",
15        Tool:       AdaptTool(tool),
16    })
17 }
```

代码 4.6 中央工具注册表与注册闭包（`internal/brainhost/team_tools.go`，第 223–238 行，原文）

每个工具在注册时，都必须当场写清它属于哪个分类、风险等级多高、允许哪些角色调用。三条最有代表性的注册声明长这样：

```
1 registerTeamTool("run_backtest", "运行策略回测，验证历史表现（仅QUANT可使用）",
2     port.CategoryBacktest, "medium", []port.AgentRole{port.RoleQuant}, backtestTool)
3 registerTeamTool("place_trade", "执行买入/卖出订单（仅TRADER可使用，且需合法DecisionObject）",
4     port.CategoryExecution, "high", []port.AgentRole{port.RoleTrader}, placeTradeTool)
5 registerTeamTool("market_sixdim_detect", "基于六维判势评估A股市场环境：...",
6     port.CategoryMarketData, "low",
7     []port.AgentRole{port.RoleCIO, port.RolePlanner, port.RoleRisk, port.RoleQuant},
8     sixdimTool)
```

代码 4.7 三条注册声明（`internal/brainhost/team_tools.go`，第 244 行、第 250 行、第 270 行，原文节选）

三行声明里，三件事同时被写死：`run_backtest` 只给 `QUANT`，风险等级 `medium`；`place_trade` 只给 `TRADER`，风险等级 `high`；`market_sixdim_detect` 给 `CIO`、`PLANNER`、`RISK`、`QUANT` 四个角色，风险等级 `low`。

注意 `place_trade` 挂的是 `high`，而六维判势挂的是 `low`。在这份注册表里，标到最高风险等级的那一个，正是唯一能下单的那一个。能力越大的工具，它的元数据越扎眼——这不是为

了好看，是为了让“谁在什么时候能调用什么”这件事变成可以逐条翻出来的记录，而不是藏在某段代码逻辑里。

注册完之后，这份注册表被逐个注入每个智能体（第 329–334 行），由智能体在运行时调用工具前再校验一次角色。装配是静态的，校验是动态的——同一条边界，在两个时刻各自确认一次。

我特意保留了这张表的“多余”：装配时已经按角色挂过工具了，为什么还要在运行时再查一遍？因为装配只发生在启动那一刻，而运行时校验发生在每一次调用。如果哪天有人加了一条新代码路径，绕过了装配直接调工具，第一道门不会拦——第二道门会。

要提醒一句：这份注册表与前面那 25 个注册入口不是同一份清单——前者按工具登记“谁能用”，后者按角色登记“我有什么”。两份清单各自列举，也各自可以被单独核对。

4.6 常见误解与纠偏

常见误解·Function Calling 是万能钥匙 “接上 Function Calling 就解决了”——这句话听起来像技术方案，其实是个情绪安慰。

工具能不能解决问题，取决于三件非常具体的事：

第一，返回值是不是结构化的。 返回一段自然语言，等于没返回。第 chapter 二 章那张权限表能成立，是因为“允许/拒绝”是两个布尔值；如果它是“原则上可以”这种文本，这张表立刻失效。

第二，取不到数据时它返回什么。 是报错，还是返回一个“默认中性 50 分”？后者看起来更“稳定”，代价是你会拿一堆假数据去做真决策。

第三，谁能调用它。 没有这一条，Function Calling 只是把模型的手变长了，没有把它关进笼子里。

常见误解·工具多就是能力强 我一度以为工具越多越好：给 CIO 挂 24 个工具，它应该更强。

事实是：工具越多，选错工具的概率越大。每个工具都有一段说明，模型要在这些说明之间做选择。工具的功能越重叠、名字越接近，它越容易挑错——而挑错工具往往不会报错，只会拿回一份“看起来相关”的数据。

所以工具数量的增长，必须伴随两件事：**每个工具有明确的“不重叠”的说明，以及每个工具有明确的可见角色。**

表 4.3 里那五个数字之所以有意义，不是因为它们大，而是因为它们**不相等**——不相等，说明有人被故意挡住了。

常见误解·模型会自己挑对的工具 工具说明不是文档，它是提示词的一部分。

模型看不到工具的内部实现，它只看到名字和一段描述。所以“这个工具被正确使用”这件事，一半的责任在写工具的人身上，而不是在模型身上。

反过来看 market_sixdim_detect 的说明：它把六个维度的名字、“输出各维度得分、冲突修正后总分、建议仓位系数 position_rate 与市场标签”、以及最要紧的那句“策略信号仓位 = 原始信号仓位 × position_rate”，全部写进了描述里。

这不是啰嗦。这是在把“该怎么用我”提前告诉调用者。工具描述写得含糊，模型就只能猜；它一猜，你就得去核对它猜的对不对——于是你又回到了这一章开头那个“没有人会发现”的处境。

4.7 真实出处、动手试一试与下一章

本章的真实出处

- 工具的执行接口与四个方法：`internal/tools/tools.go`（`Tool` 接口，第 22–27 行）
- 工具元数据与运行时校验用的注册表：`internal/port/agent.go`（`ToolMetadata`、`ToolRegistry`）与 `internal/port/port.go`（`ToolExecutor`、`ToolDefinition`）
- 代表性工具：`internal/tools/sixdim_tool.go`（`market_sixdim_detect`）、`daily_strategy_tool.go`（`select_next_day_strategy`）、`factor_review_tool.go`、`order_book_tool.go`、`cninfo_tools.go`（`get_cninfo_info`）
- 按角色装配与注册入口：`internal/brainhost/team_tools.go`（第 338–566 行）
- 中央工具注册表与各工具的角色、风险等级声明：`internal/brainhost/team_tools.go`（第 223–334 行）；元数据结构见 `internal/port/agent.go` 的 `ToolMetadata` 与 `ToolRegistry`（第 303–355 行）
- 六维判势结果的落库表：`market_sixdim_daily`（`DuckDB data/stock.duckdb`）

动手试一试（十分钟）

1. 打开系统，进“设置 → 数据源 → 市场信息数据”，找到六维判势那张卡片。
2. 不要改任何开关。只是把六个源的启用状态与优先级抄下来，然后去看“AI 团队”页里 PLANNER 的工具清单——它也有六维判势这个工具，但它没有下单工具。
3. 回到本章表 4.2，找出“只有一个角色能用”的工具。本章提到的有两个：`run_backtest`（仅 QUANT）和表 4.3 里的 `place_trade`（仅 TRADER）。
4. 自己回答一个问题：如果明天要给系统加一个“读研报”的工具，你会把它给哪几个角色？为什么不能给 TRADER？

到这里，五个角色有了权力边界，有了任务合同，也有了各自的那几把工具。可它们还是“各自在干活”——今天先做什么、多久做一次、谁在几点喊开工，还没有人安排。工具齐了，一天的时间表是谁定的？第 chapter 五 章我们跟着时钟走一整天：9:00 的盘前、9:30 到 15:00 的盘中、15:00 之后的盘后，还有 20:00 的深夜复盘。

一天是怎么过完的？——盘前、盘中、盘后、深夜

5.1 九点我在开会，十五点我在地铁上

二零二六年九月十日，星期四。

上午九点，我在会议室。投影仪上放着季度复盘，我手边一杯已经凉掉的茶。

下午三点，我在回家的地铁上，一手扶栏杆，一手拎着电脑包。

晚上八点，我在厨房洗碗，孩子在客厅写作业。

这三段时间里，我没有打开过一次行情软件。手机在口袋里，行情推送全部被我关掉了。

可就在这一天，系统至少跑了三轮完整的分析。开盘前它算过一遍，盘中每隔一段时间看一眼，收盘后把它一天的账写完，到了晚上还给第二天留了一份计划。

那天它给市场的定性是“退潮风险”。这不是我事后回想时给的形容词，是它自己写在库里的标签：修正后总分 38.43¹，对应的仓位系数 0.10²³。

我对着这一行字想了很久。

我在开会的时候，它在干什么？我在地铁上晃的时候，它凭什么觉得该看盘？我洗碗的时候，它为什么已经不看了？还有——它凭什么知道，现在这个时刻属于“盘前”而不是“盘中”？

这一章就回答一个问题：它的一天，是怎么被切开的。

5.2 先别急着讲程序，先看一张值班表

设想你住的楼下有一家社区医院。它不是急诊，所以夜里不开门，但它有门诊、有化验、有药房。

医院墙上贴着一张值班表。表上只有四行：

- 上午八点到八点半：交接班。昨天夜班把没处理完的事说清楚，护士核对药品，医生翻昨天的化验单。
- 上午八点半到十一点半：门诊。所有科室到位，病人来了就接。
- 中午十一点半到下午一点：午休。只留一个值班护士，其他人都去吃饭。
- 下午一点到两点：整理。把上午的记录归档，把明天要复查的病人列出来。

这张表最要紧的地方，不是“几点开什么科”，而是谁在岗、做什么、几点交接、哪段时间什么都不做。

¹真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

²真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

³数据来源：`market_sixdim_daily` 表，`trade_date = 2026-09-10`，只读查询于 2026-09-24。

直觉·值班表不是闹钟 闹钟只会响。值班表会告诉你“响过之后谁该干什么”。

差别在两个地方。

第一，**值班表认得午休**。闹钟到了中午十二点照样响，可医院这时候不看病——它只留一个人守着。

一个只按钟点触发的系统，会在所有人都吃饭的时候，把全套流程再跑一遍，然后抱怨没数据。

第二，**值班表认得交接**。上午的化验单必须交到下午那个人手里；不交接，下午的医生就得重新抽一次血。一个不分先后的系统，会在化验单还没出来时就去开药。

我后来意识到，我要给这个系统做的，根本不是一只闹钟，而是一张**值班表**。表上要写清楚：谁在岗、什么顺序、什么时候停工、干过的活不许再干第二遍。

5.3 如果不排这张表，会发生什么

我一开始觉得“排班”这件事有点小题大做。不就是按时间调用几个功能吗？写一个定时器，九点整触发一轮，不就行了。

后来我把不排班的代价一条一条列出来，发现有三条躲不掉。

第一条，顺序会乱，因为任务之间有真实的依赖。

盘前那串任务不是平铺的。它有一条看不见的传送带：市场扫描的结果要交给因子健康检查，因子健康检查的结果要和市场扫描一起交给选股，选股的结果再交给组合模拟。

这不是我描述出来的关系，是代码里写死的关系。后面我们会看到，每个任务创建时都会把“上家的任务编号”当成参数传进去。如果这些任务被同时触发，选股就会在一张还没算完的因子表上开工。

第二条，窗口会被错过，而开盘不等人。

盘前那段流程必须在九点半之前跑完。合规检查、隔夜风险、晨间简报，全都是“开盘之前才有效”的东西。九点半之后交付一份“今天这个组合还不合规”，等于没有交付——因为该不该买，已经在开盘那一秒决定了。

第三条，它会重复干同一件事，而且没人拦得住。

这一条我是在日志里撞见的。有一版系统，收盘之后日志还在不停地刷同一个词，三十秒一次，刷了一整个下午。查下去才发现：主循环是一个固定三十秒的定时器，而复盘阶段那三个小时里，它每三十秒都会重新判断一次“该不该复盘”，判断完发现“今天已经做过了”，然后什么都不做，但日志先把“进入复盘”写了一遍。

它没有做错事，但它做了一下午无用功，还把日志刷满了。

教训是：“什么时候该做”和“做过了就别再做”是两张不同的表。值班表只写了前者，缺了后者。

5.4 一个差不多对的模型：把一天切成四段

现在我先给你一个粗糙但方向正确的模型。

把一整天切成四段。每一段有它自己的任务，段与段之间交接，段内的任务按固定顺序跑。

这个模型很接近真相，它甚至就是系统里“任务阶段”的名字：盘前（PRE_MARKET, 9:00–9:30）、盘中（IN_MARKET, 9:30–15:00）、盘后（POST_MARKET, 15:00–16:00）、深夜复盘（NIGHT_REVIEW,

20:00–21:00)。

四段，听起来干脆。但这个模型有两处裂缝，不补上就没有办法落地。

这个近似在哪里失效

裂缝一：四段太粗，认不出午休。盘中那一段横跨 9:30 到 15:00，可中间 11:30–13:00 市场是关着门的。一个只知道“现在是盘中、该每五分钟看一次”的系统，会在午休继续看盘，然后拿到一堆不动的价格。时段必须切得比“四段”更细。

裂缝二：四段不记得自己做过什么。它只回答“现在属于哪一段”，不回答“这一段的事今天做过没有”。重启一次，它就把一天重跑一遍。阶段判定之外，还必须有幂等标记。

所以严谨版本是这样东西：一个每三十秒醒一次的状态机。它每次醒来先看“现在市场属于哪个时段”，如果时段变了，就按新时段去触发该做的事；如果没变，就检查这一段里那些“要反复做”的任务是不是又到了该做的时间；最后，凡是“一天只该做一次”的事，都要在一张纸上记下“今天做过了”。

值班表、交接单、签到本，三样东西合起来，才是完整的。

5.5 它在系统里长什么样

先看最细的那一层：市场时段。系统把它切成了八个常量，其中七个是真正常用的时段（第八个是“开盘前的准备”）。

```
1  const (
2      PhasePreMarket      MarketPhase = "PRE_MARKET"      // 盘前准备 开盘前-09:15
3      PhasePreOpen        MarketPhase = "PRE_OPEN"        // 盘前 09:15-09:30
4      PhaseMorningSession MarketPhase = "MORNING_SESSION"  // 上午交易 09:30-11:30
5      PhaseLunch           MarketPhase = "LUNCH"           // 午休 11:30-13:00
6      PhaseAfternoonSession MarketPhase = "AFTERNOON_SESSION" // 下午交易 13:00-15:00
7      PhasePostMarket      MarketPhase = "POST_MARKET"     // 盘后 15:00-15:15
8      PhaseReview          MarketPhase = "REVIEW"          // 复盘 15:15-18:00
9      PhaseClosed          MarketPhase = "CLOSED"          // 休市
10 )
```

代码 5.1 市场时段常量（节选，internal/util/trading_hours.go）

这段话在说什么：这个函数每天被调用无数次，它只回答一个问题——此刻市场处于哪一段。它先判断今天是不是交易日，不是就直接返回“休市”；是交易日，才拿当前分钟数去和上面这些窗口比。表 5.1 是这七段的完整对照。

注意到午休了吗？它单独占了一档。这就是刚才那条裂缝的补丁——系统不靠“盘中”这两个字判断该不该看盘，它靠的是分钟数落在哪个窗口里。

再看那一层“四段任务阶段”和它上面的调度循环。

```
1  func (s *AutoScheduler) schedulingLoop() {
2      ticker := time.NewTicker(30 * time.Second)
3      defer ticker.Stop()
4  }
```


表 5.1 七段市场时段与它们的时间窗

时段	枚举	时间	系统该有的动作
盘前竞价	PRE_OPEN	09:15–09:30	前一天定好的盘前流程必须已经跑完
上午交易	MORNING_SESSION	09:30–11:30	盘中监控、可交易池处理、决策补跑
午休	LUNCH	11:30–13:00	停手，不刷新
下午交易	AFTERNOON_SESSION	13:00–15:00	同上午交易
盘后	POST_MARKET	15:00–15:15	收盘价入库、盘口摘要落库
复盘	REVIEW	15:15–18:00	结算、因子复盘、明日计划
休市	CLOSED	其余时间	不交易，也不产生当日决策

另有 PRE_MARKET（开盘前–09:15）一档，用于“开盘前的准备”，不计入交易时段。
数据来源：系统源码 `internal/util/trading_hours.go`；核对时间 2026-09-24。

```
5     s.checkAndTrigger()
6     // 启动较晚（已进入盘中时段）时补偿执行错过的盘前任务
7     s.compensateMissedPreMarket()
8
9     for {
10         select {
11             case <-s.ctx.Done():
12                 return
13             case <-ticker.C:
14                 s.checkAndTrigger()
15         }
16     }
17 }
```

代码 5.2 主调度循环（节选，`internal/scheduler/auto_scheduler.go`）

这段话在说什么：系统每三十秒醒一次。醒来的第一件事不是干活，是看时段有没有变——变了，就按新时段派活；没变，就去检查那些“要反复做”的任务。

三十秒这个数字值得停一下。它不是任务执行的间隔，它是看表的间隔。看表要勤，干活要稳：看表慢了会错过九点半这个交接点，干活太勤会把自己烧穿。所以真正干活的间隔，是另外一组数字。

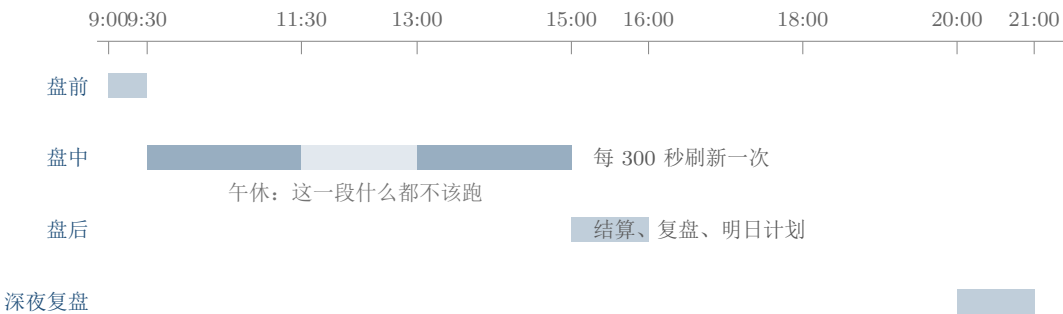


图 5.1 一天的四幕。横轴是时间，四行是四个任务阶段。盘中那一段里颜色变浅的两小时是午休——按钟点排班最容易漏掉的，就是它。

图 5.1 就是我说的那张值班表。它一眼就能看出两件事：第一，四段长短极不均匀，盘前只有半小时，盘中五个半小时；第二，盘中那一段不是连着的，中间被午休挖掉一块。

真正干活的间隔写在任务表里。盘中那三个监控任务——操盘手的“市场监控”、风控师的“风险监控”、量化分析师的“信号监控”——每一个都带一个 RepeatInterval: 300 的字段，单位是秒⁴。三百秒，五分钟。

现在回到四个任务阶段里最忙的那一段：盘前。它是全书所有故事的开头，因为“今天买什么”这件事，是在这半小时里被决定的。

盘前一共七个任务，执行顺序固定，而且每个任务在创建时都会把上家的任务编号带进来。

表 5.2 盘前阶段的七个任务（真实执行顺序）

序	角色	任务类型	它在干什么
1	Planner	CHECK_MANDATE	检查当前组合是否还符合投资委托书
2	Quant	MARKET_SCAN	隔夜市场扫描（海外、指数、消息）
3	Quant	FACTOR_HEALTH	因子健康度分析（价值、动量、质量……）
4	Quant	STOCK_SCREENING	全市场选股（依赖 2、3 的产出）
5	Quant	PORTFOLIO_SIMULATION	基于候选股票做组合模拟
6	Risk	OVERNIGHT_RISK	隔夜风险、跳空风险评估
7	CIO	MORNING_BRIEF	汇总前六份报告，形成晨间简报

执行完成后系统写下一行日志断言：“[WorkflowEngine] 盘前阶段完成，执行了 7 个任务”。
数据来源：系统源码 internal/agentworkflow/workflow_engine.go，函数 ExecutePreMarketPhase；核对时间 2026-09-24。

这张表里最值得看的是第四行的括号。选股之所以排在第三之后，不是谁规定的，是它真的需要前面两份产出。代码里就是这么写的：

```
1 // Step 1: Planner 检查投资政策
2 mandateCheckTask, err := we.taskMgr.CreateTask(
3     AgentPlanner, TaskTypeCheckMandate,
4     "检查投资政策合规性", "检查当前组合是否符合Investment Mandate要求",
5     workflowID, "HIGH", phase, nil, nil, "", // 无前置任务
6 )
7
8 // Step 4: Quant 选股——依赖 Step2 的市场扫描与 Step3 的因子健康检查
9 screenTask, err := we.taskMgr.CreateTask(
10     AgentQuant, TaskTypeStockScreening, "全市场选股", "根据因子表现进行全市场选股",
11     workflowID, "HIGH", phase,
12     []string{marketScanTask.TaskID, factorTask.TaskID}, nil, "", // 前置任务编号
13 )
14
15 // Step 7: CIO 晨间简报——把前六个任务全部收进来
16 cioBriefTask, err := we.taskMgr.CreateTask(
17     AgentCIO, TaskTypeMorningBrief, "CIO晨间简报", "汇总各智能体报告，形成晨间简报",
18     workflowID, "CRITICAL", phase,
19     []string{mandateCheckTask.TaskID, marketScanTask.TaskID, factorTask.TaskID,
20         screenTask.TaskID, simTask.TaskID, riskTask.TaskID},
21     nil, "",
22 )
```

⁴数据来源：系统源码 internal/scheduler/task_scheduler.go，三个盘中监控任务的 RepeatInterval 均为 300；核对时间 2026-09-24。

代码 5.3 盘前任务的顺序与依赖（节选，`internal/agentworkflow/workflow_engine.go`）

这段话在说什么：注意倒数第二个参数。选股那一行里，传进去的是“市场扫描的任务编号”和“因子健康检查的任务编号”；CIO 那一行里，传进去的是前面六个任务的编号。这就是“交接”在代码里的样子——不是一句“请注意上下游关系”的注释，而是一个必须被填满的数组。

还有一件小事，看起来不像设计，其实很重要：任务有五种状态——SUCCESS、RUNNING、FAILED、PENDING、SKIPPED⁵。

为什么“状态”这件事值得单列出来？因为一张值班表如果只写“几点做什么”，它就没法回答“这一步做完没有”。有了状态，调度器才敢做两件事：跳过已经成功的、重试失败的。这也是后面第 chapter 九 章要讲的容错的基础。

最后是那份“签到本”。系统把两个每天只该做一次的标记——`last_daily_review_date` 和 `last_plan_refresh_date`——写进配置文件 `config/config.json`，而且要求它在重启后还能读回来⁶。

我一开始以为把它放在内存里就够了。后来发现完全错了：这个系统的常态是“你晚上关机，早上开机”。标记只存在内存里，就等于每天早上都失忆一次——昨天已经做过的复盘，今天开盘前会再做一遍。一份不落盘的签到本，等于没有签到本。

常见误解·自动化就是实时 这是我最初最根深蒂固的误解。我以为“接了 AI、挂了定时器”就等于“实时盯盘”，越快越好。

事实正相反：这个系统的节拍器从来不是模型，是 A 股的时钟。模型再快，也得等收盘价出来；再聪明，也不能在午休时算出一个新价格。你能做的所有事，都被交易所那几张时间表卡着：9:15 集合竞价、9:30 开盘、11:30 上午收盘、13:00 下午开盘、15:00 收盘。

所以在这套系统里，“实时”这个词是有成本的，而且它的价值随着时段剧烈变化。开盘前后那半小时，一分钟的延迟都可能错过一个决策窗口；午休那一个半小时，多算一次纯属浪费；晚上八点，它算的不是“现在”，是“明天”。

一旦接受了“A 股的节奏才是节拍器”这件事，你会对系统的“慢”变得宽容，也会对它的“快”变得警惕——一个在深夜还要高频刷新的系统，不是在更努力，是在空转。

常见误解·“每五分钟”是随手定的 盘中那三个监控任务，间隔都是三百秒。我看到这个数字时的第一个反应是：随便挑的吧，取个整而已。

其实这是一个两头夹出来的折中。

往快了做，成本立刻上去。每一次刷新都不是免费的空转——盘中监控要拉实时行情、要读组合状态、有些环节还会调用语言模型。三百秒意味着一个交易日的上午加下午（四个小时）里，每个监控任务最多被调用四十次上下。如果把间隔改成三十秒，调用次数直接乘十，而多出来的那些次里，绝大多数看到的还是同一个价格。

往慢了做，时效性就没了。五分钟一次，意味着一个破位的信号最长要过五分钟才被看见。对一个

⁵数据来源：系统源码 `internal/scheduler/task_scheduler.go`，统一状态定义；核对时间 2026-09-24。

⁶数据来源：系统源码与事实清单：幂等标记持久化到 `config/config.json`，启动恢复、重启不重跑；核对时间 2026-09-24。

只做日线级别的系统来说，这个延迟可以接受；但如果你的策略是分钟级的，五分钟就是灾难。所以三百秒不是随手取的，它是“再快就亏钱、再慢就误事”之间那条缝。真正该记住的不是三百这个数，而是这句话：刷新频率是一个可以被你自己改的参数，而且它有价格。改它之前，先问清楚你多买的那些分钟，到底买到了什么。

本章的真实出处

- 盘前/决策/风控/盘中/盘后五个阶段函数：`internal/agentworkflow/workflow_engine.go`，其中 `ExecutePreMarketPhase` 执行七个任务并写下“执行了 7 个任务”的断言日志。
- 市场时段的定义与判定：`internal/util/trading_hours.go` (`GetMarketPhase`、`IsTradingDay`、`PhaseLabels`)。
- 三十秒调度循环、时段转换触发与盘中补跑：`internal/scheduler/auto_scheduler.go`。
- 任务阶段、五种状态、`RepeatInterval`：`internal/scheduler/task_scheduler.go`。
- 幂等标记 (`last_daily_review_date` / `last_plan_refresh_date`) 的持久化：`config/config.json`。
- 前端“实时活动”页（流水线进度、活跃任务、按角色筛选的日志）：`ui/src/pages/` 下的活动页面，路由表在 `ui/src/App.tsx`。

动手试一试（十分钟，挑在交易日的盘后）

1. 打开系统，进入左侧菜单的实时活动页。先看那条流水线：PLANNER → QUANT → RISK → CIO → TRADER。
2. 找到今天的活动日志，搜索“盘前阶段完成”这一行，数一数它前面有几个任务条目。它应该正好是七个。
3. 看每一行的时间戳。把盘前那七行的时间和表 5.2 的顺序对一遍——它们应该严格按 1 到 7 排。
4. 现在看页面上方的“市场时段”指示。对照表 5.1 确认它现在报的是哪一段。
5. 最后一问：如果现在是午休，页面上还有任务在跑吗？如果没有，说明那张值班表认出午休了。

值班表排好了，交接单填明白了，签到本也落盘了。可盘前那七个任务里，最后一个——CIO 的晨间简报——是要“拍板”的。

那么问题来了：它凭什么拍板？它手里只有前面六个人交上来的纸。接下来一章，我们去看它怎么把这几张纸，变成一场自己跟自己的辩论。

第六章

让 AI 自己吵一架：多空辩论

6.1 两个 AI，你信哪个

先讲一个小实验，是我自己做的。

我把同一份市场证据，给了决策官两次。

第一次，我只问它：“今天该怎么做？”它回得很干脆：方向看多，建议仓位 0.8，理由三条，条条通顺。

第二次，我把一模一样的证据给它，只在末尾加了一句：“请找出你这个判断可能错在哪里。”

它立刻换了口气。它列出了三条风险，其中一条是流动性——而这恰好是我自己看着那份证据时最担心的那一处。

同一份证据，同一个模型，两个结论。

我盯着这两次输出想了很久。问题不是它前后不一致。问题是：如果我只问了第一次，我拿到的就是一个看起来很确定的答案，而它自己明明知道存在第二条路。

后来我明白了一件事。一个只给我结论的 AI，和一个能自己找出反证的 AI，我信后者。不是因为后者更聪明，而是因为前者把不确定性藏起来了，后者把它摆在了桌面上。

直觉·法庭上的交叉询问 想象一场法庭审理。

检察官陈述完毕，他说得很完整——人证、物证、时间线，全都对得上。如果你只听他这一遍，你几乎会立刻点头。

可法庭不会就此结束。接下来是辩护律师的交叉询问。他不重复检察官的话，他专门问那些检察官没提的地方：那天晚上你的当事人到底在哪？这根绳子为什么只验了一次？

这不代表检察官一定错。代表的是：在“有罪”这个结论被采纳之前，必须有人站在对面，认真地找过它的漏洞。

交叉询问的价值，不在于它总能推翻什么。它的价值在于：它让一个结论必须在被反驳之后还站得住，才算数。

一份没人反驳过的判断，和一份被认真反驳过、还活下来的判断，分量完全不同。哪怕它们最后写着同一个词。

6.2 让同一个模型“再想一遍”，为什么没有用

我第一个想到的办法很自然：让它自查。

让它在给出结论之后，再看一遍自己写的东西，问一句“你确定吗”。便宜、简单，一次调用就搞定。

我试了。结果是：它百分之九十的情况下会告诉我“经过重新审视，我依然认为……”。

不是它嘴硬。是这件事从原理上就办不到。

在第 chapter 一章我讲过那个“一个人的公司”的问题，这里的道理是同一个，但更尖锐：让同一个模型在同一次对话里自我怀疑，等于让它在同一段计算里，同时优化“给出结论”和“推翻结论”两个目标。这两个目标是打架的，而语言模型的本能是输出连贯的文本。“我刚才错了”这句话，是一段连贯文本里最不连贯的东西。

所以它会替你圆场。它会用更细致的措辞，把同一个结论再说一遍，让你觉得它考虑过了。

两种“再想一遍”的区别

自查：同一个角色、同一段上下文、同一个目标函数，只是多了一轮。结果是“结论的修辞升级版”。

对抗：换一个角色、换一套系统提示词、给它一个不同的考核指标——它的任务不是“得出更好的结论”，而是“找出这个结论的漏洞”。

前者不产生新信息，后者才产生。区别不在问几遍，在换没换视角。

我们不需要一个更有自省能力的模型，我们需要第二个角色，而且它的立场必须是怀疑。

这个角色不能太温和。你如果告诉它“请客观评估这个结论”，它就会变成一个和事佬。你要明确地告诉它：你的职责是故意挑刺。

6.3 一个差不多对的模型：吵一架

粗糙的模型是这样的：让主张官先说一个方向，再让一个专门挑刺的角色上场，把主张官的说法按在地上摩擦一遍。

这个模型抓住了核心：反对意见必须由另一个独立的东西产生。但它太粗，漏掉了三件事。

漏掉的第一件事：吵之前得先把证据摆出来。如果双方吵的是空口白话，那这场辩论就只是两种措辞在互相覆盖。真正的辩论要有共同的证据底稿——今天的市场什么样、组合里有什么、风控怎么说、候选股票是哪几只。这些东西必须先被采集成一份清单，双方都在这份清单上发言。

漏掉的第二件事：挑刺的角色只能往保守的方向走。这一点至关重要，我们在第 chapter 七章会用一整个定理来讲它。这里先给出结论：在这个系统里，任何角色的意见只会让仓位变小，不会让它变大。挑刺的人就算想说“你应该更激进”，也递不出去。

漏掉的第三件事：争论到最后，必须留下一个能算的数。光有“看多派说了什么、看空派说了什么”是没用的。要有一个数字，告诉决策层“这场争论到底吵得有多凶”。这个数字叫共识度。

补上这三件事，我们得到一个五段的流程。

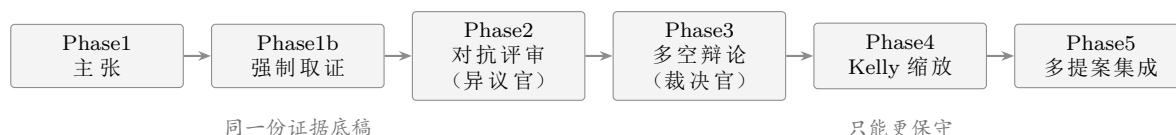


图 6.1 决策主张层的五个阶段。前两段是“把话和证据说清楚”，中两段是“吵并且折算”，最后一段是“多方对表”。

哪一段是真吵架？Phase2 和 Phase3 都是，但它们吵的方式不一样。

Phase2 是对抗评审。一个“异议审查官”上场，它的任务是从集中度、情绪过热、流动性、追高、利好出尽这些角度，找出主张官可能冒进的地方，然后给出一个它自己认为更保守的仓位建议，以及一个定性表态：可以认可、需要留意、还是明显有风险。

Phase3 是多空辩论。一个“辩论裁决官”上场。注意它的身份——它不是站在空头那边跟主张官对骂，它是主持人。它要同时替多头和空头各说几句话：站在多头角度，列出最有力的看多论据；站在空头角度，列出最有力的看空论据。然后它给出一个方向，和一个共识度。

为什么要一个“主持人”，而不是直接让多空两个角色对骂？因为对骂会产生两个结论，而决策需要一份材料。系统的目标不是收集两种情绪，是让决策者在同一张纸上同时看到两边的理由，并且知道这张纸有多脆。

6.4 它在系统里长什么样

入口函数叫 `decideProposalGate`，在 `internal/cio/proposal.go`。它做的第一件事不是问主张，而是取证。

```
1 func (c *CIOEngine) decideProposalGate(ctx context.Context, in proposalInput, ruleRate
    float64) *proposalGateResult {
2     evidence := buildEvidenceLines(in)           // Phase1b 强制取证：先摆证据
3     res := &proposalGateResult{Evidence: evidence, Effective: ruleRate, KellyScale: 1.0}
4
5     prop := c.genDecisionProposal(ctx, in)       // Phase1 主张
6     if prop == nil {
7         return res                               // 不可用 → 完全回退规则
8     }
9     // 规则风控门：主张只能更保守
10    gate := math.Min(ruleRate, clampF(prop.TargetPositionRate, 0.2, 1.0))
11    ...
12 }
```

代码 6.1 决策主张层入口（节选，`internal/cio/proposal.go`）

这段话在说什么：注意第一行和第四行。`buildEvidenceLines` 把日期、六维判势、组合快照、回撤、风控摘要、因子候选、历史反馈整理成一份清单。这份清单先于任何判断被准备好，并且会随着决策一起落库。也就是说，无论后面怎么吵，“当时看到的是什么”是被固定下来的。

然后是主张官。它的系统提示词里写死了输出的形状——一个 JSON 对象，五个字段：

```
1 {"market_direction": "BULLISH|BEARISH|NEUTRAL",
2  "market_read": "<一句话解读>",
3  "target_position_rate": 0~1,
4  "preferred_stocks": [{"code": "600519", "reason": "<理由>"}],
5  "confidence": 0~100,
6  "conclusion": "<不超过120字总结>"}
```

代码 6.2 主张官输出的字段（节选，`internal/cio/proposal.go`）

这段话在说什么：主观看多还是看空（`market_direction`）、建议用多少仓位（`target_position_rate`）、看好哪几只（`preferred_stocks`）、以及它对自己有多确信（`confidence`）。

那个 `confidence` 字段是本章的伏笔。请记住它——下一章那个把分歧折算成数字的公式，一半的原料就是它。

辩论那一段是这样的：

```
1 // 辩论裁决官输出的结构
2 type debateVerdict struct {
3     Direction string `json:"direction"` // BULLISH/BEARISH/NEUTRAL
4     Bull      []string `json:"bull"` // 看多论据
5     Bear      []string `json:"bear"` // 看空论据
6     Agreement int      `json:"agreement"` // 0~100 共识度
7     Summary   string   `json:"summary"`
8 }
9
10 // 解析失败就返回 nil —— 不影响门控
11 func (c *CIOEngine) parseDebateVerdict(content string) *debateVerdict {
12     block := extractJSONBlock(content)
13     if block == "" { return nil }
14     var deb debateVerdict
15     if err := json.Unmarshal([]byte(block), &deb); err != nil { return nil }
16     if deb.Direction == "" || len(deb.Bull) == 0 && len(deb.Bear) == 0 { return nil }
17     if deb.Agreement < 0 || deb.Agreement > 100 { deb.Agreement = 0 }
18     return &deb
19 }
```

代码 6.3 多空辩论与解析（节选，`internal/cio/proposal.go`）

这段话在说什么：三件事。

第一，输出是结构化的：一个方向、一组看多论据、一组看空论据、一个 0 到 100 的共识度。它不允许只交一份情绪。

第二，解析是苛刻的。方向为空、多空论据一条都没有、JSON 不合法——任何一种，直接返回 `nil`。

第三，`nil` 不是“没吵架”，是“这场辩论不算数”。它会被丢掉，门控流程照常往下走，只是少了一路意见。

这就是本章真正想讲的设计。系统宁可承认“这一路意见我没拿到”，也不肯硬塞一个结论进去。

常见误解·辩论是为了选出更准的方向，从而提高收益 这大概是最容易被想歪的一点。看到“多空辩论”四个字，本能反应是：这是在凑智慧，想让判断更准，从而赚得更多。

如果你带着这个预期去读代码，你会看不懂——因为这个系统里，辩论的每一步都是往下调的。

主张官说 0.8，异议官说 0.6，于是取 0.6；辩论说共识度不高，再乘一个小于 1 的系数；集成投票发现整体偏空，再把仓位压到 0.75 以下。整条链上，没有任何一步是“把仓位抬高一点”。

所以辩论的真实目标不是提高收益，而是更保守。它要解决的问题是“我凭什么相信一个单一来源

的判断”，它给出的答案是“我不相信，所以我让别的视角来削弱它”。

把这件事说透：一个会降低收益期望、但会降低错误集中度的机制，在投资里往往是划算的。因为你最怕的不是“赚得少一点”，是“在一件自己很确信的事情上亏掉一大块”。辩论买的不是收益，是保险。

常见误解·“辩论失败了就随便给个默认方向顶上” 既然是“多空辩论”，有人自然会想：那万一模型抽风、输出解析不了，是不是就默认按“中性”处理，把流程走完算了？

在别的系统里，这可能是合理的兜底。在这套系统里，它是一条红线。

看上面 `parseDebateVerdict` 的写法：任何一个环节不合法，返回的是 `nil`。而调用方 `debateProposal` 拿到 `nil` 之后，什么都不做——不生成默认方向，不填占位结论，不改变已有的仓位门控。

为什么这么“浪费”？因为一个编出来的“中性”，和一个真实的“中性”，在下游看起来一模一样，但在审计时天差地别。第 chapter 10 章会讲，这套系统的核心承诺之一，是决策发生的那一刻就把证据固化下来。如果此刻塞进去的是一个系统自己造的假结论，那整条证据链就废了。

同样的道理也写在主张官那边：LLM 没配置、调用失败、超时、输出解析不了，`decideProposalGate` 直接返回 `Effective = ruleRate`，也就是完全退回规则决策，AI 的意见一个字都不采纳。这不是“降级使用假数据”，这是“没有意见就不发表意见”。

一句话记住：在这套系统里，缺席是合法的，编造不是。

常见误解·让模型“再检查一遍”就是对抗评审 回到本章开头那个自查实验。我把“请再检查一遍”和“请找出你可能错在哪”当成一回事，实际上它们是两件不同的事。

真正的对抗评审，在系统里是另一个角色、另一套系统提示词、另一次独立的调用。异议审查官有它自己的职责说明，里面明确写着“故意挑刺”，还明确写了它的产出必须是一个更保守的建议仓位。它和主张官不共享上下文，也不共享立场。它甚至不需要同意主张官对市场的判断——它只需要回答一个问题：这个主张如果错了，会错在哪？

自查办不到这一点。自查是同一支笔写完之后，换一种说法再写一遍。对抗评审是换一个人，拿着红笔，在纸上圈出错的地方。区别不在于问了几遍，在于有没有第二个人。

那么共识度到底是什么？它不是“谁赢了”，也不是“哪边论据多”。

我用系统里真实的一天来说明这件事。

表 6.1 系统真实记录：同一天里，两个维度自己就吵起来了

交易日	技术趋势维度	市场广度维度	系统记录到的冲突数
2026-09-02	100 ¹	18 ²	2 ³

技术趋势维度看“指数有没有站上均线”，市场广度维度看“上涨家数占比”。100 与 18 是同一天、同一份行情算出来的两个数。

数据来源：`market_sixdim_daily` 表，`trade_date = 2026-09-02`；只读查询于 2026-09-24。

那一天，技术趋势维度给了满分 100——指数形态很好。可市场广度维度只给了 18——真正在涨的股票少得可怜。这两个数字没有一个是错的，它们只是互相不同意。

共识度想表达的，正是这种东西。它不告诉你“该看多还是看空”，它告诉你：你手上这些理由，彼此之间有多一致。

举个带数的例子来看这件事的另一面（这里的主观看多/看空表态是示意，因为系统库里没

有持久化当日辩论记录；公式和取值范围则是代码里的真实定义）：

- 情况 A：多头说“资金在流向同一条主线”，空头说“这条主线的估值已经不便宜了”。两句话其实在说同一件事的两面。共识度可以很高，比如 80。
- 情况 B：多头说“放量突破”，空头说“放量出货”。这是两种互相排斥的读法——同一根阳线，一个人看成进攻，一个人看成撤退。共识度只能很低，比如 25。

看出区别了吗？在情况 A 里，方向是看多还是看空不重要，因为双方对“发生了什么”没有分歧，只是对“这意味着什么”有分歧。在情况 B 里，双方连“发生了什么”都没谈拢——这种时候，任何方向都建立在一层薄冰上。

共识度衡量的不是结论的强弱，是结论底下那块冰的厚度。

本章的真实出处

- 五阶段门控入口与规则风控门： `internal/cio/proposal.go`，函数 `decideProposalGate`。
- 强制取证（证据链文本）：同文件， `buildEvidenceLines` / `buildProposalPrompt`。
- 主张官与它的系统提示词：同文件， `proposalSystemPrompt` / `genDecisionProposal` / `parseDecisionProposal`。
- 异议审查官（Phase2 对抗评审）：同文件， `skepticSystemPrompt` / `challengeProposal` / `parseSkepticVerdict`。
- 辩论裁决官（Phase3 多空辩论）：同文件， `debateSystemPrompt` / `debateProposal` / `parseDebateVerdict`。
- 各阶段均为独立的一次 LLM 调用，超时 60 秒；失败/解析失败返回 `nil`，不改动门控。
- 真实分歧样本： `market_sixdim_daily` 表。
- 前端展示：CIO 页的【多空辩论】区块。

动手试一试（十五分钟）

1. 打开系统，进入首席投资官（CIO）页，找到【多空辩论】那块卡片。
2. 看它有没有方向、共识度、以及多空两组论据。如果今天还没有决策，就先看决策日志里最近一条。
3. 把多空两组论据各读一遍。问自己一个问题：这两组论据，是在争论“发生了什么”，还是只在争论“这意味着什么”？
4. 如果它们只在争论“这意味着什么”，那今天的方向其实是稳的；如果在争论“发生了什么”，那今天这块冰是薄的，仓位理应更小。
5. 最后一问：如果这张卡片是空的，你敢不敢让它照常下单？这个问题的答案，就是本章全部的分量。

吵完了。多空两边都说了话，共识度也给出来了。

可是你注意一下，我们手上现在有什么：一个主张仓位、一个异议仓位、一个方向、一个共识度。四个词，没有一个是钱。

所以下一个问题必然要问：这些吵架的结果，谁来把它折算成一个数字，而且折算的时候不许偷偷加码？

第七章

意见不一致时，谁来拍板？——Kelly 缩放与“只降不升”

7.1 信心六十分，共识七十分，这算看好吗

上一章结束时，我们手上攒了一堆形容词：看多、看空、中性、可以认可、需要留意、明显有风险。

这一章要干的事很土：把这些话，变成一个能乘进订单里的数。

先看一个具体的场面。假设决策日志里有这样一行（这一行的措辞是示意，数字的含义下面马上讲清楚）：

主张：看多 | 信心 60 辩论：看多 | 共识 70 Kelly 仓位 $\times 0.65$

我第一次看到这行字的时候，脑子里的反应是：“两个都看多，那不就是看好嘛。”

然后我停住了。信心只有 60 分，共识只有 70 分。这真的算“看好”吗？

如果算，那它跟“信心 95、共识 90”有什么分别？如果不算，那它到底算什么？系统又凭什么用一个 0.65 去表达这种“算也不算”？

更让我不安的是另一件事。这个数是从哪冒出来的？谁给它这个权力，让它去乘我的仓位？如果某一天模型心情好，给了“信心 100、共识 100”，是不是仓位就该被放大到 1.0？

那天我翻到库里的真实记录，看到 2026-09-11 那一天的仓位系数是 0.06¹²。百分之一都不到。

我就想：在这一天，如果 AI 兴高采烈地说“信心 100、共识 100”，它能把这个仓位抬起来吗？

这个问题，就是本章的全部。答案是**不能**——而且这件事是可以`用式子证明的`。

7.2 先看一场预算审批

在我搞懂这个公式之前，我先搞懂了另一个场景：报销。

假设你在一家公司管报销。一个部门要办活动，报上来一个预算：三万。

这一份申请会经过好几个人。先是部门主管签字，然后是财务核预算余额，再是分管领导看这笔钱合不合规，最后是大老板看要不要批。

现在注意一个细节：这几个人里，没有一个人能在流程里给自己加钱。

主管可以把三万砍成两万五。财务可以说“这个季度超支了，最多一万五”。分管领导可以说“这个科目不批”。大老板可以说“再降一点”。

但没有任何一个人，能在别人的意见之上写一句“我觉得可以，加到四万”。

¹真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

²数据来源：`market_sixdim_daily` 表，`trade_date = 2026-09-11`；只读查询于 2026-09-24。

直觉·审批链是一条单向阀 审批链的形状，不是一个平均值，也不是一次投票。

平均值长这样：三个人说三万、两万、一万，最后取两万。可实际上，财务说一万五的时候，那不是“一个建议”，那是一道**上限**。

投票长这样：多数说三万，那就三万。可实际上，只要有一个部门说“这个不批”，这事就是办不成——不需要多数同意他。

审批链真正的形状是：**每一道签字，都是在上一道的基础上，把数字往下压，或者不动**。最后批下来的数，是所有签字的**最小值**。

这个形状有一个漂亮的后果：最终批下来的钱，一定不超过最初申请的那个数，也一定不超过任何一道签字允许的数。也就是说，你不需要去管前面的签字有多随意，只要它们的方向都是“只能往下”，这个系统的上限就是锁死的。

这套系统里，AI 的意见扮演的是“签字”的角色，不是“投票”的角色。

7.3 不这么设计，会怎样

我先试着想了两种“更自然”的设计，结果两种都会出事。

第一种：取平均。

主张官说仓位 0.8，异议审查官说 0.5，那就取 0.65。看起来公平、温和、考虑了双方。

问题在于，平均是**双向的**。如果主张官今天特别乐观，说 1.0，而异议官说 0.4，平均出来是 0.7——比异议官想给的还多。也就是说，只要有一个角色敢报大数，它就能通过“平均”把别人的保守意见稀释掉。

异议官的存在意义，本来是“拦住冒进”。可平均法让它的作用变成了“提供一个可以被拉平的数字”。这等于把刹车装在了油门上。

第二种：谁有信心听谁的。

主张官信心 90，异议官信心 40，那就听主张官的。

这更糟。因为它**奖励自信**。而语言模型最容易犯的毛病，恰恰是“用很确定的语气说一件没把握的事”。你在第 chapter 六 章已经见过一次了：同一个模型，同一份证据，只在末尾加一句“请找出你的错处”，它立刻换了口气。

也就是说，在这套系统里，**自信程度本身不是一个可以拿来加权的可信度指标**。它可以用来“减少下注”，但绝不能用来“增加下注”。

两次试错之后，我要的东西清楚了：

我需要一个数

我要一个系数 κ ，它同时满足三条：

一、有下界。它不能被压到零。因为信心和共识度是可以因为“这两个数据没拿到”而变成 0 的，如果把那种情况和“确定度极低”混为一谈，一次数据缺失就会把仓位清零。

二、有上界，且上界不超过 1。它可以降低仓位，永远不能提高仓位。

三、单调。信心或共识度越高，这个系数只能越大或持平，绝不能出现“信心从 60 涨到 80，系数反而变小”这种荒唐事。

一个单调、有界、只在 $[0.4, 1]$ 里走的函数，就够了。

7.4 一个差不多对的模型，和一个严谨的版本

近似模型非常朴素：确定度 = 信心 × 共识度，确定度越高，仓位越接近满仓。

这个模型的直觉是对的。信心是“我有多确信我的判断”，共识度是“我的理由之间有多一致”（上一章讲过，它是那块冰的厚度）。两件事都强，才叫真的确定。

但它有两处会出事，必须补。

第一处：纯乘积没有下界。如果共识度是 1（几乎完全分歧），乘积就贴到 0，仓位跟着归零。可这是一种**过度反应**——“吵得厉害”意味着该小注，不意味着该空仓。空仓是另一个更重的决定，不应该被一个口径模糊的数字顺手做掉。

第二处：纯乘积没有上界保护。“确定度 1 就该满仓”这句话听起来没问题，但它把“这一路意见”和“规则层给的仓位基准”混成了一件事。第 chapter 二十一 章会讲清楚：判断“今天该下多少注”是市场判势的职责，AI 的责任只是“在它之上，帮忙踩一脚刹车”。所以这个系数的作用域必须被限制在 1 以下。

补完这两处，严谨版本长这样。

定义 7.1 (确定度与 Kelly 缩放系数). 设归一化置信度 $c \in [0, 1]$ 与归一化共识度 $a \in [0, 1]$ 。其中 c 由主张官字段 `confidence` 除以 100 得到， a 由辩论裁决官字段 `agreement` 除以 100 得到。称

$$\text{certainty}(c, a) = c \cdot a \quad (7.1)$$

为确定度，并定义 **Kelly 仓位缩放系数**

$$\kappa(c, a) = \min(1, 0.4 + 0.6ca). \quad (7.2)$$

式 (7.2) 在讲什么：括号里那一坨 $(0.4 + 0.6ca)$ 是一个从 0.4 走到 1.0 的滑杆。两边都满格 ($c = a = 1$)，滑杆推到底，得 1.0，也就是“不缩放”；确定度归零，滑杆落到底座 0.4，也就是“最多留下四成”。外面那个 $\min(1, \cdot)$ 是一道围栏，防止它越过 1。

现在把三条要求逐条证明。

定理 7.2 (κ 的有界性). 对任意 $c, a \in [0, 1]$ ，式 (7.2) 定义的 $\kappa(c, a)$ 满足

$$0.4 \leq \kappa(c, a) \leq 1. \quad (7.3)$$

这个定理在说什么：不管信心和共识度给成什么样，这个系数永远落在 $[0.4, 1]$ 这一段里——既不会把仓位砍到零，也永远不会超过 1。

证明. 分四步。

第一步：先定住乘积 ca 。由 $c \in [0, 1]$ 、 $a \in [0, 1]$ ，两个数都非负，故 $ca \geq 0$ ；又 $ca \leq 1 \cdot 1 = 1$ 。于是

$$ca \in [0, 1]. \quad (7.4)$$

第二步：定住内层表达式。把式 (7.4) 乘以 0.6，得 $0.6ca \in [0, 0.6]$ ；再加 0.4，得

$$g(c, a) := 0.4 + 0.6ca \in [0.4, 1.0]. \quad (7.5)$$

第三步：取下界。 $\min(1, \cdot)$ 关于它的第二个自变量是单调不减的：若 $u \leq v$ ，则 $\min(1, u) \leq \min(1, v)$ 。由式 (7.5) 知 $g(c, a) \geq 0.4$ ，于是

$$\kappa(c, a) = \min(1, g(c, a)) \geq \min(1, 0.4) = 0.4. \quad (7.6)$$

第四步：取上界。由定义， $\kappa(c, a) = \min(1, g(c, a))$ 是 1 与另一个数的较小者，故恒有

$$\kappa(c, a) \leq 1. \quad (7.7)$$

综合第三、四步，得 $0.4 \leq \kappa(c, a) \leq 1$ 。 ■

定理 7.3 (κ 的单调性). 固定 $a \in [0, 1]$ ，映射 $c \mapsto \kappa(c, a)$ 在 $[0, 1]$ 上单调不减；固定 $c \in [0, 1]$ ，映射 $a \mapsto \kappa(c, a)$ 在 $[0, 1]$ 上单调不减。特别地，当 $c = 0$ 或 $a = 0$ 时， κ 取到它的下限

$$\kappa(0, a) = \kappa(c, 0) = 0.4. \quad (7.8)$$

这个定理在说什么：信心和共识度是“越多越好”的两个旋钮，往哪个方向拧，系数都只会往上走或者不动，绝不会反过来。而且只要有一个旋钮拧到底（归零），系数就落在 0.4 这个底座上——这是“最保守档”，不是“清仓档”。

证明。先证明关于 c 的单调性。

第一步：把内层看成 c 的函数。固定 a ，令

$$g_a(c) = 0.4 + 0.6ac. \quad (7.9)$$

这是关于 c 的一次函数，斜率是 $0.6a$ 。对 c 求偏导：

$$\frac{\partial g_a}{\partial c} = 0.6a \geq 0, \quad (7.10)$$

因为 $a \geq 0$ 而 $0.6 > 0$ 。斜率非负，故 g_a 在 $[0, 1]$ 上单调不减。

第二步：复合上 $\min$ 。由式 (7.10)，对任意 $0 \leq c_1 \leq c_2 \leq 1$ 有 $g_a(c_1) \leq g_a(c_2)$ 。又 $\min(1, \cdot)$ 单调不减，于是

$$\kappa(c_1, a) = \min(1, g_a(c_1)) \leq \min(1, g_a(c_2)) = \kappa(c_2, a). \quad (7.11)$$

即 $c \mapsto \kappa(c, a)$ 单调不减。

第三步：关于 a 同理。固定 c ，令 $g_c(a) = 0.4 + 0.6ca$ ，对 a 求偏导得

$$\frac{\partial g_c}{\partial a} = 0.6c \geq 0, \quad (7.12)$$

同样得到 $a \mapsto \kappa(c, a)$ 单调不减。

第四步：边界值。令 $c = 0$ ，则 $g(0, a) = 0.4 + 0 = 0.4$ ，故 $\kappa(0, a) = \min(1, 0.4) = 0.4$ 。令 $a = 0$ 同理得 $\kappa(c, 0) = 0.4$ 。结合定理 7.2 的下界 0.4，这正是 κ 能取到的最小值。 ■

注 7.4 (公式与实现的一处差异，必须说清楚). 定理 7.3 说的是：当确定度归零时，系数落到下限 0.4。但系统代码里还有一道守卫——当输入的信心或共识度小于等于零时，函数直接返回 1.0，也就是“完全不缩放”。

这不是矛盾，是两个不同的语义被写在了同一个位置上。在系统里，0 这个值出现的场合，是“这个数没拿到”（模型没给出、解析失败、字段缺失），而不是“信心真的很低”。对一次数据缺失就动刀，等于让一次失联决定仓位，所以实现选了“不动”。

换句话说：式 (7.2) 描述的是“数据都在时的保守度曲线”，而代码的守卫描述的是“数据不在时干脆不做判断”。读者自己接这套代码时，这两件事必须分开考虑，否则会得到一个“AI 一失败、仓位自动腰斩”的系统——这跟本章反复强调的“AI 单点故障不得阻断交易”正好相反。

最后一块拼图。有了 κ ，谁来保证它真的只是“踩刹车”？答案是那个 $\min$ 。

引理 7.5 (只降不升). 设 $A = \text{ruleRate}$ 为规则层给出的仓位基准， p 为主张官的建议仓位， o 为异议审查官的建议仓位， κ 为式 (7.2) 的缩放系数。记

$$\text{Effective} = \min\left(A, \text{clamp}(p, 0.2, 1.0), \text{clamp}(o, 0.2, 1.0), \kappa\right), \quad (7.13)$$

则

$$\text{Effective} \leq A = \text{ruleRate}. \quad (7.14)$$

这个引理在说什么：生效仓位是四个数的较小者，所以它一定不超过其中任何一个。特别地，它不超过规则层给的 A 。AI 无论说什么，都不可能把仓位抬过规则层划的那条线。

证明. 第一步：min 的基本性质。设 $S = \{A, \text{clamp}(p, 0.2, 1.0), \text{clamp}(o, 0.2, 1.0), \kappa\}$ 是一个由四个实数组成的有限集合。由 min 的定义， $\min S$ 是 S 中的最小元素，因此对 S 中每一个元素 x 都有

$$\min S \leq x. \quad (7.15)$$

这是“最小者不超过任何成员”的直接推论，不需要任何额外假设。

第二步：代入 A 。取 $x = A$ ，得

$$\text{Effective} = \min S \leq A = \text{ruleRate}, \quad (7.16)$$

即式 (7.14)。

第三步：说明“放大”不可能发生。把式 (7.13) 里四项逐一检查它们的上界：由 theorem 7.1 的 clamp 定义， $\text{clamp}(p, 0.2, 1.0) \leq 1.0$ 且 $\text{clamp}(o, 0.2, 1.0) \leq 1.0$ ；由定理 7.2， $\kappa \leq 1$ 。也就是说，除 A 以外的三项顶到天也只有 1，它们只能往下拉，不可能往上顶。而 A 是由规则层决定的、AI 无权修改的量。所以：生效仓位超过 ruleRate 的唯一路径，是 ruleRate 自己变大；而 AI 在这条路径上没有任何话语权。

第四步（附带结论）：下界。由 $\text{clamp}(\cdot, 0.2, 1.0)$ 的定义， $B := \text{clamp}(p, 0.2, 1.0) \geq 0.2$ ， $C := \text{clamp}(o, 0.2, 1.0) \geq 0.2$ ；由定理 7.2， $\kappa \geq 0.4$ 。故

$$\text{Effective} = \min(A, B, C, \kappa) \geq \min(A, 0.2). \quad (7.17)$$

当 $A \geq 0.2$ 时，生效仓位落在 $[0.2, 1]$ 内；而当 $A < 0.2$ 时（下面会看到，真实记录里确实有过 $A = 0.06$ 的日子），生效仓位的上界就是 A 本身，比 0.2 更小。 ■

7.5 它在系统里长什么样

函数本身只有五行。

```
1 // kellyPositionScale 用「LLM 主张置信度 × 多空共识度」计算 Kelly 仓位缩放系数。
2 // certainty = conf/100 × agreement/100 in [0,1]; scale = 0.4 + 0.6×certainty in [0.4,1]。
3 // 只在信心或多空共识不足时下调买入仓位，绝不放大到超出规则门。
4 func kellyPositionScale(conf, agreement int) float64 {
5     if conf <= 0 || agreement <= 0 {
6         return 1.0
7     }
8     certainty := clampF(float64(conf)/100*float64(agreement)/100, 0, 1)
9     return clampF(0.4+0.6*certainty, 0.4, 1.0)
10 }
```

代码 7.1 Kelly 仓位缩放（internal/cio/proposal.go）

这段话在说什么：第 3 行就是注 7.4 里那道守卫。第 5 行算确定度，顺手用 `clampF` 夹进 $[0, 1]$ 。第 6 行就是式 (7.2)，只不过这里没有写 $\min(1, \cdot)$ ——因为输入已经被夹在 $[0.4, 1]$ 之间了， $\min(1, \cdot)$ 变成了多余的保险。两者等价。

这道系数在哪里被用上？在 `decideProposalGate` 里：

```
1 gate := math.Min(ruleRate, clampF(prop.TargetPositionRate, 0.2, 1.0))
2 if ver := c.challengeProposal(ctx, in, prop); ver != nil { // Phase2 异议
3     gate = math.Min(gate, clampF(ver.SuggestedRate, 0.2, 1.0))
4 }
5 if db := c.debateProposal(ctx, in, prop); db != nil { // Phase3 辩论
6     res.KellyScale = math.Min(1.0, kellyPositionScale(prop.Confidence, db.Agreement))
7     gate = math.Min(gate, res.KellyScale) // Phase4 Kelly
8 }
9 res.Effective = gate
```

代码 7.2 Kelly 缩放进入门控（节选，internal/cio/proposal.go）

这段话在说什么：三个 `math.Min` 摺在一起，就是式 (7.13) 逐项落地的过程。每加一道意见，就把 `gate` 和那道意见取一次较小值。 κ 也只是其中一道——它没有任何特权，不会覆盖别人，只会再压一层。

顺带说明：Phase5 的多提案集成还会再压一层（明显偏空时收敛到 0.5 以下，轻度偏空收敛到 0.75 以下），那是第 chapter 八 章的内容。它同样只有 `min`，没有 `max`。

数字会被留下来吗？会。`attachProposal` 把辩论结果、Kelly 系数、集成结果一起塞进 `CIODecision.Evidence` 的一个结构里（`decisionProposalPayload`），字段名 `kelly_position_scale`。

```
1 type decisionProposalPayload struct {
2     SuggestedPositionRate float64      `json:"suggested_position_rate"`
3     EffectivePositionRate float64      `json:"effective_position_rate"`
4     Debate                 *debateVerdict `json:"debate,omitempty"`
5     KellyScale              float64      `json:"kelly_position_scale,omitempty"`
6     Ensemble                *ensembleVerdict `json:"ensemble,omitempty"`
7     Evidence                []string      `json:"evidence"`
8     GeneratedAt             string        `json:"generated_at"`
9 }
```

代码 7.3 证据链负载（节选，`internal/cio/proposal.go`）

这段话在说什么：注意 `suggested_position_rate`（AI 建议的）和 `effective_position_rate`（实际生效的）是两个字段，并排放着。这是刻意设计——第 chapter 十 章会讲，审计时要能一眼看出“AI 说了多少”和“最后用了多少”之间的差距。如果这两个数被合并成一个，事后追因就无从下手。

现在用真实数据检验一遍引理 7.5。

表 7.1 引理 7.5 在真实记录上的表现：生效仓位的上界就是 `ruleRate`

交易日	修正后总分	标签	ruleRate（真实）	生效仓位上界
2026-08-31	52.30	结构性震荡	0.80 ³	Effective $\leq$ 0.80
2026-09-02	41.82	偏弱	0.21 ⁴	Effective $\leq$ 0.21
2026-09-09	40.40	偏弱	0.20 ⁵	Effective $\leq$ 0.20
2026-09-10	38.43	退潮风险	0.10 ⁶	Effective $\leq$ 0.10
2026-09-11	25.71	退潮风险	0.06 ⁷	Effective $\leq$ 0.06

最后一列不是算出来的另一个数，而是引理 7.5 的直接结论：不论 AI 说了什么，那一格的生效仓位都不可能超过它左边的 `ruleRate`。

数据来源：`market_sidim_daily` 表全部 5 条真实记录，区间 2026-08-31 至 2026-09-11；只读查询于 2026-09-24。

2026-09-11 那一行最说明问题。那天规则层给的 $A = 0.06$ 。无论主张官多有信心、辩论多一致，生效仓位都不可能超过 0.06。AI 的最高权限，是把这个 0.06 再往下压一点。

再单独看 κ 这条曲线。

第三行就是本章开头那一行：“信心 60、共识 70”，乘出确定度 0.42，代进滑杆得 0.652。所以那行“Kelly 仓位 $\times 0.65$ ”的意思不是“系统很看好”，是“系统把这一路意见按六五折采纳”。

表 7.2 κ 的取值（按式 (7.2) 计算）

信心 c (示意)	共识度 a (示意)	确定度 ca	κ	含义
1.00	1.00	1.000	1.000	两样都满格：不缩放
0.80	0.70	0.560	0.736	都偏高：几乎不缩放
0.60	0.70	0.420	0.652	本章开头那一行
0.50	0.50	0.250	0.550	两样都一般
0.20	0.30	0.060	0.436	都很低：接近底座
0.00	任意	0.000	0.400	公式取到下限（实现见注 7.4）

置信度与共识度两列在系统库中没有持久化的辩论记录可供引用（该库未保存当日 `confidence` 与 `agreement` 原始值）⁸，此处取值用于演示式 (7.2) 的形态； κ 一列严格按该式计算，可由读者手算复核。

数据来源：公式出处 `internal/cio/proposal.go` 的 `kellyPositionScale`；置信度/共识度示意。

常见误解 · 这个 Kelly 就是凯利公式，是最优下注比例 名字撞车，是这件事最容易出错的地方。教科书里的凯利公式，回答的是“给定赔率和胜率，长期增长率最大的下注比例是多少”。它的形式里有一项是**赔率**——赢的时候赚多少、输的时候亏多少。它给出的数字可以大于 1（意味着该用杠杆），它的目标是**最大化长期增长率**。

本章这个 κ 有三个地方和它完全不同：

第一，它**没有赔率**。整个公式里只有两个输入：信心和共识度。它压根不知道这笔交易赢面多大。

第二，它有**上界 1**，靠 `min` 和 `clamp` 双保险锁死。真正的凯利公式不会给你锁上界。

第三，它的目标是**更保守**，不是最大化增长率。它有下界 0.4，就是为了防止“确定度低”被过度解释成“空仓”。

所以请把它当成一个**保守度控制器**，而不是一个下注比例。它的作用只有一个：把 **AI 那一路意见**，按它的自我确信程度打个折。名字里带“Kelly”只是因为它借用了“确定度越高、仓位越足”这个直觉，而不是因为它继承了那个公式的任何数学性质。

如果有一天你把它换成真正的凯利公式，你会得到一个能加杠杆的系统——那和这套系统“AI 只能踩刹车”的设计前提是直接冲突的。

常见误解 · AI 出故障时，就按它上次说的仓位打个折 这是我曾经觉得最“稳妥”的做法：如果今天模型没响应，那就沿用昨天的仓位系数，或者干脆取个保守值 0.5。

这个做法的问题在于：它让**一次技术故障变成了一个交易决策**。

看代码里那条硬回退线。LLM 没配置、调用失败、超时、输出解析不了——主张官返回 `nil`，`decideProposalGate` 立刻带着 `A = ruleRate` 返回，`Effective = A`。也就是说，**生效仓位完全等于规则层给的基准，一个百分点都不打折，也一个百分点都不多加**。

为什么不是“打个折更保险”？因为打折是一个**判断**，判断需要理由，而此刻系统手里恰好没有任何新理由——它手里只有一个“我失败了”。用一次失败去推动仓位变化，是在把不确定性从一个地方搬到另一个地方。

更严重的是它会**掩盖故障**。如果 AI 挂了之后仓位自动变成 0.5，那么从最终订单上看，你和“AI 正常但不太看好”的情况几乎分不出来。等到哪天你发现决策质量莫名其妙变差了，再回头查，就什么都查不到了。

规则层给出的 `A`，本来就是这套系统里“**没有 AI 也该执行**”的答案。AI 的价值是让它更保守，不是替代它。所以 AI 失联时，正确答案是“回到 `A`”，而不是“回到 `A` 的某个折扣”。

本章的真实出处

- Kelly 缩放函数: `internal/cio/proposal.go`, `kellyPositionScale`。
- 生效仓位四项 `min` 门控与硬回退: 同文件, `decideProposalGate` (`Effective` = `min(ruleRate, clamp(主张, 0.2, 1), clamp(异议, 0.2, 1),  $\kappa$ )`)。
- 证据链落库 (含 `kelly_position_scale`、`suggested_position_rate`、`effective_position_rate`): 同文件, `attachProposal` 与 `decisionProposalPayload`。
- 真实 `ruleRate` 记录: `market_sixdim_daily` 表 (5 条, 2026-08-31 至 2026-09-11)。
- 前端展示: CIO 页【多空辩论】区块里的“Kelly 仓位 $\times$ 系数”, 以及决策日志中的证据链。

动手试一试 (十分钟)

1. 拿出一张纸, 手算一遍本章开头那一行: $c = 0.60$ 、 $a = 0.70$, 求 `certainty` 与 κ 。答案应该是 0.42 与 0.652。
2. 再算 $c = 0.20$ 、 $a = 0.30$, 答案是 0.06 与 0.436。你会发现: 信心和共识度都掉了一半, 系数却只从 0.652 掉到 0.436——底座 0.4 一直在下面托着。
3. 现在算 $c = 1.00$ 、 $a = 1.00$ 。答案必须是 1.000, 不是 1.4、也不是 1.6。这就是 `min` 和 `clamp` 的作用。
4. 打开系统, 进入首席投资官 (CIO) 页, 找一条有 Kelly 系数的决策, 把它的 `Effective` 与当天的 `ruleRate` 对一下。检验引理 7.5: 前者一定不超过后者。
5. 最后一问: 如果明天你把 0.4 改成 0.1, 系统会更容易满仓还是会更容易空仓? 先想清楚再动手 (改完记得改回来)。

到这里, 一路意见的折算讲完了: 主张说了多少、异议说了多少、辩论一致到什么程度, 全都折算成了一个不超过 1 的系数。

可你数一数: 主张、异议、辩论、规则——手上已经有四路意见了。它们方向不同、语气不同、权重也不该一样。

那么下一个问题很自然: 如果不止两个意见呢? 这四路话, 怎么合成一句?

投票能不能比专家更靠谱？——多提案集成

8.1 四个人，四个说法

二零二六年九月十日，星期四。

那一天系统自己留下了记录：六维判势算出的修正总分是 38.43¹，标签定为“退潮风险”，建议的仓位系数只有 0.10²。这是系统里最保守的那一档。

然后，决策层的另外三个来源也表了态。把它们和规则来源摆在同一张纸上，是这样的：

- **主张官：**看多。理由是“技术趋势维度给了 85 分，指数还站在 MA20 上方”。
- **异议审查官：**RISK。理由是“市场广度只有 8.69 分——指数在涨，可真正在涨的股票少得可怜”。
- **辩论裁决官：**中性，共识度 50。多空各摆三条论据，谁也压不倒谁。
- **规则来源：**偏空。因为它读到的标签里有“退潮”两个字。

四个来源，两个说空，一个说多，一个说不知道。

我盯着这张纸想了很久。这不是我编出来的困境，注意看前两条：技术趋势的 85 分和市场广度的 8.69 分，是同一天、同一份行情算出来的两个数，谁都没有算错。它们只是互相不同意。

真正的问题是：这四个说法，最后应该变成哪一个决定？

这里要先说清楚一件不体面的事。上面四条里，“规则”那一条来自系统真实记录；另外三条表态，是为了把机制讲清楚而设的示意值。系统确实会把它们随决策一并落库（第 chapter 十章会讲这件事），但我手上没有可以引用的运行期决策库。所以：算例是示意的，下面要走的公式和参数，是代码里的真公式和真参数。

8.2 直觉：陪审团不改变上限，它只防止一个人说了算

直觉·十二个普通人 法庭上那个陪审团，是十二个普通人，不是十二个法律专家。

如果你问“十二个普通人加起来，会不会比法官更懂法条”，答案显然是不会。可陪审团的意义从来不在那儿。它的意义在于：要让判决错，得让十二个人同时错。

董事会表决也是同一个道理，不过多了一个细节：董事长的票比普通董事重。这叫加权。但请注意加权的能力边界——它只能把票往大家已经说出来的那些方向上加，不能让结论跑到所有人意见的范围之外去。哪怕董事长一个人的票重到能压过全场，结论也只能停在“最看多的那个人”和“最看空的那个人”之间，越不出去。

这两件事合起来，就是本章全部的分量：

投票不提高上限，它只降低单点错误。

¹真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

²真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

最聪明的那个来源有多聪明，加权的投票并不能让它更聪明；但“某一个来源犯了错”这件事，从此不再等于“整个系统犯了错”。

8.3 不引入投票，会怎样

不用投票，系统照样能跑。因为主张官本来就给出方向，规则门本来就给出仓位上限。它少掉的是三样东西。

第一，方向失去了独立复核。主张官说看多，没有人用另一种方式再问一遍“真的偏多吗”。一个来源的一次幻觉，就是一个方向。

第二，少了一个“整队一起偏空”的收敛点。收紧仓位的路径本来就很多（主张官的保守建议、异议官的降仓、Kelly 系数），但它们都挂在同一条链上。缺少一个把多个来源的方向合起来看的环节，就没有“所有人都觉得不对”这种集体信号。

第三，分歧没有被变成一个数。回头看一下本章开头那张纸：四个人四个说法。如果没有净多分，这张纸就只是一段争论；有了净多分，它就变成一行可以落库、可以复核的记录（这一点第 chapter 十 章会接着讲）。

我要把话说得再老实一点：这个机制的力量是有限的。它不改变方向、不抬高上限，唯一的动作是“当多数来源整体偏空时，把仓位上限再收紧一档”。为什么要为一个“只能往下压”的东西写一整章？因为在一个已经处处往下压的系统里，唯一被允许的动作方向就是保守，而值得讨论的从来不是“能不能加码”，而是“这一档该压到多低、以及凭什么”。

8.4 一个差不多对的模型：数人头

最粗糙、也最符合直觉的模型是这样的：每个来源一票，多者胜，平票算中性。

这个模型抓住了核心——决定不再由一个人做出。但它太粗，漏掉了四件事。

漏掉的第一件：一票不等于一票。主张官说看多、信心 90，和主张官说看多、信心 30，在这个模型里是同一票。可它们显然不是同一件事：前者是“我很确定”，后者是“我倾向这样，但你别太当真”。

漏掉的第二件：人数不等于分量。三个来源两票看多，和四个来源两票看多（另一个弃权），在这个模型里也是一回事。前者是“多数”，后者是“相对多数里的一方”。

漏掉的第三件：平票没有定义。两个看多、两个看空，谁赢？这个模型在这里会卡住，或者说不出理由。

漏掉的第四件：它会跳变。从“一半对一半”到“多一票”，结论突然翻面。中间没有一段过渡区，告诉我们“今天其实很接近，别太用力”。

把这四件事补上，就得到本章的严谨版本。

8.5 严谨版本：把四个说法折成一个数

先说这个模型想干什么：把每一路意见既能“表态”、又能“表明自己有多重”，最后合成一个落在 -1 与 $+1$ 之间的分数。分数为正表示整体偏多，为负表示整体偏空，接近 0 表示双

方拉平。

定义 8.1 (来源表态与权重). 设有 n 个来源。第 i 个来源给出一个**表态分** $v_i \in [-1, 1]$: +1 表示明确看多, 0 表示中性, -1 表示明确看空; 同时给出一个**权重** $w_i \geq 0$, 表示这一路意见该占多大分量。系统里 n 最多为 4: 主张、异议、辩论、规则。

定义 8.2 (净多分). 记 $S = \sum_{i=1}^n w_i$ 。当 $S > 0$ 时, 定义

$$\text{net} = \frac{\sum_{i=1}^n w_i v_i}{\sum_{i=1}^n w_i}. \quad (8.1)$$

当 $S = 0$ (一个来源都没有, 或所有来源的权重都是 0) 时, 约定 $\text{net} = 0$ 。

这个约定需要一句解释。 $S = 0$ 意味着“没有任何来源参与表决”, 此时分子分母都没有意义, 式 eq. (8.1) 是一张写不出答案的空表。这种情况下的正确做法不是猜一个方向, 而是**承认自己没有方向**: 0 恰好就是“中性”, 它同时会让后续的仓位收敛失效 (后面会看到, 只有净多分为负时才会动手), 于是规则门原样生效。一句话: **没票就等于弃权, 不等于投一张假票**。

定理 8.3 (净多分的有界性). 在定义 8.1、8.2 下, 恒有 $\text{net} \in [-1, 1]$ 。更进一步, net 落在所有表态分的最小值与最大值之间:

$$\min_i v_i \leq \text{net} \leq \max_i v_i. \quad (8.2)$$

这个定理在说什么: 不管权重怎么分配、怎么加权, 投票的结果都跑不出大家意见的范围。加权能改变结果落在区间里的位置, 但改变不了区间的边界。这正是前面那句“董事长的票再重, 也越不出全场意见”的数学版本。

证明. 先证下界。对每个来源, 由 $v_i \geq -1$ 与 $w_i \geq 0$, 在不等式两边同乘 w_i (乘一个非负数, 不等号方向不变) 得

$$-w_i \leq w_i v_i.$$

对 i 从 1 到 n 求和, 得

$$\sum_{i=1}^n w_i \cdot (-1) = -\sum_{i=1}^n w_i \leq \sum_{i=1}^n w_i v_i.$$

再证上界。同理由 $v_i \leq 1$ 与 $w_i \geq 0$ 得 $w_i v_i \leq w_i$, 求和得

$$\sum_{i=1}^n w_i v_i \leq \sum_{i=1}^n w_i \cdot 1 = \sum_{i=1}^n w_i.$$

两式合起来:

$$-\sum_{i=1}^n w_i \leq \sum_{i=1}^n w_i v_i \leq \sum_{i=1}^n w_i.$$

当 $S = \sum_i w_i > 0$ 时，三边同除以 S 。 S 是正数，除以正数不改变不等号方向，于是

$$-1 \leq \frac{\sum_i w_i v_i}{\sum_i w_i} = \text{net} \leq 1,$$

即 $\text{net} \in [-1, 1]$ 。

当 $S = 0$ 时，按定义 8.2 的约定 $\text{net} = 0$ ，而 $0 \in [-1, 1]$ ，结论同样成立。

把上界证明里的 1 换成 $\max_i v_i$ 、把下界证明里的 -1 换成 $\min_i v_i$ ， $w_i \geq 0$ 这一条保证乘除不改变不等号方向，两步可以照抄一遍，即得不等式 eq. (8.2)。■

引理 8.4 (多数方向与净多分的符号一致). 把来源按表态分成三组：看多组 ($v_i = +1$)、看空组 ($v_i = -1$)、中性组 ($v_i = 0$)。记

$$W_+ = \sum_{v_i=+1} w_i, \quad W_- = \sum_{v_i=-1} w_i, \quad W_0 = \sum_{v_i=0} w_i.$$

若 $W_+ + W_- + W_0 > 0$ ，则

$$\text{sign}(\text{net}) = \text{sign}(W_+ - W_-).$$

特别地， $W_+ = W_-$ 时 $\text{net} = 0$ 。

这个引理在说什么：净多分的符号，和“偏多票的权重和减去偏空票的权重和”的符号，是同一件事。也就是说，加权平均并没有发明一种新的判决方式——它就是把“数人头”这件事写成了一个连续的分数的，顺便把“这部分人有多重”也算进去了。

证明. 由定义，每个表态分只取 +1、0、-1 三个值之一。于是

$$\sum_{i=1}^n w_i v_i = \sum_{v_i=+1} w_i \cdot (+1) + \sum_{v_i=-1} w_i \cdot (-1) + \sum_{v_i=0} w_i \cdot 0 = W_+ - W_-,$$

最后一项因为乘 0 而消失。记分母 $D = W_+ + W_- + W_0 = \sum_i w_i > 0$ 。代入式 eq. (8.1)：

$$\text{net} = \frac{W_+ - W_-}{D}.$$

$D > 0$ ，除以正数不改变符号，故 $\text{sign}(\text{net}) = \text{sign}(W_+ - W_-)$ 。当 $W_+ = W_-$ 时分子为 0，于是 $\text{net} = 0$ 。■

最后一步，把这个分数变成对仓位的约束。设 $\text{rate} \in [0, 1]$ 是当前已经生效的仓位系数（它可能来自六维判势，也可能已经被主张官、异议官、Kelly 系数压过好几轮）。

定理 8.5 (只降不升的收敛映射). 定义映射 $f: [-1, 1] \rightarrow [0, 1]$ ：

$$f(\text{net}) = \begin{cases} \min(\text{rate}, 0.5), & \text{net} < -0.5, \\ \min(\text{rate}, 0.75), & \text{net} \geq -0.5. \end{cases} \quad (8.3)$$

则对一切 $\text{rate} \in [0, 1]$ 与一切 $\text{net} \in [-1, 1]$ ：

(i) $f(\text{net}) \leq \text{rate}$;

(ii) f 关于 net 单调不减。

这个定理在说什么，两句话。**第一句：投票不会让仓位变大。**不管净多分多高、多低，出来的仓位永远不超过原来那个数——最多持平。**第二句：不会出现反常。**净多分越高，允许的仓位上限只会越大或不变，绝不会“越看多反而压得越狠”。

证明. (i) 分情况讨论。

情况一： $\text{net} < -0.5$ 。此时由式 eq. (8.3) 有 $f(\text{net}) = \min(\text{rate}, 0.5)$ 。对任意实数 a, b ，由 $\min(a, b)$ 的定义（取两者中较小者）有 $\min(a, b) \leq a$ 。取 $a = \text{rate}$ 、 $b = 0.5$ ，得 $\min(\text{rate}, 0.5) \leq \text{rate}$ ，即 $f(\text{net}) \leq \text{rate}$ 。

情况二： $\text{net} \geq -0.5$ 。此时 $f(\text{net}) = \min(\text{rate}, 0.75) \leq \text{rate}$ ，依据同样是 $\min(a, b) \leq a$ 。

两种情况的并集覆盖整个 $[-1, 1]$ ，因此对一切 $\text{net} \in [-1, 1]$ 与一切 $\text{rate} \in [0, 1]$ 都有 $f(\text{net}) \leq \text{rate}$ 。这一条不需要用到 $\text{rate} \leq 1$ ，它是“ $\min$ 的第一个参数不会被超过”的直接结果。

(ii) 分段说明单调性。取任意 $\text{net}_1 < \text{net}_2$ ，需要证明 $f(\text{net}_1) \leq f(\text{net}_2)$ 。

在开区间 $(-1, -0.5)$ 上， f 恒等于常数 $\min(\text{rate}, 0.5)$ ；在区间 $[-0.5, 1]$ 上， f 恒等于常数 $\min(\text{rate}, 0.75)$ 。常数函数显然单调不减，所以只要 net_1 与 net_2 落在同一个区间里，结论成立。

剩下的情况只有“跨点”： $\text{net}_1 < -0.5 \leq \text{net}_2$ 。此时

$$f(\text{net}_1) = \min(\text{rate}, 0.5), \quad f(\text{net}_2) = \min(\text{rate}, 0.75).$$

由 $0.5 \leq 0.75$ ，且固定 rate 时 $\min(\text{rate}, \cdot)$ 关于第二个参数单调不减（第二个参数变大，取小值的结果不会变小），得

$$\min(\text{rate}, 0.5) \leq \min(\text{rate}, 0.75),$$

即 $f(\text{net}_1) \leq f(\text{net}_2)$ 。跨点处 f 不下降。

综上， f 在 $[-1, 1]$ 上单调不减。 ■

注 8.6 (实现里的封顶函数是三段，不是两段). 本章依据的公式与源码核对后有一处差别，必须交代清楚，否则读者会对着代码对不上：**实现只在 $\text{net} < 0$ 时才动手**。也就是封顶函数实际是

$$c(\text{net}) = \begin{cases} 0.5, & \text{net} \leq -0.5, \\ 0.75, & -0.5 < \text{net} < 0, \\ 1, & \text{net} \geq 0, \end{cases} \quad f_{\text{实现}}(\text{net}) = \min(\text{rate}, c(\text{net})).$$

它与式 eq. (8.3) 的差别只出现在 $\text{net} \geq 0$ 那一段：两段式版本会把仓位封到 0.75，实现版本不封顶。

定理 8.5 的结论对实现版本同样成立，理由只有两步： c 单调不减 ($0.5 \leq 0.75 \leq 1$)，

$\min(\text{rate}, \cdot)$ 也单调不减，复合之后 $f_{\text{实现}}$ 单调不减；而 $\min(\text{rate}, c(\text{net})) \leq \text{rate}$ 是恒成立的。所以“只降不升”这条纪律没有打折。

差别在于性格。两段式是“只要不是明显看多，就一律压一档”；实现是“只有集成方向真的转空，我才伸手”。后者更弱，也更符合这个环节的定位：它是用来发现问题的，不是用来表达赞同的。

8.6 它在系统里长什么样

四个来源怎么变成权重，权重怎么变成净多分，源码在 `internal/cio/ensemble.go` 一个函数里，名字叫 `ensembleVote`。

```

1 func ensembleVote(prop *llmDecisionProposal, sk *skepticalVerdict,
2     db *debateVerdict, regime string) *ensembleVerdict {
3     votes := make(map[string]string)
4     net := 0.0
5
6     if prop != nil { // 来源一：主张官
7         votes["主张"] = prop.MarketDirection
8         net += dirScoreStr(prop.MarketDirection) *
9             clampF(float64(prop.Confidence)/100, 0, 1)
10    }
11    if sk != nil { // 来源二：异议评审
12        switch strings.ToUpper(sk.Verdict) {
13            case "AGREE": votes["异议"] = "AGREE"; net += 0.3 // 认可 → 偏多
14            case "RISK": votes["异议"] = "RISK"; net -= 0.7 // 明显风险 → 偏空
15            default: votes["异议"] = "CAUTION"; net -= 0.2 // 需留意 → 轻偏空
16        }
17    }
18    if db != nil { // 来源三：多空辩论
19        votes["辩论"] = db.Direction
20        net += dirScoreStr(db.Direction) *
21            clampF(float64(db.Agreement)/100, 0, 1)
22    }
23    if rt := strings.TrimSpace(regime); rt != "" { // 来源四：六维规则信号
24        if strings.Contains(rt, "强势") { votes["规则"] = "BULLISH"; net += 0.3 }
25        if strings.Contains(rt, "退潮") || strings.Contains(rt, "弱势") {
26            votes["规则"] = "BEARISH"; net -= 0.3
27        }
28    }
29
30    net = clampF(net, -1, 1) // 截断到 [-1,1]
31    dir := "NEUTRAL"
32    if net > 0.25 { dir = "BULLISH" } else if net < -0.25 { dir = "BEARISH" }
33    return &ensembleVerdict{Direction: dir, Votes: votes,
34        NetScore: math.Round(net*100) / 100}
35 }

```

代码 8.1 多提案集成投票（节选，`internal/cio/ensemble.go`；为便于阅读合并了分支赋值）

这段话在说什么，四件事。

第一，每个来源的表态都有一个配套的分量。主张官看多，加的分是“看多”这一个方向分乘上它的信心；辩论裁决官看多，乘的是共识度；它们都是 $[-1, 1]$ 里的数。异议审查官不谈方向，它谈的是**立场**：认可、需要留意、还是明显有风险，对应 $+0.3$ 、 -0.2 、 -0.7 三个固定分。规则来源最干脆：标签里有“强势”就 $+0.3$ ，有“退潮”或“弱势”就 -0.3 。

第二，权重全都在 $[0, 1]$ 里，但每一路的“量纲”并不统一。这一点非常重要，也是本章必须老实交代的地方：主编的权重是“信心除以 100”，辩论的权重是“共识度除以 100”，异议和规则则是三个写死的常数。这四路权重加起来，可能远大于 1。

第三，最后一步不是除以权重和，而是截断。源码里写的是 `clampF(net, -1, 1)`，不是定义 8.2 的除法。这两者在“来源方向一致”时结果相同（四个都说多，加权求和与加权平均都饱和到 $+1$ ），但在“方向分歧”时结果不同。差别有多大，下一节的表格会给一个真实算例。

第四，方向判定有一圈“死区”。净多分大于 0.25 才算看多，小于 -0.25 才算看空，中间算中性。所以严格按引理 8.4 说，实现是“符号判定”的加宽版。

表 8.1 把四个来源的权重规则整理成一张表。

表 8.1 四路来源的表态分与权重

来源	表态分 v_i	权重 w_i	它代表什么
主张 prop	看多 $+1$ / 中性 0 / 看空 -1	<code>confidence</code> /100	主张官对自己方向判断的把握
异议 sk	固定分（见右）	写死的 0.3 / 0.2 / 0.7	AGREE 偏多、 CAUTION 轻偏空、 RISK 明显偏空
辩论 db	看多 $+1$ / 中性 0 / 看空 -1	<code>agreement</code> /100	多空双方对“发生了什么”的一致程度
规则 regime	强势 $+1$ / 其他 0 / 退潮·弱势 -1	写死的 0.3	六维判势标签的规则化读法

四路权重之和没有归一化：主张与辩论的权重在 $[0, 1]$ 内，异议的权重是 0.7 或 0.2 或 0.3，规则的权重是 0.3。因此加权和可能超出 $[-1, 1]$ ，由 `clampF` 截断。

数据来源：系统源码 `internal/cio/ensemble.go`，函数 `ensembleVote`；核对时间 2026-09-24。

系统自带的单元测试里有三个真实算例，输入和输出都是可复算的。表 8.2 把它们算了一遍。

第二行那个算例值得停一下。它的加权和是 -1.10 ，被截断成 -1.00 ：**四路来源全部反对主张官一个人**，集成给出的净多分直接压到最低。这就是本章开头那个“四个人四个说法”的极端情形——只不过这个例子里不是两两对半分，而是四比一。

现在来看前面留的那个悬念：归一化与截断到底差多少。同一个算例，如果用定义 8.2 的加权平均，权重和是 $0.8 + 0.7 + 0.9 + 0.3 = 2.7$ ，于是

$$\text{net}_{\text{归一化}} = \frac{-1.10}{2.7} \approx -0.407.$$

两个口径给出两个答案：实现口径 -1.00 (≤ -0.5 ，仓位压到 0.5)，归一化口径 -0.407 (> -0.5 ，

表 8.2 三个可复算的算例：从四路表态到净多分与方向

场景	主张	异议	辩论	加权求和	净多分 / 方向
全看多、信心高	看多, 信心 90	缺	看多, 共识 85	$+0.90 + 0.85 + 0.30$	$+1.00$ / 看多
看多遇反对	看多, 信心 80	RISK	看空, 共识 90	$+0.80 - 0.70 - 0.90 - 0.30$	-1.00 / 看空
中性遇空头	中性, 信心 60	缺	看空, 共识 70	$0 - 0.70$	-0.70 / 看空

第一行三路来源的加权和为 2.05；第二行为 -1.10 ；第三行为 -0.70 。前两行都被 `clampF` 截断到 ± 1 ，第三行未越界。按式 eq. (8.3)，第二、三行的净多分都不高于 -0.5 ，仓位上限各被压到 0.5；第一行净多分为正，仓位不受影响。

数据来源：输入取自系统单元测试 `internal/cio/ensemble_test.go`；加权求和与方向由本章按 `internal/cio/ensemble.go` 的同一公式复算（核对时间 2026-09-24）。此处输入并非市场真实数据，只用于演示公式。

仓位只压到 0.75)。

这不是谁算错了。归一化口径把“四路来源意见分歧”这件事摊薄了：权重和越大，分歧越被稀释；实现口径选择让分歧饱和到 -1 ，因此更保守。用一句话记住这个差别：加权平均告诉你“平均而言大家怎么想”，加权求和加截断告诉你“到底有没有人强烈反对”。这个系统选的是后者——因为它要的不是平均值，是一个“该收手了”的信号。

这个结果最后去了哪里？它被写进决策记录里。Phase5 的那几行是这样：

```

1 // Phase5 多提案集成投票
2 res.Ensemble = ensembleVote(prop, res.Skeptic, res.Debate, in.Regime)
3 if res.Ensemble.NetScore < 0 {
4     if res.Ensemble.NetScore <= -0.5 {
5         gate = math.Min(gate, 0.5) // 明显偏空 → 仓位收敛到 <= 0.5
6     } else {
7         gate = math.Min(gate, 0.75) // 轻度偏空 → 仓位收敛到 <= 0.75
8     }
9 }
```

代码 8.2 集成结果如何收敛仓位（节选，`internal/cio/proposal.go`）

这段话在说什么：`gate` 就是“当前生效的仓位系数”。整个 Phase5 只有两个动作，都写在 `math.Min` 里——要么把它压到不超过 0.5，要么压到不超过 0.75。没有任何一条分支是把它抬上去的。

结果被放进 `CIODecision.Evidence`，随决策一起落库，字段名是 `ensemble`，里面有方向、`net_score`、以及四路来源各自的表态 `votes`。前端的 CIO 页专门有一块【多提案集成】，把它渲染成一行字。事实上，你看到的会是类似这样的一行：

CIO 页上那一行长什么样

【多提案集成】方向看空 · 净多 -100% · 来源主张 = 看多, 异议 = RISK, 辩论 = 看空, 规则 = 看空

这一行里，“净多 -100% ”就是 `net_score` 乘 100 之后取整。它把本章开头那张四个人四个说法的纸，压缩成了四个字的结论加上一个可核对的数。

8.7 常见误解与纠偏

常见误解·投票是为了选出更准的方向，从而提高收益 这是最容易想歪的一点。看到“集成投票”，本能反应是“三个臭皮匠顶个诸葛亮”——人多判断更准，判断更准就赚得更多。

前半句在这个系统里是错的，后半句更错。

投票改变的不是判断的**准确度**，是判断的**脆弱度**。请回头看定理 8.3 的推论：净多分永远落在所有来源表态的最小值与最大值之间。这句话有一个刺眼的含义——**如果最会判断的那个来源给出了全场最好的意见，投票也变不出比它更好的意见来**。投票的产出，永远被最好的那个来源封顶。

那它买到了什么？买到了“单个来源犯错不再致命”。用统计的话说：在“各路来源的误差相互独立、且都不系统性地偏”这个前提下（这是一个假设，不是定理，市场里它常常不成立），取平均不会改变误差的中心，但会降低误差的波动。**它优化的是方差，不是均值**。

所以正确的期待是：投票不会让收益期望变高，它降低的是“被单一错误观点带走”的概率。这两件事在投资里的价值差别很大——你怕的通常不是赚得少，而是在一件自己很确信的事情上亏掉一大块。

常见误解·来源缺了一个，就该补一个“中性票”把位置占满 既然讲的是“四路投票”，很自然会想：某一路没拿到意见（模型超时、解析失败），是不是应该补一个“中性”进去，保持四票齐全？源码的写法正好相反：`if prop != nil、if sk != nil、if db != nil`——谁缺席，谁的那一项就**整段不执行**，既不加分也不记票。净多分就在剩下的来源上算。

看起来这很“不严谨”，其实这恰恰是严谨。补一张中性票，等于**替一个没说话的来源表了态**；而“中性”这个值会参与加权求和，也就实实在在改变了净多分。你以为你在补齐格式，其实你在伪造意见——而且伪造得很隐蔽，因为下游看到的只是一个数字的微小变化。

系统对这件事的边界处理得很干净。单元测试里专门有一条“全部来源都为空”的用例：四个参数全传 `nil`、`regime` 传空串，`ensembleVote` 依然返回一个非 `nil` 的结果，净多分 0、方向 `NEUTRAL`、`votes` 为空表，不 `panic`。而净多分 0 不是负数，所以按式 eq. (8.3) 它不会触发任何封顶——规则门原样生效。

一句话记住：**降级不是缺陷，是设计**。缺数据就少投票，而不是补一个假票；没票时系统没有变弱，它只是没有加码。

常见误解·净多分是个 0 到 100 的“看多概率” `net_score` 落在 $[-1, 1]$ ，屏幕上又按百分数显示（“净多 -100%”、“净多 30%”），很容易被理解成“看多的概率是 30%”。

它不是概率。它是一个**加权后的表态分**：`+1` 表示所有参会来源都明确看多，`-1` 表示都明确看空，`0` 表示拉平——或者根本没有来源参会。请注意最后这一种，它是这个数最容易骗人的地方：同样的 `0`，既可能是“四方激烈对冲到拉平”，也可能是“今天一个来源都没拿到”。这两种情况在数字上完全一样，要区分它们，只能去看同一行里的来源个数和 `votes` 明细。

一个分数如果可以被两种完全不同的原因产生，那它就必须和它的解释一起读。这条规则不只适用于本章，第 chapter 十 章会把它升级成整个系统的纪律。

本章的真实出处

- 集成投票的完整实现（四路来源、权重、截断、方向死区）：`internal/cio/ensemble.go`，函数 `ensembleVote` 与 `dirScoreStr`。
- 结果结构 `ensembleVerdict` (`direction`、`votes`、`net_score`、`summary`)：同文件。

- *Phase5* 的调用与仓位封顶: *internal/cio/proposal.go*, 函数 *decideProposalGate*; 封顶动作为 $gate = \text{math.Min}(gate, 0.5)$ 或 $gate = \text{math.Min}(gate, 0.75)$ 。
- 落库路径: *CIODecision.Evidence* 的 *ensemble* 字段, 赋值处为 *attachProposal* 中的 *decisionProposalPayload*。
- 三个真实算例与“全空来源不 *panic*”用例: *internal/cio/ensemble_test.go*, *TestEnsembleVote* 与 *TestEnsembleVoteNilSourcesNoPanic*。
- 前端展示: *CIO* 页的【多提案集成】区块(方向 / 净多分 / 来源表态), 见 *ui/src/pages/CIO.tsx*。
- 四路来源各自的原始定义: 主张官与异议官见第 *chapter 六* 章, 辩论见第 *chapter 六* 章, *Kelly* 系数见第 *chapter 七* 章, 六维标签见第 *chapter 十九* 章。

动手试一试(十五分钟)

1. 打开系统, 进入首席投资官(CIO)页, 在决策日志里找到最近一条带【多提案集成】的记录。
2. 抄下它的净多分和四个来源的表态。
3. 拿一张纸, 按表 8.1 的规则手算一遍: 主张官那一项 = 方向分 × 信心; 辩论那一项 = 方向分 × 共识度; 异议按 AGREE/CAUTION/RISK 取 +0.3、-0.2、-0.7; 规则按标签取 ±0.3。四项加起来, 再截断到 [-1, 1]。
4. 看它和屏幕上的数对不对得上。对不上的话, 多半是某一路来源今天缺席了——去数一数 *votes* 里有几个来源。
5. 最后一问: 把净多分改成那个“归一化加权平均”的定义, 今天这一天的仓位上限会不会不一样? 如果不一样, 哪一个更合你的脾气?

投票投完了, 四路来源也都留了痕。

可是这里有一个问题我一直绕不过去: 这四个来源, 本质上都是同一个东西在说话——它们全都是那一个语言模型的输出。如果那个模型本身没了呢? 如果网络断了、接口挂了、进程崩了呢?

系统活不活得下来? 这是下一章的事。

第九章

如果 AI 突然失联，交易还继续吗？

9.1 窗口没了

那是一个交易日的下午。

我在做别的事，系统在后台跑。屏幕右下角的托盘图标还在，可当我点击主窗口的时候，它闪了一下就消失了。整台程序没了。

我先去看日志。日志文件的最后一行是正常的——“[QuantBot] Logging system initialized successfully”之后一路都是常规输出，没有报错、没有警告、没有异常堆栈。它写到某一刻，就停了。

然后我去看 Windows 的事件查看器。什么都没有。

一个程序在我面前消失，而我手上没有任何一条线索。那种感觉不是“出错了”，而是“出错这件事没有被记录下来”。我甚至不能说清它是几点几分死的、死在哪个环节。

直觉·自动驾驶的机械备份 想象一辆自动驾驶汽车。它的转向、制动都由电子系统控制——效率高、响应快，这是它存在的理由。

但没有任何一家车厂会说：“因为电子系统很可靠，所以方向盘可以拆掉。”

方向盘留着的理由，不是“电子系统不够聪明”，而是“**电子系统可以坏**，而当它坏的那一刻，车上的人必须还有办法把车停下来”。

机械备份的设计有一个共同点：它不追求和主系统一样好，它只追求在主系统消失之后，车还是可控的。

那台消失的程序，就是“主系统”；而它当时没有方向盘。

9.2 不给它方向盘，会付出三种代价

我在复盘这件事的时候发现，问题不是一个，而是三个，而且它们的严重程度是递增的。

代价一：一次调用失败，等于一天不做决定。如果那把“AI 失联就停下”的规则写死，那么模型接口一次抖动、一次超时、一次返回了格式不对的文本，当天的决策就整个空了。系统还活着，但它交出了方向盘——今天不选股、不调整、不出方案。

代价二：一个环节失败，拖垮后面的所有环节。盘前、决策、风控、盘中、盘后是一条流水线。如果第一个环节抛出一个错误就让整条流水线停下来，那么“盘前因某个数据源超时而失败”会连带导致“盘后不复盘、不结算”。用户第二天打开软件，会看到账面数据是断的。

代价三：进程直接死掉，什么都没留下。这是最严重的一种，也是我遇到的那种。前两种是“功能退化了”，第三种是“**连退化这个词都无从谈起**”——因为没有任何记录，你无法判断它是功能退化还是根本没跑。

所以“AI 失联了怎么办”这个问题，正确的问法不是一句话，而是五句话：

- 单次调用失败，怎么办？
- 单个阶段失败，怎么办？
- 单个后台任务崩了，怎么办？
- 整个进程崩了，怎么办？
- 那个“决策脑”本身坏了，怎么办？

本章要讲的，就是这五个问题各自的答案。

9.3 一个差不多对的模型：挂了就什么都不做

我先想到的模型最简单，也最安全：**AI 一失联，今天就什么都不做。**

这个模型有它的道理。它的逻辑是：既然决策依赖 AI，那 AI 不在，就不该做决策——总比乱做强。在很多系统里，这是对的。

但它在 A 股里有一个明显的裂缝：“什么都不做”本身也是一个决定，而且是有代价的决定。一个持仓已经触发止损的交易日，你不卖出，就是把风险留着过夜。所以“安全停摆”并不安全——它把“不决策”当成了中立，可市场不会因为你不决策就停止给你计分。

第二个裂缝更隐蔽：这个模型把“AI 失联”当成一个**整体事件**，而它其实是一个**局部事件**。系统里一次决策要用到三次独立的模型调用（主张、异议、辩论），外加晨间简报这类各自独立的任务，它们不会一起挂。多数时候是某一路挂了，另外两路好好的。

把这两条裂缝补上，模型就变成三层：

本章的骨架：退化的三层

第一层，退回规则。AI 给不出意见，就用规则本来就算出的那个数——规则门本身就是一份完整、可执行、可复盘的决定，不是“假数据”。

第二层，局部降级。某一路意见缺席，就少一路；某个阶段失败，就记一笔，其余阶段照跑。缺席是合法的，编造不是。

第三层，进程不灭。后台任务崩溃要留在日志里，致命崩溃要留在磁盘上，那个“决策脑”不允许被中途拆走。

这个模型的失效点在哪里？在于它只保证“系统继续运转”，不保证“系统继续像原来那么聪明”。一个靠规则门跑完的一天，和一个四路来源齐全地跑完的一天，产出质量不是一回事。所以这三层的目标必须写准：它们要保的是“不停工”和“有据可查”，不是“保证水平不下降”。想清楚这一点，才不会在事后把“退化运行”误读成“正常运行”（第 chapter 十 章会讲怎么区分）。

9.4 第一层：单次调用失败，退回规则

先看最小的一格：一次 LLM 调用失败了。

这里的“失败”包括四种：模型没配置、调用报错、超时、以及返回了内容但解析不出结构。四种的处理方式完全一样——**完全回退到规则决策**，一个字的意见都不采纳。源码在这里

写得很直白：

```

1 func (c *CIOEngine) decideProposalGate(ctx context.Context, in proposalInput, ruleRate
    float64) *proposalGateResult {
2     evidence := buildEvidenceLines(in)
3     res := &proposalGateResult{Evidence: evidence, Effective: ruleRate, KellyScale: 1.0}
4
5     prop := c.genDecisionProposal(ctx, in) // Phase1 主张
6     if prop == nil {
7         log.Printf("[CIO] 决策主张不可用，完全回退规则决策（有效仓位=%.2f)", ruleRate)
8         return res // 有效仓位 = ruleRate
9     }
10    ...
11 }

```

代码 9.1 硬回退（节选，internal/cio/proposal.go）

这段话在说什么：注意第二行 `Effective: ruleRate`——在问 AI 之前，答案就已经先填成“规则给的那个数”了。这是一种很聪明的写法：回退不是“出错时才去补的补丁”，而是“默认状态”。AI 的意见如果成功拿到，就把这个默认值改小（后面所有的动作都只能把它改小）；拿不到，默认值原样留下。

`ruleRate` 是从哪来的？它是六维判势给出的仓位系数（第 chapter 十九章）。这里有一个真实的细节值得记下来：六维判势本身也不可用时，`ruleRate` 取 1.0——也就是“不因为 AI 挂了而额外惩罚，也不因为 AI 挂了而额外放行”，让后面的风控层去做它该做的事。

另外三个 LLM 阶段（异议评审、多空辩论，以及主张官本身）各自设了 60 秒的超时。超时之后调用返回失败，于是走的就是上面那条路：返回 `nil`、不改变门控。超时不是一个异常状态，它是一个设计好的出口。

9.5 第二层：一个阶段失败了，其余阶段照跑

现在把视角放大一格：如果失败的不是一次调用，而是一整个阶段呢？

系统的日常投资工作流有五步，固定顺序是：盘前 → CIO 决策 → 风控审查 → 盘中订单 → 盘后。它的写法不是“遇到错误就返回”，而是“把错误收集起来，继续往下走”：

```

1 var phaseErrs []string
2
3 // Step 1: 盘前阶段
4 if err := we.ExecutePreMarketPhase(instance.InstanceID); err != nil {
5     log.Printf("[WorkflowEngine] 盘前阶段失败(继续后续阶段): %v", err)
6     phaseErrs = append(phaseErrs, "盘前:"+err.Error())
7 }
8
9 // Step 2: CIO决策阶段
10 if err := we.ExecuteCIODecisionPhase(instance.InstanceID); err != nil {
11     log.Printf("[WorkflowEngine] CIO决策阶段失败(继续后续阶段): %v", err)
12     phaseErrs = append(phaseErrs, "CIO决策:"+err.Error())

```



```

13     decision = &InvestmentDecision{} // 失败时用空决策占位，避免后续阶段解引用 nil
14 }
15
16 // Step 3: 风控审查阶段
17 if err := we.ExecuteRiskReviewPhase(instance.InstanceID, decision.RelatedTaskIDs); err !=
18     nil {
19     phaseErrs = append(phaseErrs, "风控审查:"+err.Error())
20 }
21 // Step 4-6: 盘中订单 → 盘后，同样各自收集错误，各自继续

```

代码 9.2 单步失败不中断（节选，internal/agentworkflow/workflow_engine.go）

这段话在说什么，三件事。

第一，`phaseErrs` 是一张“欠账清单”。每个阶段失败，就往里记一笔，然后继续。全部跑完之后，如果清单不空，函数返回一个错误，日志里写着“日常投资 workflow 完成（含 N 个阶段失败）”。注意这句话的措辞：它说的是“完成（含失败）”，不是“中止”。

第二，空决策占位这个小动作，救的是下一行。看第二段：如果 CIO 决策阶段失败了，代码不是让 `decision` 保持 `nil`，而是把它赋成一个空的 `InvestmentDecision`。为什么？因为第三段要读 `decision.RelatedTaskIDs`——如果 `decision` 是 `nil`，这一读就是空指针解引用，在网络编程里叫“访问了不存在的东西”，在 Go 里就是一次 panic。一个为“防止下一行崩溃”而存在的空壳对象，是容错设计里最不起眼、也最不可省的一块砖。

第三，这套写法有一个必须说清楚的代价。阶段会带着“上一步没成功”的状态继续跑。所以每个阶段自己必须能处理“输入不全”的情况——比如风控审查拿到的是一张空的 `任务 ID` 列表。容错从来不是免费的，它的账单开在“每个下游都要能接受残缺的输入”上。

调度层还有一条同源的规则：盘后分析的主周期如果出错，每日结算、盘口日频摘要、每日复盘这三件事仍然照常执行（internal/scheduler/auto_scheduler.go 的 `runPostMarketAnalysis`）。理由很实在——盈亏结算和复盘一断，第二天的账面数据就是错的，而这个错误会一路传下去。

9.6 第三层：后台任务崩了，进程还得活着

再放大一格：如果一个后台任务内部 panic 了呢？

在 Go 里，一个 goroutine 里没有 `recover` 的 panic 会直接终止整个进程。这句话的分量要体会一下：你为了不阻塞主流程而开出去的那个后台任务，反过来有能力杀死整个程序。

所以系统的规矩是：后台 goroutine 一律带兜底。工具函数 `SafeGo`（internal/util/goroutine.go）就是干这个的——它启动 goroutine 时先挂上一个 `defer recover()`，崩了就把名字和原因写进日志，然后安静地结束：

```

1 func SafeGo(name string, fn func()) {
2     go func() {
3         defer func() {
4             if r := recover(); r != nil {
5                 log.Printf("[Goroutine] Panic recovered in %s: %v", name, r)
6             }

```

```

7      }()
8      fn()
9      }()
10 }

```

代码 9.3 后台 goroutine 的兜底（节选，`internal/util/goroutine.go`）

但兜底只兜得住“可恢复的”崩溃。这就是我那次闪退的真相所在——它根本不是一次普通的 panic。

9.6.1 那次闪退的根因：DLL 被拆走了

系统的“决策脑”是一个独立的 DLL 文件 `agent.dll`。它是一个 c-shared 的 Go 动态库，独立资产，被主程序加载进同一个进程使用。

问题出在那张“用完就还”的清单上。写代码的时候，我给它的加载器配了一个释放入口 `Close()`，里面调的是 `a.dll.Release()`——像一个规矩的程序员那样，加载了的东西用完就卸。

出事的地方在于：这个 DLL 内部有它自己的后台 `goroutine`。主程序这边把 DLL 卸掉、内存回收，那些还活着的内部 `goroutine` 并不知道自己脚下的代码已经被搬走了。它们继续执行，于是访问了一段已经释放的内存。

Windows 给这种行为起了个名字：`ACCESS_VIOLATION`，错误码 `0xC0000005`。它不是“程序逻辑错了”，它是“执行了一段不属于你的内存”。这种错误不经过 Go 的 `log` 包，也不经过任何一层 `recover`——它是操作系统级别的、立即生效的终止。

于是就有了本章开头那一幕：程序消失、日志干净、事件查看器无记录。

修复方式只有一句话：不再卸载它。让 DLL 在进程的整个生命周期里常驻——加载之后就不再释放。库里的那个 `Close()` 入口还在，但决策脑不去调它。

常见误解·“概率极小的崩溃”，在设计阶段可以忽略 我当时的想法很朴素：这个 DLL 只在决策时用，一天用一次，卸载的时候恰好撞上内部 `goroutine` 在跑，能有多大概率？先把这个“概率”算清楚。先做两个明确的假设。

假设 9.1 (独立同概率的崩溃单元). 设系统每天执行 N 次“有可能触发该缺陷的单位操作”，每次触发的概率都是同一个 p ，且各次相互独立。

在假设 9.1 之下，一年（按 365 天）里至少触发一次的概率是

$$\mathbb{P}(\text{至少一次}) = 1 - (1 - p)^{365N}. \quad (9.1)$$

现在代入两个数。系统的调度主循环是 30 秒一次 `ticker`，一天就是 $N = 2880$ 次（这是一个真实的参数，见 `internal/scheduler/auto_scheduler.go`）。

若单次概率 $p = 10^{-8}$ （一亿分之一），代入式 eq. (9.1) 得一年 1.0%；若 $p = 10^{-6}$ （百万分之一），得一年 65%。

两个数放在一起看：同一个“极小概率”，一亿分之一意味着大约九十五年才碰上一次，而百万分之一则是“大概率今年一定会遇上”。 7×24 运行的系统把次数放大了两千八百多倍—— N 越大， p 有

多小就越不重要。

结论不是“要小心一点”，而是：在这类系统里，“会不会发生”不该是设计依据，“发生了会怎样”才是。判断标准应该换成一条：这个错误一旦发生，会不会让整个进程消失？如果会，它就必须有专门的、不依赖 recover 的记录通道。这才是那次闪退教我的东西。

常见误解·Go 的崩溃都会写进 log，所以日志干净就说明没崩 那次事故里，我最长一段时间被这个想法困住：日志最后一行很正常，所以我一直在查“那个正常的时刻之前发生了什么业务逻辑”，查了很久。

方向从一开始就错了。Go 运行时的致命 panic 不走 log 包，它直接写向标准错误输出；而这个程序是一个 GUI 程序（Wails），它没有给人看的控制台。于是致命崩溃的完整堆栈被写到一片没人接收的地方，然后消失。

修复办法是给这条通道单独接一根管子：在主程序启动的最早阶段，用 debug.SetCrashOutput 把运行时崩溃输出重定向到一个独立文件。

```
1 crashPath := filepath.Join(logDir, "crash_dump.txt")
2 if cf, cerr := os.OpenFile(crashPath, os.O_CREATE|os.O_WRONLY|os.O_TRUNC, 0666); cerr ==
   nil {
3     // SetCrashOutput 持有该文件供运行时写入，运行时负责其生命周期，勿手动 Close
4     debug.SetCrashOutput(cf, debug.CrashOptions{})
5 }
```

代码 9.4 把致命崩溃接出来（节选，main.go）

这段话在说什么：从这一刻起，任何没被 recover 的致命 panic，都会把完整堆栈写进 log/crash_dump.txt。注意代码注释里那句“运行时负责其生命周期，勿手动 Close”——这份文件的生命周期被交给了运行时，因为写它的时刻，程序已经在死了，你没法指望它还能优雅地关闭谁。

这一条的教训可以推得很远：可观测性不是框架附赠的，它是一个必须被专门设计的部件。日志系统在正常工作的时候，你看不出它会不会漏掉什么；只有到崩溃那天，你才知道它到底兜住了哪一层。所以对“最后一层”必须单独安排：一条不经过业务代码、不依赖任何中间件的通路。

常见误解·“退回规则”是一种降级，等于拿假数据凑数 这是对“回退”最常见的误读。很多人一听“AI 失败就退回规则”，第一反应是：那今天的结果是不是半真半假？

不是。要区分两件事：降级使用假数据，和更换决策来源。

规则门给出的仓位系数，是系统用真实行情算出来的、原本就一直在用的数——在引入 AI 之前，系统就是靠它运转的。AI 的主张之所以被加进来，是因为它能在规则之上再收紧一点；它挂掉，只是“收紧的那只手松开了”，规则门本身仍然是一份完整、可执行、可复盘的决策。

真正的红线在另一个地方，而且前面几章反复出现过：不许为了跑完流程而填一个假意见。辩论解析不了，返回 nil，不塞一个“中性”；主张拿不到，退回规则，不编一个方向。第 chapter 八 章那条“缺数据就少投票，而不是补一个假票”的规矩，和“AI 挂了就退回规则，而不是编一个主张”的规矩，是同一条规矩的两种说法。

把上面三层拼起来，就是表 9.1 这张全景表。

表 9.1 五级失效点与各自的兜底方式

失效的层级	系统的行为	真实出处
单次 LLM 调用 (未配置 / 失败 / 超时 / 解析失败)	完全回退规则；生效仓位直接取 <code>ruleRate</code> ；六维不可用时 <code>ruleRate</code> 取 1.0	<code>internal/cio/proposal.go</code>
单个工作流阶段失败	错误记入 <code>phaseErrs</code> ，其余阶段继续；CIO 决策失败时用空决策占位防 nil 解引用	<code>internal/agentworkflow/workflow_engine.go</code>
盘后主周期失败	每日结算、盘口日频摘要、每日复盘仍照常执行	<code>internal/scheduler/auto_scheduler.go</code>
后台 goroutine 内 panic	<code>defer recover()</code> 记下任务名与原因，进程继续	<code>internal/util/goroutine.go</code> 、 <code>main.go</code>
未被 recover 的致命崩溃	堆栈写入 <code>log/crash_dump.txt</code> ，不再静默消失	<code>main.go</code>
决策脑 DLL 被卸载	不再卸载：DLL 在进程整个生命周期内常驻	<code>internal/agentdll/loader_windows.go</code>

表中“超时”指各 LLM 阶段各自的 60 秒调用超时；超时与失败走同一条回退路径。
数据来源：系统源码 `internal/cio/proposal.go`、`internal/agentworkflow/workflow_engine.go`、`internal/scheduler/auto_scheduler.go`、`internal/util/goroutine.go`、`main.go`、`internal/agentdll/loader_windows.go`；事故与修复见本项目真实修复记录；核对时间 2026-09-24。



再往下一层：goroutine 兜底 → 崩溃输出文件 → 进程不灭

图 9.1 五阶段流水线的容错方向：错误向上汇聚成一张清单，而不是向下传导成一次中断。

9.7 真实出处与动手试一试

本章的真实出处

- 五阶段工作流与 `phaseErrs`、空决策占位: `internal/agentworkflow/workflow_engine.go`, 函数 `ExecuteCompleteDailyWorkflow`。
- 硬回退规则决策与 60 秒超时: `internal/cio/proposal.go`, 函数 `decideProposalGate / genDecisionProposal / challengeProposal / debateProposal`。
- 盘后主周期失败仍照常结算与复盘: `internal/scheduler/auto_scheduler.go`, 函数 `runPostMarketAnalysis`。
- 后台 `goroutine` 兜底: `internal/util/goroutine.go` 的 `SafeGo`, 以及 `main.go` 的 `appRecover`。
- 致命崩溃输出重定向: `main.go`, `debug.SetCrashOutput` → `log/crash_dump.txt`。
- 决策脑 `DLL` 的加载与释放入口: `internal/agentdll/loader_windows.go` (`Close()` 调用 `a.dll.Release()`); “不再卸载”是本项目的真实修复决定。
- 调度节拍 (30 秒 `ticker`): `internal/scheduler/auto_scheduler.go`。

动手试一试 (二十分钟)

1. 打开系统安装目录下的 `log/` 文件夹。里面应该至少有两类文件：按日期滚动的运行日志，和一个 `crash_dump.txt`。
2. 先看 `crash_dump.txt`。如果它是空的（长度为 0），说明到目前为止还没有发生过致命崩溃——这是好事，不是 Bug。
3. 然后做一个安全的破坏实验：在设置页里把 **API Key** 填成一个明显无效的值，保存，然后手动触发一次“生成今日决策”之类的动作。
4. 观察两件事：系统有没有报错？以及它有没有停止工作？正确的表现是——决策日志里能查到“决策主张不可用、完全回退规则决策”这类记录，而当天的仓位照常由规则门给出。
5. 最后一问：把 **API Key** 恢复回去之前，先问自己一句——刚才那一天，系统是“用假数据顶上了”，还是“换了个人拍板”？

到这里，第一部分最难的一半已经过去了：这个团队有分工、有权限、有契约、有工具，会吵、会投票，AI 挂了也倒不了。

但它活下来这件事，只证明了它没死——没证明它没瞎干。

亏钱的那一天，我打开系统，能不能查到它当时到底看到了什么、想了什么、谁否了谁？下一章我们去看那个“黑匣子”。

黑匣子：怎么证明这个决策不是瞎猜的？

10.1 亏钱那天，你能查到什么

我先不说“可解释性”这三个字，先请你做一个动作。

想象某一天收盘，你打开账户，发现是亏的。亏得不多，但亏得莫名其妙——因为按你的理解，那天市场看起来还行。

现在请你打开这套系统，回答下面五个问题：

1. 那天它看到了什么？
2. 那天它判断是什么？
3. 那天谁不同意它？不同意在哪里？
4. 那天它本来想下多少注，最后实际下了多少？
5. 那一笔买入，是哪个信号让它下的单？

如果这五个问题里有任何一个你答不上来，那么“这个决策是不是瞎猜的”这个问题，你也就永远答不上来——不是因为它瞎猜了，而是因为你没有留给自己调查的资格。

我用系统自己的真实记录来把这件事说具体。翻开判势记录，二零二六年九月十一日那一天是这样的：六维判势的原始总分 32.14¹，冲突数 5²，修正后总分 25.71³，标签“退潮风险”，建议仓位系数只有 0.06⁴。

这一条记录本身已经能回答第一个问题的一部分了。更进一步，它的 `sources` 字段里逐维度写着数字从哪来——同一天的情绪维度写着“涨停 36 / 跌停 15 / 炸板率 33%（炸板 18）”，技术维度写着“MA20=3936.8 MA60=3944.7”。

也就是说：“那天它看到的是真实数据，而不是编的”这句话，是可以被逐条核对的。这正是本章要讲的整套机制。

直觉·飞行记录仪与手术记录单 飞机上那个橙色盒子，学名叫飞行记录仪。它有两个通道：一个记录仪表读数，一个记录驾驶舱里的对话。

它设计上有一个关键特征：它是被撞不坏、烧不坏的。因为设计者早就知道，它唯一被需要的那一天，正是飞机最惨的那一天——那天别的记录都可能没了，只有它必须留下。

手术记录单是另一个版本的故事。一台手术结束后，医生要写一份记录：术前诊断是什么、切开了哪里、发现了什么、做了什么处理、谁签的字。

这两样东西有一个共同点：它们不是“事后总结”，它们是过程本身的副本。写它们的时刻，就是事

¹真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

²真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

³真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

⁴真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

情正在发生的时刻；一旦写完，它就不再依赖任何人的记忆。

本章讲的，就是这套系统里的橙盒子。

10.2 没有它，会坏在哪

“留痕”这件事，听起来像行政要求，不像技术设计。我一开始也是这么想的，直到我把“不留痕”的代价一条一条列出来。

第一条：你分不清“想错了”和“做错了”。一个决策的链条很长——看到什么、判断什么、谁反对、折算成多少仓位、下多少单。如果只知道结果，那么“亏了”这件事至少对应三种完全不同的病因：判断错了（认知问题）、折算错了（参数问题）、执行错了（工程问题）。三种病因的修法毫不相干。没有证据链，你连去修哪一个都不知道。

第二条：你会不由自主地编理由。这是最危险的一条，而且它发生在你身上，不发生在系统身上。人有一种很强的本能：事后回头看一个结果，会自发地给这个结果配一个通顺的解释，然后把这个解释当成“当时就是这么想的”。假如证据是事后补录的，那这份记录就**天然被这份解释污染过**，它就不再是证据，而是辩护词。

第三条：说不清“这一单凭什么”。买一只股票，可能是策略信号让它买的，也可能是因子模型选的，也可能是 AI 自己偏好的。这三条路的责任归属完全不同。如果订单上不写明来源，事后就只能靠猜。

10.3 一个差不多对的模型：写日志

我最初的做法符合直觉：**把关键动作写进日志文件**。每做一件事就写一行，出错也写一行。

这个模型抓住了最本质的东西——**有记录比没记录强**。它在排查“程序有没有跑”“哪一步报错了”这类问题时非常有效，前面第 chapter 九 章那次闪退的排查，用的就是它。

但作为“决策证据”，它有四个致命的不足。

不足一：它是给机器读的，不是给结论读的。一行日志通常只有一个维度：时间、级别、一句话。而一个决策有十几个维度（方向、信心、反对意见、净多分、仓位折算……）。把这些挤进一行文本里，读的人要自己拿正则表达式去拆。

不足二：它是碎的。证据分散在几十行里，中间还夹着行情刷新、缓存命中之类无关的输出。要回答“谁否了谁”，你得像拼图一样把相关的几行挑出来，还得保证没挑漏。

不足三：它可能被冲掉。日志有滚动策略（这套系统里审计日志默认保留 180 天）。你三个月后想复查一次决策，那条记录可能已经不在。

不足四——也是最要命的一条：它和你什么时候写，是两件事。日志可以在决策之后补写。只要补写的可能存在，“记录被事后解释污染”这个风险就永远在。

补上这四条，模型就变成了本章的严谨版本：**在决策发生的那一瞬间，把证据序列化成结构化的对象，和决策结果写进同一条记录里，并且不依赖你事后去回忆任何东西。**

这个模型失效在哪里？在于它保证不了“证据链是完整的”。它可以保证“已经写下来的那些是真实的、同刻的”，但保证不了“该写的都写了”——如果某个环节忘了往证据里塞一个字

段，那个字段就是空的，而且没人会报警。所以这套机制能挡住的，是“事后编理由”和“事后查不到”；挡不住的，是“当时就没记”。想清楚这一点，你才知道该定期去翻那些字段，看它们是不是常年为空。

10.4 它在系统里长什么样

先说决策记录本身。每一次 CIO 决策，都带一个证据对象，字段名是 `Evidence`，类型叫 `decisionProposalPayload`，定义在 `internal/cio/proposal.go`。表 10.1 是它的字段清单。

表 10.1 决策证据链的字段，以及每个字段负责回答的问题

字段	它回答的问题
<code>market_direction / market_read</code>	它当时怎么看的，一句话解读是什么
<code>suggested_position_rate / effective_position_rate</code>	它想下多少注；最后实际生效多少
<code>governed_by_sixdim</code>	这次是不是被六维判势压下来的
<code>preferred_stocks / confidence / conclusion</code>	它看好谁、有多确信、总结说了什么
<code>skeptic_verdict / skeptic_concerns</code>	谁否了它，否在哪几条具体风险上
<code>debate (方向 / 多 / 空 / 共识度 / 总结)</code>	多空两边各自摆出了什么论据
<code>kelly_position_scale</code>	共识与信心折算成的那个折扣系数
<code>ensemble (方向 / net_score / votes)</code>	四路来源各自怎么表态、净多分多少
<code>feedback</code>	上一次结算之后的真实战绩，作为这次决策的输入之一
<code>evidence[] / generated_at</code>	当时看到的原始输入清单，以及这些结论生成的时刻

数据来源：系统源码 `internal/cio/proposal.go`，结构体 `decisionProposalPayload`；字段随 `CIODecision.Evidence` 持久化；核对时间 2026-09-24。

请注意这张表的形状。它不是一个“结果表”，它是一张对账表：左边一列是决策过程里的每一个角色和每一步折算，右边一列是它们各自的产出。第 chapter 六 章到第 chapter 八 章讲的所有机制——主张、异议、辩论、Kelly 折扣、集成投票——最后都落在这一张表里，一条都不少。

写进去的动作发生在 `attachProposal` 里：它把主张、异议、辩论、Kelly 系数、集成结果，以及盘前采集的那份输入清单，一起塞进 `decision.Evidence`。在这之前，`buildEvidenceLines` 已经先把“当时看到了什么”整理成了一份清单——日期、六维判势摘要、组合快照、回撤等级、风控摘要、因子候选、历史反馈。这份清单在任何判断发生之前就准备好了，这也是它能当证据的原因。

再往上一层，是两条合规防线。它们都只拦买入，这一点很重要，下面会专门讲。

防线一：股票池白名单，而且是“失败即拒绝”。任何自主建仓的买入订单，都要先过一遍可交易股票池 (`tradeable_stocks`)。不在池里的标的，一个不留地剔除：

```
1 allowed := c.tradeablePoolBuyableCodes()
2 if len(allowed) == 0 {
3     log.Println("[CIO] 可交易股票池为空，拒绝基于多因子的自主建仓（白名单强制）")
4     return orders          // 池子空 → 一笔都不下
5 }
6 for _, s := range topStocks {
7     if allowed[normalizePlanCode(s.Code)] {
```



```

8     poolFiltered = append(poolFiltered, s)
9 } else {
10     log.Printf("[CIO] 白名单剔除: %s(%s) 未在可交易股票池, 禁止买入", s.Name, s.Code)
11 }
12 }

```

代码 10.1 白名单的失败即拒绝（节选，`internal/cio/cio.go`）

这段话在说什么：注意第二个 `if`。池子是空的、或者读池子失败了，系统做的事情不是“先按默认买几只”，而是一笔都不下。这种设计有个专门的名字，叫 `fail-closed`（失败即关闭）——拿不准的时候，选择“不动”，而不是选择“按常识猜一个”。

把池子交给程序自己维护，它会在读不到的时候放行；把池子建成白名单，它就会在读不到的时候拦住。这两种选择的区别，就是“一次事故”和“永远不会因为这个原因出事”的区别。

防线二：每一笔买入都要写清信号来源。订单结构 `OrderIntent` 里有一个字段 `signal_source`（`internal/port/agent.go`），每笔买入订单都会被标注它是被哪个信号驱动的，并且这个来源还会被写进订单的理由文本里——形如“因子优选买入某某（代码）某某股 @ 某某（复合得分 = 某某，信号来源 = 某某）”。同一天里的加仓订单，同样带这个字段。

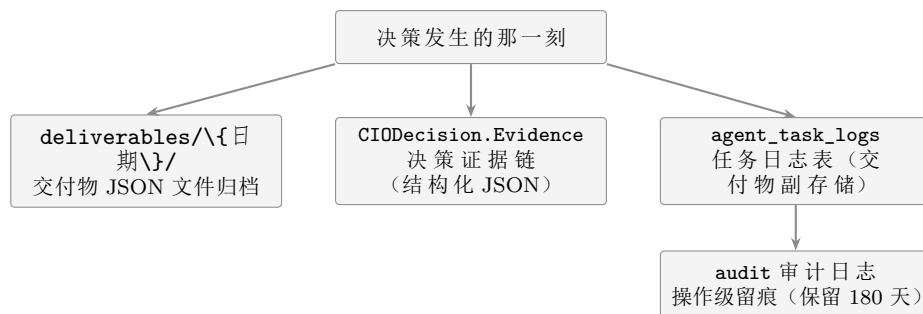


图 10.1 三条落库通道：决策证据链挂在决策记录上，交付物同时进文件与数据库，操作级动作另进审计日志。

接下来是另外三条通道。

交付物双存储。智能体每个任务的产出，同时写两个地方：一个是文件系统 `deliverables/{日期}/`，文件命名形如 `YYYYMMDD_HHMMSS_DELIVERABLE_TYPE_交付物名称.json`；另一个是 SQLite 的 `agent_task_logs` 表，对应字段是 `deliverable_type`、`deliverable_name`、`deliverable_data`。为什么要存两份？文件适合“翻出来看”，数据库适合“查和筛”。一份记录如果只有文件，你三个月后想按类型找它就得靠文件名猜；只有数据库，你又没法把原始 JSON 直接递给别人。

审计日志。系统里有一个统一的审计服务（`internal/audit/service.go`），处理的是操作级的留痕：谁在什么时候做了什么、结果如何。它的工程细节都是围绕“必须记下来”设计的——写入走异步批量（批量上限 100 条、每 5 秒刷一次），不阻塞主流程；写入失败最多重试 3 次；日志默认保留 180 天，每 24 小时清理一次过期记录；重要的操作走同步写入版本，确保落库成功才继续。它按业务场景分了很多入口：选股、投资规划、回测、AI 分析、CIO 每日复盘、配置变更、实时活动等等，并且提供了带筛选和分页的查询接口。

过程透明度。还有一份更细的记录，专门描述“一次会话里发生了什么”：

`internal/transparency/tracker.go` 里的 `SessionTransparency`，包含三个数组——`data_sources`（这一轮读了哪些数据源、成功还是失败）、`algorithm_records`（跑了哪些算法、输入输出各是什么、耗时多少毫秒）、`decision_steps`（第几步、哪个角色、做了什么动作）。它的价值在于把“结论”和“结论的原料”分开存：`Evidence` 存的是结论侧的证据，这份存的是数据与算法侧的流水账。

最后，这些记录是被消费的——不然存了也没意义。`CIO` 页的决策日志（`GetCIOJournal`）会把每条决策渲染成一张卡片，里面包含【多空辩论】区块和【多提案集成】区块。也就是说，第 chapter 六 章那条“看多 3 条、看空 2 条、共识 61”、和第 chapter 八 章那条“净多 -100%、来源主张 = 看多、异议 = RISK、辩论 = 看空、规则 = 看空”，就是从这里读出来的。你怎么读它，和系统怎么记它，是同一件事。

10.5 常见误解与纠偏

常见误解 · 可解释性 = 事后能把理由说圆 这是我想得最久、也错得最久的一条。

我一开始以为，“可解释”指的是：决策做完了，系统能给我一段人能看懂的理由。按这个标准，一套写得漂亮的总结文字就够了。

后来我发现这个标准毫无意义。因为一段“说得圆的理由”，在决策正确时和错误时都一样圆。它是为读者服务的，不是为调查服务的。

真正的标准应该换成这样一句话：**证据必须在决策发生的那一刻就已经存在，而不是在你想看的时候才被生产出来**。用系统里的话说：`buildEvidenceLines` 在任何判断之前运行，`attachProposal` 把结果和证据写进同一条记录，两者之间没有“等一会儿再补”的空间。

为什么这一点不能妥协？因为一旦允许事后补，“证据”就退化成了“叙述”，而人对叙述有一种很强的加工本能——你会不自觉地把结果合理化。第 chapter 八 章讲的那个“净多分 0 到底是没有来源还是激烈对冲”，一旦允许事后补记录，你就会永远得到一个对自己有利的答案。

常见误解 · 合规防线只拦买入、放行卖出，这是双标 看代码很容易产生这个印象：白名单卡的是买入订单，`signal_source` 标的也是买入订单；而卖出那一侧，几乎没有什么东西拦它。这不是“管一半漏一半”吗？

这个不对称是刻意的，而且是一条贯穿全书的纪律：**离场永远比进场更应该被放行**。

理由很朴素。买入是“增加风险敞口”，它的最坏情况是“多了一个你本不该有的仓位”——这个错误可以等一天、可以下不为例。而卖出是“减少风险敞口”，尤其在被止损、被熔断、市场转弱这些情形下，晚一小时都是实打实的损失。如果让白名单去管卖出，那么“这只股票被移出池子”就会变成一个“不能卖出”的理由——你想把风险赶出去，白名单反倒把门锁上了。

同样的不对称在前面的章节里已经出现过两次：第 chapter 十九 章会讲的六维仓位系数只缩放买入订单、不缩放卖出订单；第 chapter 八 章的集成投票只往下压仓位上限、从不往上抬。它们都是同一句话的不同写法：**所有约束的作用方向都必须指向“更少风险”，而不能顺手把出口也堵上**。

常见误解 · 有日志就等于可审计 “系统有日志”和“系统可审计”，是两种东西。

这套系统里，这两者的差距可以被一个具体的问题量出来：**那天谁否了谁？**

日志能告诉你“风控审查通过了”“决策完成了”。但它回答不了上面的问题——因为答案藏在两个具体的字段里：一个是 `skeptic_verdict`（异议审查官的表态是 `AGREE`、`CAUTION` 还是 `RISK`）加上

`skeptic_concerns`（它到底担心哪几条），另一个是 `ensemble` 里的 `votes`（四路来源各自投了什么）。这两个字段如果没被单独存下来，而是被揉进一句“综合各方意见后决定”的总结里，那么这个决策在事后就变成了不可审的——你能证明它做过了，不能证明它想过了。

所以“可审计”的门槛不是“有没有记录”，而是：能不能在不依赖任何回忆和推断的前提下，回答“谁不同意、不同意在哪”。

10.6 真实出处与动手试一试

本章的真实出处

- 决策证据链结构 `decisionProposalPayload` 与写入动作 `attachProposal`: `internal/cio/proposal.go`; 随 `CIODecision.Evidence` 持久化。
- 决策前采集的输入清单 `buildEvidenceLines`: 同文件。
- 股票池白名单与失败即拒绝: `internal/cio/cio.go`, 函数 `generateBuyOrdersFromFactors`, 白名单来源函数 `tradeablePoolBuyableCodes`; 池数据表 `tradeable_stocks`。
- 买入订单的信号来源字段 `signal_source`: `internal/port/agent.go`, 结构 `OrderIntent`。
- 操作级审计服务: `internal/audit/service.go`, 结构 `AuditService` (批量写入、重试、保留期与清理)。
- 过程透明度记录: `internal/transparency/tracker.go`, 结构 `SessionTransparency` 与 `Tracker.StartSession`。
- 交付物双存储: 文件目录 `deliverables/{taskDate\}/` 与 `SQLite` 表 `agent_task_logs` (`deliverable_type` / `deliverable_name` / `deliverable_data`)。
- 前端消费: `CIO` 页决策日志 (`GetCIOJournal`) 与【多空辩论】【多提案集成】区块, `ui/src/pages/CIO.tsx`。
- 真实判势证据样本: `market_sixdim_daily` 表 (含逐维度的 `sources` 字段)。

动手试一试（二十分钟）

1. 打开系统，进入首席投资官（CIO）页，把决策日志往下翻，找一条带完整证据的决策。
2. 在这一条里，按表 10.1 逐个找字段：方向、建议仓位、实际生效仓位、异议表态、辩论共识度、Kelly 系数、集成净多分。
3. 试着用它回答本章开头那五个问题。答不上来的那个，就是你想再要一个字段的的地方。
4. 换到设置 → 审计日志页，查一查这段时间里发生过哪些操作级事件。
5. 最后打开安装目录的 `deliverables/` 文件夹，找一个日期目录，看里面有多少个 JSON——数一数，和你刚刚在决策日志里看到的任务数对不对得上。

10.7 这一部分的收尾：他们到底在算什么

第一部分到这里就结束了。我们现在可以回过头，把第 chapter 一章那个问题再回答一次。那个问题是：一个人，怎么假装自己有一支投资团队？

十章的答案拼起来是这样的：

先分工——把“看市场、找机会、做研究、选股、回测、控风险、管组合、交易、复盘”这九件事，拆成五个岗位（第 chapter 一章）。

再定权——每条权力边界都写进代码里的权限表，尤其把“否决”这一票单独交给一个角

色（第 chapter 二 章）。

然后签合同——每一次派活都有明确的输入、禁止项、产出格式、停止条件，连“我答不上来”都有专门的拒答码（第 chapter 三、chapter 四 章）。

接着排班——用市场自己的时段当节拍器，什么时间谁在岗（第 chapter 五 章）。

让他们吵——把反对意见交给另一个视角产生，而不是让同一个模型自我怀疑（第 chapter 六 章）。

只许往下调——所有角色的意见都只能把仓位压小，不能抬高（第 chapter 七 章）。

有事投票——四路来源折成一个可核对的净多分，缺票就少投，绝不补假票（第 chapter 八 章）。

打不死——AI 失联退回规则、阶段失败不中断、进程不许静默消失（第 chapter 九 章）。

留得下——决策发生的那一刻，把看到了什么、想了什么、谁否了谁，一起写进记录（本章）。

所以答案不是“AI 很聪明”。答案恰恰相反：

第一部分的一句话总结

一个人假装有一支团队的办法，不是找到了五个聪明的 AI，而是把分工、权限、契约、证据、容错和诚实，一条一条写进了代码里。

智能体是这套制度的执行者，不是它的替代品。

团队有了，纪律有了，证据链也有了。

但你现在应该能感觉到，有一个问题我们一直在绕：这套系统里反复出现的那些数字——六维判势的八个维度、因子的权重、形态评分、仓位系数——它们到底是怎么算出来的？

一个“退潮风险”的标签，是怎么从几万只股票、一千多根 K 线、几个外部数据源里被算出来的？凭什么相信它不是凑出来的？

第 chapter 十一 章开始，我们把团队放在一边，去拆开它脑子里那台计算器。第一部分到此结束：我们知道谁来干活了。第二部分要问的是，他们到底在算什么。

第二部分

量化交易原理：把“感觉”变成“可以计算的东西”

什么叫“这只股票不错”？——从感觉到因子

11.1 我的备注栏里，写着三十七个“不错”

二零二六年九月十日，星期四，收盘之后。

那天市场很难看。收盘后系统落库的判势记录里，市场标签是**退潮风险**，建议仓位系数 0.10¹²。

我关掉行情软件，打开自选股。备注栏是我自己加的小格子，平时随手写两个字，方便下次点进来时想起当初为什么把它放进来。

那天我往下翻了一遍，数了数：备注栏里写着“不错”的，有三十七个。

我挑出其中一只，试着把“为什么不错”讲清楚。讲得出完整理由的，不到五只。其余那些，我能说出口的句子长这样：

- “感觉它要动了。”
- “有人在进。”
- “跌不动了。”

这三句话有一个共同点：它们都不可被检验。三个月以后再回来看，我甚至没法判断当初说对了没有——“要动了”没有一把尺子可以对。

这就是我打算从这一章开始解决的事。第二部分整部分的题目只有一个：投资里哪些东西真的可以被计算，哪些不能；以及被计算的那部分，误差从哪来。

11.2 如果你说“不错”，我问你三个问题

那天晚上，我给自己出了三道题。每题都只问一件事，而且都必须用一个数来回答。

第一题：它比同行贵，还是便宜？不是问它的股价高不高，而是问它的价格相对它自己赚的钱、相对它自己的净资产，在同一个行业里排第几。我答不上来——因为我根本不知道同行现在的估值是多少。

第二题：它的利润在增长吗？我有“印象”，但印象来自三个月前看过的某篇公众号。我更该知道的是：最近一期已公告的报表里，营业收入同比是多少，净利润同比是多少，公告日是哪天。这三个数我一个都报不出来。

第三题：它最近是在走强，还是在走弱？这个我以为最不该答错。可我发现我连“最近”是指五个交易日还是二十个交易日都说不清，更别说跟大盘比、跟同板块比。

三题问完，我得到一个不太舒服的结论：

¹真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

²数据来源：系统运行库 `market_sixdim_daily` 的 2026-09-10 记录；核对时间 2026-09-24。

你答不上来，那你说出口的就不是观点，是情绪。

情绪不是坏东西。它是我按下买入键的最后那点推力。但它不能是那个按键的理由——理由必须能拿出来给别人看，也要能在三个月后拿出来给我自己看。

直觉·体检报告 你四十岁，跟医生说“我身体挺好”。医生没跟你争，他开了一张单子：抽血、验尿、心电图、血压、肝功能、肾功能、血脂……二十项。

三天后你拿到报告，其中三项标了红。

你依然感觉“挺好”。可你已经不能再说“我身体挺好”了——你只能说“我有三项偏离了参考值，其中一项偏离得不轻”。

“挺好的”和“有三项偏高”，差别不在准不准，而在能不能被核对。医生不需要相信你的感觉，他只需要看那二十个数字，并且知道每个数字是谁在什么时候测的。

11.3 不把它变成数字，会发生什么

我把“不量化”的代价数了一遍，一共四条。

无法检验。“要动了”这句话永远对。涨了叫“果然”，跌了叫“被大盘拖累”。一句标准可以随时改口径的话，等于没有信息。

无法复制。三个月后我想再做一次同样的判断，可我连当初依据的是哪几个数都找不回来。不能复制的东西，学不会，也改不动。

无法排序。这是最要命的一条。假设自选股里有十只都被我判断为“不错”，我要买几只？买哪几只？资金只有五万，最多买五只。没有一把共同的尺子，我就只能凭“谁最近更让我心动”来排——那又回到了情绪。

无法归咎。一笔亏损发生之后，我不知道该改哪一环。是选股选错了，还是买点错了，还是仓位给大了？如果没有把每个环节拆成可核对的小格，我的每一次“复盘”最后都会变成一句“下次注意”。

四条代价指向同一件事：我需要一个中间层，把“不错”翻译成一组能被别人重复算出来的数。那个中间层，就是因子。

直觉·挑西瓜 一位老瓜农敲一敲就知道甜不甜。你问他凭什么，他说“手感”。

手感是好东西——他自己靠它赚了几十年。但他没法把手感教给学徒，也没法验证自己今天手感有没有失灵。

把“手感”拆开，它就变成几件别人也能测的事：敲击声的频率、瓜蒂的颜色、底部黄斑的面积。这三件事各自都不等于“甜”，但把它们合起来，就能得到一把学徒也能用、也能被检验的尺子。

因子就是被拆开、被称过重量的手感。

11.4 因子，是一种可以被检验的偏见

先说一个“差不多对”的说法：因子就是给每只股票打一个分，分高的排前面。

这句话抓住了七成。它漏掉的二成，恰恰是工程上最容易翻车的地方——这个“分”是从哪里来的。所以下面把它写严谨一点。

定义 11.1 (因子). 给定一个交易日 t 和一个股票集合 $\mathcal{U}$ ，一个因子 f 是把每只股票映射成一个

实数的函数

$$f: \mathcal{U} \rightarrow \mathbb{R}, \quad u \mapsto f(u).$$

$f(u)$ 叫股票 u 在该因子上的**原始值**。同一交易日 t 上所有股票的原始值，构成一个**横截面**。

如果只写到这里，“因子”和“随便给个分”没有区别。要让它真的可用，必须同时满足三个条件。

一是能取到。原始值必须能从可复现的数据源里按公式算出来。取不到就报错，不许用别的数据凑。系统里所有的行情与财务都来自本地行情库 `stock.ohlcv` 与 `stock.financial_report`；选股入口先校验行情库是否挂载，未挂载时直接抛错“行情数据库 (stock.duckdb) 未挂载”，不返回假数据。

二是能对齐。只有在同一个时间点上取到的值，才能放在一起比较。这里最常见的病叫**前视偏差**：拿二零二六年四月才公告的年报，去给二零二五年十二月的股票打分，等于翻到答案再考试。系统的做法是财务数据一律按公告日 `ann_date` 取数，选股时只取“截至当天已经披露”的那一期。

三是能排序。原始值必须能比大小，而且比的方向要写清楚。“波动率”这个因子，数值越大越差；“账面市值比”这个因子，数值越大越好。同一个“越大”，含义相反。方向写反一次，整张榜就倒过来了。

“可计算”的三个条件

能取到（有可复现的来源） $\Rightarrow$ 能对齐（同一时间截面） $\Rightarrow$ 能排序（方向明确）。

三条缺一，得到的都不是因子，是说法。

三个条件都满足了，原始值仍然不能直接相加：市盈率 8 倍和 80 倍之间的距离，跟“动量 3% 和 30%”之间的距离，不是同一种距离。所以要先做**横截面标准化**，也就是常说的 z -score：

$$z_i = \frac{x_i - \mu}{\sigma}, \quad \mu = \frac{1}{n} \sum_{j=1}^n x_j, \quad \sigma = \sqrt{\frac{1}{n} \sum_{j=1}^n (x_j - \mu)^2}, \quad (11.1)$$

这里 $x_1, \dots, x_n$ 是横截面上 n 只股票的因子原始值。

这个符号在讲什么： μ 是这一群股票的平均水平， σ 是它们的分散程度。 z_i 回答的是“这只股票比平均高了几个身位”。贵不贵、强不强，都被换算成了同一个单位——身位。

这个换算有三个我一开始没想到的后果，它们全都属于“误差从哪来”：

其一， z 只在当天这一群股票里有意义。今天算出的 $z = 1.5$ ，是在今天的股票池里比平均高 1.5 个身位；换一个股票池，这个数就变了。它不是这只股票的固有属性。

其二， σ 太小时会爆掉。如果横截面上所有股票的值几乎一样， σ 接近零，除法会把噪声放大成天文数字。系统在 `internal/screener/factor_engine.go` 里的处理是：当 $\sigma < 10^{-10}$ 时强制令 $\sigma = 1$ 。等于承认“这一群股票在这个因子上没有可分辨的差异”，于是谁都拿不到加分。

其三，反向因子必须翻号。 z_i 越大分越高，这对“越小越好”的因子是反的。低估的股票会被排到最后一名去。

到这一步，我终于可以把“不错”翻译成一句可以被核对的话了：

“这只股票不错”的可检验版本

在二零二六年九月十日的全市场截面上，这只股票的价值因子分 78、质量因子分 65、动量因子分 41、低波动因子分 52、盈利稳定因子分 60、流动性因子分 88，按均衡模板加权后总分落在候选池前 5%。

这句话可以被检验、可以被复制、可以被排序，也可以被归咎——如果它错了，我知道该去查哪一项。

11.5 它在系统里长什么样

上面讲的都是“应该怎样”。系统里对应的是两个文件。

第一件事，是把原始值变成分数。`internal/screener/factor_engine.go` 里有两把尺子，一把给正向因子，一把给反向因子：

```
1 // 正向因子：值越大分越高
2 func zScoreTo0100(value, mean, stdDev float64) float64 {
3     z := (value - mean) / stdDev
4     sigmoid := 1.0 / (1.0 + math.Exp(-z))
5     return sigmoid * 100
6 }
7
8 // 反向因子（值越低越好）
9 func inverseZScoreTo0100(value, mean, stdDev float64) float64 {
10    z := (mean - value) / stdDev
11    sigmoid := 1.0 / (1.0 + math.Exp(-z))
12    return sigmoid * 100
13 }
```

代码 11.1 两把尺子：正向与反向（节选自 `internal/screener/factor_engine.go`）

这段话在说什么：注意这里用的不是一条直线，而是一条 *S* 形曲线（logistic 函数）。它把 z 压到 0 到 100 之间，中间陡、两头平。第 12 章我会证明“压到两头平”意味着什么，也会讲清它和“线性截断”的区别。

第二件事，是把分数合成一个总分。`internal/screener/pro_factor_engine.go` 的主流程是这样一行一行加起来的：

```
1 var totalScore float64
2 for factorID, weight := range weights {
3     if fs, ok := factorScores[factorID]; ok {
4         totalScore += fs.Score * weight
5     }
6 }
7 totalScore = roundScore(totalScore)
```

代码 11.2 加权总分（节选自 `internal/screener/pro_factor_engine.go`）

这段话在说什么：总分没有别的玄机，就是把每个因子得分乘上它的权重，再加起来。注意那个 `if ok`——只有这只股票在这个因子上有有效分数时才加。这个不起眼的判断，后面第 12 章会指出它是“不同股票之间可比性被打破”的入口。

第三件事，是规定因子的展示顺序。因子在界面上、在公式字符串里、在日志里，都必须按同一套顺序出现——否则同一份策略，两个人看到的公式会不一样：

```
1 factorOrder := []FactorID{
2     FactorValue,           // value           价值
3     FactorQuality,         // quality        质量
4     FactorMomentum,        // momentum       动量
5     FactorLowVolatility,    // low_volatility 低波动
6     FactorEarningsStability, // earnings_stability 盈利稳定
7     FactorLiquidity,        // liquidity       流动性
8     FactorOrderBook,        // order_book      盘口
9     FactorCointegration,    // cointegration   配对/协整
10 }
```

代码 11.3 固定因子顺序（节选自 `internal/screener/factor_engine.go`）

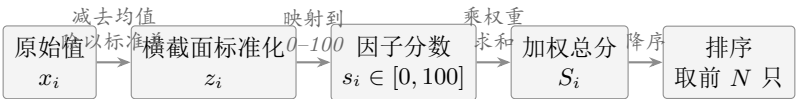
八个因子的清单见表 table 11.1。请注意最后一列——它就是我前面说的“能排序”里那个方向，写错了整张榜会倒过来。

表 11.1 系统里的八个因子（固定顺序）

常量	序列化名	中文	方向（分数越高代表什么）
FactorValue	value	价值	正向：相对更便宜
FactorQuality	quality	质量	正向：更能赚钱、负债更轻
FactorMomentum	momentum	动量	正向：近期走势更强
FactorLowVolatility	low_volatility	低波动	反向：波动越小越好（走反向尺子）
FactorEarningsStability	earnings_stability	盈利稳定	正向：越稳越好
FactorLiquidity	liquidity	流动性	正向：越容易买卖越好
FactorOrderBook	order_book	盘口	正向：买盘越主动越好
FactorCointegration	cointegration	配对协整	反向：相对市场越超卖越有吸引力（走反向尺子）

序列化名即 `FactorQuality` 表中的 `factor_name` 键；固定展示顺序由 `buildFormulaString` 保证。
数据来源：系统源码 `internal/screener/factor_engine.go`、`internal/screener/pro_factor_engine.go`；核
对时间 2026-09-24。

整条链路画出来只有五个格子：



式 eq. (11.1)

图 11.1 从“感觉”到“可排序”的五步。前三步在横向截面上完成，后两步才产生名次。

常见误解 · 把因子当预测魔法 我最早的一个误解是：既然因子能排名次，那选出来的第一名应该就是明天涨得最多的那只。

不是。因子是一个排序工具，它只回答一个问题：在这群股票里，谁更靠前。

“靠前”是一个相对的、统计意义上的说法。它意味着，把历史上所有这样的横截面都拿出来，靠前一组的平均表现好于靠后那一组。它完全不承诺今天这一只、明天这一周会怎样。

理解了这一点，就能接受两件让新手很难受的事：第一，因子选出的票照样会跌，而且经常跌得毫不含糊；第二，判断一个因子好不好，不能用“它上次选的那只涨没涨”，而要用第 14 章要讲的 IC 那类统计量——那是拿几百个横截面一起算出来的东西。

把相对结论当绝对承诺，是量化里最常见的一种自伤。

常见误解·指标越多越准 “那我多加几个因子，是不是就更准了？”——我干过这件事，结果更差了。

原因在于总分是加权求和（清单 代码 11.2）。每加一个因子，就等于在总分里掺进一股新的成分：

- 如果这个因子携带的信息和已有的因子高度重复，它带来的不是新信息，而是权重的偏移——某一类特征被重复计了两次，总分开始偏向那一边；
- 如果这个因子和自己的排名毫不相关，它带来的主要是噪声，会把真正有区分度的那几个因子的信号稀释掉。

所以因子的数量不是越多越好，筛子反而是越干净越好：留下能吃住不同误差来源的那些因子，把重复的、无效的剔除。第 14 章会给出判断“有没有效”的具体指标，第 15 章会给出判断“是不是碰巧有效”的检验。

11.6 本章的真实出处

本章对应的真实代码

- 原始值统计量与 σ 退化保护 ($\sigma < 10^{-10} \Rightarrow \sigma = 1$) : `internal/screener/factor_engine.go` 的 `computeStats`
- 正向 / 反向两把尺子: `internal/screener/factor_engine.go` 的 `zScoreTo0100`、`inverseZScoreTo0100`
- 固定因子顺序与公式字符串: `internal/screener/factor_engine.go` 的 `buildFormulaString`
- 加权总分主流程: `internal/screener/pro_factor_engine.go` 的 `computeWeightedProScores`
- 原始值收集（含财务按公告日取数） : `internal/screener/pro_factor_engine.go` 的 `collectProFactorValues`
- 选股入口与行情库校验: `internal/screener/screener_service.go` 的 `ScreenStock`
- 前端选股页与公式展示: `ui/src/pages/` 下的选股页面，路由表在 `ui/src/App.tsx`

动手试一试（十分钟）

1. 打开系统，进入**选股**页，选上任意一个策略模板，让它在全市场跑一次。
2. 在结果列表里挑一只票，展开它的因子明细，看八个因子的分数。
3. 把界面上显示的公式抄下来。对照表 table 11.1 的顺序，确认它确实按 value 到 order_book 的顺序在展示。
4. 现在自己回答三个问题：这只票为什么排在前面？它在哪一项上赢了？如果它的流动性因子分只有 20 分，它还会出现在这张表里吗？（答案在第 12 章。）
5. 最后做一个思想实验：把这只票的原始值 x 换成全市场最高的一只票的原始值，它的分数会

变成多高？再想一想：如果全市场的 σ 忽然变得很小，会发生什么？（回到本节讲 σ 退化保护那一段。）

到这里，“不错”已经被翻译成了八个 0 到 100 之间的数。但只写定义式是不够的：把 z 变成分数这一步，究竟丢掉了什么信息？把八个分数加权求和，又凭什么说总分高的一定“都不差”？

下一章会把这两件事各写成一个命题，一步一步证给你看，然后再把六个体检科和盘口因子逐个拆开——它是我在系统里第一次遇到“今天几点几分的数据，明天才敢用”的地方。

第十二章

给股票做一次体检：六大因子与盘口

12.1 我的第一版选股，只量了一件事

上一章讲完因子，我做的第一件事是给代码加一个“涨幅榜选股”：谁最近二十天涨得最多，就排在前面。

跑出来的前五名我到现在还记得——三只只是同一个板块里的两融标的，走势几乎重叠。它们同一天涨，也同一天跌。

那一刻我才明白我干了一件很蠢的事：我以为自己买了五只票，其实只下了一个注。这五只票的风险来自同一件事，它们只是同一件事的五种写法。

更麻烦的是，我自己完全看不出来。涨幅榜只量了“最近涨了多少”这一件事，而让它们同时下跌的那个原因，恰好不在这一件事里。

直觉·医院化验单 你去做体检，医生不会只给你量一项。

量体重，看不出肝脏；查肝功能，看不出血脂；做心电图，看不出甲状腺。所以单子上列了二十项，每项都对应一类会出的问题。

注意这张单子的写法：它不承诺“二十项全绿就等于健康”，也不承诺“有一项红就一定生病”。它给的是一张分工明确的清单——“这一项负责盯这个病，那一项负责盯那个病”。

股市里的因子表就是这张化验单。区别只有一个：化验单是按器官分的，因子表是按人性偏差分的。

12.2 六个科室，各防一种病

为什么要六个，而不是一个或者二十个？上一章已经说过“指标多了会稀释信号”。所以这六个不是拍脑袋凑的，每一个都对应一类在真实市场里反复出现的、能被说清楚的偏差。

下面这张表就是六个体检科的门牌。第三列是我最想让你记住的——每一项化验单，防的是哪种病。

六个科室里，价值与质量用的是**真金白银的财报**：价值因子优先取真实财务的 EP（每股收益除以现价）与 BP（每股净资产除以现价），也就是“按赚钱能力和家底算，这个价钱贵不贵”；质量因子取 ROE、毛利率、净利率，并把资产负债率**取负号**加进来。两者都在财务数据缺失时才退回技术代理值。

动量因子取的是 1 月、3 月、6 月三个周期的收益，按 0.2/0.3/0.5 合成——短周期给低权重、长周期给高权重，因为短周期更容易被噪声带跑。

低波动因子走的是反向尺子：波动越小，分越高。盈利稳定因子用的是成交量的变异系数取负号（成交越稳，越像一家“有正经生意”的公司）。流动性因子用近十个交易日的平均成交额，它同时是打分因子，也是下一节的硬门槛。

表 12.1 六个体检科：每个因子试图捕捉哪种人性偏差、防哪种病

因子（科室）	它试图捕捉的偏差	它防的那种病
价值	市场对“故事”反应过度、对“便宜”反应不足，好公司长期被定低价	为故事付出太高的价钱
质量	市场常把“增长快”当成“会赚钱”，忽略了盈利能不能变成现金	把会计利润错当成还款能力
动量	新信息出来以后，市场消化得比想象中慢，趋势会延续一段	抄底抄在半山腰、早早卖飞强势股
低波动	投资者偏爱“彩票型”资产，高波动股票被系统性追捧、低波动被冷落	账户天天坐过山车，拿不住
盈利稳定	业绩忽上忽下的公司更难被预测，市场为这种不确定性打折	重仓踩上业绩爆雷
流动性	小票有流动性溢价，但对小资金来说“买了跑不掉”是致命的	想卖的时候没人接

每个因子的方向与实现见 `internal/screener/pro_factor_engine.go`；“防哪种病”是本书对因子经济含义的归纳，不是源码里的注释。

数据来源：系统源码 `internal/screener/pro_factor_engine.go`、`internal/screener/types.go`；核对时间 2026-09-24。

常见误解·“我买了五只，所以我很分散” 这是我最早也最贵的一个教训。分散不是说“持仓数量大于一”，而是说“持仓的风险来源不止一个”。上一章我用涨幅榜选出来的五只票，持仓数量是五，风险来源是一。判断自己是真分散还是假分散，不需要复杂的模型：把候选股的因子得分表打开，看那一列因子分数是不是齐刷刷朝同一个方向。如果五只票都是动量 90 分、价值 20 分，那你持有的不是五只股票，是一个方向的一倍杠杆。这也是为什么总分的加权是“把不同科室的结论加起来”，而不是“看谁喊得响”——第 13 章会证明这句话的数学含义。

12.3 盘口：唯一一个“今天几点几分”的因子

六个科室看的都是日线级别的历史。第七个因子不一样，它看的是**盘口**——也就是买卖挂单和成交方向。

它想回答的是一个非常具体的问题：此刻这笔资金，是抢着买，还是急着卖？它由两半合成：
一半是量比。量比衡量“今天的成交量是平时的几倍”。放量说明这只票今天有人来了。
一半是内外盘。主动卖出的成交额算**内盘**，主动买入的算**外盘**。内盘占比越低，说明买盘越主动。所以这一半用的是 $(1 - \text{内盘占比})$ 。
两半各占一半，合成一个 0 到 1 的原始值：

$$\text{raw}_{\text{ob}} = \left(0.5 \cdot \tilde{v} + 0.5 \cdot (1 - \rho_{\text{in}}) \right) \cdot (1 - p), \tag{12.1}$$

其中 $\tilde{v} = \frac{\min(\max(\text{量比}, 0), 5)}{5} \in [0, 1]$ 是把量比先截断在 $[0, 5]$ 再归一到 0 到 1， $\rho_{\text{in}} \in [0, 1]$ 是内盘占比， p 是状态罚分。

这个符号在讲什么：前一个括号是“这只票今天有多活跃、买盘有多主动”，取值 0 到 1；

后一个括号是“但今天它的交易状态值不值得信”， p 越大，折得越狠。

罚分 p 是这一节最有意思的地方——它不是按“涨得多还是跌得多”来罚的，而是按“能不能成交”来罚的：

表 12.2 盘口状态罚分 p

盘口状态	罚分 p	为什么这样罚
跌停封死	1.0	卖单压在跌停上排队，根本卖不掉
跌停打开	1.0	跌停被打开意味着随时可能再封回去，同样是“跑不掉”
巨量跌停	1.0	量比 ≥ 2 且触及跌停，放量出逃
涨停封死	0.6	买单排到涨停上，买不进，选出来也成交不了
正常 / 涨停打开	0	能正常成交，不折价

盘口五态的判定在 `internal/orderbook/orderbook.go` 的 `Classify`；封死阈值 `sealThreshold` 取 $\max(1000 \text{ 手}, \text{当日成交量} \times 10\%)$ 。

数据来源：系统源码 `internal/screener/pro_factor_engine.go` 的 `calcOrderBookFactorValue`、`internal/orderbook/orderbook.go`；核对时间 2026-09-24。

注意“跌停封死”罚满（ $p = 1$ ）意味着原始值直接变成零——这只票在盘口因子上拿不到任何分。这是刻意的：一只今天卖不掉的票，无论它多便宜，都不该在“明天要买什么”的名单里排前面。

直觉 · 相亲打分表

相亲角的打分表上有身高、学历、收入、性格。每项都有权重。

有意思的不是打分表，而是那张表背面写着的东西：为什么要有“收入”这一项？因为它防的是“聊得很来但完全没法过日子”；为什么要有“性格”这一项？因为它防的是“条件都好但待在一起很累”。

每一项都不是为了夸人，是为了防一种失败。

因子表也一样。价值防“买贵了”，低波动防“拿不住”，流动性防“跑不掉”，盘口防“买不进也卖不出”。你看到一份因子表，先别问“哪项最重要”，先问“每一项在防什么”。

12.4 从原始值到分数：一个近似模型和它的三个结论

原始值算出来以后，还要变成 0 到 100 的分数。这一步我在上一章给了公式，但没有证明它到底干了什么。现在补上。

为了把性质看清楚，先用一个“差不多对”的模型：把 z 线性放大再截断，也就是

$$s_i = \text{clamp}(50 + 10 z_i, 0, 100), \quad \text{clamp}(u, 0, 100) := \min(\max(u, 0), 100). \tag{12.2}$$

这条规则的直觉是： $z = 0$ （等于截面平均）给 50 分，每高一个标准差加 10 分，每低一个标准差减 10 分，超出 0 到 100 的部分一律切平。

系统真正用的不是这条直线，而是一条 S 形曲线（上一章那两把尺子里的 logistic 函数 $100/(1 + e^{-z})$ ）。这个近似在哪里失效，我下面马上写清楚。先用直线，是因为它的性质能被一眼看穿。

四个结论，先看结论再看证明

- (1) 排序不会被打乱：原始值高的，分数不会更低（命题 theorem 12.1）。
- (2) 但“高一点”不等于“分高一点”：两端会被压平（命题 theorem 12.1）。
- (3) 反向因子只是把整张名次表倒过来，一个数都不用重算（命题 theorem 12.2）。
- (4) 加权总分是“处处不差就一定不差”的（定理 theorem 12.3），前提是权重非负、和为 1。

命题 12.1 (分数映射的单调性与平坦化). 设某个交易日的截面原始值为 $x_1, \dots, x_n \in \mathbb{R}$, $\sigma > 0$, 令 $z_i = (x_i - \mu)/\sigma$, $s_i = \text{clamp}(50 + 10z_i, 0, 100)$ 。则

- (i) 若 $x_i \leq x_j$, 则 $z_i \leq z_j$ 且 $s_i \leq s_j$;
- (ii) 存在截面使 $x_i < x_j$ 而 $s_i = s_j$ (严格序会被压平); 反过来, 若 $z_i < z_j$ 且 $z_i, z_j \in (-5, 5)$, 则 $s_i < s_j$ 。

证明. 先把两条链拆开。 z 是通过“先减常数、再乘正常数”得到的：

$$x \mapsto x - \mu \mapsto \frac{x - \mu}{\sigma}.$$

(i) 第一步是平移，保持大小关系；第二步乘以 $\sigma^{-1} > 0$ ，也保持大小关系（乘以正数不改变不等号方向）。所以 $x_i \leq x_j \Rightarrow z_i \leq z_j$ 。

再看 $z \mapsto s$ 这一步步：

$$z \mapsto 50 + 10z \mapsto \max(50 + 10z, 0) \mapsto \min(\max(50 + 10z, 0), 100).$$

第一个映射 $z \mapsto 50 + 10z$ 是斜率 $10 > 0$ 的直线，非降；第二个 $u \mapsto \max(u, 0)$ 非降（两个非降函数逐点取最大仍非降）；第三个 $v \mapsto \min(v, 100)$ 非降。三个非降函数的复合仍然非降，故 $z_i \leq z_j \Rightarrow s_i \leq s_j$ 。结合上一段，(i) 得证。

(ii) 第二段的映射在两处变成常数： $z \geq 5$ 时 $50 + 10z \geq 100$ ，被 $\min(\cdot, 100)$ 一直压到 100； $z \leq -5$ 时 $50 + 10z \leq 0$ ，被 $\max(\cdot, 0)$ 一直压到 0。在两块平地上，函数不是单射，这是“平坦化”的来源。

给一个具体截面把它坐实。取 $n = 60$ ：其中 58 只股票的原始值是 0，另外两只分别是 100 与 100.5。于是

$$\mu = \frac{58 \cdot 0 + 100 + 100.5}{60} = 3.342 \text{ (保留三位)},$$

$$\sigma = \sqrt{\frac{58 \cdot (0 - 3.342)^2 + (100 - 3.342)^2 + (100.5 - 3.342)^2}{60}} = 17.995.$$

两只“极端票”的 z 分别是

$$z_1 = \frac{100 - 3.342}{17.995} = 5.371, \quad z_2 = \frac{100.5 - 3.342}{17.995} = 5.399.$$

两个都超过 5，所以两个都被压到

$$s_1 = s_2 = 100,$$

尽管原始值 $100 < 100.5$ 。这就是“分数平坦化”：它不是代码写错了，而是 clamp 的语义本身就是“超过上限的一律记成上限”。系统在提示“这只票极其便宜 / 极其强势”时，本来就无法区分“极其”到底有多极其。

（本例是为命题构造的数学反例，不是从数据库里取出的真实截面；构造细节已写在证明里，任何人都能重算。）

最后证充分条件。若 $z_i < z_j$ 且两者都落在 $(-5, 5)$ 内，则 $u_i = 50 + 10z_i$ 与 $u_j = 50 + 10z_j$ 都落在 $(0, 100)$ 内，此时 $\text{clamp}(u_i, 0, 100) = u_i$ 、 $\text{clamp}(u_j, 0, 100) = u_j$ （截断没有生效），于是

$$s_i = u_i = 50 + 10z_i < 50 + 10z_j = u_j = s_j.$$

（端点 $z = \pm 5$ 处截断也恰好取原值 0 或 100，所以条件可放宽为 $z_i, z_j \in [-5, 5]$ 。）

命题 theorem 12.1 说的是：这条尺子不会把好的说成差的，但它会在两端把差距抹掉。现在回到开头的问题——近似在哪里失效。

系统用的是 logistic 映射

$$s_i = \frac{100}{1 + e^{-z_i}}.$$

它严格递增：对 z 求导得 $\frac{100 e^{-z}}{(1 + e^{-z})^2} > 0$ ，恒为正。所以对任意 $z_i < z_j$ 都有 $s_i < s_j$ ——严格序永远不会丢，永远不会出现上面那种“原始值不同、分数完全相同”的情况。它丢的是分辨率：回到刚才那个截面， $z_1 = 5.371$ 与 $z_2 = 5.399$ 对应的分数是 99.537 与 99.550，相差 0.013 分。

一句话总结这两个模型的关系：线性截断把极端样本之间的差距归零，logistic 把差距压到看不出。病是同一个病，只是程度不同。所以下面这条命题和上面那条，对两者都成立——“反向因子必须翻号”这件事，和用哪条曲线无关。

命题 12.2 (反向因子的处理). 设正向因子取 $x'_i = -x_i$ 。则截面均值变为 $-\mu$ 、标准差不变仍为 σ ，标准化值满足 $z'_i = -z_i$ ，分数满足

$$s'_i = \text{clamp}(100 - s_i, 0, 100),$$

且 s' 的名次与 s 的名次完全相反。

证明. 第一步算均值。因为求和是线性的，

$$\mu' = \frac{1}{n} \sum_{i=1}^n x'_i = \frac{1}{n} \sum_{i=1}^n (-x_i) = -\frac{1}{n} \sum_{i=1}^n x_i = -\mu.$$

第二步算标准差。每个离差都只差一个负号，平方以后负号消失：

$$(x'_i - \mu')^2 = (-x_i - (-\mu))^2 = (-(x_i - \mu))^2 = (x_i - \mu)^2,$$

所以 $\sigma' = \sqrt{\frac{1}{n} \sum_i (x_i - \mu)^2} = \sigma$ 。注意结论是“标准差不变”，不是“标准差取负”——标准差是一个距离的平均，永远非负。

第三步算标准化值：

$$z'_i = \frac{x'_i - \mu'}{\sigma'} = \frac{-x_i + \mu}{\sigma} = -\frac{x_i - \mu}{\sigma} = -z_i.$$

到这里可以看清一件事：系统里 `inverseZScoreTo0100` 做的事是 $(\text{mean} - \text{value})/\text{std}$ ，这正是“先把原始值取负、再套正向公式”，与 $z'_i = -z_i$ 是同一件事。

第四步算分数。先用一条关于 `clamp` 的恒等式：

$$100 - \text{clamp}(u, 0, 100) = \text{clamp}(100 - u, 0, 100) \quad (\forall u \in \mathbb{R}). \quad (12.3)$$

分三段验证：若 $u < 0$ ，左边 $= 100 - 0 = 100$ ；右边因为 $100 - u > 100$ 也被压到 100，相等。若 $0 \leq u \leq 100$ ，左边 $= 100 - u \in [0, 100]$ ；右边 $100 - u$ 本来就在 $[0, 100]$ 内，截断不生效，也等于 $100 - u$ ，相等。若 $u > 100$ ，左边 $= 100 - 100 = 0$ ；右边 $100 - u < 0$ 被压到 0，相等。三段合并，(式 eq. (12.3)) 对一切实数 u 成立。

于是，用 eq. (12.3) 把内层的截断搬到外面：

$$\begin{aligned} s'_i &= \text{clamp}(50 + 10z'_i, 0, 100) = \text{clamp}(50 - 10z_i, 0, 100) \\ &= \text{clamp}(100 - (50 + 10z_i), 0, 100) \\ &= 100 - \text{clamp}(50 + 10z_i, 0, 100) = 100 - s_i. \end{aligned}$$

又因为 $s_i \in [0, 100]$ ，所以 $100 - s_i$ 自动落在 $[0, 100]$ 内，外层那个 `clamp` 只是形式上的保险，不会改变数值，故 $s'_i = \text{clamp}(100 - s_i, 0, 100)$ 。

最后证名次相反。若 $s_i < s_j$ ，两边取负号加 100（这是一个严格递减的变换）得

$$s'_i = 100 - s_i > 100 - s_j = s'_j.$$

反过来若 $s_i > s_j$ 则 $s'_i < s'_j$ ；若 $s_i = s_j$ 则 $s'_i = s'_j$ （并列保持并列）。也就是说， $x \mapsto -x$ 这一步把整张名次表**整体倒转**，一个数都不用重算。 ■

这条命题解释了系统为什么必须准备两把尺子。低波动因子的原始值方向是“越小越好”，协整因子的原始值方向也是“越小越好”（相对市场跌得越多，越有均值回归的吸引力）。如果它们误用了正向尺子，名次表就会整个反过来——最好的被排到最后一名，而这不是一个“差一点”的错误，是一个“完全相反”的错误。

定理 12.3 (加权综合评分的排序合理性). 设权重 $w_k \geq 0$ 且 $\sum_k w_k = 1$ ，每个因子分数 $s_{i,k} \in [0, 100]$ ，总分

$$S_i = \sum_k w_k s_{i,k}.$$

则：

(i) $S_i \in [0, 100]$;

(ii) 若 $s_{i,k} \geq s_{j,k}$ 对一切 k 成立，则 $S_i \geq S_j$;

(iii) 若在 (ii) 的前提下还存在某个 k^* 使得 $s_{i,k^*} > s_{j,k^*}$ 且 $w_{k^*} > 0$ ，则 $S_i > S_j$ 。

证明. (i) 下界：每一项 $w_k s_{i,k} \geq 0$ （非负权重乘非负分数），有限个非负数之和非负，故 $S_i \geq 0$ 。

上界：每一项 $w_k s_{i,k} \leq 100 w_k$ （因为 $s_{i,k} \leq 100$ 且 $w_k \geq 0$ ），求和得

$$S_i = \sum_k w_k s_{i,k} \leq \sum_k 100 w_k = 100 \sum_k w_k = 100.$$

所以 $0 \leq S_i \leq 100$ 。

(ii) 逐项作差并求和（求和可以逐项进行，因为都是有限项）：

$$S_i - S_j = \sum_k w_k s_{i,k} - \sum_k w_k s_{j,k} = \sum_k w_k (s_{i,k} - s_{j,k}).$$

在这个和式里，每一项都有 $w_k \geq 0$ 且 $s_{i,k} - s_{j,k} \geq 0$ ，所以每一项都非负；有限个非负数之和非负，故 $S_i - S_j \geq 0$ ，即 $S_i \geq S_j$ 。

(iii) 把和式拆成“第 k^* 项”和“其余项”：

$$S_i - S_j = \underbrace{w_{k^*} (s_{i,k^*} - s_{j,k^*})}_{>0} + \underbrace{\sum_{k \neq k^*} w_k (s_{i,k} - s_{j,k})}_{\geq 0}.$$

第一项严格为正： $w_{k^*} > 0$ 且括号里严格为正，正数乘正数仍为正。第二项按 (ii) 的逐项论证非负。于是 $S_i - S_j > 0$ ，即 $S_i > S_j$ 。 ■

这个定理在说什么：“处处不差”的股票，总分一定不低；“至少在一项上更好、而这一项还有话语权”的股票，总分一定更高。换句话说，加权求和不会平白无故把一只更好的股票排到后面去——这是它比“投票”更值得信任的地方。

不过它有两个前提，缺了任何一个，结论就失效，这属于“误差从哪来”里最容易被忽略的一类：

前提一是权重和为 1。 系统对均衡模板给出的权重之和其实不是 1：六个主因子加起来是 1.00，再加盘口 0.05 与协整 0.04，合计 1.09。此时定理的上界要改成 $100 \times 1.09 = 109$ ，逐项比较的结论一字不改（因为证明里只用到“权重非负”和“求和是线性的”）。这也是为什么归一化缩放（第 13 章的 `scale`）必须整体进行——它是一个正的公共倍数，不影响任何名次。

前提二是每只股票在同一组因子上都有分。 回到上一章那个不起眼的判断 `if ok`：如果因子 k 对股票 i 有分、对股票 j 没有分（数据不足被降级跳过），那么 j 在 k 上的贡献是 0 而不是一个真值。此时“ $s_{i,k} \geq s_{j,k}$ ”这句话本身就没有意义，定理的前提被破坏，两只股票的总分不再可比。这不是理论上的边角情况——盘口因子在样本不足时就是整项不参与的，原因下一

节马上讲。

注 12.4. 系统在 S_i 之外还有一层：把总分除以 10 四舍五入后 clamp 到 $[1, 10]$ ，得到界面上那个“AI 评分”。这层只是展示用的压缩，它单调非降，因此不改变名次；但它会把 100 分和 109 分压成同一个 10 分。

12.5 它在系统里长什么样

三个模板决定了“六个科室谁说话更响”。系统内置三套：**balanced**（均衡配置）、**defensive**（红利价值多头）、**growth**（成长型）。它们的权重是一组写死的常量，具体数值我放到第 13 章的表 table 13.1 里逐格列出——因为那一章要论证“权重才是真正的观点”，放在那里更顺手。这里只说它们的脾气：

均衡配置把六个主因子摊得很平（价值、质量、动量各 0.20，低波动、盈利稳定各 0.15，流动性 0.10），另外给盘口 0.05、协整 0.04。

红利价值多头把 0.75 的权重压在质量、低波动、盈利稳定三项上（0.30/0.25/0.20），流动性给 0.00。

成长型把 0.55 压在动量与质量上（0.30/0.25），价值只留 0.10。

注意“流动性 0.00”这一格：它在防御模板里不参与打分，但不意味着流动性不重要——下一段你会看到，流动性还有一道不靠权重的硬门槛。

当用户带着画像进来时，系统走的是智能组合：`internal/screener/smart_composer.go` 的 `CalculateSmartWeights` 以模板权重为基础，叠加画像、市场状态、因子健康度与历史表现做**保守微调**（单因子调整幅度不超过其基础权重的 15%），最后归一化；如果调整后与模板的相似度过低，还会按 0.6 : 0.4 混回模板，保证“策略还是那个策略”。

接着是两道**硬筛选**。它们和“权重”不是一回事：权重是加减分，硬筛选是直接剔除，不降权、不保留。

1. **流动性硬下限**。`internal/screener/screener_service.go` 的 `applyLiquidityFloor` 直接看流动性因子分：低于 `minLiquidityScore=30` 的一律剔除。代码注释把理由写得很直白：小基金买了以后要能顺利离场，流动性是**拒买门槛**，不是一个“低权重扣分项”。它还有一条兜底——如果过滤后一只都不剩，就退回原结果，宁可放弃硬过滤，也不给用户一个空池。
2. **流通市值区间**。`minFloatCap=20` 亿、`maxFloatCap=200` 亿。流通市值 = 最新收盘价 × 流通股本（缺流通股本时用总股本兜底）；小于 20 亿的微盘弹性太差、波动太剧，大于 200 亿的超大盘弹性太小，两端都剔除。缺股本资料时**放行**——不因为数据缺失误伤正常选股。

最后是盘口因子的两条“自我约束”：

一是样本门槛。取盘口原始值以后，系统先数一遍有效样本：`orderBookSamples ≥ 20` 才让它参与评分（协整因子同理，还要求配对基准可用）。样本不够时，整个因子**不参与**，而不是“参与但分数不可靠”。这就是定理 theorem 12.3 前提二在工程上的直接体现：宁可少一个因子，也

不要一个歪的因子。

二是零前视。盘口摘要由每日收盘后的 `OrderBookDailyProcessor` 上的 `RecordOrderBookDaily` 方法写入 `order_book_daily`，供次日盘口因子使用。也就是说，今天选股用到的盘口，来自上一个交易日的收盘——前视偏差为零。

一道现在就能验证的留白

系统运行库里 `order_book_daily` 当前有 0^a 条记录。这意味着：盘口因子在真实数据上这一版还没有参与过任何一次评分，有效样本数为 0，连 20 的门槛都不到。

所以我不能给你任何一只股票的盘口因子真实得分——那种数字只能靠编。本书的做法是留白：表格里凡是盘口真实值的位置，一律写 “—^b”。这不是漏写，是这一章最重要的一个示范——数据取不到的时候，正确的动作是停下来，而不是填一个看起来合理的数。

^a数据缺失：该系统库中此项无有效记录。本书遵照系统“数据不可用即报错、不降级、不伪造”的原则，此处留白、不填充估计值。

^b数据缺失：该系统库中此项无有效记录。本书遵照系统“数据不可用即报错、不降级、不伪造”的原则，此处留白、不填充估计值。

12.6 常见误解与纠偏

常见误解·多因子是投票，不是加权 “六个因子的意见合起来”——这句话听起来像开会：三项说买、两项说卖，投票多数说了算。

系统里没有投票。总分是加法，加法里真正决定结果的不是“多少个因子同意”，而是“同意的那几个因子各占多重”。

举个极端的例子：假设某只票质量 90 分、价值 20 分，其余都是 50 分。在成长模板下（质量 0.25、价值 0.10），质量贡献 $0.25 \times (90 - 50) = +10$ ，价值贡献 $0.10 \times (20 - 50) = -3$ ，净赚；在均衡模板下（两者都是 0.20），一个 +8 一个 -6，还是赚，但赚得少了。

“几只票说买”不重要，“说话的那个人有几票”才重要。所以在调参界面上，真正需要你盯着看的一列是权重，不是因子数量。第 13 章整章都在讲这件事。

常见误解·样本不够，就先用着 盘口因子只在有效样本 ≥ 20 只时才参与评分。我第一次读到这条规则时觉得浪费：明明已经有 12 只票有盘口数据了，为什么整项都不要？

因为 z 是在截面上算的。样本太少时，估出来的均值 μ 和标准差 σ 本身就是噪声——12 只票算出的“平均量比”和“量比的波动”，跟全市场真实水平可能差得很远。用这样的 μ, σ 去标准化，得到的分数不是“不精确”，而是方向都可能反。

更要紧的是它发生在横截面上：一只票的 μ, σ 用歪了，被影响的不只是它自己，而是同一天所有票的相对名次——它们共享同一套 μ, σ 。少一个因子，损失的是一点信息；多一个歪因子，污染的是整张榜。

12.7 本章的真实出处

本章对应的真实代码

- 盘口因子原始值与状态罚分: `internal/screener/pro_factor_engine.go` 的 `calcOrderBookFactorValue`、`penaltyForState`
- 盘口因子样本门槛 (≥ 20): `internal/screener/pro_factor_engine.go` 的 `computeAllProFactorScores`
- 盘口五态判定与封单阈值: `internal/orderbook/orderbook.go` 的 `Classify`
- 盘口摘要落库(供次日使用): `OrderBookDailyProcessor.RecordOrderBookDaily`, 写表 `order_book_daily`
- 三套策略模板与默认权重: `internal/screener/types.go` 的 `DefaultStrategies`
- 智能组合权重: `internal/screener/smart_composer.go` 的 `CalculateSmartWeights`
- 流动性硬下限: `internal/screener/screener_service.go` 的 `applyLiquidityFloor`
- 流通市值硬区间 (20 亿 - 200 亿): `internal/screener/pro_factor_engine.go` 的 `minFloatCap / maxFloatCap` 与 `computeWeightedProScores`
- 前端选股页 (因子明细、公式、模板切换): `ui/src/pages/` 下的选股页面

动手试一试 (十分钟)

1. 进选股页，同一个股票池分别用均衡配置和成长型跑一遍（注意不要改别的设置）。
2. 把两次结果的公式抄下来贴在一起看：同一批股票，两次的总分不一样，名次也不一样。找出“名次变化最大”的那只，看它的因子明细——它是不是在动量或质量上分数很高、在被降权的因子上分数很低？
3. 打开任意一只票的因子明细，找盘口 (`order_book`) 那一行。它现在有分数吗？如果整列都是空的，回到本章那个留白框，确认一下原因。
4. 自己构造一个截面：五只股票的动量原始值取 0, 0, 0, 0, 100，手算 μ 、 σ ，再按式 eq. (12.2) 算出五只票的分数。做完再想一想：这就是命题 theorem 12.1 里说的那种“被压平”吗？
5. 最后做一次反向检验：把上面五只票的动量原始值全部取负，按命题 theorem 12.2 预测新的名次，然后再手算一遍核对。

六个体检科说完了，硬门槛也说完了。但有一个问题从头到尾我都在绕开：为什么均衡模板给动量 0.20，而成长模板给它 0.30？为什么不是 0.50？这些数字不是从数据里推出来的，它们是三张被写死的表。那系统有没有办法让权重自己动起来？

下一章讲自适应权重——以及它为什么必须给自己装一个 0.5 到 1.5 的笼子。

第十三章

每个因子该占多少分？——权重从哪里来

13.1 我盯着那个 0.20，越看越不顺眼

动量最近很准。

这句话是我在某次因子复盘之后对自己说的。复盘页上，动量那一行的表现明显好于其他几行，其他几个因子看起来平平无奇。

于是我打开了权重配置。均衡模板里，动量的权重是 0.20。我把光标停在那上面，越看越不顺眼：既然它是最近最准的那个，凭什么只给它 20%？

我把小数点后的第一个数字改成了 5。0.20 变成 0.50。

那几天我心里挺舒服的，觉得自己终于“让数据说了话”。

三天后我开始怀疑自己。不是因为亏了钱——恰恰相反，那几天我手上几只票走得不错。我怀疑，是因为我说不出“为什么是 0.50，而不是 0.35 或者 0.65”。我改动的依据只有一句“它最近很准”，而从“很准”到“0.50”之间，没有任何一步是可复算的。

后来我在系统的修复清单里看到一条更早的记录，几乎是我这次犯的错的另一个版本：

症状：因子质量分被极值带偏。

根因：自适应加权用均值做参考基准。

修复：改用中位数 medianFactorQuality。

那一刻我才明白，权重的来源问题有两个层次：第一层是“要不要让权重自己动”；第二层是“动的时候，你拿什么当基准”。我卡在了第二层。

直觉·球队教练怎么排上场时间 一支球队的主教练，不会因为某个球员上一场砍了 30 分，就让他下一场打满 48 分钟。

上场时间是分配出来的资源，它有三条纪律：

第一，状态好可以多打，但有上限。再厉害的球员也有体力极限，打满全场会拖垮防守。

第二，看的是全队的整体水平，不是平均值。如果一个替补突然砍了 60 分，把全队的场均分拉高了一大截，主教练不会因此把其他所有人的上场时间一起砍半——他知道那是一个异常值，不是全队变强了。

第三，每一次调整都要能复盘。“这场为什么让他多打 6 分钟”必须能回答。回答不出来的调整，下次就不知道是该继续还是该撤回。

因子权重和上场时间是同一种东西：都是有限话语权的分配。所以它遵守同样的三条纪律。

13.2 不调整，或者放开调

先说不调整会怎样。

权重是一张写死的表，那表里每一个数都是一次“事前判断”。可市场是事后的：某个因子

会有一段时间特别有效，另一段时间完全失灵。如果权重永远不动，一个已经连续几个月失灵的因子，仍然拿着它那份话语权（均衡模板下可能是 0.20），在每一张榜上投出等量的噪声票。第 12 章那句话在这里要改写一下：不是“多一个歪因子污染整张榜”，而是“一个已经歪掉的因子，每天都在污染整张榜”。

那放开调呢？把权重完全交给“最近谁准”来决定，会得到另一个更糟的结果。

一个因子可能因为某段时间的运气，短期内指标非常漂亮。如果权重随它涨到 0.50、0.70，那整张榜其实已经变成单因子榜了。第 12 章我吃过这个亏——五只票其实是同一个注。权重失控的后果，就是一个人用一套看起来很高级的多因子系统，重新做了一遍单因子赌注。

所以两头都不能去：权重必须会动，但动幅必须有笼子。

本章要回答的两个问题

1. 一个因子凭什么多拿话语权？——靠它的质量分，不是靠感觉。
2. 最多能多拿多少？——用乘数 `mult` 封在 0.5 到 1.5 之间（命题 theorem 13.2）。

13.3 一条“差不多对”的规则，和它的两个漏口

“差不多对”的版本只有一句话：近期质量好的因子多给点权重，质量差的少给点。

这句话的问题在于“多给点”是多少。如果不定量，它就退化成我前面那个 $0.20 \rightarrow 0.50$ 的手改动作。定量以后，系统的规则是

$$\text{mult}_k = \text{clamp}\left(\frac{q_k}{q_{\text{med}}}, 0.5, 1.5\right), \quad w'_k = w_k \cdot \text{mult}_k, \quad (13.1)$$

其中 w_k 是该因子的基础权重， q_k 是它最近一次因子复盘落库的质量分， q_{med} 是所有有效质量分的中位数。

这个符号在讲什么： q_k/q_{med} 回答“这个因子比一屋子因子的中等水平好多少”；外面套的 `clamp` 把答案夹在 0.5 到 1.5 之间——好得再多，也只乘 1.5；差得再多，也只乘 0.5。

这套规则有两个漏口，我在真机上各踩过一次。

漏口一：基准用均值。 q_{med} 这个名字里的 `med` 是后来改上去的，最早写的是均值。用一个爆表的因子把均值抬高，其余所有因子就会被一起压到 0.5 的下限——整个自适应机制退化成了“全体降权”，等于什么都没调。这正是本章开头那条修复记录说的事。

漏口二：只缩放，不再归一。乘数乘完以后，权重之和一般不再是原来的值。比如三个权重各 0.20 的因子，乘数分别是 1.5, 1.5, 1.0，调整后是 0.30, 0.30, 0.20，和为 0.80。如果就这么拿去打分，总分这一天最高只能到 80，第二天又可能到 110——跨天的总分不再可比，第 12 章定理 theorem 12.3 的前提也跟着塌了。所以必须再做一次整体缩放，让权重之和回到原来的基准（下一节的 `scale`）。

13.4 把它写严谨：一个假设、两条命题

假设 13.1 (乘数的取值区间). 设共有 K 个参与调整的因子。对每个 k , 基础权重 $w_k > 0$, 质量分 $q_k > 0$; 基准 $q_{\text{med}} > 0$ 取 $\{q_1, \dots, q_K\}$ 的中位数; 乘数按式 eq. (13.1) 定义为 $\text{mult}_k = \text{clamp}(q_k/q_{\text{med}}, 0.5, 1.5)$, 其中 $\text{clamp}(u, \alpha, \beta) = \min(\max(u, \alpha), \beta)$ 。调整后权重 $w'_k = w_k \text{mult}_k$, 归一化权重 $W_k = w'_k / \sum_j w'_j$ 。

先证最基础的一条：这个笼子确实把乘数关住了，而且笼子里的调整方向是对的。

命题 13.2 (乘数的有界性与单调性). 在假设 theorem 13.1 下：

- (i) $\text{mult}_k \in [0.5, 1.5]$ 对一切 k 成立;
- (ii) 作为质量分 q 的函数 $\text{mult}(q) = \text{clamp}(q/q_{\text{med}}, 0.5, 1.5)$ 在 $q > 0$ 上单调不减;
- (iii) 若 $q_{\text{med}} \rightarrow 0^+$, 则对任意固定的 $q > 0$ 有 $\text{mult}(q) \rightarrow 1.5$; 当 $q_{\text{med}} = 0$ 时该定义不再成立。

证明. (i) $\text{clamp}(u, 0.5, 1.5)$ 的定义就是 $\min(\max(u, 0.5), 1.5)$ 。无论 u 取什么实数, $\max(u, 0.5) \geq 0.5$, 所以 $\text{clamp}(u, 0.5, 1.5) \geq \min(0.5, 1.5) = 0.5$; 同时 $\min(\cdot, 1.5) \leq 1.5$ 。两端合起来, $\text{clamp}(u, 0.5, 1.5) \in [0.5, 1.5]$ 对一切实数 u 成立。取 $u = q_k/q_{\text{med}}$ (由假设 $q_k > 0, q_{\text{med}} > 0$ 知它是正实数, 定义无碍), 即得 (i)。

(ii) 把它拆成两段映射的复合:

$$q \mapsto u(q) = \frac{q}{q_{\text{med}}} \mapsto \text{clamp}(u, 0.5, 1.5).$$

第一段是斜率 $1/q_{\text{med}} > 0$ 的直线, 严格递增; 第二段 clamp 非降 (它依次由“平移/取最大/取最小”这三个非降操作构成, 与第 12 章命题 theorem 12.1 里对 $\text{clamp}(50 + 10z, 0, 100)$ 的论证完全同型)。两个非降映射的复合仍非降, 故 mult 在 $q > 0$ 上单调不减。

顺带把 mult 的三段形状写出来, 因为它就是“封底 / 线性 / 封顶”的来源:

$$\text{mult}(q) = \begin{cases} 0.5, & 0 < q \leq 0.5 q_{\text{med}}, \\ \frac{q}{q_{\text{med}}}, & 0.5 q_{\text{med}} \leq q \leq 1.5 q_{\text{med}}, \\ 1.5, & q \geq 1.5 q_{\text{med}}. \end{cases} \quad (13.2)$$

中间那一段才是“按质量线性说话”的区间: 质量分正好等于中位数时乘数恰好为 1 (保持原权重); 高多少就乘多少, 低多少就乘多少。超出 1.5 倍的部分和低于 0.5 倍的部分被压平——这是刻意的, 理由和上一章一样: 不要把一次极端读数当成长期结论。

(iii) 固定 $q > 0$, 令 $q_{\text{med}} \rightarrow 0^+$, 则 $u(q) = q/q_{\text{med}} \rightarrow +\infty$; 由式 eq. (13.2) 第三段, $\text{mult}(q) \rightarrow 1.5$ 。注意这个极限与 q 无关: 每一个因子的乘数都趋于 1.5。而所有因子乘上同一个正数 1.5, 在归一化之后会完全约掉:

$$W_k = \frac{w_k \cdot 1.5}{\sum_j w_j \cdot 1.5} = \frac{w_k}{\sum_j w_j},$$

也就是回到了基础权重本身。所以 $q_{\text{med}} \rightarrow 0$ 不是“调整变弱”，而是整套自适应机制彻底失效——花了一套机制的成本，换回一张静止的权重表。

当 $q_{\text{med}} = 0$ 时， q_k/q_{med} 在实数意义下未定义（浮点运算里得到 $+\infty$ ，而若同时 $q_k = 0$ 则得到 $0/0$ 的 NaN，任何比较都返回假，clamp 会把 NaN 原样放出去，污染整张权重表）。因此定义域上必须排除 $q_{\text{med}} = 0$ 。

工程上的做法是两条：基准只在一个“有效质量分”的因子集合上求中位数，且要求这个集合非空——medianFactorQuality 在集合为空时返回 0，调用方随即用 $\text{meanQ} > 0$ 做门禁，条件不满足就整支退回基础权重。至于“至少 1 个有效样本”：若集合里只有 1 个因子，中位数就是它自己， $q_k/q_{\text{med}} \equiv 1$ ，所有乘数都是 1，调整不发生，但也不会出错。这是完备性要求，不是效果要求。 ■

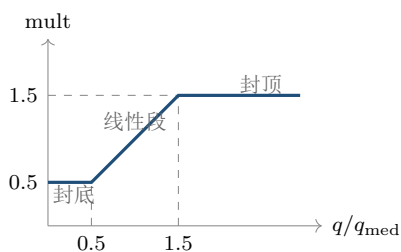


图 13.1 乘数 mult 关于相对质量 q/q_{med} 的形状（式 eq. (13.2)）：封底 0.5、线性、封顶 1.5。

接着证第二条：缩放完、归一化之后，权重仍然是一张“和固定、谁也抹不掉”的表。

定理 13.3 (归一化后权重和恒为 1，且相对话语权被夹在 3 倍以内)。在假设 theorem 13.1 下，归一化权重 $W_k = w'_k / \sum_j w'_j$ 满足：

- (i) $\sum_k W_k = 1$ ，且每个 $W_k > 0$ ；
- (ii) 对任意两个因子 k, l 有 $\frac{1}{3} \cdot \frac{w_k}{w_l} \leq \frac{W_k}{W_l} \leq 3 \cdot \frac{w_k}{w_l}$ ；
- (iii) 对每个 k 有 $\frac{1}{3} \cdot \frac{w_k}{\sum_j w_j} \leq W_k \leq 3 \cdot \frac{w_k}{\sum_j w_j}$ 。

证明. 先检查分母。由假设 $w_j > 0$ 且 $\text{mult}_j \geq 0.5 > 0$ （命题 theorem 13.2(i)），每个 $w'_j = w_j \text{mult}_j > 0$ ；有限个正数之和为正，故 $\sum_j w'_j > 0$ ， W_k 的定义合理。

(i) 求和号里的 k 与分母里的 j 只是哑标，指的是同一个和：

$$\sum_k W_k = \sum_k \frac{w'_k}{\sum_j w'_j} = \frac{\sum_k w'_k}{\sum_j w'_j} = 1.$$

每个 $W_k > 0$ 则直接来自“分子为正、分母为正”。

(ii) 把比值写成两部分：

$$\frac{W_k}{W_l} = \frac{w'_k}{w'_l} = \frac{w_k \text{mult}_k}{w_l \text{mult}_l} = \frac{w_k}{w_l} \cdot \frac{\text{mult}_k}{\text{mult}_l}.$$

由 $\text{mult}_k, \text{mult}_l \in [0.5, 1.5]$ ，中间那个比值被夹住：

$$\frac{\text{mult}_k}{\text{mult}_l} \geq \frac{0.5}{1.5} = \frac{1}{3}, \quad \frac{\text{mult}_k}{\text{mult}_l} \leq \frac{1.5}{0.5} = 3.$$

(分子最小、分母最大给出下界；分子最大、分母最小给出上界。) 两边同乘正数 w_k/w_l ，即得 (ii)。

(iii) 分母满足 $\sum_j w_j \text{mult}_j \leq 1.5 \sum_j w_j$ (每个 $\text{mult}_j \leq 1.5$)，于是

$$W_k = \frac{w_k \text{mult}_k}{\sum_j w_j \text{mult}_j} \geq \frac{0.5 w_k}{1.5 \sum_j w_j} = \frac{1}{3} \cdot \frac{w_k}{\sum_j w_j}.$$

同理 $\sum_j w_j \text{mult}_j \geq 0.5 \sum_j w_j$ ，且 $\text{mult}_k \leq 1.5$ ，于是

$$W_k \leq \frac{1.5 w_k}{0.5 \sum_j w_j} = 3 \cdot \frac{w_k}{\sum_j w_j}.$$

(iii) 得证。 ■

这个定理在说什么：加权这件事本身是“干净的”——它不会在归一化的过程中凭空制造或消灭权重。而且它给了一个很硬的数量级保证：**任何因子的相对话语权，最多被放大或缩小 3 倍**。基础权重是 2:1 的两个因子，调整完最极端也就变成 6:1 或者 2:3；绝不会出现“一个因子吃掉整张表”的情况。第 12 章那个“假分散”的坑，在这里被一道不等式挡住了。

注 13.4. 系统实际的整体缩放不是把权重和缩到 1，而是缩到基础权重和 `baseSum`：`scale = baseSum/adjSum`。这是一个正的公共倍数，同时出现在每个 W_k 的分子和分母里，因此定理 theorem 13.3 的 (ii)、(iii) 一字不改，(i) 变成 $\sum_k W_k = \text{baseSum}$ 。之所以不缩到 1，是为了让“今天的总分”和“昨天的总分”还有可比性。

注 13.5 (诚实下线会把下界再压低). 系统还有一条额外规则：若某因子的分半检验**真实可判定**且判定为不稳定，它的乘数会先封顶到 0.6 再砍半，即 $\text{mult} \leftarrow \min(\text{mult}, 0.6) \times 0.5$ 。由于 $\text{mult} \in [0.5, 1.5]$ ， $\min(\text{mult}, 0.6) \in [0.5, 0.6]$ ，所以这种因子的乘数落在 $[0.25, 0.3]$ 。

此时定理 theorem 13.3 的下界要改成 $0.25/1.5 = 1/6$ ，即 $\frac{1}{6} \cdot \frac{w_k}{w_l} \leq W_k/W_l \leq 3 \cdot \frac{w_k}{w_l}$ 。上界不变——这套机制只把不稳定因子往下压，从不上调，这与第 22 章要讲的“只降不升”是同一种设计习惯。

最后一条是“为什么基准必须用中位数”的定量理由。我先把它写成引理，再用一个具体的小样本算给大家看。

引理 13.6 (中位数与均值对离群值的敏感度). 设 K 个因子的质量分为 $q_1, \dots, q_K$ 。中位数的击穿点为 $\lceil K/2 \rceil / K$ ，均值（平均数）的击穿点为 $1/K$ 。也就是说，把中位数推到任意远需要污染超过一半的样本，而把均值推到任意远只需要污染一个点。

证明. 先看均值。记 $S = \sum_i q_i$ 。把任意一个观测 q_j 改成任意大的 M ，新的均值为

$$\frac{S - q_j + M}{K}.$$

令 $M \rightarrow \infty$ ，上式 $\rightarrow \infty$ ，与 K 无关，也与其他观测无关。也就是说，只要改一个点（占样本的 $1/K$ ），就能把均值推到任意远。因此在 $K \rightarrow \infty$ 时，均值能承受的污染比例为 0。

再看中位数。把 K 个数按大小排好，中位数是第 $\lceil K/2 \rceil$ 小的那个（偶数个时取中间两个的平均，结论不变）。想让中位数变成一个任意大的数，就必须让“第 $\lceil K/2 \rceil$ 小”的那个数任意大；而要做到这一点，至少得有 $\lceil K/2 \rceil$ 个数都被换成任意大的值。若被污染的观测少于 $\lceil K/2 \rceil$ 个，中位数仍落在一批未被污染的观测中间，取值被这些观测夹住，无法被拉远。所以中位数能承受的污染比例是 $\lceil K/2 \rceil / K$ ，在 $K \rightarrow \infty$ 时趋于 50%。

把 $K = 7$ 代进去：中位数要失控需要 $\lceil 7/2 \rceil = 4$ 个点被污染，比例 $4/7 \approx 57\%$ ；而均值只要 1 个点。 ■

具体算一遍。取七个因子的质量分（系统的质量分口径落在 $[0, 1]$ ）：

0.08, 0.09, 0.10, 0.11, 0.12, 0.13, 1.00

最后一个 1.00 是那个“爆表”的因子。中位数是第 4 小的数，等于 0.11；均值是

$$\frac{0.08 + 0.09 + 0.10 + 0.11 + 0.12 + 0.13 + 1.00}{7} = \frac{1.63}{7} = 0.2329.$$

用中位数当基准，七个乘数（按式 eq. (13.1) 计算，保留三位）是

0.727, 0.818, 0.909, 1.000, 1.091, 1.182, 1.500.

六个正常因子全部落在**中间线性段**，各自按自己相对中位数的比例说话；只有那个爆表的因子撞到封顶 1.5。

换成均值当基准，七个乘数变成

0.500, 0.500, 0.500, 0.500, 0.515, 0.558, 1.500.

四个正常因子的乘数被**直接压到 0.5 的下限**——它们之间的质量分差异（0.08 到 0.11）在这张表上被抹平了，因为它们都被同一个被抬高的基准压弯了。这就是本章开头那条修复记录里的“被极值带偏”，一个因子的一次异常读数，改写了其余所有因子的话语权。

结论： q_{med} 用中位数，不是“感觉更稳”，而是“要多污染一半样本才能动它”（引理 theorem 13.6）。

13.5 它在系统里长什么样

自适应权重的实现在 `internal/cio/cio.go`。核心就是上面那几条式子的逐行落地：

```
1 // medianFactorQuality 统计有记录的因子质量分中位数（比均值对极值更稳健），空时返回 0。
2 func medianFactorQuality(m map[string]float64) float64 {
3     if len(m) == 0 {
4         return 0
```



```

5     }
6     arr := make([]float64, 0, len(m))
7     for _, v := range m {
8         arr = append(arr, v)
9     }
10    sort.Float64s(arr)
11    n := len(arr)
12    if n%2 == 1 {
13        return arr[n/2]
14    }
15    return (arr[n/2-1] + arr[n/2]) / 2
16 }

```

代码 13.1 因子质量分的稳健基准（节选自 internal/cio/cio.go）

这段话在说什么：先排序，再取正中间那个；因子数是偶数时取中间两个的平均。注意第一行的空集保护——集合为空返回 0，把“没有基准”这件事用 0 明确地表示出来，让调用方能一眼看出并绕开（而不是偷偷用一个默认值继续算下去）。

```

1  meanQ := medianFactorQuality(qmap) // 基准用中位数，对极值更稳健
2  baseSum := 0.0
3  for _, w := range base { baseSum += w }
4
5  weights := map[string]float64{}
6  adjSum := 0.0
7  for name, w := range base {
8      mult := 1.0
9      if q, ok := qmap[name]; ok && meanQ > 0 && q > 0 {
10         mult = clampF(q/meanQ, 0.5, 1.5)
11         // 诚实下线：仅当分半检验「真实可判定」且判定为不稳定时
12         if h, okH := half[name]; okH && !h.stable && h.ic != "" {
13             mult = math.Min(mult, 0.6)
14             mult *= 0.5
15         }
16     }
17     aw := w * mult
18     weights[name] = aw
19     adjSum += aw
20 }
21 if adjSum > 0 && baseSum > 0 {
22     scale := baseSum / adjSum // 整体缩放：权重和回到基础基准
23     for k, v := range weights { weights[k] = v * scale }
24 }

```

代码 13.2 自适应加权主流程（节选自 internal/cio/cio.go）

这段话在说什么，有三处值得盯住：

第一处，`meanQ > 0 && q > 0`。这个门禁就是命题 theorem 13.2(iii) 的工程形状。质量分

为 0（没有记录）或基准为 0（集合为空）时，mult 保持 1.0——也就是说，“不知道好不好”不会被当成“很差”来罚。这一条很关键：如果让缺失的质量分也走 clamp， $q = 0$ 会得到乘数 0.5，等于因为“没有这条记录”就砍掉这个因子一半的话语权。

第二处，`clampF(q/meanQ, 0.5, 1.5)`。这就是式 eq. (13.1)，也就是图 fig. 13.1 那条折线。

第三处，`scale := baseSum / adjSum`。adjSum 是调整后权重之和，baseSum 是基础权重之和；除以它们的比值，就是把这张表整体搬回到原来的量纲。注意它既不是除以 adjSum 硬缩到 1，也不是不缩——前者会让总分跨天不可比，后者会让权重和漂移。这就是前面“漏口二”的修法。

基准权重从哪来？来自 `internal/factors/engine.go` 的 `GetDefaultFactorWeights(regime)`：按市场状态给出三套基础权重表（BULLISH / BEARISH / 默认），每套 16 个因子，名字是 `momentum_5d`、`momentum_20d`、`reversal_5d`、`ep_ratio`、`bp_ratio`、`roe_score`、`gross_margin_score`、`volatility_20d`、`idiosyncratic_vol`、`turnover_rate_score`、`amihud_illiq`、`log_market_cap`、`dividend_yield_score`、`beta`、`sentiment_score` 这一类。举两个真实数字：BULLISH 下 `momentum_20d` 是 0.10、`log_market_cap` 是 0.10；BEARISH 下 `dividend_yield_score` 是 0.12、`ep_ratio` 是 0.10。

这里要提醒一句，因为很容易混淆：这一套 CIO 侧的因子（16 个细粒度名字）和第 12 章选股引擎的六大因子不是同一套命名。选股引擎用的是 `value`、`quality`、`momentum` 这样的粗粒度名字（表 table 11.1），自适应缩放是在 CIO 那套细粒度名字上做的。两套权重各管一段，改一套不会自动改另一套。第 14 章的因子复盘，产出的就是后一套名字对应的质量分。

至于“三套模板的默认权重”，它们属于选股引擎那一侧，写死在 `internal/screener/types.go` 里：

表 13.1 三套模板的默认因子权重（写死的常量，不是从数据里推出来的）

因子	balanced	defensive	growth	说明
价值 value	0.20	0.15	0.10	成长模板最不在意便宜
质量 quality	0.20	0.30	0.25	三套模板都重视
动量 momentum	0.20	0.10	0.30	成长模板的第一权重
低波动 low_volatility	0.15	0.25	0.10	防御模板的第二权重
盈利稳定 earnings_stability	0.15	0.20	0.15	防御模板的第三权重
流动性 liquidity	0.10	0.00	0.10	防御模板不靠它打分，靠硬门槛
盘口 order_book	0.05	—	—	模板未配置即为 0
协整 cointegration	0.04	—	—	模板未配置即为 0
合计	1.09	1.00	1.00	均衡模板含盘口与协整

“—”表示该模板的权重表里没有这一项，合成总分时不参与；防御模板的流动性权重是 0.00，但流动性硬下限仍会照常执行。

数据来源：系统源码 `internal/screener/types.go` 的 `DefaultStrategies`；核对时间 2026-09-24。

这张表本身就是本章最好的例证：权重不是算出来的，是被人选出来的三套“价值观”。自适应机制是在这三套价值观之上做微调，它不会、也不应该把三套模板调成一个样子——这就是 `ensureStrategyCoherence` 存在的理由（相似度太低时按 0.6 : 0.4 混回模板）。

常见误解·用均值做基准 这是本章唯一一个我真踩过的坑，所以我把它单独拎出来说一次。

用均值做基准，听起来完全合理：“平均水平以上就多加，以下就少加。”问题在于均值会被单个极端值整体抬走，而中位数不会。

引理 theorem 13.6 里的七因子例子把后果算得很清楚：一个爆表的 1.00 把均值抬到 0.2329 之后，七个因子中有四个被试乘数压到 0.5 的下限。也就是说，那次异常的不是那一个因子，而是其余六个因子——它们被一个不是自己的原因削了权。

判断方法很简单：把当天的质量分按大小排一排，算两个数（中位数、均值），比一比。如果均值比中位数高出很多，说明样本里有一两个离群点，这时候用均值做基准就是在让离群点替所有人做决定。

常见误解·权重变了，却没人能说清为什么变 自适应权重有一个隐蔽的坏习惯：它会偷偷地变。

每天夜里因子复盘刷一次质量分，第二天的权重就跟着动一点。几天下来，同一个策略的权重已经和上周完全不同，而界面上如果只显示“均衡配置”四个字，你根本看不出来。

这会造成一个致命的后果：你无法判断收益的变化是模型变好了，还是运气变了。如果权重在跑，回测的对照就不成立——你所谓的“改进”可能只是把历史权重向未来偷看了一眼。

所以权重变化必须满足两条：**可回滚**（能一键回到基础权重）、**可解释**（能说出这次每个乘数是多少、是哪个质量分推的）。系统里这两条都有落点：基础权重来自 `GetDefaultFactorWeights(regime)` 的常量表（随时可回滚）；每个因子的质量分按 `factor_name` 存在 `FactorQuality` 表里，`Detail` 字段记着 `|RankIC|`、准确率、标准差、覆盖率、衰减五个分量（随时可解释——这正是第 14 章的内容）。

改权重之前，先问自己一句：如果明天它没变，我还能复现今天的榜吗？

13.6 本章的真实出处

本章对应的真实代码与表

- 自适应加权主流程与整体缩放： `internal/cio/cio.go` 的 `factorWeightsFor`
- 中位数基准： `internal/cio/cio.go` 的 `medianFactorQuality`
- 区间截断： `internal/cio/cio.go` 的 `clampF`
- 基础权重表（16 因子 × 三种市场状态）： `internal/factors/engine.go` 的 `GetDefaultFactorWeights`
- 三套模板默认权重： `internal/screener/types.go` 的 `DefaultStrategies`
- 模板一致性兜底： `internal/screener/smart_composer.go` 的 `ensureStrategyCoherence`
- 质量分与溯源明细： `internal/tools/factor_review_tool.go` 的 `persistFactorQuality`，落库表 `FactorQuality` (`Detail` 记 `|RankIC|` / 准确率 / 标准差 / 覆盖率 / 衰减)
- 诚实下线的判据（分半检验）： `internal/tools/factor_review_tool.go` 的 `splitHalfStability`

动手试一试（十五分钟）

1. 打开存放质量分的表（`FactorQuality`），把每个因子的 `Quality` 值抄出来，按大小排序。
2. 手算两个数：中位数与均值。比一比它们的差。差得越大，说明你的因子越可能有“爆表选手”。
3. 按式 eq. (13.1) 逐个算乘数：先除以中位数，再夹到 `[0.5, 1.5]`。数一数有几个撞到封顶、几个

撞到封底。

4. 再把上一步的中位数换成均值重算一遍。看看有几个因子被压到 0.5——这就是引理 theorem 13.6 那个例子的现场版。
5. 最后给自己定一条纪律：如果某一项因子看起来“特别准”，你打算给他多少权重？写下这个数，再写下你凭什么定在这个数上。写不出理由的，就把它定回去。

权重解决了“谁说话更响”。可它整套机制都建立在同一个输入上： q_k ，那个因子质量分。

问题是，这个 q_k 是谁算出来的？一个因子说“我这周很准”，我凭什么信？它准到什么程度？覆盖了多少只股票？这个“准”能维持几天，还是一天就衰减完了？

下一章给每个因子发一张体检报告——IC、RankIC、分组收益差、覆盖率、前瞻准确率、稳定性、衰减，一项一项拆开。

第十四章

因子也要写体检报告：IC、RankIC、分组与衰减

14.1 九月十一日，盘后那行正数

二零二六年九月十一日，周五，收盘之后。

那天盘前的六维判势给的是“退潮风险”：调整后总分 25.71，仓位系数 0.06 系统表 `market_sixdim_daily` 的 2026-09-11 记录：`adjusted_total_score = 25.71`，`position_rate = 0.06`，`market_tag = 退潮风险`。¹。系统那天没让我买入任何东西。

同一份盘后复盘里，动量因子的 RankIC 是一个正数。

我盯着这个正数看了很久，然后问了它三句话。

第一句：准到什么程度？它是一个正数，可它是 0.004 还是 0.08？这两个数字对应的含义完全不同：前者是噪声，后者是一年里都难得见到的好因子。

第二句：覆盖了多少股票？如果这个正数是由全市场九千多只股票算出来的，那它是一份普查结果；如果它是由八只股票算出来的，那它是一次抛硬币。

第三句：能维持几天？如果它只在最近五个交易日有效，那我下周就得重新评估它；如果它已经连续二十周为正，那我有理由把它写进权重里。

三句问完，我发现我手里的这个“正数”，什么都没回答。

这不是复盘工具坏了。这是我在用一把只能量长度的尺子，去量一件有三四个维度的事。

所以我决定给每个因子写一份体检报告——不是一行数，是一张表。

14.2 跟着一个天气预报员上班

我想到的是一个很笨的办法：跟着一个天气预报员上一整年的班，每天记录他一件事。

直觉·天气预报员的四栏记录 我在本子上画四栏。

第一栏：他报得准不准。他说明天有雨，第二天真下雨了吗？这一栏记的是方向和幅度——他报“小雨”却下了暴雨，和一个只报“有雨”的人，水平不一样。

第二栏：他敢不敢报。一年三百六十五天，他只报了二十天。那二十天他全准了——可这一年里剩下的三百四十五天，我一点信息都没拿到。准确率再高，也不敢用，因为他覆盖的天气太少。

第三栏：他报得早不早。他是昨晚就报，还是今天早上出门前才报？昨晚就能预报的人，我可以在昨天就决定带不带伞；只能在出门前预报的人，我只能站在门口等。

第四栏：把本子撕成两半，两半的看法一致吗。前半年他排的“最能下雨的日子”和下半年他排的，是同几天吗？如果两半段排出来的东西完全不一样，那他的“第一名”就不是本事，是抽签。

四栏记满，我才敢说这个预报员“行”或者“不行”。

这个笨办法，就是我们马上要写进系统的因子体检。翻成术语，四栏分别是：信息系数（*IC*）、覆盖率（*coverage*）、前瞻准确率（*forward accuracy*）、衰减（*decay*），再加上头的“分组收益

¹真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

差”和收尾的“分半稳定性”。

我第一次做这件事的时候，以为它只是“多算几个数”。后来发现它改变的是我的判断方式：我不再问“这个因子准不准”，我开始问“这个因子在哪种情况下准，在哪种情况下会撒谎”。

14.3 不做体检，要付三种代价

在动手算之前，我得先把“为什么必须算”说清楚。因为如果代价不明显，任何人都会选择省事。

代价一：你会在一个只剩八只股票的因子上押注。

动量因子在退潮期的覆盖率会掉。这不是设计缺陷，这是现实：市场下跌时，能同时满足“有足够长的历史 K 线”“有正常的成交量”“不在涨跌停上封着”的股票本来就少。如果覆盖率掉到百分之几，那么 RankIC 这个数基本上就是这八只股票的排位结果。你把它当成“全市场的规律”，就会在下次调仓时发现它完全不管用。

代价二：你会为一个只在昨天有效的因子付手续费。

这是我最心疼的一笔账。因子准，意味着它每天给出的排序都在变；排序每天变，意味着持仓每天要换。A 股的摩擦不是零：买入收佣金（万三，最低 5 元），卖出收佣金加印花税 0.05%²，另外还有市场冲击（这笔账在第 17 章会精确算，那里会给出平方根冲击律）。

一个每天让你换掉三成持仓的因子，即便它的 RankIC 很漂亮，它挣的钱也可能刚好被这三项摩擦吃完。体检报告里的“衰减”一栏，就是为了提前发现这种事：一个只对 1 天有效的因子，它的收益全落在最短、摩擦最重的那一档上。

代价三：你没法回答“它变差了没有”。

如果不留存历史体检结果，那么“这个因子最近还行吗”这个问题永远只能靠感觉回答。系统的做法是把每次体检按因子名覆盖写入一张表（下面第五节会看到），于是“昨天 0.05，今天 0.02”这件事就变成可查的事实。差异一旦可查，你就能判断到底是因子衰退了，还是你自己的权重调错了。

这三条代价合起来就是一句话：一个因子只有一行数，等于没有报告。

14.4 先给一个“差不多对”的模型

现在动手。我先给自己的直觉版本：因子值和下期收益画成一张散点图，如果点斜着往上排，这个因子就有效。

这句话基本是对的，它抓住了“相关”的本质。但它有两个裂缝，我必须当场指出来，否则后面所有的数都会被它骗。

裂缝一：少数几个点能决定整条斜线。一张散点图上，只要有一个点落在极端位置，它能自己把整条拟合线拉歪，而另外几十个点几乎不参与。

裂缝二：斜线是直的，但现实常常是弯的。如果某只股票“因子值从 60 涨到 70”和“从 90 涨到 100”带来的收益完全不一样（这在动量、情绪类因子中非常常见），那么“一条直线的

²数据来源：交易口径见 `internal/portfolio` 与 `order_cost_tool` 的成本模型；核对时间 2026-09-24。

斜率”就描述不了它。

这两个裂缝分别对应两个不同的指标：Pearson 相关（*Pearson correlation*）和 Spearman 秩相关（*Spearman rank correlation*）。我们要用第二个去补第一个。

14.4.1 IC 与 RankIC：同一张散点图，两种读法

设某一天收盘后，我们对横截面上的 n 只股票各拿到两个数：因子值 x_i ，以及它未来 h 个交易日的收益 y_i （ $i = 1, \dots, n$ ）。

定义 14.1 (IC 与 RankIC). 记 ρ 为 x 与 y 的 Pearson 相关系数， ρ_S 为它们的 Spearman 秩相关系数。本书沿用系统里的叫法：

$$\text{IC} = \rho(x, y) = \frac{\text{Cov}(x, y)}{\sigma_x \sigma_y}, \quad (14.1)$$

$$\text{RankIC} = \rho_S(x, y) = \rho(\text{rank}(x), \text{rank}(y)). \quad (14.2)$$

其中 $\text{rank}(\cdot)$ 把每个值换成它在当天的名次（从 1 到 n ；并列取平均名次）。

这句话在讲什么：IC 直接拿数值去算相关系数，RankIC 先把数值换成名次，再对名次算一次同样的相关系数。两者的差别只在于“用不用数值的大小”。

假设 14.2. 本章所有定理都假设当天的因子值与下期收益都没有并列（名次 $1, \dots, n$ 各不相同）。若存在并列，用平均名次替代，全部结论不变，只是记号变长。

现在来回答第一个真正要紧的问题：这两个数会不会打架？

先看一个不证自明的事实，它是后面一切的起点。

引理 14.3 (保序引理). 设 $g : \mathbb{R} \rightarrow \mathbb{R}$ 是严格单调增函数。若 $y_i = g(x_i)$ ，则对每个 i 都有 $\text{rank}_i(y) = \text{rank}_i(x)$ 。

证明. 严格单调增意味着 $x_i < x_j \iff y_i = g(x_i) < g(x_j) = y_j$ 。所以在把 $\{x_1, \dots, x_n\}$ 从小到大排队时，任意两个样本的先后关系与把 $\{y_1, \dots, y_n\}$ 排队时完全相同；由假设 14.2，两串名次逐位相等。 ■

定理 14.4 (IC 与 RankIC 的关系). 设 (x_i, y_i) , $i = 1, \dots, n$ 为当天截面样本，并满足假设 14.2。

(i) 若存在严格单调增函数 g 使 $y_i = g(x_i)$ 对全部 i 成立，则

$$\text{RankIC} = 1. \quad (14.3)$$

(ii) 秩变换对“严格单调增”不敏感：对任意严格单调增的 g ,

$$\rho_S(x, g(y)) = \rho_S(x, y). \quad (14.4)$$

而对严格单调减的 g , 有 $\rho_S(x, g(y)) = -\rho_S(x, y)$ 。也就是说，**RankIC** 只看得见次序是否一致，看不见数值的大小。

(iii) 因此 IC 与 RankIC 的符号**不必相同**：存在具体样本使 $IC < 0$ 而 $RankIC > 0$ 。

这个定理在说什么：如果把收益换成因子值的“任何一路保序改写”，名次不变，所以 RankIC 一点不动；而 IC 会因为数值被改写而改变，甚至翻号。

证明. (i) 由引理 14.3，两个名次向量逐位相等，即 $r_i^{(x)} = r_i^{(y)}$ 对所有 i 。eq. (14.2) 于是变成“一个向量与自己的 Pearson 相关”：

$$RankIC = \frac{Cov(r^{(x)}, r^{(x)})}{\sigma_{r^{(x)}} \sigma_{r^{(x)}}} = \frac{Var(r^{(x)})}{Var(r^{(x)})} = 1. \quad (14.5)$$

最后一步用了 $\sigma > 0$ （由假设 14.2，名次互不相同，方差严格为正）。

(ii) 对严格单调增的 g ，由引理 14.3（把 x 换成 y 、把 y 换成 $g(y)$ ）得 $rank_i(g(y)) = rank_i(y)$ ，故 eq. (14.2) 右端一点不变，eq. (14.4) 成立。

对严格单调减的 g ： $x_i < x_j \iff g(x_i) > g(x_j)$ ，所以名次被整体反号，即 $rank_i(g(y)) = n + 1 - rank_i(y)$ 。对向量 r 与 $(n + 1)\mathbf{1} - r$ 求相关，常数项 $(n + 1)\mathbf{1}$ 平移不影响协方差，也平移不影响方差，于是

$$\rho_S(x, g(y)) = \frac{Cov(r^{(x)}, -r^{(y)})}{\sigma_{r^{(x)}} \sigma_{r^{(y)}}} = -\rho_S(x, y). \quad (14.6)$$

(iii) 只需给出一个样本。取 $n = 5$ ，因子值与下期收益分别为

$$x = (0, 1, 2, 3, 4), \quad y = (0, 100, 1, 2, 3). \quad (14.7)$$

先算 IC。 x 的均值是 2，偏差为 $(-2, -1, 0, 1, 2)$ ， $\sum_i (x_i - \bar{x})^2 = 10$ 。 y 的均值是 21.2，偏差为 $(-21.2, 78.8, -20.2, -19.2, -18.2)$ ，于是

$$\sum_i (x_i - \bar{x})(y_i - \bar{y}) = 42.4 - 78.8 + 0 - 19.2 - 36.4 = -92.0, \quad (14.8)$$

$$\sum_i (y_i - \bar{y})^2 = 449.44 + 6209.44 + 408.04 + 368.64 + 331.24 = 7766.8. \quad (14.9)$$

所以

$$IC = \frac{-92.0}{\sqrt{10 \times 7766.8}} = \frac{-92.0}{278.69} = -0.330. \quad (14.10)$$

再算 RankIC。名次为 $rank(x) = (1, 2, 3, 4, 5)$ ， $rank(y) = (1, 5, 2, 3, 4)$ 。两串名次的均值都是 3，偏差分别为 $(-2, -1, 0, 1, 2)$ 与 $(-2, 2, -1, 0, 1)$ ，因此

$$\sum_i (rank_i(x) - 3)(rank_i(y) - 3) = 4 - 2 + 0 + 0 + 2 = 4, \quad (14.11)$$

$$\sum_i (rank_i(y) - 3)^2 = 4 + 4 + 1 + 0 + 1 = 10, \quad \sum_i (rank_i(x) - 3)^2 = 10, \quad (14.12)$$

$$\text{RankIC} = \frac{4}{\sqrt{10 \times 10}} = 0.400. \quad (14.13)$$

于是同一个截面样本， $\text{IC} = -0.330 < 0$ ，而 $\text{RankIC} = 0.400 > 0$ 。符号相反，eq. (14.3) 与 eq. (14.4) 的结论同时得到了实例检验。 ■

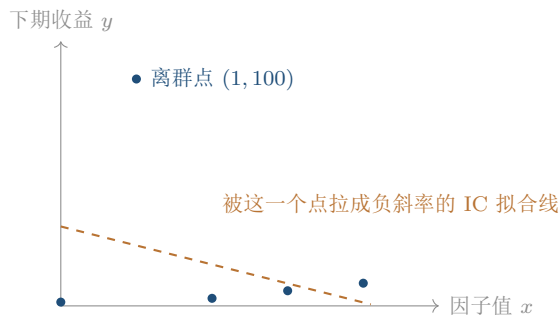


图 14.1 eq. (14.7) 的五个点。四个点（除 $x = 1$ 那一个）次序与收益一致，名次相关为正；但 (1, 100) 这一个点的杠杆把 Pearson 相关拉成负数。

注 14.5 (常识里那句“IC 和 RankIC 差不多”什么时候成立). 定理 14.4 说的是“一般情形下两者可以翻号”。那么在什么情形下它们确实差不多？

经典结果给出：当 (x, y) 服从二元正态、相关系数为 ρ 时，

$$\rho_S = \frac{6}{\pi} \arcsin\left(\frac{\rho}{2}\right). \quad (14.14)$$

把 $\arcsin u = u + \frac{u^3}{6} + O(u^5)$ 代入 eq. (14.14) ($u = \rho/2$):

$$\rho_S = \frac{6}{\pi} \left(\frac{\rho}{2} + \frac{\rho^3}{48} + O(\rho^5) \right) = \frac{3\rho}{\pi} \left(1 + \frac{\rho^2}{24} + O(\rho^4) \right). \quad (14.15)$$

因为 $3/\pi = 0.9549$ ，所以当 $|\rho|$ 很小时（截面因子的真实量级正是这个区间），

$$\rho_S = 0.9549 \rho + O(\rho^3), \quad (14.16)$$

两者相差不到百分之五；而 eq. (14.14) 右端关于 ρ 严格单调增，符号必然一致。这才解释了为什么在“分布规矩”的市场里 IC 与 RankIC 从不打架，而在 eq. (14.7) 这种“有离群点”的截面里会当场翻号。

本书只在弱信号区间使用 eq. (14.15) 的展开形式，不依赖 eq. (14.14) 的精确恒等式。

14.4.2 分组收益差：把一个相关系数翻译成钱

体检报告的第二栏我想写成钱的语言。因为“相关系数”这个词对人不友好，而“头尾两组的收益差”是任何人一看就懂的。

做法很土：把当天所有股票按因子值从大到小排，取最前面一批（top 组）和最后面一批（bottom 组），分别算它们的平均下期收益，两者相减。

系统里选的是三等分——最上三分之一与最下三分之一³。

那么问题来了：分组收益差和 IC 是不是同一件事的两种说法？在“分布规矩”的前提下，答案是肯定的，而且比例系数可以算出来。

假设 14.6. 截面上的因子值 x 与下期收益 y 服从二元正态分布： $x \sim N(\mu_x, \sigma_x^2)$, $y \sim N(\mu_y, \sigma_y^2)$, 相关系数为 ρ 。 $\sigma_x > 0$, $\sigma_y > 0$ 。

引理 14.7 (截断正态的一阶矩). 设 $Z \sim N(0, 1)$, 其密度函数记为 φ , 分布函数记为 Φ ($\varphi'(z) = -z\varphi(z)$)。对任意实数 c :

$$\mathbb{E}[Z \mid Z > c] = \frac{\varphi(c)}{1 - \Phi(c)}, \quad \mathbb{E}[Z \mid Z < c] = -\frac{\varphi(c)}{\Phi(c)}. \quad (14.17)$$

证明. 以第一式为例。条件密度的定义与全概率公式给出

$$\mathbb{E}[Z \mid Z > c] = \frac{1}{\mathbb{P}(Z > c)} \int_c^\infty z \varphi(z) dz. \quad (14.18)$$

被积函数是一个完全微分：由 $\varphi'(z) = -z\varphi(z)$ 得 $z\varphi(z) = -\varphi'(z)$, 于是

$$\int_c^\infty z \varphi(z) dz = \left[-\varphi(z) \right]_c^\infty = -\varphi(\infty) + \varphi(c) = \varphi(c), \quad (14.19)$$

其中用了 $\varphi(z) \rightarrow 0$ ($z \rightarrow \infty$, 因为密度含因子 $e^{-z^2/2}$)。而 $\mathbb{P}(Z > c) = 1 - \Phi(c)$, 代回即得第一式。

第二式同理： $\int_{-\infty}^c z \varphi(z) dz = [-\varphi(z)]_{-\infty}^c = -\varphi(c)$, 除以 $\mathbb{P}(Z < c) = \Phi(c)$ 得到 $-\varphi(c)/\Phi(c)$ 。 ■

定理 14.8 (正态假设下分组收益差与 IC 成比例). 在假设 14.6 下, 取 $q \in (0, 1)$, 把 x 高于其 $1 - q$ 分位点的样本称为 *top* 组、低于其 q 分位点的样本称为 *bottom* 组。记

$$\Delta = \mathbb{E}[y \mid x > z_{1-q}] - \mathbb{E}[y \mid x < z_q], \quad (14.20)$$

其中 z_{1-q} 、 z_q 分别为 x 的 $1 - q$ 分位点与 q 分位点。则

$$\Delta = \rho \sigma_y \cdot \frac{2\varphi(z_{1-q}^*)}{q}, \quad z_{1-q}^* = \frac{z_{1-q} - \mu_x}{\sigma_x}. \quad (14.21)$$

特别地, Δ 与 ρ 同号, 且当 q 与 σ_y 固定时 Δ 与 ρ 成正比例。

这个定理在说什么：分组收益差不是另一个“独立指标”，它就是 IC 换了一身衣服——两者只差一个由分组比例 q 和收益波动 σ_y 决定的常数。

³数据来源：系统源码 `internal/tools/factor_review_tool.go` 的 `groupSpread: n/3` 分组，返回 `top_group_ret`、`bottom_group_ret`、`long_short_spread`。核对时间 2026-09-24。

证明. 第一步，写出一元条件期望。由多元正态的条件分布公式（对二元正态，给定 x 时 y 的条件分布仍为正态，其均值等于最优线性预测），

$$\mathbb{E}[y | x] = \mu_y + \frac{\text{Cov}(x, y)}{\text{Var}(x)} (x - \mu_x) = \mu_y + \rho \frac{\sigma_y}{\sigma_x} (x - \mu_x). \quad (14.22)$$

第二步，把 top 组的上界换成标准正态刻度。令 $a = z_{1-q}$ ， $\alpha = (a - \mu_x)/\sigma_x$ 。由分位点定义， $\mathbb{P}(x > a) = q$ ，而 $\mathbb{P}(Z > \alpha) = 1 - \Phi(\alpha) = q$ 也正是 $Z > \alpha$ 的概率；把 x 标准化即得 $\alpha = z_{1-q}^*$ 。

第三步，算 top 组的条件期望。对 eq. (14.22) 两边在事件 $\{x > a\}$ 下取条件期望（期望的线性性），再把 x 标准化为 $(x - \mu_x)/\sigma_x$ ，它在这个条件下正是“截断在 α 右侧”的标准正态：

$$\begin{aligned} \mathbb{E}[y | x > a] &= \mu_y + \rho \frac{\sigma_y}{\sigma_x} (\mathbb{E}[x | x > a] - \mu_x) \\ &= \mu_y + \rho \sigma_y \mathbb{E}\left[\frac{x - \mu_x}{\sigma_x} \mid x > a\right] \\ &= \mu_y + \rho \sigma_y \mathbb{E}[Z | Z > \alpha] = \mu_y + \rho \sigma_y \frac{\varphi(\alpha)}{1 - \Phi(\alpha)} = \mu_y + \rho \sigma_y \frac{\varphi(z_{1-q}^*)}{q}, \end{aligned} \quad (14.23)$$

其中倒数第二步用了引理 14.7 第一式，最后一步用了 $1 - \Phi(\alpha) = q$ 。

第四步，算 bottom 组的条件期望。同理令 $b = z_q$ ， $\beta = (b - \mu_x)/\sigma_x$ ，则 $\mathbb{P}(x < b) = q$ 且 $\Phi(\beta) = q$ ，于是

$$\begin{aligned} \mathbb{E}[y | x < b] &= \mu_y + \rho \sigma_y \mathbb{E}[Z | Z < \beta] \\ &= \mu_y - \rho \sigma_y \frac{\varphi(\beta)}{\Phi(\beta)} = \mu_y - \rho \sigma_y \frac{\varphi(z_q^*)}{q}. \end{aligned} \quad (14.24)$$

第五步，相减。由标准正态的对称性 $\varphi(u) = \varphi(-u)$ ，以及分位点的对称性 $z_q = -z_{1-q}$ （两者的标准化刻度互为相反数），有 $\varphi(z_q^*) = \varphi(z_{1-q}^*)$ 。于是

$$\begin{aligned} \Delta &= \mathbb{E}[y | x > a] - \mathbb{E}[y | x < b] \\ &= \rho \sigma_y \frac{\varphi(z_{1-q}^*)}{q} + \rho \sigma_y \frac{\varphi(z_q^*)}{q} = \rho \sigma_y \cdot \frac{2 \varphi(z_{1-q}^*)}{q}, \end{aligned} \quad (14.25)$$

即 eq. (14.21)。

第六步，符号与比例。由假设 14.6， $\sigma_y > 0$ ； $\varphi(\cdot) > 0$ 恒成立； $q \in (0, 1)$ 故 $1/q > 0$ 。因此 eq. (14.25) 右端的 ρ 之外的全部因子都是与 ρ 无关的严格正数，故 $\text{sign}(\Delta) = \text{sign}(\rho)$ ，且 Δ/ρ 与 ρ 无关。定理证完。 ■

推论 14.9 (系统实际使用的三等分). 取 $q = 1/3$ ，则 $z_{1-q}^* = z_{2/3} = 0.4307$ ，

$$\varphi(0.4307) = \frac{1}{\sqrt{2\pi}} e^{-0.4307^2/2} = 0.39894 \times e^{-0.09275} = 0.39894 \times 0.91142 = 0.36358, \quad (14.26)$$

$$\Delta = \rho \sigma_y \cdot \frac{2 \times 0.36358}{1/3} = 6 \times 0.36358 \times \rho \sigma_y = 2.1815 \rho \sigma_y. \quad (14.27)$$

这段话在说什么：在“收益服从正态”的前提下，头尾各三分之一组，收益差就是 2.18 倍的“IC 乘收益标准差”。分组收益差和 IC 是同一件事的两种刻度——当分组比例改成 $q = 0.1$ 时，这个系数会跳到 $2\varphi(z_{0.9})/0.1 = 2 \times 0.17550/0.1 = 0.3510/0.1 = 3.510$ ，也就是说“分得更极端”，同样的 IC 会显得更赚钱。

这正是我在系统里看到的“分组收益差比 IC 好看”的原因，它不是发现了新东西，它是把同一件事换了个放大倍数来说。

14.4.3 这个等价性在哪里失效

必须把话说透：theorem 14.8 的比例性完全建立在假设 14.6（二元正态）上。现实中它至少在三个地方失效。

失效一：胖尾。A 股的日收益分布比正态厚得多（涨停、一字板、事件驱动跳空）， φ 的尾部密度不再是准确的刻度。此时 $\text{sign}(\Delta) = \text{sign}(\rho)$ 仍然成立（因为 y 对 x 的平均依赖若为单调，头尾组的均值差方向不会反），但 Δ/ρ 这个常数会随 ρ 、 q 一起漂移，“2.18 倍”这个换算不再可靠。

失效二：非线性但单调。若 $y = g(x) + \varepsilon$ 且 g 严格单调增、 ε 与 x 独立、 $\text{Var}(\varepsilon) > 0$ ，则由定理 14.4(ii) 的思想，名次一致意味着 RankIC 会很高，而 IC 被 g 的非线性与噪声 ε 压下来。于是“IC 小但分组收益差大”完全可能，比例性被破坏。**符号仍然一致**——这是本书要保留的那一半结论。

失效三：用样本均值估计头尾。 Δ 是用有限样本的头尾均值估出来的，它的方差随 q 变小而急剧变大（ $q = 1/3$ 时每组约占 $n/3$ 个样本； $q = 0.1$ 时每组只剩 $n/10$ ）。定性地看，eq. (14.21) 里的 $1/q$ 是“放大系数”，而估计误差并不会被同样地放大，因此极端分组会给出一根噪声极大的曲线。

常见误解·把分组收益差当成独立于 IC 的第二个证据 我在第一版复盘里同时列了“RankIC = 0.03”和“头尾差 = 2.1%”两行，并且把它们当成两个互相印证的证据：一个说相关性还行，一个说确实赚得到钱。

读完上面这一节我才明白：在正态前提下它们是同一个数。用同一个数的两种刻度当作两个证据，是经典的重复计数。

正确的用法是把分组收益差当作“可读性”工具，而不是“独立性”工具：它帮你把相关系数翻译成成人能感觉到的钱，但它不能给你额外的置信度。要额外的置信度，得靠第 15 章的分半检验。

14.4.4 剩下四项：覆盖率、准确率、稳定性、衰减

体检报告剩下的四栏，都能在三行以内说清。

覆盖率。当天真正参与了这次复盘计算的股票数，除以当天的股票池总数：

$$\text{coverage} = \frac{n}{N_{\text{universe}}} \times 100\%. \quad (14.28)$$

系统里 N_{universe} 取的就是这次复盘解析出来的股票池大小⁴。分母取 $\max(\cdot, 1)$ 是为了防止除零

⁴数据来源：系统源码 `internal/tools/factor_review_tool.go`, `buildFactorItem` 中的 `coverage :=`

——这是一个不起眼但必要的工程细节。

前瞻准确率。在按因子值排出的前三分之一里，下期收益为正的比例：

$$\text{acc} = \frac{\#\{i \in \text{top}_{1/3} : y_i > 0\}}{\#\text{top}_{1/3}}. \quad (14.29)$$

这一栏是“你能不能在赌桌上赢钱”的最直白版本：它不问你赢多少，只问你赢的那次比输的那次多不多。

稳定性（区分度）。系统里这一栏算的是因子得分自身的横截面标准差 std^5 ——越大说明这个因子把股票“分得越开”。

这里我必须诚实写下一件事：这一栏的名字叫“稳定性”，但它量的其实是“区分度”。第一次读到这个名字时我以为它量的是“IC 的时间序列标准差”（那才是统计意义上的稳定性，而这件事由下一章的分半检验承担）。两者混起来会误判：一个因子完全可以是“每天得分都分得很开、但每天的排序都不同”，这种因子在 `stability_std` 上很好看，在实际选股上却是噪声。所以读这一栏时，脑子里要把它读成“区分度”，并且知道真正的稳定性另有其人。

衰减。同一批样本，分别按 1 日与 20 日前瞻收益算两次 RankIC，取差：

$$\text{decay} = \text{RankIC}_{20d} - \text{RankIC}_{1d}. \quad (14.30)$$

这个差的符号很要紧：为负，说明这个因子在 1 日尺度上更有效、拉到 20 日就衰减了；为正，说明它是个慢因子，短周期里是噪声，把窗口拉长才显出来。系统注释里明确写着“负值表示短周期有效、长周期衰减”⁶。

直觉·三栏合起来才有信息 单独看“衰减 = -0.03”没有意义。合起来看才有意义：如果同时“覆盖率 12%”、`stability_std` 很高、衰减为负，那这幅画很可能是——少数几只高波动小票在短周期里疯了一把，把 1 日 `RankIC` 抬了起来。

体检报告的价值不在任何一栏，在于栏与栏之间能不能互相解释。一份所有栏都很好看的报告，反而要更小心。

14.5 它在系统里长什么样

纸上讲完了，现在拆机器。

因子复盘在系统里是一个工具：`factor_review`，实现在目录 `internal/tools/factor_review_tool.go` 里，类名 `FactorReviewTool`。它由 `NewFactorReviewTool` 构造，需要四样东西：行情库管理器、SQLite 管理器、可交易池、以及一个共享的情绪引擎⁷。

它的位置在盘前 workflow 里。第 5 章讲过盘前七个任务的顺序，其中第二个就是量化分析师的因子健康度检查（`FACTOR_HEALTH`）。复盘结果再往两个方向走：一个方向是当天的权重（由

`float64(n) / float64(maxInt(totalUniverse, 1)) * 100`。核对时间 2026-09-24。

⁵数据来源：系统源码 `internal/tools/factor_review_tool.go` 的 `scoreStd`：对当日因子得分求标准差。核对时间 2026-09-24。

⁶数据来源：系统源码 `internal/tools/factor_review_tool.go` 的 `calcDecay`。核对时间 2026-09-24。

⁷数据来源：系统源码 `internal/tools/factor_review_tool.go` 第 35 行的构造签名。核对时间 2026-09-24。

factorWeightsFor 读走)，另一个方向是次日选股。

一次复盘要落库的东西，是六个指标加上几个溯源字段，一起写进 SQLite 的 FactorQuality 表：按 factor_name 为键做 UPSERT，即同一因子只保留最新一次体检结果⁸。这样这张表永远只有“最近一次体检”的含义，不会越长越大。

表 14.1 是这六栏在代码里的名字，以及它们各自在质量分里占的权重。

表 14.1 因子体检的六栏，与它们在质量分里的权重

体检栏	代码字段	权重	它防的病
IC (Pearson)	pearson_ic	—	参考项，不直接进质量分
RankIC	rank_ic / abs_rank_ic	0.35	信号弱、被离群点翻转
分组收益差	long_short_spread	—	只给人看的可读性翻译
覆盖率	coverage_pct	0.10	样本太少的假结论
前瞻准确率	top_accuracy	0.30	排前面却不涨
区分度	stability_std	0.15	得分挤在一起、没有区分力
衰减	decay_20d_minus_1d	0.10	只在极短周期有效
分半稳定性	split_half_ic / split_half_stable	见第 15 章	多重比较产生的假阳性

权重总和 $0.35 + 0.30 + 0.15 + 0.10 + 0.10 = 1.00$ 。

数据来源：系统源码 internal/tools/factor_review_tool.go 的 factorQualityFromItem；核对时间 2026-09-24。

质量分的构造值得细看，因为“诚实”两个字就写在这几行里：

```
1 // IC 贡献: |RankIC|=0.04 视为满分 0.35
2 icScore := clampFloat(absIC/0.04, 0, 1) * 0.35
3 if !halfStable && sampleCount >= 30 {
4     icScore *= 0.5           // 诚实下线: IC 贡献减半
5 }
6 accScore := clampFloat(accuracy, 0, 1) * 0.30
7 // 区分度: scoreStd 越接近 0.15 区分度越理想
8 staScore := (1 - clampFloat(absFloat(stability-0.15)/0.15, 0, 1)) * 0.15
9 covScore := clampFloat(coverage/80, 0, 1) * 0.10
10 // 短周期衰减: decay<=-0.02 满分, >=0.02 归零
11 decScore := (1 - clampFloat((decay+0.02)/0.04, 0, 1)) * 0.10
12 return clampFloat(icScore+accScore+staScore+covScore+decScore, 0, 1)
```

代码 14.1 质量分的五项加权（节选，去掉了注释）

这段话在说什么：质量分是把六栏压成一个 [0, 1] 的数。有几个具体的刻度很重要：

- $|RankIC| = 0.04$ 就算满分。这个数字比很多入门教程里说的 0.05、0.06 更宽松，说明它并不指望因子很强，只要求它“能被看见”。
- 覆盖率覆盖 80% 的股票池算满分，也就是这一项几乎总是拿不满——它防的是极端低覆盖。
- 区分度以 0.15 为理想值：太挤（接近 0）说明因子分不开股票，太散（远离 0.15）说明有极端值把标准差撑大。

⁸数据来源：系统源码 internal/tools/factor_review_tool.go 的 persistFactorQuality: db.Where("factor_name = ?", name).First(&existing) 命中即更新、未命中即 Create。核对时间 2026-09-24。

- 衰减以 -0.02 为满分、 $+0.02$ 为零分。注意这是一个窄窗：它奖励“短周期更有效”，但用的是 20 日与 1 日的差，不是收益本身。

写完这张表之后，我在系统里还发现了一个东西：情绪因子 `sentiment_score` 也被纳进了同一套体检。

它的实现方式是并发预取：以 5 个并发位（`sem := make(chan struct{}, 5)`）遍历股票池，每只股票单独设 20 秒超时，取不到就跳过这一只、不阻塞整场复盘⁹。这段代码让我改了一个旧习惯：我以前觉得“因子复盘”只是算数，跑多久都无所谓。事实上它每天盘前要跑，一只股票卡住 20 秒乘上几千只，就是一场事故。复盘工具必须自带超时纪律，这不是优化，是可用性。

最后是一件必须标注清楚的事。体检报告里有一栏是盘口因子，而盘口数据来自 `order_book_daily`。核对时点上看，这张表里没有任何记录 `order_book_daily` 在核对时点（2026-09-24）为 0 行。本书不为此填充任何估计值。¹⁰，因此盘口因子的复盘在当前数据库里拿不到可用样本。这件事本身就是体检报告要防的病之一：一个因子的数据源是空的，它的所有指标都不该被当成有效结论。

14.6 常见误解与纠偏

常见误解·IC 高就等于赚钱 这是我在整本书里最想刻进读者骨头的一句话：IC 是一个统计量，不是一个收益。它至少漏掉了四笔支出。

第一笔是覆盖率：IC 只统计参与计算的那些股票，如果它只覆盖了 5% 的股票，你根本没有 5% 之外的交易机会。第二笔是换手成本：排序每天都在变，你每次调仓都要付佣金、印花税与市场冲击。第三笔是容量：一个能赚钱的因子，规模一大就会自己把价差吃掉。第四笔是执行：涨停封死买不进、跌停封死卖不掉，这些在第二十二章会讲到。

正确的问法是：“这个因子扣掉成本之后，还剩多少？”——这个问题要到第十七章才有完整的答案。

常见误解·用“均值的相关系数”评价一个因子 我犯过这个错，而且犯了很久。

我把每天的 IC 记成一条时间序列，然后取平均：“这个因子过去 250 天的平均 IC 是 0.028，不错。”这句话把两件完全不同的事压成了一个数：每天都稳定地小有贡献，和十天空着、一天爆发。

后者的均值可能一样漂亮，但你不能用它——因为你不知道明天是空着的那十天，还是爆发的那一天。而且均值对极端值毫无抵抗力：一天 IC 冲到 0.5，能把 249 天的平淡全部盖章认证。

反制有两条：看分布而不只看均值（分位数、正号天数占比），以及下一章的分半检验。把一条序列压成一个数，是自欺最容易发生的那一步。

常见误解·把“稳定性”读成“IC 的时间稳定性” 上面说过一次，这里再单独列出来，因为它特别容易踩：`stability_std` 这个字段名字叫稳定性，算的是当日因子得分的横截面标准差，量的是区分度。

⁹数据来源：系统源码 `internal/tools/factor_review_tool.go` 的 `computeSentimentScores`：信号量容量 5、`context.WithTimeout(ctx, 20*time.Second)`、单只失败仅记录日志。核对时间 2026-09-24。

¹⁰数据缺失：该系统库中此项无有效记录。本书遵照系统“数据不可用即报错、不降级、不伪造”的原则，此处留白、不填充估计值。

一个因子可以天天都“分得很开”，同时天天排的顺序都不同。这种因子在 `stability_std` 上很漂亮，但在选股上等于抽签。

要判断“排序本身稳不稳”，看的是 `split_half_ic` 那一栏——下一章的全部内容。

14.7 真实出处

本章的真实出处

- 因子复盘工具 (`IC` / `RankIC` / 分组 / 覆盖率 / 准确率 / 区分度 / 衰减 / 分半) : `internal/tools/factor_review_tool.go`
- 指标计算细节：同文件中的 `pearsonIC`、`spearmanIC`、`rankIC`、`groupSpread`、`topAccuracy`、`scoreStd`、`calcDecay`、`rankFloat`
- 质量分构造与落库：同文件中的 `factorQualityFromItem`、`persistFactorQuality`、`blendQuality`
- 情绪因子并发预取：同文件中的 `computeSentimentScores`
- 落库表：`SQLite FactorQuality` (字段 `Quality`、`RankIC`、`Accuracy`、`Coverage`、`Stability`、`Decay`、`SplitHalfStable`、`SplitHalfIC`、`Detail`、`AutoCalibrate`)
- 质量分被谁读走：`internal/cio/cio.go` 的 `factorWeightsFor` (第 十三 章讲过它的 `clamp` 与归一化)
- 盘口因子数据源：`DuckDB order_book_daily`

动手试一试（十分钟）

1. 打开系统的 SQLite 库 `database/quantpilot.db`，查 `FactorQuality` 表。看两件事：一共几个因子有记录？每个因子的 `Quality` 是多少？
2. 挑质量分最低的那个因子，读它的 `Detail` 字段。那是一行形如 `factor_review@ 日期 |RankIC|=... acc=... std=... cov=... decay=... half=...` 的文本。逐项对上本章六栏，看看是哪一栏把它压低了。
3. 再挑质量分最高的那个因子，做同一件事。如果它靠的是覆盖率那一栏，你要意识到这个结论很脆——覆盖率一变，它就不是第一名了。
4. 最后一个练习不需要电脑：把本章那个五点的反例抄在纸上，手工算一遍 `IC` 与 `RankIC`，确认符号相反。

一份体检报告写到这里，还缺最关键的一道防线。上面所有数字都是“用全部样本算一次”，而真实世界里，一个因子之所以看起来很好，往往只是因为我们在两千只股票、二十个窗口上试了太多遍，试出了一个巧合。

下一章要做的，就是把这份报告撕成两半——看两半段会不会说同一种话。

第十五章

怎样识破一个“看起来很厉害”的因子？

15.1 一个 0.08 的因子，第二天全崩了

我至今记得那个周二。

周一盘后，公式跑完，有一个因子的 RankIC 是 0.08。这个数值在 A 股截面上属于“高得不像话”——按第 十四 章那张体检表的刻度， $|\text{RankIC}| = 0.04$ 就已经算满分¹。0.08 是它的两倍。

我当时的反应是把它加进权重，第二天按它选了一遍股票。

周三收盘，那一批股票的表现是全场后半段。

我第一反应是“市场变了”。于是我回去看这个因子在更长时间窗口上的表现——它不像是一个坏因子，它在过去三个月里大概七成的时间都为正。这让我更困惑：一个七成时间为正的因子，为什么偏偏在我相信它的那一天失效？

那天晚上我才想到一个更要紧的可能：也许它没有失效，只是我从一开始就高估了它。

我用的评价办法是这样的：把最近一段时间每天的 IC 算出来，取平均，看这个平均值。这个办法的问题是，它把一件有三个维度的事压成了一个数——第 十四 章结尾我提过一次，但它值得单独用一整章来讲，因为这一步是整条因子流水线上最容易被自己骗过去的地方。

我需要的不是更好的因子，是一个能说出“不行”的检验。

15.2 把全班分成两半

直觉·全班的排名可信吗 假设我要选一个“全班最会做题的人”，但我手上只有一次测验的排名。

我做了一件事：把全班同学按学号奇偶分成两组（A 组和 B 组），然后看 _____——

如果两组排出来的“前三名”是同一批人，那这个排名有共同的原因，它大概是真本事。

如果 A 组的第一名在 B 组排到第三十，B 组的第一名在 A 组排到第二十五，那么“第一名”这个称号就是抽签抽出来的。不是那个同学不行，而是那次测验量到的全是噪声。

关键就在这里：我并没有增加任何数据，我只是把同一批数据切成两半，然后问它们会不会说同一种话。

还有一个细节非常重要：分组不能由我“看着办”。如果我知道谁成绩好，我就可能不自觉地把他分到同一组去。所以分组必须是我事先定好、事后不能改的——一个固定的规则，一个固定的随机种子。

这就是本章全部的技术核心：用一个固定的规则，把样本撕成两半，看两半段会不会说同一种话。

¹数据来源：系统源码 `internal/tools/factor_review_tool.go` 的 `factorQualityFromItem: absIC/0.04` 封顶。核对时间 2026-09-24。

15.3 不留“反对席”，检验就变成表彰

在讲做法之前，我得先说清不这么做的代价，否则没人愿意多跑一遍计算。

代价一：你会把一个巧合写进权重。

第十三章讲过，因子质量分会通过 $\text{clamp}(q/q_{\text{med}}, 0.5, 1.5)$ 影响次日选股的权重。这条链是闭环的：一个因子在昨天看起来好，它明天就自动占更大的权重。如果这个“看起来好”是一次巧合，那么闭环会自动把错误放大——你没有任何一步会拦住它。

代价二：你会以为自己在做研究，其实在做筛选。

这是我最想纠正的一个自我认知。我在两千多只股票、二十来个时间窗口上反复看同一个因子，总会找到一个窗口里它表现很好。然后我把那个窗口的结果记下来，说“你看，它有效”。

这不是研究，这是筛选。我试了多少次，我从来没数过。

代价三：你会失去“承认不确定”的能力。

真正让我警觉的是一件很小的事：我发现我从来没有办法在报告里写“这个因子我判断不了”。报告里要么写“有效”，要么写“无效”。而现实中大量因子的真实状态是“信息不足，无法判断”。如果这句话不许说，那么所有“信息不足”都会被逼成“有效”或者“无效”这两个假答案。

所以系统的做法里有一条我特别看重：样本不够时不下结论，而不是下个结论再打折。

15.4 分半检验：从朴素版本到严谨版本

先用一句话说清朴素版本：把当天的截面样本随机分成两半，两半各算一个 RankIC；两个数方向一致、且都不太小，就判“稳定”。

朴素版本是对的，但它有三处需要严谨化，否则它只是把主观判断换成了另一种主观判断：怎么分才算随机？“都不太小”是多小？判错的概率有多大？

15.4.1 怎么分：固定种子的交替分配

先说“怎么分”。系统的实现可以完整写成一段流程。

定义 15.1 (分半划分). 设当天的截面样本按输入顺序编号为 $s_1, \dots, s_n$ 。取固定种子 $\text{seed} = 20240701$ ，由一个伪随机数发生器生成 $\{1, \dots, n\}$ 的一个排列 $\pi = (\pi_1, \dots, \pi_n)$ 。定义

$$A = \{s_{\pi_1}, s_{\pi_3}, s_{\pi_5}, \dots\}, \quad B = \{s_{\pi_2}, s_{\pi_4}, s_{\pi_6}, \dots\}, \quad (15.1)$$

即按排列后的次序“隔一个分一个”。于是

$$|A| = \left\lceil \frac{n}{2} \right\rceil, \quad |B| = \left\lfloor \frac{n}{2} \right\rfloor, \quad (15.2)$$

两半段样本量最多相差 1。

这段话在讲什么：先把样本顺序打乱（打乱的结果由种子 20240701 完全决定），然后像发

牌一样一张给 A、一张给 B。系统注释把这一步叫“按固定种子伪随机两分（交替分配到两半，数量均衡）”²。

这里有一个必须讲清的“矛盾”：既然是固定种子，那它到底算不算随机？

算，也不算。准确地说是：它对同一次输入是确定的，对不同的样本编号顺序是随机的。

命题 15.2 (固定种子交替分配的可复现性与无偏性). 在 *theorem 15.1* 的记号下：

(i) (确定性) 对固定的 n 、固定的种子与固定的输入顺序，划分 A, B 是一个确定的结果：用同样的输入重跑，得到同样的两半。因此该检验可复现。

(ii) (交换性) 设样本的属性 $X(s)$ 只依赖于样本自身的取值，与它在输入中的编号无关（假设 15.3）。若把输入顺序看作一个均匀随机的排列，则

$$\mathbb{E}[X_A] = \mathbb{E}[X_B], \quad (15.3)$$

其中 X_A, X_B 分别表示 A 半、 B 半上属性 X 的（同一口径的）统计量。

假设 15.3. 样本在输入序列中的编号顺序，与样本的属性 X 独立——换句话说，编号顺序里不含信息。

这个命题在说什么：分组规则是死的（可复现），但当编号本身不带信息时，分出来的两半在统计上没有系统差别——不会出现“A 半系统性偏爱大票”这种事。

证明. (i) 由随机数发生器的定义，`NewSource(20240701)` 产生的是一个确定的比特流；`Perm(n)` 是这条流的一个确定的函数。因此 π 是输入长度 n 的确定函数，eq. (15.1) 给出的 A, B 也是 $(n, \text{输入顺序})$ 的确定函数。同一输入 $\Rightarrow$ 同一划分。

(ii) 记 T 为输入顺序所对应的排列（把“第 i 个位置上是哪个样本”读成一个双射）。 π 是种子与 n 的函数，与 T 独立。设 τ 为交换位置 1 与 2 的对换，考虑映射

$$\varphi: \pi \mapsto \pi \circ \tau. \quad (15.4)$$

因为对换 τ 是双射， φ 是 $\{1, \dots, n\}$ 上全部排列之集到自身的一个双射；又因 π 在假设下均匀分布， φ 保持均匀性，即 $\pi \circ \tau$ 与 π 同分布。

现在看这个映射对划分做了什么。 $\pi \circ \tau$ 就是把 π 的第 1 位与第 2 位交换，其余不动。而 eq. (15.1) 是按位置奇偶分流，交换第 1、2 位的结果正是把 s_{π_1} 从 A 挪到 B、把 s_{π_2} 从 B 挪到 A，其余样本不动。因此若记

$$F(\pi) = (X_A(\pi), X_B(\pi)) \in \mathbb{R}^2, \quad (15.5)$$

则 $F(\pi \circ \tau) = (X_B(\pi), X_A(\pi))$ ，也就是把两个分量对调。

²数据来源：系统源码 `internal/tools/factor_review_tool.go` 的 `splitHalfStability: rand.New(rand.NewSource(20240701)).Perm(n)` 后按 `i mod 2 == 0` 分流。核对时间 2026-09-24。

由于 $\pi \circ \tau$ 与 π 同分布，随机向量 $F(\pi)$ 与 (X_B, X_A) 同分布，即 (X_A, X_B) 与 (X_B, X_A) 同分布（交换性）。对同分布的对称向量取期望，两个分量必然相等：

$$\mathbb{E}[X_A] = \mathbb{E}[X_B]. \quad (15.6)$$

■

注 15.4 (两条边界，必须写清). 第一，命题 15.2 的 (ii) 完全依赖假设 15.3。如果输入顺序本身携带信息——例如样本是“按股票代码字典序”排列的切片——那么固定种子就等价于一个**固定的、隔一个取一个的抽样**。这种抽样一旦与板块或代码段相关，两半就不再可比。这不是理论洁癖：这个项目真实发生过一次“按代码字典序取 300 只，结果样本几乎全落在北交所”的事故³。所以样本顺序这件事，在统计上和工程上都要单独盯。

第二，命题里说的是“期望相等”，不是“样本量完全相等”或“方差相等”。两半的样本量由 eq. (15.1) 保证最多差 1，但它们的方差并不相同，这在下一个小节会成为一个具体的数字问题。

15.4.2 判据，与它判错的概率

判据本身很短。系统里的规则是一行：

$$\text{判“稳定”} \iff \text{sign}(\hat{\rho}_1) = \text{sign}(\hat{\rho}_2) \text{ 且 } |\hat{\rho}_1| \geq 0.02 \text{ 且 } |\hat{\rho}_2| \geq 0.02, \quad (15.7)$$

其中 $\hat{\rho}_1, \hat{\rho}_2$ 是 A、B 两半段各自算出的 RankIC⁴。

三个条件缺一不可。方向必须一致（同号），强度必须各段达标（0.02）。系统把这两半段的 IC 原样存进 SplitHalfIC 字段，格式是 "ic1/ic2"：用 `fmt.Sprintf` 把两半段 IC 各保留四位小数、中间以斜杠连接⁵，方便日后审计。

那么问题来了：如果这个因子根本无效，它有多大概率被判成“稳定”？

要回答这个问题，我需要把“无效”变成一个可以计算的假设。

假设 15.5. (i)（无效原假设）该因子真实无效，即母体秩相关为 $\rho = 0$ 。

(ii)（渐近正态）两半段的估计量 $\hat{\rho}_1, \hat{\rho}_2$ 在 $\rho = 0$ 下近似服从均值为 0、方差为 $1/(m-1)$ 的正态分布，其中 m 为每半段的样本量。

(iii)（独立）两半段的样本互不重叠，故 $\hat{\rho}_1$ 与 $\hat{\rho}_2$ 相互独立。

³数据来源：系统事实清单第 15 节第 4 条：旧版市场广度按 ORDER BY symbol LIMIT 300 字典序截断，样本几乎全落在北交所与沪 600。

⁴数据来源：系统源码 `internal/tools/factor_review_tool.go`: `const minStrength = 0.02, 判定式为 math.Signbit(ic1)==math.Signbit(ic2) && math.Abs(ic1)>=minStrength && math.Abs(ic2)>=minStrength`。核对时间 2026-09-24。

⁵数据来源：系统源码 `internal/tools/factor_review_tool.go` 的 `buildFactorItem` 中由 `halfICKey` 拼接两个四位小数的 IC。核对时间 2026-09-24。

定理 15.6 (分半检验的假阳性概率上界). 在假设 15.5 下, 记

$$c = 0.02\sqrt{m-1}, \quad (15.8)$$

则判据 eq. (15.7) 误判“稳定”的概率满足

$$\mathbb{P}(\text{误判}) = 2(1 - \Phi(c))^2 \leq 4(1 - \Phi(c))^2, \quad (15.9)$$

其中 Φ 是标准正态分布函数。进一步, 用尾部不等式 $1 - \Phi(c) \leq \varphi(c)/c$ ($c > 0$, φ 为标准正态密度) 还可给出

$$\mathbb{P}(\text{误判}) \leq 4 \frac{\varphi(c)^2}{c^2}. \quad (15.10)$$

这个定理在说什么: 它把“我有多容易被骗”变成一个数, 而且这个数只取决于每半段样本量 m 和阈值 0.02。

证明. 先把单半段的越阈概率算出来。由假设 15.5(ii), $\hat{\rho}_j$ 的标准差是 $1/\sqrt{m-1}$, 直接把它标准化, 令

$$Z_j = \hat{\rho}_j \sqrt{m-1}, \quad j = 1, 2. \quad (15.11)$$

由 (ii), Z_j 近似服从 $N(0, 1)$ 。于是 $|\hat{\rho}_j| \geq 0.02$ 等价于 $|Z_j| \geq 0.02\sqrt{m-1} = c$ 。由标准正态的对称性,

$$\mathbb{P}(|\hat{\rho}_j| \geq 0.02) = \mathbb{P}(|Z_j| \geq c) = 2(1 - \Phi(c)). \quad (15.12)$$

第一步, 在同号条件下把两个半段合起来。要把一个事件分解成互斥的几块, 才能各自用独立性相乘。判“稳定”这个事件可以拆成两块:

$$\{\text{同号且都超阈}\} = \underbrace{\{\hat{\rho}_1 \geq 0.02, \hat{\rho}_2 \geq 0.02\}}_{\text{两部分正}} \cup \underbrace{\{\hat{\rho}_1 \leq -0.02, \hat{\rho}_2 \leq -0.02\}}_{\text{两部分负}}, \quad (15.13)$$

两块互斥。由假设 15.5(iii) (独立) 与 eq. (15.12),

$$\mathbb{P}(\text{两部分正}) = \mathbb{P}(\hat{\rho}_1 \geq 0.02) \mathbb{P}(\hat{\rho}_2 \geq 0.02) = (1 - \Phi(c))^2, \quad (15.14)$$

$$\mathbb{P}(\text{两部分负}) = \mathbb{P}(\hat{\rho}_1 \leq -0.02) \mathbb{P}(\hat{\rho}_2 \leq -0.02) = (1 - \Phi(c))^2, \quad (15.15)$$

其中两处都用了“正态分布对称于是 $\mathbb{P}(Z \geq c) = \mathbb{P}(Z \leq -c) = 1 - \Phi(c)$ ”。相加得

$$\mathbb{P}(\text{误判}) = 2(1 - \Phi(c))^2. \quad (15.16)$$

第二步, 放宽成 eq. (15.9)。由 $1 - \Phi(c) \leq 1$ (分布函数的取值不超过 1) 立得

$$2(1 - \Phi(c))^2 \leq 4(1 - \Phi(c))^2. \quad (15.17)$$

第三步，算出 Mills 型上界。对 $c > 0$,

$$1 - \Phi(c) = \int_c^\infty \varphi(z) dz \leq \int_c^\infty \frac{z}{c} \varphi(z) dz = \frac{1}{c} \int_c^\infty (-\varphi'(z)) dz = \frac{\varphi(c)}{c}, \quad (15.18)$$

其中第一个不等号用了 $z \geq c$ 内的被积变量满足 $z/c \geq 1$ 且 $\varphi(z) > 0$ ，最后一个等号用了 $\varphi'(z) = -z\varphi(z)$ (与第 14 章引理 14.7 同一个恒等式)。把它代入 eq. (15.9) 即得 eq. (15.10)。■

推论 15.7 (两个样本量下的实际数值). 取 $t = 0.02$, 分别代入 $n = 60$ (每半 $m = 30$) 与 $n = 400$ (每半 $m = 200$)。

$n = 60$: $\sqrt{m-1} = \sqrt{29} = 5.38516$, 故 $c = 0.02 \times 5.38516 = 0.10770$; 由标准正态表 (或级数 $\Phi(z) \approx 0.5 + 0.39894z - 0.06649z^3$ 在 $z = 0.1077$ 处的值) 得 $\Phi(0.10770) = 0.54288$, 故 $1 - \Phi(c) = 0.45712$,

$$\mathbb{P}(\text{误判}) = 2 \times 0.45712^2 = 0.41791, \quad 4(1 - \Phi(c))^2 = 0.83582. \quad (15.19)$$

$n = 400$: $\sqrt{m-1} = \sqrt{199} = 14.10674$, 故 $c = 0.02 \times 14.10674 = 0.28213$; $\Phi(0.28213) = 0.61106$, 故 $1 - \Phi(c) = 0.38894$,

$$\mathbb{P}(\text{误判}) = 2 \times 0.38894^2 = 0.30254, \quad 4(1 - \Phi(c))^2 = 0.60509. \quad (15.20)$$

表 15.1 无效因子被判“稳定”的假阳性概率上界 (阈值 0.02)

总样本 n	每半 m	c	$1 - \Phi(c)$	同号版 $2(\cdot)^2$	放宽版 $4(\cdot)^2$
60	30	0.1077	0.4571	0.4179	0.8358
400	200	0.2821	0.3889	0.3025	0.6051

表内数值不是系统数据，而是本章按 eq. (15.9) 与 eq. (15.12) 直接代入 Φ 的函数值算出的结果，算式已全部写在 theorem 15.7 中。

数据来源：本章按 theorem 15.6 的公式计算； m 取 $\lfloor n/2 \rfloor$ ；核对时间 2026-09-24。

这张表第一次算出来的时候，我盯着它看了很久，因为它说的是一个我不太愿意接受的事实：在 $n = 60$ 这种规模上，“稳定”这个标签几乎拦不住任何东西。

看 $n = 60$ 那一行：一个完全无效的因子，有超过四成的概率被判成“稳定”。原因是 0.02 这个阈值太小了：每半段的估计量标准差是 $1/\sqrt{29} = 0.1857$ ，而门槛只立在 0.02 处，也就是 0.11 个标准差的位置。门槛立在这么近的地方，绝大多数噪声都能跨过去。

所以这个检验真正在干活的，是另外两件事，我得把它们分清楚，不能糊在一起。

第一件：同号条件把假阳性正好砍掉一半。

这一条可以从比较里直接看出来。如果不要求同号，只要求“两半都超阈”，那么四个符号组合（正正、正负、负正、负负）全算数：

$$\mathbb{P}(\text{两半都超阈}) = [2(1 - \Phi(c))]^2 = 4(1 - \Phi(c))^2, \quad (15.21)$$

而加了同号条件只保留其中两块，于是

$$\frac{\mathbb{P}(\text{同号且都超阈})}{\mathbb{P}(\text{都超阈})} = \frac{2(1 - \Phi(c))^2}{4(1 - \Phi(c))^2} = \frac{1}{2}. \quad (15.22)$$

这不是巧合，是因为在原假设下 $\hat{\rho}$ 的符号是左右对称的，四个象限各占一份——要求“两半段说同一个方向”，等于把一半的偶然一致直接排除掉。

第二件：0.02 这个阈值的作用不是“筛强”，而是“筛方向”。

它低到几乎不筛强度（ $n = 60$ 时越阈概率高达 0.914），它真正排除的是那些“一半为正一半为负”的因子——那种因子的信号连方向都不确定。要让阈值真正起到“筛强度”的作用，样本量必须大得多：看比值 $t/(1/\sqrt{m-1}) = 0.02\sqrt{m-1} = c$ ，从 $m = 30$ 时的 0.108 涨到 $m = 200$ 时的 0.282，阈值才逐渐站出来。

注 15.8 (Mills 上界在这里为什么是空的). eq. (15.10) 给的是一个“不含 Φ 、只用密度就能算”的上界。把它代入上面两个数： $c = 0.1077$ 时 $\varphi(c) = 0.39663$ ， $\varphi(c)/c = 3.6826$ ，于是 $4\varphi(c)^2/c^2 = 54.3$ ； $c = 0.2821$ 时 $\varphi(c) = 0.38337$ ， $\varphi(c)/c = 1.3588$ ，于是 $4\varphi(c)^2/c^2 = 7.39$ 。两个都大于 1，也就是说这条上界在概率的意义上什么也没说。

要让它有意义，需要 $\frac{2\varphi(c)}{c} \leq 1$ ，即 $c \geq 0.65$ 。在阈值固定为 0.02 时，这等价于 $\sqrt{m-1} \geq 32.2$ ，也就是每半段样本量要在 1000 以上。

我把这个失败的推导也写进来，是因为它比成功的推导更能说明问题：0.02 这个门槛低到连一个非平凡的理论上下界都撑不起来。它的价值在于“方向一致性”和“给权重侧一个诚实下线的开关”，而不在于它自己有多强的判别力。

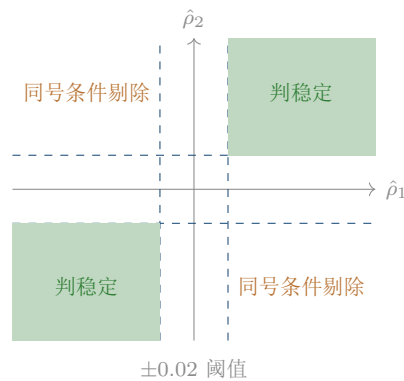


图 15.1 判据 eq. (15.7) 的四个象限。深蓝虚线是 ± 0.02 阈值，绿色两块是“同号且都超阈”。原假设下四个象限各占一份，同号条件只留两块，假阳性正好减半。

15.4.3 诚实下线：同一个判断，两把尺子

判“不稳定”之后怎么办，才是这一章真正的重点。因为“发现它不稳”这件事本身不改变任何行为——一个不影响次日参数的检验，等于没有检验（这句话在第 三十二 章还会再出现一次）。

系统里给“不稳定”准备了两道下线，都在 `internal/cio/cio.go` 与 `internal/tools/factor_review_tool.go` 中：

下线一（质量分侧）：IC 贡献减半。在 `factorQualityFromItem` 里，IC 那一项的权重本来是 0.35；如果分半判定不稳定，且总样本量达到 30，就把这一项乘 0.5，变成 0.175⁶。

下线二（权重侧）：先封顶，再砍半。在 `factorWeightsFor` 里，如果这个因子被“真实判定”为不稳定，那么先把它本来的自适应倍数 `mult` 封顶到 0.6，再乘 0.5：

$$\text{mult} \leftarrow \min(\text{mult}, 0.6) \times 0.5 \leq 0.6 \times 0.5 = 0.3. \quad (15.23)$$

也就是说，一个被判定不稳定的因子，次日最多只能拿到默认权重的三成⁷。

为什么要“先封顶再砍半”，而不是直接乘 0.5？因为 `mult` 本身已经有 [0.5, 1.5] 的区间。如果直接乘 0.5，一个质量分影像特别好的因子会拿到 $1.5 \times 0.5 = 0.75$ ——这仍然不小，“不稳定”三个字就白说了。先封顶到 0.6 再砍半，保证结果不超过 0.3，与“它可能只是一次巧合”这个判断相称。

两道下线共用的前提：不能“无中生有地判不稳定”。样本不足时，两半段的 IC 主要是噪声，“不稳定”这个结论本身不可靠。系统为此设了两道样本下限：

- $n < 8$ ：直接返回“不稳定”，且不返回两半段 IC。这时 `SplitHalfIC` 是空字符串⁸。
- 总样本 < 30 ：视为“无法判定”。质量分侧因此不减半——注释写得很清楚：“样本不足的因子其覆盖率/IC 本来就极低，已被 `covScore/icScore` 自然压低，无需额外减半”。

于是“不能判定”与“判定为不稳定”在系统里是两个不同的状态，而且都有落点：前者 `SplitHalfIC` 为空，后者 `SplitHalfIC` 是 "ic1/ic2" 这样的两个数。

注 15.9 (两把尺子的口径差异). 这里有一处我在读代码时反复确认过的细节，值得写下来，因为它会真实影响权重。

质量分侧的下线用的是“总样本 ≥ 30 ”这个门槛；权重侧的下线用的是“`SplitHalfIC` 非空”这个门槛（即 $n \geq 8$ ）。于是在 $8 \leq n < 30$ 这个区间里，两个门槛给出的答案不一样：质量分那一项没有被减半，但次日权重照样被砍到不超过 0.3。

这不是错误，是两个层次的设计选择（质量分关心“这个数准不准”，权重关心“要不要把风险降下来”），但你在读一个因子的实际权重时必须知道：它可能不是被质量分压低的，而是被权重侧的二次降权压低的。两条路径的出处分别在 `factorQualityFromItem` 与 `factorWeightsFor`。

15.5 它在系统里长什么样

把上面所有内容对到代码上，只有几个名字。

检验函数：`splitHalfStability(samples []fSample) (halfIC []float64, stable bool)`。它返回两半段的 IC（一个长度 2 的切片）与一个布尔判定。实现里有一处工程细节我

⁶数据来源：系统源码 `internal/tools/factor_review_tool.go` 的 `factorQualityFromItem`: `if !halfStable && sampleCount >= 30 { icScore *= 0.5 }`。核对时间 2026-09-24。

⁷数据来源：系统源码 `internal/cio/cio.go` 的 `factorWeightsFor`: `mult = math.Min(mult, 0.6)` 紧接 `mult *= 0.5`。核对时间 2026-09-24。

⁸数据来源：系统源码 `internal/tools/factor_review_tool.go` 的 `splitHalfStability`: `if n < 8 { return nil, false }`；以及 `if len(sc1) < 3 || len(sc2) < 3 { return nil, false }`。

特别欣赏：种子是写死在函数体内的字面量 20240701，不是从配置里读的⁹。这意味着任何人、任何时间、在任何机器上，对同一批样本都会得到同一个划分——审计才有可能。

落库字段：FactorQuality 表的 SplitHalfStable（布尔）与 SplitHalfIC（形如 "0.0312/-0.0104" 的文本）。再加上 Detail 里那一段 half=...✓ 稳定的标记，一共三处记录了这次检验¹⁰。为什么要记三遍？因为这三个字段的读者不同：SplitHalfStable 给程序读，SplitHalfIC 给权重侧的守卫条件读，Detail 给人读。

参考基准：medianFactorQuality。“好因子上调、差因子下调”总得有个基准线，系统用的是中位数而不是均值¹¹。原因和第 十三章讲过的一致，但在这里更锋利：如果基准用均值，那么一个被诚实下线、质量分被打到很低的不稳定因子，会拉低均值，从而让其他因子的倍数被动上调——一次惩罚变成了对所有因子的奖励。

```
1 mult := 1.0
2 if q, ok := qmap[name]; ok && meanQ > 0 && q > 0 {
3     mult = clampF(q/meanQ, 0.5, 1.5)
4     // 仅当分半检验「真实可判定」且判定为不稳定(false)时才强力降权
5     // （先封顶 0.6 再砍半 → 总倍数 <= 0.3）
6     if h, okH := half[name]; okH && !h.stable && h.ic != "" {
7         mult = math.Min(mult, 0.6)
8         mult *= 0.5
9     }
10 }
```

代码 15.1 权重侧的诚实下线（节选）

这段话在说什么：h.ic != "" 这个判断就是“分半是否真实可判定”的开关。它让“样本不足、无法判定”的因子免于被降权——这正是本章最重要的一条设计原则落到了代码上。

还有一条链要说清，否则读者会以为这套检验只是“多存了几个字段”。它其实是三环相扣的：

- (1) splitHalfStability 算出 stable 与两半段 IC；
- (2) factorQualityFromItem 用 stable 决定 IC 贡献是否减半，得到 [0, 1] 的质量分；
- (3) factorWeightsFor 用落库后的 SplitHalfStable 与 SplitHalfIC 决定次日权重倍数是否被砍到 ≤ 0.3 。

第 (3) 步读的不是内存里的变量，而是数据库里上一次写进去的值。这个细节有意义：今日的验证结果，明日才生效；而“明日生效”这件事本身可以被回查。第 三十二章 会讲这条闭环怎么避开“重启就重跑一遍”的问题。

⁹数据来源：系统源码 internal/tools/factor_review_tool.go: rand.New(rand.NewSource(20240701))。核对时间 2026-09-24。

¹⁰数据来源：系统源码 internal/tools/factor_review_tool.go 的 persistFactorQuality 与 stableMark。核对时间 2026-09-24。

¹¹数据来源：系统源码 internal/cio/cio.go 的 medianFactorQuality：排序后取中位，偶数个取中间两个的平均；空映射返回 0。核对时间 2026-09-24。

15.6 常见误解与纠偏

常见误解·过拟合就是“参数太多” 这是对过拟合最流行、也最误导人的一种理解。

真正的过拟合不是“参数多”，而是你没有给反对证据留位置。一个只有三个参数、但我试了两千种组合、并且只保留最好的那一组的策略，比一个二十个参数、但从从不回头看结果的策略更容易过拟合。

判据很简单：在得到这个结论之前，我有没有一个会被这个结论打败的检验？如果不管结果怎样，我的结论都不会变，那我不是在做研究，我是在做确认。

分半检验之所以放在这一章，就是为了给“反对证据”安排一个固定席位。它有一个事先定好的、不许改的分组规则（种子 20240701），和一条事先定好的判据（eq. (15.7)）。“事先”两个字是它全部的价值来源。

常见误解·样本不足就不下结论，是偷懒 我一开始真的是这么想的：算不出结论就写“数据不足”，这不是把责任推给数据吗？

后来我算清 theorem 15.7 里那两行数，才明白过来：在 $n = 60$ 的规模上，判据 eq. (15.7) 对一个无效因子的假阳性上界是 0.42；在 $n = 8$ 的规模上，这个上界会更接近 1。在信息不足的时候逼出一个结论，得到的一定是假结论。

所以“样本不足就不下结论”不是偷懒，是统计诚实：它把“我不确定”这件事保留下来，而不是用一个看起来像答案的数字把它盖住。系统里那个 `h.ic != ""` 的判断，写的正是这句话。

这条原则全书通用。第 十九 章会看到同一件事的另一种形态：六维判势的样本数不足 100 时，必须在 `sources` 里标注低可信度，而不是悄悄给一个正常分数。

常见误解·分半检验会惩罚所有因子 这是我在第一次上线它之后担心的事：如果检验太严，是不是所有因子都会被降权？

算完数就知道这个担心把方向搞反了。真正的机制是相对的：`mult` 是质量分除以中位数。当所有因子都被同等下调时，归一化那一步（把权重总和缩回默认总和）会把它们整体还原——相对排序几乎不变。

所以诚实的问法不是“它会不会惩罚因子”，而是“它会不会改变因子的相对排序”。答案是会，而且这正是我们要的：一个稳定、覆盖广、衰减正常的因子，会相对那些“看起来很好但两半段说不到一起”的因子占更大的权重。

15.7 真实出处

本章的真实出处

- 分半稳定性检验（固定种子 20240701、交替分配、数量均衡、 $|IC| \geq 0.02$ 阈值、 $n < 8$ 直接判不稳定）：`internal/tools/factor_review_tool.go` 的 `splitHalfStability`
- 两半段 `IC` 的格式化与落库：同文件的 `buildFactorItem(split_half_ic)`、`persistFactorQuality`、`stableMark`
- 质量分侧的诚实下线（`IC` 贡献减半，门槛 $n \geq 30$ ）：同文件的 `factorQualityFromItem`
- 权重侧的诚实下线（先封顶 0.6 再 $\times 0.5$ ，门槛 `SplitHalfIC` 非空）：`internal/cio/cio.go` 的 `factorWeightsFor`
- 中位数参考基准：同文件的 `medianFactorQuality`

- 落库表与字段：SQLite *FactorQuality* (*SplitHalfStable*、*SplitHalfIC*、*Quality*、*Detail*)

动手试一试（十五分钟）

1. 查 SQLite 的 *FactorQuality* 表，只取两列：*FactorName* 与 *SplitHalfIC*。
2. 分类数一下：有几行的 *SplitHalfIC* 是空字符串（无法判定），有几行是 "ic1/ic2" 这种带斜杠的文本（真实判定）。
3. 在带斜杠的那些行里，找出两个数符号相反的行。这些就是判不出来方向的因子——它们的 *SplitHalfStable* 应当是 false，去看看是不是。
4. 拿两个数都为正、且都大于 0.02 的行，算一下 *SplitHalfStable* 是不是 true。如果不是，你要检查一下自己的判断和代码里的判据 eq. (15.7) 是不是同一句话。
5. 最后做一个“反实验”：把 theorem 15.7 里 $n = 400$ 那一行重新手算一遍，验证 $c = 0.02\sqrt{199}$ 确实落在 0.282 附近。

到这里，因子的部分就讲完了：它怎么被拆成六栏体检（第 十四 章），怎么被撕成两半来识破（本章）。可是因子只是“给股票排名”。

下一个问题不一样：排名之后，谁在决定什么时候买、什么时候卖？——那些名字听起来很像十八般武艺的“策略”，究竟在做什么。

第十六章

十七种策略：它们其实只做一件事

16.1 同一天，十七个策略吵起来了

这件事发生在我做「每日策略」那个功能的时候。

想法很简单：既然系统内置了十七个策略，那就让它们在**同一只股票、同一段日 K 线**上各跑一遍，把结果拉出来排个序，看哪个更好。

排序做不下去。因为它们的输出不是分数，是方向。

同一根 K 线上，双均线说买，RSI 说卖，KDJ 说再等等，DMI 说有趋势但强度不够，CMF 说资金在往外走。十七个策略，给了我七八个不同的答案。

我第一反应是“参数不对”。我把几个策略的周期挨个调了一遍，重新跑。答案还是打架，只是换了个位置打架。

第二个反应才是我真正需要的：**我根本不知道这十七个东西分别在算什么**。我一直在问“哪个策略更好”，却从来没有问过“它们在算同一件事吗”。

那天晚上我把十七个策略的实现逐个读了一遍。读完的结论让我有点意外：**它们确实只做一件事**。

16.2 工具箱里的十七把螺丝刀

直觉·螺丝刀不分工种，只分工件 我桌上有个小工具箱：一字、十字、内六角、梅花。四种刀头，形状完全不同。

可它们干的是同一件事：把螺丝转进去、转出来。挑用哪把的时候，我从来不看“哪把刀更好”，只看**这颗螺丝是什么型号**。

策略也一样。十七个名字听起来像十八般武艺，其实每一个都只回答同一个问题：在这个时点，我该买、该卖，还是该不动？

它们的区别不在功能，而在它们**盯的是行情里的哪一种形状**——有人盯趋势的斜率，有人盯价格偏离均值多远，有人盯成交量与资金方向。同一段行情里，这几种形状常常互相矛盾，所以答案互相矛盾。

从这个角度说，“十七个策略哪个最好”这个问题，跟“一字螺丝刀和十字螺丝刀哪个更好”是同一个问题：**问错了对象**。

还有一层很像的地方：螺丝刀之所以能换刀头，是因为握把是同一个。策略也一样——它们能放在一起比较、一起回测、一起调参，是因为**接口是同一个，实现是可插拔的**。

16.3 没有统一接口，要付三种代价

在动手之前，我得先把“为什么必须统一”说清楚，否则“统一接口”听起来只是程序员的口味问题。

代价一：你没法比较任何两个策略。

如果每个策略自己定义输入和输出——一个吃日 K，一个吃分钟 K，一个吐“信号”，一个吐“目标仓位”——那么“A 比 B 好”这句话就没有公共口径。你手里的比较结果，比的其实是两套不同的记账方式。

代价二：每加一件新东西，成本是乘法不是加法。

回测、参数网格扫描、每日策略选优、盘中策略减仓，这是四条独立的功能链。如果每个策略都要为这四条链各写一套适配，那么策略数 17 与链路数 4 的乘积就是 68 处需要维护的接缝。接缝越多，出错概率越大。

代价三：出了问题，你分不清是信号错了还是执行错了。

如果“策略判断行情”和“系统怎么用它”混在一个函数里，那么一次亏损就有两种完全不同的解释：要么是信号本身在这个环境里失效，要么是引擎把它用错了。这两件事的修法完全不同，混在一起就永远查不清。

系统的答案短得有点不讲道理：一个只有两个方法的接口，定义在 `internal/backtest/strategies.go`。

16.4 一个接口，三种输出

```

1 // Strategy 策略接口
2 type Strategy interface {
3     Name() string
4     GenerateSignals(bars []BarData) []StrategySignal
5 }
6
7 const (
8     SignalNone StrategySignal = 0 // 不动
9     SignalBuy  StrategySignal = 1 // 买
10    SignalSell StrategySignal = -1 // 卖
11 )

```

代码 16.1 策略接口与信号常量（节选）

两个方法：一个报出自己的名字，一个吃一串 K 线、吐一串信号。

注意输出的是“信号序列”而不是“买卖决策”。信号序列和 K 线等长，第 i 位上的值属于 $\{-1, 0, +1\}$ ：策略只是在说“第 i 根 K 线上出现了一个买点”，它不知道账户里有多少钱、当时仓位多少、这只股票是不是躺在涨停板上。那些全部由后面的引擎负责。

这正是“可插拔”的准确含义：策略负责形状，引擎负责后果。

那么十七种策略的“不同”，全藏在 `GenerateSignals` 的函数体里。我把它们读完，归纳出一个有点扫兴的结论。

16.4.1 十七个函数体，其实只有两种算法

第一种：比较两条线。先把 K 线算成两条（或一条线与一条阈值线），再看谁在上面、有没有交叉。

- 双均线：MA15 与 MA20
- EXPMA：EMA12 与 EMA50
- KDJ 金叉：K 与 D（再加 J 的高低门槛）
- TRIX：TRIX 与它自己的均线
- DMI：PDI 与 MDI（再加 ADX 的强度门槛）
- HMA：收盘价与 HMA
- Aroon：AroonUp 与 AroonDown
- 布林带突破：收盘价与上下轨
- 超级趋势：收盘价与最终轨道

第二种：把一个数拿去和阈值比。

- RSI：和 30 / 70 比
- KDJ 里的 J 值：和 30 / 70 比
- MFI：和 30 / 70 比
- MTM：和 0 比
- CCI：和 ± 100 比
- CMF：和 ± 0.05 比
- 海龟：和 40 日最高价 / 5 日最低价比
- 捉妖大师：几个周期的涨幅分别和门槛比，再要求“共振”

机器学习策略看起来是第三种：它吐出一个概率。可它最后还是把概率和 0.5 / 0.3 比——又回到了第二种。

所以“十七种策略”更准确的说法是：**十七套把 K 线变成数字的公式，加上两种比较方式。**

而第一种里的绝大多数成员，可以合并成同一个数学对象。这一步值得认真做，因为它能一次性回答“金叉为什么会出现在这根 K 线上”。

定义 16.1 (线性过滤器). 设 p_t 为收盘价序列。称

$$L_t = \sum_{k=0}^K c_k p_{t-k} \quad (16.1)$$

是一个**线性过滤器**， $c_0, \dots, c_K$ 是它的权重。若某个策略用来比较的量可以写成这样的 L_t ，就称这个策略是线性的。

16.4.2 均线信号的可分解性与水平位移不变性

定理 16.2 (均线交叉信号的可分解性与水平位移不变性). 设 $1 \leq F < S$ 。记快、慢均线为

$$MA_F(t) = \frac{1}{F} \sum_{k=0}^{F-1} p_{t-k}, \quad MA_S(t) = \frac{1}{S} \sum_{k=0}^{S-1} p_{t-k}, \quad (16.2)$$

并记差分 $D_t = MA_F(t) - MA_S(t)$ 。则：

(i) (可分解性) D_t 是一个线性过滤器：

$$D_t = \sum_{k=0}^{S-1} c_k p_{t-k}, \quad c_k = \begin{cases} \frac{1}{F} - \frac{1}{S}, & 0 \leq k \leq F-1, \\ -\frac{1}{S}, & F \leq k \leq S-1. \end{cases} \quad (16.3)$$

(ii) (零和) $\sum_{k=0}^{S-1} c_k = 0$ 。

(iii) (水平位移不变性) 若把整条价格序列整体抬高一个常数，即 $p_t \mapsto p_t + a$ (a 为任意实数)，则 D_t 逐位不变。

(iv) (正齐次性) 若 $p_t \mapsto \lambda p_t$ ($\lambda > 0$)，则 $D_t \mapsto \lambda D_t$ 。因为 $\lambda > 0$ ，金叉条件 $\{D_{t-1} \leq 0 < D_t\}$ 与死叉条件 $\{D_{t-1} \geq 0 > D_t\}$ 的判定完全不变。

(v) (平盘无信号) 若 $p_t \equiv p$ 为常数，则 $D_t \equiv 0$ ，金叉与死叉都不触发。

这个定理在说什么：一句“MA15 上穿 MA20”，数学上就是“一个权重和为零的线性过滤器，从非正变成正”。它只看价格的形状，完全不看价格的绝对水平。

证明. (i) 两个和式的长度不同 ($S > F$ ，慢线更长)，先把它们对齐到同一个下标范围 $\{0, \dots, S-1\}$ ：

$$\begin{aligned} D_t &= \frac{1}{F} \sum_{k=0}^{F-1} p_{t-k} - \frac{1}{S} \sum_{k=0}^{S-1} p_{t-k} \\ &= \underbrace{\sum_{k=0}^{F-1} \left(\frac{1}{F} - \frac{1}{S} \right) p_{t-k}}_{\text{快线覆盖的前 } F \text{ 个}} + \underbrace{\sum_{k=F}^{S-1} \left(0 - \frac{1}{S} \right) p_{t-k}}_{\text{只被慢线覆盖的 } S-F \text{ 个}}. \end{aligned} \quad (16.4)$$

第一段对应 $k = 0, \dots, F-1$ ，系数为 $1/F - 1/S$ ；第二段对应 $k = F, \dots, S-1$ ，系数为 $-1/S$ 。写成分段形式即 eq. (16.3)。

(ii) 把两段系数分别求和：

$$\sum_{k=0}^{S-1} c_k = \underbrace{F \left(\frac{1}{F} - \frac{1}{S} \right)}_{F \text{ 个}} + \underbrace{(S-F) \left(-\frac{1}{S} \right)}_{S-F \text{ 个}} = \left(1 - \frac{F}{S} \right) - \left(1 - \frac{F}{S} \right) = 0. \quad (16.5)$$

(iii) 用 (i) 的分解式，把每一个 p_{t-k} 换成 $p_{t-k} + a$ ：

$$D_t(p + a\mathbf{1}) = \sum_{k=0}^{S-1} c_k (p_{t-k} + a) = \underbrace{\sum_{k=0}^{S-1} c_k p_{t-k}}_{=D_t(p)} + a \underbrace{\sum_{k=0}^{S-1} c_k}_{=0} = D_t(p). \quad (16.6)$$

关键在最后一步：整体位移带来的那一项 $a \sum_k c_k$ ，被 (ii) 直接清零了。

(iv) 同样展开：

$$D_t(\lambda p) = \sum_{k=0}^{S-1} c_k \lambda p_{t-k} = \lambda D_t(p). \quad (16.7)$$

$\lambda > 0$ 时， $\lambda D_{t-1} \leq 0$ 当且仅当 $D_{t-1} \leq 0$ ； $\lambda D_t > 0$ 当且仅当 $D_t > 0$ 。两个不等式的方向都不变，所以金叉的判定不变；死叉同理。

(v) $p_t \equiv p$ 时，两个均线的每一个窗口平均都等于 p ，故 $\text{MA}_F(t) = \text{MA}_S(t) = p$ ，于是 $D_t = 0$ 对一切 t 成立。金叉要求 $D_t > 0$ 、死叉要求 $D_t < 0$ ，两者都永远不成立。■

推论 16.3 (水平位移不变性的一般判据). 对一般线性过滤器 $L_t = \sum_{k=0}^K c_k p_{t-k}$ ，“整体加常数后不变”当且仅当 $\sum_k c_k = 0$ 。

特别地，“收盘价上穿 n 日均线”也是零和的：它的权重是 $c_0 = 1 - 1/n$ ， $c_1 = \cdots = c_{n-1} = -1/n$ ，求和得 $1 - 1 = 0$ 。所以它与“快线上穿慢线”属于同一个家族。

反过来，RSV、RSI 这类比值型指标不属于这一家族：它们靠“除以区间宽度”免疫于位移，代价是引入了非线性。

这句推论值得单独说一句：线性类策略的共同点是“看形状不看价位”，比值类策略的共同点是“看相对位置不看绝对位置”。两类都不会因为一只股票是 5 元还是 500 元而改变信号——这是它们能在全市场几千只股票上直接复用的前提。

注 16.4 (一个我照实写下来的不一致). 定理 16.2 的 (v) 说的是“价格恒定时不触发信号”。这句话有一个隐藏前提：参与比较的均线必须是有效值。

系统里的简单均线在窗口未满时返回的是零占位¹。于是取 $F = 15$ 、 $S = 20$ 时，从 0 计数的第 14 根 K 线上，快线刚刚变成有效值、慢线仍然是 0；交叉判定函数看到的是“一条线从 0 上穿到正数”，会认成一次金叉。

后加入的四个策略都显式把预热区关掉了：super_trend 与 aroon、hma 都在 $i < \text{period}$ 时直接跳过，cmf 跳过 $i < \text{period}-1$ ²。而最早写入的双均线与 EXPMA 策略，循环从 $i = 1$ 开始，没有这道闸门。

我没有把这个不一致偷偷改掉，而是把它写在这里，因为它正好说明一件事：“接口统一”不等于“实现对不齐的问题自动消失”。凡是可能返回占位值的指标，调用方都必须自己把预热区关掉——这是策略作者的责任，不是接口的责任。

¹数据来源：系统源码 internal/backtest/indicators.go 的 MA：仅在 $i \geq \text{period}-1$ 时写入均值，更早的位置保持切片初始值 0。核对时间 2026-09-24。

²数据来源：系统源码 internal/backtest/strategies.go 中四个 GenerateSignals 的循环头部。核对时间 2026-09-24。

16.4.3 超级趋势：让通道跟着波动呼吸

第二种算法里的“阈值”是死的（30、70、 ± 0.05 ）；第一种算法里的两条线又太钝。有没有一种办法，让阈值自己随市场波动调整？有，而且它只用到一个统计量。

定义 16.5 (真实波幅与 ATR). 设 H_t, L_t, C_t 为第 t 根 K 线的高、低、收。定义真实波幅

$$\text{TR}_t = \max\{H_t - L_t, |H_t - C_{t-1}|, |L_t - C_{t-1}|\} \quad (t \geq 1), \quad \text{TR}_0 = H_0 - L_0. \quad (16.8)$$

给定周期 k ，平均真实波幅 ATR（Wilder 平滑）为

$$\text{ATR}_k = \frac{1}{k} \sum_{j=1}^k \text{TR}_j, \quad \text{ATR}_t = \frac{(k-1) \text{ATR}_{t-1} + \text{TR}_t}{k} \quad (t > k). \quad (16.9)$$

这段定义里有一个细节值得停一下： TR_t 把“当日振幅”和“跳空”都算进去了—— $H_t - L_t$ 管日内， $|H_t - C_{t-1}|$ 与 $|L_t - C_{t-1}|$ 管隔夜跳空。取三者最大，等于不管波动出现在日内还是隔夜，都算数。而 ATR 的递推式说明它是“把当前波动以 $1/k$ 的权重滚进平均值”。

有了 ATR，超级趋势把中线与两条基本轨定义成

$$M_t = \frac{H_t + L_t}{2}, \quad B_t^U = M_t + m \text{ATR}_t, \quad B_t^L = M_t - m \text{ATR}_t. \quad (16.10)$$

系统里取的默认参数是 $k = 10$ 、 $m = 3.0^3$ 。

定理 16.6 (超级趋势 ATR 通道的宽度、单调性与有界性). (i) (宽度) 两条基本轨之间的宽度为

$$B_t^U - B_t^L = 2m \text{ATR}_t, \quad (16.11)$$

且当 $\text{ATR}_t > 0$ 时，它对 $m > 0$ 严格单调增： m 越大，通道越宽。

(ii) (单向收紧) 设最终上轨由收敛规则

$$F_t^U = \begin{cases} B_t^U, & B_t^U < F_{t-1}^U \text{ 或 } C_{t-1} > F_{t-1}^U, \\ F_{t-1}^U, & \text{否则} \end{cases} \quad (16.12)$$

生成。若 $C_{t-1} \leq F_{t-1}^U$ (上一根 K 线没有突破上轨)，则

$$F_t^U = \min\{F_{t-1}^U, B_t^U\} \leq F_{t-1}^U, \quad (16.13)$$

即上轨只降不升。对偶地，若 $C_{t-1} \geq F_{t-1}^L$ ，则 $F_t^L = \max\{F_{t-1}^L, B_t^L\} \geq F_{t-1}^L$ ，下轨只升不降。

(iii) (有界性) 若价格始终落在 $[P_{\min}, P_{\max}]$ 内，记 $R = P_{\max} - P_{\min}$ ，则对一切 $t \geq k$ ，

$$0 \leq \text{TR}_t \leq R, \quad 0 \leq \text{ATR}_t \leq R, \quad 0 \leq B_t^U - B_t^L \leq 2mR. \quad (16.14)$$

³数据来源：系统源码 `internal/backtest/strategies.go` 的 `SuperTrendStrategy` 与 `BuiltinStrategies` 中 `super_trend` 的 `ConfigJSON`: `period=10`、`multiplier=3.0`。核对时间 2026-09-24。

这个定理在说什么：通道的宽度是“波动的 $2m$ 倍”，不是“价格的某个固定百分比”；轨道像棘轮一样只往内收；而 ATR 有界，所以一次异常 K 线不会让通道永久变形。

证明. (i) 两式相减：

$$B_t^U - B_t^L = (M_t + m \text{ATR}_t) - (M_t - m \text{ATR}_t) = 2m \text{ATR}_t. \quad (16.15)$$

右端关于 m 求导得 2ATR_t 。当 $\text{ATR}_t > 0$ 时导数严格为正，故关于 m 严格单调增。

(ii) 在条件 $C_{t-1} \leq F_{t-1}^U$ 下，收敛规则里的第二个条件 ($C_{t-1} > F_{t-1}^U$) 为假，于是规则退化成两分支：“若 $B_t^U < F_{t-1}^U$ 则取 B_t^U ，否则取 F_{t-1}^U ”。这两种情形分别给出 $F_t^U = B_t^U$ 与 $F_t^U = F_{t-1}^U$ ，而在两种情形里取小者恰好都等于 $\min\{F_{t-1}^U, B_t^U\}$ 。所以 $F_t^U = \min\{F_{t-1}^U, B_t^U\}$ 。

对任意两个实数，最小值不大于其中任何一个；取被比较者是 F_{t-1}^U ，即得 $F_t^U \leq F_{t-1}^U$ 。

下轨完全对偶：把条件换成 $C_{t-1} \geq F_{t-1}^L$ ，规则退化为“取大者”， $\min$ 换成 $\max$ ， $\leq$ 换成 $\geq$ ，逐字重复上述推理即可。

(iii) 先证 TR 有界。因为 H_t 、 L_t 、 C_{t-1} 都落在 $[P_{\min}, P_{\max}]$ 内，三个候选量的绝对值都不超过 R ：

$$H_t - L_t \leq P_{\max} - P_{\min} = R, \quad |H_t - C_{t-1}| \leq R, \quad |L_t - C_{t-1}| \leq R. \quad (16.16)$$

三者取最大仍不超过 R ，故 $\text{TR}_t \leq R$ ；另一侧 $\text{TR}_t \geq 0$ 由 $H_t \geq L_t$ 以及两个绝对值非负得到。

再证 ATR 有界，用归纳法。基例 $t = k$ ： $\text{ATR}_k = \frac{1}{k} \sum_{j=1}^k \text{TR}_j$ 是 k 个落在 $[0, R]$ 内的数的算术平均，仍落在 $[0, R]$ 内。

归纳步：设 $\text{ATR}_{t-1} \in [0, R]$ 。递推式可写成

$$\text{ATR}_t = \frac{k-1}{k} \text{ATR}_{t-1} + \frac{1}{k} \text{TR}_t, \quad (16.17)$$

这是 ATR_{t-1} 与 TR_t 的一个凸组合：两个系数 $\frac{k-1}{k}$ 与 $\frac{1}{k}$ 都非负，且和为 $\frac{k-1}{k} + \frac{1}{k} = 1$ 。区间 $[0, R]$ 是凸集，凸组合仍落在其中，故 $\text{ATR}_t \in [0, R]$ 。

最后由 (i) 直接得到 $0 \leq B_t^U - B_t^L = 2m \text{ATR}_t \leq 2mR$ 。 ■

三条结论各有一个工程含义，我用三句话说完。

(i) 是自适应。宽度 $2m \text{ATR}_t$ 随波动率缩放。一只日均振幅 2% 的股票和一只 8% 的股票，用同一组 $(k, m) = (10, 3.0)$ ，前者通道自动窄、后者自动宽。所以参数 m 是倍数，不是价格区间——这跟“设一个 5% 的止损”是两种完全不同的东西：后者在低波动股票上形同虚设，在高波动股票上天天被打掉。

(ii) 是棘轮。没有被突破的上轨只能往下走。这带来一个副作用，我直说：趋势走得越久、通道收得越紧，一次像样的回调就足以翻空。所以它同时兼任两件事——顺势持有，以及自动上移的止损位（多头持仓时跟踪线取的是下轨 $M_t - m \text{ATR}_t$ ）。

(iii) 是防失控。ATR 是平均值，一次一字板带来的巨大 TR 只会以 $1/k$ 的权重进来。 $k = 10$

时权重是 0.1，不会被一根 K 线永久带偏。

还有一个细节属于工程层面，但它直接影响“第一笔交易什么时候发生”：ATR 在 $[0, k)$ 区间是零占位， $i = k$ 才是第一个有效值。所以超级趋势策略在 $i < k$ 时一律跳过；而在 $i = k$ 这一根上，如果方向已经是多头，就直接补一个买入信号——因为引擎只在收到买入信号时才建仓，若不补这一下，一段上来就是多头的行情会被整段错过⁴。

16.5 它在系统里长什么样

先把十七个策略摆成一张表。

表 16.1 17 个内置策略：族、类型、默认参数与信号方向

族	策略类型	默认参数	信号逻辑（买 → 卖）
趋势	ma_cross	15, 20	MA15 上穿 MA20 → 下穿
趋势	expma_cross	12, 50	EMA12 上穿 EMA50 → 下穿
趋势	dmi_trend	14, 25	PDI 在 MDI 上方且 ADX 超过 25 → 反向
趋势	trix_cross	12	TRIX 上穿自身均线 → 下穿
趋势	super_trend	$k = 10, m = 3.0$	方向由空翻多 → 由多翻空
趋势	hma	20	收盘上穿 HMA → 下穿
趋势	aroon	25	AroonUp 上穿 AroonDown → 下穿
动量	mtm_momentum	30	MTM 上穿 0 → 下穿 0
动量	zhuoyao_momentum	20/90/180	多周期共振转强 → 转弱
反转	rsi_reversal	14, 30/70	RSI 低于 30 → RSI 高于 70
反转	bollinger_breakout	20, 2.0	收盘跌破下轨 → 突破上轨
反转	kdj_golden	9/3, 30/70	K 上穿 D 且 J 低于 30 → K 下穿 D 且 J 高于 70
量价	mfi_volume	14, 30/70	MFI 低于 30 → MFI 高于 70
量价	cmf	20, ± 0.05	CMF 上穿 +0.05 → 跌破 -0.05
突破	turtle_breakout	40, 5	收盘破 40 日高点 → 破 5 日低点
突破	cci_breakout	14, ± 100	CCI 上穿 +100 → 下穿 -100
AI	ml_model	5 模型集成	集成概率不低于 0.5 → 不高于 0.3

“族”是本书的归纳，代码里没有这个字段；类型名取自 `BuiltinStrategies` 的 `StrategyType`，参数取自同处 `ConfigJSON`。CMF 的 ± 0.05 之间是观望带，不产生任何信号。

数据来源：系统事实清单第 10 节的内置策略表，与 `internal/backtest/strategies.go` 的 `BuiltinStrategies`、`GetStrategyByType` 逐条对照；核对时间 2026-09-24。

表里的“17”不是估出来的，是数出来的：`BuiltinStrategies` 这个切片里恰好有 17 条记录，它也是策略目录的单一数据源⁵。

单一数据源的意思是：策略的名字、类型、说明、默认参数、止损止盈，都只在这一个地方写一遍，其它地方读它。这样“系统里到底有几个策略”这个问题只有一个答案。

⁴数据来源：系统源码 `internal/backtest/strategies.go` 的 `SuperTrendStrategy.GenerateSignals`： $i < \text{period}$ 跳过， $i == \text{period}$ 时若 `trend[i] == 1` 补 `SignalBuy`。核对时间 2026-09-24。

⁵数据来源：系统源码 `internal/backtest/strategies.go` 的 `BuiltinStrategies`（共 17 条）与 `IsBuiltinStrategyType`。核对时间 2026-09-24。

16.5.1 新增一个策略，要同时动三处

单一数据源管住了“策略是什么”，但“策略怎么被用”还牵涉三张表。加一个策略，要同时在这三处登记：

```

1 // 1) 构造实例: internal/backtest/strategies.go  GetStrategyByType
2 case "super_trend":
3     return &SuperTrendStrategy{Period: 10, Multiplier: 3.0}
4
5 // 2) 参数映射: internal/backtest/strategy_config.go  strategyParamFieldMap
6 //     (JSON 配置字段名 -> 策略结构体字段名)
7 "super_trend": {"period": "Period", "multiplier": "Multiplier"}
8
9 // 3) 调参扫描范围: 同文件  defaultTuneRanges
10 "super_trend": {
11     "period":    {Type: "int",   Min: 8,   Max: 20,  Step: 2},
12     "multiplier": {Type: "float", Min: 2.0, Max: 4.0, Step: 0.5},
13 }
```

代码 16.2 新策略的三处注册（以超级趋势为例，均为节选）

这三处的分工是：`GetStrategyByType` 决定“能不能造出这个策略”；`strategyParamFieldMap` 决定“配置文件里的字段能不能落到结构体上”；`defaultTuneRanges` 决定“参数网格扫描时这个参数在什么范围里试”。

少任何一处，症状都很隐蔽：少了第一处，策略选中了却跑不起来；少了第二处，默认参数被数据库里的值悄悄覆盖掉、你却以为改了；少了第三处，回测能跑但“策略调参”这个功能对它视而不见。

后加入的四个策略（超级趋势、Aroon、HMA、CMF）三处都已登记⁶。

16.5.2 指标为什么自己写

超级趋势、Aroon、HMA、CMF、以及它们依赖的 ATR、WMA，全部写在 `internal/backtest/indicators.go` 里，没有引用外部指标库⁷。

这不是省事，也不是图快。理由有两条，第二条比第一条重要：

第一条是许可证。指标库的许可证可能对分发有约束，离线打包的桌面程序尤其要注意。

第二条是公式可审计。指标是这本手册里所有数字的源头，它必须能被读懂、能被指出来。所以系统里的每个指标都是**纯函数**：输入一个切片，输出一个等长切片，不持有状态、不访问数据库、不依赖调用顺序。这让同一套指标能被回测、因子计算、盘面分析三处共用，而任何一处发现异常，都可以单独拿一组数据去核对。

⁶数据来源：系统源码 `internal/backtest/strategies.go` 的 `GetStrategyByType`、`internal/backtest/strategy_config.go` 的 `strategyParamFieldMap` 与 `defaultTuneRanges`。核对时间 2026-09-24。

⁷数据来源：系统源码 `internal/backtest/indicators.go` 文件头的注释：与公开领域的标准指标公式思路一致，但按本方切片接口自实现，不引入 AGPL-3.0 依赖。核对时间 2026-09-24。

16.6 常见误解与纠偏

常见误解·策略有好坏，看回测收益排个序就行 这是最省事、也最容易被现实打脸的想法。

同一个策略在不同环境里的表现可以完全相反。双均线在单边趋势里很漂亮——它只要“上车”然后一直坐着；可它在震荡市里会反复被假交叉打脸，一次行情来回磨三遍，手续费和滑点全交出去了。布林带正好相反：震荡里高抛低吸得很舒服，趋势里一突破就站在了错误的一边。

所以正确的问法不是“哪个策略最好”，而是“**这个策略适合哪一种行情**”。

系统的做法跟这个想法是一致的：它不指定一个“永远最好”的策略，而是每天重新选一次——按 $\text{StrategySelectionScore} = 0.4 \times \text{总收益} + 0.3 \times \text{夏普} + 0.3 \times \text{胜率}$ 在活跃策略里选优，写进当日的策略计划^a。

这个选法本身是个启发式（它赌“最近的相对强弱会延续一段”），但它承认了本章最重要的那个直觉：**没有永远最好的策略，只有当天更合适的策略**。

^a数据来源：系统事实清单第 10 节：select_next_day_strategy 未指定类型时按该加权分自动选优，写 daily_strategy_plans。核对时间 2026-09-24。

常见误解·自己写指标，是重复造轮子 我一开始也这么觉得：ATR、CMF 这些指标到处都有现成实现，抄一个过来不是更省时间？

读完系统的指标实现我才明白，自己写的价值不在于“实现”，而在于**每一个边界条件都是自己决定并写明理由的**。举两个真实例子：

例子一：KDJ 预热期的 RSV 默认取 50 而不是 0。窗口未满足时算不出 RSV，如果按零值处理，那么这个 0 会被卷进 K、D 的平滑递推里，导致指标初期异常、信号失真。所以系统显式地把预热期的 RSV 设成中性的 50，并把理由写在注释里^a。

例子二：CMF 的预热区恰好“无害”，但仍然显式跳过。 $i < \text{period} - 1$ 时 CMF 返回 0，而 0 正好落在 $[-0.05, +0.05]$ 的观望带里，所以即便不跳过也不会误发信号。系统的选择是仍然在循环里显式 continue^b。

为什么不利用这个巧合？因为**巧合不是设计**。今天参数是 ± 0.05 ，明天调参调到 ± 0.03 ，或者阈值被改成 0，这个“无害”立刻变成“每天开盘就发信号”。把正确性建立在“当前参数恰好让它无害”上，是借来的正确。

^a数据来源：系统源码 internal/backtest/indicators.go 的 KDJ: for i := 0; i < n-1; i++ { rsv[i] = 50 } 及紧随其后的注释。核对时间 2026-09-24。

^b数据来源：同上文件 CMF 与 internal/backtest/strategies.go 的 CMFStrategy.GenerateSignals: if i < period-1 { continue }。核对时间 2026-09-24。

常见误解·信号出来了，就该按信号做 策略只输出 $-1/0/+1$ ，它对“多少钱、多少仓位、买不买得进”一无所知。

同一个买入信号，落在不同的市场环境里，后果完全不同。如果当天的六维判势给的是“退潮风险”、仓位系数 0.06，那么这笔订单会被缩到几乎可以忽略的大小。你会在成交记录里看到一笔很小的买单，然后以为“策略失效了”。

它没有失效。是决策层在干活：信号回答“什么时候”，仓位系数回答“买多少”，风控回答“能不能买”。这三件事分属三个模块，把它们混在一句话里，就会得出错误的结论。第 十九 章会专门讲中间那层。

16.7 真实出处

本章的真实出处

- 策略接口与信号常量 (*Name*、*GenerateSignals*、*SignalNone/Buy/Sell*) : *internal/backtest/strategies.go* 的 *Strategy* 接口
- 十七个内置策略的单一数据源 (名称、类型、说明、默认 *ConfigJSON*、止损止盈) : 同文件的 *BuiltinStrategies* 与 *IsBuiltinStrategyType*
- 各策略的构造与默认参数: 同文件的 *GetStrategyByType*
- 参数映射与调参扫描范围: *internal/backtest/strategy_config.go* 的 *strategyParamFieldMap*、*defaultTuneRanges* 与 *GetDefaultTuneRanges*
- 指标自实现 (*MA*、*EMA*、*ATR*、*WMA*、*HMA*、*Aroon*、*CMF*、*SuperTrend*) : *internal/backtest/indicators.go*
- 预热期处理: *super_trend/aroon/hma* 的 $i < period$ 、*cmf* 的 $i < period-1$ 、*KDJ* 的 *RSV* 中性初值
- 每日策略选优与落库 (*StrategySelectionScore*、*daily_strategy_plans*): 系统事实清单第 10 节所述的 *select_next_day_strategy*

动手试一试 (二十分钟)

1. 打开 *internal/backtest/strategies.go*，数一数 *BuiltinStrategies* 里到底有多少条记录。数出来的数字应当是 17——如果不是，说明有人加了策略却没同步这张表。
2. 在同一个文件里搜 "super_trend"，看它在 *BuiltinStrategies* 与 *GetStrategyByType* 里各出现一次。再到 *internal/backtest/strategy_config.go* 里搜一次，它应当出现两次 (参数映射、调参范围)。三处齐了，才算登记完整。
3. 拿一张纸，把表 16.1 里 17 个策略按“比较两条线”和“和阈值比”分两堆。你会发现有一些可以从一堆挪到另一堆——能挪说明它们本质相同。
4. 用 eq. (16.3) 手算 $F = 15$ 、 $S = 20$ 时的 20 个系数：前 15 个是 $1/15 - 1/20 = 1/60$ ，后 5 个是 $-1/20$ 。把它们相加，验证得到 0；再乘上任意常数 a 相加，验证还是 0。这就是水平位移不变性的全部机制。
5. 对超级趋势做一次“参数敏感度”手算：取某根 K 线的 $ATR_t = 0.42$ ，分别用 $m = 2.0$ 和 $m = 4.0$ 算通道宽度 $2m ATR_t$ 。宽度翻倍，但中线一动不动——这就是“乘倍数而不是设价格”的直观含义。

到这里，“什么时候买、什么时候卖”这件事已经拆开了：十七个策略只是十七套公式加两种比较方式，接口统一、实现可插拔，指标自己写、边界自己定。

可是还有一个问题没回答。我在本章开头想让十七个策略排个序，排不出来；现在我知道它们只是形状探测器，可形状探测器也需要被检验。

怎么知道一个策略在这个环境里到底行不行？

答案只有一个：回测。而回测恰恰是整本书里最容易骗人的一件事——它的假象主要不是来自数据本身，而是来自你没算进去的东西。下一章就来讲，回测到底在验证什么。

回测到底在验证什么？——成本、前视与样本池

17.1 年化 60% 的策略，实盘第一周亏了 3%

二零二六年九月七日，星期一。我把上一章挑出来的那条双均线策略塞进回测框，按下运行。

四十分钟后，屏幕上给出的成绩单是：年化收益 61.4%、夏普 2.31、最大回撤 12.0%、胜率 70.8%、总成交 348 笔。曲线漂亮得像一条被熨过的斜线。

我把它接进了模拟盘。九月八日到九月十二日，五个交易日，账面累计 -3.1%。

一根 K 线都没改，一个参数都没动。同一套规则，在“历史”里是印钞机，在“这五天”里是碎钞机。

我一开始的反应很自然：运气不好。第二反应更自然：市场变了。这两个解释都不可检验，所以都等于没解释。

真正有用的那个解释，是我把回测的账本一栏一栏摊开之后才看出来的。摊开之后我发现，我在“历史”里赚的那 61.4%，有相当一部分来自三笔我从来没付过的钱：

1. 摩擦没算：我在回测里把成交当成“点一下就成了”，没扣佣金、没扣印花税、没扣自己下单把价格顶上去的那部分；
2. 未来偷看：我用当天的收盘价算信号，又用当天的收盘价成交——这在 T+1 的市场里根本不可能；
3. 样本抽歪：我的“70.8% 胜率”是从一只股票的三笔交易里数出来的。

这一章就是把这三句话，一句一句写成可以被检验的东西。

17.2 台架上的发动机，和装在车上的发动机

直觉·实验室台架 vs 真实工地 一台发动机在台架上测出 300 马力。装到车上，跑了三个月，车主说“感觉没那么有劲”。

台架错了吗？没有。台架测的是发动机本身。车跑起来之后，马力要分给变速箱、水泵、空调压缩机，还要对抗风阻和轮胎滚动阻力。

差额不在发动机里，在发动机和地面之间——在那条从曲轴一直连到轮胎的传动链上。

回测里的成本模型，就是这条传动链的账本。它不加马力，它只告诉你：从曲轴出来的 300 匹，到轮胎上还剩多少。

回测这件事，很多人对它的期待是错的。他们把回测当成“预测器”：回测好，所以未来好。

它更像一个筛子，而且只干一件事：把那些明显错的策略筛掉。它的价值在于证伪，不在于证明。一个策略通过了回测，说明“它至少没有犯那些一眼就能看出来的错误”；它完全不说明这个策略下个月能赚钱。

这个定位一变，前面那句“实盘亏 3%”就不再是矛盾了：回测从来没有承诺过 61.4%，它只说“在扣掉我算的那几项成本之后，历史上是这样”。

所以要紧的问题只有一个：那几项成本，到底算了没有？

17.3 把摩擦写成公式：三因子成本模型

先说一个“差不多对”的版本：交易成本就是佣金，买卖各收万分之三。

这个版本抓住了三成。它漏掉的是三件在 A 股每天都真实发生的事：（1）佣金有**最低收费**；（2）卖出还要交**印花税**；（3）你的单子会把价格推走，这叫**市场冲击**。

系统在 `internal/backtest/cost_model.go` 里把这三件事写成了一项一项相加的成本函数。它的官方注释直接把对标对象写了出来：

$$\text{cost} = a \cdot |x| + b \cdot \sigma \cdot \frac{|x|^{1.5}}{\sqrt{V}} + c \cdot x, \quad (17.1)$$

其中 x 是这一笔的成交金额（元）， σ 是标的的日波动率， V 是当日的成交金额（元）。三项的含义分别是：

- $a|x|$ ：**比例费用**。A 股里就是佣金（买卖都收，最低 5 元）加上印花税（只在卖出时收 0.05%）；
- $b\sigma|x|^{1.5}/\sqrt{V}$ ：**市场冲击**。这叫 Almgren–Chriss 平方根冲击律；
- $c x$ ：**收盘偏置**。因为回测里信号和成交都用“收盘价”，口径一致时这项取 0。

系统的默认参数写在 `DefaultCostModel()` 里，我把它们抄成表 table 17.1。

表 17.1 系统的默认成本模型参数（A 股口径）

字段	默认值	名称	说明
<code>CommissionRate</code>	0.0003	佣金率	万分之三， 买卖都收
<code>MinCommission</code>	5	最低佣金	单笔不足 5 元按 5 元收
<code>StampDutyRate</code>	0.0005	印花税	0.05%， 仅卖出
<code>ImpactCoeff</code>	1.0	冲击系数 b	设为 0 即关闭冲击项
<code>ImpactExponent</code>	1.5	冲击指数	即式 eq. (17.1) 里的 1.5 次方
<code>BiasCoeff</code>	0.0	收盘偏置 c	收盘价成交且口径一致时为 0
<code>VolWindow</code>	60	波动率窗口	用最近 60 根 K 线估 σ

买入成本 = 佣金 + 冲击 + 偏置；卖出成本 = 佣金 + 印花税 + 冲击 + 偏置。

数据来源：系统源码 `internal/backtest/cost_model.go` 的 `DefaultCostModel()` 与 `BuyCost/SellCost`；核对时间 2026-09-24。

现在我要把表里第一个数字单独拎出来说，因为它藏着一件几乎所有小资金账户都会踩的事。

17.3.1 最低佣金：一条给小额交易设的税率

命题 17.1 (最低佣金抬高有效费率). 设佣金率为 a 、最低佣金为 M （元），一笔金额为 $x > 0$ 的成交，实收佣金为

$$\text{fee}(x) = \max\{ax, M\}. \quad (17.2)$$

记有效佣金率 $\alpha(x) = \text{fee}(x)/x$ 。则

- (i) 当 $x \geq x^* := M/a$ 时, $\alpha(x) = a$, 是个常数;
- (ii) 当 $0 < x < x^*$ 时, $\alpha(x) = M/x$, 关于 x 严格递减, 且 $\lim_{x \rightarrow 0+} \alpha(x) = +\infty$;
- (iii) 因此 α 在 $x = x^*$ 处取到最小值 a , 且 $\alpha(x) \geq a$ 对一切 $x > 0$ 成立。

这个命题在说什么：它把“最低 5 元”这句话翻译成了一条税率曲线。交易金额越小，你实际付的费率越高，而且可以高到离谱。代入系统参数： $a = 0.0003$ 、 $M = 5$ ，得阈值

$$x^* = \frac{5}{0.0003} = 16666.67 \text{ 元}. \quad (17.3)$$

表 17.2 最低佣金下的有效佣金率 ($a = 0.0003$, $M = 5$)

成交金额 x (元)	实收佣金 (元)	有效费率 $\alpha(x)$	相对名义费率
3,000	5.00	0.1667%	$5.56 \times$
5,000	5.00	0.1000%	$3.33 \times$
10,000	5.00	0.0500%	$1.67 \times$
16,667	5.00	0.0300%	$1.00 \times$
50,000	15.00	0.0300%	$1.00 \times$

数据来源：本书按 `internal/backtest/cost_model.go` 的 `fee(x)=max(0.0003x,5)` 复算；参数取自同文件 `DefaultCostModel()`。

证明. (i) 若 $x \geq M/a$, 则 $ax \geq M$, 于是 $\max\{ax, M\} = ax$, 除以 x 得 $\alpha(x) = a$ 。

(ii) 若 $0 < x < M/a$, 则 $ax < M$, 于是 $\max\{ax, M\} = M$, $\alpha(x) = M/x$ 。对 x 求导：

$$\alpha'(x) = -\frac{M}{x^2} < 0 \quad (M > 0, x > 0),$$

故在 $(0, x^*)$ 上严格递减；又 $\lim_{x \rightarrow 0+} M/x = +\infty$ 。

(iii) 由 (i)(ii), α 在 $(0, x^*)$ 上为 $M/x > x^*$ 的单调下降段, 在 $[x^*, \infty)$ 上恒为 a , 且左极限 $\lim_{x \rightarrow x^*-} \alpha(x) = M/x^* = a$ 。故 α 在 x^* 处连续且取最小值 a ；其余各处 $\alpha(x) \geq a$ 。■ ■

这条命题能直接解释一件事：为什么“把 10 万元拆成 20 只每只 5 千”这种看起来很稳健的分散，在成本上是灾难。每只 5,000 元，佣金率是名义费率的 3.33 倍；而卖出时还要再叠一层印花税。

17.3.2 平方根冲击律：为什么大单必须拆

现在看式 eq. (17.1) 里最不像“费用”的那一项。冲击项的形式是 $|x|^{1.5}/\sqrt{V}$ ，两个指数都不好看。但把它除以 x ，它会突然变得极干净。

定义 17.2 (参与率). 一笔金额为 x 的委托，相对于当日成交金额 V ，其参与率定义为

$$\rho = \frac{x}{V} \in (0, 1). \quad (17.4)$$

命题 17.3 (平方根冲击律). 在上述成本模型下, 令 $I(x) = b\sigma x^{1.5}/\sqrt{V}$ 为单笔冲击成本 (元), 则

(i) 单位金额的冲击成本为

$$\frac{I(x)}{x} = b\sigma\sqrt{\rho}, \quad (17.5)$$

即冲击成本率正比于参与率的平方根, 与 x 、 V 的绝对大小无关, 只通过 ρ 起作用;

(ii) 单调性: $I(x)$ 关于 x 严格递增; (17.5) 关于 ρ 严格递增且上凸 (凹函数);

(iii) 可加性尺度: 若把金额 x 与成交量 V 同时放大 λ 倍, 则冲击成本率不变、冲击成本放大 λ 倍;

(iv) 拆分定理: 把一笔金额 x 等分成 k 笔分别下单, 总冲击成本下降为原来的 $1/\sqrt{k}$ 。

这个命题在说什么: 第 (i) 条是整章的枢纽——真正决定冲击成本的, 不是“我买了多少钱”, 而是“我买了当日成交额的百分之几”。第 (iv) 条则回答了一个所有人都问过的问题: “大单为什么要拆着下?” 答案不是玄学, 是 $\sqrt{k}$: 拆成 4 笔, 冲击降到一半; 拆成 16 笔, 降到四分之一。

证明. (i) 由 $I(x) = b\sigma x^{1.5}/\sqrt{V}$ 得

$$\frac{I(x)}{x} = b\sigma \frac{x^{0.5}}{\sqrt{V}} = b\sigma \sqrt{\frac{x}{V}} = b\sigma\sqrt{\rho}.$$

(ii) 先看 I : $x^{1.5}$ 在 $x > 0$ 上严格递增, 故 I 严格递增。再看 $g(\rho) = b\sigma\sqrt{\rho}$: $\sqrt{\cdot}$ 在 $(0, \infty)$ 上严格递增且严格上凸 (二阶导 $-\frac{1}{4}\rho^{-3/2} < 0$); $b, \sigma > 0$ 不改变单调性与凸性。

(iii) 代入 $x \rightarrow \lambda x$ 、 $V \rightarrow \lambda V$:

$$\frac{I(\lambda x; \lambda V)}{\lambda x} = \frac{b\sigma(\lambda x)^{1.5}/\sqrt{\lambda V}}{\lambda x} = b\sigma \frac{\lambda^{1.5}x^{0.5}}{\lambda^{0.5}\sqrt{V} \cdot \lambda} = b\sigma\sqrt{\rho} = \frac{I(x; V)}{x},$$

即成本率不变; 而 $I(\lambda x; \lambda V) = \lambda x \cdot b\sigma\sqrt{\rho} = \lambda I(x; V)$, 成本放大 λ 倍。

(iv) 每笔金额 x/k , 由 (i) 每笔冲击成本为

$$\frac{x}{k} \cdot b\sigma \sqrt{\frac{x/k}{V}} = b\sigma \frac{x^{1.5}}{k^{1.5}\sqrt{V}},$$

k 笔相加得

$$k \cdot b\sigma \frac{x^{1.5}}{k^{1.5}\sqrt{V}} = \frac{b\sigma x^{1.5}}{\sqrt{V}} \cdot \frac{1}{\sqrt{k}} = \frac{I(x)}{\sqrt{k}}. \quad \blacksquare$$

注 17.4 (平方根律在哪里失效). 式 (17.5) 不是自然定律, 它是一条经验近似。三处会失效:

第一, V 是外生的。模型把当日成交金额 V 当成你不在场时的量。当 ρ 变大 (比如超过 10%), 你自己就是 V 的主要成分, V 会随你变化, 公式自相矛盾。

第二，冲击有永久与暂时之分。真实冲击里有一部分会随时间衰减回去，另一部分留在了价格里。这里把两者合并成一个瞬时项。

第三，拆分不是免费的。命题 17.3(iv) 只算冲击，没算拆单带来的时间风险和撤单/漏单概率。 $k \rightarrow \infty$ 时冲击趋零，但你的持仓暴露时间趋于无穷——所以最优拆分数是一个有内部最小值的优化问题，不是“越拆越好”。

17.3.3 把上面两笔账合起来：一次往返要付多少

现在算一次完整的“买入 → 卖出”。用九月二十三日库里那只真实标的做例子：sh600519，当日收盘 1251.24¹ 元，最近 60 个交易日对数收益率的标准差 $\sigma = 0.014433$ ²³。

假设我下单 50,000 元，当日成交金额取 $V = 1.33222 \times 10^9$ 元（该标的当日库内记录的 amount）。代入：

$$\text{买入佣金} = \max\{0.0003 \times 50000, 5\} = 15.00 \text{ 元}, \quad (17.6)$$

$$\text{买入冲击} = 1.0 \times 0.014433 \times \frac{50000^{1.5}}{\sqrt{1.33222 \times 10^9}} = 4.42 \text{ 元}, \quad (17.7)$$

$$\text{卖出佣金} = 15.00 \text{ 元}, \quad (17.8)$$

$$\text{卖出印花税} = 0.0005 \times 50000 = 25.00 \text{ 元}, \quad (17.9)$$

$$\text{卖出冲击} = 4.42 \text{ 元}. \quad (17.10)$$

往返合计 63.84 元，占本金的 0.1277%。

这个数单独看很小。但它乘上交易频率就变了性质：若这条策略一年往返 50 次，光成本就是 $0.1277\% \times 50 = 6.39\%$ 。你回测里看到的 61.4% 年化，是要先扣掉这 6.39% 的。

本章第一个结论

摩擦不是“扣几个点”，它是与交易频率相乘的项。

频率 × 单次成本率 = 年化成本。

所以一条高换手策略的真实门槛，比它的回测收益率低得多——低多少，取决于它一年来回多少次。

17.4 比成本更贵的错：偷看未来

成本是“少赚”，前视偏差是“假赚”。后者危险得多，因为它会让一个根本不成立的策略在回测里封神。

¹真实数据：本数字直接取自系统运行库 data/stock.duckdb，未做任何模拟或平滑。

²复算数据：该数字为本书按系统实现中的同一公式、在系统的真实数据上重新计算所得；公式出处已在正文标注。

³数据来源：只读查询 stock.ohlcv 中 sh600519 的前 60 根日线，按 $\sigma = \text{std}(\ln(P_t/P_{t-1}))$ 复算；查询日 2026-09-24。

⁴真实数据：本数字直接取自系统运行库 data/stock.duckdb，未做任何模拟或平滑。

直觉·翻到答案再考试 一个学生每次模拟考都接近满分。你翻他的卷子，发现他在“已给出答案”的那一页折了角。

他没有作弊——他只是把答案那页翻开了。而在真实考场上，那一页是密封的。

回测里的“当日收盘价成交”，就是那一页折角：你用一个当天的收盘价算出信号，又用这个收盘价成交。真实世界里，你要在收盘那一刻同时完成“看到收盘价”和“以这个价格成交”两件事，这在物理上不可能。

把这件事写严谨，需要一个词：**信息集**。

定义 17.5 (信息集与可测决策). 设 $P_t = (P_t^{\text{open}}, P_t^{\text{high}}, P_t^{\text{low}}, P_t^{\text{close}}, V_t)$ 为第 t 个交易日的完整行情向量。定义到第 t 日收盘为止的信息集

$$\mathcal{F}_t = \sigma(P_0, P_1, \dots, P_t), \quad (17.11)$$

即“由第 0 到第 t 日全部行情生成的信息”。若某个决策变量 D_t 是 $\mathcal{F}_t$ -可测的，就说它是**无前视**的；若 D_t 用到 $P_{t+1}, P_{t+2}, \dots$ 中的任何分量，就说它有**前视偏差**。

命题 17.6 (成交价口径决定有无前视). 设策略在第 t 日的信号为 $s_t = g(\mathcal{F}_t)$ 。比较两种成交口径：

(i) 次日开盘口径：成交价取 P_{t+1}^{open} ；

(ii) 当日收盘口径：成交价取 P_t^{close} 。

则 (i) 无前视偏差；而 (ii) 隐含了一个无法实现的假设——要求下单与成交发生在“观察到 P_t^{close} ”同一瞬间、以零延迟完成。

证明. (i) 次日开盘口径下，第 t 日产生的委托在第 $t+1$ 日开盘撮合。委托的全部输入 $s_t = g(\mathcal{F}_t)$ 是 $\mathcal{F}_t$ -可测的，成交价 P_{t+1}^{open} 出现在 $t+1$ 日，晚于决策时刻。决策与成交满足“先决策、后成交”的因果顺序，故无前视。

(ii) 若成交价取 P_t^{close} ，则成交时刻 = 收盘时刻 τ_t^{close} 。而 P_t^{close} 由收盘集合竞价在 τ_t^{close} 处产生——在 τ_t^{close} 之前， P_t^{close} 不属于任何参与者已知的信息。要以 P_t^{close} 成交，参与者必须在 τ_t^{close} 之前提交订单，而他提交订单时依据的 $s_t = g(\mathcal{F}_t)$ 中不含 P_t^{close} 。因此“用 P_t^{close} 算信号、又用 P_t^{close} 成交”这一对条件不能同时由同一次决策实现。■

系统里的做法是**强制**第一条口径。回测服务 `internal/backtest/service.go` 里有一个开关叫 `ExecNextOpen`，含义是“在下一根 K 线的开盘价撮合”。它的注释把理由写在明处：消除前视。同一层还按标的自适应涨跌停、并按**披露日**注入真实财务数据，防止“用年报给年报发布前的股票打分”。

常见误解·回测是预测，所以回测好 = 将来好 这是我最早、也是最贵的一个误解。

回测不能预测，它只能**证伪**。一个策略通过了回测，只说明“在我检查过的那些错误里，它没有犯”；它不说明这个策略在明天会赚钱。

把这两件事分清之后，一件让人难受的事就变得可以接受了：**回测好、实盘差，是常态而不是异常**。回测的价值在于它能把“明显不成立”的策略用很低的成本淘汰掉。它是一道初筛，不是一份承诺。

真正需要警惕的是相反的情形：回测特别好。一个年化远高于市场、回撤又特别小的结果，首先应该被怀疑有前视、有幸存者偏差、或者样本太少——本章余下的部分就是在教你怎么怀疑它。

17.5 胜率是怎么被一只股票骗出来的

现在讲第三笔账：样本。

九月七日那天，我看到的“胜率 70.8%”，分母是 348 笔成交——听起来很扎实。可是这 348 笔是从哪里来的？

我查了一下：它们全部来自一只股票。

问题不在于“一只股票能不能做回测”，而在于单只股票的成交笔数少、且互相关。当我们只在一只股票上跑一条策略，来回反复，样本之间的独立性很弱，而笔数又少到统计量没有意义。极端情况下，一只股票上只成交了三笔、三笔全赢，屏幕上就会显示“胜率 100%”。

系统的修复办法叫样本池回测，实现在 `internal/strategy/service.go` 的 `RefreshStrategyMetrics` 里。它的做法是：

1. 样本池优先取当前持仓（`position_status='open'` 的标的）；持仓为空则回退到基准指数 `sh000300`，再回退 `sh000001`；
2. 每个样本加载 `sampleLookback = 1500` 根 K 线（比单次回测长得多，目的就是让单只标的也能产生足量成交）；
3. 胜率不再按标的算，而是把所有样本的已平仓交易汇总起来算： $\text{aggWins}/(\text{aggWins} + \text{aggLosses})$ ；
4. K 线只加载一次，用有界 worker 池并行跑（并发数上限 8），避免重复读库。

第 3 条看起来只是“换个写法”，其实它决定了一个统计量有没有意义。下面把它写成命题。

命题 17.7 (交易聚合胜率与标的平均胜率不相等). 设样本池有 m 只标的。第 j 只标的的已平仓交易中，盈利 w_j 笔、亏损 ℓ_j 笔，总笔数 $n_j = w_j + \ell_j > 0$ 。定义

$$\hat{p}_{\text{交易}} = \frac{\sum_{j=1}^m w_j}{\sum_{j=1}^m n_j} \quad (\text{交易聚合胜率}), \quad (17.12)$$

$$\hat{p}_{\text{标的}} = \frac{1}{m} \sum_{j=1}^m \frac{w_j}{n_j} \quad (\text{标的平均胜率}). \quad (17.13)$$

则

- (i) $\hat{p}_{\text{交易}} = \hat{p}_{\text{标的}}$ 当且仅当所有 n_j 相等；
- (ii) $\hat{p}_{\text{交易}}$ 是“从全部成交中均匀随机取一笔，该笔为盈利”这一事件的无偏频率估计；而 $\hat{p}_{\text{标的}}$ 一般不是；
- (iii) 存在例子使 $\hat{p}_{\text{标的}} = 100\%$ 而 $\hat{p}_{\text{交易}}$ 远低于 100%。

这个命题在说什么：把胜率写成“哪个公式”不是风格问题。(i) 说这两个数只有在所有标的成交笔数完全一样时才相等——而现实中永远不会；(ii) 说交易聚合口径才是“随机抽一笔不会赢”的答案，这正是我们关心的那个量；(iii) 则给出了那个最刺眼的反例。

证明. (i) 记 $p_j = w_j/n_j$ 。把 $\hat{p}_{\text{交易}}$ 写成一个笔数加权平均：

$$\hat{p}_{\text{交易}} = \frac{\sum_j n_j p_j}{\sum_j n_j} = \sum_j \omega_j p_j, \quad \omega_j = \frac{n_j}{\sum_i n_i}.$$

而 $\hat{p}_{\text{标的}} = \sum_j \frac{1}{m} p_j$ ，是等权平均。两者都是 p_j 的凸组合，但权重向量不同： (ω_j) 对 $(\frac{1}{m})$ 。由均值恒等式

$$\hat{p}_{\text{交易}} - \hat{p}_{\text{标的}} = \sum_j \left(\omega_j - \frac{1}{m} \right) p_j,$$

该差为零对所有 (p_j) 成立，当且仅当 $\omega_j \equiv \frac{1}{m}$ ，即 n_j 全相等。

(ii) 把全部已平仓交易编号，令第 i 笔的盈利指示变量 $X_i \in \{0, 1\}$ 。全样本共 $n = \sum_j n_j$ 笔， $\sum_i X_i = \sum_j w_j$ 。若“随机抽一笔”是在 n 笔上均匀抽，则

$$\mathbb{P}(X = 1) = \frac{\sum_i X_i}{n} = \frac{\sum_j w_j}{\sum_j n_j} = \hat{p}_{\text{交易}},$$

故 $\hat{p}_{\text{交易}}$ 是该概率的样本频率。而 $\hat{p}_{\text{标的}}$ 是 p_j 的等权平均，只有在“先均匀随机选一只标的、再在该标的上随机抽一笔”这个抽样机制下才是无偏的——那是另一个问题。

(iii) 取 $m = 2$ ，标的 1 有 $w_1 = 3, \ell_1 = 0$ ；标的 2 有 $w_2 = 0, \ell_2 = 97$ 。则

$$\hat{p}_{\text{标的}} = \frac{1}{2} \left(\frac{3}{3} + \frac{0}{97} \right) = \frac{1}{2} (1 + 0) = 50\%,$$

$$\hat{p}_{\text{交易}} = \frac{3 + 0}{3 + 97} = \frac{3}{100} = 3\%.$$

更极端的 $m = 1$ 情形：单只标的成交 3 笔全赢，则 $\hat{p}_{\text{标的}} = \hat{p}_{\text{交易}} = 100\%$ ——分母只有 3，统计量本身没有意义。这正是修复前的失真来源。■

顺带说明：收益、夏普、回撤、换手这些路径型指标无法“合并成一笔”，系统的处理是对活跃样本取平均；而胜率之所以要特别处理，恰恰因为它是唯一一个“数笔数”的指标，最容易被单只股票的笔数结构污染。

17.6 把参数选好看的代价：walk-forward

最后一笔账，是“我怎么知道这组参数不是碰巧”。

直觉·在历年真题上找规律 一个学生在过去十年的真题上反复练习，把每道题的答案都背了下来。他再考同一套卷子，满分。
这不叫“学会了”，这叫在答案上过拟合。

要判断他有没有学会，唯一的办法是给他一套他从没见过的题——样本外测试。

参数寻优是同一回事：你在历史上试了一百组参数，选了成绩最好的那一组。那一组的成绩里，有一部分是“真实能力”，另一部分是“碰巧对了”。

这一步的偏差是可以精确写出来的。

命题 17.8 (最大值统计量的正偏). 设 K 组候选参数的真实表现 (如真实夏普) 为 $\theta_1, \dots, \theta_K$, 估计值为

$$\hat{\theta}_i = \theta_i + \varepsilon_i, \quad \mathbb{E}[\varepsilon_i] = 0, \quad i = 1, \dots, K, \quad (17.14)$$

其中 ε_i 为估计噪声。则

(i) $\mathbb{E}[\max_i \hat{\theta}_i] \geq \max_i \theta_i$;

(ii) 若噪声非退化 ($\text{Var}(\varepsilon_i) > 0$) 且 $\max_i \hat{\theta}_i$ 落在一个“并列”区间内, 则上述不等式严格成立;

(iii) 特别地, K 越大, 偏差 $\mathbb{E}[\max_i \hat{\theta}_i] - \max_i \theta_i$ 一般越大。

这个命题在说什么：它解释了为什么“我试了一百组参数，挑最好的”这个动作本身就会系统性地吧成绩抬高。抬高多少不由你的诚意决定，只由你试了多少组决定。

证明. (i) $\max_i(\cdot)$ 是 $\mathbb{R}^K$ 上的凸函数 (逐点取最大是凸的：对任意 i , $\max$ 是线性函数的逐点上确界，而线性函数既是凸也是凹，凸函数的上确界仍凸)。由 Jensen 不等式，对凸函数 f 有 $f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)]$ ，取 $X = \hat{\theta} = (\hat{\theta}_1, \dots, \hat{\theta}_K)$ ：

$$\max_i \mathbb{E}[\hat{\theta}_i] \leq \mathbb{E}[\max_i \hat{\theta}_i].$$

而 $\mathbb{E}[\hat{\theta}_i] = \theta_i$ (噪声零均值)，故左边 = $\max_i \theta_i$ ，得到 (i)。

(ii) 由 Jensen 不等式取等的条件：凸函数 f 上 $\mathbb{E}[f(X)] = f(\mathbb{E}[X])$ 当且仅当 X 在 $\mathbb{E}[X]$ 的每一个 f -活跃面上几乎必然取常值。对 $\max$ 函数，这意味着 $\hat{\theta}$ 几乎必然落在某个单一的“活跃面” (即最大值指标 $i^* = \arg \max_i \hat{\theta}_i$ 恒定) 上。若存在至少两个指标 $i \neq j$ 使 $\theta_i = \theta_j = \max_k \theta_k$ 且 $\text{Var}(\varepsilon_i - \varepsilon_j) > 0$ ，则 $\arg \max$ 会随噪声在 i, j 间跳动，不满足取等条件，于是不等式严格。

(iii) 记 $M_K = \max_{i \leq K} \hat{\theta}_i$ 。因为 $\max$ 关于指标集单调不减 ($M_{K+1} \geq M_K$ 逐样本成立)，故 $\mathbb{E}[M_{K+1}] \geq \mathbb{E}[M_K]$ ：加入新的候选参数不可能降低期望最大值。在候选参数彼此独立同分布、噪声非退化的常见情形下， $\mathbb{E}[M_K]$ 关于 K 严格递增。又由 (i) 得偏差 = $\mathbb{E}[M_K] - \max_i \theta_i \geq 0$ 且随 K 单调不减。■

系统的对策叫 **walk-forward 时间切分**，写在 `internal/backtest/grid.go` 里：

```
1 // walk-forward 时间切分：样本内前70%，样本外后30%
2 trainEnd := len(bars) * 7 / 10
3 train, test := bars[:trainEnd], bars[trainEnd:]
4
5 // 逐组回测：只在前 train 段评分、选参
6 for t := range jobCh {
7     r := runComboOnSegment(req.StrategyType, t.params, train, req, dm)
```

```
8     results[t.idx] = GridComboResult{ /* 记录 IsSharpe / IsReturn ... */ }
9 }
10
11 // 排序: 0 成交的组合一律排在后面 (防止选出 "零成交的虚假最优")
12 sort.SliceStable(results, func(i, j int) bool {
13     a, b := results[i], results[j]
14     if (a.IsTrades == 0) != (b.IsTrades == 0) { return a.IsTrades > 0 }
15     if obj == GridObjReturn { return a.IsReturn > b.IsReturn }
16     return a.IsSharpe > b.IsSharpe
17 })
18
19 // 只在样本外 (后30%) 报告被选中参数的真实表现
20 best := results[0]
21 oos := runComboOnSegment(req.StrategyType, best.Params, test, req, dm)
```

代码 17.1 walk-forward 切分与样本外验证 (节选自 internal/backtest/grid.go)

这段话在说什么：选参这件事只允许看到前 70% 的数据；被选中的那组参数，要拿到它没见过的后 30% 上去跑一遍，把那个成绩单单独报出来。表 table 17.3 是网格扫描的输出结构——注意它同时给出 BestIsSharpe（样本内）和 OosSharpe（样本外）两个数，这是刻意设计：不让你只看那个好看的数字。

表 17.3 网格扫描的输出字段 (GridScanResult)

字段	含义	为什么要有它
CombosTested	实际测试的组合数	命题 17.8(iii)：它越大，样本内成绩越不可信
TrainBars / TestBars	样本内 / 样本外根数	报告切分位置，可复现
BestParams	最优参数	只在样本内选出
BestIsSharpe	最优组的样本内夏普	好看但不作数
OosSharpe / OosReturn / OosMaxDD	样本外夏普 / 年化 / 回撤	唯一作数的成绩
Method	防前视方法说明	把“怎么防的”写进结果，方便审计

组合数超过 200 会被截断为前 200 组；扫描要求至少 40 根 K 线。

数据来源：系统源码 internal/backtest/grid.go 的 GridScanResult 与 scanGrid；核对时间 2026-09-24。

常见误解 · 用随机划分做训练/测试，或用全量数据 fit 归一化器 这是量化里最常见、也最隐蔽的一种自欺，所以我要先把它点名，下一章专门拆开。

随机划分：把全部历史打乱，随机分 70% 训练、30% 测试。对时间序列这是错的，因为打乱之后“未来”会流进训练集——你的模型见过它。正确做法是按时间切分。

全量 fit scaler：先在全样本上算均值和标准差，再分别标准化训练集和测试集。这看起来无害，但均值和标准差里已经包含测试集的信息，泄漏已经发生。

这两条合起来的效果是：回测指标会变得更漂亮，而不是报警。系统的事实清单里记录了同一个坑在 ML 策略上的完整症状（第 #26 条）——模型指标虚高、实盘失效。第 chapter 十八 章会给出它的形式化版本。

常见误解·样本池里多塞几只票就“更稳健”了 样本池的目的是让统计量有意义，不是“让曲线更好看”。

样本池回测优先取当前持仓，是因为持仓标的代表了你会实际交易的宇宙；它回退到指数，是因为持仓为空时也要有一个可算的对象。

需要留意的是聚合口径带来的一个副作用：当样本池里各标的的成交笔数极不均衡时，交易聚合胜率会被笔数多的那只主导。这不是缺陷——这正是命题 17.7 说的“交易加权”的正确含义——但你必须知道你在读的是“交易平均”还是“标的平均”，两者不等。

17.7 它在系统里长什么样

把本章的四个部件按调用顺序摆出来：数据入口统一（自动识别市场、优先 DuckDB、回退 TDX、日期过滤）→ 撮合口径强制 ExecNextOpen → 三因子成本逐笔扣减 → 参数扫描时做 walk-forward 切分 → 样本池聚合出胜率并回写策略表。

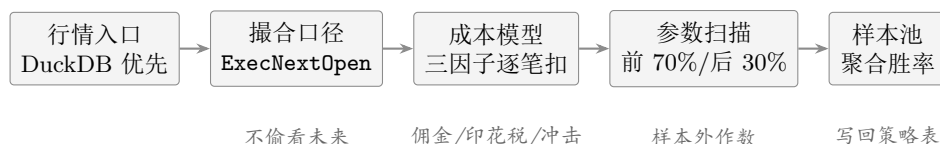


图 17.1 回测链路的五个环节。前三环决定“数字真不真”，后两环决定“数字算不算数”。

本章对应的真实代码

- 三因子成本模型与默认参数、逐笔买入/卖出成本：`internal/backtest/cost_model.go` 的 `DefaultCostModel`、`BuyCost`、`SellCost`、`impact`
- 次日开盘撮合与前视消除、按披露日注入财务、自适应涨跌停：`internal/backtest/service.go`
- `walk-forward` 参数网格扫描与样本外验证、0 成交兜底排序：`internal/backtest/grid.go` 的 `scanGrid`
- 样本池回测、1500 根回看、有界 `worker` 池、胜率聚合与指标回写：`internal/strategy/service.go` 的 `RefreshStrategyMetrics`
- 交易口径（买入仅佣金、卖出佣金 + 印花税）：`internal/portfolio` 与 `OrderCostTool`

动手试一试（二十分钟）

1. 打开回测页，选一条你熟悉的策略（比如双均线），跑一次。把结果页上的“年化收益”“夏普”“最大回撤”“胜率”四个数抄下来。
2. 在同一个页面上找到参数扫描入口，选目标 `sharpe`，让它跑。跑完对比两个数：`BestIsSharpe`（样本内）和 `OosSharpe`（样本外）。它们之间的落差，就是命题 17.8 的实测版本。
3. 去策略管理页，点一次「刷新」。观察“胜率”那一格有没有变化——如果它变了，说明你上一次看到的是单标的算出来的旧值，现在是样本池聚合值。
4. 现在做一道手算题：用表 table 17.2 的公式，算一算如果你把 10 万元分成 20 份、每份 5,000 元买入，佣金一共要付多少？再算只分 2 份、每份 5 万元，佣金一共多少？两次的差，就是“过度分散”的票价。
5. 最后想一想：把 50,000 元的单子拆成 4 笔，冲击成本降为 $1/\sqrt{4} = 1/2$ ；那拆成 100 笔是不是降为 $1/\sqrt{100} = 1/10$ 分之一？请回到注 17.4 第三条，说说为什么“越多越好”在这里不成立。

成本、前视、样本池——这三笔账算完，我至少知道我的回测“少算了几项”。可是还有一笔更大的账没算：那些看起来很聪明的模型。

第九章挑出来的那条机器学习策略，回测三个指标漂亮得不像话。下一章我们就去看它为什么经常打不过三条均线——以及那五个模型集成在一起，到底是真的更稳，还是只是把同一个错误抄了五遍。

机器学习的陷阱：五个模型为什么常常输给三条均线

18.1 五个模型，打不过一条均线

九月十日，我在策略管理页上按下「刷新」。

那条叫做 `ml_model` 的策略跑完了。结果页写着五个模型集成——XGBoost、LightGBM、随机森林、逻辑回归、多层感知机。名字排成一列，像一份简历。

同一张表里，紧挨着它的是 `ma_cross`（双均线交叉），全部参数只有两个数：短周期、长周期。

那次的表现是：集成模型没有赢。它输得也不算难看，只是平平淡淡地输——五个模型、一堆超参数、一整套训练流程，换来的是一条“股价上穿 20 日均线就买”也能拿到的成绩，有时还不如。

我盯着这个结果想了很久。因为“机器学习没用”和“机器学习有用、但我用错了”这两个结论，在结果页上长得一模一样。

后来我确认是后者。而且错的地方不在模型里，在模型外面那圈管道上。

18.2 用历年真题训练出来的学生

直觉·让一个学生刷十年真题 你给学生十年的历年真题，让他反复练。练完之后，你拿同一套卷子考他。他满分。

你会说“他学会了”吗？不会。你只会说“他把答案背下来了”。

要判断他到底有没有学会，唯一的办法是给他一套他没见过的题。

机器学习在量化里的全部困难，都压缩在这句话里：什么叫“没见过”？

金融数据有两个和图像、文本完全不同的性质。

第一，它有时间顺序。打乱它，就制造出了一个不可能存在的世界——在那个世界里，你可以用下个月的新闻解释这个月的价格。

第二，它的信噪比极低。手写数字识别里，像素和标签的关系几乎是确定的；而日频收益预测里，能被解释的那部分方差常常只有几个百分点。信噪比越低，“背答案”更容易发生，也越难被发现。

这两条合起来，就指向机器学习在量化里最容易走的那条岔路：**不是模型过拟合，是管道泄漏。**

18.3 泄漏是从哪儿漏进去的

先说一个“差不多对”的说法：过拟合就是参数太多、模型太复杂。

这个说法对图像模型基本成立。但在量化里，真正致命的泄漏往往发生在**模型还没开始训**

练之前——在你造特征的那一步。

系统的真实事故清单第 #26 条把症状写得很全：

真实事故 #26（摘录自系统事实清单）

症状：模型指标虚高、实盘失效。

根因：跨股票滚动窗口污染 + 无效标签混入 + 随机 `train_test_split` + `scaler` 用全量 `fit`（测试泄漏）。

修复：按 `symbol` 分组、时间切分、`scaler` 仅训练集 `fit`。

四件事里，后两件是同一个病：测试集的信息提前进入了训练过程。先把它定义清楚。

定义 18.1 (泄漏). 设数据按时间排序为 $T_1 < \dots < T_n$ 。学习流程是“预处理 φ （造特征、算统计量） $\rightarrow$ 在训练集 S_{tr} 上拟合模型，输出规则 h ”。若 h 对某个样本 i 的取值依赖晚于第 i 个样本的数据，或依赖测试集自身的标签，就说这个流程存在数据泄漏。

命题 18.2 (随机切分必然泄漏，时间切分不泄漏). 对时间有序数据 $T_1 < \dots < T_n$ ：

(i) 若把下标集合 $\{1, \dots, n\}$ 随机分成训练集 S_{tr} 与测试集 S_{te} ，两边都非空，则泄漏事件

$$\mathcal{L} = \{\min S_{te} < \max S_{tr}\} \quad (18.1)$$

的概率为

$$\mathbb{P}(\mathcal{L}) = 1 - \frac{1}{\binom{n}{m}}, \quad m = |S_{tr}|, \quad (18.2)$$

它随 $n \rightarrow \infty$ (m/n 趋于常数 $p \in (0, 1)$) 而趋于 1；

(ii) 若按时间切分 $S_{tr} = \{1, \dots, m\}$ 、 $S_{te} = \{m+1, \dots, n\}$ ，则 $\min S_{te} > \max S_{tr}$ ， $\mathcal{L}$ 不可能发生；预处理只使用 S_{tr} 时， h 对任意测试样本的取值仅依赖 T_m 之前的信息，无泄漏。

这个命题在说什么：随机切分在“样本独立同分布”的前提下是合法的，在时间有序的数据上不合法。(i) 说泄漏几乎必然发生，而且发生概率可以精确写成 $1 - 1/\binom{n}{m}$ ；(ii) 说按时间切分是关掉它的最直接办法。

证明. (i) 事件 $\mathcal{L}$ 的补集是 $\{\min S_{te} > \max S_{tr}\}$ ，即“ S_{te} 的每一个元素都大于 S_{tr} 的每一个元素”。在时间顺序下，这等价于“存在唯一的切分点 m ，使 $S_{tr} = \{1, \dots, m\}$ 、 $S_{te} = \{m+1, \dots, n\}$ ”。

从 $\{1, \dots, n\}$ 中随机取 m 个作为 S_{tr} ，共有 $\binom{n}{m}$ 种等可能取法（这里 $|S_{tr}| = m$ 固定）。满足上述条件的取法只有 1 种，即 $S_{tr} = \{1, \dots, m\}$ 。故

$$\mathbb{P}(\{\min S_{te} > \max S_{tr}\}) = \frac{1}{\binom{n}{m}}, \quad \mathbb{P}(\mathcal{L}) = 1 - \frac{1}{\binom{n}{m}}.$$

当 $m = \lfloor pn \rfloor$ 、 $p \in (0, 1)$ 固定时， $\binom{n}{m}$ 随 n 指数增长，故 $\mathbb{P}(\mathcal{L}) \rightarrow 1$ 。

(ii) 时间切分下，对任意 $j \in S_{tr}$ 有 $j \leq m$ ；对任意 $i \in S_{te}$ 有 $i \geq m+1 > m \geq j$ 。因为时间是递增的 ($T_1 < \dots < T_n$)，所以 $T_j < T_i$ 恒成立，从而 $\min S_{te} \geq m+1 > m \geq \max S_{tr}$ ， $\mathcal{L}$ 为不可能事件。

若 φ 的输入仅为 S_{tr} ，则 φ 的全部统计量都是 $T_1, \dots, T_m$ 的函数； h 对测试样本 i 的取值只额外用到第 i 个样本自身的同期特征。二者合起来不含任何 $T_{>m}$ 的信息，故无泄漏。■ ■

下面看泄漏能把成绩“修”得多漂亮。这个例子只有两行数学，但它就是事故 #26 的内核。

命题 18.3 (目标编码泄漏：没有信号也能做出满分)。设有 n 个样本 (X_i, Y_i) ， $Y_i \in \{0, 1\}$ ，且 $Y \perp X$ (即 X 对 Y 毫无预测力)。把样本 i 所属的类别记为 $c(i)$ ，类别总数为 k ，第 c 类的样本数记为 n_c 。用全样本做目标编码

$$\varphi_S(x_i) = \bar{Y}_{c(i)} := \frac{1}{n_{c(i)}} \sum_{j: c(j)=c(i)} Y_j, \quad (18.3)$$

再随手配一个阈值规则 $h(x) = \mathbf{1}\{\varphi_S(x) > 1/2\}$ 。则

- (i) 若 $k = n$ (每个样本自成一类)，则 $\varphi_S(x_i) = Y_i$ 对一切 i 成立， h 在任意测试集 (包括被随机划出去的那一半) 上精确率都等于 100%；
- (ii) 若 k 固定而 $n \rightarrow \infty$ ，则 $\varphi_S(x_i) \rightarrow \mathbb{E}[Y_i]$ ，规则退化为无信息，泄漏自行消失。

这个命题在说什么：第 (i) 条是一条完美的骗局——一个和结果毫无关系的特征，只要它的粒度足够细，就能让任何模型拿到满分。第 (ii) 条给出诊断口径：泄漏的严重程度，正比于“类别粒度相对于样本量的大小”。这也解释了为什么“按 symbol 分组”是关键修复：只要一只股票不跨越训练集与测试集，粒度就被摁住了。

证明。 (i) 若 $k = n$ ，则每个类别只含一个样本，即 $c(i)$ 类中唯一的元素就是 i 本身，故 $n_{c(i)} = 1$ ，求和式只有一项 $j = i$ ：

$$\varphi_S(x_i) = \frac{Y_i}{1} = Y_i.$$

于是 $h(x_i) = \mathbf{1}\{Y_i > 1/2\} = \mathbf{1}\{Y_i = 1\} = Y_i$ ，预测恒等于真值，在任意测试集上精确率都是 1。

(ii) 由 $Y \perp X$ ，同一类别内的 Y_j 与类别无关，是均值 $\mu = \mathbb{E}[Y]$ 、方差有限的独立同分布序列 (样本随机抽取且 X 与 Y 独立)。当 $n \rightarrow \infty$ 且各类样本数 $n_{c(i)} \rightarrow \infty$ 时，由强大数定律 $\bar{Y}_{c(i)} \rightarrow \mu$ 。于是 $\varphi_S(x_i) \rightarrow \mu$ ，阈值规则不再随 X 变化，退化为常数预测。■ ■

常见误解 · “模型指标很好看，说明它学会了” 这是机器学习在量化里最常见的一号自伤。

指标好看有四个来源：真实信号、泄漏、过拟合、样本太少。其中只有第一个是真的。而泄漏恰恰是唯一一个会让你越改进管道越严重的来源——特征工程做得越精细、粒度越细，漏得越狠。

所以看一个 ML 策略的结果，第一件事不是看准确率，而是问三句话：训练和测试是怎么切的？标准化统计量是在哪一段上算的？有没有任何一项特征是用全样本或未来算出来的？这三句里只要有一句答不上来，那个指标就不必看。

系统的做法是把它们全部钉死在代码里：按 symbol 分组、按时间切分、scaler 只在训练集上 fit、无效标签直接过滤。

18.4 五个模型合起来，为什么不一定更稳

泄漏讲完了，还有第二个问题：集成。

“五个模型投票”听起来天然比一个好。但集成能不能提高，取决于一件很少被明说的事：这五个模型彼此有多像。

命题 18.4 (等权重集成的方差缩减与它的天花板). 设 k 个模型的预测误差为 $e_1, \dots, e_k$, 满足

$$\text{Var}(e_i) = \sigma^2 > 0, \quad \text{Corr}(e_i, e_j) = \rho \quad (\forall i \neq j), \quad \rho \in [0, 1]. \quad (18.4)$$

则等权平均预测误差 $\bar{e} = \frac{1}{k} \sum_{i=1}^k e_i$ 的方差为

$$\text{Var}(\bar{e}) = \rho \sigma^2 + \frac{1-\rho}{k} \sigma^2. \quad (18.5)$$

于是

- (i) $\text{Var}(\bar{e})$ 关于 k 单调不减, 下确界为 $\rho \sigma^2$ ($\rho < 1$ 时取不到);
- (ii) 若 $\rho = 1$ (所有模型完全相同), $\text{Var}(\bar{e}) = \sigma^2$, 集成不带来任何收益;
- (iii) 若 $\rho = 0$ (误差不相关), $\text{Var}(\bar{e}) = \sigma^2/k$, 收益随 k 线性放大;
- (iv) 集成只能降低方差, 不能降低偏差。

这个命题在说什么：第 (i) 条是最该记住的一句——集成的效果有一个天花板，天花板就是 $\rho \sigma^2$ 。模型从 5 个加到 500 个也突破不了它。(ii) 说明为什么“把同一个模型训练五遍、换个随机种子”几乎没有用；(iii) 说明真正有用的集成，需要让模型犯不同的错。

证明. 先算两两协方差：对 $i = j$, $\text{Cov}(e_i, e_j) = \text{Var}(e_i) = \sigma^2$ ；对 $i \neq j$, 由相关系数定义 $\text{Cov}(e_i, e_j) = \rho \sqrt{\text{Var}(e_i) \text{Var}(e_j)} = \rho \sigma^2$ 。于是

$$\text{Var}(\bar{e}) = \frac{1}{k^2} \sum_{i=1}^k \sum_{j=1}^k \text{Cov}(e_i, e_j) = \frac{1}{k^2} [k\sigma^2 + k(k-1)\rho\sigma^2].$$

把括号内第二项拆开 $k(k-1) = k^2 - k$ ：

$$\text{Var}(\bar{e}) = \frac{k\sigma^2 + k^2\rho\sigma^2 - k\rho\sigma^2}{k^2} = \rho\sigma^2 + \frac{1-\rho}{k}\sigma^2,$$

即式 (18.5)。

(i) 把 k 视作连续变量求导：

$$\frac{\partial}{\partial k} \text{Var}(\bar{e}) = -\frac{(1-\rho)\sigma^2}{k^2} \leq 0,$$

故关于 k 单调不减；等号成立当且仅当 $\rho = 1$ 。又 $\frac{1-\rho}{k}\sigma^2 \rightarrow 0$ ($k \rightarrow \infty$)，故下确界为 $\rho \sigma^2$ ； $\rho < 1$ 时该项对每个有限 k 都严格为正，下确界取不到。

(ii) 代入 $\rho = 1$ ： $\text{Var}(\bar{e}) = 1 \cdot \sigma^2 + \frac{0}{k}\sigma^2 = \sigma^2$ ，与单个模型相同。

(iii) 代入 $\rho = 0$ ： $\text{Var}(\bar{e}) = \frac{\sigma^2}{k}$ 。

(iv) 期望是线性算子，故

$$\mathbb{E}[\bar{e}] = \frac{1}{k} \sum_{i=1}^k \mathbb{E}[e_i],$$

即各模型偏差的等权平均。无论 k 取何值，这个平均值都等于各偏差的算术平均，不可能因 k 增大而变为 0。■

把两件事放在一起看

集成的收益取决于 ρ ；而 ρ 又取决于五个模型有没有吃同一个泄漏。

若五个模型共享同一份被污染的特征、同一段被随机切分的数据，它们的错误就高度同向 ($\rho \rightarrow 1$)。于是你得到的是：五个模型，同一个错误，抄了五遍。——这正好解释了本章开头那张表。

18.5 它在系统里长什么样

系统里的 ML 策略叫 `ml_model`，注册在策略目录的单一数据源 `BuiltinStrategies` 里。它的配置原文是：

```
1 {
2     Name: "机器学习策略", StrategyType: "ml_model",
3     Description: "基于机器学习的AI选股策略：5模型集成（XGBoost/LightGBM/随机森林/
4                 逻辑回归/MLP）概率平均选股，纯Go本地推理无需Python；
5                 不可用时回退到多因子本地评分模型。",
6     ConfigJSON: `{"capital":100000,"max_position":10,"stop_loss":0.08,
7                 "take_profit":0.25,
8                 "model_names":["xgboost_astock_v1","astock_lgbm_v1","astock_rf_v1",
9                 "astock_logistic_v1","astock_mlp_v1"],
10                "ensemble_method":"probability",
11                "buy_threshold":0.5,"sell_threshold":0.3}`,
12     StopLossPct: 0.08, TakeProfitPct: 0.25,
13 },
```

代码 18.1 ML 策略的默认配置（节选自 `internal/backtest/strategies.go`）

这段话在说什么：`ensemble_method = "probability"` 就是命题 18.4 里那个“等权平均”——五个模型各自给出一个上涨概率，取平均，高于 `buy_threshold= 0.5` 买，低于 `sell_threshold= 0.3` 卖。中间那一段（0.3 到 0.5）是观望带，不产生任何信号。阈值不对称是刻意的：买入比卖出需要更高的把握，和第一部分讲的“只降不升”是同一条纪律。

第二个细节更值得单独说：描述里那句“纯 Go 本地推理无需 Python”。这不是技术炫耀，而是一个部署决定——若推理依赖一个 Python 服务，那么回测、实盘、策略刷新三条路径就得同时维护一套 Python 环境；一旦某条路径的模型文件路径不一致，就会出现事故 #25 那种情况：你重训了模型，但系统加载的还是旧的。

还有一条纯粹是“版本兼容坑”，但它花掉的时间一点不少。Python 侧训练脚本用的是 `sklearn 1.7.2`，而这一版的 `MLPClassifier` 不支持 `class_weight` 参数，直接写会抛 `TypeError`。修法是把类别权重换算成逐样本权重，改用 `fit(..., sample_weight=train_sw)`。这两件事

表 18.1 ML 策略在系统里的四个工程要点

环节	做法	它防的错
特征与标签	按 symbol 分组内单独计算	跨股票滚动窗口污染（一只票的信息漏进另一只）
训练/测试切分	时间切分，不随机划分	命题 18.2(i) 的泄漏
标准化	scaler 只在训练集上 fit	测试集统计量泄漏
标签	过滤掉无效标签后再训练	用不可交易日的样本训练
集成	5 模型概率平均 + 观望带	命题 18.4 的方差缩减（有天花板）

数据来源：系统事实清单第 15 节事故 #26；系统源码 `internal/backtest/strategies.go` 的 `ml_model`；核对时间 2026-09-24。

在数学上等价，在接口上一个能跑一个不能——而它们造成的现象（训练脚本报错退出）和模型问题（指标虚高）完全不同，容易被混为一谈。

常见误解·“机器学习”是均线的升级版 “用了 XGBoost，所以比均线强”——这个推论在量化里几乎总是错的。

均线策略是一个低方差、有偏的估计器：它的假设很硬（趋势会延续），所以容易在特定环境下系统性犯错，但很少给出荒诞结果。ML 模型是一个高方差、低偏的估计器：理论上能拟合任意形状，代价是它对管道里的每一处细节都极度敏感——包括那些你自己都没意识到的泄漏。

在信噪比极低的日频数据上，这个组合的典型结果就是：ML 在回测里赢，在实盘里输。原因不是 ML 不行，而是它的高方差特性正好会放大你管道里的每一个瑕疵。

本书不反对用 ML。它反对的是：在没把时间切分、分组、标准化、标签过滤这四件事做完之前，就去看 ML 的指标。

常见误解·集成就是多训练几个模型 “我训练十个模型平均一下，肯定更稳。”

回到命题 18.4(ii)(iii)：如果这十个模型用同一份数据、同一套特征、同一个标签定义，只是换了随机种子或树的棵数，那么它们犯的错高度同向， $\rho \rightarrow 1$ ，方差几乎不下降。你付出的是十倍的训练时间，换来的是一个几乎相同的预测。

真正能降低 ρ 的做法，是让模型从不同的角度看同一件事：不同的信息子集、不同的损失函数、不同的预测目标。系统里那五个模型（树模型、线性模型、神经网络）之所以被放在一起，正是因为它们的归纳偏置确实不同——但如果你给它们的特征完全一样、标签完全一样，这份差异就会被压扁。判断集成有没有用，不看模型数量，看它们预测的相关性。

18.6 最后一句：它失败了会怎样

补一句容错。`ml_model` 的描述里有一句容易被跳过的话：“不可用时回退到多因子本地评分模型”。

意思是：如果模型文件加载失败，这条策略不会静默返回“今天没有信号”，而是退到一个可解释的、由多因子加权构成的本地评分模型上。

这和第一部分第 chapter 九 章是同一条原则：AI 的单点故障不得阻断交易。区别只在于，第 9 章讲的是“整个 AI 决策层失联”，这里讲的是“其中一个模型加载不出来”。处理方式一样——换一条已知的、确定性的路走，而不是停在原地，也不是编一个结果出来。

本章对应的真实代码与事实

- *ML* 策略定义、五模型清单、集成方式与买卖阈值: *internal/backtest/strategies.go* 的 *BuiltinStrategies* 中 *ml_model* 条目
- 策略构造与模型加载路径: *internal/backtest/strategies.go* 的 *GetStrategyByType*; 模型加载 *models.LoadMLModelByName*
- 数据泄漏四项修复 (按 *symbol* 分组、时间切分、*scaler* 仅训练集 *fit*、无效标签过滤): 系统事实清单第 15 节事故 #26
- *sklearn 1.7.2* 的 *class_weight* 兼容坑与 *sample_weight* 替代: 事故 #24
- 训练产物命名与加载路径必须一致: 事故 #25
- 回测撮合与成本口径: *internal/backtest/service.go*、*internal/backtest/cost_model.go* (见第 *chapter 十七* 章)

动手试一试 (十五分钟)

1. 打开策略管理页, 找到 *ml_model* 那一行, 展开参数。数一数 *model_names* 里有几个名字, 核对是不是 5 个。
2. 看 *buy_threshold* 与 *sell_threshold* 两个数: 0.5 与 0.3。自己回答: 0.3 到 0.5 之间会发生什么? (提示: 那是一个不产生信号的观望带。)
3. 对同一条 *ml_model* 和一个简单的 *ma_cross*, 各跑一次同区间回测, 把样本外指标抄下来并排看。不要看样本内。
4. 做一道手算题: 设五个模型的误差方差都是 $\sigma^2 = 1$, 两两相关系数 $\rho = 0.9$ 。用式 (18.5) 算集成后方差; 再把 ρ 换成 0.1 算一遍。两次之差, 就是“让模型犯不同的错”值多少钱。
5. 最后回答一个判断题: 把 5 个模型换成 50 个, 集成方差能不能降到 0? 回到命题 18.4(i) 说出理由。

到这里, 单只股票上的事——因子、权重、检验、策略、成本、回测、模型——都讲完了。可是上面所有结论都建立在一个隐含前提上: 今天这个市场, 值不值得下注。

同一只股票、同一个信号, 在牛市该买, 在退潮期就不该买。下一章我们去看系统怎么把“今天”本身也算成一个数——市场六维判势。它也是全书里我唯一能给你看五条完整真实记录的地方: 从原始分到冲突修正, 再到最终仓位系数, 每一步都可以手工验算。

第十九章

今天能不能下注？（上）——市场六维判势怎么打分

19.1 同一只股票，两只手

九月十日，星期四，开盘前。

如果只看单只股票，这一天我挑出来的几只和昨天挑出来的几乎是同一批——因子分没变多少，策略信号也没翻。可是系统给我的仓位系数是 0.10¹²。

0.10 是什么概念？它意味着：就算选股层挑出了一只“满分”的股票、就算策略层说了“买”，这次买入的名义金额也只能是它原本该是的一成。

我第一反应是“这也太胆小了”。第二反应是把两天的记录并排摆出来看，然后我就不说话了：

- 八月三十一日，仓位系数 0.80³；
- 九月十一日，仓位系数 0.06⁴。

同一套选股逻辑、同一套策略、同一个人的账户。唯一变化的是今天这个市场本身。

这一章要讲的就是那件事：把“今天”变成一个 0 到 100 之间的数，再把那个数变成“允许下多少注”。

19.2 出门前为什么要看天气

直觉·没有“今天天气多少分” 早上出门前，你会看几样东西：温度、湿度、风、紫外线、空气质量、有没有预警。

你不会把它们合成一个“今天天气 73 分”，然后据此决定穿什么。可你确实会做三件很具体的事：**要不要带伞、穿不穿外套、走不走高架。**

这六样东西各自回答一个不同的问题：温度管保暖、湿度管体感、风管舒适度、紫外线管晒、空气质量管呼吸、预警管要不要干脆别出门。它们**不能互相替代**，也不需要合成一个总分才能用。

市场判势是同一件事：不是问“今天涨还是跌”，而是问**今天这个环境允许我下多少注**。

先说清楚它**不是什么**：六维判势不是涨跌预测。它输出的是仓位系数，不是方向。

这个区分极其重要，因为一个新手最容易犯的错就是把它当成“多空信号”：看到分数低于 40 就去做空，或者看到分数高就满仓。系统里对这两个用法都有硬拦：分数低了它只做一件事——** 把新的买入关掉 **，而不是反过来做空（A 股个人投资者也没有方便的低成本做空工具）；分数高了它也不会“加杠杆”，只是不再额外收缩。

¹真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

²数据来源：系统运行库 `market_sixdim_daily` 中 2026-09-10 的记录；核对时间 2026-09-24。

³真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

⁴真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

19.3 六个维度，六种问法

系统的市场六维判势（*MarketSixDim*）用的是六个维度、各自一个权重，权重之和为 1。定义写在 `internal/marketsixdim/sixdim.go` 里，我抄成表 table 19.1。

表 19.1 市场六维判势的六个维度与权重

key	维度	权重	它问的那个问题
tech	技术趋势	0.15	指数站上 20 日 / 60 日均线了吗，趋势是升是降
breadth	市场广度	0.20	上涨家数占比、新高新低比、涨跌停比、有几条主线
volume	量能流动性	0.15	成交额相对 20 日均是放是缩，价量配不配合
capital	资金结构	0.15	北向是否连续净流入，资金是集中还是到处游击
sentiment	情绪赚钱效应	0.25	涨停多少、炸板率高不高、非 ST 跌停几家
external	外部约束	0.10	隔夜外围与重大事件带来的开盘压力

权重之和为 1。情绪维度权重最大（0.25），因为它最接近“今天有没有赚钱效应”这件事本身。

数据来源：系统源码 `internal/marketsixdim/sixdim.go` 的 `DimWeights`、`DimOrder`、`DimChinese`；核对时间 2026-09-24。

每个维度的打分方式都是同一个套路：从 50 分出发，按证据加减分，最后截断到 $[0, 100]$ 。50 分是“中性”，不是“及格线”。这么做有一个实际的好处：一个维度缺数据时，可以什么都不加、什么都不减，它就停在 50——不会因为“今天没数据”而被误判为“今天很糟”。

举两个维度看形态。

19.3.1 技术趋势：三件事，加减各一档

$$s_{\text{tech}} = 50 + \underbrace{(\pm 15)}_{\text{站上/跌破 MA20}} + \underbrace{(\pm 20)}_{\text{站上/跌破 MA60}} + \underbrace{(\pm 15)}_{\text{趋势状态}\pm 1} \quad (19.1)$$

三档全对上（站上 20 日、站上 60 日、趋势向上）就是 $50 + 15 + 20 + 15 = 100$ ；全反就是 0。没有中间态被漏掉。

19.3.2 情绪赚钱效应：涨停加分，炸板扣分

$$s_{\text{sentiment}} = 50 + \min(\text{涨停家数}, 100) \times 0.3 - \min(\text{炸板率}, 1) \times 50 - \min(\text{非 ST 跌停数}, 10) \times 3. \quad (19.2)$$

注意三个 min：它们把每一项都封了顶。涨停再多，加分不超过 30；炸板率再高，扣分不超过 50；跌停再多，扣分不超过 30。这种“封顶”不是怕算错，是怕一个极端维度把总分彻底带走——第 chapter 十三 章讲权重时用的是同一条思路，只不过那里封的是权重，这里封的是分。

19.4 六个分数怎么合成一个总分

定义 19.1 (加权总分). 给定六个维度的得分 $s = (s_{\text{tech}}, \dots, s_{\text{external}}) \in [0, 100]^6$ 与权重

$$\begin{aligned} w_{\text{tech}} &= 0.15, & w_{\text{breadth}} &= 0.20, & w_{\text{volume}} &= 0.15, \\ w_{\text{capital}} &= 0.15, & w_{\text{sentiment}} &= 0.25, & w_{\text{external}} &= 0.10. \end{aligned} \quad (19.3)$$

定义原始总分

$$\text{raw}(s) = \sum_k w_k s_k. \quad (19.4)$$

命题 19.2 (加权总分的性质). raw 满足:

- (i) 有界性: $\text{raw}(s) \in [0, 100]$; 更精确地, $\min_k s_k \leq \text{raw}(s) \leq \max_k s_k$;
- (ii) 单调性: raw 关于每一个 s_k 严格递增 (偏导 $\partial \text{raw} / \partial s_k = w_k > 0$);
- (iii) 权重即边际: 第 k 个维度每提高 1 分, 总分提高 w_k 分。所以情绪维度的一句话, 抵得上外部约束的两句半 (0.25 对 0.10)。

证明. (i) 因 $w_k \geq 0$ 且 $\sum_k w_k = 1$ 、 $s_k \in [0, 100]$,

$$\text{raw}(s) = \sum_k w_k s_k \leq \sum_k w_k \cdot 100 = 100, \quad \text{raw}(s) \geq \sum_k w_k \cdot 0 = 0.$$

再看夹逼: 记 $s_{\min} = \min_k s_k$ 、 $s_{\max} = \max_k s_k$, 则对每个 k 有 $s_{\min} \leq s_k \leq s_{\max}$ 。乘以非负权重 w_k 后求和 (保序), 并用 $\sum_k w_k = 1$:

$$\sum_k w_k s_{\min} = s_{\min} \leq \text{raw}(s) \leq \sum_k w_k s_{\max} = s_{\max}.$$

(ii) 固定其余五个维度, 对 s_k 求偏导: $\partial \text{raw} / \partial s_k = w_k > 0$ (六个权重全为正), 故严格递增。

(iii) 直接由 (ii): s_k 增加 1, raw 增加 w_k 。比较两个维度: $w_{\text{sentiment}} / w_{\text{external}} = 0.25 / 0.10 = 2.5$ 。■

19.4.1 冲突修正: 当市场自己都不一致

原始总分算出来了, 但它还没被使用。系统在 **Evaluate** 里对它做了一次修正。

定义 19.3 (冲突计数与修正算子). 令

$$H(s) = |\{k : s_k \geq 75\}| \quad (\text{高分维度数}), \quad (19.5)$$

$$L(s) = |\{k : s_k \leq 40\}| \quad (\text{低分维度数}), \quad (19.6)$$

冲突计数定义为

$$c(s) = \begin{cases} H(s) + L(s), & H(s) > 0 \text{ 且 } L(s) > 0, \\ 0, & \text{否则.} \end{cases} \quad (19.7)$$

修正算子为

$$\text{adj}(s) = \begin{cases} 0.8 \cdot \text{raw}(s), & c(s) \geq 2, \\ \text{raw}(s), & c(s) < 2, \end{cases} \quad \text{最后截断到}[0, 100]. \quad (19.8)$$

这段话在说什么： H 数“几个维度很强”， L 数“几个维度很弱”。只有当两边同时不为零时，才算发生了冲突，冲突数就取两边相加；只要有一边是零，冲突数就是 0。修正很简单：冲突数到 2 或以上，总分打八折。

第一次读到这里，我以为这只是一个“缩表”的土办法。后来我发现它有一个必须被证明的性质，以及一个必须被承认的副作用。

命题 19.4 (修正算子的三条性质). 记 adj 如式 (19.8)。

- (i) 只降不升： $\text{adj}(s) \leq \text{raw}(s)$ 对一切 s 成立；等号成立当且仅当 $c(s) < 2$ 或 $\text{raw}(s) = 0$ ；
- (ii) 同冲突态内保序：固定 c ， adj 关于 raw 严格递增；
- (iii) 跨冲突态不保序，且逆序幅度有上界：存在 s, s' 使 $\text{raw}(s) < \text{raw}(s')$ 而 $\text{adj}(s) > \text{adj}(s')$ ；并且当 $c(s) < 2 \leq c(s')$ 时，逆序比满足

$$\frac{\text{adj}(s)}{\text{adj}(s')} = \frac{1}{0.8} \cdot \frac{\text{raw}(s)}{\text{raw}(s')} < \frac{1}{0.8} = 1.25. \quad (19.9)$$

这个命题在说什么：第 (i) 条是纪律——修正永远不会把分数抬高，它和第一部分第 chapter 七章“只降不升”是同一条设计原则，只不过对象从仓位换成了评分。第 (ii)(iii) 条是一条诚实：八折修正不是一个保序的评分变换，它会让一个原始分更高、但自相矛盾的市场，排在原始分更低、但内部一致的市场后面。而第 (iii) 条给出它的边界——被颠倒的两个市场，原始分最多差 25%。

证明. (i) 若 $c(s) < 2$ ，则 $\text{adj} = \text{raw}$ ，等号成立。若 $c(s) \geq 2$ ，则 $\text{adj} = 0.8\text{raw}$ 。因 $\text{raw} \geq 0$ ，得 $\text{adj} = 0.8\text{raw} \leq \text{raw}$ ；等号成立当且仅当 $0.8\text{raw} = \text{raw}$ ，即 $\text{raw} = 0$ 。

(ii) 固定 c 。若 $c < 2$ ， $\text{adj} = \text{raw}$ ，严格递增；若 $c \geq 2$ ， $\text{adj} = 0.8\text{raw}$ ，系数 $0.8 > 0$ ，严格递增。

(iii) 构造反例。取

$$s = (60, 60, 60, 60, 60, 60) \Rightarrow \text{raw}(s) = 60, H(s) = 0, L(s) = 0, c(s) = 0, \text{adj}(s) = 60.$$

再取

$$s' = (100, 100, 20, 20, 60, 60).$$

先算它的总分：

$$\text{raw}(s') = 0.15 \cdot 100 + 0.20 \cdot 100 + 0.15 \cdot 20 + 0.15 \cdot 20 + 0.25 \cdot 60 + 0.10 \cdot 60 = 15 + 20 + 3 + 3 + 15 + 6 = 62.$$

高分维度： $s'_{\text{tech}} = 100 \geq 75$ 、 $s'_{\text{breadth}} = 100 \geq 75$ ，故 $H = 2$ ；低分维度： $s'_{\text{volume}} = 20 \leq 40$ 、 $s'_{\text{capital}} = 20 \leq 40$ ，故 $L = 2$ 。两者都大于零， $c(s') = 2 + 2 = 4 \geq 2$ ，于是

$$\text{adj}(s') = 0.8 \times 62 = 49.6.$$

比较： $\text{raw}(s) = 60 < \text{raw}(s') = 62$ ，但 $\text{adj}(s) = 60 > 49.6 = \text{adj}(s')$ 。逆序成立。

上界：在 $c(s) < 2 \leq c(s')$ 的情形下， $\text{adj}(s) = \text{raw}(s)$ 、 $\text{adj}(s') = 0.8 \text{raw}(s')$ ，故

$$\frac{\text{adj}(s)}{\text{adj}(s')} = \frac{\text{raw}(s)}{0.8 \text{raw}(s')} = \frac{1}{0.8} \cdot \frac{\text{raw}(s)}{\text{raw}(s')}.$$

又由假设 $\text{raw}(s) < \text{raw}(s')$ 得比值 $\text{raw}(s)/\text{raw}(s') < 1$ ，从而整个比值 $< 1/0.8 = 1.25$ 。■ ■

八折修正到底在做什么

它不是一个更“准”的评分，它是一个**风险折扣**。
内部一致的市场（六维同向）拿原始分；自相矛盾的市场（既有人喊强、又有人喊弱）拿八折。
代价是：它会让排序出现最大 25% 的逆序。
这个代价是被接受的——因为一个自相矛盾的市场，本来就不配和一个内部一致的市场拿同一个分数。宁可排错，也不肯给矛盾高分。

19.5 用五条真实记录把它算一遍

上面全是公式。现在换一种讲法：打开系统运行库，把 `market_sixdim_daily` 里**全部五条记录**拿出来，一条一条手工验算。

这张表是判势结果的落库表，以交易日为主键。库里目前恰好有五条：

表 19.2 系统运行库中的市场六维判势真实记录（全部 5 条）

日期	tech	breadth	volume	capital	sentiment	external	raw	冲突	adjusted	标签
2026-08-31	60	55	50	45	52	50	52.30	0	52.30	结构性震荡
2026-09-02	100	18	50	60	50	46.78	52.28	2	41.82	偏弱
2026-09-09	45	64.20	52.80	80	25.40	46.45	50.50	2	40.40	偏弱
2026-09-10	85	8.69	52.80	80	35.85	46.71	48.04	4	38.43	退潮风险
2026-09-11	15	12.84	52.80	80	14.13	38.71	32.14	5	25.71	退潮风险

表中 **raw**、**冲突**、**adjusted** 三列是库中原样数值（`raw_total_score / conflict_count / adjusted_total_score`）；六个维度分取自 `dim_scores` 的 JSON。数值保留了库中精度，展示时统一到两位小数。
数据来源：只读查询 `market_sixdim_daily` 全部 5 条记录；查询日 2026-09-24。

现在验算。我用式 (19.4) 和式 (19.8)，逐个走一遍。

2026-09-02。六个分数 100, 18, 50, 60, 50, 46.778967965281055:

$$\begin{aligned} \text{raw} &= 0.15(100) + 0.20(18) + 0.15(50) + 0.15(60) + 0.25(50) + 0.10(46.778967965) \\ &= 15 + 3.6 + 7.5 + 9 + 12.5 + 4.6778967965 = 52.2778967965 \Rightarrow \mathbf{52.28} \checkmark \end{aligned} \quad (19.10)$$

高分: tech = 100 $\geq$ 75, 共 1 个。低分: breadth = 18 $\leq$ 40, 共 1 个。两者皆非零, $c = 1 + 1 = 2 \geq 2$, 故

$$\text{adj} = 0.8 \times 52.2778967965 = 41.8223174372 \Rightarrow \mathbf{41.82} \checkmark$$

2026-09-09。六维分数依次为 tech 45、breadth 64.2、volume 52.8、capital 80、sentiment 25.4、external 46.44938591807174:

$$\begin{aligned} \text{raw} &= 6.75 + 12.84 + 7.92 + 12 + 6.35 + 4.6449385918 \\ &= 50.5049385918 \Rightarrow \mathbf{50.50} \checkmark \end{aligned} \quad (19.11)$$

高分: capital = 80 $\geq$ 75, 1 个。低分: sentiment = 25.4 $\leq$ 40, 1 个。 $c = 2$, 故

$$\text{adj} = 0.8 \times 50.5049385918 = 40.4039508734 \Rightarrow \mathbf{40.40} \checkmark$$

2026-09-10。六维分数依次为 tech 85、breadth 8.692307692307693、volume 52.8、capital 80、sentiment 35.85、external 46.71042113410559:

$$\begin{aligned} \text{raw} &= 12.75 + 1.7384615385 + 7.92 + 12 + 8.9625 + 4.6710421134 \\ &= 48.0420036519 \Rightarrow \mathbf{48.04} \checkmark \end{aligned} \quad (19.12)$$

高分: tech 85、capital 80, 2 个。低分: breadth 8.69、sentiment 35.85, 2 个。 $c = 4$, 故

$$\text{adj} = 0.8 \times 48.0420036519 = 38.4336029215 \Rightarrow \mathbf{38.43} \checkmark$$

2026-09-11。六维分数依次为 tech 15、breadth 12.836363636363636、volume 52.8、capital 80、sentiment 14.133333333333333、external 38.71379548577629:

$$\begin{aligned} \text{raw} &= 2.25 + 2.5672727273 + 7.92 + 12 + 3.5333333333 + 3.8713795486 \\ &= 32.1419856092 \Rightarrow \mathbf{32.14} \checkmark \end{aligned} \quad (19.13)$$

高分: capital = 80, 1 个。低分: tech 15、breadth 12.84、sentiment 14.13、external 38.71, 共 4 个。 $c = 1 + 4 = 5$, 故

$$\text{adj} = 0.8 \times 32.1419856092 = 25.7135884874 \Rightarrow \mathbf{25.71} \checkmark$$

四条记录，逐位吻合。最后是那条对不上的。

注 19.5 (一条对不上的记录，以及为什么我不改它). **2026-08-31**. 分数 60, 55, 50, 45, 52, 50:

$$\text{raw} = 0.15(60) + 0.20(55) + 0.15(50) + 0.15(45) + 0.25(52) + 0.10(50) = 52.25, \quad (19.14)$$

而库里 `raw_total_score` 写的是 **52.30**。差 0.05。冲突数为 0 (没有 ≥ 75 的，也没有 ≤ 40 的)，所以 `adjusted` 应等于 `raw`——库里的 `adjusted_total_score` 正好也是 52.30，自洽。

但更值得说的是下一列：库里这一天的 `position_rate` 是 **0.80**、标签是**结构性震荡**。按当前代码的映射（下一节），52.30 落在 $[40, 60)$ 这一档，应该得到“偏弱”和 0.26 左右的仓位系数。两者对不上。

我核对过这一天的 `sources` 字段——它只写了 `"tech": "DuckDB 真实指数 K 线"` 一条，其余五个维度的来源**全部缺失**。而九月九日之后的记录，`sources` 每一维都有明确交代。

结论很清楚：八月三十一日这条是早期版本写进去的，那时的评分与映射口径与现在不同。

我不改它，也不替它圆场。理由和第一章讲的是同一件事：这条记录本身就是一个教材——一个系统跑起来之后，口径会变；如果口径变了但历史记录不带版本号，你三个月后再看这五行数字，就分不清哪条是“当时的真相”、哪条是“当时的实现”。这也正是后面第 chapter 二十章要专门讲“数据从哪来、缺数怎么办”的原因。

19.6 从分数到仓位：一张分段映射表

修正后总分要变成仓位系数。这一段实现在 `ScoreToPosition` 里，是 4 段线性映射，外加标签。

表 19.3 修正后总分 $\rightarrow$ 仓位系数 r_{pos} $\rightarrow$ 市场标签

区间	公式	取值	标签
$\text{adj} \geq 80$	$r_{\text{pos}} = 0.8 + \frac{\text{adj}-80}{20} \times 0.2$	$[0.8, 1.0]$	强势
$60 \leq \text{adj} < 80$	$r_{\text{pos}} = 0.4 + \frac{\text{adj}-60}{20} \times 0.2$	$[0.4, 0.6]$	结构性震荡
$40 \leq \text{adj} < 60$	$r_{\text{pos}} = 0.2 + \frac{\text{adj}-40}{20} \times 0.1$	$[0.2, 0.3]$	偏弱
$\text{adj} < 40$	$r_{\text{pos}} = \frac{\text{adj}}{40} \times 0.1$	$[0.0, 0.1]$	退潮风险

每段都是线性的，四段拼起来是一条不减、但不连续的阶梯。 r_{pos} 计算后保留两位小数。

数据来源：系统源码 `internal/marketsixdim/sixdim.go` 的 `ScoreToPosition`；核对时间 2026-09-24。

命题 19.6 (分段映射的单调性与跳跃点). 设 $r_{\text{pos}} = g(\text{adj})$ 由表 19.3 定义。则

- (i) g 在 $[0, 100]$ 上单调不减；
- (ii) g 在每段内部严格递增；

(iii) g 在点 $\text{adj} = 40, 60, 80$ 处不连续，且都是向上跳跃：

$$\begin{aligned}\lim_{\text{adj} \rightarrow 40^-} g &= 0.10, & g(40) &= 0.20; \\ \lim_{\text{adj} \rightarrow 60^-} g &= 0.30, & g(60) &= 0.40; \\ \lim_{\text{adj} \rightarrow 80^-} g &= 0.60, & g(80) &= 0.80.\end{aligned}\tag{19.15}$$

(iv) 跳跃幅度随分数升高而变大： $\Delta_{40} = 0.10$ 、 $\Delta_{60} = 0.10$ 、 $\Delta_{80} = 0.20$ 。

这个命题在说什么：第 (i) 条保证“市场越好、允许下注越多”这条直觉方向不错。第 (iii) 条是它的脾气——阈值处会跳一格。79.99 分给 0.60，80.00 分给 0.80：一念之间，仓位差三分之一。

证明. (ii) 四段的斜率分别为 $0.2/20 = 0.01$ 、 $0.2/20 = 0.01$ 、 $0.1/20 = 0.005$ 、 $0.1/40 = 0.0025$ ，全为正，故每段严格递增。

(iii) 用左极限与右值分别代入。

- 在 $\text{adj} = 40$ ：第四段左极限 $g(40^-) = \frac{40}{40} \times 0.1 = 0.10$ ；第三段在 40 处取值 $0.2 + \frac{40-40}{20} \times 0.1 = 0.20$ 。跳跃 $\Delta = 0.20 - 0.10 = 0.10$ 。
- 在 $\text{adj} = 60$ ：第三段左极限 $g(60^-) = 0.2 + \frac{60-40}{20} \times 0.1 = 0.2 + 0.1 = 0.30$ ；第二段在 60 处取值 $0.4 + \frac{60-60}{20} \times 0.2 = 0.40$ 。跳跃 $\Delta = 0.10$ 。
- 在 $\text{adj} = 80$ ：第二段左极限 $g(80^-) = 0.4 + \frac{80-60}{20} \times 0.2 = 0.4 + 0.2 = 0.60$ ；第一段在 80 处取值 $0.8 + \frac{80-80}{20} \times 0.2 = 0.80$ 。跳跃 $\Delta = 0.20$ 。

(i) 在每段内部由 (ii) 严格递增。跨段只需检查三个边界：由 (iii) 的三个跳跃全为非负 ($0.10, 0.10, 0.20 \geq 0$)，且每段右端的值不超过下一段右端的值（例如第三段最大值 0.3 等于第二段最小值 0.4 的左侧极限，未越界），故拼接后整体不减。又第一段上界为 1.0，与代码中的 $r \leq 1$ 截断一致。■

用这五个真实记录验一遍映射（只验当前口径下应当成立的四个）：

表 19.4 四条真实记录从 `adjusted_total_score` 到 `position_rate` 的验算

日期	adjusted	命中区间	手算 r_{pos}	库中 r_{pos}	标签
2026-09-02	41.82	[40, 60)	$0.2 + \frac{1.82}{20} \times 0.1 = \mathbf{0.21}$	0.21 ⁵	偏弱
2026-09-09	40.40	[40, 60)	$0.2 + \frac{0.40}{20} \times 0.1 = \mathbf{0.20}$	0.20 ⁶	偏弱
2026-09-10	38.43	[0, 40)	$\frac{38.43}{40} \times 0.1 = \mathbf{0.10}$	0.10 ⁷	退潮风险
2026-09-11	25.71	[0, 40)	$\frac{25.71}{40} \times 0.1 = \mathbf{0.06}$	0.06 ⁸	退潮风险

手算值四舍五入到两位小数后与库中 `position_rate` 逐条吻合。2026-08-31 那条因口径变更不参与本表验算，见注 19.5。

数据来源：只读查询 `market_sixdim_daily`；本书按 `internal/marketsixdim/sixdim.go` 的 `ScoreToPosition` 复算；查询日 2026-09-24。

到这一步，有一条规律值得单独拎出来。请看九月十一日那条： $s_{\text{tech}} = 15$ ，但 $s_{\text{capital}} = 80$ ——技术面已经跌破到接近最差，资金结构却还是满分。这正是“冲突”最典型的形态：价格在

跌，但钱还在进。系统的回答不是“听谁的”，而是两个都听，然后打八折。

19.7 它在系统里长什么样

三层职责分得很开，这是这个包最值得学的一点：

- **评分是纯函数。**`internal/marketsixdim/sixdim.go` 里没有任何 I/O——输入 `MarketInput`、输出 `MarketReport`，可测试、可复现；
- **取数是独立的一层。**`internal/marketsixdim/fetcher.go` 负责把真实数据填进 `MarketInput`（下一章讲它怎么降级）；
- **落库是第三层。**`internal/marketsixdim/persist.go` 把结果写进 `market_sixdim_daily`，以交易日为主键——所以同一天重复跑判势不会产生重复记录，只会覆盖，这让“今天到底算了什么”永远只有一个答案。

面向智能体时，这一整套被打包成一个工具：`market_sixdim_detect`。它的可用角色是 CIO、PLANNER、RISK、QUANT 四个——这是第 chapter 一章那张工具可见性表里的第二个例子：判势是共享上下文，不是任何人的私产。它还有一个 `save` 参数，用来控制这次调用是否落库；盘前决策会落库，盘中复算不落库。

常见误解·判势是涨跌预测 这是我被问得最多的一个误解。

判势完全不预测涨跌。它输出的不是“明天涨还是跌”，而是“今天允许下多少注”。同一个判势分数，在两种用法下会导出完全相反的结论：

错误用法：“今天 25.71 分（退潮风险），所以要做空/清仓。”判势从来不给清仓建议——它只拦新买入，不会要求你卖掉已有持仓。真正决定卖不卖的是策略信号和风控。

正确用法：“今天 25.71 分，所以任何新的买入，名义金额上限是原来的 6%。”就这一件事。

为什么要把这个区分做这么硬？因为它决定了判势错了的后果有多严重。如果判势是方向预测，它错了就是亏钱；而它只是一个缩放系数时，它错了最多是“今天少赚”。把仓位系数当方向信号用，等于把一个保守器改装成了油门。

常见误解·六维中有一项爆表，总分就该爆表 “今天涨停 200 家，情绪维度直接拉满，总分应该很高。”

看第二段的公式：涨停项的加分被 $\min(\cdot, 100) \times 0.3$ 封在 30 分。再加基准 50，情绪维度最多到 100；而它权重是 0.25。也就是说，哪怕情绪维度撞到天花板，它对总分的贡献也只有 25 分。

这不是设计缺陷。它的用意是：任何单一维度都不能把总分带走。反过来说，如果一个市场的总分很高，那一定是多个维度同时好——这正是“强势”要被定义成的样子。

顺带说一个更要紧的边界：这些“封顶”全部是实现里的常数，不是理论最优值。它们可以被修改，但修改之后必须重新用真实记录验算一遍——本章表 19.4 那四行，就是你要维护的回归测试。

本章对应的真实代码与数据

- 六维权重、固定顺序、中文名：`internal/marketsixdim/sixdim.go` 的 `DimWeights`、`DimOrder`、`DimChinese`
- 六个维度的评分函数与截断：`internal/marketsixdim/sixdim.go` 的 `dim1TechScore` ...`dim6ExternalScore`、`clampScore`

- 冲突计数与修正算子: `internal/marketsixdim/sixdim.go` 的 `CheckDimConflict`、`Evaluate`
- 分数 → 仓位 → 标签映射: `internal/marketsixdim/sixdim.go` 的 `ScoreToPosition`
- 判势结果的落库表与五条真实记录: `market_sixdim_daily`
- 面向智能体的工具: `market_sixdim_detect` (可用角色 `CIO/PLANNER/RISK/QUANT`, `save` 控制落库)

动手试一试（二十分钟，全程可核对）

1. 打开系统，进 **CIO 决策中心**，找到【市场六维判势】卡片。它默认显示最近 7 天，最多 30 天。
2. 把卡片上六个维度的分数抄下来，用式 (19.4) 手算一遍 `raw_total_score`，和界面上的数对一下。误差应当在四舍五入范围内。
3. 数一数有几个维度 ≥ 75 、几个 ≤ 40 。两者都非零时，冲突数就是两者之和；达到 2 就打八折。核对界面上显示的 `adjusted_total_score`。
4. 用表 19.3 的公式算仓位系数，和界面上的 `position_rate` 对一下。
5. 最后做一个思想实验：把某一天的 `external` 从 46.71 改成 90，其余不动。总分变多少？冲突数会不会变？标签会不会跨档？——想清楚之后再去动手改权重（下一章之后的第 30 章会讲改参数的安全做法）。

六个维度、一个加权和、一次八折、一张阶梯表——这就是判势的全部算法。它是纯函数，只要输入确定，输出就确定。

可问题恰恰在这里：那六个输入的分数，是从哪儿来的？九月二日那条记录里，市场广度的来源写着“采样 300 只 20 日新高/新低 (98/8)”，而涨跌家数那一项写着“全市场涨跌家数 (涨 0/总 9533) = 0%”。一个说采样、一个说全市场，涨 0 家对 9533 家——这显然不对。

下一章就去看每一个数字的真实来源，以及取不到数的时候，系统选择做什么。那是我所知道的、全书最贵的一课：抽样偏差不会报错，它只会安静地给你一个错的答案。

今天能不能下注？（下）——数据从哪来，缺数怎么办

20.1 一个“总是很淡”的市场广度

上一章末尾我留了一个尾巴：九月二日那条判势记录的市场广度，只有 18¹ 分。

可我记得那几天盘面并不难看。涨停板上是有票的，只是系统说“上涨家数 0”。

我去查了这一天的 `sources`，广度的三个来源写得很清楚：

- `breadth_highlow`：采样 300 只 20 日新高/新低 (98/8)；
- `breadth_rise`：全市场涨跌家数 (涨 0/总 9533)=0%；
- `breadth_limit`：采样涨停/跌停 (0/0)。

“涨 0 / 总 9533” 这行字，我当时读了三遍才反应过来：在 9533 只股票里，没有一只在上涨。这在任何一天都不可能发生，除非那天的行情库坏了。

行情库没坏。坏的是取样本的那一句 SQL。

20.2 体检设备的精度说明

直觉·“本设备精度 $\pm 0.5^{\circ}\text{C}$ ” 你去体检，拿到一台家用体温计，盒子上印着“精度 $\pm 0.5^{\circ}\text{C}$ ”。你不会因为这台机器不准就把它砸了。你会换一种用法：它测出 37.8，你不会据此判断“发烧了”，你会说“可能有点低烧，再用水银的确认一下”。

更关键的是：如果某天它显示“-”，你知道那意味着没测出来，而不是“体温是零”。

数据源也是同一回事。每一个外部数据源都有它的精度、时效和失效方式：一个负责任的系统，要做的不是假装它们都一样准，而是把每一格的来源和可信度写在旁边。

这一章只回答一个问题：上一章那六个维度的分数，每一下是从哪儿取来的；取不到的时候，系统做什么。

答案有一条贯穿始终的底线。它写在这个包的源代码文件头上：

包级硬约束（原文，`internal/marketsixdim/fetcher.go` 文件头注释）

“数据来源原则（对应项目硬约束「严禁伪造数据」）：……缺失源用真实代理指标并标注……保证「严禁伪造数据」——任何无真实数据源的维度都明确标注为中性默认，不猜测。”

这句话拆成三条可执行的规则：

1. 能取到真实值，就用真实值，并把来源写进 `sources`；
2. 取不到真实值，可以退而用“代理指标”，但必须在 `sources` 里写明“这是代理”，并说清代理是什么；

¹真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

3. 连代理都没有，就给中性默认（50 分 / 0 / false），同样标注，绝不猜一个“看起来合理”的数。

20.3 先讲最贵的那个错：抽样抽歪了

上面那条“涨 0 / 总 9533”，来自一个现在已经修掉、但值得单独立一章的错误。系统的真实事故清单把它列在第 #4 条：

真实事故 #4（摘录自系统事实清单）

症状：市场广度“总是很淡”，与盘面感觉不符。

根因：旧版 ORDER BY symbol LIMIT 300 字典序截断，样本几乎全落在北交所 + 沪 600。

修复：改分层代表样本 + 固定种子 20240901 可复现。

这个 bug 最值得讲的地方是：它不会报错。SQL 语法正确，返回 300 行，程序照常往下走。它只是安静地给你一个错的答案。

先把“错”写成可证明的东西。

定义 20.1 (板块分层与真实占比). 设股票宇宙 $\mathcal{U}$ 被划分为 B 个板块（沪主板、深主板、创业板、科创板、北交所），第 b 个板块含 N_b 只股票， $N = \sum_b N_b$ 。记真实占比

$$p_b = \frac{N_b}{N}, \quad \sum_{b=1}^B p_b = 1. \quad (20.1)$$

设第 b 个板块内“上涨股票”的比例为 $\theta_b \in [0, 1]$ ，则全市场上涨比例为

$$\theta = \sum_{b=1}^B p_b \theta_b. \quad (20.2)$$

任一抽样方案给出板块构成的估计 $\hat{p}_b$ ，其与真值的全变差距离为

$$d_{\text{TV}}(\hat{p}, p) = \frac{1}{2} \sum_{b=1}^B |\hat{p}_b - p_b| \in [0, 1]. \quad (20.3)$$

命题 20.2 (字典序截断的偏差). 设符号列 *symbol* 的字典序把 B 个板块排成一个严格顺序，其中板块 b^* 的全部符号都小于其余所有板块的符号。令 n 为样本量，且 $n \leq N_{b^*}$ 。则字典序截断 (ORDER BY symbol LIMIT n) 得到的样本满足

$$\hat{p}_{b^*} = 1, \quad \hat{p}_b = 0 \ (b \neq b^*). \quad (20.4)$$

于是

(i) 全变差距离 $d_{\text{TV}} = 1 - p_{b^*}$ ，即偏差完全由被过度代表的那一个板块决定；

(ii) 样本上涨比例 $\hat{\theta} = \hat{\theta}_{b^*}$ ，其期望为

$$\mathbb{E}[\hat{\theta}] = \theta_{b^*}, \quad \text{偏差} \quad \mathbb{E}[\hat{\theta}] - \theta = \sum_{b \neq b^*} p_b (\theta_{b^*} - \theta_b); \quad (20.5)$$

(iii) 偏差为零当且仅当所有板块的上涨比例相等 ($\theta_b \equiv \theta$)。

这个命题在说什么：它把“抽样抽歪”从一句抱怨变成一条公式。第 (iii) 条最要紧——只要各板块表现不同（而它们永远不同），字典序截断就必然有偏，而且这个偏差和样本量 n 无关：你把 300 加到 3000，偏差一模一样，只是更自信地错。

证明. (i) 由 $n \leq N_{b^*}$ ，“最小的 n 个符号”全部落在板块 b^* 内，故 $\hat{p}_{b^*} = 1$ ，其余 $\hat{p}_b = 0$ 。于是

$$d_{\text{TV}} = \frac{1}{2} \left(|1 - p_{b^*}| + \sum_{b \neq b^*} |0 - p_b| \right) = \frac{1}{2} \left((1 - p_{b^*}) + (1 - p_{b^*}) \right) = 1 - p_{b^*}.$$

(ii) 样本全在板块 b^* 内，故样本上涨比例只反映该板块： $\hat{\theta} = \hat{\theta}_{b^*}$ 。若板块内为简单随机抽样，则 $\mathbb{E}[\hat{\theta}_{b^*}] = \theta_{b^*}$ ，于是 $\mathbb{E}[\hat{\theta}] = \theta_{b^*}$ 。再用 $\sum_b p_b = 1$ 与 $\theta = \sum_b p_b \theta_b$ ：

$$\theta_{b^*} - \theta = \theta_{b^*} \sum_b p_b - \sum_b p_b \theta_b = \sum_b p_b (\theta_{b^*} - \theta_b) = \sum_{b \neq b^*} p_b (\theta_{b^*} - \theta_b),$$

因为 $b = b^*$ 那一项为零。

(iii) 由式 (20.5)，偏差是 $\{\theta_{b^*} - \theta_b\}_{b \neq b^*}$ 关于权重 p_b 的加权和。若所有 θ_b 相等，每项为零，偏差为零；反之若存在 b 使 $\theta_b \neq \theta_{b^*}$ 而 $p_b > 0$ ，则该项非零，且因其余项不改变符号（在极端情形下全同号），加权和不为零。■

用库里真实的样本量算一遍，就知道这个偏差有多大。stock_basic 里可交易个股的板块分布是：沪主板 1710²、深主板 1497³、创业板 1409⁴、科创板 617⁵、北交所 351⁶，合计 5584⁷。

若按字典序截断取 300 只，且“北交所”恰好排在字典序最前 ($b^* = \text{北交所}$, $N_{b^*} = 351 \geq 300$)，则：

修复方案也写在这个文件里。核心是三行代码：

```
1 // repSampleSeed 代表性抽样的固定随机种子。固定种子保证多次判势/重启间
2 // 采样结果可复现，避免样本在两次判势间随机漂移导致广度信号抖动。
3 const repSampleSeed = 20240901
4
5 // 各板块按其在全宇宙中的占比等比例配额
6 rng := rand.New(rand.NewSource(repSampleSeed))
```

²真实数据：本数字直接取自系统运行库 data/stock.duckdb，未做任何模拟或平滑。

³真实数据：本数字直接取自系统运行库 data/stock.duckdb，未做任何模拟或平滑。

⁴真实数据：本数字直接取自系统运行库 data/stock.duckdb，未做任何模拟或平滑。

⁵真实数据：本数字直接取自系统运行库 data/stock.duckdb，未做任何模拟或平滑。

⁶真实数据：本数字直接取自系统运行库 data/stock.duckdb，未做任何模拟或平滑。

⁷真实数据：本数字直接取自系统运行库 data/stock.duckdb，未做任何模拟或平滑。

表 20.1 字典序截断 300 只 vs. 分层等比例配额 300 只 (真实板块规模)

板块	全市场只数	真实占比 p_b	字典序截断	等比例配额 n_b	配额占比
沪主板	1710 ⁸	30.62%	0	91	30.33%
深主板	1497 ⁹	26.81%	0	80	26.67%
创业板	1409 ¹⁰	25.23%	0	75	25.00%
科创板	617 ¹¹	11.05%	0	33	11.00%
北交所	351 ¹²	6.29%	300	18	6.00%
合计	5584 ¹³	100%	300	297 + 3	100%

字典序截断一列的全变差距离 $d_{TV} = 1 - 0.0629 = 0.937$ ——样本的板块构成与总体几乎完全无关。等比例配额一列按 $\lfloor nN_b/N \rfloor$ 取整, 得 297 只, 余下 3 只按顺序补给仍有容量的板块, 凑满 300。

数据来源: 只读查询 `stock_basic`, 按 `sh6 / sz00 / sz30 / sh688 / bj` 前缀分板块计数; 配额按 `internal/data/duckdb.go` 的 `ListRepresentativeSymbols` 公式复算; 查询日 2026-09-24。

```

7 order := []string{"sh", "sz", "cy", "kc", "bj", "other"}
8 quota := map[string]int{}
9 assigned := 0
10 for _, b := range order {
11     buckets[b] = shuffleSymbols(rng, buckets[b]) // 板内可复现随机排列
12     q := n * len(buckets[b]) / len(all)          // 等比例配额
13     if q < 0 { q = 0 }
14     quota[b] = q
15     assigned += q
16 }
17 // 余量分配给仍有容量的板块 (按顺序), 保证总数正好 n

```

代码 20.1 分层等比例配额抽样 (节选自 `internal/data/duckdb.go`)

这段话在说什么: 三件事。第一, 配额公式 $q_b = \lfloor nN_b/N \rfloor$ 就是定义 20.1 里那个 p_b 的样本版本; 第二, 板内用固定种子打乱, 所以同样的宇宙必然得到同样的样本——这叫可复现; 第三, 余量补足, 保证样本量正好是 300。

命题 20.3 (等比例配额的样本占比无偏). 设各板块按 $n_b = \lfloor nN_b/N \rfloor$ 取配额, 板块内为简单随机抽样 (不放回), 板块 b 内上涨比例 θ_b 的估计为 $\hat{\theta}_b$. 记样本总量 $n^* = \sum_b n_b \leq n$. 则

- (i) $\sum_b \frac{n_b}{n^*} \hat{\theta}_b$ 是 θ 的无偏估计, 当配额可整除 ($n_b = nN_b/N$, $\sum_b n_b = n$) 时;
- (ii) 板块占比估计 $\hat{p}_b = n_b/n$ 满足 $\mathbb{E}[\hat{p}_b] = p_b$ (整除时), $d_{TV} \rightarrow 0$ ($n \rightarrow \infty$, 整除);
- (iii) 与命题 20.2 对比: 字典序截断的偏差不随 n 减小, 而等比例配额的偏差随 n 消失。

证明. (i) 板块 b 内为简单随机抽样, 故 $\mathbb{E}[\hat{\theta}_b] = \theta_b$ (样本比例是总体比例的无偏估计)。于是

$$\mathbb{E}\left[\sum_b \frac{n_b}{n^*} \hat{\theta}_b\right] = \sum_b \frac{n_b}{n^*} \mathbb{E}[\hat{\theta}_b] = \sum_b \frac{n_b}{n^*} \theta_b.$$

若配额整除, $n_b = nN_b/N$, 则 $n^* = \sum_b nN_b/N = n$, 且

$$\frac{n_b}{n^*} = \frac{nN_b/N}{n} = \frac{N_b}{N} = p_b,$$

代入得 $\sum_b p_b \theta_b = \theta$ ，故无偏。

(ii) $\hat{p}_b = n_b/n$ 为常数（配额是确定性的），整除时 $\hat{p}_b = nN_b/(Nn) = N_b/N = p_b$ ，于是 $d_{TV} = \frac{1}{2} \sum_b |p_b - p_b| = 0$ 。配额的取整误差 $|n_b - nN_b/N| < 1$ ，故 $\hat{p}_b \rightarrow p_b$ ， $d_{TV} \rightarrow 0$ 。

(iii) 命题 20.2 的偏差式 (20.5) 里没有 n ：它只由各板块的 θ_b 差异与 p_b 决定，加大样本量不改变它。而 (i) 的偏差恰为零，且 (ii) 的 $d_{TV} \leq \frac{B}{2n} \rightarrow 0$ 。■

这一节真正要记住的一句话

抽样偏差不会报错。SQL 会成功，数字会返回，曲线会画出来。

它只在你看不见的地方，把你的读数朝着某个方向推——推到你自已都相信“市场就是这么淡”为止。

所以任何“采样”都要能回答两个问题：样本的构成是什么？样本量是多少？

20.4 第二个安静的错误：差一根 K 线

抽样偏差是“选错了对象”，还有一类错误是“数错了格子”。事故 #5 就是它。

真实事故 #5（摘录自系统事实清单）

症状：标注“20 日新高”，但实际只比对了 19 根。

根因：bars[len-20:] 把当前 bar 混入窗口。

修复：改为 bars[len-21 : len-1]。

命题 20.4 (窗口混入当前 bar 的后果). 设第 t 日的收盘价为 c_t 。按定义，第 t 日为“20 日新高”当且仅当

$$c_t > \max\{c_{t-20}, c_{t-19}, \dots, c_{t-1}\} \quad (\star)$$

即当前价高于此前 20 个交易日最高收盘。设长度为 L 的数组 bars 以 bars[L-1] 为当前 bar。则

- (i) 正确窗口为 bars[L-21 : L-1]，其下标集合是 $\{L-21, \dots, L-2\}$ ，恰好 20 个元素，对应 $c_{t-20}, \dots, c_{t-1}$ ；
- (ii) 若误用 bars[L-20 : L]，其下标集合是 $\{L-20, \dots, L-1\}$ ，含当前 bar，对应 $\{c_{t-19}, \dots, c_t\}$ ；
- (iii) 此时：若判定写 $c_t \geq \max(\cdot)$ ，条件退化为“ c_t 不低于前 19 日”，即 19 日新高，标签与口径不符；若判定写严格不等号 $c_t > \max(\cdot)$ ，则条件恒假，新高计数恒为 0（新低同理）。

这个命题在说什么：一根 K 线的错位，会让两个截然不同的结论同时发生——要么把“20 日新高”偷换成“19 日新高”，要么让它永远数不出一个。而两种情况在界面上都表现为“新高很少”，也就是事故清单里那句“市场广度总是很淡”。

证明. (i) 数组下标从 0 计，总长 L ，当前 bar 下标 $L-1$ ，对应 t 。切片 bars[L-21:L-1] 含左端不含右端，下标集合为

$$\{L-21, L-20, \dots, L-2\},$$

元素个数为 $(L-2)-(L-21)+1=20$ 。逐项对应：下标 $L-21$ 即 $t-20$ ，……，下标 $L-2$ 即 $t-1$ 。故恰为 $\{c_{t-20}, \dots, c_{t-1}\}$ ，与 $(\star)$ 的窗口一致。

(ii) 切片 `bars[L-20:L]` 的下标集合为 $\{L-20, \dots, L-1\}$ ，元素数 $(L-1)-(L-20)+1=20$ ，对应 $\{c_{t-19}, \dots, c_t\}$ ，其中最后一项 c_t 就是当前 bar。

(iii) 记 $M = \max\{c_{t-19}, \dots, c_t\}$ 。

- 若判定为 $c_t \geq M$ ：因 c_t 是取最大值的候选之一， $M \geq c_t$ 恒成立，故 $c_t \geq M \iff c_t = M \iff c_t = \max\{c_{t-19}, \dots, c_{t-1}\}$ （最后一步用了 $c_t = M$ 时 M 由其余项决定）。即“不低于前 19 日的最高收盘”，是 19 日新高的定义，而非 20 日。
- 若判定为 $c_t > M$ ：因 $c_t \leq M$ 恒成立，故 $c_t > M$ 恒为假，计数恒为 0。新低同理（ $c_t < \min\{c_{t-19}, \dots, c_t\}$ 恒假）。



修复后的两个好处值得点出来：一是窗口正好 20 根，口径与标签一致；二是窗口严格落在当前 bar 之前，所以它在时间上只用了过去，没有偷看今天——这 and 第 chapter 十七 章讲的前视偏差是同一条纪律。

20.5 六维的来源与降级链

现在把六个维度的真实来源摆出来。这张表是本章的核心，它回答的是“每个数从哪儿来”。

表 20.2 六个维度的首选源、备选源与兜底代理

维度	首选源	备选 / 兜底（逐级降级）
技术趋势	DuckDB 真实指数日 K	直接取 <code>sh000001</code> ；不够 60 根时该维度留中性
市场广度	DuckDB 真实行情 + 代表样本	涨跌停比优先东财 <code>push2ex</code> 全市场池，失败回退代表样本
量能流动性	DuckDB 全市场成交额	相对 20 日均值；数据不足则该维度留中性
资金结构	同花顺 <code>hsgtApi</code> 北向实时净流入	（1）东财融资余额趋势 → （2）兜底代理“成交额连续放量天数”
情绪赚钱效应	同花顺官方涨停/跌停/炸板池 + 连板天梯	（1）东财 <code>push2ex</code> → （2）DuckDB 全市场统计 → （3）采样日线代理
外部约束	腾讯全球指数实时（4s 超时 + 5min 缓存）	兜底代理：上证指数开盘跳空幅度

“代理”不是“伪造”：代理是一个真实但含义不同的指标，用它时必须写明它代理的是什么。表中每一行的实际取值都会写进 `sources` 字段。

数据来源：系统源码 `internal/marketsixdim/fetcher.go` 与系统事实清单第 8 节；核对时间 2026-09-24。

把这张表和第一章末尾那个真实的 `sources` 对照着读，会看出系统的“成长”：

```

1 # 2026-09-02（早期版本：多处降级、口径不一）
2 breadth_highlow : 采样300只20日新高/新低(98/8)
3 breadth_rise    : 全市场涨跌家数(涨0/总9533)=0%
4 capital         : 北向无实时源→成交额连续放量天数代理=0；资金集中度=行业Top3成交额占比>=40%(true)
5 external        : 无隔夜美股/事件实时源→用上证指数开盘跳空代理(开盘缺口-0.16%(中性))
6 sentiment       : 采样300只日K(全市场统计不可用降级)：涨停0/跌停0/炸板率0%(日线代理)
7
8 # 2026-09-09（改善后：多源就位，逐项标注）

```

```

9 breadth_rise      : 代表样本涨跌家数(涨171/总300)=57%
10 breadth_highlow  : 代表样本20日新高/新低(57/16)
11 capital          : 北向实时(同花顺hsgtApi): 当日净流入370亿→连续流入口径5
12 external         : 真实隔夜外围(tencent(qt.gting.cn) 全球指数实时:偏空)融合指数跳空(开盘缺口0
                      .07%(中性))
13 sentiment        : 同花顺官方特色数据(实时): 涨停48/跌停7/炸板率36%(炸板27)
14 volume           : 全市场成交额120584亿/20日均值105122亿=1.15, 5日/20日量能趋势=0.95(综合1.07)

```

代码 20.2 sources 字段的两个真实版本（原文摘录，仅截取关键片段）

这段话在说什么：九月二日的每一行几乎都带着“无实时源”“→ 代理”“降级”这样的字样，而且广度四成分的口径是混的——“采样 300 只”和“全市场”混在一起用，结果就是“涨 0/总 9533”这种自相矛盾的读数。九月九日之后，四个成分统一到同一个代表样本上（涨 171/总 300），情绪换成了同花顺官方实时池，资金结构拿到了北向实时净流入。

20.5.1 缓存与限流：不是性能优化，是纪律

外部数据源（同花顺、东财、腾讯）都有一个共同特点：请求太密会被封。而盘中判势每 5 分钟跑一次，每次要覆盖约 300 只样本的最新价。

系统的处理写在 `internal/marketsixdim/fetcher.go` 里：一个包级的 `ttlCache`，实时快照的 TTL 设成 30 秒；外部源另有最小请求间隔限流。

```

1 // 批量实时快照缓存：广度维度每次判势会请求整组代表样本(~300只)的实时快照，
2 // 是判势链路最贵的外部调用。
3 realtimeSnapCache = newTTLCache[realtimeSnapsEntry](30 * time.Second)
4 ...
5 // 非交易时段实时报价即昨收，直接用 DuckDB 历史，避免对 ~300 只样本的无谓外部请求。
6 if !inTradingSession { /* 走 DuckDB, 不请求外部源 */ }
7 ...
8 // 覆盖数为 0 时必须告警，而不是静默降级
9 if !cached && len(snaps) > 0 && rt.covered == 0 {
10     log.Printf("[SixDim] 实时快照覆盖 0/%d ...", len(codes))
11 }

```

代码 20.3 实时快照的 TTL 缓存与非交易时段短路（节选自 `internal/marketsixdim/fetcher.go`）

这段话在说什么：三个动作。**第一**，加 **TTL**——30 秒内的重复请求直接吃缓存，既省请求也去抖。**第二**，非交易时段短路——收盘后“最新价”就是收盘价，库里有，不必去外部问一遍，这是事故 #18 的修复。**第三**，覆盖数为 0 时报出来——这是事故 #28 的修复；这一条最像本书反复讲的那个原则：降级可以是设计，但降级必须被看见。

还有一条口径标注规则：样本数小于 100 时，在 `sources` 里标“低可信度”。

20.6 它在系统里长什么样

把本章的东西串起来，是四层：



图 20.1 六维判势的取数链。左边是首选，越往右越是兜底；无论走到哪一站，来源都必须写进 `sources`。

本章对应的真实代码与数据

- 分层代表样本抽样、固定种子 20240901、等比例配额与余量补足：`internal/data/duckdb.go` 的 `ListRepresentativeSymbols`、`repSampleSeed`
- 六维取数与降级链、TTL 缓存、非交易时段短路、覆盖数告警：`internal/marketsixdim/fetcher.go` 的 `fetchSampleBars`、`aggregateSampleStats`、`realtimeSnapCache`、`ttlCache`
- 判势结果落库：`internal/marketsixdim/persist.go` 与 `market_sixdim_daily`
- 五个维度的真实 `sources` 原文：`market_sixdim_daily` 的 `sources` 字段
- 真实板块规模：`stock_basic`（沪主板 1710 / 深主板 1497 / 创业板 1409 / 科创板 617 / 北交所 351）
- 相关事故：`#4` 字典序抽样落到北交所、`#5` 20 日新高 `off-by-one`、`#18` 六维延迟与限流、`#28` 覆盖数 0 无告警、`#29` 样本不足未提示

常见误解·“数据不全就先拿别的顶上” 这是最危险的一句话，因为它的动机是“把活干完”。

本章里所有的兜底（北向没有就用融资余额、再没有就用成交额连续放量天数；外围没有就用指数跳空）看起来正像是“拿别的顶上”。它们和“编一个数”的区别在哪里？

区别只有一个：代理指标是真实发生的另一个量，而且它的名字被写在旁边。

“北向连续流入 5 天”和“成交额连续放量 3 天”是两个不同的东西，后者不是前者。你可以在拿到前者之前先用后者做一次粗判断，但你必须知道自己在用后者——否则三个月后读这条记录的人（很可能是你自己）会以为那天真有北向数据。

系统把这条写进了包约束：“代理变量必标注”。而“数据缺失”这一类（`order_book_daily` 表当前 0 条记录，见第 chapter 十二章）就老老实实留白，连代理都不编。

常见误解·样本量够大就没偏差 “我取 300 只不行，那我取 3000 只。”

看命题 20.2 的偏差式 (20.5)——里面没有 n 这个符号。字典序截断的偏差完全由“各板块的差异”决定，样本量再大也不减一分。你只是从“300 只全错”升级到了“3000 只全错”，而且更理直气壮。这是抽样偏差最反直觉的地方：它随样本量增大而变得更自信，而不是更准确。无偏才是第一位的，样本量是第二位的。

系统里对应的一处细节值得抄笔记：样本数 < 100 时标注低可信度。这句话在数学上什么都没保证，但它承认了一件事——可信度是一个需要被说出来的东西。宁可标注“这次样本少，请谨慎”，也不假装每一条记录同样可靠。

动手试一试（二十分钟，全部可在界面上核对）

1. 打开 CIO 决策中心的【市场六维判势】卡片，展开任意一天的明细，找到它的来源说明（对应库里的 `sources`）。
2. 逐维读一遍：哪几个维度写着“真实源”，哪几个写着“→ 代理”，哪几个写着“降级”。把它们列成两栏。

3. 打开 设置 → 数据源 → 市场信息数据，看【六维判势】那张卡片。它列出了 6 个源的启用开关与优先级——这里面每一个开关，对应的就是表 20.2 里的一行。
4. 做一道手算题：把表 20.1 里“字典序截断”那一列换成“只取前 50 只”。新的 d_{TV} 是多少？偏差会变大还是变小？为什么？（提示：看命题 20.2(i)。）
5. 最后做一个思想实验：如果某一维度的首选源和所有备选源全部失效，系统的选择是什么？请回到本节开头那三条规则，说出它的具体动作，以及你会在哪里看到这个动作的痕迹。

到这里，问题“今天能不能下注”有了一个数：仓位系数。可它是怎么变成真金白银的下单量的？选股说“买 10 万”，判势说“今天只能下四成”，最后到底买了多少、这个缩放发生在哪一层、为什么卖出不受它影响——下一章讲这件事。那里还会看到一条被反复强调的不对称设计：离场永远比进场更该被放行。

决定“下多少注”：仓位系数是怎么算出来的

21.1 同一条决策里，出现了两个数字

盘前 09:00，量化分析师把选股结果交上来：一只票因子分排第一，建议买入，预算 10 万元。10 万这个数字听起来很确定，它是按组合规模和单票上限算出来的。

可那天 CIO 的日志里同时打印出另一串数字：总分 25.71（修正后 25.71），冲突数 5，仓位系数 0.06，标签「退潮风险」0.06¹。

10 万，和 0.06。我一开始以为这是打折：0.06 就是打六折吧？六折是 0.6，不是 0.06。0.06 是百分之六。10 万乘以 0.06 等于 6000² 元。

更让我意外的是后面的结果：那天的成交记录里，连 6000 元这一笔都没有。订单根本没生成。

同一周的另一个交易日，标签是「偏弱」、仓位系数 0.200.20³，这一天闸门没关，10 万变成了 2 万元⁴。

同一个策略、同一个买入信号，最终的下单量在 0、2 万、6 千之间来回跳。这一章就是要回答这件事：到底是谁在决定“下多少注”。

第 chapter 十九 章，我们把一天的市场压成一个 0 到 100 的分数；第 chapter 二十 章，我们追清楚了这六个维度的数字从哪来。现在到了最后一公里——这个分数怎么落到下单量上。

21.2 发动机和限速牌，从来不吵架

直觉·油门与限速 一辆车的速度由两样东西共同决定：发动机和限速牌。

发动机决定“我能跑多快”，你把油门踩到底，它能给你 200。限速牌决定“我今天允许多快”，路口写着 60，你就只能跑 60。

关键在于：限速牌从来不会让车变快。它只会让车变慢，或者什么也不管。你把发动机换成更强的，限速牌还是 60；你把限速牌从 60 换成 40，发动机再强也得慢下来。

还有一层：这两套系统互不干扰。发动机不会因为看到限速牌就熄火，限速牌也不会因为发动机弱就自动放宽——它们各自独立生效，最后那个“实际速度”取的是两者中更保守的一个。

策略就是发动机：它回答“这个机会值多少注”。六维判势就是限速牌：它回答“今天的市场允许你下多少注”。最后的下单量，取的是更保守的那个。

策略和判势是两件不同的事，我一开始把它们搅在一起，吃了不少苦头（这件事写在后面

¹真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

²复算数据：该数字为本书按系统实现中的同一公式、在系统的真实数据上重新计算所得；公式出处已在正文标注。

³真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

⁴复算数据：该数字为本书按系统实现中的同一公式、在系统的真实数据上重新计算所得；公式出处已在正文标注。

的纠偏里)。

21.3 不装限速牌，会付出什么代价

先把“为什么需要”讲清楚，否则“仓位系数”听起来只是程序员多加的一个字段。

代价一：全市场退潮的时候，你按满仓的力气下注。

看两条真实记录。2026-09-10 那天，六维判势的技术趋势维度并不差，可市场广度与情绪两个维度同时低迷、彼此冲突，修正后总分 38.43，标签「退潮风险」38.43⁵。第二天更差，修正后总分 25.71，标签还是「退潮风险」25.71⁶（两天的逐项维度分已在第 chapter 二十 章逐条核对）。

这两天，市场自己给出的信号是“退潮”。如果没有限速牌，策略只要发出买入信号，系统就会按 10 万的尺度去建仓——等于在系统自己认定的危险期里把预算一次性押出去。

代价二：你分不清“信号错了”还是“下注太多”。

一个策略亏损有两种完全不同的解释：一种是信号本身在这种环境里失效，另一种是信号没错、只是这个环境不该下这么大的注。如果下单量只由策略决定，这两种情况在成交记录里长得一模一样，你永远查不清到底是哪一个。

代价三：胜负被单一变量决定。

策略是“什么时候买”，仓位是“买多少”。如果只有前者、没有后者，那么整个系统的收益就完全押在“信号准不准”这一件事上。而限速牌提供的是另一条独立的控制通道：信号可以照发，下注量由市场环境来收缩。

21.4 近似模型：乘一个系数；严谨版本：取一个最小值

先给一个差不多对的模型。最省事的做法是“乘一个系数”：

$$\text{信号仓位} = \text{原始信号仓位} \times r_{\text{pos}}. \quad (21.1)$$

这就是系统里 `scaleOrdersByPositionRate` 那个函数干的事，也是这一章最核心的一句话。 r_{pos} 越接近 1，限量越松；越接近 0，限量越紧。

但这个近似在哪里失效？乘法的缺点是：它永远乘不到 0。只要系数是 0.06，10 万还是能买 6 千；系数是 0.01，10 万还是能买 1 千。可“今天根本不该建仓”这件事，用乘法表达不出来——乘法没有“关机”的那一档。

所以严谨版本有两层：

- **软缩放**：在允许建仓的日子里，用系数把买入量按比例缩小（式 21.1）；
- **硬闸门**：在系统判定“不该建仓”的日子里，直接不生成买入订单，而不是把它缩成一个很小的数。

⁵真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

⁶真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

这两层的分界线，是本章最有意思的地方，因为它正好是一次真实翻车后补上的（见纠偏一）。

现在把软缩放那层写成严格的数学。生效仓位不是“乘一个数”，而是“在好几个来源里取最小值”。

假设 21.1 (门控四个输入的取值域). 本章讨论的四个来源各自归一化到 $[0, 1]$:

- (1) 规则门 $r_0 \in [0, 1]$ ，就是六维判势给出的仓位系数 r_{pos} ；
- (2) 主张 $a \in [0, 1]$ 、异议 $d \in [0, 1]$ ，二者在进入最小值之前先被夹到 $[0.2, 1.0]$ ；
- (3) Kelly 缩放系数 $\kappa \in [0.4, 1]$ ，它的界由 chapter 七 给出，本章先引用；
- (4) 任一来源缺失时，把该项从最小值里移除，而不是填一个默认值。

引理 21.2 ($\min$ 与 clamp 的两个技术性质). (i) 对一组固定常数 $y_1, \dots, y_m$ ，映射

$$x \mapsto \min\{x, y_1, \dots, y_m\} \quad (21.2)$$

关于 x 非减；并且对任意两组实数，

$$\left| \min_k x_k - \min_k y_k \right| \leq \max_k |x_k - y_k|. \quad (21.3)$$

(ii) 夹取函数 $\text{clamp}(x, 0.2, 1.0) = \min\{1, \max\{0.2, x\}\}$ 关于 x 非减、连续、且是 1-Lipschitz 的；它的值域恰好是闭区间 $[0.2, 1.0]$ 。

这个引理在说什么： $\min$ 的动作是“就低不就高”， clamp 的动作是“卡在下限与上限之间”。两个动作都不可能把数字抬上去——这就是整章“只降不升”的机械原理。

证明. (i) 非减性。设 $x \leq x'$ 。对任意 k ， $\min\{x, y_k\} \leq \min\{x', y_k\}$ ，取 k 上的最小值仍保持不等号，故映射非减。

(i) Lipschitz 界 eq. (21.3)。记 $X = \min_k x_k$ 、 $Y = \min_k y_k$ ，不妨设 $X \leq Y$ （另一情形把 x, y 互换即可）。设下标 j 使 $x_j = X$ 。由 Y 是 $\{y_k\}$ 的最小值，有 $Y \leq y_j$ ；而 $y_j \leq x_j + |x_j - y_j| = X + |x_j - y_j| \leq X + \max_k |x_k - y_k|$ 。于是 $|X - Y| = Y - X \leq \max_k |x_k - y_k|$ 。

(ii) 夹取函数。 $x \mapsto \max\{0.2, x\}$ 是“下界截断”：它非减、连续，且是 1-Lipschitz 的（改变 x 一单位，输出改变不超过一单位）。 $u \mapsto \min\{1, u\}$ 是“上界截断”，同理非减、连续、1-Lipschitz。两个非减 Lipschitz 函数的复合仍非减、Lipschitz 常数为其常数之积 $1 \cdot 1 = 1$ ，且连续。

值域： $\max\{0.2, x\} \geq 0.2$ ，故 $\text{clamp} \geq 0.2$ ；又 $\min\{1, \cdot\} \leq 1$ ，故 $\text{clamp} \leq 1$ 。两端都能取到（ $x \leq 0.2$ 时输出 0.2； $x \geq 1$ 时输出 1），所以值域恰为 $[0.2, 1.0]$ 。 ■

定理 21.3 ($\min$ 门控的“只降不升”)。在假设 21.1 下，定义生效仓位

$$r^* = \min\{r_0, \text{clamp}(a, 0.2, 1.0), \text{clamp}(d, 0.2, 1.0), \kappa\}. \quad (21.4)$$

则：

- (a) 不放大性：恒有 $r^* \leq r_0$ ；
- (b) 逐源单调性： r^* 关于 r_0 、 a 、 d 、 κ 每一个自变量分别非减；并且关于 κ 与两个夹取输入是连续的；
- (c) 有界性： $r^* \in [0, 1]$ ；特别地，当四个来源都齐备时 $r^* \geq \min\{r_0, 0.2\}$ ；
- (d) 缺项的单调性：记可用来源集合为 S 、相应结果为 r_S^* ，若 $S' \subseteq S$ ，则 $r_{S'}^* \geq r_S^*$ 。

这个定理在说什么，我翻译成一句人话：任何一个来源都只能把仓位往下按，没有任何一个来源有资格把它往上抬。某一路意见转好了，仓位最多是“不那么低”，绝不会因为别人变好而反过来变小；某一路数据缺失了，仓位只会“不减”，系统不会偷偷帮它补一票假票。

证明. (a) 最小值不超过它所比较的任何一个数。把 r_0 看作被取最小值的成员之一，由 theorem 21.2(i)（令 $x = r_0$ 、其余三项为常数）即得 $r^* \leq r_0$ 。

(b) 单调性。把 eq. (21.4) 看成四个输入的函数。对 r_0 ：直接由 theorem 21.2(i) 得到非减。对 a ： $\text{clamp}(\cdot, 0.2, 1.0)$ 由 theorem 21.2(ii) 非减，再经 $\min$ (theorem 21.2(i)) 复合，非减； d 同理。对 κ ： $\kappa \mapsto \min\{c_1, c_2, c_3, \kappa\}$ (c_i 为其余三项) 是 $x \mapsto \min\{x, \text{常数}\}$ 的形式，非减。

(b) 连续性。 $\text{clamp}(\cdot, 0.2, 1.0)$ 连续 (theorem 21.2(ii))； $\min$ 是有限个连续函数的最小值，连续。于是 r^* 作为 (a, d, κ) 的函数是连续函数的复合，连续。此外由 eq. (21.3) 与 theorem 21.2(ii) 的 1-Lipschitz 性， r^* 关于每一个自变量都是 1-Lipschitz 的——比“连续”更强：任何一路意见变动一小步，生效仓位的变化不会超过同样大小。

(c) 有界性。由假设 21.1，四项分别落在 $[0, 1]$ 、 $[0.2, 1]$ 、 $[0.2, 1]$ 、 $[0.4, 1]$ ，都是 $[0, 1]$ 的子集。有限多个 $[0, 1]$ 内元素的最小值仍落在 $[0, 1]$ 内，故 $r^* \in [0, 1]$ 。

下界：后三项分别满足 $\text{clamp}(a, \cdot) \geq 0.2$ 、 $\text{clamp}(d, \cdot) \geq 0.2$ 、 $\kappa \geq 0.4$ 。对最小值有

$$r^* = \min\{r_0, \geq 0.2, \geq 0.2, \geq 0.4\} \geq \min\{r_0, 0.2, 0.2, 0.4\} = \min\{r_0, 0.2\}. \quad (21.5)$$

(d) 缺项。删掉一个来源，等于从被取最小值的那个有限集合里去掉一个元素。集合 T' 是 T 的子集时， T' 的最小值不小于 T 的最小值——因为 T 里多出来的那些元素只会把最小值“拉得更低或持平”。所以 $S' \subseteq S \Rightarrow r_{S'}^* \geq r_S^*$ 。 ■

这个定理有一个立刻能用的推论，它解释了系统的“硬回退”为什么安全。

推论 21.4 (谁有资格把仓位压到两折以下)。在假设 21.1 下：

- (a) 若 LLM 主张层失效 (未配置、超时、解析失败)，系统退回到 $S' = \{r_0\}$ ，此时 $r_{S'}^* = r_0 \geq r^*$ 。也就是说，主张层缺席只会让仓位变大或持平，绝不会更小——这是一个安全的降级方向；
- (b) 若 $r_0 \geq 0.2$ ，则 $r^* \geq 0.2$ 。主张与异议各自的下界 0.2 是“LLM 最多只能把仓位砍到两折”的硬底；
- (c) 因此，只有规则门 r_0 有资格把生效仓位压到 0.2 以下，也只有它有资格把仓位压到建仓闸门的那条线 (0.15) 之下。

证明. (a) 由定理 21.3(d), 取 $S' = \{r_0\} \subseteq S$ 得 $r_{S'}^* \geq r_S^*$; 而单个来源的最小值就是它本身, $r_{S'}^* = r_0$ 。

(b) 由定理 21.3(c) 的 $r^* \geq \min\{r_0, 0.2\}$, 在 $r_0 \geq 0.2$ 时右端等于 0.2。

(c) 由 (b): 只要 $r_0 \geq 0.2$, 无论主张、异议、Kelly 怎么低, r^* 都不低于 0.2。所以要出现 $r^* < 0.2$, 必要条件就是 $r_0 < 0.2$, 即只能由规则门造成。 ■

这条推论的工程含义很实在：**别指望 LLM 帮你把关机键**。它最多按到两折，真正的“关机”必须由规则门和闸门完成。

21.4.1 缩放本身：保方向、不放大敞口

定理 21.3 管的是“系数怎么算出来”。还有一个问题：把系数乘到订单上，会不会改变交易的方向？会不会把风险放大？下面这个定理回答它。

先把缩放写清楚。策略生成一批买入订单 $\{(q_i, p_i)\}_{i=1}^n$, $q_i > 0$ 是数量（股）、 $p_i > 0$ 是价格。按 A 股的整手规则，缩放后的数量向下取整到 100 股：

$$q'_i(r) = 100 \left\lfloor \frac{q_i r}{100} \right\rfloor, \quad r \in (0, 1). \quad (21.6)$$

定理 21.5 (缩放保方向与敞口单调). 设 $r \in (0, 1)$, $\{(q_i, p_i)\}$ 为一组买入订单。则：

(a) **方向不变**: $q'_i(r) \geq 0$, 缩放的符号永远非负，绝不会把买入翻成卖出；且 $q'_i(r) \leq q_i$, 单笔数量不被放大；

(b) **敞口单调**: 组合的名义敞口

$$N(r) = \sum_{i=1}^n p_i q'_i(r) \quad (21.7)$$

关于 r 非减: $0 < r_1 \leq r_2 < 1 \Rightarrow N(r_1) \leq N(r_2)$;

(c) **总敞口不放大**: $N(r) \leq \sum_{i=1}^n p_i q_i$ 。

这个定理在说什么：缩放只能把“买多买少”往下调，调不动“买还是卖”这个方向，也永远不会让总敞口超过原样下单。它就是限速牌的定义——限速牌不能给车加油门。

证明. (a) 因为 $q_i > 0$ 、 $r > 0$, 有 $q_i r > 0$, 故 $\lfloor q_i r / 100 \rfloor \geq 0$, 于是 $q'_i(r) = 100 \lfloor q_i r / 100 \rfloor \geq 0$: 符号非负，方向不变。

不放大: $\lfloor q_i r / 100 \rfloor \leq q_i r / 100$, 两边乘 100 得 $q'_i(r) \leq q_i r$; 又 $r < 1$, 故 $q'_i(r) \leq q_i r < q_i$ 。

顺带一个边界: 若 $q_i r < 100$, 则 $q'_i(r) = 0$ ——整手取整会让过小的订单直接退化为“不成交”。这不是 bug, 是 A 股整手买卖的必然结果（第 chapter 二十三 章会专门讲整手）。

(b) 设 $r_1 \leq r_2$ 。因为 $q_i > 0$, 有 $q_i r_1 \leq q_i r_2$; 除以正数 100 后不等号不变, 再取整 ($\lfloor \cdot \rfloor$ 非减) 得 $\lfloor q_i r_1 / 100 \rfloor \leq \lfloor q_i r_2 / 100 \rfloor$; 乘正数 100、再乘正数 p_i , 每一项都满足 $p_i q'_i(r_1) \leq p_i q'_i(r_2)$ 。对 i 求和, $N(r_1) \leq N(r_2)$ 。

（这一步只能是“非减”而不是“严格增”：取整是阶梯函数，在同一个台阶内 r 变大而 N 不变。）

(c) 由 (a) 的 $q'_i(r) \leq q_i$ 逐项成立，乘以 $p_i > 0$ 后在 i 上求和，即得 $N(r) \leq \sum_i p_i q_i$ 。 ■

定理 21.5 只在“只对买入订单做缩放”的前提下成立。卖出订单根本不在这个映射的定义域里——这件不对称的事，值得单独说（见纠偏三）。

21.5 它在系统里长什么样

先看那句核心代码。它短到有点不像话。

```
1 // scaleOrdersByPositionRate 按六维判势仓位系数缩放买入订单（仅 BUY）：
2 // 策略信号仓位 = 原始信号仓位 × position_rate，卖出订单不受影响。rate>=1 时原样返回。
3 func scaleOrdersByPositionRate(orders []agents.OrderIntent, rate float64) []agents.
    OrderIntent {
4     if rate <= 0 || rate >= 1 {
5         return orders
6     }
7     scaled := make([]agents.OrderIntent, 0, len(orders))
8     for _, o := range orders {
9         if o.Side == "BUY" {
10            o.MaxNotional = math.Round(o.MaxNotional*rate*100) / 100
11            o.TargetWeight = math.Round(o.TargetWeight*rate*10000) / 10000
12            o.Reason += fmt.Sprintf("（六维判势×%.2f）", rate)
13        }
14        scaled = append(scaled, o)
15    }
16    return scaled
17 }
```

代码 21.1 按仓位系数缩放买入订单（节选，internal/cio/cio.go）

这段话在说什么：遍历订单列表，只看 Side == "BUY" 的那几笔，把它们 MaxNotional（最大名义金额）和 TargetWeight（目标权重）各乘一次 rate，并在下单理由后面追加一行“（六维判势 × 0.20）”，方便事后追溯。卖出订单原样放进结果列表，一个字段都没动。

两个工程细节值得停一下：一是缩放发生在金额和权重两个字段上，而不是数量字段——数量由后面的整手归一化决定；二是 rate <= 0 || rate >= 1 时直接返回原列表，即“不缩放”。rate >= 1 时不缩放是对的（限速牌高于发动机的能力，不干预）；rate = 0 时不缩放则有风险，所以系统不靠这个函数来阻止建仓——靠的是下面那扇硬闸门⁷。

21.5.1 硬闸门：该不该建仓，是另一个问题

```
1 // 六维判势硬阻断建仓（修复：暂停建仓门控）：
2 // 当六维判势给出“退潮风险”或仓位系数极低（<= 0.15）时，判定为“强制暂停建仓”，
3 // 直接禁止一切新建仓/加仓（BUY 不生成），但卖出/减仓/止损/调仓不受影响。
4 buildBlocked := sixDim != nil && (sixDim.MarketTag == "退潮风险" || sixDim.PositionRate <=
    0.15)
```

⁷数据来源：系统源码 internal/cio/cio.go 的行 1024–1038。核对时间 2026-09-24。

代码 21.2 六维判势硬阻断建仓（节选，`internal/cio/cio.go`）

判据只有一行：标签是「退潮风险」，或者仓位系数不大于 0.15⁸。一旦命中，`buildBlocked` 为真，后面所有生成买入订单的分支（建仓、加仓、调仓、防守布局）都被这同一个开关挡住——卖出、减仓、止损、调仓则完全不受影响。

这就是本章开头那天的故事： $r_{\text{pos}}=0.06$ ，命中 $0.06 \leq 0.15$ ，闸门关闭，10 万那笔订单根本不会生成。所谓“连 6 千都没买成”，来源就是这一行。

21.5.2 盘中：同一把标尺，只量不记账

盘前判势会落库，盘中则不一样。交易时段内，调度器周期性调用 `MonitorIntradayInvestmentPlan`，它复用同一次判势，但把落库关掉：

```
1 // 六维判势（盘中复用，不落库以免覆盖盘前记录）
2 sixDim := c.sixDimReport(ctx, false)
3 if sixDim != nil {
4     if sixDim.PositionRate < 0.2 || sixDim.MarketTag == "退潮风险" {
5         marketWeak = true
6         c.addActivity("CIO", "SIXDIM_ALERT",
7             fmt.Sprintf("六维判势告警：市场标签「%s」，仓位系数%.2f，建议降低风险敞口、暂缓建仓",
8                 sixDim.MarketTag, sixDim.PositionRate), sixdimPayload)
9     }
10 }
```

代码 21.3 盘中复用判势与告警（节选，`internal/cio/cio.go`）

盘中的阈值是 `PositionRate < 0.2` 或标签含「退潮」，一旦命中就记一条 `SIXDIM_ALERT` 告警并转入观望⁹。不落库的理由很直接：盘前那条记录是当天的“正式判势”，盘中再写一次会把它覆盖掉，事后复盘就看不到盘前那一刻真实的判断了。

盘前闸门用的是 0.15、盘中告警用的是 0.2，这两个数不一样。我把它照实写在这里（见后面的工程笔记）。

21.5.3 把 5 条真实记录摆出来

系统库里 `market_sixdim_daily` 表当时共有 5 条真实记录。把判势结果和缩放结果并排放，这一章的全部机制一眼可见。

表里能看到两件事。第一，缩放是连续可调的：52.30 分给 0.80，41.82 分给 0.21，系数随分数滑。（第一行那个 0.80 是早期版本的记录，与当前 `ScoreToPosition` 公式给 52.30 分的 0.26¹⁰ 不一致，详见第 chapter 二十 章的纠偏。）第二，闸门是离散的：0.10 和 0.06 这两天，缩

⁸数据来源：系统源码 `internal/cio/cio.go` 的行 1123–1131。核对时间 2026-09-24。

⁹数据来源：系统源码 `internal/cio/cio.go` 的行 3925–3940；盘前落库的入口在同文件行 1119 的 `c.sixDimReport(ctx, true)`。核对时间 2026-09-24。

¹⁰复算数据：该数字为本书按系统实现中的同一公式、在系统的真实数据上重新计算所得；公式出处已在正文标

表 21.1 5 条真实六维判势记录：总分 → 仓位系数 → 建仓闸门

交易日	修正后总分	仓位系数	市场标签	10 万订单缩放后	是否被闸门拦下
2026-08-31	52.30	0.80	结构性震荡	8.00 万	否
2026-09-02	41.82	0.21	偏弱	2.10 万	否
2026-09-09	40.40	0.20	偏弱	2.00 万	否
2026-09-10	38.43	0.10	退潮风险	0（禁止买入）	是
2026-09-11	25.71	0.06	退潮风险	0（禁止买入）	是

“10 万订单缩放后”一列为本书按系统公式（买入金额 × 仓位系数；命中闸门时为 0）在真实系数上的复算；“是否被闸门拦下”依据 `MarketTag=="退潮风险" || PositionRate<=0.15`。

数据来源：前四列取自系统 `data/stock.duckdb` 的 `market_sixdim_daily` 表（真实数据，共 5 条，核对时间 2026-09-24）；后两列为本书按系统实现同公式复算。

放公式本来会给出 1 万和 6 千，但闸门把它们一起变成了 0。

21.5.4 前端怎么把这个数露出来

CIO 页面有一张【市场六维判势】卡片。它调用 `GetMarketSixDimReports(limit)`，参数 `limit` 默认 7、上限 30¹¹。CIO 页实际传的是 `GetMarketSixDimReports(7)`，也就是看最近 7 个交易日的判势¹²。接口侧还有一层 30 秒的短缓存，避免页面反复刷新时反复查库¹³。

判势结果还会随决策一起持久化：盘中与盘前构造的 `CIODecision` 都带一个 `SixDim` 字段，保存的是同一次判势的摘要（总分、修正分、冲突数、仓位系数、标签、来源），所以事后复盘时，“那天为什么只买这么点”能在决策记录里直接找到原文¹⁴。

注 21.6（一条我照实写下来的阈值不一致）。盘前的建仓闸门用的是 `PositionRate <= 0.15`，盘中的告警用的是 `PositionRate < 0.2`。这两个数不一样。

它们的任务也不同：盘前 0.15 是硬阻断（不生成买入订单），盘中 0.2 是告警并观望。于是中间那一段 `[0.15, 0.2)` 会出现一种状态：盘前允许买、盘中却会告警。我没有把它抹平成同一个数，因为两个阈值服务两个不同动作——这跟第 chapter 十六 章里“接口统一、实现对不齐”的故事属于同一类问题。

21.6 常见误解与纠偏

常见误解 · 有了缩放，就不需要闸门了 这是我在系统里真实修过的一个 bug，值得完整讲一遍。最初的实现只有 `scaleOrdersByPositionRate`。逻辑听着很顺：判势说退潮，我就把订单乘一个小系数，缩小不就行了？问题在于乘法乘不到 0。判势给 0.06 的时候，那笔买入还是会按原预算的 6%^a 落地；系数再低一点，还是会落地，只是更小。系统一边告警“退潮风险、建议暂缓建仓”，一边照样建仓——只不过

注。

¹¹数据来源：系统源码 `app_market.go` 的行 661–666：`limit <= 0` 置为 7，`limit > 30` 置为 30。核对时间 2026-09-24。

¹²数据来源：前端源码 `ui/src/pages/CIO.tsx` 的行 478。核对时间 2026-09-24。

¹³数据来源：系统源码 `app_market.go` 的行 628–704：`sixDimReportsCache` 与 `expireAt` 组成 30 秒 TTL。核对时间 2026-09-24。

¹⁴数据来源：结构体字段 `internal/port/agent.go` 行 176 的 `SixDim interface{\}`；赋值见 `internal/cio/cio.go` 行 1121 与行 3982。核对时间 2026-09-24。

下注小了一点。这就是源码注释里写的“越权路径”^b。

补上 `buildBlocked` 之后，机制才闭环：软缩放负责“买少一点”，硬闸门负责“今天不买”。一个管比例，一个管开关；两者不能互相替代。表 21.1 里 0.10 和 0.06 那两天的“0”，是闸门给的，不是缩放给的。

^a复算数据：该数字为本书按系统实现中的同一公式、在系统的真实数据上重新计算所得；公式出处已在正文标注。

^b数据来源：系统源码 `internal/cio/cio.go` 行 1126；注释原文为“彻底修复‘六维判势已提示退潮/暂停，却仍通过 `scaleOrdersByPositionRate` 仅缩小仓位、照常建仓’的越权路径”。核对时间 2026-09-24。

常见误解·选股和判势是一回事 “选股决定买什么，判势决定买多少”——这两件事经常被合成一句话：“今天市场不好，所以不买。”

听起来对，其实把两个模块混成了一个。市场不好，不等于没有值得买的股票；判势判的是环境，不是标的。这两件事的正确分工是：

- 选股（第 chapter 十九 章之前那一整套因子与筛选）在几千只票里挑出“买什么”，它给出的是**标的与相对排序**；
- 判势（第 chapter 十九、chapter 二十 章）给整个市场打一个分，它给出的是**当天的仓位尺度**；
- 仓位系数把两者接起来：好标的照挑，但今天统一下注小一点；环境太差时，连下注都不允许。

混在一起后果很具体：你会以为“市场不好”这句话已经把选股结果也否掉了，于是不再去看选股输出；或者反过来，你以为选出了好票就一定要买满。分开之后，系统里“选股一条链、判势一条链、缩放放在中间”的结构才立得起来——表 21.1 里同一个“10 万订单”，只在“下多少注”这一栏上不同^a。

^a数据来源：表 21.1 的修正后总分、仓位系数与标签三项取自 `market_sixdim_daily`，真实数据，核对时间 2026-09-24。

常见误解·为什么卖不跟着缩放？这不公平 读完 代码 21.1 的第一反应通常是这个：凭什么只缩放买入，卖出不动？

因为它本来就不该对称。买入和卖出在风险上的地位完全不同：

买入是增加风险敞口。你判断错了，损失是“新加进去的那部分”乘上后面的跌幅。**卖出是减少风险敞口。**你判断错了，代价只是“少赚”或者“卖早了”，而不是“多亏”。

所以在市场环境转差的时候，唯一需要收紧的方向是买入；卖出方向反而应该尽量放行。这条不对称设计在系统里出现了不止一处：六维判势的硬闸门只拦买入^a；盘中的六维保护与早盘保护同样“只拦截买入/新建仓/加仓，策略卖出/回撤熔断等离场风控不受影响”^b；策略驱动的减仓更是明确“优先于市场状态机，且不受六维仓位系数缩放影响”。**离场永远比进场更应该被放行**——这条原则，第 chapter 二十二 章讲三层风控时还会再遇到一次。

^a数据来源：系统源码 `internal/cio/cio.go` 行 1124-1125 的注释：“直接禁止一切新建仓/加仓（BUY 不生成），但卖出/减仓/止损/调仓不受影响。”核对时间 2026-09-24。

^b数据来源：系统源码 `internal/cio/cio.go` 行 3944-3945 的注释。核对时间 2026-09-24。

21.7 真实出处

本章的真实出处

- 缩放函数 `scaleOrdersByPositionRate` (只缩放 `BUY` 的 `MaxNotional` 与 `TargetWeight`) : `internal/cio/cio.go` 行 1024-1038
- 生效仓位 `Effective = min(ruleRate, clamp(主张, 0.2, 1.0), clamp(异议, 0.2, 1.0), KellyScale)` 与主张层: `internal/cio/proposal.go`; 调用点 `internal/cio/cio.go` 行 1283
- 建仓硬闸门 `buildBlocked` (退潮风险 或 `PositionRate <= 0.15`): `internal/cio/cio.go` 行 1123-1131、行 1223/1231/1262 的买入分支
- 盘中复用判势与 `SIXDIM_ALERT` (`PositionRate < 0.2` 或 退潮风险): `internal/cio/cio.go` 行 3925-3952; 入口 `MonitorIntradayInvestmentPlan` 行 3849
- 判势落库与读取: `internal/marketsixdim/persist.go` 的 `LatestReports`、`SaveReport`; 表 `market_sixdim_daily`
- 生效仓位持久化: `CIODecision.SixDim`, 字段定义 `internal/port/agent.go` 行 176, 赋值 `internal/cio/cio.go` 行 1121、行 3982
- 前端接口与卡片: `GetMarketSixDimReports(limit)` (默认 7、上限 30) `app_market.go` 行 648-707; `CIO` 页调用 `ui/src/pages/CIO.tsx` 行 478
- 分数到系数的映射 `ScoreToPosition` (修正后分数 $\rightarrow$ 仓位系数 + 标签) : `internal/marketsixdim/sixdim.go` 行 221-237

动手试一试（二十分钟）

1. 手算一次门控。取 $r_0 = 0.6$ 、主张 $a = 0.9$ 、异议 $d = 0.4$ 、 $\kappa = 0.78$ ，按 eq. (21.4) 算 $r^* = \min\{0.6, \text{clamp}(0.9, 0.2, 1.0), \text{clamp}(0.4, 0.2, 1.0), 0.78\}$ 。答案应当是 0.4。再留意一件事：就算你把异议 d 从 0.4 一路提到 0.9， r^* 也只是从 0.4 升到 0.6——永远不会超过规则门 0.6。这就是定理 21.3(a)。
2. 核对“只缩放买入”。打开 `internal/cio/cio.go`，在 `scaleOrdersByPositionRate` 里找到 `if o.Side == "BUY"` 这一行。把 `SELL` 的订单从头到尾看一遍，你会确认它除了被追加进结果列表，一个字段都没被改动。
3. 亲手算一遍表 21.1。挑 2026-09-02 那天： $10 \text{ 万} \times 0.21 = 2.10 \text{ 万}$ ；再挑 2026-09-10： $10 \text{ 万} \times 0.10 = 1 \text{ 万}$ ，但标签是「退潮风险」，按闸门判据 `MarketTag==" 退潮风险" || PositionRate<=0.15` 应当拦下，结果是 0。两笔对不上，说明你对闸门的理解还差一点。
4. 在系统里看六维卡片。打开 `CIO` 页，找到【市场六维判势】卡片，它显示的是最近 7 个交易日的判势。把卡片上的仓位系数和标签，跟本章表 21.1 的 5 条记录对一遍。
5. 验证“缺数据即少投票”。在源码里搜 `Effective = ruleRate`，找到主张层失败时的硬回退分支。按推论 21.4(a)，这个回退是“不减”的：它只会让仓位变大或持平，绝不变小。

到这一步，一整套“今天能不能下注、下多少注”的链路就通了：选股决定买什么，判势决定允许多快， $\min$ 门控保证没有任何一路意见能把仓位抬上去，硬闸门保证环境太差时干脆不下注。

可是还有一个问题悬在半空。这套机制全建立在“判断是对的”这个前提上——万一判势错了呢？万一买进去以后行情反向呢？

风控不是一堵墙，是三层走廊。下一章就来讲，当判断本身出错时，系统靠什么兜底。

第二十二章

风控不是一堵墙，是三层走廊

22.1 「最多亏 10%」这句话，能兑现吗

有人问我：我把最大回撤容忍设成 10%，是不是就保证最多亏 10%？

我张口就答「是」。后来在纸上画出四种亏法，才发现答错了。一条线拦不住亏损，因为穿过这条线的方式有很多种。

第一种，阴跌。每天跌 1%，十天亏 10%。这最「讲道理」：系统每天算回撤，8% 预警、12% 熔断降仓，线在这里有效。

第二种，跳空。早上出重大利空，开盘直接 -10%。昨天收盘回撤还是 0，今天开盘已亏 10%。线不是被磨穿，是**被跨过去了**——昨晚设的那个数，上午就作废。

第三种，停牌。票停牌两周，复牌后连续一字跌停，想卖也卖不掉——跌停价上的单子排成长队。回撤线早破了，可你连止损的动作都做不出来。

第四种，整体性。组合里十只票各跌 1%，却是同一天一起跌的。回撤不是某只票造成的，是整个水位一起下去了；你逐只检查，每只都「没超限」。

四种亏法说的是同一件事：**回撤是路径上的一个数，风险是一个分布**。拿一个数管一个分布，注定有漏洞。

这件事真的发生过一次。早期版本的单票上限与回撤容忍写在三处，数值却不一致——一处 30%、一处 20%¹。策略引擎以为自己在 20% 的笼子里跑，执行端按 30% 放行。**硬限制有两套口径，就等于没有硬限制。**

这一章只回答一件事：怎么把「最多亏多少」从一句愿望，变成三层真的会拦住你的走廊。

22.2 飞机不靠「更小心」

直觉·冗余不是加倍小心，是多一套系统 飞机有三套操纵系统：液压的、电动的、纯机械的（钢索直连）。它们不是「同一系统的三个备份」，而是**三条独立的路**：第一套坏了走第二套，第二套也坏了还能手动拉。飞行员并不比别人「更小心」，他只是坐在一架**坏一套也不会掉下来的**机器里。风控也一样。加一句「大家注意风险」不是风控，那只是语气；真正起作用的是：同一笔买单，要通过三条彼此独立的路，每条路都能单独把它拦下来。

「独立」是关键。三条路若共用同一个判断函数，它错一次，三条路一起错。所以三层必须由不同东西判断：第一层看写死的数，第二层看讲理（AI 评审），第三层看开关（熔断与阻断）；失效模式不一样，才叫冗余。

¹数据来源：系统事实清单第 15 节第 7 条。核对时间 2026-09-24。

22.3 为什么只有一堵墙会漏

一堵墙——「设一个阈值，破了就停」——漏在四处：**一是事后**，回撤是算出来的，你看见它到 10%，钱已经亏掉 10%；**二是单维**，它只问亏了多少，不问亏得多集中、多难卖；**三是静态**，阈值是你今天拍的数，风险明天换形状；**四是没接线**，事故 #7 就是这样——限制写在那里，两个模块各信各的。

所以系统不用一堵墙，用三层走廊：**你必须依次走过三个房间，任何一个房间都可以把你请出去。**

22.4 它在系统里长什么样：三层走廊

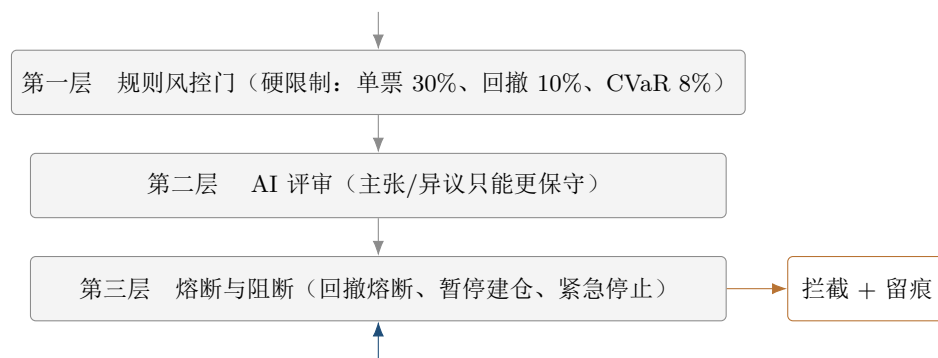


图 22.1 三层风控走廊：买单自上而下依次通过，任何一层都可以单独拦下它。拦截不是「静默丢单」，而是带结构化拒绝原因的留痕。

22.4.1 第一层：规则风控门——把数写死，并且只写一遍

第一层是写死的数字。它们集中在 `internal/policy/engine.go` 的 `HardLimits` 里。

表 22.1 九项硬限制的默认值，以及它在拦什么

限制名（结构体字段）	默认值	它在拦什么
<code>MaxSinglePosition</code>	0.30	单票占组合总资产上限
<code>MaxSectorExposure</code>	0.50	单行业合计暴露上限
<code>MaxPortfolioLeverage</code>	1.50	总杠杆（当前不融资，留接口）
<code>MaxDailyLoss</code>	0.05	单日损失上限
<code>MaxDrawdown</code>	0.10	组合最大回撤容忍（事故 #7 统一后的口径）
<code>MaxOrderSize</code>	100000	单笔名义金额上限（元）
<code>MaxTurnover</code>	1.00	年化换手率上限
<code>MaxSlippage</code>	0.005	单笔滑点上限
<code>MinLiquidity</code>	0.10	流动性下限（越界即视为流动性风险）

值为 `NewPolicyEngine` 的默认硬限制；「年化换手率」指单位资本一年内累计成交额倍数。

数据来源：系统源码 `internal/policy/engine.go` 的 `HardLimits` 结构体与 `NewPolicyEngine` 默认值；核对时间 2026-09-24。

表里有两个数要单独说：**单票 30%、回撤 10%**。它们是事故 #7 之后统一的权威口径，必须^①在三处同时成立：

- 风控师（`RiskLimits`）：写进审查文案；

- **策略引擎** (HardLimits, 上表): 下单前据此算违规;
- **风险中心** (internal/riskcenter/riskcenter.go 的 buildCompliance): 把「单票持仓上限」「最大回撤」做成两条独立校验, 阈值取 hl.MaxSinglePosition 与 hl.MaxDrawdown²。

第三处不是重复劳动, 而是**对账**: 前两处任一被改坏, 报告里都会亮出「超限」。

还有两条硬约束: 组合 $CVaR(95) \leq 8\%$ 、压力损失 $> 15\%$ 即否决³。8% 也用在综合风险评分: computeRiskScore 把它当 CVaR 归一化基准 (VaR 0.05、波动率 0.25、回撤 0.15)⁴。

22.4.2 第二层: AI 评审——只能更保守

第一层是死数, 不会思考; 第二层补的是「这段逻辑讲不讲得通」。做法是: 规则先给一个建议仓位 ruleRate, 主张官与异议者各说一句, 最后取最小:

$$\text{Effective} = \min(\text{ruleRate}, \text{clamp}(\text{主张}, 0.2, 1.0), \text{clamp}(\text{异议}, 0.2, 1.0), \kappa). \quad (22.1)$$

这个符号在讲什么: 四个数取最小, 意味着任何一个环节都不许把仓位放大。主张官说「加倍买」没用, 因为 clamp($\cdot$, 0.2, 1.0) 把它锁在 1.0 以内; ruleRate 若只有 0.3, 最终就是 0.3。

更关键的是**硬回退**: AI 未配置、调用失败、超时或输出解析不了时, 直接退回 ruleRate, 不给一个「猜的」仓位⁵。这与第 chapter 二十一章同源: AI 只能让仓位变小; 想放大必须回到第一层改硬限制, 那是需要人签字的地方。

22.4.3 第三层: 熔断与阻断——把开关按下去

前两层都在算「这一笔该不该放行」。第三层管的是「今天整个车间还开不开机」。它有三个开关。

开关一: 净值回撤熔断。系统设两条线: 预警线 8%、止损线 12%⁶。盘中触发预警线 → 暂停新建仓; 触发止损线 → 生成 DecisionPauseTrading 决策并强制降仓: 股票市值压回「总资产 $\times 0.5$ 」的现金目标, 卖出量向下取整到 100 股⁷。

开关二: 暂停建仓 (共享阻断开关)。它独立于紧急停止, 真源在 Policy 引擎: pauseBuilding() 读 policyEngine.IsPauseBuilding()⁸。它只拦买入, 卖出与止损照常。执

²数据来源: 系统源码 internal/riskcenter/riskcenter.go 的 buildCompliance。核对时间 2026-09-24。

³数据来源: 系统事实清单第 11 节; 情景构造见 internal/intelligence/risk_engine.go 的 generateStressScenarios。核对时间 2026-09-24。

⁴数据来源: 系统源码 internal/intelligence/risk_engine.go 的 computeRiskScore。核对时间 2026-09-24。

⁵数据来源: 系统事实清单第 7 节: LLM 未配置/失败/超时/解析失败时硬回退。核对时间 2026-09-24。

⁶数据来源: 系统源码 internal/cio/cio.go 的常量 drawdownWarnPct = 8.0、drawdownStopPct = 12.0, 判定函数 drawdownStatus。核对时间 2026-09-24。

⁷数据来源: 系统源码 internal/cio/cio.go 的 drawdownRiskReduceOrders (cashTargetPct = 0.5, sell = (sell/100)*100)。核对时间 2026-09-24。

⁸数据来源: 系统源码 internal/cio/cio.go 的 pauseBuilding 与 PauseBuilding。核对时间 2026-09-24。

行端在 `engine.Buy` 最前面生效——连手动点「买入」都会被拦⁹。

开关三：紧急停止。这是最重的那个。它同时拦买入和卖出，覆盖 CIO 建仓、自动股票池、下单工具、审批补确认所有入口¹⁰。

除这三个开关，还有一道来自第 chapter 十九 章的闸门：六维判势为「退潮风险」或仓位系数 $r_{\text{pos}} \leq 0.15$ 时，禁止生成新买入订单。库里 `market_sixdim_daily` 的真实记录中有两天正踩在闸门内侧：2026-09-10 的 r_{pos} 为 0.10¹¹、2026-09-11 的 r_{pos} 为 0.06¹²，两天的标签都是「退潮风险」¹³。那两天，系统只允许卖，不允许买。

22.4.4 执行端的最后一厘米：buyRiskGate

就算前三条走廊都放行了，扣现金之前还有一道硬复核：`buyRiskGate`¹⁴。它做两件事：

```
1 const (
2     maxSinglePositionPct = 0.30 // 单票占组合市值上限（与 policy 默认一致）
3     maxBuyCongestPct     = 0.08 // 单笔买入名义金额 <= 当日成交额的比例
4 )
5 // 1) 单票集中度：（已有持仓市值 + 本笔）/ 总资产 <= 30%
6 afterPct := (existingVal + gross) / totalAssets
7 if afterPct > maxSinglePositionPct { return "单票集中度超限：..." }
8 // 2) 买入流动性拥挤度：本笔名义金额 > 当日成交额 * 8% 则拒绝
9 if snap.Turnover > 0 && gross > snap.Turnover*maxBuyCongestPct {
10     return "买入流动性拥挤：薄盘大单难成交/难离场，已拦截"
11 }
```

代码 22.1 buyRiskGate：执行端硬风控（节选）

这段话在说什么：这道门只看买入，不看卖出，不过就返回结构化拒绝原因，绝不伪造成交易。第二道校验的物理含义是：这一笔若超过当天该票成交额的 8%，买进容易、卖出就难——在薄盘上，你的仓位就是你的对手盘。成交额读不到时跳过，不因缺数据误伤正常买入。

这里如实记一笔口径差异：事实清单第 11 节把第二道校验概括为「近 20 日换手率与名义成交额需达阈值」，源码落地的判据却是「本笔名义金额 $\leq$ 当日成交额 $\times 8\%$ 」——以源码为准。这种「文档一句话、代码一个实现」的偏差，正是三层走廊要防的。

22.5 把「最多亏多少」变成可算的数

现在回到开头那句话。要让「最多亏 10%」有意义，先得把它从「一条路径上的一个数」翻译成「一个分布上的一个量」。

⁹数据来源：系统源码 `internal/portfolio/engine.go` 的 `Buy`：入口处 `buyStopped()` 为真即返回错误，注释写明「执行层最终防线，独立于紧急停止——仅拦截买入」。核对时间 2026-09-24。

¹⁰数据来源：系统源码 `internal/portfolio/engine.go` 的 `emergencyStopped` 门控，同时出现在 `Buy` 与 `Sell` 开头；对照 `internal/portfolio/engine_test.go` 的 `TestEmergencyStopGateBlocksTrades`。核对时间 2026-09-24。

¹¹真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

¹²真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

¹³数据来源：系统事实清单第 13.3 节 `market_sixdim_daily` 的 5 条真实记录。核对时间 2026-09-24。

¹⁴数据来源：系统源码 `internal/portfolio/engine.go` 的 `buyRiskGate`（行 888 起），调用点在 `Buy` 中现金校验之前。核对时间 2026-09-24。

先给一个差不多对的模型。只看一天的损失，记成随机变量 L （亏损为正）。最朴素的想法是：「我有 95% 的把握不会亏超过某个数」——把「把握」当分位数，这个数就是后面要严格定义的风险价值（*Value at Risk*）。它在哪里失效？一是只报一个数，不告诉你超过之后有多惨；二是可能不满足「分散化一定不亏」的直觉——这一点用反例说清楚。

下面把它严谨化。先立三条假设，后面每一处引用都会点名。

假设 22.1 (H1: 可积性). 损失随机变量 L 满足 $\mathbb{E}|L| < \infty$ 。这条保证后文出现的分位数、条件期望与积分都有限、有意义。

假设 22.2 (H2: 分布连续). L 的分布函数 $F_L(\ell) = \mathbb{P}(L \leq \ell)$ 连续且在尾部严格递增。它保证「分位数形式」与「条件期望形式」的 CVaR 相等（见引理 22.9）。去掉这条，两种形式会在有原子的地方分家——本章的离散算例会当场演示这一点。

假设 22.3 (H3: 二阶矩). 需要用到波动率、Sharpe、Beta 这些指标时，额外假设 $\mathbb{E}[L^2] < \infty$ 。本章的三个定理都不需要它，标出来是为了说明：波动率类指标比 VaR/CVaR 类指标对分布的要求更强。

22.5.1 VaR：一个分位数

定义 22.4 (VaR). 给定置信水平 $\alpha \in (0, 1)$ ，损失 L 的 α -分位损失为

$$\text{VaR}_\alpha(L) = \inf\{\ell \in \mathbb{R} : \mathbb{P}(L \leq \ell) \geq \alpha\}. \quad (22.2)$$

口语一点：在任何「损失不超过 ℓ 」的概率已经不低于 α 的 ℓ 里，取最小的那个。

记 $\mathcal{A}_\alpha(L) = \{\ell : \mathbb{P}(L \leq \ell) \geq \alpha\}$ 。 $\mathbb{P}(L \leq \ell) \rightarrow 1$ ($\ell \rightarrow +\infty$) 说明它非空，分布函数单调不减说明它向上封闭， $\mathbb{P}(L \leq \ell) \rightarrow 0$ ($\ell \rightarrow -\infty$) 说明它下方有界——所以上面的下确界是有限实数，这个观察后面反复要用。

定理 22.5 (VaR 的三条好性质). 在假设 22.1 下，对任意 $\alpha \in (0, 1)$ ：

- (i)（单调性）若 $L_1 \leq L_2$ 几乎处处，则 $\text{VaR}_\alpha(L_1) \leq \text{VaR}_\alpha(L_2)$ 。
- (ii)（正齐次性）对一切 $\lambda > 0$ ， $\text{VaR}_\alpha(\lambda L) = \lambda \text{VaR}_\alpha(L)$ 。
- (iii)（平移不变性）对一切 $c \in \mathbb{R}$ ， $\text{VaR}_\alpha(L + c) = \text{VaR}_\alpha(L) + c$ 。

这个定理在说什么：(i) 亏得更多的那个，风险值不会更小；(ii) 仓位放大三倍，风险值放大三倍；(iii) 确定性地多亏 c 元，风险值整体平移 c 。三条都符合直觉——VaR 在「形状」上是规矩的。

证明. 先做一个统一的技术观察：三个部分的证明都归结为集合 $\mathcal{A}_\alpha$ 的比较——集合越大，下确界越小。

(i) 设 $L_1 \leq L_2$ 几乎处处。固定任意 ℓ 。因为每个满足 $L_2(\omega) \leq \ell$ 的 ω 都满足 $L_1(\omega) \leq L_2(\omega) \leq \ell$ ，除去零概率例外集后有

$$\{L_2 \leq \ell\} \subseteq \{L_1 \leq \ell\} \implies \mathbb{P}(L_1 \leq \ell) \geq \mathbb{P}(L_2 \leq \ell). \quad (22.3)$$

于是只要 $\ell \in \mathcal{A}_\alpha(L_2)$ ，就有 $\mathbb{P}(L_1 \leq \ell) \geq \mathbb{P}(L_2 \leq \ell) \geq \alpha$ ，即 $\ell \in \mathcal{A}_\alpha(L_1)$ 。故

$$\mathcal{A}_\alpha(L_2) \subseteq \mathcal{A}_\alpha(L_1). \quad (22.4)$$

一个集合变大，下确界只会变小（集合里多了元素，可选的候选更多），所以 $\inf \mathcal{A}_\alpha(L_1) \leq \inf \mathcal{A}_\alpha(L_2)$ ，即 (i)。

(ii) 固定 $\lambda > 0$ 。对任意 ℓ ，

$$\mathbb{P}(\lambda L \leq \ell) = \mathbb{P}\left(L \leq \frac{\ell}{\lambda}\right) = F_L\left(\frac{\ell}{\lambda}\right). \quad (22.5)$$

于是

$$\ell \in \mathcal{A}_\alpha(\lambda L) \iff F_L\left(\frac{\ell}{\lambda}\right) \geq \alpha \iff \frac{\ell}{\lambda} \in \mathcal{A}_\alpha(L) \iff \ell \in \lambda \mathcal{A}_\alpha(L), \quad (22.6)$$

即 $\mathcal{A}_\alpha(\lambda L) = \lambda \mathcal{A}_\alpha(L)$ 。因为 $\lambda > 0$ 保持实数大小次序不变，映射 $\ell \mapsto \lambda \ell$ 把下确界映为下确界： $\inf(\lambda \mathcal{A}) = \lambda \inf \mathcal{A}$ 。故 $\text{VaR}_\alpha(\lambda L) = \lambda \text{VaR}_\alpha(L)$ 。

(iii) 认真算一遍。对任意 ℓ ，

$$\mathbb{P}(L + c \leq \ell) = \mathbb{P}(L \leq \ell - c) = F_L(\ell - c). \quad (22.7)$$

于是 $\ell \in \mathcal{A}_\alpha(L+c) \iff \ell - c \in \mathcal{A}_\alpha(L) \iff \ell \in \mathcal{A}_\alpha(L) + c$ ，即 $\mathcal{A}_\alpha(L+c) = \mathcal{A}_\alpha(L) + c$ 。加法是保序的平移，平移不改变下确界相对结构， $\inf(\mathcal{A}+c) = \inf \mathcal{A} + c$ 。故 $\text{VaR}_\alpha(L+c) = \text{VaR}_\alpha(L) + c$ 。■

22.5.2 VaR 的致命缺陷：它不是次可加的

三条好性质里少了一条。风险度量最该有的一条叫次可加性（*subadditivity*）：

$$\text{VaR}_\alpha(L_1 + L_2) \leq \text{VaR}_\alpha(L_1) + \text{VaR}_\alpha(L_2). \quad (22.8)$$

它的现实含义是「分散化不会更糟」：两个东西合起来看的风险，不该超过各自风险之和。VaR 做不到，而且不是偶尔不成立，是结构性地不成立。

定理 22.6 (VaR 一般不是次可加的). 存在同一概率空间上的损失 L_1, L_2 与 $\alpha \in (0, 1)$ ，使

$$\text{VaR}_\alpha(L_1 + L_2) > \text{VaR}_\alpha(L_1) + \text{VaR}_\alpha(L_2). \quad (22.9)$$

也就是说，eq. (22.8) 不成立；VaR 因此不是一致风险度量。

这个定理在说什么：把两只「单独看都很安全」的资产放在一个组合里，VaR 会突然变大——它会把分散化读成风险上升。这就是引入 CVaR 的直接动机。

例 22.7 (两只独立债券：把反例算干净). 设组合里有两只债券 L_1, L_2 ，各自独立，违约概率都是 $p = 0.04$ ，违约损失为 100 元，不违约损失为 0。取 $\alpha = 0.95$ 。

单只的 VaR。 对 L_1 ： $\mathbb{P}(L_1 \leq 0) = 0.96 \geq 0.95$ ，而对任何 $\ell < 0$ 有 $\mathbb{P}(L_1 \leq \ell) = 0 < 0.95$ 。

故 $\mathcal{A}_{0.95}(L_1) = [0, +\infty)$, $\text{VaR}_{0.95}(L_1) = 0$. L_2 同理, 也是 0. 两者之和为

$$\text{VaR}_{0.95}(L_1) + \text{VaR}_{0.95}(L_2) = 0. \quad (22.10)$$

组合的 **VaR**. 和 $L_1 + L_2$ 的分布是

$$\begin{aligned} \mathbb{P}(L_1 + L_2 = 0) &= 0.96^2 = 0.9216, \\ \mathbb{P}(L_1 + L_2 = 100) &= 2 \times 0.04 \times 0.96 = 0.0768, \\ \mathbb{P}(L_1 + L_2 = 200) &= 0.04^2 = 0.0016. \end{aligned} \quad (22.11)$$

因为 $0.9216 < 0.95 \leq 0.9984 = 0.9216 + 0.0768$, 而对 $\ell \in [0, 100)$ 有 $\mathbb{P}(L_1 + L_2 \leq \ell) = 0.9216 < 0.95$, 所以

$$\mathcal{A}_{0.95}(L_1 + L_2) = [100, +\infty), \quad \text{VaR}_{0.95}(L_1 + L_2) = 100. \quad (22.12)$$

结论. $100 > 0 + 0$, 次可加性被打破: 两个各自「95% 把握一分不亏」的资产, 合在一起却有 5% 以上的概率至少亏 100 元¹⁵. 注意这里 $p = 0.04 < 1 - \alpha = 0.05$ 是关键: 单只落在尾部之外, 合起来就把尾部填满了。

定理 22.6 的证明. 取 theorem 22.7 中的 L_1, L_2 与 $\alpha = 0.95$. 按定义 22.4 逐项计算 (详细数值推导见上例), 得

$$\text{VaR}_{0.95}(L_1) = \text{VaR}_{0.95}(L_2) = 0, \quad \text{VaR}_{0.95}(L_1 + L_2) = 100, \quad (22.13)$$

于是 $100 > 0 + 0$, eq. (22.8) 的反例成立。 ■

22.5.3 CVaR: 把分位数换成尾部均值

VaR 报的是一条线。既然这条线会骗人, 那就别只要线——顺便问一句「线的那一边, 平均有多惨」。

定义 22.8 (CVaR). 对 $\alpha \in (0, 1)$, 损失 L 的 α -条件风险价值定义为分位数对置信水平的积分平均:

$$\text{CVaR}_\alpha(L) = \frac{1}{1 - \alpha} \int_\alpha^1 \text{VaR}_u(L) du. \quad (22.14)$$

当假设 22.2 成立时, 它还有一种更好记的写法:

$$\text{CVaR}_\alpha(L) = \mathbb{E}[L \mid L \geq \text{VaR}_\alpha(L)]. \quad (22.15)$$

口语一点: 只看最坏的那 $1 - \alpha$ 段, 把它们的损失取平均。

¹⁵数据来源: 本章为说明定理 22.6 构造的手算例子, 非系统运行数据; 两只债券独立、违约概率 0.04、违约损失 100 元。

引理 22.9 (两种写法在连续分布下相等). 设假设 22.1、22.2 成立, 记 $q = \text{VaR}_\alpha(L)$. 则 eq. (22.14) 的积分等于 eq. (22.15) 的条件期望。

证明. H2 说 F_L 连续严格递增, 故它可逆, 且 $\text{VaR}_u(L) = F_L^{-1}(u)$ 、 $\mathbb{P}(L \geq q) = 1 - F_L(q) = 1 - \alpha$ 。

对 eq. (22.14) 做换元 $u = F_L(x)$, 则 $du = f_L(x) dx$, 且 $F_L^{-1}(F_L(x)) = x$; 下限 $u = \alpha$ 对应 $x = q$, 上限 $u = 1$ 对应 $x \rightarrow +\infty$ 。于是

$$\int_{\alpha}^1 \text{VaR}_u(L) du = \int_q^{+\infty} x f_L(x) dx = \mathbb{E}[L \mathbf{1}\{L \geq q\}]. \quad (22.16)$$

两边同除以 $1 - \alpha = \mathbb{P}(L \geq q)$:

$$\frac{1}{1 - \alpha} \int_{\alpha}^1 \text{VaR}_u(L) du = \frac{\mathbb{E}[L \mathbf{1}\{L \geq q\}]}{\mathbb{P}(L \geq q)} = \mathbb{E}[L \mid L \geq q]. \quad (22.17)$$

即 eq. (22.14) 与 eq. (22.15) 相等。 ■

注 22.10 (H2 不是装饰). theorem 22.9 用到两处 F_L 连续: 可逆, 以及 $\mathbb{P}(L \geq q) = 1 - \alpha$ 。 L 若含原子 (离散分布), F_L 在跳跃点不可逆, 两种写法就分家。theorem 22.7 就是证据: 按积分形式 eq. (22.14) 算得 80, 按条件期望形式算得 $\mathbb{E}[L_1 \mid L_1 \geq 0] = 4$ 。同一个名字, 两个数。系统的历史法实现取积分/尾部集合的口径。

22.5.4 CVaR 的一致性

一致风险度量要满足四条: 单调、正齐次、平移不变、次可加。VaR 前三条有、第四条没有; CVaR 四条都有。

定理 22.11 (CVaR 满足四条一致性公理). 设假设 22.1、22.2 成立, $\alpha \in (0, 1)$ 。则:

- (i) (单调性) $L_1 \leq L_2$ 几乎处处 $\Rightarrow \text{CVaR}_\alpha(L_1) \leq \text{CVaR}_\alpha(L_2)$;
- (ii) (正齐次性) $\lambda > 0 \Rightarrow \text{CVaR}_\alpha(\lambda L) = \lambda \text{CVaR}_\alpha(L)$;
- (iii) (平移不变性) $c \in \mathbb{R} \Rightarrow \text{CVaR}_\alpha(L + c) = \text{CVaR}_\alpha(L) + c$;
- (iv) (次可加性) $\text{CVaR}_\alpha(L_1 + L_2) \leq \text{CVaR}_\alpha(L_1) + \text{CVaR}_\alpha(L_2)$ 。

这个定理在说什么: VaR「把分散化读成风险上升」的毛病, CVaR 没有了。前三条继承自积分, 第四条是它真正的价值; 完整推导在 chapter F。

引理 22.12 (CVaR 的共轭 (上确界) 表示). 设假设 22.1、22.2 成立, 记 $\lambda_\alpha = \frac{1}{1-\alpha}$ 。令

$$\mathcal{Z}_\alpha = \left\{ Z \mid Z \text{ 可测}, 0 \leq Z \leq \lambda_\alpha \text{ a.s.}, \mathbb{E}Z = 1 \right\}. \quad (22.18)$$

则

$$\text{CVaR}_\alpha(L) = \sup_{Z \in \mathcal{Z}_\alpha} \mathbb{E}[ZL]. \quad (22.19)$$

这个引理在说什么: 把所有「重新加权概率、最多放大 λ_α 倍」的测度拿来, 对损失求期望再取最大——最大值恰好是 CVaR。它把「尾部平均」翻译成「最坏重加权的平均」。

证明. 第一步：证明一个中间表示（Rockafellar–Uryasev 形式）

$$\text{CVaR}_\alpha(L) = \inf_{t \in \mathbb{R}} \left\{ t + \lambda_\alpha \mathbb{E}[(L - t)^+] \right\}, \quad (x)^+ = \max(x, 0), \quad (22.20)$$

并且 $t = q = \text{VaR}_\alpha(L)$ 处取到下确界。

先证「 $\leq$ 」。对任意 t ，每个 u 都有 $\text{VaR}_u(L) \leq t + (\text{VaR}_u(L) - t)^+$ ，两边在 $[\alpha, 1]$ 上积分：

$$\int_\alpha^1 \text{VaR}_u(L) du \leq (1 - \alpha)t + \int_\alpha^1 (\text{VaR}_u(L) - t)^+ du. \quad (22.21)$$

再看被积项：因假设 22.2 下 $\text{VaR}_u(L) = F_L^{-1}(u)$ ，换元 $u = F_L(x)$ 得

$$\int_{F_L(t)}^1 (F_L^{-1}(u) - t)^+ du = \int_t^{+\infty} (x - t) f_L(x) dx = \mathbb{E}[(L - t)^+]. \quad (22.22)$$

而 $[\alpha, 1]$ 与 $[F_L(t), 1]$ 的位置关系决定方向：若 $F_L(t) \leq \alpha$ ，被积函数非负，区间更短的 $[\alpha, 1]$ 上积分不超过 $[F_L(t), 1]$ 上的；若 $F_L(t) > \alpha$ ，则 $[\alpha, F_L(t)]$ 上 $F_L^{-1}(u) \leq t$ 、被积函数为 0，两段相等。合起来：

$$\int_\alpha^1 (\text{VaR}_u(L) - t)^+ du \leq \mathbb{E}[(L - t)^+]. \quad (22.23)$$

代入并除以 $1 - \alpha$ 得 $\text{CVaR}_\alpha(L) \leq t + \lambda_\alpha \mathbb{E}[(L - t)^+]$ 对一切 t 成立，故

$$\text{CVaR}_\alpha(L) \leq \inf_t \left\{ t + \lambda_\alpha \mathbb{E}[(L - t)^+] \right\}. \quad (22.24)$$

再证「 $\geq$ 」，即在 $t = q$ 处取到。由 H2， $F_L(q) = \alpha$ ，上面那个换元在这一情形取等号，故 $\int_\alpha^1 (\text{VaR}_u - q)^+ du = \mathbb{E}[(L - q)^+]$ 。于是

$$\begin{aligned} q + \lambda_\alpha \mathbb{E}[(L - q)^+] &= q + \lambda_\alpha \int_\alpha^1 (\text{VaR}_u(L) - q) du \\ &= q + \lambda_\alpha \int_\alpha^1 \text{VaR}_u(L) du - \lambda_\alpha q(1 - \alpha) = \text{CVaR}_\alpha(L), \end{aligned} \quad (22.25)$$

最后一步用了 $\lambda_\alpha(1 - \alpha) = 1$ 。故下确界不超过 $\text{CVaR}_\alpha(L)$ ，与上一段合起来即得 eq. (22.20)。

第二步：证明 eq. (22.19) 的两个方向。

($\leq$) 任取 $Z \in \mathcal{Z}_\alpha$ 与任意 $t \in \mathbb{R}$ 。由 $L - t \leq (L - t)^+$ 且 $Z \geq 0$ 得 $(L - t)Z \leq (L - t)^+ Z$ ；再由 $(L - t)^+ \geq 0$ 且 $Z \leq \lambda_\alpha$ 得 $(L - t)^+ Z \leq \lambda_\alpha (L - t)^+$ 。取期望：

$$\mathbb{E}[ZL] = \mathbb{E}[Z]t + \mathbb{E}[(L - t)Z] = t + \mathbb{E}[(L - t)Z] \leq t + \lambda_\alpha \mathbb{E}[(L - t)^+]. \quad (22.26)$$

(第二个等号用了 $\mathbb{E}Z = 1$ 。) 对 t 取下确界并用 eq. (22.20) 得 $\mathbb{E}[ZL] \leq \text{CVaR}_\alpha(L)$ ；再对 Z 取上确界： $\sup_{Z \in \mathcal{Z}_\alpha} \mathbb{E}[ZL] \leq \text{CVaR}_\alpha(L)$ 。

($\geq$) 取 $q = \text{VaR}_\alpha(L)$ ，构造 $Z^* = \lambda_\alpha \mathbf{1}\{L > q\}$ 。它合法： $0 \leq Z^* \leq \lambda_\alpha$ ；由 H2 的连续性

$\mathbb{P}(L > q) = \mathbb{P}(L \geq q) = 1 - \alpha$ ，故

$$\mathbb{E}Z^* = \lambda_\alpha \mathbb{P}(L > q) = \frac{1 - \alpha}{1 - \alpha} = 1. \quad (22.27)$$

再算期望，用 theorem 22.9:

$$\mathbb{E}[Z^*L] = \lambda_\alpha \mathbb{E}[L \mathbf{1}\{L > q\}] = \mathbb{E}[L \mid L > q] = \text{CVaR}_\alpha(L). \quad (22.28)$$

所以 $\sup_{Z \in \mathcal{Z}_\alpha} \mathbb{E}[ZL] \geq \text{CVaR}_\alpha(L)$ 。两个方向合起来就是 eq. (22.19)。■

定理 22.11 的证明. (i) 由定理 22.5(i)，对每个 u 有 $\text{VaR}_u(L_1) \leq \text{VaR}_u(L_2)$ 。被积函数逐点不大，积分就不大，两边同乘 $\lambda_\alpha > 0$ 得

$$\text{CVaR}_\alpha(L_1) \leq \text{CVaR}_\alpha(L_2). \quad (22.29)$$

(ii) 由定理 22.5(ii)， $\text{VaR}_u(\lambda L) = \lambda \text{VaR}_u(L)$ 对每个 u 成立。常数可以提出积分：

$$\text{CVaR}_\alpha(\lambda L) = \lambda_\alpha \int_\alpha^1 \lambda \text{VaR}_u(L) du = \lambda \lambda_\alpha \int_\alpha^1 \text{VaR}_u(L) du = \lambda \text{CVaR}_\alpha(L). \quad (22.30)$$

(iii) 由定理 22.5(iii)， $\text{VaR}_u(L + c) = \text{VaR}_u(L) + c$ 。于是

$$\text{CVaR}_\alpha(L + c) = \lambda_\alpha \int_\alpha^1 (\text{VaR}_u(L) + c) du = \text{CVaR}_\alpha(L) + \lambda_\alpha c (1 - \alpha) = \text{CVaR}_\alpha(L) + c, \quad (22.31)$$

末步用 $\lambda_\alpha(1 - \alpha) = 1$ 。

(iv) 这一步只能走共轭表示——积分对「和的尾部区间」没有直接控制。任取 $Z \in \mathcal{Z}_\alpha$ ，期望对加法线性：

$$\mathbb{E}[Z(L_1 + L_2)] = \mathbb{E}[ZL_1] + \mathbb{E}[ZL_2] \leq \text{CVaR}_\alpha(L_1) + \text{CVaR}_\alpha(L_2), \quad (22.32)$$

末步对两项分别用引理 22.12。右边与 Z 无关，故对 Z 取上确界仍不超过它：

$$\text{CVaR}_\alpha(L_1 + L_2) = \sup_{Z \in \mathcal{Z}_\alpha} \mathbb{E}[Z(L_1 + L_2)] \leq \text{CVaR}_\alpha(L_1) + \text{CVaR}_\alpha(L_2). \quad (22.33)$$

(第一个等号再次用引理 22.12；这张引理之所以要证得那么细，就是因为这里。) (i)–(iv) 即四条一致性公理。■

顺手把 theorem 22.7 里的 CVaR 算出来对照： $\text{CVaR}_{0.95}(L_1) = \text{CVaR}_{0.95}(L_2) = 80$ ， $\text{CVaR}_{0.95}(L_1 + L_2) = 103.2$ ，于是 $80 + 80 \geq 103.2$ ——**VaR** 破的那条线，**CVaR** 守住了¹⁶。

¹⁶数据来源：同一算例按定义 22.8 的积分形式手算；非系统运行数据。

22.5.5 CVaR 至少和 VaR 一样大

最后补一条直觉上「必须成立」的性质，它说明 CVaR 不是换了个东西，而是 VaR 的加严版。

引理 22.13 (VaR 关于置信水平单调). 设 $0 < \alpha \leq \beta < 1$ ，则 $\text{VaR}_\alpha(L) \leq \text{VaR}_\beta(L)$ 。

证明. 若 $\ell \in \mathcal{A}_\beta(L)$ ，则 $\mathbb{P}(L \leq \ell) \geq \beta \geq \alpha$ ，故 $\ell \in \mathcal{A}_\alpha(L)$ ，即 $\mathcal{A}_\beta(L) \subseteq \mathcal{A}_\alpha(L)$ ，取下确界得 $\text{VaR}_\alpha(L) \leq \text{VaR}_\beta(L)$ 。■

定理 22.14 ($\text{CVaR} \geq \text{VaR}$). 设假设 22.1 成立，则对一切 $\alpha \in (0, 1)$ ，

$$\text{CVaR}_\alpha(L) \geq \text{VaR}_\alpha(L). \quad (22.34)$$

这个定理在说什么：别以为「换个更高级的指标」能把风险数字变小。CVaR 只会更悲观——它算尾部整段的平均，VaR 只看那段最轻的一点。

证明. 由引理 22.13，函数 $u \mapsto \text{VaR}_u(L)$ 在 $[\alpha, 1]$ 上非减，故对每个 $u \in [\alpha, 1]$ 有 $\text{VaR}_u(L) \geq \text{VaR}_\alpha(L)$ 。在区间上逐点比较后积分：

$$\int_\alpha^1 \text{VaR}_u(L) du \geq \int_\alpha^1 \text{VaR}_\alpha(L) du = (1 - \alpha) \text{VaR}_\alpha(L). \quad (22.35)$$

两边同除 $1 - \alpha > 0$ ，得 $\text{CVaR}_\alpha(L) \geq \text{VaR}_\alpha(L)$ 。■

22.5.6 系统里的历史法：指标是怎么算出来的

系统里的风险指标有两个引擎，各司其职：

- **传统风控** (`internal/risk/engine.go`): `RiskMetrics` 里排开 `VaR95`、`VaR99`、`CVaR95`、`Volatility`、`SharpeRatio`、`MaxDrawdown`、`Beta`、`SortinoRatio`、`CalmarRatio`（外加 `R2`、`DownsideDev`、`UpsideDev`）；止损止盈在 `DefaultStopLossConfig`：初始止损 0.05、移动止损 0.03、ATR 倍数 2.0、最大持仓 120 天、止盈锁定 0.20/回撤 0.10。
- **计算中台** (`internal/intelligence/risk_engine.go`): `ComputePortfolioRisk` 输出 `VaR/ES`、年化波动率、最大回撤、`Beta`、相关性、集中度 (`HHI`)、流动性代理与压力测试六情景，合成 `OverallScore` 与 `NORMAL`/`WATCH`/`WARNING`/`CRITICAL` 四档（分界 0.3/0.5/0.7）。

两边都是**历史法**：排序收益率序列，用序号定位分位数。以 `internal/risk/engine.go` 为例，`CalculateVaR` 取 `idx = \lfloor (1 - \text{confidence}) \cdot n \rfloor`，`CalculateCVaR` 取排序后前 `idx + 1` 个（最坏尾部）的绝对值均值¹⁷。这与本章理论口径有三处差别：样本有限（分位数落在某根真实 K 线上，不插值）、序号取整、损失取绝对值。压力测试六情景的冲击写死在源码里：-15%/-

¹⁷数据来源：系统源码 `internal/risk/engine.go` 的 `CalculateVaR`、`CalculateCVaR`；`internal/intelligence/risk_engine.go` 的 `computeHistoricalVaR`、`computeHistoricalCVaR` 同口径。核对时间 2026-09-24。

5%/-8%/-10%/-12%/-7%，估计损失按「基准波动率 × 倍数 × 冲击 × 10」计算并封顶到 1.0¹⁸。

这些指标的输入也要交代。组合日收益率由 `computePositionReturns` 给出：每个持仓取最近 120 天收盘价算日收益率，按市值占比加权，再按日期对齐相加；历史不足 10 个点的标的跳过，记「无法计算可靠的 VaR/ES」¹⁹。这条「不够就跳过、不伪造」的纪律跟全书一致。

这些指标本章只给口径、不给数值：口径可核对，数值不猜²⁰。`MaxDrawdown` 是净值曲线峰谷最大跌幅；`Beta`、`Sortino`、`Calmar` 分别是对基准回归、下行偏差、年化收益比回撤。

22.6 风控的最后一米：盘口五态

前三层管的是「你的账」，最后一米管的是「市场答不答应」：想卖得有人接，想买得有人卖。A 股把这最后一米压成五种状态。

`internal/orderbook/orderbook.go` 的 `Classify` 是个纯函数：吃进现价、涨跌停价、买一卖一的价与量、内外盘、量比、当日成交量，吐出一个状态，靠两条经验阈值：

$$\text{封单量阈值} = \max(1000 \text{ 手}, \text{当日成交量} \times 10\%), \quad \text{放量阈值} = 2.0. \quad (22.36)$$

表 22.2 盘口五态：判定条件与后果

状态	英文枚举	判定条件与后果
涨停封死	<code>sealed_limit_up</code>	触涨停 且 买一价 ≥ 涨停价 且 买一量 ≥ 封单阈值
涨停打开	<code>opened_limit_up</code>	触涨停但封单薄，可成交
跌停封死	<code>sealed_limit_down</code>	触跌停 且 卖一价 ≤ 跌停价 且 卖一量 ≥ 封单阈值
跌停打开	<code>opened_limit_down</code>	触跌停但封单薄（常伴巨量换手），可离场
正常	<code>normal</code>	未触及涨跌停

另两条信号：巨量跌停 = 触及跌停且量比 ≥ 2.0；抛压占优 = 内盘占比 > 0.60 (`InVolDominant`)。涨跌停判定带浮点容差 10^{-6} 。

数据来源：系统源码 `internal/orderbook/orderbook.go` 的 `Classify`、`sealThreshold` 与常量 `MinSealVolHands=1000`、`SealVolRatio=0.10`、`HugeVolumeRatio=2.0`、`InVolDominant=0.60`；核对时间 2026-09-24。

这五种状态怎么变成拦截规则？执行端只做三件事²²：

买入拦两样：跌停板，和涨停封死。拦跌停板是拦「接飞刀」：现价 ≤ 跌停价（容差 10^{-9} ）即拒；拦涨停封死是拦「排队买不进」：挂上去大概率当天成交不了，系统不装作成交。

卖出只拦一样：跌停封死。封死时你的单子排在长队里，当天卖不掉；拦下来反而保留持

¹⁸数据来源：系统源码 `internal/intelligence/risk_engine.go` 的 `generateStressScenarios` (`shock` 依次为上列六值)。核对时间 2026-09-24。

¹⁹数据来源：系统源码 `internal/cio/cio.go` 的 `computePositionReturns` (`historyDays = 120`)；事实清单第 11 节：无历史时用沪深 300 参考。核对时间 2026-09-24。

²⁰数据缺失：该系统库中此项无有效记录。本书遵照系统“数据不可用即报错、不降级、不伪造”的原则，此处留白、不填充估计值。

²¹数据来源：本书写作时未取到可核对的运行数值；系统库中对应记录不作为引用依据。核对时间 2026-09-24。

²²数据来源：系统源码 `internal/portfolio/engine.go` 的 `buyBlockReason` 与 `sellBlockReason`；`LimitPctForSymbol` 提供各板块涨跌停幅度。核对时间 2026-09-24。

仓。

跌停打开要放行。这条最反直觉：跌停打开意味着封单薄、换手大——门开着，赶紧按跌停价走。原则很明确：拦截是为了「别做无效的事」，不是「留住仓位」——封死的保留，打开的放行；行情读不到时一律不拦。

22.7 跑不掉的仓位等于负债：流动性锁定指标

买了卖不掉的仓位，等于没有流动性。系统为此单独算一个指标：`liquidityLockMetrics`²³——遍历持仓，逐只按实时快照判定盘口状态，「跌停封死」的市值算进「锁定市值」。

表 22.3 流动性锁定指标的字段与含义

字段	含义
<code>liquidity_locked_positions</code>	「跌停封死」的持仓只数
<code>liquidity_locked_value</code>	这些持仓的市值合计（元）
<code>position_value</code>	全部持仓市值合计（元）
<code>liquidity_locked_ratio_pct</code>	锁定市值占持仓市值百分比
<code>liquidity_locked_note</code>	数据来源与降级说明（无持仓 / 引擎未初始化时写明原因）

这组指标与收益历史无关：即使组合收益率不足、算不出 VaR，它照样算得出来。
数据来源：系统源码 `internal/tools/investment_tools.go` 的 `liquidityLockMetrics` 与 `PortfolioRiskMetricsTool.Execute`；核对时间 2026-09-24。

库里没有可引用的锁定比例记录，留白：²⁴²⁵。
盘口数据本身会落库。每天收盘后，`OrderBookDailyProcessor.RecordOrderBookDaily` 把可交易股票池当日的盘口摘要写进 `order_book_daily`，供次日的盘口因子使用——是次日，所以没有前视偏差²⁶。这张表必须点名留白：写入路径存在，但当前表内 0 行，本章任何「盘口因子回测结果」都无从引用 0 行²⁷²⁸。

22.8 常见误解与纠偏

常见误解·风控的目标是「不亏」 这也是我一开始的想法：风控做得越严，亏得越少，最好一分不亏。
错在把「风控」和「不交易」画了等号。风控的目标只有一句话：不让钱以你无法承受的方式亏掉。能承受的亏损照样会发生，那是换收益的成本。系统的拦截规则都贯彻这条：拦接飞刀、拦薄盘大单、拦排队买不进，却从不拦按计划的止损——跌停打开那天它甚至主动放行。

²³数据来源：系统源码 `internal/tools/investment_tools.go` 的 `liquidityLockMetrics`（行 734 起），供 `PortfolioRiskMetricsTool` 使用。核对时间 2026-09-24。
²⁴数据缺失：该系统库中此项无有效记录。本书遵照系统“数据不可用即报错、不降级、不伪造”的原则，此处留白、不填充估计值。
²⁵数据来源：该指标依赖实时盘口快照，`data/stock.duckdb` 中无对应历史落库记录。核对时间 2026-09-24。
²⁶数据来源：系统源码 `internal/scheduler/auto_scheduler.go` 的 `RecordOrderBookDaily`、`internal/data/duckdb.go` 中该表的建表与写入。核对时间 2026-09-24。
²⁷数据缺失：该系统库中此项无有效记录。本书遵照系统“数据不可用即报错、不降级、不伪造”的原则，此处留白、不填充估计值。
²⁸数据来源：系统事实清单第 13.2 节：`order_book_daily` 实测 0 行，引用时必须标「数据缺失」。核对时间 2026-09-24。

一个把仓位全部冻结的系统看起来「零风险」，其实是在替你做一个没授权的决定。这就是「暂停建仓」与「紧急停止」必须分成两个开关的原因：前者只拦买入，后者才全部停下——严格程度必须分级，「可控」才是目的。

常见误解·硬限制写在哪里都一样，反正都是同一个数 这是本章最重要的纠偏，因为系统真的在这里翻过车。

事故 #7 的根因是：单票上限和回撤容忍在两个模块里数值不一致，一处 30%、一处 20%（出处见本章第 1 节的同一事故脚注）。写的人以为「都是限制」，读的人以为「都一致」；结果是策略层按 20% 算违规、执行层按 30% 放行，谁也没错，限制却不存在。

修法不是「把两处改成一样」这么轻，而是让同一件事有多个可对账的地方：三处都写这两个数，且风险中心把「单票持仓上限」「最大回撤」做成报告里两条独立条目——谁被改坏，报告里马上出现「超限」。一致性不是靠约定，是靠对账。

这条经验对全部配置都成立：同一个限制出现在两个地方，就必须有一处是对账方而非执行方。

22.9 真实出处

本章的真实出处

- 硬限制九项默认值 ($MaxSinglePosition=0.30$ 、 $MaxDrawdown=0.10$ 、 $MaxDailyLoss=0.05$ 等) 与第三条对账口径: `internal/policy/engine.go` 的 `HardLimits`、`NewPolicyEngine`; `internal/riskcenter/riskcenter.go` 的 `buildCompliance`
- 执行端硬风控 (单票集中度复核 + 买入流动性拥挤度, $maxSinglePositionPct=0.30$ 、 $maxBuyCongestPct=0.08$): `internal/portfolio/engine.go` 的 `buyRiskGate`
- 盘口拦截 (`buyBlockReason`、`sellBlockReason`)、暂停建仓 (`buyStopped`)、紧急停止 (`emergencyStopped`): 开关真源 `internal/policy/engine.go` 的 `IsPauseBuilding`
- 净值回撤熔断 (预警 8%、止损 12%、降仓到现金 50%): `internal/cio/cio.go` 的 `drawdownStatus`、`drawdownRiskReduceOrders`
- AI 评审门控与硬回退 ($Effective = \min(ruleRate, \text{clamp}(\cdot, 0.2, 1.0), \kappa)$): 系统事实清单第 7 节
- 风险指标双引擎、历史法 $VaR/CVaR$ 、止损止盈默认参数、压力测试六情景: `internal/risk/engine.go`、`internal/intelligence/risk_engine.go`
- 组合日收益率序列 (每持仓 120 天、按市值加权、样本不足跳过) : `internal/cio/cio.go` 的 `computePositionReturns`
- 盘口五态与阈值: `internal/orderbook/orderbook.go` 的 `Classify`、`sealThreshold`
- 流动性锁定指标: `internal/tools/investment_tools.go` 的 `liquidityLockMetrics`
- 盘口日频摘要落库与读取: `internal/scheduler/auto_scheduler.go` 的 `RecordOrderBookDaily`、`internal/data/duckdb.go` 的 `order_book_daily`、`internal/screener/pro_factor_engine.go` 的 `loadOrderBookCache`
- 长推导 (共轭表示、Rockafellar-Uryasev 形式): [chapter F](#)

动手试一试

1. 手算反例。两条独立债券：违约概率各 0.04、违约损失 100，取 $\alpha = 0.95$ 。按定义 22.4 算得 $VaR_{0.95}(L_1) = 0$ 、 $VaR_{0.95}(L_1 + L_2) = 100$ ；把违约概率改成 0.01 再算—— $VaR_{0.95}(L_1 + L_2)$

变回 0，次可加性这次成立。可见 VaR 的失效取决于尾部概率与置信水平的具体关系。

2. 核对三处一致。`internal/policy/engine.go` 的 `NewPolicyEngine` (`MaxSinglePosition`、`MaxDrawdown`)、`internal/riskcenter/riskcenter.go` 的 `buildCompliance` (两条同名校验)、`internal/portfolio/engine.go` 的 `buyRiskGate` (`maxSinglePositionPct`)。三处都应是 0.30 / 0.10，任何一处不同就是事故 #7 复现。
3. 验证流动性锁定口径。读 `internal/tools/investment_tools.go` 的 `liquidityLockMetrics`，看它把哪一类盘口状态算作「锁定」；再到 `internal/orderbook/orderbook.go` 看该状态的条件——它需要卖一价与卖一量，不是只看现价。
4. 压力测试复算。用 `generateStressScenarios` 的公式 (基准波动 × 倍数 × 冲击 × 10，封顶 1.0) 算六个情景。若沪深 300 年化波动率为 0.20，「市场暴跌」情景 (倍数 3.0、冲击 -0.15) 的损失是多少？是否触发「> 15% 否决」？

到这里风控闭环了：三层走廊保证「不放大」，`buyRiskGate` 保证「拦得住」，盘口五态保证「拦得对」——拦无效动作、放有效动作。数学上，「最多亏多少」从一个数变成两个量：VaR 告诉你线在哪，CVaR 告诉你线那边有多惨；只有 CVaR 满足四条一致性公理，所以组合层面能相加、能优化的只有它。

可是这一切都有个前提：市场允许你交易。A 股自己还有脾气——今天买进的 1000 股今天不能卖；想卖时它可能躺在跌停板上；钱刚够买 90 股，规则却只认 100 股整手。这些不是模型的误差，是规则的形状。第 chapter 二十三 章就讲这些硬规矩。

第二十三章

A 股的几个硬规矩：T+1、整手与涨跌停

23.1 第一天的废单

我认识一个人，他在美股做过几年日频策略。策略本身简单得像一句口号：收盘前扫一遍动量榜，挑最壮的几只，第二天开盘价买入，收盘前全部换成当天新的最强名单。净值曲线很干净，回撤小，换手高——他管这叫“永远骑在最快的马上”。

他把同一套逻辑搬到了 A 股。

第一天上午九点四十，他按名单买齐了五只票。中午其中一只跌了六个点，按他的规则该立刻换掉。他点了卖出，系统弹回来一行字：

- 1 违反A股T+1交易规则：sh600xxx 可卖数量为 0 股（昨日持仓），
- 2 今日买入 1000 股不可卖出，请求卖出 1000 股

代码 23.1 一次真实的拒单（来自执行层校验）

这段话在说什么：他今天买进的 1000 股，今天一股都卖不掉。可卖数量是“当前持仓减去今日买入量”，而当前持仓里今天买入的那部分，在当天被冻结。

他愣了几秒，以为是自己下单数量写错了，改小，还是不行。改小到 100 股，依然不行——因为问题不在数量，在日期。

那天下午他一只都动不了，只能眼看着那五只票继续跌。第二天开盘他才把它们卖掉，而这一夜，恰好是他的策略里假设“根本没有隔夜敞口”的那一夜。

怪策略吗？不怪。他的回测没有算错，只是那段回测是在另一套交通规则下跑的。规则没有写进模型，模型就假装它不存在——这正是这一部分从第 12 章一路追到现在的主题：误差不是从数据来的，是从你没算进去的东西来的。

23.2 你的车再好，红灯也得停

直觉·车与路 一辆跑车在直线加速上能秒掉任何家用车。可它开到路口，红灯一亮，它也停在白线后面，和旁边那辆小面包并排。

为什么？因为速度是车的属性，怎么走是路的规矩，这两件事是分开的。

交易也是这样。“信号有多准、因子有多强、回测夏普有多高”，那是车。“当天买进能不能当天卖、一次最少买多少股、涨停了还能不能挂单”，那是路。车好不等于可以不看路。

更要紧的一层：路规是外生的，不会因为你模型漂亮而改一个数字。你唯一能做的，是在设计模型时就把红绿灯算进去；等下单那一刻才想起来，红绿灯已经在拦你了。

这些规矩大多是制度设计而不是技术限制：T+1 压日内投机，涨跌停给情绪装断路器，整手为了撮合后可结算。它们跟“市场有效性”是两码事，只回答一个底层问题：在 A 股，什么东西是你根本不许做的？

23.3 不把这些算进去，会付出四种代价

我把代价数了一遍，一共四种。

代价一：回测骗你。如果回测假装“当天买进当天就能卖”，它给出的就是**不存在的世界里**的收益。回测里那些漂亮的日内止损、日内换仓，实盘一笔都执行不了——多出来的收益不是策略赚的，是规则漏洞借给你的。

代价二：订单被直接拒绝。买入数量不是 100 的整数倍时，执行层会当场拒绝，而不是帮你凑整：

```
1 // A股最小单位检查
2 if quantity%100 != 0 {
3     return nil, fmt.Errorf("A股买入数量必须是100的整数倍")
4 }
```

代码 23.2 整手校验（节选）

这段话在说什么：一百股是 A 股买入的最小台阶，不是建议而是硬门槛——不是一百整数倍的下单量进不了账户。

代价三：信号对了，但成交不了。这是最容易被忽略的一种。策略说“买”，方向完全正确，可那只票此刻正躺在涨停板上封死，你的买单排在几万手后面，成交概率是零。这属于盘口问题，第 chapter 二十二章已经把盘口五态拆开讲过，这一章要把它和“制度”接上。

代价四：你选到了一个跑不掉的池子。如果选股偏爱“弹性大”的微盘股，建仓那天会很满意——涨得快；等要走时才发现，成交量只有几百万，仓位一天也走不完。

四种代价，第一种在回测里，后三种在下单那一刻，都指向同一件事：**制度和摩擦，是模型外面的一层硬壳。**

23.4 近似模型：现金只够买整手

先只关心一件事：手上有 C 元现金，每股价格 p 元，一手一百股，我最多能买多少股？

23.4.1 整手取整

定义 23.1 (整手取整算子). 取一手 $L = 100$ 股。对实数 $x \geq 0$ ，定义

$$\lfloor x \rfloor_L = L \left\lfloor \frac{x}{L} \right\rfloor. \quad (23.1)$$

这个定义在讲什么：先把手数向下取整再乘回一百，所以 $\lfloor x \rfloor_L$ 永远是整百的数。

引理 23.2 (整手取整的三条性质). 对任意 $x \geq 0$: (i) $\lfloor x \rfloor_L \leq x < \lfloor x \rfloor_L + L$; (ii) $\lfloor x \rfloor_L$ 是 L 的非负整数倍; (iii) 若 $n \in \mathbb{N}$ 、 $L \mid n$ 且 $n \leq x$ ，则 $n \leq \lfloor x \rfloor_L$ 。

这个引理在说什么：(i) 取整一定不放大，丢掉的部分不到一手；(ii) 结果永远落在最小台阶上；(iii) 任何不超过 x 的整手数都不会越过取整结果——这句是后面证明最优性的关键。

证明. (i) 由下取整的定义,

$$\left\lfloor \frac{x}{L} \right\rfloor \leq \frac{x}{L} < \left\lfloor \frac{x}{L} \right\rfloor + 1. \quad (23.2)$$

三边同乘 $L > 0$, 得

$$L \left\lfloor \frac{x}{L} \right\rfloor \leq x < L \left\lfloor \frac{x}{L} \right\rfloor + L, \quad (23.3)$$

左端与右端第一项正是 $\lfloor x \rfloor_L$, 即得 $\lfloor x \rfloor_L \leq x < \lfloor x \rfloor_L + L$.

(ii) $x \geq 0$ 时 $\lfloor x/L \rfloor$ 是非负整数, 记它为 k , 则 $\lfloor x \rfloor_L = Lk$, 是 L 的非负整数倍。

(iii) 由 $n \leq x$ 得 $n/L \leq x/L$ 。而 n/L 是整数 (因为 $L \mid n$), 一个整数不超过实数 x/L , 必然不超过它的下取整: $n/L \leq \lfloor x/L \rfloor$ 。两边同乘 L , 得 $n \leq L\lfloor x/L \rfloor = \lfloor x \rfloor_L$ 。■

23.4.2 现金约束下的最大可行购买量

先把费用放一边, 只要求“按名义金额 pq 算, 钱够”。

假设 23.3 (整手约束). 可买数量只能是整手: $q = kL$, 其中 $k \in \mathbb{N}$, $L = 100$ 。

假设 23.4 (买入成本模型). 买入的单位成本含佣金与最低佣金项,

$$c(p, q) = pq + \max\{f pq, c_{\min}\}, \quad f = 0.0003, \quad c_{\min} = 5 \text{ 元}. \quad (23.4)$$

买入不收印花税。 $c(p, q)$ 关于 q 单调不减。

这两种假设都要对到源码上: 假设 23.3 说“数量只能踩在一百股的台阶上”, 假设 23.4 说“买入付佣金, 成交额的万分之三, 不足 5 元按 5 元收”——那个 $\max$ 就是“最低 5 元”的写法。

定义 23.5 (可行购买集). 给定现金 $C \geq 0$ 、价格 $p > 0$, 称

$$Q = \{q = kL : k \in \mathbb{N}, k \geq 1, c(p, q) \leq C\} \quad (23.5)$$

为可行购买集。它装的是“至少买一手、而且付得起”的全部数量。

定理 23.6 (整手取整与现金约束下的可行购买量最大性). 在假设 23.3、23.4 下:

- (a) (存在性判据) $Q \neq \emptyset$ 当且仅当 $C \geq c(p, L)$, 即“预算至少够买一手”。
- (b) (简化模型的最优性) 记忽略费用时的可行集

$$Q_0 = \{q = kL : k \geq 1, pq \leq C\}, \quad q^* = \lfloor C/p \rfloor_L, \quad (23.6)$$

则 q^* 是 Q_0 的最大元: 对一切 $q \in Q_0$ 都有 $q \leq q^*$; 且

$$q^* = L \left\lfloor \frac{C}{pL} \right\rfloor, \quad (23.7)$$

这与系统实现里“先按现金算最大股数、再向下取整到手”的写法逐字一致。

(c) (现金不足自动回退整手) 令费用感知的最大手数

$$k^\dagger = \max\{k \geq 1 : c(p, kL) \leq C\}, \quad q^\dagger = k^\dagger L. \quad (23.8)$$

则集合 $\{k \geq 1 : c(p, kL) \leq C\}$ 是下闭的, $k^\dagger$ 唯一存在, 并且 $q^\dagger \leq q^*$; 更进一步, 若 $c(p, q^*) > C$, 则 $q^\dagger \leq q^* - L$, 即至少要回退一手。

(d) (低价股买不起一手的判据) 连一手都买不成的充要条件是

$$C < pL + \max\{f pL, c_{\min}\}. \quad (23.9)$$

当 $pL < c_{\min}/f = 16666.67$ 元时最低佣金项起主导, 判据退化为 $C < pL + 5$ 。

这个定理在说什么: 三句话。(a) 能不能开张, 只看“钱够不够买一手”; (b) 系统那句 $\text{int}(C/p)/100*100$ 算出来的, 就是现金能覆盖的最大整手数量, 不多一股也不少一股; (c) 如果加上手续费后最大的那个数量仍然买不起, 正确做法是往下退一手, 而退到的那一手是退过之后的最大可行量; (d) 什么时候算“穷到连一手都买不起”, 有一个闭式的判据。

证明. (a) ($\Leftarrow$) 若 $C \geq c(p, L)$, 取 $q = L$ (即 $k = 1$), 它满足 $c(p, L) \leq C$, 故 $L \in Q$, $Q \neq \emptyset$ 。

($\Rightarrow$) 设 $Q \neq \emptyset$, 取任意 $q = kL \in Q$, 则 $k \geq 1$, 于是 $q = kL \geq L$ 。由假设 23.4 中 c 关于 q 单调不减, 得 $c(p, q) \geq c(p, L)$ 。又 $q \in Q$ 给出 $c(p, q) \leq C$ 。串联:

$$C \geq c(p, q) \geq c(p, L). \quad (23.10)$$

两边合起来, $Q \neq \emptyset \iff C \geq c(p, L)$ 。

(b) 记 $k^* = \lfloor C/(pL) \rfloor$, 则 $q^* = Lk^*$ 。

可行性: 由 $\lfloor \cdot \rfloor \leq \cdot$ 得 $k^* = \lfloor C/(pL) \rfloor \leq C/(pL)$, 两边乘 $pL > 0$:

$$p q^* = p L k^* \leq p L \cdot \frac{C}{p L} = C, \quad (23.11)$$

故 $q^* \in Q_0$ (只要 $k^* \geq 1$, 即 $q^* > 0$)。

最大性: 任取 $q = kL \in Q_0$, 有 $pq \leq C$, 即 $pkL \leq C$ 。两边除以 $pL > 0$:

$$k \leq \frac{C}{pL}. \quad (23.12)$$

左端 k 是整数, 一个整数不超过实数 $C/(pL)$, 必不超过其下取整: $k \leq \lfloor C/(pL) \rfloor = k^*$ 。再乘 L :

$$q = kL \leq k^* L = q^*. \quad (23.13)$$

所以 q^* 是 Q_0 的最大元, 任何可行数量都不超过它。

与代码一致: 由引理 23.2 的形式, $\lfloor C/p \rfloor_L = L \lfloor \lfloor C/p \rfloor / L \rfloor$ 。对正整数 L 有恒等式 $\lfloor \lfloor x \rfloor / L \rfloor =$

$\lfloor x/L \rfloor$ (取 $x = C/p$), 故

$$\lfloor C/p \rfloor_L = L \left\lfloor \frac{C}{pL} \right\rfloor = q^*, \quad (23.14)$$

即 eq. (23.7)。系统里的 $\text{int}(C/p)/100*100$ 正是“对 C/p 先取整、再向下取整到一百的整数倍”；对非负数而言 $\text{int}()$ 就是下取整，所以它与 eq. (23.7) 是同一个数。

(c) 先证下闭性。设 k 使 $c(p, kL) \leq C$ 。对任意整数 j 满足 $1 \leq j \leq k$ ，有 $jL \leq kL$ ，由 c 关于 q 单调不减得 $c(p, jL) \leq c(p, kL) \leq C$ ，故 j 也在集合里。这说明集合形如 $\{1, 2, \dots, k^\dagger\}$ ， $k^\dagger$ 存在且唯一。

再证上界 $q^\dagger \leq q^*$ 。对任意 $q > 0$ ，

$$c(p, q) = pq + \max\{fpq, c_{\min}\} \geq pq, \quad (23.15)$$

因为额外项 $\max\{fpq, c_{\min}\} \geq 0$ 。于是 $c(p, k^\dagger L) \leq C$ 蕴含 $pk^\dagger L \leq C$ ，即 $k^\dagger \leq C/(pL)$ 。左端为整数，故 $k^\dagger \leq \lfloor C/(pL) \rfloor = k^*$ ，从而 $q^\dagger = k^\dagger L \leq k^* L = q^*$ 。

最后证明“至少回退一手”。若 $c(p, q^*) > C$ ，则 k^* 不属于集合 $\{k : c(p, kL) \leq C\}$ 。而该集合是 $\{1, \dots, k^\dagger\}$ ，不含 k^* 说明 $k^\dagger < k^*$ ，即 $k^\dagger \leq k^* - 1$ 。乘 L ：

$$q^\dagger = k^\dagger L \leq (k^* - 1)L = q^* - L. \quad (23.16)$$

(d) 由 (a)， $Q = \emptyset$ 当且仅当 $C < c(p, L)$ 。把 $q = L$ 代入 eq. (23.4)：

$$c(p, L) = pL + \max\{fpL, c_{\min}\}, \quad (23.17)$$

即 eq. (23.9)。再定哪一项在 $\max$ 里占优： $fpL \geq c_{\min}$ 当且仅当

$$pL \geq \frac{c_{\min}}{f} = \frac{5}{0.0003} = 16666.67 \text{ 元}. \quad (23.18)$$

所以当 $pL < 16666.67$ 时 $\max = 5$ ，判据化为 $C < pL + 5$ 。■

把定理 23.6 的 (b)、(c) 连起来读，就是系统实际执行的三个动作：先把请求量向下取整到整手，再算出现金能覆盖的最大整手，最后取两者中较小的那个。这正是“整手归一化 + 现金不足自动回退整手”的全部机制。

23.4.3 回退是幂等的

推论 23.7 (归一化映射的幂等性与一致性). 设 $q^* = \lfloor C/p \rfloor_L$ 如 eq. (23.7)。定义映射

$$\Phi(q) = \min\{\lfloor q \rfloor_L, q^*\}. \quad (23.19)$$

则：(i) $\Phi(q)$ 恒为 L 的非负整数倍；(ii) Φ 幂等，即 $\Phi(\Phi(q)) = \Phi(q)$ ；(iii) $\Phi(q) \leq q^*$ ，且 $p\Phi(q) \leq C$ (在 $q^* > 0$ 时)。

这个推论在说什么：同一个数量归一化一次和两次，结果一模一样。这在工程里很值钱——归一化会在“策略算一次、工具校验一次、执行层再校验一次”的链路上被反复调用，若不幂等，每过一道手数量就掉一个台阶，最后成交的量会莫名变小。

证明. (i) $\lfloor q \rfloor_L = L \lfloor q/L \rfloor$ 是 L 的整数倍， q^* 也是；两个 L 的整数倍取小仍是 L 的整数倍（非负）。

(ii) 令 $m = \Phi(q)$ 。由 (i)， m 已是 L 的整数倍，故 $\lfloor m \rfloor_L = m$ 。于是

$$\Phi(m) = \min\{\lfloor m \rfloor_L, q^*\} = \min\{m, q^*\} = m, \quad (23.20)$$

最后一步因为 $m = \min\{\lfloor q \rfloor_L, q^*\} \leq q^*$ 。

(iii) $\Phi(q) \leq q^*$ 由 $\min$ 的定义直接得到。又由 (b) 的可行性， $p q^* \leq C$ ，故 $p \Phi(q) \leq p q^* \leq C$ 。

■

■

23.4.4 费用项与“低价股”过滤

定理 23.6(d) 的判据有一个很实际的后果：股票越贵，越可能“一手都买不起”。

假设单标的预算是 B 元，能买得起一手的条件约等于 $B \geq pL$ ，即 $p \leq B/100$ 。资金五万、每只分到一万元时， $p \leq 100$ 元才买得起一手；价格超过一百元的票，一万元预算连一手都凑不齐。

系统的处理不是“硬扛”，而是在选股阶段就把这类标的挡在外面：当单标的预算买不到一手时，把筛选取值收窄到低价股与 ETF。选股工具里有一个价格上限参数，说明写得非常直白：

```
1 "max_price": map[string]interface{}{
2     "type": "number",
3     "description": "最大股价上限（元），用于小资金低价股筛选
4     （如5万以下需 股价×100 <= 单标的资金，可传 50），>0 才生效",
5 },
```

代码 23.3 低价股筛选参数（节选）

这段话在说什么：小资金别买高价股，因为“股价乘一百”必须落在单标的预算内；传一个价格上限，工具就只在这个价位以下选。

23.5 涨跌停：想买买不进，想卖卖不掉

制度里还有一条更硬的：涨跌停。它和第 chapter 二十二 章的盘口五态是同一件事的两面——盘口五态描述此刻能不能成交，涨跌停描述为什么不能。先把判据写清楚。

假设 23.8 (涨跌停价与盘口五态)。涨停价与跌停价由行情快照给出的涨跌幅限制 limitPct 决定：

$$p^{\text{up}} = \text{round}(\text{前收} \times (1 + \text{limitPct}), 2), \quad p^{\text{down}} = \text{round}(\text{前收} \times (1 - \text{limitPct}), 2). \quad (23.21)$$

盘口五态由封装好的分类函数给出：涨停封死、涨停打开、跌停封死、跌停打开、正常。封单量阈值

$$\theta = \max\{1000 \text{ 手}, 0.1 \times V\}, \quad V = \text{当日成交量}, \quad (23.22)$$

其中“涨停封死”的判定是：现价触及 p^{up} ，且买一价已达 p^{up} ，且买一封单量不低于 θ 。“跌停封死”完全对偶。

定义 23.9 (可成交集合与成交换手). 对一个新提交的委托，记 $\mathcal{T} \subseteq \{\text{五态}\} \times \{\text{买, 卖}\}$ 为可成交集合： $(s, d) \in \mathcal{T}$ 表示在状态 s 下、方向为 d 的委托有机会成交。成交遵循价格优先、时间优先：同一价位上，新委托排在已有委托之后，只有队列前部被消耗殆尽，它才可能成交。

定理 23.10 (涨跌停封死下的成交概率为零). 在假设 23.8 下：(i) 若目标股处于“涨停封死”，则买单在 $\mathcal{T}$ 中不可成交，成交概率为 0；(ii) 若目标股处于“跌停封死”，则卖单不可成交，成交概率为 0；(iii) 若处于“跌停打开”，则卖单可成交（按跌停价离场）。

这个定理在说什么：“信号正确”和“能成交”是两件事。止损信号再正确，碰上跌停封死，卖单当天出不去；买入信号再正确，碰上涨停封死，买单排在队尾，一手都吃不到。

证明. (i) 设状态为涨停封死。由假设 23.8，买一价等于 p^{up} ，且该价位的已封买单量 $B \geq \theta > 0$ 。

先看远端： p^{up} 是制度允许的最高委托价（价格上界），任何理性买单的报价都不超过 p^{up} ；卖方亦不能报出高于 p^{up} 的成交价。因此新买单能成交的价格只能落在 $[p^{\text{ask}}, p^{\text{up}}]$ 之内，其中 p^{ask} 是当时最优卖价。

再排队。由定义 23.9 的时间优先，新买单在价位 p^{up} 上排在已存在的 B 手之后；它要成交，必须先有 B 手被消耗。而“封死”这一判定本身就表示：在这个时点窗口内，该价位的封单 B 维持不低于阈值 θ ，也就是队列前部没有被消耗。设窗口内愿意在 p^{up} 卖出的总量为 X ，则留给新买单的可成交量至多为 $X - B$ ；“封死”蕴含 $X \leq B$ ，故 $X - B \leq 0$ 。于是

$$\mathbb{P}(\text{新买单成交} \mid \text{涨停封死}) = 0. \quad (23.23)$$

(ii) 完全对偶：把“涨停/买”整体换成“跌停/卖”， p^{up} 换成 p^{down} ，价格下界取代上界，“卖一价等于 p^{down} 且卖一封单量不低于 θ ”取代买一，逐字重复 (i) 的推理，得卖单成交概率为 0。

(iii) 跌停打开的状态判据是封单量不足阈值（封单薄或伴随巨量换手）。此时 (ii) 中的不等式 $X \leq B$ 不再成立，存在 $X - B > 0$ ，即新卖单存在可成交量，故可成交；系统据此对跌停打开放行，允许按跌停价立即离场。■

推论 23.11 (信号与成交是两个条件的“与”). 一笔委托真正成交，需要两个条件同时成立：

$$\text{成交} \iff (\text{信号给出方向}) \wedge ((s, d) \in \mathcal{T}). \quad (23.24)$$

因此“信号正确”不蕴含“能成交”。特别地，当 $s = \text{涨停封死}$ 而 $d = \text{买}$ ，或者 $s = \text{跌停封死}$ 而 $d = \text{卖}$ 时，

$$\mathbb{P}(\text{成交} \mid \text{信号正确}) = 0. \quad (23.25)$$

这个推论在说什么：成交是一个“与”关系。信号只是一半，另一半在盘口手里；你把信号做得再准，也只是把左边那一半做到极致，右边那一半为假时，结果仍然是零。

系统对此的处理是在执行层直接拦截，而不是“试着报一单看能不能成”：买入侧拦跌停板与涨停封死，卖出侧拦跌停封死，跌停打开放行。拦截返回一句结构化的拒绝原因，绝不伪造成交。底气正是定理 23.10：封死状态下成交概率为零，与其挂一笔永远不成交的委托，不如当场说清为什么不做。

23.6 它在系统里长什么样

上面每一条在系统里都有具体落点，我把它们排成一张表。

表 23.1 A 股三条硬规矩与它们在系统里的落点

硬规矩	取值	系统落点
T+1	当日买入当日不可卖	卖出前算“可卖量 = 持仓 - 今日买入量”，不足则拒单 (internal/portfolio/engine.go)
交易时段	09:30–11:30、13:00–15:00	IsTradingHour；周末与节假日返回休市 (internal/util/trading_hours.go)
整手归一化	买入量为 100 股整数倍	floorLots 向下取整；执行层再校验 quantity%100!=0 (internal/tools/position_manager.go、internal/portfolio/engine.go)
现金不足回退	取“请求整手”与“现金整手上限”之较小者	下单工具里 qty=(quantity/100)*100 与 affordable=int(Cash/price)/100*100 取小 (internal/tools/tools.go)
买入费用	佣金万三、最低 5 元；不收印花税	internal/portfolio/engine.go 的 Buy
卖出费用	佣金万三、最低 5 元 + 印花税 0.05%	同文件 Sell (fees = grossAmount*0.0005)
持仓数量	按资金分档动态调整	PortfolioSizing (internal/util/portfolio_sizing.go)
流通市值筛选	20 亿–200 亿	minFloatCap/maxFloatCap，收盘价 × 股本 (internal/screener/pro_factor_engine.go)
流动性硬下限	流动性因子分 ≥ 30	applyLiquidityFloor (internal/screener/screener_service.go)

“最低 5 元”是佣金下限，不是印花税。“现金整手上限”忽略手续费，故加费后仍可能不足，需再回退一手——这正是定理 23.6(c)。

数据来源：系统事实清单第 9、10、11、12 节，与所列源码逐条对照；核对时间 2026-09-24。

持仓数量那一行值得展开，它是“按资金规模动态调整”的具体形态。分档只有四档：这张表解释了开头那个策略为什么“一次买五只”：不是他的名单只有五只，而是五万元这个资金量下，系统认为分散到五只合适。资金越大、档位越高，允许的持仓数越多、单标的越轻。为什么是“越多标的越轻”？因为每只股票有一个一百股的最小重量。五万元分成十份、每份五千元时，很多股票连一手都买不起（回到定理 23.6(d)）。小资金只能聚焦少数标的，大资

表 23.2 总资产规模与最大候选只数、单标的预算

总资产	最大候选（持仓）数	单标的预算占比
3 万以下	4	约 25%
3 万-20 万（含 20 万）	5	约 20%
20 万-50 万	8	约 12%
50 万以上	10	约 10%

同一套分档同时用于投资规划与盘中自动建仓，两处口径一致。5 万资金落在第二档，推荐 5 只，单标的约 1 万元。

数据来源：`internal/util/portfolio_sizing.go` 的 `PortfolioSizing`；核对时间 2026-09-24。

金才谈得上分散。

23.7 两道硬筛选：流通市值与流动性

选股阶段有两道德式硬门槛，它们和上面的交易规矩是同一个精神——不是降权，是直接剔除。

第一道：流通市值 20 亿-200 亿。口径是“收盘价 × 流通股本”，流通股本缺失时用总股本兜底；低于 20 亿的微盘和高于 200 亿的超大盘都被剔除。这里有一个我很欣赏的设计：**无股本资料时放行**——缺数据不等于数据差，因为拿不到股本就把正常股票剔除掉，是把数据的缺口错当成标的的缺陷。

第二道：流动性硬下限。流动性因子分低于门限值（默认 30，约在均值下方半个标准差处）的标的直接剔除。系统把这条写成了“拒买门槛”而不是普通打分因子，源码注释里的理由只有一句：

```
1 // 流动性硬下限：换手/成交额不足的“僵尸股”即便高分也直接剔除（不降权、不保留）。
2 // 小基金最怕“买了跑不掉”，流动性必须是拒买门槛而非只是一个打分因子。
3 const minLiquidityScore = 30.0
```

代码 23.4 流动性硬下限（节选）

这段话在说什么：流动性差的股票，别的因子满分也不买——买入后能不能顺利卖出是另一个问题，那部分风险打分表兜不住。过滤有兜底：若过滤后一只不剩就退回原始结果，不硬过滤，也不让用户面对空池。

关于盘口五态本身还有一个诚实的数据说明：本地行情库里当日盘口摘要表没有有效记录当日 `order_book_daily` 记录数为 0，本书遵照系统“数据不可用即报错、不降级、不伪造”的原则留白¹，所以本章对五态的讨论全部建立在判定逻辑上，而不是当日的统计分布上。

¹数据缺失：该系统库中此项无有效记录。本书遵照系统“数据不可用即报错、不降级、不伪造”的原则，此处留白、不填充估计值。

23.8 常见误解与纠偏

常见误解·这些都是“交易细节”，把模型做好就够了 我一开始就是这么想的：T+1、整手、涨跌停，听起来都是“客服手册”里的东西，跟因子、跟模型、跟数学有什么关系？

后来我在回测里做了一次对照，才明白这句话错得有多离谱。同一个策略，一套回测假设“当天可卖”，另一套假设“T+1 冻结”，两条净值曲线差出一大截，而差距主要来自那些被迫持仓过夜的仓位。这不是细节，这是收益的一部分。

更普遍地说：**制度和摩擦，常常比模型更能决定最终收益**。模型决定“你想赚什么”，制度决定“你到底能不能赚到”。开头那位朋友的策略，恰恰只能在理想世界里赚钱。

常见误解·把微盘股剔除掉是偏见，微盘弹性大、涨得快 这是很有诱惑力的一条：翻一翻历史，很多翻倍股确实从中微盘里冒出来，“弹性大”不是空穴来风。

但“涨得快”只说了故事的一半，另一半是你走得掉吗。一只日均成交额只有几百万的微盘股，你放进去十万元，可能当天就占掉它相当比例的成交；等你要减仓，能接盘的量根本不够，你想卖的那一刻，价格已经先跌为敬。

所以“剔除微盘”不是对市值的偏见，而是对**退出能力**的要求：**跑不掉的仓位等于负债**。两道硬筛选——20 亿以下不选、流动性因子分低于 30 不选——都是在成交流动性上设闸，而不是在收益预期上做品味判断。

23.9 真实出处

本章的真实出处

- T+1 校验与拒单文案（“可卖数量为 N 股（昨日持仓）”）：`internal/portfolio/engine.go` 的 `Sell`，含“按交易日判断是否过了 T+1 结算日”的解锁逻辑
- 整手校验（`quantity%100!=0` 拒绝）、买入费用（佣金万三、最低 5 元、买入无印花税）、卖出费用（佣金 + 印花税 0.05%）：同文件 `Buy` 与 `Sell`
- 交易时段与休市判定：`internal/util/trading_hours.go` 的 `GetMarketPhase`、`IsTradingHour`（上午 09:30-11:30、下午 13:00-15:00）
- 整手归一化函数 `floorLots`（`int(shares/100)*100`）：`internal/tools/position_manager.go`
- 现金不足自动回退整手：`internal/tools/tools.go` 的 `PlaceTradeTool.Execute`，`qty=(quantity/100)*100` 与 `affordable=int(Cash/price)/100*100` 取小，不足一手则拒绝
- 持仓数量动态分档：`internal/util/portfolio_sizing.go` 的 `PortfolioSizing`
- 流通市值硬筛选（20 亿-200 亿、总股本兜底、无资料放行）：`internal/screener/pro_factor_engine.go` 的 `minFloatCap/maxFloatCap` 与 `floatShares`
- 流动性硬下限：`internal/screener/screener_service.go` 的 `applyLiquidityFloor`（`minLiquidityScore=30`）
- 组合优化器（均值-方差 Markowitz、风险平价、风险预算；MVO 风险厌恶系数 λ 默认 4.0）：`internal/portfolio/optimizer.go`；工具 `optimize_portfolio`
- 盘口五态与封单量阈值：`internal/orderbook/orderbook.go` 的 `Classify`；买卖拦截：`internal/portfolio/engine.go` 的 `buyBlockReason` / `sellBlockReason`

动手试一试（半小时）

1. 手算一次最大购买量。设可用现金 $C = 100000$ 元，股价 $p = 37.6$ 元，一手 $L = 100$ 股。先算 $C/(pL) = 100000/3760 = 26.60$ ，向下取整到手得 $k^* = 26$ ，即 $q^* = 2600$ 股该结果按系统 `floorLots` 的同一公式在给定输入上手算所得，仅演示取整算术^a。再验证 27 手： $2700 \times 37.6 = 101520 > 100000$ ，确实超预算。最后算费用： $2600 \times 37.6 \times 0.0003 = 29.33$ 元 ≥ 5 元，故含佣金净支出 97789.33 元，仍在预算内， $q^\dagger = q^*$ 。
2. 核对取整方向。在 `internal/tools/position_manager.go` 里搜 `floorLots`，确认它是向下取整：宁可少买，绝不超买。再去 `internal/tools/tools.go` 的 `PlaceTradeTool.Execute` 里搜 `affordable`，确认“现金上限”也是向下取整。
3. 在选股页看流通市值筛选。跑一次选股，在结果里找每只票的流通市值列。你会看到它们都落在 20 亿–200 亿之间——这不是巧合，是 `minFloatCap/maxFloatCap` 两道门槛在起作用。
4. 核对交易时段常量。打开 `internal/util/trading_hours.go`，找到 `PhaseMorningSession` 与 `PhaseAfternoonSession`，确认时间是 09:30–11:30 与 13:00–15:00。再把电脑时间改到晚上八点，试着手动下单一次，看系统回什么。

^a复算数据：该数字为本书按系统实现中的同一公式、在系统的真实数据上重新计算所得；公式出处已在正文标注。

原理讲完了。这一部分从头到尾在追同一件事：误差是从哪里来的？它可能来自一个用错口径的因子、一个漏算的成本项、一次抽歪的抽样、一个把当日买入当成可卖的假设。这一章补上了最后、也最容易被忽略的一类来源——制度和摩擦：它们不在数据里，也不在因子里，却实实在在扣在你的收益上。

到这里，第二部分合上了：十七种策略、六维判势、三层风控、盘口五态、T+1 与整手——这套机器我们已从里到外拆过一遍。可拆开看懂不等于能用：一台机器不会因为你读完说明书就自己转起来。

所以接下来要换一种语气。第三部分不再是“为什么”，而是“怎么做”：把它装到你的电脑上，接上模型，连上数据，让它明天早上真的开始上班。第 chapter 二十四章就是第一天。

第三部分

实际操作：让它在今天就开始工作

第二十四章

第一天：安装、接模型、连数据

24.1 下载、解压、双击——然后呢？

上周有个朋友把安装包拖到桌面，解压，双击里面那个 `.exe`。窗口弹出来了，屏幕上开始一行一行往外跳字。他没看，端着水杯等了三分钟，然后问了我一个问题：

“好了吗？”

我说：你现在盯着的第一屏，正在替你回答另一个问题——这台机器，今天到底能不能用。

他有点意外：“不就是装个软件吗？装完不就能用了？”

我把这句话记了下来，因为它正好是这一章要拆掉的东西。装软件最难的一步，从来不是下载和双击。是知道自己在等什么，以及知道什么东西坏了。

这一部分的语气会跟前两部分不一样：前面我们在实验室里拆机器、算误差，现在我坐到你的电脑旁边，手把手。本章只干三件事：装好软件、接上 AI 的大脑、打开今天的数据通路。

24.2 装一台新机器：先通电，再通水，最后开机自检

直觉·先通电，再通水，最后按自检键 我家里装过一台洗衣机。装完的那天，师傅没有立刻按“开始洗衣”。他做了三件事，顺序很固定：

第一，通电。插头插上，面板亮不亮。这一步只关心“它认不认自己的电源”。

第二，通水。进水管接上、水阀打开，看会不会漏水。这一步关心“它的输入从哪来、通不通”。

第三，按自检键。面板上一排灯依次亮起、依次熄灭，全部亮完说明每一条回路都活着；只要有一盏灯是红的，他就把机器停在那里，不洗。

这三步和 QuantBot 的开机一模一样，而且一一对应：

通电，是**配置**——你告诉它有多少钱、用哪个 AI、钥匙是什么。落在 `config/config.json` 这个文件里。

通水，是**数据**——行情库 `data/stock.duckdb` 能不能被打开、里面的 K 线新不新。

自检，是**启动加载页**——六个组件挨个点亮，其中几个是“红灯就停机”的关键件。

顺序不能倒。先通水后通电、或者跳过自检直接开洗，机器不会更聪明，只会更危险。而“自检不通过就停机”这件事，本身就是一个需要被设计出来的决定——下一节我们就来说它。

24.3 为什么“带病开机”比“开不了机”更糟

按本书的规矩，讲“是什么”之前先讲“不这么做会怎样”。所以这里先假设一个反面设计：启动时不检查、失败了就静默降级悄悄继续。

这个设计在界面上很讨喜：加载页一路绿灯，你进选股页点“开始选股”，返回 0 只。你大概率会想“今天没什么好票”，而不是“行情库根本没挂上”。

更坏的一种降级是“用假数据顶上”：行情库读不到就拿上一次的缓存当今天用；API Key

没配，就用一句模板话术装作 AI 已经回答过。演示时很连贯，带来的却是一个看起来完全正常的错误决策。

到这里必须接上第 chapter 二十三 章的那条规则：A 股是 T+1，当日买入当日不能卖。也就是说，一个基于错误数据的买入决定，当天没有撤回键。系统明明有能力告诉你“我坏了”，却选择沉默，你就得为这份沉默付钱。

所以系统的取舍写在明处：关键组件失败必须弹窗报错，严禁降级、严禁使用虚假数据；同时保留一个人为出口——失败后可以由你自己点一次“进入系统”继续（此时你是知情者，不是被蒙在鼓里的人）¹。

那么“停机”和“带病继续”哪个更划算？直觉上大家都知道停机更保险，但“更保险”三个字不是证明。我们把它变成一个可以算的问题。

24.4 把“开机”写成一个状态机

先说一个“差不多对”的模型，再补严谨版。

近似模型：开机就是一道与门（AND）。六个组件全部正常，才允许进入工作态；有一个不正常，就停。这个模型抓住了本质，但它把“停机”当成零成本，把“继续”当成零损失——这两笔账都没算。

严谨版本：把开机看成一个判定函数，把两条路的期望损失算出来相减。先给定义。

定义 24.1 (启动状态机). 设参与启动判定的组件集合为 $C = \{c_1, \dots, c_n\}$ ，每个组件的状态取

$$s_i \in \{\text{ok}, \text{fail}\}, \quad i = 1, \dots, n, \quad (24.1)$$

整个系统的状态是一个向量 $s = (s_1, \dots, s_n) \in \{\text{ok}, \text{fail}\}^n$ 。记失效集合为

$$F(s) = \{i : s_i = \text{fail}\}, \quad (24.2)$$

并设关键性标记 $\text{crit}(i) \in \{0, 1\}$ 把组件分成关键子集

$$C^* = \{i : \text{crit}(i) = 1\} \subseteq C. \quad (24.3)$$

启动判定是一个映射

$$\Sigma : \{\text{ok}, \text{fail}\}^n \rightarrow \{\text{RUN}, \text{ABORT}\}. \quad (24.4)$$

本书比较两个具体判定：

$$\Sigma_{\text{fc}}(s) = \text{ABORT} \iff F(s) \cap C^* \neq \emptyset, \quad (24.5)$$

$$\Sigma_{\text{deg}}(s) = \text{RUN} \quad \text{对一切 } s. \quad (24.6)$$

¹数据来源：前端 `ui/src/components/InitialLoading.tsx`: `criticalFailed` 为真时渲染红色错误块，并显示“进入系统”按钮（`showContinue`）；错误清单取自后端 `IsReady()` 的 `errors` 字段。核对时间 2026-09-24。

称 Σ_{fc} 为失败即拒（fail-closed）判定， Σ_{deg} 为降级继续判定。

这个定义在说什么：开机不是“跑起来就算成功”，而是一个函数——输入是六个组件的死活清单，输出只有两个字，跑或停。式 (24.5) 说的是：只要失效清单里沾上一个关键组件，输出就是“停”。

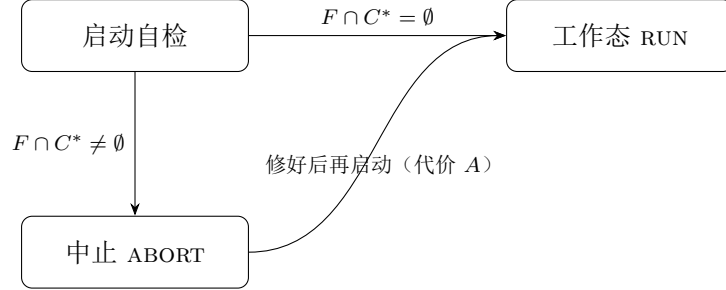


图 24.1 fail-closed 启动状态机：只有关键组件全绿才进入工作状态；否则停在 ABORT 分支，修好后重新走一遍自检。

接下来要算账，就得先规定“损失”是什么。

假设 24.2 (损失模型). (i) 走到 ABORT 分支的损失记作 A , $0 < A < \infty$: 这是“今天不开工”的代价（推迟一天、错过一次机会），且中止不产生任何错误决策。

(ii) 关键组件全部正常、进入工作状态运行一天，期望损失记作 $L_0 \geq 0$ （正常持仓波动与交易成本）。

(iii) 关键组件 c_i 失效却仍然运行，它会以概率 $p_i \in [0, 1]$ 产出一个“看起来正常、实际错误”的决策，单次期望损失为 $\ell_i > 0$ 。给定失效集合 F ，这一天的期望错误损失为

$$D(F) = \sum_{i \in F \cap C^*} p_i \ell_i, \quad (24.7)$$

即各失效组件的错误损失可加（近似；见后文注 24.7）。

假设 24.3 (失效先验). 记“至少有一个关键组件失效”这个事件为 $\mathcal{F} = \{s : F(s) \cap C^* \neq \emptyset\}$ ，其先验概率

$$\rho = \mathbb{P}(\mathcal{F}) \in (0, 1), \quad \bar{D} = \mathbb{E}[D(F) \mid \mathcal{F}] > 0. \quad (24.8)$$

由假设 24.2(iii) 的 $p_i \ell_i > 0$ 立即得 $\bar{D} > 0$ ：只要发生关键失效，期望错误损失就是严格正的。

假设 24.4 (代价不对称).

$$A < \bar{D}. \quad (24.9)$$

说人话：“今天不开工”的代价，小于“带着坏组件开工一天”的期望错误损失。这不是数学事实，是工程取舍：中止可逆（明天再来），而错误决策在 T+1 与涨跌停制度下往往不可撤销（chapter 二十三）； A 由系统自己控制（一个弹窗、一次手动确认），而 ℓ_i 没有先验上界（错误数据可以驱动一次不该发生的资金操作）。

命题 24.5 (fail-closed 的期望损失严格更优). 在假设 24.2–24.4 下：

(i) 两条路径的期望损失分别为

$$\mathbb{E}_{\text{fc}} = \rho A + (1 - \rho)L_0, \quad \mathbb{E}_{\text{deg}} = L_0 + \rho\bar{D}; \quad (24.10)$$

(ii) 两者之差只由“中止代价”与“错误损失”两项决定：

$$\mathbb{E}_{\text{fc}} - \mathbb{E}_{\text{deg}} = \rho(A - \bar{D}); \quad (24.11)$$

(iii) 特别地， $A < \bar{D}$ 时

$$\mathbb{E}_{\text{deg}} - \mathbb{E}_{\text{fc}} = \rho(\bar{D} - A) > 0, \quad (24.12)$$

即 *fail-closed* 的期望损失严格小于降级继续，差额随失效概率 ρ 线性放大。

这个命题在说什么：它把“该不该停机”这句口号换成了一个不等式：停机的代价 A 如果比“带着坏组件瞎跑”的期望损失 $\bar{D}$ 小，就该停机，而且差额是 $\rho(\bar{D} - A)$ 。注意它同时说明了反面：如果 $A > \bar{D}$ （停机代价大得离谱、错误损失小得可以忽略），那么这个策略就不再占优——所以 *fail-closed* 是有条件的正确，不是永远的正确。

证明. (i) 先算 $\mathbb{E}_{\text{fc}}$ 。按式 (24.5)， Σ_{fc} 的输出由事件 $\mathcal{F}$ 决定，分两种情况：

当 $\mathcal{F}$ 发生（概率 ρ ）时，判定为 ABORT，按假设 24.2(i) 损失恰为 A ，且不产生错误决策；当 $\mathcal{F}$ 不发生（概率 $1 - \rho$ ）时，判定为 RUN，按假设 24.2(ii) 期望损失为 L_0 。两段相加即

$$\mathbb{E}_{\text{fc}} = \rho \cdot A + (1 - \rho) \cdot L_0. \quad (24.13)$$

再算 $\mathbb{E}_{\text{deg}}$ 。按式 (24.6)， Σ_{deg} 永远输出 RUN，所以两种情况下都进入工作态。当 $\mathcal{F}$ 不发生（概率 $1 - \rho$ ）时，损失就是 L_0 ；当 $\mathcal{F}$ 发生（概率 ρ ）时，损失为 $L_0 + D(F)$ ，其中 $D(F)$ 由式 (24.7) 给出。用全期望公式：

$$\mathbb{E}_{\text{deg}} = \rho \mathbb{E}[L_0 + D(F) \mid \mathcal{F}] + (1 - \rho) L_0. \quad (24.14)$$

把 L_0 从条件期望里提出来（它是常数，不依赖 F ），并用假设 24.3 定义的 $\bar{D} = \mathbb{E}[D(F) \mid \mathcal{F}]$ ：

$$\mathbb{E}_{\text{deg}} = \rho(L_0 + \bar{D}) + (1 - \rho)L_0 = L_0 + \rho\bar{D}. \quad (24.15)$$

这就是式 (24.10)。

(ii) 直接相减，把 L_0 项消掉：

$$\mathbb{E}_{\text{fc}} - \mathbb{E}_{\text{deg}} = [\rho A + (1 - \rho)L_0] - [L_0 + \rho\bar{D}] = \rho A + \underbrace{(1 - \rho)L_0 - L_0}_{=-\rho L_0} - \rho\bar{D} = \rho(A - \bar{D}). \quad (24.16)$$

这一步的关键是： L_0 在两条路径里都以 $(1 - \rho)$ 的同样权重出现（全绿时两路径都必须运行），所以它整项相消——两条路径的差别只发生在失效情形里。

(iii) 由假设 24.4 得 $A - \bar{D} < 0$ ，两边乘以 $-\rho$ 并把 $\rho > 0$ (假设 24.3) 代入：

$$\mathbb{E}_{\text{deg}} - \mathbb{E}_{\text{fc}} = \rho(\bar{D} - A) = \underbrace{\rho}_{>0} \cdot \underbrace{(\bar{D} - A)}_{>0} > 0. \quad (24.17)$$

两个正数之积严格为正，故不等号严格；又因该式对 ρ 是一次齐次的，差额随失效频率线性放大。 ■

推论 24.6 (关键集合扩张只会更保守). 设两组关键标记给出 $C_1^* \subseteq C_2^*$ (把更多组件“升级为关键”), 其余记号不变。则

(i) 判定更保守: $\{\Sigma_{\text{fc}}^{C_1^*} = \text{ABORT}\} \subseteq \{\Sigma_{\text{fc}}^{C_2^*} = \text{ABORT}\}$;

(ii) 差额不缩小: $\rho(\bar{D}_2 - A) \geq \rho(\bar{D}_1 - A)$ 。

这句推论在说什么：把某个组件从“坏了也能凑合”改成“坏了必须停”，只会让系统更早停下，绝不会让它在更坏的状态下多跑一会儿——这是“只降不升”的一个实例。

证明. (i) 取任意状态 s , 设 $\Sigma_{\text{fc}}^{C_1^*}(s) = \text{ABORT}$ 。按定义 24.1 的(24.5), 这意味着存在 $i \in F(s) \cap C_1^*$ 。由 $C_1^* \subseteq C_2^*$ 得 $i \in C_2^*$, 于是 $i \in F(s) \cap C_2^*$, 从而 $\Sigma_{\text{fc}}^{C_2^*}(s) = \text{ABORT}$ 。由于 s 任意, 第一个中止集合被第二个包含。

(ii) 对任意失效集合 F , 由式 (24.7) 与 $C_1^* \subseteq C_2^*$ 得 $F \cap C_1^* \subseteq F \cap C_2^*$; 被求和集扩大而每一项 $p_i \ell_i > 0$ 非负, 故级数不减:

$$D_2(F) = \sum_{i \in F \cap C_2^*} p_i \ell_i \geq \sum_{i \in F \cap C_1^*} p_i \ell_i = D_1(F). \quad (24.18)$$

对条件分布取期望保序 (期望单调性: 逐点 $X \geq Y$ 蕴含 $\mathbb{E}[X] \geq \mathbb{E}[Y]$), 得 $\bar{D}_2 \geq \bar{D}_1$ 。于是 $(\bar{D}_2 - A) - (\bar{D}_1 - A) = \bar{D}_2 - \bar{D}_1 \geq 0$, 乘以 $\rho > 0$ 结论成立。 ■

注 24.7 (这个近似在哪里失效). 我不想让上面这套推导显得比它实际更强, 所以把三个松动的地方写清楚。

第一, 可加性 (假设 24.2(iii)). 现实中失效是耦合的: 行情库挂了, 判势、选股、回测会一起错, 损失不是简单相加而是相互放大。好消息是耦合通常是**正相关**, 正相关只会使 $\bar{D}$ 更大, 命题结论不变; 只有当失效之间**互相抵消** (例如一个错误方向的信号恰好被另一个错误盖住) 时, D 才可能偏小, 结论才可能翻转。

第二, ρ 与 $\bar{D}$ 不可直接观测。 式 (24.12) 给的是判据, 不是一个数。本书没有这两个量的实测值, 也不打算编一个: 它们取决于你机器上的具体环境 (第 chapter 二十五 章会讲, 库没同步好是最常见的失效来源)。

第三, “进入系统” 没有推翻命题。 系统的“进入系统”按钮确实允许你在失败后走 RUN 分支。但那是在你看到了失败清单之后做的决定, 信息集变了; 命题比较的是默认策略, 而不是否定人的知情选择。

24.5 它在系统里长什么样

理论部分到这里就够用了，下面全部是手把手。

24.5.1 第一步：一个不用安装的文件夹

QuantBot 是免安装 ZIP：解压即用，没有安装程序。解压之后，所有东西都在同一个目录里，最要紧的是这几样：

- 主程序 `.exe`：双击它；
- `config/`：配置目录，里面是 `config/config.json`；
- `data/`：行情库目录，里面是 `data/stock.duckdb`；
- `database/`：SQLite 库目录，里面是 `database/quantpilot.db`；
- `log/`：日志目录，每次启动生成一个 `log/quantbot_< 时间戳 >.log`；
- `log/crash_dump.txt`：崩溃转储。

“数据和程序同目录”这件事值得单独说一句：程序里所有路径都是从可执行文件所在目录推出来的²，不是从系统盘符里找。所以换一台电脑 = 拷一个文件夹：把整个目录复制过去，配置、数据、历史记录一起搬走。

顺带记一个“出事了去哪找原因”：GUI 程序没有控制台，Go 运行时的致命崩溃默认写到标准错误、那里没人接，系统把它重定向到 `log/crash_dump.txt`³。

24.5.2 第二步：四步向导，一次把三件事问完

首次运行时，程序不会直接进主界面，而是先走一个四步向导⁴：

1. **市场与初始资金**。市场固定为 A 股（那个按钮是禁用状态，点不动，不是你没找到）；初始资金是一个数字输入框，默认 100000⁵ 元⁶。
2. **AI 提供商**。`deepseek / openai / claude / other` 四个按钮，默认选中 `deepseek`；下面一行是 API Key 输入框，占位符 `sk-...`。
3. **风险操作系统（RiskOS）**。这一步是一张说明卡片，交代风控层的存在。
4. **完成**。显示刚才填的初始资金，并可切换界面语言（中文 / EN）。

这里有一个很容易按错的地方：向导只在最后一步保存。你翻到第 2 步填了 Key，然后关掉程序——什么都没存下来。真正落库发生在第 4 步点下那个按钮的瞬间，它做两件事：

²数据来源：系统源码 `main.go` 的 `GetExecDir()` (`os.Executable()` 取目录) 与 `setupLogging()` (日志写 `exeDir/log/`)；崩溃转储用 `debug.SetCrashOutput` 写到 `exeDir/log/crash_dump.txt`。核对时间 2026-09-24。

³数据来源：系统源码 `main.go` 的 `setupLogging()`；`debug.SetCrashOutput(cf, debug.CrashOptions{})`。核对时间 2026-09-24。

⁴数据来源：前端 `ui/src/pages/WelcomeWizard.tsx`：step 从 1 到 4，步进条为四个圆点；第 4 步的按钮触发 `handleFinish`。核对时间 2026-09-24。

⁵真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

⁶数据来源：默认值来自 `WelcomeWizard` 里 `useState('100000')`，与后端 `internal/config/config.go` 的 `defaultConfig()` 中 `InitialCapital: 100000.0` 一致。核对时间 2026-09-24。

`SetInitialCapital` 写入资金；如果你的 `Key` 非空，再调 `SetAISettings` 写入提供商、模型、接口地址和 `Key`⁷。

默认接的是哪条 AI 通路？后端的出厂配置写得很明确：提供商 `deepseek`、模型 `deepseek-v4-flash`、接口地址 `https://api.deepseek.com`⁸。

注 24.8 (一个我照实写下来的不一致)。上面的模型名来自后端出厂配置。而向导自己内置的那张小表里，`deepseek` 这一行写的模型是 `deepseek-chat`⁹。也就是说：如果你在向导里填了 `Key`，写进配置的模型名会取自这张表；如果你跳过 `Key`、后来在设置页填，用的就是后端那套默认。

我不改它，也不替它圆场，因为这件事本身是本章要教的：同一个“默认值”写在两个地方，就等于有两个默认值。想知道你这台机器上到底在用哪个模型，不要去猜，去设置页的 AI 页签看那一行字。

24.5.3 第三步：开机自检——六个组件与十步初始化

点下完成之后，进程重启进入主流程。此时前端显示的是启动加载页，它每秒向后端问一次“好了吗”，最多等 30 秒¹⁰。

后端那边在干什么？它真的去“通水、通电”，而不是只把标志位置上。启动时执行一次真实连通性探测¹¹：

- **数据库**：对 SQLite 做一次真实 `PingContext`（3 秒超时）。
- **行情**：通过数据完整性守卫查 DuckDB 能否取到最新交易日与股票数量，并检查是否更新到“今天的前一个交易日”。
- **AI**：向配置里的真实接口发一次极简请求（"ping"），5 秒超时。`Key` 为空、地址为空、客户端未初始化，各有各的错误文案。

这三项的结果缓存在内存里供轮询读取——所以加载页上那几盏灯不是装饰，它背后是一次真实的网络往返。

加载页把“能不能开工”拆成了六项组件状态，另外还多显示一行组合数据预热。我把它抄成一张表。

```
1 // 启动超时看门狗：25 秒内没完成初始化，就强制标记“就绪”
2 go func() {
3     timer := time.NewTimer(25 * time.Second)
```

⁷数据来源：前端 `ui/src/pages/WelcomeWizard.tsx` 的 `handleFinish`：先校验资金为正数，再 `SetInitialCapital(capitalNum)`，`apiKey.trim()` 非空才调 `SetAISettings(provider, model, baseUrl, apiKey)`，失败时在卡片里显示红色错误行。核对时间 2026-09-24。

⁸数据来源：系统源码 `internal/config/config.go` 的 `defaultConfig():AIProvider`：“`deepseek`”、`AIModel`：“`deepseek-v4-flash`”、`AIBaseURL`：“`https://api.deepseek.com`”；同一函数里默认交易模式为 `simulated`、单票上限 `MaxPositionSize`：0.30。核对时间 2026-09-24。

⁹数据来源：前端 `ui/src/pages/WelcomeWizard.tsx` 的 `AI_PROVIDER_DEFAULTS`：`deepseek`：{
`model`: 'deepseek-chat', `baseUrl`: 'https://api.deepseek.com'}，`openai` 为 `gpt-4o`，`claude` 为 `claude-3-5-sonnet`。核对时间 2026-09-24。

¹⁰数据来源：前端 `ui/src/components/InitialLoading.tsx`：`maxWaitTime = 30000`（单位毫秒），`setTimeout(poll, 1000)` 顺序轮询，超时后置 `initFailed=true` 并显示“进入系统”按钮。核对时间 2026-09-24。

¹¹数据来源：系统源码 `App.go` 的 `runStartupConnectivityCheck()`。核对时间 2026-09-24。

表 24.1 启动加载界面的六个组件，以及失败意味着什么

组件	状态字段	关键	失败意味着什么
配置管理器	config_ready	是	配置读不出来，后面所有参数都无处可依
SQLite 数据库	conn_db_ok	是	持仓、成交、决策日志没有账本可记
股票时序库（行情）	conn_ds_ok	是	选股、判势、回测全部失去输入
LLM 大模型服务	conn_ai_ready	是	AI 团队无法思考，只能回退规则
投资组合引擎	portfolio_ready	否	资金与仓位不可用
智能体执行器	agent_executor_ready	否	任务派不出去，自动 workflow 停摆
（额外）组合数据预热	portfolio_warm	是	持仓行情/昨收/结算数据没备好

前六行是本书所称的“六个组件”，第七行是加载页额外显示的一项预热状态，源码里同样被标为关键件。加载页对关键件用红叉、非关键件只做灰点提示。

数据来源：前端 `ui/src/components/InitialLoading.tsx` 的 `componentStatus` 数组（`name` 显示名、`critical` 标记）与后端 `App.go` 的 `IsReady()` 返回字段逐条对照；`ready` 总判据为 `a.ready && configReady && sqliteReady && a.connDbOK && a.connDsOK && a.connAIReady`。核对时间 2026-09-24。

```
4     <-timer.C
5     if !a.ready {
6         a.ready = true           // 强制放行（降级模式）
7         log.Println("STARTUP TIMEOUT: Forced system ready after 25s (degraded mode)")
8     }
9 }()
10
11 // 整个 startup 外面再兜一层 panic 恢复
12 defer func() {
13     if r := recover(); r != nil {
14         log.Printf("[QuantBot] PANIC RECOVERED: %v", r)
15         a.ready = true           // 同样强制放行，但日志里留下完整堆栈
16     }
17 }()
```

代码 24.1 后端的启动看门狗与 panic 兜底（节选）

这段代码在说什么：后端的初始化是在协程里跑的，它有两条“兜底绳”。第一条是 25 秒的看门狗——如果初始化卡住太久，就强制把 `ready` 置真，让界面别再无限等下去（日志里明确写作降级模式）¹²；第二条是 `panic` 恢复——初始化过程中任何一处 `panic` 都不会让进程带着半个状态沉默地死掉，而是先落栈、再放行。

所以“等待”这件事有三层时间常数，我把它们摆在一起，免得你以后被某一层搞混：后端看门狗 25 秒，前端最多等 30 秒，AI 连通性探测本身 5 秒超时。至于整台机器的实际启动耗时，各人机器不同，本书不给估计值（启动耗时随机器与数据规模变化，系统库中无固定可引用的记录）¹³。

那“十步初始化”呢？日志里把它编号得很清楚，一共 10 步¹⁴：配置 → SQLite → DuckDB

¹²数据来源：系统源码 `App.go` 的 `startup(ctx)` 开头：`time.NewTimer(25 * time.Second)` 的看门狗协程，以及紧随其后的 `defer func() { if r := recover(); ... }()`。核对时间 2026-09-24。

¹³数据缺失：该系统库中此项无有效记录。本书遵照系统“数据不可用即报错、不降级、不伪造”的原则，此处留白、不填充估计值。

¹⁴数据来源：系统源码 `App.go` 的 `startup(ctx)`：日志依次打印 [Step 1/10] 配置、[Step 2/10] SQLite、[Step

→ TDX 服务 → Harness → LLM 与 MCP → 任务执行器 → 智能体与引擎 → 服务与审计 → 调度与编排。

这里有两个顺手可以学的工程习惯。第一，TDX 初始化被单独包了一个局部 `recover`，因为第三方库有已知的日历越界 panic，包住它就不会因为一个可选组件把后面的组合引擎一起拖死。第二，每步都打印耗时——出问题时你能一眼看出是“卡在哪一步”，而不是“好像很慢”。

24.5.4 第四步：点 × 不是退出

现在讲最容易被误会的两个行为，先说那个叉号。

标题栏右上角那个 ×，点了之后窗口消失了。程序没有退出。它做的是“隐藏到系统托盘”：进程还在跑，后台调度器还在按点上班（盘前、盘中、盘后各有任务，主循环每 30 秒扫一次时段）¹⁵。想找回界面，去任务栏右下角找托盘图标：左键单击直接显示主界面，右键弹出菜单，里面两项——“显示主界面”和“退出”。

真的要退出，必须走托盘菜单里的“退出”。这个“退出”走的是 Wails 的生命周期函数而不是 `os.Exit(0)`，区别在于前者会执行关停清理（交易审批清理、调度器停止、审计日志刷新、数据库关闭）¹⁶。这一点和你在别的软件上的习惯正好相反，值得在第一次运行时试一次——本章最后的“动手试一试”里就安排了这一条。

托盘本身是自实现的：Wails v2 在 Windows 侧没有提供托盘 API，系统直接用原生 `Shell_NotifyIconW` 做了一个隐藏消息窗口，并在一条锁定的独立线程上跑它自己的消息泵¹⁷。为什么要额外一条线程？因为托盘回调消息只发给“创建那个窗口的线程”，而启动流程往往不在主线程上——不单独取消息，左键右键都会永远无人响应。

再说第二件事：单实例。你双击了两次，只会有一个程序。机制很短：进程一启动就去申请一个命名互斥量，名字是 `QuantBot_SingleInstance_Mutex_v5`；如果系统告诉你“这个名字已经存在”（Windows 错误码 183），说明已经有一个实例在跑，于是弹一个错误对话框然后退出¹⁸。

这条设计不只是防手抖，它是一致性的前提：两个实例同时开着一份 SQLite 和一份 DuckDB，两份调度器会同时往同一个账本里写决策、同时去跑同一段回测。互斥量把这件事从“可能发生的数据打架”变成了“根本不会发生”。

本章讲的“开机自检”，在界面上留下痕迹的地方是「设置·关于」页。

3/10] DuckDB、[Step 4/10] TDX 服务、[Step 5/10] Harness、[Step 6/10] LLM 客户端与 MCP、[Step 7/10] 任务执行器、[Step 8/10] 智能体与引擎、[Step 9/10] 服务与审计、[Step 10/10] 调度与编排。核对时间 2026-09-24。

¹⁵数据来源：系统源码 `App.go` 的 `HideToTray()`：注释写明“隐藏主窗口到系统托盘（标题栏 ×）：进程与后台任务（调度/回测/监控）继续运行”。调度主循环 30 秒 ticker 见系统事实清单第 6 节。核对时间 2026-09-24。

¹⁶数据来源：系统源码 `App.go` 的 `WindowQuit()` 与 `tray_windows.go` 的 `showTrayContextMenu()`：菜单项 ID 1 为“显示主界面”、2 为“退出”，回调分别触发 `onTrayShow` / `onTrayQuit`。核对时间 2026-09-24。

¹⁷数据来源：系统源码 `tray_windows.go`：`runTrayAsync` 里 `runtime.LockOSThread()` 后创建窗口并进入 `GetMessage` 循环；文件头注释记录了“Wails startup 常跑在非主线程且不带消息泵，托盘消息无人派发导致菜单弹不出来”这一真实问题。另监听 `TaskbarCreated` 以便资源管理器重启后重建图标。核对时间 2026-09-24。

¹⁸数据来源：系统源码 `single_instance_windows.go` 的 `SingleInstance()`：`CreateMutexW + GetLastError()` == `ERROR_ALREADY_EXISTS(183)`，非首实例会 `CloseHandle` 并返回 `false`；`main.go` 中 `!isFirst` 分支调用 `showAlreadyRunningDialog()` 后 `os.Exit(0)`。核对时间 2026-09-24。



图 24.2 设置 • 关于：版本信息、组件状态与数据基准日

24.6 常见误解与纠偏

常见误解 · 先把策略和参数调好，再回来装环境 这是顺序反了，而且反得很自然。人总想先做“有技术含量”的那部分——挑策略、调参数、看回测曲线。可是请回想本章开头那句“好了吗”：在你连“今天能不能工作”都不知道之前，所有的策略讨论都建立在沙子上。

举个非常具体的场景。你花了两个小时把双均线的参数调到历史回测最漂亮的组合，然后回到设置页一看，行情库的最新交易日停在上周，AI 的 Key 也从来没配过。那你这两个小时调出来的东西，是在一段已经过期的数据上调出来的，而它连一次实时的判断都做不了。

正确的顺序就是本章的标题：**安装 → 接模型 → 连数据**，然后才是第 chapter 二十五 章的数据维护、再往后的方案与策略。先让机器“今天能工作”，再谈它“工作得好不好”。

常见误解 · 报错就是报错，先想办法绕过它 几乎每个人的第一反应都是这样：弹窗挡路了，就把弹窗点掉；加载页说行情库不可用，就按“进入系统”接着用。反正界面能进去。

但请把这句话读两遍：“能跑但数据是假的”，比“跑不起来”危险得多。跑不起来，你今天的动作是零；能跑但数据是假的，你会得到一个看起来很正常的错误决策，并且——按第 chapter 二十三 章的 T+1——它当天不可撤销。

系统把这件事做成了一条硬约定：关键组件失败必须弹窗报错，**严禁降级、严禁使用虚假数据**。它宁可停在加载页上让你看见红灯，也不肯给你一个“用旧缓存顶上”的顺滑假象。那个“进入系统”的按钮是给你知情之后用的（比如你确认今天只是想让它模拟盘上空转），不是给你绕过红灯用的。

顺带一提，同样的诚实也写在数据层：行情源全部失败时，系统返回空，而不是伪造一条看起来合

理的行情——这一条在第 chapter 二十五 章会反复出现。

24.7 真实出处

本章的真实出处

- 首次运行向导（四步、默认初始资金、提供商与 `Key` 的保存时机、内置提供商默认表）：
`ui/src/pages/WelcomeWizard.tsx`
- 启动加载页的组件状态与关键标记、30 秒等待与“进入系统”按钮：
`ui/src/components/InitialLoading.tsx`
- 启动初始化 `startup(ctx)`：十步日志、25 秒看门狗、`panic` 恢复、真实连通性探测
`runStartupConnectivityCheck()`、`IsReady()` 状态字段：`App.go`
- 点 × 隐藏到托盘 (`HideToTray`) 与真正退出 (`WindowQuit`): `App.go`、`ui/src/components/TitleBar.tsx`
- 原生 Win32 系统托盘 (隐藏窗口 + 独立线程消息泵 + `TaskbarCreated` 重建): `tray_windows.go`
- 单实例命名互斥量与“已在运行”对话框: `single_instance_windows.go`、`main.go`
- 配置文件路径与出厂默认值 (资金、提供商、模型、接口地址、交易模式) : `config/config.json`、
`internal/config/config.go` 的 `defaultConfig()`
- 可执行文件目录推导、日志与崩溃转储路径: `main.go` 的 `GetExecDir()`、`setupLogging()`

动手试一试（十五分钟，今天就能做完）

1. 把压缩包解压到一个不含中文和空格的路径（例如 D: 盘根目录下的 `quantbot` 文件夹），双击主程序，走完四步向导。第 1 步把初始资金留成默认的 100000^a，第 2 步选 `deepseek` 并粘上你的 `Key`。
2. 盯着启动加载页，逐行看那几盏灯。你想要的终点是**六项组件全绿**（第七行“组合数据预热”也要绿）。只要有一盏红的，先别点“进入系统”，把红字读完——它已经告诉你缺什么了。
3. 进主界面后打开设置页，切到 AI 页签，核对三样东西：提供商、模型名、接口地址。再看一眼 `Key` 是否有效——如果启动时那次“ping”失败，加载页会明确告诉你 AI 不可用，而不是留一个空壳界面给你猜。
4. 点标题栏的 ×。窗口消失了，但程序没退：去任务栏右下角找到托盘图标，左键单击，看它是不是原样回来了。
5. 再双击一次主程序。你应该看到“程序已在运行中”的错误对话框，而不是第二个窗口——这说明单实例互斥量生效了。想真正退出，走托盘右键菜单里的“退出”。

^a**真实数据**：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

到这里，机器通电了、模型接上了、通路也自检过了。你手上有一个“今天可以工作”的系统：它知道自己有多少钱、用哪个大脑、数据在哪。

但“能打开行情库”和“行情库里的米是新鲜的”是两件事。下一章我们就来干这件最像家务、也最能决定后面所有结论的事——先把米下锅：数据维护的正确顺序。顺序错了，后面全是假结论。

先把米下锅：数据维护的正确顺序

25.1 我急着选股，系统却问我“米呢”

装好、配好、连好之后，我做的第一件事是点开选股页，想看看这台机器到底能挑出什么股票。

界面上弹出一句话，短短几个字，但把我卡住了：

行情数据库 (stock.duckdb) 未挂载

我当时的反应是“我明明装了”。找了一圈才明白，问题不在装没装，而在装进去的是空缸——数据库文件在，里面却没有可用的日 K 线。就像厨房装修得很漂亮，一拧开水龙头，是干的。

我翻回去重看第 chapter 二十四 章里那个启动加载界面：几个组件都亮着，“股票时序库”也亮着。可“亮着”只代表组件起来了，不代表里面有米。

那天我学到一个必须记住的事：选股是最后一步，数据维护才是第一步。而且顺序也不能随便。

25.2 下厨之前，先想清楚备料的次序

直觉·下厨前先备料 你一定见过这种场面：火已经开了，油已经热了，主料还在冰箱里没解冻。先把米淘好、把菜洗净切好、把调料摆到手边，然后才开火。你要做的是“青椒炒肉”，结果肉还没解冻就下了青椒，那盘菜的名字就应该改成“糊青椒”。

数据维护就是这样一桌备料。行情是米，复权因子是盐，交易日历是灶上的火候表，财务数据是配菜，盘口摘要是出锅前那一勺明油。它们各有用处，而且有先后。把复权因子放到行情前面下锅，行不行？也能执行，但后面那道前复权视图会是空的——因为它要去连接的数据还没到。

所以这一章先讲清楚：这六样料各干什么，少一样会坏哪一步菜。具体怎么点按钮，最后我有一张清单。

25.3 为什么顺序不能颠倒：六步备料，各自的“跳过会怎样”

先把顺序整体说一遍，下面再一步一步拆开。系统的「设置 → 数据维护」把这件事拆成了同花顺数据页签下的六件事，正确的次序是：

- 第 1 步： 设置通达信 vipdoc 目录，同步 .day 日线（本地兜底与手工补位）
- 第 2 步： 全量导入（近 3 年）（同花顺官方全市场日 K）
- 第 3 步： 一键同步全部（复权因子 + 交易日历）
- 第 4 步： 财务数据维护（同花顺官方三表）

第 5 步：每天 16:00 的自动拉取与告警（这一步不用你动手，但要知道它在盯着）

第 6 步：收盘后的盘口摘要（落 `order_book_daily`，供次日盘口因子）

顺序的道理只有一句：后一步要读前一步的结果。复权因子要拿 `stock.ohlcv` 的收盘价去推算比率；前复权视图 `stock.ohlcv_qfq` 又要拿复权因子去连接行情；盘口因子明天要读今天落的盘口摘要。谁先谁后，不是偏好，是依赖。

25.3.1 第一步：先把米缸接上——通达信 `vipdoc` 目录

第一件事是告诉系统“你的本地数据从哪来”。系统会去枚举 `vipdoc` 下的日线文件，规则写在 `internal/data/maintenance.go` 里：路径形如

```
\{tdxPath}\vipdoc\{bj|sh|sz}\1day\{market}\{code\}.day
```

文件名必须是“2 位市场前缀 + 6 位代码 + .day”，也就是 `sh600519.day` 这种整整 12 个字符的名字；不合规的文件会被直接跳过。文件内容是通达信固定的 32 字节小端记录：日期、开、高、低、收（单位为“分”）、成交额、成交量。解析出来的每根 K 线，被合并进 `stock.stock_daily`，合并规则是 `ANTI JOIN`——只插入库里没有的 (`symbol,date`)，重复自动跳过。所以它叫“增量”：再点一次，数据不会翻倍。

跳过会怎样？如果你没有通达信本地数据、也没配同花顺官方源，那么 `stock.stock_daily` 底表就是空的。选股引擎在进 `ScreenStock` 之前会先校验数据库是否可用，查不到就直接报错、不返回假数据¹。“未挂载”这四个字，就是这么来的。

还要如实说明一件事：通达信这条路，系统已经取消了自动执行，只保留手工补位²。它现在的定位是“备用粮仓”——同花顺官方源临时不可用时，你还能手工把最新几天的日 K 补进去。

25.3.2 第二步：全量导入（近 3 年）

第二步才是主粮。前端那个按钮的文案是“全量导入（近 3 年）”，不是“全量导入”，括号里三个字很重要。

后端口径就写死在 SQL 里：`ImportTHSDailyK(ctx, "full")` 只保留 `date >= now() - INTERVAL '3 years'` 的行³。

为什么是 3 年？因为数据多并不等于数据好——你真正会用到的窗口通常就是最近这几年；把十几年前的老黄历塞进来，只会拖慢查询，还容易把制度变迁的断层混进统计。3 年是“够用且不拖累”的折中。

跳过会怎样？这一步跳过，`stock.ohlcv` 只有你手工补进来的零星几天，选股样本会少得可

¹数据来源：系统事实清单第 9 节：`ScreenStock(req)` 先校验 `DuckDB` 与 `HasStockDB()`，否则报错“行情数据库 (`stock.duckdb`) 未挂载…”，不返回假数据。核对时间 2026-09-24。

²数据来源：系统事实清单第 16 节：通达信数据自动执行已取消，仅在「数据维护-通达信数据」手工补位。核对时间 2026-09-24。

³数据来源：系统源码 `internal/data/thd_dailyk.go`：mode == "full" 时 `dateFilter = " AND date >= CAST(now() - INTERVAL '3 years' AS DATE)"`。核对时间 2026-09-24。

怜，任何统计都不可信。两者互补：通达信负责“存量兜底 + 最新补位”，同花顺负责“铺满近 3 年底子”。

25.3.3 第三步：一键同步全部（复权因子 + 交易日历）

这一屏里有两个按钮，主按钮是“一键同步全部（复权因子 + 交易日历）”，次按钮是“仅导入复权因子”。点主按钮时，系统按固定次序执行两件事：

`ImportTHSAdjFactors` → `SyncTHSTradingCalendar`

结果在界面上分两行用 ✓ 显示，你一眼就能看出是哪一行没成功⁴。

复权因子在做的事，我留到下一节单独讲，因为它是本章唯一需要证明的东西。这里先说结论：它把 `stock.ohlc` 和 `stock.adj_factor` 连起来，生成前复权视图 `stock.ohlc_qfq`。

交易日历在做的事是把“哪天开市”从猜变成查。过去系统靠“周末休市 + 一张内置节假日表”来判断，现在改成从同花顺官方拉近一年的交易日序列，写进 `stock.trading_calendar`。刷新入口是 `util.SyncTHSTradingHolidays`，窗口固定为 [今日-1 年，今日]，按 Asia/Shanghai 计算，没有入参——你不需要告诉它拉哪一年，它自己按窗口算⁵。

跳过会怎样？分两种：

- 跳过复权因子：除权那天价格会“凭空”掉一截（比如每 10 股派 2 元现金，前收就少 0.2 元）。均线类策略只看价格形状，会把这次分红造成的台阶误读成一次真实下跌，发出完全错误的卖出信号。这不是“精度差 1%”，是方向错了。
- 跳过交易日历：系统退回“周末休市”的近似假设。碰到国庆、春节这类长假，它会把休市日当成交易日，去判断“上一交易日”“数据是否最新”，判断全错。

25.3.4 第四步：财务数据维护

行情是价格，财务是“这家公司赚不赚钱”。第四步把同花顺官方的三张表（利润表、资产负债表、现金流量表）拉下来，按报告期对齐、算好同比与各项比率，写进 `stock.financial_report`。

这一步有一条硬规矩：没配 API Key，直接拒绝。`NewTHSFinancialFetcher` 第一件事就是取官方客户端，取不到就返回错误提示，不会伪造、也不会跳过⁶。

全市场几千只股票 × 三张表，是本章最“重”的一步。系统给它配了三层稳压器，全在 `internal/thssdk` 里：

- 并发限流：同时最多 5 个请求在飞 (`maxInflight = 5`)；

⁴数据来源：系统事实清单第 14 节：一键同步全部依次调用 `ImportTHSAdjFactors` → `SyncTHSTradingCalendar`，结果分两行 ✓；「仅导入复权因子」为次级按钮。核对时间 2026-09-24。

⁵数据来源：系统源码 `internal/util/trading_calendar.go: SyncTHSTradingHolidays(tradingDays []string)` 以 `windowStart := today.AddDate(-1,0,0)`，逐日对工作日判断是否在交易日列表中。核对时间 2026-09-24。

⁶数据来源：系统源码 `internal/data/financial_ths.go: c := thssdk.Default(); if c == nil \{ return nil, fmt.Errorf(" 同花顺官方数据源未配置..." ) \}`。核对时间 2026-09-24。

- **全局速率令牌桶**：`maxRPS = 15`，也就是全局限速 15 请求/秒，平均单次请求间隔约 67 毫秒⁷；
- **指数退避重试**：只对真正“临时性”的失败重试——HTTP 429、5xx，以及业务码 5003（官方数据源临时不可用）。退避等待是 $1 \ll (\text{attempt}-1) * 500\text{ms}$ ，即 0.5s、1s、2s，加上首次共最多尝试 4 次；其余确定性错误（比如 1002 代码不存在）不重试，直接返回⁸。

再加一层保险：**断点续传**。同步按 `symbol` 逐个走，某只失败只记一次失败、继续下一只；重复点“增量”时，起点取该股表内已有的最大报告期 `maxFinReportDate`，不会重头再来⁹。

还有一条本章必须写明的真实事故：**财务表过去只有手动同步才会补数据**——自动调度里根本没带这一步。所以你会看到财务表长时间停在某个旧的报告期，直到你手动点一次¹⁰。价值因子、质量因子都吃这碗饭，这一步空着，它们给出的基本面得分就是没有依据的。

25.3.5 第五步：每天 16:00，让它自己下锅

前四步是“一次性铺底”，从第五步起是“每天续上”。

系统内置一个每日 16:00 的定时器，只做一件事：把当天（或最近一个交易日）的日 K 拉进来。它只走同花顺官方增量拉取，拉完还会**重新判位**——看数据到底有没有覆盖到目标交易日，而不只是看接口有没有报错¹¹。

如果没到位，它会弹窗告警：

同花顺数据同步未成功

这不是提醒你“接口又限流了”，而是提醒你：**明天开盘前，你的数据可能还停在昨天之前**。告警里还给了下一步建议（到下个交易日 9 点前补数据）¹²。

这一步最容易被忽略——你什么都没按，是它在替你按。如果你把托盘程序退出了，16:00 就没人拉数据，第二天开盘你会看到“昨天的数据”。

25.3.6 第六步：收盘后的盘口摘要

最后一步是收盘后落一份“盘口摘要”到 `order_book_daily`。它记录的是每只票当天收盘时刻的盘口状态（涨跌停、封单、量比、内外盘等），供次日的盘口因子使用。

⁷数据来源：系统源码 `internal/thssdk/thss.go: maxInflight = 5, maxRPS = 15`，令牌桶容量与速率均为 15。1000/15 约等于 67。核对时间 2026-09-24。

⁸数据来源：系统源码 `internal/thssdk/thss.go` 的 `get: const maxAttempts = 4, 退避 time.Duration(1<<(attempt-1)) * 500 * time.Millisecond`，仅当 429、5xx 或 5003 时继续重试。核对时间 2026-09-24。

⁹数据来源：系统源码 `internal/data/financial_sync.go: StartFinancialSync` 循环内 `startReport = dm.maxFinReportDate(sym)`，单只失败 `failed++`；`continue`。核对时间 2026-09-24。

¹⁰数据来源：系统事实清单第 15 节事故 #12：财务表只有手动同步才补数据，根因是自动调度不含财务同步。核对时间 2026-09-24。

¹¹数据来源：系统源码 `app_scheduler.go: startTHSDailyKSyncTimer` 计算下一个 16:00，触发 `syncDailyKlineAuto`，返回后由 `dailyKlineCaughtUp(want)` 判定是否到位。核对时间 2026-09-24。

¹²数据来源：系统源码 `app_scheduler.go` 的 `emitDailyKSyncWarning`：消息以“同花顺数据同步未完全成功：”开头，并提示在下个交易日 9 点前补充股票时序数据。核对时间 2026-09-24。

这里有一条重要的设计原则：它是收盘后落库的，所以次日用时看到的是“已经发生过的昨天”，没有前视偏差——你不会用到“今天的收盘盘口”去决定“今天该怎么交易”这种时间倒流的事¹³。

而现在要如实说明它的状态：它是空的。本书核对时 `order_book_daily` 没有任何记录 0 行¹⁴。所以本章凡提到“今天的盘口因子”，都应理解为“设计上如此、当前数据缺失”。盘口因子权重固定为 0.05，有效样本不足 20 只时整个因子不参与评分——表是空的，它自然不参与。

25.4 一条容易被忽略的硬约束：同一天的多条复权事件

现在回头认真讲第三步里那个“复权因子”。

它要先下载官方全市场复权事件 Parquet，再自己推算日频因子。这里埋着一个系统真实修过的坑：同一只股票、同一天，官方文件里可能不止一条记录。比如“每 10 股派 2 元”又叠加“每 10 股送 3 股”，会作为两行落在同一天。

而 `stock.adj_event` 的主键是 `(symbol, ex_date)`——两行撞同一个主键，直接冲突。系统当年的报错就出在这里。

修复办法是“先聚合，再写库”。这一聚合，就产生了一个必须证明的命题。

25.4.1 先给一个“差不多对”的近似，再看它在哪失效

外行的直觉是这样的：既然同一天有两件事，那就加起来不就完了？现金分红相加、送股比例相加、配股比例相加，配股价……平均一下。听起来很自然。

现金、送股、配股比例取和，这三件事是对的（下一节的命题会证明）。但配股价取“平均”这一步，平均的方式藏着一个陷阱。直觉上大家会取算术平均 $\bar{p} = \frac{1}{k} \sum_i p_i$ ；可算术平均会把配股的总募资额改掉，而募资额恰恰是决定除权参考价的那个量。

用一个构造反例就能看清。设同一天有两条配股事件：

事件一：配股比例 $r_1 = 1$ ，配股价 $p_1 = 5$ 元；事件二：配股比例 $r_2 = 3$ ，配股价 $p_2 = 10$ 元。

两条事件真实要募集的钱是 $r_1 p_1 + r_2 p_2 = 1 \times 5 + 3 \times 10 = 35$ 元。总配股比例 $R = r_1 + r_2 = 4$ 。

- 若取加权平均 $\bar{p} = \frac{r_1 p_1 + r_2 p_2}{r_1 + r_2} = \frac{35}{4} = 8.75$ ，则 $R\bar{p} = 4 \times 8.75 = 35$ ，与真实募资额分毫不差。
- 若取算术平均 $\bar{p}_{arith} = \frac{5 + 10}{2} = 7.5$ ，则 $R\bar{p}_{arith} = 4 \times 7.5 = 30$ ，比真实的 35 少了 5 元，偏差 $5/35 \approx 14\%$ 。

同样的道理：用算术平均，等于给“比例大、价格低”的事件派了高价，又给“比例小、价格高”的派了低价，一加权就全歪了。加权平均不是“更好看的平均”，它是唯一能让募资额守恒的平均。下面把它写成命题。

¹³数据来源：系统事实清单第 11 节：每日收盘后 `OrderBookDailyProcessor.RecordOrderBookDaily` 写 `order_book_daily`，供次日盘口因子，无前视偏差。核对时间 2026-09-24。

¹⁴数据缺失：该系统库中此项无有效记录。本书遵照系统“数据不可用即报错、不降级、不伪造”的原则，此处留白、不填充估计值。

25.4.2 命题与证明

我要先说明系统实际用的公式，因为本书的一切都以源码为准。推算复权比率的那一行 SQL，分母是 `prev_close - d + r * p`，分子是 `prev_close * (1.0 + s + r)`，其中 `d` 是每股现金红利、`s` 是每股送股比例、`r` 是配股比例、`p` 是配股价，`prev_close` 是除权日前一交易日收盘价¹⁵。等价地，把除权参考价写成

$$p_{\text{ex}} = \frac{p_{t-1} - d + r p}{1 + s + r}, \quad \text{于是} \quad \text{ratio} = \frac{p_{t-1}}{p_{\text{ex}}} = \frac{p_{t-1} (1 + s + r)}{p_{t-1} - d + r p}. \quad (25.1)$$

聚合的那一段 SQL，正是“求和 + 按 r 加权平均”：`SUM(d)`、`SUM(s)`、`SUM(r)`，配股价 `CASE WHEN SUM(r) > 0 THEN SUM(r * p) / SUM(r) ELSE 0 END`。

命题 25.1 (复权事件按 `(symbol, ex_date)` 聚合去重的守恒性). 设某股票在同一除权日 `ex_date` 上有 k 条官方事件 (d_i, s_i, r_i, p_i) , $i = 1, \dots, k$ ，除权日前一交易日收盘价为 p_{t-1} ($p_{t-1} > 0$)。系统按下式把它们聚合成一条事件：

$$D = \sum_{i=1}^k d_i, \quad S = \sum_{i=1}^k s_i, \quad R = \sum_{i=1}^k r_i, \quad \bar{p} = \begin{cases} \frac{\sum_{i=1}^k r_i p_i}{R}, & R > 0, \\ 0, & R = 0. \end{cases} \quad (25.2)$$

并记单条事件 $(D, S, R, \bar{p})$ 的除权参考价为 $\varphi = p_{\text{ex}}(D, S, R, \bar{p})$ (见式 eq. (25.1))。则：

(i) (现金守恒) $D = \sum_i d_i$ ，聚合后的每股现金红利等于原各条之和；乘上股本，总现金分红分文不差。

(ii) (送股守恒) $S = \sum_i s_i$ ，每股送股比例不变。

(iii) (配股募资守恒 / 配股等价) 当 $R > 0$ 时， $R\bar{p} = \sum_i r_i p_i$ ：按加权平均配股价算出的总配股募资额，与逐条事件之和完全相等。

(iv) (参考价不变) 聚合后的除权参考价 φ ，与“把 k 条事件视为一个合并事件” (总现金 D 、总送股 S 、总配股 R 、总募资 $\sum_i r_i p_i$) 算出的参考价完全相同；因此复权比率不变，前复权价格在事件日前后连续、无人工跳空。

这个命题在说什么：一句话，“同一天的多件事”可以放心地合并成“一件事”，只要配股价用加权平均——合并前后，分红、送股、配股募资这三笔账的总额一点不变，除权参考价也不变。所以前复权序列里不会因为合并，多出一个假的涨跌。

证明. (i)、(ii) 由式 eq. (25.2) 中 D 、 S 的定义直接得到：它们本来就是各条之和。合并不改变总量，只是把多行写成一行。

¹⁵数据来源：系统源码 `internal/data/thx_adjfactor.go` 的 `ratios` CTE: `(kp.prev_close * (1.0 + eff.s + eff.r)) / NULLIF(kp.prev_close - eff.d + eff.r * eff.p, 0) AS ratio`。核对时间 2026-09-24。

(iii) 当 $R > 0$ 时，把 $\bar{p}$ 的定义代入：

$$R\bar{p} = R \cdot \frac{\sum_{i=1}^k r_i p_i}{R} = \sum_{i=1}^k r_i p_i. \quad (25.3)$$

分子分母里的 R 相消，右端正是逐条募资额之和。这就是“配股等价”的全部内容。

作为对照，若改用算术平均 $\bar{p}_{\text{arith}} = \frac{1}{k} \sum_i p_i$ ，则 $R\bar{p}_{\text{arith}} = \frac{R}{k} \sum_i p_i$ ，一般不等于 $\sum_i r_i p_i$ ；只有所有 r_i 相等时才偶然相等。这说明加权平均不是口味问题，而是等式能否成立的前提。

(iv) 先把“合并事件”的参考价写出来：总现金 D 、总送股 S 、总配股 R 、总募资 $\sum_i r_i p_i$ ，代入式 eq. (25.1) 得

$$\varphi_{\text{merged}} = \frac{p_{t-1} - D + \sum_{i=1}^k r_i p_i}{1 + S + R}. \quad (25.4)$$

再把聚合事件的参考价写出来。由 (iii) 得 $R\bar{p} = \sum_i r_i p_i$ ，代入得分子为 $p_{t-1} - D + R\bar{p} = p_{t-1} - D + \sum_i r_i p_i$ ，分母为 $1 + S + R$ 。两式逐项相同，故

$$\varphi = \varphi_{\text{merged}}. \quad (25.5)$$

参考价相同，则由式 eq. (25.1)， $\text{ratio} = p_{t-1}/\varphi$ 也相同。

最后看“无跳空”。设除权日为该事件生效的有效交易日 t^* 。复权因子的构造是“ t^* 起累计乘上 ratio”（源码里 `backward_factor` 按 `EXP(SUM(LN(ratio)))` 在时间轴上累乘，再除以最新日的值归一化得到前复权因子）。记归一化分母为 B^* （与 t 无关的常数），则相邻两日的前复权价格为

$$q_{t^*-1} = p_{t^*-1} \cdot \frac{B_{t^*-1}}{B^*}, \quad q_{t^*} = p_{t^*} \cdot \frac{B_{t^*-1} \cdot \text{ratio}}{B^*}, \quad (25.6)$$

两者相除， B_{t^*-1}/B^* 消掉：

$$\frac{q_{t^*}}{q_{t^*-1}} = \frac{p_{t^*}}{p_{t^*-1}} \cdot \text{ratio} = \frac{p_{t^*}}{p_{t^*-1}} \cdot \frac{p_{t^*-1}}{\varphi} = \frac{p_{t^*}}{\varphi}. \quad (25.7)$$

这个结果正是“剔除了除权机制影响之后的真实涨跌”：若当日实际收盘恰等于理论除权价（ $p_{t^*} = \varphi$ ），则 $q_{t^*}/q_{t^*-1} = 1$ ，前复权价格连续、不跳。也就是说，除权造成的机械台阶被因子精确地补平了；剩下的起伏，才交给策略去看。■

注 25.2 (为什么前复权要“归一到最新日为 1”)。源码里还有一步归一化：`forward_factor[t] = backward_factor[t] / backward_factor[最新日]`。这一步让最新一天的前复权价等于原始价，你看到的现价不会被缩放。代价是：每来一次新的除权事件，整条历史序列都会被重新缩放一次。所以“复权序列”是随着最新事件动态变化的，不是一个固定不变的数表——这也是为什么它建成了视图（`stock.ohlcv_qfq`）而不是一张静态表。

25.5 它在系统里长什么样：真实库容

把上面的道理落到地上，就是本章最重要的那张表。所有数字都是核对时刻从 `data/stock.duckdb` 只读查出来的，没有任何估算。

表 25.1 行情库的表与视图及其真实规模

表 / 视图	行数	覆盖与说明
stock.ohlcv	10,041,343 ¹⁶	未复权日 K，覆盖 2021-08-02 至 2026-09-24
stock.stock_basic	19,068 ¹⁷	标的字典；含 ETF/基金，其 name 为 NULL
stock.adj_event	14,172 ¹⁸	除权事件 (d,s,r,p)
stock.adj_factor	9,997,923 ¹⁹	日频复权因子 backward/forward
stock.trading_calendar	248 ²⁰	交易日；最新一条为 2026-09-17
stock.financial_report	315,680 ²¹	财务三表对齐结果
order_book_daily	0 ²²	盘口摘要，当前无任何记录
stock.ohlcv_qfq	视图	前复权 K 线，由 stock.ohlcv 左连接 stock.adj_factor 生成

stock.ohlcv 去重后覆盖 9,584²³ 个 symbol；四只基准指数 sh000001、sz399001、sh000300、sh000905 均在库，各 1249²⁴ 根。stock.ohlcv_qfq 是无因子的行按因子 1 返回的视图，故行数为空（不落盘）。
数据来源：库 `data/stock.duckdb` 的具体表，核对时间 2026-09-24；行数来自系统事实清单第 13.2 节。

这张表里有三处必须展开讲，否则你会看错。

第一，交易日历是“滞后”的。stock.trading_calendar 最新只到 2026-09-17，而行情库 stock.ohlcv 已经覆盖到 2026-09-24。这不是 bug，是两类数据的刷新节奏不同：交易日历按“近一年窗口”整段重建，行情库按日增量。你核对时看到日历“落后几天”，属于正常窗口错位；但真要判断“今天是不是交易日”，最好确认日历已经覆盖到你关心的那天。

第二，stock.stock_basic 不是“全是个股”。它里面有 ETF 和基金份额，比如 sz167301、sz200570，这些代码的 name 字段是 NULL²⁵。所以你做“全市场个股统计”时必须先过滤，否则分母里会混进一堆没有名字的基金，广度、涨跌家数之类的指标会被稀释。这是第 chapter 十九 章讲市场广度时反复强调过的事。

第三，唯一一行空的就是盘口摘要。order_book_daily 是 0 行。本书遵照系统“数据不可用即报错、不降级、不伪造”的原则，这一格留白，不填任何估计值。

“数据从哪来”这件事在界面上只有一个入口：「设置 · 数据源」。下面这张图是它的实际样子。

25.6 常见误解与纠偏

常见误解 · 先把策略跑起来，数据慢慢补 这是最自然的冲动：我想快点看到结果，为什么不能先跑着？因为你跑出来的每一个数字，都会被“数据不全”翻译成一句看着很确定、其实是假的结论。选股样本少了一半、复权没做、财务还是两年前的——系统不会告诉你“这些分数可能不可信”，它只会老老实实把分数算出来给你看。而你会信。

系统的态度是相反的：进选股前先校验数据库，查不到就报错退出，宁可让你做不成，也不给你一

²⁵数据来源：系统事实清单第 13.2 节：stock_basic 含 ETF/基金类 symbol（如 sz167301、sz200570），其 name 为 NULL。核对时间 2026-09-24。



图 25.1 设置·数据源：五个可选数据源，以及“腾讯财经只覆盖实时行情，研究中心仍读本地 DuckDB”的口径说明

份假结果。这条“fail-closed”的原则，贯穿在它每一次数据不可用的处理里。你也应该用同样的态度对待自己的操作：数据没备齐，就别开火。

常见误解·复权因子是锦上添花，不做也行 很多人把复权理解成“让图表好看一点”，于是觉得它是可选的美化。它一点都不“锦上添花”。均线类策略只认价格的形状。除权那天，价格会真实地掉一个台阶（这就是式 eq. (25.1) 里的 p_{ex} ）。如果不做复权，这个台阶会被策略当成“一次下跌”，快线随之跌破慢线——一个方向完全错误的卖出信号就这么诞生了。

注意错误的量级：这不是“少赚一点”，是“在你根本不该卖的时候卖”。而且分红、送股越频繁的股票，这种假信号越多，恰好会系统性地把高分红的好公司从你的候选里筛掉。所以复权因子不是可选项，是数据入口的一部分——它和行情本身同属“米”，不属于“盐”。

还有个相关的坑：复权做成视图、每次除权都会重算整条历史，所以别把“前复权价”当成永远不变的历史事实。它是“以今天为基准回看”的镜头，基准一变，整卷胶片都要重洗。

25.7 真实出处

- 本章的真实出处
- 通达信 .day 解析、vipdoc 枚举、ANTI JOIN 增量合并: `internal/data/maintenance.go`
 - 同花顺官方全市场日 K 导入 (`ImportTHSDailyK`, 近 3 年窗口): `internal/data/th_dailyk.go`
 - 复权事件聚合与日频因子推算、前复权视图: `internal/data/th_adjfactor.go` (表 `stock.adj_event`、`stock.adj_factor`、视图 `stock.ohlc_qfq`)
 - 官方交易日历同步 (`SyncTHSTradingCalendar`) 与 `util.SyncTHSTradingHolidays`: `internal/data/th_calendar.go`、`internal/util/trading_calendar.go` (表

`stock.trading_calendar)`

- 财务数据同步(限流并发 5 / 15 req/s 令牌桶 / 429•5xx•5003 退避 / 断点续传): `internal/thssdk/thss.go`、`internal/data/financial_sync.go`、`internal/data/financial_ths.go` (表 `stock.financial_report`)
- 每日 16:00 日 K 定时同步与“同花顺数据同步未成功”告警: `app_scheduler.go` 的 `startTHSDailyKSyncTimer`、`syncDailyKlineAuto`、`emitDailyKSyncWarning`; 调度层 `internal/scheduler`
- 盘口摘要落库 (表 `order_book_daily`): `internal/orderbook/orderbook.go` 的 `RecordOrderBookDaily`
- 真实库容数字: `data/stock.duckdb`, 核对时间 2026-09-24; 明细亦见附录 chapter B

动手试一试：按顺序点一遍三项同步

1. 打开「设置 → 数据维护 → 同花顺数据」页签。先确认顶部“同花顺官方金融数据服务”已启用并填好 API Key——没配的话，下面每一步都会直接报错，这是设计如此，不是坏了。
2. 点“一键同步全部（复权因子 + 交易日历）”，盯住结果区：它应当分两行各出现一个 ✓，第一行是复权因子、第二行是交易日历。哪一行没有 ✓，就是哪一步没成功。
3. 点“全量导入（近 3 年）”，等它跑完。这一步是把近 3 年的日 K 底子铺满，耗时取决于网络，属于正常的长任务。
4. 用一条只读 SQL 核对成果（例如 `SELECT COUNT(*) FROM stock.ohlcv;` 与 `SELECT MIN(date), MAX(date) FROM stock.ohlcv;`）。对照表 table 25.1 的行数与日期区间：行数应远大于几千，日期区间应覆盖到最近一个交易日。
5. 再查一次盘口摘要：`SELECT COUNT(*) FROM stock.order_book_daily;`。核对时它是 0^a——如果是 0，说明它确实还没开始落库，本章里所有依赖它的因子都应视为“数据缺失”。

^a“数据缺失：该系统库中此项无有效记录。本书遵照系统“数据不可用即报错、不降级、不伪造”的原则，此处留白、不填充估计值。

到这一步，米缸是满的、盐和火候表都摆好了。第 3 步的那个 ✓，不只是“同步成功了”四个字——它背后是命题 theorem 25.1 证明过的守恒性：你接下来看到的每一条均线、每一个前复权价，都不会被一次分红或送股骗过去。

现在数据齐了，可系统还不知道你是谁。一个能承受 30% 回撤的年轻人和一个只求跑赢存款的中年人，应该拿到的方案完全不同。第 chapter 二十六 章，我们就来回答那个听起来很啰嗦、其实每个字都算数的问题：该给你生成一份什么样的投资方案？

生成属于你的第一份投资方案

26.1 十二个问题，一个都不能少

第一次点开「投资规划」，我以为迎接我的是一个“一键生成”按钮。

它先不给我股票，先问我问题。证券投资经验几年？买过哪些股票类产品？靠什么做决策——别人推荐、新闻政策、基本面、技术分析，还是量化模型？风险承受等级选 R1 到 R5 哪一个？亏了怎么办？能接受的最大亏损是多少？这笔钱投多久？图的是保值、稳收益、平衡，还是增值？要不要随时能取？准备投多少？偏好哪种风格？看好哪些行业？

十二道题，一道一道来。

我当时的反应很实在：你能直接把代码给我吗？我是来用系统的，不是来答卷的。

我忍着填完，然后做了一个小动作：把“风险承受能力”从 R4 改成 R3。屏幕上的东西立刻变了——三套候选方案里最激进的那套（C）被标了红，注明与我的风险等级不匹配；下面那张“建议持仓”表里的名字，也换了一批。

这下我明白了。这 12 个答案不是入场登记，它们是系统“给谁配什么”的全部输入。其中任何一个改了，我最后该拿的股票就变了——而且是牵一发动全身地变。

本章讲的就是这条链：12 个答案 → 一份投资风格画像 → 三套候选方案 → 一份真实的股票池 → 一张带权重的持仓表，以及这套系统每天怎么自动把它刷新一遍。

26.2 先量身体，再定训练量

直觉·体检套餐 + 私人教练计划 健身房里最常见的错误，是直接抄别人的训练计划。

你朋友每周五练、深蹲 100 公斤，你照着抄，第三天腰就废了。因为那份计划是给他的身体开出来的。正确顺序是：**先体检**——量体重、测体脂、查血压、看旧伤；**再由教练开训练量**。体检不是仪式，它是训练计划的分母。

投资一模一样。“12 题问卷”是体检，“A/B/C 三套候选”是教练给出的三种训练强度，“建议持仓表”是最后那组动作。跳过体检直接抄持仓，等于拿别人的腰做自己的深蹲。

这个类比还有第二层，更要紧：体检报告上“血压偏高”这四个字，会改掉教练的整份计划；问卷里“这笔钱随时要用”这一个答案，也会改掉你该持有的股票。一个输入动了，整条链都得跟着动。

26.3 不做画像，要付什么代价

先把“为什么非做不可”讲透，否则这 12 题看起来就是走过场。

假设两个人，账面上都是 10 万元，同一天进来，拿到了同一张持仓表。

第一个人 30 岁，这笔钱五年不用，能接受三成回撤。市场跌 25%，他拿得住，甚至还有闲钱补仓。

第二个人下个月要用这笔钱付首付。同样是跌 25%，他不是“能不能扛”的问题，是“必须卖”的问题——他会在最低点被迫清仓，而这件事和股票选得好不好毫无关系。

同一张持仓表，一个能扛住，一个被逼死在最低点。差别不在股票，在画像。

所以问卷的真正作用，是给后面每一个数字定标尺：单票敢下多重、总仓位留多少现金、弱市该不该减。这些标尺全都写在答案里。跳过它，系统就只能猜；而“猜”在这个领域里的另一个名字叫“编”。

26.4 十二个维度，量的到底是什么

系统把这 12 题做成一份选择题问卷，参考的是国内私募基金募集时的风险测评问卷。每个维度在代码里都有稳定的字段名，不是随手写的。

表 26.1 画像问卷的十二个维度及其对方案的影响

维度（题干）	字段名	怎么影响后面的数字
证券投资经验	investment_experience	新手会把激进方案自动降一档
股票型私募产品经验	product_experience	反映对权益类产品的熟悉度
投资决策依据	decision_basis	记录你信哪一类证据
风险承受能力	risk_tolerance	决定风险等级（R1/R2→ 保守，R3→ 均衡，R4/R5→ 成长）
亏损应对态度	loss_attitude	“立即止损”会把激进方案降一档
可接受最大亏损	max_drawdown	决定任务书里的回撤上限
计划投资期限	investment_horizon	短期资金会压制激进方案
主要投资目标	investment_objective	决定策略类型与收益预期
流动性需求	liquidity_requirement	决定现金缓冲与再平衡频率
计划投资金额	capital	决定候选只数与单标的预算
投资风格偏好	investment_style_preference	决定因子权重与偏好行业
行业配置偏好	industry_preference	决定行业倾斜

题干文字取自前端问卷配置；字段名是系统内部键。风险等级与策略类型的映射见

`internal/plannerhost/planner.go` 的 `buildPlanFromProfile`。

数据来源：前端问卷配置 `ui/src/pages/InvestmentPlanner.tsx` 的 `CHOICE_QUESTIONS`；系统事实清单第 12 节。
核对时间 2026-09-24。

我第一遍读这张表，以为最要紧的是“风险承受”。读完源码才发现，真正决定“买几只”的，是最不起眼的那一栏——计划投资金额。

26.4.1 问卷的答案，先变成一份投资风格

填完 12 题，系统先算出一份投资风格（`InvestmentStyle`）：风险等级、预期收益、最大回撤、夏普比率，以及资产配置和行业配置两块饼图。以三档为例：

这里有一条我照实写下来的规则：风格不是选完就定的，它还会被别的答案往下拽。下手最狠的是这几条——新手会把激进降成均衡，均衡再降成保守；短期资金、保值目标、资金不足 5 万，都会把激进方案降一档¹。

方向永远是只降不升。这一点和后面归一化要“只收不摊”是同一种性格：宁可保守一点，绝不自作主张加码。

¹数据来源：前端 `ui/src/pages/InvestmentPlanner.tsx` 的 `generateStyle: NOVICE / SHORT / PRESERVATION / capital < 50000` 的降级分支。核对时间 2026-09-24。

表 26.2 三档风格的预期收益、回撤与资产配置

风格	风险等级	预期收益	最大回撤	资产配置
保守（C1/C2）	保守型	7%–9%	10%–16%	红利价值股 70% / 沪深 300 20% / 现金 10%
均衡（C3）	平衡型	8%–12%	15% 以内	量化多头股 55% / 中证 500 25% / 现金 20%
成长（C4/C5）	进取型	12%–18%	25%–30%	成长龙头股 70% / 科创创业指 20% / 现金 10%

这是前端给出的“风格预期”；后端引擎（`buildPlanFromProfile`）另有一套年化参数：保守 4.5%/3.5%，均衡 9.5%/11.5%，成长 13.5%/20%（收益/波动）。两处口径不同，涉及真实收益预期以后端引擎值为准。
数据来源：前端 `ui/src/pages/InvestmentPlanner.tsx` 的 `styleMap` 与 `generateStyle`；后端 `internal/plannerhost/planner.go` 的 `buildPlanFromProfile`。核对时间 2026-09-24。

26.4.2 同一份画像，三套候选方案

画像定完，系统不是直接给一个组合，而是给三套候选：A 红利价值、B 量化多因子、C 成长龙头。它们对应三档风险，让你自己挑，而不是替你做主。

表 26.3 A/B/C 三套候选策略的核心指标

标签	名称	预期收益	波动	最大回撤	夏普
A	红利价值多头策略	9.0%	13%	16%	0.85
B	量化多因子多头策略	9.5%	11.5%	15%	0.95
C	成长龙头多头策略	14.5%	22%	28%	0.82

三套候选的因子画像直接取自选股引擎的同一个策略模板（`defensive/balanced/growth`），因此规划页与选股页不会各说各话。若风险等级为保守，C 会被标为 `REJECTED`、风险分降到 0.30。
数据来源：`internal/plannerhost/planner.go` 的 `generateCandidates`；核对时间 2026-09-24。

我最初以为 A/B/C 只是“三档收益、让你挑个刺激的”。看完代码才知道，它们的区别是三套不同的因子配方：A 压在红利和低波上，B 是质量 + 价值 + 动量 + 低波四因子，C 偏成长和动量。收益预期不同，是因为它们赌的东西不同。

方案不是许愿池。你以为在挑“我想要多少收益”，其实在挑“我愿意承担哪一类波动”。这句话后面还会再讲一次。

26.5 从候选到股票池：选股引擎接棒

三套候选是“战略”，接下来要落到“战术”——具体买哪几只股票。

系统不在这里硬编码股票。它把风险等级翻译成选股模板（保守 → `defensive`，均衡 → `balanced`，成长 → `growth`），然后调用选股引擎跑全市场真实因子打分，最多取 200 只候选²。

这里的“真实”两个字是硬约束：选股引擎先校验行情数据库是否挂载，没挂载就直接报错“行情数据库（`stock.duckdb`）未挂载”，不返回假数据³。这也是 chapter 二十五 里“先把米下锅”的原因——数据没备好，这一整章都跑不起来。

选出来的结果会落到可交易股票池（`tradeable_stocks`）。这张表有两个身份：

²数据来源：`internal/plannerhost/planner.go` 的 `buildRealStockPool: ScreeningRequest\{StrategyID, Market:"all", MaxResults:200, MinScore:0, SmallCapital, InvestorProfile, DiversifySeed\}`。核对时间 2026-09-24。

³数据来源：系统事实清单第 9 节：`ScreenStock(req)` 先校验 `DuckDB`、再校验 `HasStockDB()`，未挂载直接报错。核对时间 2026-09-24。

- 它是投资方案的股票池落点；
- 它同时是 CIO 决策买入的白名单来源——不在池里的股票，禁止生成买入订单；池空或读取失败，则整体拒绝自主建仓（fail-closed，细节见 chapter 十）。

这就是那道“第一道防线”：先有资格，再谈买多少。

顺带一层：同一个风险等级、同一个资金规模，两个人拿到的股票池也不该完全一样。系统把用户 ID 哈希成一个确定性种子（`userDiversifySeed`）对权重做微扰，并把整份画像透传给选股引擎，让风险承受、风格、期限真实影响因子权重⁴。“千人一面”在这里是不被接受的。

26.5.1 买几只，由本金决定

真正决定“买几只”的，还是钱。

表 26.4 本金区间与候选只数、单标的预算

本金区间	候选只数	单标的预算	一句话
3 万以下	4	约 25%	钱少，只能集中
3–20 万	5	约 20%	最常见的档（默认 10 万落在这一档）
20–50 万	8	约 12%	开始谈分散
50 万以上	10	约 10%	分散到位

这是“候选名额”与“单标的预算”，不是最终权重；最终权重还要过单票上限收口（见定理 theorem 26.4）。

数据来源：`internal/plannerhost/planner.go` 的 `portfolioSizing`。核对时间 2026-09-24。

默认初始资金 10 万元落在“3–20 万”这一档，所以候选是 5 只。这就是投资方案页上“建议持仓”表里那一排股票的由来。

26.6 单票上限：保守 15%，均衡 20%，成长 30%

有了候选和权重，还有一个问题：一只股票最多能占多少？系统按风险等级分三档：

表 26.5 各风险等级的单票上限与基础现金缓冲

风险等级	单票上限 u	基础现金	说明
保守 conservative	15%	10%	回撤容忍低，不许重仓单票
均衡 balanced	20%	8%	默认档
成长 growth	30%	5%	与全局风控单票上限 30% 对齐

上限写在投资任务书（Mandate）的 `PositionLimit.MaxSingleStock` 里，随方案一起落库；行业上限也按等级给（保守 25%，均衡 25%–35%，成长 40%）。

数据来源：`internal/plannerhost/planner.go` 的 `buildMandateFromProfile`；系统事实清单第 12 节。核对时间 2026-09-24。

为什么不一刀切？因为上限要跟“你能扛多大回撤”对齐。一个接受了 28% 回撤的人，把三成押在一只票上，是他的选择；一个只接受 5% 回撤的人这么做，一次次个股黑天鹅就足以打穿整个组合。上限不是收益旋钮，是回撤旋钮。而成长档的 30% 正好等于系统全局风控的单票硬限制——规划层可以贴天花板，但绝不能捅破它。

⁴数据来源：`internal/plannerhost/planner.go` 的 `userDiversifySeed` 与 `ScreeningRequest.DiversifySeed`。核对时间 2026-09-24。

26.7 小额成长方案：弱市也要把钱用起来

这里有一个我一开始想不通、后来觉得挺妙的设计。

成长型、本金不到 50 万的小账户，如果碰上弱市，按常理会怎么做？提高现金、减少持仓、保守再保守。系统确实提高了现金——但它做了一个反向的补充动作：

- **候选保底**：因子分线过严导致候选不足时，按 0.05 一档往下放宽，直到凑够至少 3 只，最低放宽到 0.50 为止；极端情况下（行情数据不足）取评分最高者，保证股票池非空⁵。
- **现金约 10%**：弱市里普通成长方案的现金会从 5% 抬到 20%（×4，最高 50%）；但**成长型 + 本金 < 50 万**这种情况被单独保护，只抬到约 10%⁶。

为什么？因为小账户的钱本来就少，再大面积闲置，收益会被现金拖死。小资金最怕的不是波动，是闲置。所以弱市里它宁可留住几只候选、把仓位用起来，但现金仍保留一成作为安全垫。弱市的判定只看市场状态机的 Regime 是否落在 bear 或 bear_range⁷。

一句提醒：这是给小额成长账户的专门安排，不是通用规则。你把风险等级改成保守，现金立刻回到 10% 起步、弱市还要往上抬——这正是“一个答案改了，持仓就变了”的又一个实例。

26.8 归一化之后，必须再收口一次

现在到了本章唯一需要动笔算的地方。前面说“候选名额 5 只、单标的预算 20%”，你可能以为权重就是每只 20%，等权。不是。系统给权重的方式是按因子分差异化分配，分高的权重更大。这就带来一个必须处理的陷阱。

26.8.1 先说一个“差不多对”的模型

最直觉的做法是归一化：给每只候选一个原始分数 $\tilde{w}_i$ ，然后让它们按比例分掉“股票那一部分预算”。占比就是

$$w_i = \frac{\tilde{w}_i}{\sum_{j \in S} \tilde{w}_j}, \quad (26.1)$$

这样 $\sum_i w_i = 1$ ，看起来干净利落。

26.8.2 这个模型在哪里失效

失效的地方是：候选太少的时候，归一化会自动“加杠杆”。

假设某天全市场只筛出 2 只够格的候选（弱市里很常见）。归一化不管你只有 2 只——它照样把 100% 的预算摊到这 2 只身上。如果其中一只分数远高于另一只，它可能被分到 90%。

90% 的单票权重，对一个单票上限 20% 的组合意味着什么？意味着风控那条线被整个跨过

⁵数据来源：internal/plannerhost/planner.go 的 buildCandidatePool：阈值兜底循环 len(scoredList) < 3 && minScore >= 0.50。核对时间 2026-09-24。

⁶数据来源：internal/plannerhost/planner.go 的 applyThreeLayerWeighting：if riskLevel == "growth" && totalAssets < 500000 \{ boostedCash = 0.10 \}。核对时间 2026-09-24。

⁷数据来源：internal/plannerhost/planner.go 的 bearish()。核对时间 2026-09-24。

去了。归一化自己没有做错任何一步，它只是忠实地执行了“总和等于 1”；是“候选太少”这个前提，让“总和等于 1”变成了“单票被迫超限”。

所以必须补第二步：收口（clamp），并把收口挤出来的那部分钱转进现金。

26.8.3 把两步写成算法，再证它对

假设 26.1 (候选与上限的合法取值). 候选集合 S 非空，原始权重满足 $\tilde{w}_i > 0$ ($i \in S$)；单票上限 $u \in (0, 1]$ ，总仓位上限 $\Theta \in (0, 1]$ 。风险等级给出的 u 只取三个值之一：0.15（保守）、0.20（均衡）、0.30（成长）。

定义 26.2 (归一化—收口—溢出转现金). 给定 $\tilde{w}$ 、 u 、 Θ ，定义两步算法：

$$w_i = \frac{\tilde{w}_i}{\sum_{j \in S} \tilde{w}_j} \quad (\text{第一步：归一化}) \quad (26.2)$$

$$w_i^* = \min(w_i, u) \quad (\text{第二步：收口}) \quad (26.3)$$

剩余量 $\rho = 1 - \sum_{i \in S} w_i^*$ 全部转为现金权重 $w_{\text{cash}} = \rho$ 。

这个定义在说什么：第一步让所有股票按比例分蛋糕；第二步给每只股票加一个“最多只能拿 u ”的盘子盖，盖剩下漏出来的，全部扫进现金。

引理 26.3 (归一化的单调性). 固定除 i 以外的所有 $\tilde{w}_j$ ，把 w_i 看成 $\tilde{w}_i$ 的函数 $\varphi(x) = x/(x+C)$ ，其中 $C = \sum_{j \neq i} \tilde{w}_j \geq 0$ 。则 φ 在 $(0, \infty)$ 上非减；当 $C > 0$ 时严格增。

证明. 若 $C = 0$ （即 $|S| = 1$ ）， $\varphi(x) = x/x = 1$ 为常数，非减性显然。

若 $C > 0$ ，求导：

$$\varphi'(x) = \frac{(x+C) - x}{(x+C)^2} = \frac{C}{(x+C)^2} > 0. \quad (26.4)$$

导数在 $(0, \infty)$ 上处处为正，故 φ 严格增，从而非减。 ■

定理 26.4 (归一化—收口的可行性、单调性、保序性与加杠杆机制). 在假设 26.1 与定义 26.2 下：

(a) (可行与有界) $0 \leq w_i^* \leq u$ 对一切 $i \in S$ 成立； $\sum_{i \in S} w_i^* \leq 1$ ； $w_{\text{cash}} = \rho \geq 0$ ；且权重之和恒为 1：

$$\sum_{i \in S} w_i^* + w_{\text{cash}} = 1. \quad (26.5)$$

(b) (单调性与保序) 固定其余 $\tilde{w}_j$ ， w_i^* 关于 $\tilde{w}_i$ 非减；且若两只票都未触上限，则它们的相对次序与原始权重一致： $\tilde{w}_i \geq \tilde{w}_j$ 且 $w_i^* < u$ ， $w_j^* < u \implies w_i^* \geq w_j^*$ （把 $\geq$ 换成 $>$ 也成立）。

(c) (候选过少必生正现金) 若 $|S| \cdot u < 1$ ，则 $\rho > 0$ 。并且，若跳过第二步只做归一化，单票权重可能突破上限：存在反例使 $\max_i w_i > u$ 。

(d) (叠加总仓位上限) 再令 $w_i^{**} = \Theta w_i^*$ 、 $w_{\text{cash}}^{**} = 1 - \Theta \sum_i w_i^*$ 。则 $0 \leq w_i^{**} \leq u$ ， $\sum_i w_i^{**} \leq \Theta$ ，且 $\sum_i w_i^{**} + w_{\text{cash}}^{**} = 1$ 。

这个定理在说什么：这套“先归一化、再收口、溢出转现金”的两步法，绝不会让任何一只票超过上限，也不会让权重和跑出 100%；分数高的票在没被盖住时仍然排在前面；候选越少，被挤出来的现金越多；如果还要额外压低总仓位，把所有股票权重同乘一个系数就行，上限和预算仍然双双成立。

证明. (a) 由假设 26.1, $\tilde{w}_i > 0$ 且分母 $\sum_j \tilde{w}_j > 0$, 故 $w_i > 0$; 又 $u > 0$, 于是

$$w_i^* = \min(w_i, u) \geq 0, \quad w_i^* = \min(w_i, u) \leq u. \quad (26.6)$$

这是“min 不大于任一被比较者”的直接结论。

再证 $\sum_i w_i^* \leq 1$ 。逐项 $w_i^* = \min(w_i, u) \leq w_i$, 求和并用 eq. (26.1):

$$\sum_{i \in S} w_i^* \leq \sum_{i \in S} w_i = 1. \quad (26.7)$$

于是 $\rho = 1 - \sum_i w_i^* \geq 0$, $w_{\text{cash}} = \rho \geq 0$ 。最后

$$\sum_{i \in S} w_i^* + w_{\text{cash}} = \sum_{i \in S} w_i^* + \left(1 - \sum_{i \in S} w_i^*\right) = 1, \quad (26.8)$$

即 eq. (26.5)。

(b) (单调性) 由 eq. (26.2), $w_i = \varphi(\tilde{w}_i)$, φ 由引理 26.3 给出。 $\tilde{w}_i$ 增大时 $\varphi(\tilde{w}_i)$ 非减; 而 u 是常数, 映射 $y \mapsto \min(y, u)$ 是非减的 (对任意 $y_1 \leq y_2$ 有 $\min(y_1, u) \leq \min(y_2, u)$)。两个非减函数复合仍非减, 故 $w_i^* = \min(\varphi(\tilde{w}_i), u)$ 关于 $\tilde{w}_i$ 非减。

(b) (保序) 两只票共用同一个分母 $\sum_j \tilde{w}_j$ 。若 $\tilde{w}_i \geq \tilde{w}_j$, 则

$$w_i = \frac{\tilde{w}_i}{\sum_j \tilde{w}_j} \geq \frac{\tilde{w}_j}{\sum_j \tilde{w}_j} = w_j. \quad (26.9)$$

又题设 $w_i^* < u$ 且 $w_j^* < u$, 说明两只都没有被 min 截断, 于是 $w_i^* = w_i$ 、 $w_j^* = w_j$, 代入上式即得 $w_i^* \geq w_j^*$ 。

严格版: 若 $\tilde{w}_i > \tilde{w}_j$, 上面第一个不等号取严格 (分母为正), 得 $w_i > w_j$; 再因两者未被截断, $w_i^* > w_j^*$ 。

(c) 逐项 $w_i^* \leq u$, 故

$$\sum_{i \in S} w_i^* \leq \sum_{i \in S} u = |S|u < 1, \quad (26.10)$$

于是 $\rho = 1 - \sum_i w_i^* \geq 1 - |S|u > 0$ 。这就证明了“候选太少必然留下正现金”, 也就是那句“买不满”的机制: 盖在每只票上的盘子再多, 也盖不出 100% 的股票仓位。

反例 (只归一化不收口会突破上限): 取 $|S| = 2$ 、 $\tilde{w} = (9, 1)$ 、 $u = 0.20$ 。归一化得 $w = (0.9, 0.1)$, 于是

$$\max_i w_i = 0.9 > 0.20 = u, \quad (26.11)$$

单票权重高出上限四倍多。注意此时 $|S|u = 0.4 < 1$ ，正好落在 (c) 的前提里——若老老实实做收口，得到 $w^* = (0.2, 0.1)$ 、 $\sum_i w_i^* = 0.3$ 、 $\rho = 0.7$ ：现金七成，权重和仍为 1，且没有一只越界。两步差一步，结论天翻地覆。

(d) 因 $\Theta \leq 1$ 且 $w_i^* \leq u$ ，

$$w_i^{**} = \Theta w_i^* \leq w_i^* \leq u, \quad (26.12)$$

单票上限满足；又 $w_i^* \geq 0$ 、 $\Theta > 0$ ，故 $w_i^{**} \geq 0$ 。

总仓位（权益部分）：

$$\sum_{i \in S} w_i^{**} = \Theta \sum_{i \in S} w_i^* \leq \Theta \cdot 1 = \Theta. \quad (26.13)$$

现金权重 $w_{\text{cash}}^{**} = 1 - \Theta \sum_i w_i^*$ ，由 (a) 中 $\sum_i w_i^* \leq 1$ 得 $w_{\text{cash}}^{**} \geq 1 - \Theta \geq 0$ 。最后权重和：

$$\sum_{i \in S} w_i^{**} + w_{\text{cash}}^{**} = \Theta \sum_{i \in S} w_i^* + \left(1 - \Theta \sum_{i \in S} w_i^*\right) = 1. \quad (26.14)$$

■

■

26.8.4 定理 26.4 在代码里长什么样

系统里 Θ 就是“1 减去现金比例”。代码先算出所有持仓的权重之和 `totalWeight`，再取缩放系数

$$\text{scale} = \frac{\Theta}{\text{totalWeight}}, \quad \Theta = 1 - \text{cashRatio}, \quad (26.15)$$

逐只相乘，然后再一次按单票上限收口，把超额累加成 `overflow` 并并入现金——这正是定理 theorem 26.4 的 (a)(c)(d) 三部分的落地⁸。

源码注释里留着一次真实翻车的痕迹：“该计划曾因此生成 40% 单票”——归一化把已经收口好的权重又放大回去了。修法就是“缩放后再次收口、溢出转现金”。这条注释是本章所有数学的现实版本。

26.9 方案每天会自动换股

最后讲一件容易被忽略、但每个交易日都会发生的事：你的方案不是一次性快照，它每天会自动重建一次。

链路是这样的：

```
1 // 调度器（复盘阶段，幂等：同一交易日只跑一次）
2 PlanRefreshProcessor.RebuildInvestmentPlan(ctx)
3   -> App.RebuildInvestmentPlan(ctx)           // app_review.go
4   -> planner.RebuildCurrentPlanPortfolio()    // plannerhost/planner.go
5   // 用最新全市场真实选股重建 construction 并落库
```

⁸数据来源：internal/plannerhost/planner.go 的 `applyThreeLayerWeighting: scale := (1 - cashRatio) / totalWeight`，随后 `if tw > MaxSingleStock { overflow += tw - MaxSingleStock; tw = MaxSingleStock }`，最后 `cashRatio += overflow`。核对时间 2026-09-24。

代码 26.1 每日自动重建方案的调用链（节选）

这段代码在说什么：不是“每天给你造一份新方案”，而是“把你现有那份方案里那张持仓表，用当天最新的全市场选股重新算一遍，再覆盖回去”。任务书、风险等级、方案状态都不动。

幂等由两件事保证：内存里的 `lastPlanRefreshDate` 记着“今天重建过没有”，同一个交易日直接跳过；而这个标记还会持久化到 `config/config.json` 的 `last_plan_refresh_date`⁹。所以程序重启不会重跑——这条是真实事故换来的：幂等标记原来只放在内存里，重启后就又跑一遍。

重建时还会顺手刷新一次可交易股票池，让“方案持仓”和“白名单”保持一致¹⁰。

至于这套每日重建怎么和复盘、改权重、换策略连成“越用越聪明”的闭环，那是 chapter 三十二 的内容。

26.9.1 一个必须分清的细节：两个动作，两种行为

这里有一个真实的产品细节，我照实写，因为它极容易让人误解。

投资方案页上那张“AI 投资组合（建议持仓）”表里的股票，来自 `PlanJSON.construction.positions`——这是创建方案时算好、随后写进 JSON 的一份快照。前端只把它反序列化出来展示，自己不算权重。

页面上另有一个「重新生成股票池」按钮，点它会走 `GeneratePlanStockPool`：重新跑一次选股、写 `tradeable_stocks`，并把结果计数记进 `PlanJSON.stockPool` 的元数据¹¹。但它不重算 `construction.positions`。所以：

- 想要真的换股——等每日自动重建（复盘阶段自动跑）；
- 点了「重新生成股票池」——只是刷新那条“股票池”卡片上的元数据计数，持仓表不会跟着变。

这不是 bug，是两个动作的分工不同。分清它们，你就不会对着“我明明重新生成了股票池，怎么持仓没变”发呆。

一份方案生成出来之后，在界面上长这样：

⁹数据来源：internal/scheduler/auto_scheduler.go 的 rebuildInvestmentPlanDay 与 persistSchedulerState。核对时间 2026-09-24。

¹⁰数据来源：app_review.go 的 App.RebuildInvestmentPlan：重建后调用 generatePlanStockPool，失败只记日志、不影响持仓更新。核对时间 2026-09-24。

¹¹数据来源：app_planner.go 的 GeneratePlanStockPool 与 generatePlanStockPool。核对时间 2026-09-24。



图 26.1 投资方案详情：风险等级与四项指标、投资任务书，以及资产配置后的具体持仓权重

26.10 常见误解与纠偏

常见误解·方案是许愿池，我想要多少收益就选哪套 这是最容易踩的一脚。“C 成长龙头预期收益 14.5%，A 只有 9%，那当然选 C。”

那张表左右两边是绑在一起的：C 的 14.5% 对应 22% 的年化波动和 28% 的最大回撤^a。你不是“选了高收益”，你是“选了高波动”——收益是它的结果，波动是它的代价，两者不可拆卖。

系统的做法比“提醒你一句”更硬：如果你风险承受是 R3（均衡），C 那套会被标注为不匹配；如果你保守，C 的 RiskOSStatus 会变成 REJECTED、风险分掉到 0.30^b。它不禁止你看，但它拒绝替你背书。

^a数据来源：见 table 26.3 及其出处。核对时间 2026-09-24。

^b数据来源：internal/plannerhost/planner.go 的 generateCandidates 末尾：if riskLevel == "conservative" {\ candidates[2].RiskScore = 0.30; candidates[2].RiskOSStatus = "REJECTED" \}。核对时间 2026-09-24。

常见误解·归一化就是“等比缩放”，做完就完事 “总权重归一到 1”听起来是个纯数学动作，做完权重自然就对了。

错在“候选数量”这个前提上。归一化默认了“候选足够多、每只都分不到多少”，一旦候选只有两三只（弱市里天天发生），它就会把单票权重放大到吓人的水平——定理 theorem 26.4(c) 的反例里，两只票就让第一只到了 90%。

所以系统的第二步不是可选项：收口 + 溢出转现金。少了这一步，你的组合会在纸面上“合法地”违反单票上限；风控那边一旦对齐口径，订单就会被拒，或者更糟——先建了仓才发现超限。

一句话记住：归一化保证“加起来是 100%”，收口保证“每一只都不越线”，这两件事必须一起做。

26.11 真实出处

本章的真实出处

- 12 题问卷的题干与选项：ui/src/pages/InvestmentPlanner.tsx 的 CHOICE_QUESTIONS
- 问卷答案落库与完整度计算：internal/plannerhost/planner.go 的 SaveProfileAnswers、calculateCompleteness
- 风险等级/收益/波动/回撤推导：同文件的 buildPlanFromProfile
- 投资任务书（单票上限、行业上限、现金区间、再平衡政策）：同文件的 buildMandateFromProfile
- 组合构建（候选池、三层权重、归一化收口、溢出转现金、现金仓位）：同文件的 buildPortfolioConstruction 与 applyThreeLayerWeighting
- 真实全市场选股与画像透传：同文件的 buildRealStockPool；选股引擎 internal/screener/screener_service.go
- A/B/C 三套候选：同文件的 generateCandidates
- 每日自动重建：internal/plannerhost/planner.go 的 RebuildCurrentPlanPortfolio；app_review.go 的 App.RebuildInvestmentPlan；internal/scheduler/auto_scheduler.go 的 PlanRefreshProcessor、rebuildInvestmentPlanDay、persistSchedulerState
- 股票池白名单与 fail-closed：internal/cio/cio.go；策略引擎 internal/policy/engine.go 的硬限制与暂停建仓开关（详见 chapter 十）
- 数据表：tradeable_stocks（可交易池）、investment_plans（投资方案）；幂等标记 last_plan_refresh_date 持久化于 config/config.json

动手试一试（半小时）

1. 打开「投资规划」，把 12 题如实填完，记下 A/B/C 三套候选的预期收益与最大回撤；然后只改一题——把风格偏好从“成长投资”改成“红利价值”——再看一遍：B 那套的名称与收益预期会跟着变。
2. 把“风险承受能力”从 R4（进取）改成 R3（均衡），并把“可接受最大亏损”改到 15% 以内。对照 table 26.5：你的单票上限应当从成长档的 30% 掉到均衡档的 20%——去看方案详情里任意一只股票的权重，确认没有超过 20%。
3. 点一次「生成股票池」，然后到「我的投资 / 投资方案」页确认：股票池卡片上出现了策略模板（防御型/均衡/成长型）、提交只数、生成时间这些元数据。
4. 下一个交易日复盘时段之后，再打开同一份方案，比较持仓表里的股票代码：变了，说明每日自动重建真的跑了；没变，去 config/config.json 看 last_plan_refresh_date 是否已等于今天。
5. 拿一张纸手算定理 theorem 26.4(c) 的反例：取 $\tilde{w} = (9, 1)$ 、 $u = 0.20$ 。先只归一化算 $\max_i w_i$ ，再按两步法算 w^* 与现金 ρ 。两次结果一对照，“自动加杠杆”就变成了你自己算出来的数。

到这里，问卷到持仓的整条链你走过一遍了：12 个答案定标尺，画像定风格，三套候选定战略，选股引擎出股票池，权重经过“归一化 + 收口”落到每一只票，第二天系统自动再算一遍。

可是方案生成完，屏幕上会冒出一堆数字：总资产、今日盈亏、累计收益、持仓市值、现金、CIO 状态、风险监控。先看哪个？哪个能信、哪个只是过程量？下一章，我们站到仪表盘前面去，把 chapter 二十七 的那三块屏幕逐个读懂。

第二十七章

读懂仪表盘：控制中心、AI 团队与实时活动

27.1 屏幕上全是数字，先看哪一个

第 chapter 二十六 章结束时，系统里已经有一份属于你的投资方案和一个股票池。第二天我打开程序，落在了控制中心。

我数了一下那一屏：五张指标卡、一句 CIO 今日状态、四张小头像、三个风险数字、一张持仓表、一列决策卡片。十几个数字同时在动。

我当时的做法很没出息：我盯着“今日盈亏”看。因为它是红的，因为它最像“成绩单”，因为我唯一看得懂它。

接下来的半小时里我做了三件蠢事。第一件，我看见 AI 团队页里操盘手那张卡写着“待命”，立刻以为它挂了。第二件，我看见实时活动页整页写着“休市中，等待下一交易日”，更确信系统坏了。第三件，我盯着那个不动的“今日盈亏”来回刷新，想让它动起来。

三件事的根因是同一个：我默认这三个页面是同一份数字的三个视图。它们不是。它们回答三个完全不同的问题，而且刷新节奏不同——所以同一时刻，三个页面上的数字本来就可以对不上。

本章要做的，就是把这三个页面按“先看什么”的顺序摆清楚。

27.2 先看故障灯，再看速度，最后看油量

直觉·驾驶座上的目光顺序 坐进车里的第一眼，我看的是仪表盘上那一排故障灯：机油、水温、电瓶、刹车。灯亮了，速度表和油表都别看了——先靠边停车。

灯全灭，我再看速度：车在动吗，动得快不快，有没有超速。

最后才看油量：还能跑多远，什么时候该进加油站。

这个顺序不是习惯，是被逼出来的逻辑。故障灯回答的是“还能不能继续”，它决定后面两类数字还有没有意义；速度回答的是“正在发生什么”；油量回答的是“存量还剩多少”。

存量类的数字永远最后看。因为它变化最慢，也最不紧急——它是前两者长期累积出来的结果，你盯着它看，它也不会因为你看得快而涨得快。

三个页面正好对上这三层。先看图 27.1。

“故障灯层”在界面上长得不像灯：它是控制中心顶栏那个绿色“正常”圆点（或黄色“停止”）、风险监控面板最后那行“风控规则正常”、AI 团队卡片上的红色“异常”徽章、实时活动右上那个红色“执行器不可用”。它们散在三个页面里，但回答的是同一件事：这套系统现在还能不能干活。

所以真正的操作顺序是：先把三个页面各自扫一眼有没有红黄的东西，确认绿灯之后再谈收益。这一条听起来像废话，可在真实故事里，绝大多数“亏钱”都发生在用户还没看故障灯的时候。



图 27.1 看板信息的三层结构。每一层回答一个不同的问题，而且只有上层的答案是“好”的时候，下层的数字才值得读。

27.3 三个页面，三个问题

27.3.1 控制中心：钱还剩多少

控制中心是默认页。它在 `ui/src/App.tsx` 的路由 `switch` 里对应那个 `default` 分支——也就是说，地址栏里的 `hash` 是空的、或者写了任何页面都不认识的值，落到的都是它。

它的第一行是五张卡片，从左到右：

1. **总资产**（存量总量）；
2. **今日盈亏**（带百分比，当日增量）；
3. **累计收益**（把当日视角拉到全周期）；
4. **持仓市值**（钱有多少在股票里）；
5. **持仓/现金**（“N 只”加一行“现金 ...”，钱有多少在账上）。

这个顺序是“从结果到构成”：前两张是我最想知道的答案，第三张告诉我这个答案在过去一段时间里是什么形状，最后两张回答“它是由什么拼出来的”。如果我把第四、五张挪到第一张的位置，我会先看到一个几千几万的数字，却不知道它是赚来的还是亏出来的。

这五个数字的具体取值，本书不引用。它们由你按下刷新键那一刻数据库里的快照决定，本书不是某一天的运行记录——按本书的数据规范，这里留白¹。

第二行两块面板。左边是 **CIO 今日状态**：一句话（源码里就是 `cioStatus.todayMessage`），加四张小卡。

这里有个我照实写下来的细节：四张小卡是 **CIO**、**QUANT**、**RISK**、**TRADER**，没有 **PLANNER**²。这是排版取舍：投资规划师的活是“每天一次”的目标与方案维护，不是盘中状态；控制中心这块要的是“此刻谁在盯盘”。五个角色是完整的组织，四个头像只是“值班表”。

右边是 **风险监控**：持仓集中度（取所有持仓权重的绝对值最大值，超过 30 标成黄色——

¹数据缺失：该系统库中此项无有效记录。本书遵照系统“数据不可用即报错、不降级、不伪造”的原则，此处留白、不填充估计值。

²数据来源：系统源码 `ui/src/pages/Dashboard.tsx` 的 `teamStatus` 数组（4 条记录），以及第 chapter 二 章讨论过的五个角色定义 `internal/agentworkflow/models.go`。核对时间 2026-09-24。

与硬限制里的单票 30% 是同一个口径)、最大单票名字、未实现盈亏,最后一行是风控规则状态。它属于第一层:这行字一变黄变红,上面的收益数字就先别看了。

第三行是持仓分布表和 AI 今日决策列表。持仓表每页 5 条(源码里 HOLDINGS_PAGE_SIZE = 5),三列:股票名称(代码)、持仓量、盈亏。分的不是“重要不重要”,是“屏幕高度有限”:持仓数按资金规模动态调整,5 万资金推荐 5 只,钱多了就不止 5 只。

AI 今日决策读的是 GetCIOJournal(7)——最近 7 天的 CIO 决策日志,屏上最多展示 5 条。每条上有三样东西值得看:决策类型的中文标签(建仓、加仓、减仓、调仓、观望、持有、买入、卖出、暂停)、风控审批的那三个符号(通过 / 否决 / 审核中)、以及理由那一行字。第 chapter 二十八 章会把这一行字逐块拆开,这里只需要知道它在这个页面上是“摘要版”。

最后是刷新。控制中心的自动刷新闻隔是 5 分钟³。而且它不是单纯重读一遍数字,每一轮先调 RefreshPrices 拉一次最新价,再读组合状态、再读决策日志。这一句很重要,本章第四节会用它解释“为什么这个页面不敢刷新得太快”。

控制中心还有简洁/详细两种显示模式(右上角那个“简洁/详细”按钮切换),简洁模式只留总资产、今日盈亏、累计收益加一行状态。它是给“我每天只看一眼”的那类用户留的。

本节起,界面部分的说明都配一张实拍截图。这些截图取自系统真实运行时的界面(深色主题,原始分辨率约 2.6K),未作美化、未作裁剪;图中出现的股票代码与金额,就是那一刻数据库里的真实记录。

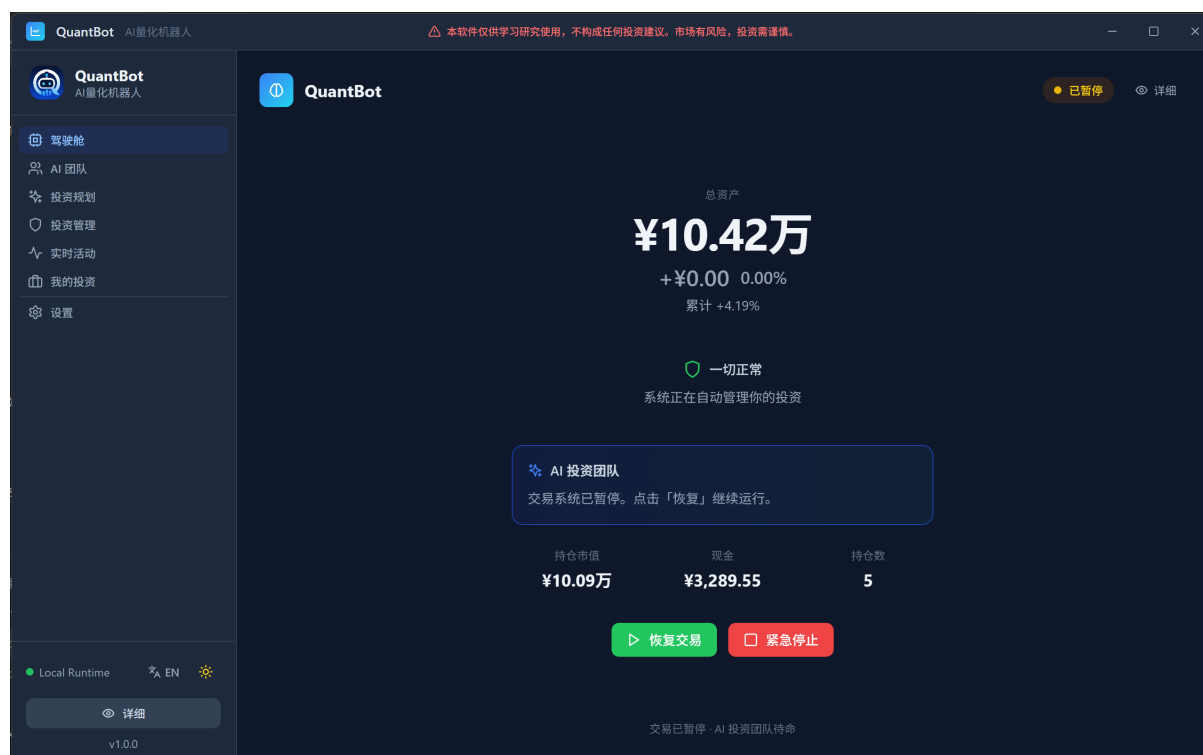


图 27.2 控制中心(驾驶舱):顶部五张卡片、CIO 今日状态、风险监控、持仓分布与 AI 今日决策

³数据来源:系统源码 ui/src/pages/Dashboard.tsx: 轮询函数末尾 timer = window.setTimeout(poll, 300000), 即 300 000 ms = 5 分钟; 且是“顺序轮询”——本次刷新跑完才安排下一次。核对时间 2026-09-24。

27.3.2 AI 团队：谁在干活、谁有权干什么

AI 团队的地址是 `/#ai-team`。它问的问题是：“此刻这五个智能体在干什么，而且它们各自被允许干什么。”

第一屏最上面不是卡片，是一行时段指示：`phaseLabels[currentPhase]`，也就是“上午交易 (09:30-11:30)”这类中文时段名，右边跟着当前时间。为什么把时段放在第一屏？因为这个页面上几乎所有的“状态”都是时段推导出来的，不是查出来的。

下面那五张智能体卡片，每张有三样：一个三色状态标签（正常 = 绿、工作中 = 黄、异常 = 红）、一句“当前动作”、一个任务状态徽章（执行中 / 已完成 / 有错误 / 等待中，后面跟“N 项”）。

状态是怎么来的，值得一句一句说清楚，因为“AI 在休息”这个误解就是从这儿长出来的：

- 开盘前（准备段）：一排“盘前准备，等待开盘”；
- 盘前竞价段：一排“盘前准备中”；
- 上午、下午交易段：按分工分道——操盘手“执行交易中”，风控师“盘中风险监控”，首席投资官“决策评估中”，量化分析师“等待交易信号”，规划师“监控市场变化”；
- 午休：整排“午休中”（源码里的状态常量是 `RESTING`）；
- 盘后：一排“盘后确认交易中”；
- 复盘段：17 点之后（或 16 点之前）显示“今日任务已完成”，首席投资官换成“今日投资报告已生成”；16 点到 17 点之间显示“执行盘后任务中”；
- 休市：一排“休市中，等待下一交易日”。

也就是说，“休息”和“完成”是这个系统设计好的状态，不是故障。它直接呼应第 chapter 五章那张市场时段表和“每三十秒醒一次”的调度循环。

再往下是 **任务规格清单**：五个角色各一行，点开有左“可以做”、右“不能做”两张清单，下面是任务 ID 清单。清单里的条目是真实的：五个角色合计 17 项任务 ID（规划师 2 项、量化 4 项、首席投资官 4 项、风控 4 项、操盘手 3 项）⁴。同一份代码里，注释说 16，清单是 17——我没改它，因为这类“注释漂移”正是第 chapter 二章讲过的同一件事：判断权限和职责，要看表里写了什么，不要看注释写了什么。

这一页的第三样东西是 **权限矩阵**：8 个操作权限 × 5 个角色，绿色表示可执行、灰色表示不可执行。本节把其中最要紧的 8 条抽出来排成表 27.1。

读这张表有一个技巧：**先横着看，再竖着看**。横着看，每一行只有一个“可”——说明这 8 件事互不重叠，没有一个人能自己走完流程。竖着看，风控那一列全是“可”，却一条订单类权限都没有；操盘手那一列只有订单——它能买能卖，但不知道买什么。

最后是刷新：这一页的任务状态每 30 秒刷一次（源码里 `setInterval(loadAgentTasks, 30000)`），时段状态每 60 秒重算一次⁵。它比控制中心快 10 倍，第四节会解释为什么它敢这么

⁴数据来源：系统源码 `ui/src/pages/AITeam.tsx` 的 `agentTasks` 逐条计数；同一段代码上方的注释写的是“与后台 `getOptimizedTasks` 一一对应，共 16 任务”，与清单实际条数不一致，本书照实记录。核对时间 2026-09-24。

⁵数据来源：系统源码 `ui/src/pages/AITeam.tsx`：任务轮询 30 000 ms，时段与状态重算 60 000 ms。核对时间

表 27.1 8 个操作级权限 × 5 个角色（默认矩阵中“恰好只有一个角色可执行”的那些条目）

权限项	中文	规划师	量化	风控	CIO	操盘手
CREATE_INVESTMENT_PLAN	创建投资方案	可	—	—	—	—
GENERATE_STOCK_CANDIDATES	生成候选池	—	可	—	—	—
SELECT_ASSET	选股	—	—	—	可	—
SET_TARGET_WEIGHT	设定目标权重	—	—	—	可	—
APPROVE_INVESTMENT	批准投资	—	—	—	可	—
RISK_REJECT	风险否决	—	—	可	—	—
EMERGENCY_FREEZE	紧急冻结	—	—	可	—	—
SUBMIT_ORDER	提交订单	—	—	—	—	可

这 8 行是本书从代码里的默认矩阵里挑出的“操作级”条目；矩阵本身按角色分行、条目更多（首席投资官一行就有 9 条）。矩阵里还写着负向权限：规划师、量化、风控三行都挂着“禁止批准投资”与“禁止执行订单”，操盘手一行挂着“禁止选股”“禁止修改目标权重”“禁止绕过风控”。

数据来源：系统源码 `internal/agentworkflow/models.go` 的 `PermissionMatrix` 与权限常量区（第 49–143 行）；前端渲染口径见 `docs/用户操作手册.md` 第 5 节与本书系统事实清单第 3 节。核对时间 2026-09-24。

快。

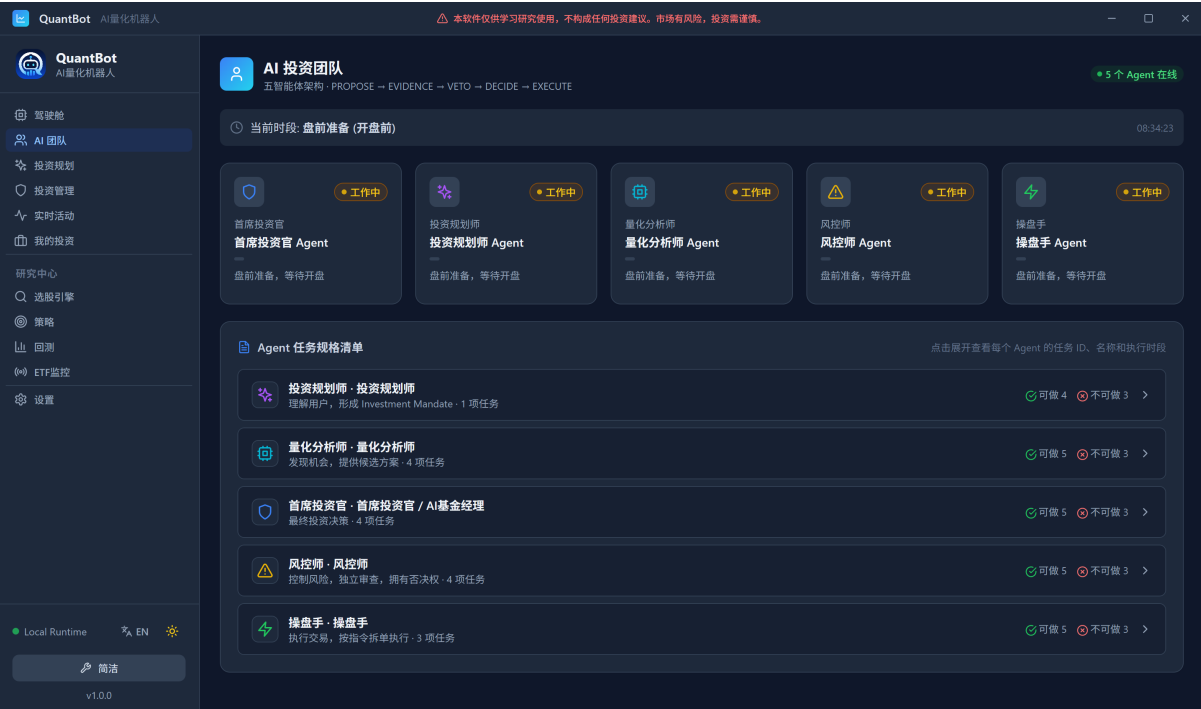


图 27.3 AI 团队页：五智能体架构（PROPOSE → EVIDENCE → VETO → DECIDE → EXECUTE）与 Agent 任务规格清单

27.3.3 实时活动：现在跑到哪一步

实时活动的地址是 `/#activity`；如果你用的是旧地址 `/#workflow`，也会落到同一页——路由表里这两个分支返回的是同一个组件⁶。这一页问的问题是：“现在跑到哪一步了。”

第一块是 **投资流水线**：五格横排，`PLANNER → QUANT → RISK → CIO → TRADER`,

2026-09-24。

⁶数据来源：系统源码 `ui/src/App.tsx` 的路由 `switch: case '/workflow'` 与 `case '/activity'` 都返回 `Activity`。核对时间 2026-09-24。

每格下面写着它在这条链上唯一被允许推进的那一步——提出方案、提供证据、风险否决、最终决策、执行交易（PROPOSE、EVIDENCE、VETO、DECIDE、EXECUTE）。正在干活的那格有一条进度条，干完的打勾，午休那种“休息中”显示黄字“休息中”。

为什么用进度条，而不是像下面那块一样直接铺日志？因为这两个东西回答的问题不同：日志回答“之前发生过什么”，进度条回答“此时此刻卡在链条的哪一环”。我在开头那半小时里，就是缺了这一格：我盯着日志从下往上翻，翻到“午休中”才反应过来，其实最上面那排格子早就告诉我全队都在休息。

第二块是 **活跃任务详情**：从五个角色里找出当前状态属于“工作中 / 执行中 / 监控中”的那一个，显示它的进度百分比、当前任务名、以及今日任务的前三条。如果一个活跃的都没有，它就按时段给两句不同的话——交易时段说“当前无活跃任务 · 交易时段 Agent 正在监控市场”，非交易时段说“当前无活跃任务 · 非交易时段 Agent 自动进入休眠”。

第三块是 **活动日志**：可以按角色筛（全部 / CIO / Planner / Quant / Risk / Trader），记录按五类着色（研究分析、决策、风控检查、执行交易、警报），最多保留 200 条，最新一条加个“NEW”的高亮。这条流的上游是两处：今日的团队活动，加最新一批智能体任务日志。

再往下是 **5 个 Agent 工作明细**：五张小卡各带一条进度条，点一张卡，下面就是这位智能体今天的任务日志。

还有两块小东西：一块是 **市场指数卡**，只留上证指数、深证成指、创业板指——这就是 AI 实际读到的实时行情；另一块是右上角的 **调度器状态徽章**，绿色“调度器运行中”，或者红色“执行器不可用”。后面这个属于故障灯层，看见了就得去查，别等它自己好。

一定要说清楚的是自动刷新，因为它在三个页面里最特殊：它的间隔不是写死的 5 分钟，而是从设置里读一个数（`activity_refresh_minutes`，取值范围 1-60 分钟，默认 5）⁷；而且它只在交易时段刷新，非交易时段整页就是静止的；右上角还能把自动刷新整个暂停。所以这一页上的时间戳“最近刷新：...”是判断它有没有在干活的最快办法。

最后，我要为一条手册里写了、但代码里还没接上的功能留一句实话：用户手册把“今日交易记录”（今日买入/卖出笔数与明细）列为这一页的功能，但当前代码里负责转换交易记录的那个函数带下划线前缀（`_convertTradeToActivity`），也就是没有被调用⁸。这种“文档说有了、代码里是预留”的地方，我不替它圆场——你去看页面，看不到那张表，就是没接上。

27.4 近似模型 → 严谨版本：刷新间隔决定陈旧度

到这里，“三个页面各自回答什么”已经清楚了。剩最后一件事，也是我在开头那半小时里真正困惑的事：为什么控制中心 5 分钟刷一次，AI 团队 30 秒刷一次？为什么不能都设成 1 秒？

先给一个差不多对的模型：**刷新越快，你看到的越新。**

这个模型在“越快越好”的方向上是对的，但它漏了两件事：一是你看到的永远是过去某个时刻的值，“实时”是不存在的；二是每次刷新是有成本的，而且成本还不一样。把这两件事

⁷数据来源：系统源码 `ui/src/pages/Activity.tsx` 与 `AIQuant/internal/config/config.go`（`ActivityRefreshMinutes` 字段）；设置项见「设置-通用」。核对时间 2026-09-24。

⁸数据来源：系统源码 `ui/src/pages/Activity.tsx`：`TodayTrade` 接口与 `_convertTradeToActivity` 均存在，但函数未被任何组件引用；对照 `docs/用户操作手册.md` 第 7 节“今日交易记录”。核对时间 2026-09-24。

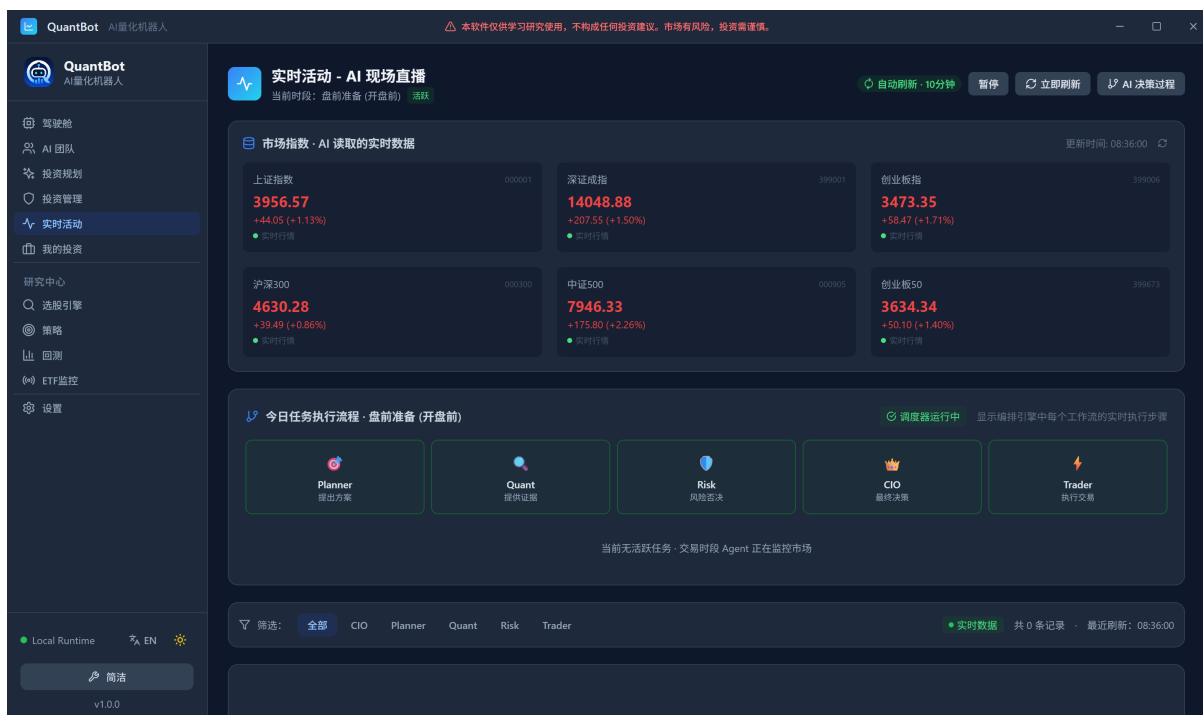


图 27.4 实时活动页：投资流水线五格（提出方案 → 提供证据 → 风险否决 → 最终决策 → 执行交易）、AI 实际读取到的市场指数，与可按角色筛选的活动日志

写下来，就成了下面这个命题。

先约定记号。设某个看板指标的真实取值随时间变化，写成函数 $x(t)$ ；看板每隔 Δ 秒刷新一次，所以看到的是 x 在上一次刷新时刻的值。两次刷新之间隔了多久，取决于你什么时候抬眼——把这个“相位”记成 $\delta(t)$ ，它落在 $[0, \Delta)$ 里。于是你看到的陈旧程度是

$$e(t) = x(t) - x(t - \delta(t)), \quad (27.1)$$

$|e(t)|$ 越大，说明屏上的数字离真相越远。这个 e 就是陈旧度 (staleness)。

假设 27.1 (指标变化率的 Lipschitz 界)。存在常数 $M \geq 0$ ，使得对一切 t, s 都有

$$|x(t) - x(s)| \leq M |t - s|. \quad (27.2)$$

称 M 为该指标的最大变化率。

这个假设在讲什么： x 不会有“瞬间跳变”。一秒之内它最多变 M 。真实的总资产、盈亏都满足它（价格是有限变化率的量）；它失效的情形是“瞬时跳变”——比如 15:00 收盘价入库那一下快照口径切换，那一刻属于离散事件，不属于本章讨论的连续过程。

命题 27.2 (刷新闻隔、陈旧度与成本的三方权衡)。设指标 x 满足假设 27.1，刷新闻隔为 $\Delta > 0$ ，刷新时刻取 $\tau_k = k\Delta$ ($k \in \mathbb{Z}$)，相位 $\delta(t) = t - \Delta \lfloor t/\Delta \rfloor$ 。则：

(a) (陈旧度上界) 对一切 t ,

$$|e(t)| \leq M\Delta. \quad (27.3)$$

(b) (上界是紧的) 取 $x(t) = Mt$ ($M > 0$), 则 $\sup_t |e(t)| = M\Delta$, 即 eq. (27.3) 不能再收紧。

(c) (给定陈旧度上限的最小刷新频率) 若要求对一切 t 有 $|e(t)| \leq \varepsilon$ ($\varepsilon > 0$), 则必须

$$\Delta \leq \frac{\varepsilon}{M}, \quad \text{也就是} \quad \frac{1}{\Delta} \geq \frac{M}{\varepsilon}. \quad (27.4)$$

(d) (成本) 设单次刷新的成本为 $c > 0$ (它可以是钱, 也可以是接口配额或电量), 单位时间成本预算为 $\hat{C} > 0$ 。则预算约束等价于

$$\Delta \geq \frac{c}{\hat{C}}, \quad (27.5)$$

可行区间为 $[c/\hat{C}, \varepsilon/M]$, 它非空当且仅当

$$Mc \leq \hat{C}\varepsilon. \quad (27.6)$$

在可行时, 最不陈旧的刷新间隔是 $\Delta^* = c/\hat{C}$ 。

这个命题在说什么: **精度给刷新间隔一个上限, 成本给它一个下限**。你想看得更准, 就得刷得更频繁; 你不想花更多成本, 就不得不刷得更慢。两头一夹, 剩下的就是你能挑的区间; 这个区间可能空——空的时候, 你只能放宽对“准”的要求, 或者加预算, 没有第三条路。

证明. 先把相位的取值范围算实。设 $k(t) = \lfloor t/\Delta \rfloor$, 则按下取整的定义 $k(t)\Delta \leq t < (k(t)+1)\Delta$ 。三段同减 $k(t)\Delta$, 得 $0 \leq \delta(t) < \Delta$ 。

(a) 注意 $t - \delta(t) = k(t)\Delta$ 恰好是一个刷新时刻, 所以可以直接把假设 27.1 用在 t 与 $t - \delta(t)$ 这组点上:

$$|e(t)| = |x(t) - x(t - \delta(t))| \leq M |t - (t - \delta(t))| = M \delta(t) < M\Delta. \quad (27.7)$$

最后一步用了 $\delta(t) < \Delta$, 于是 eq. (27.3) 成立 (取 $\leq$ 是因为要的是上界)。

(b) 取 $x(t) = Mt$ 。先算它的 Lipschitz 常数: $|x(t) - x(s)| = M|t - s|$, 所以 M 正是它的最大变化率, 假设 27.1 被恰好取等。代入 eq. (27.1):

$$e(t) = Mt - M(t - \delta(t)) = M\delta(t). \quad (27.8)$$

现在取一串 $t_\eta = \Delta - \eta$, 让 η 从正的一侧趋近于 0。因为 $t_\eta \in (0, \Delta)$, 它落在第一个刷新周期里, $\delta(t_\eta) = t_\eta = \Delta - \eta$, 于是 $|e(t_\eta)| = M(\Delta - \eta)$ 。令 $\eta \rightarrow 0^+$, 得到 $\sup_t |e(t)| \geq M\Delta$; 结合 (a) 的 $\leq M\Delta$, 两边相等。上界取到, 所以它是紧的。

(c) 用反证。设 $\Delta > \varepsilon/M$ 。取 $t = \varepsilon/M + \eta$, 其中 $\eta > 0$ 取得足够小, 使 $t < \Delta$ (这样的 η 存在, 因为 $\varepsilon/M < \Delta$)。此时 $t \in (0, \Delta)$, 故 $\delta(t) = t$, 代回 (b) 里算过的那条等式:

$$|e(t)| = Mt = M\left(\frac{\varepsilon}{M} + \eta\right) = \varepsilon + M\eta > \varepsilon. \quad (27.9)$$

这与“对一切 t 有 $|e(t)| \leq \varepsilon$ ”矛盾。所以必须 $\Delta \leq \varepsilon/M$ ，两边取倒数（都是正数，不等号反向）得 $1/\Delta \geq M/\varepsilon$ ，即 eq. (27.4)。

(d) 单位时间里发生的刷新次数是 $1/\Delta$ ，所以单位时间成本是 c/Δ 。要求它不超过预算：

$$\frac{c}{\Delta} \leq \hat{C} \iff c \leq \hat{C} \Delta \iff \Delta \geq \frac{c}{\hat{C}}, \quad (27.10)$$

第二个等价号两边同除以正数 $\hat{C}$ ，第三个等价号两边同除以正数 Δ 并移项——每一步都是同解变形，不等号方向不变。这就是 eq. (27.5)。

把它和 (c) 的 $\Delta \leq \varepsilon/M$ 合并， Δ 必须同时落在两个区间里，故可行集为 $[c/\hat{C}, \varepsilon/M]$ 。实数区间非空当且仅当下端点不超过上端点，即 $c/\hat{C} \leq \varepsilon/M$ ，两边同乘正数 $\hat{C}M$ 得 $Mc \leq \hat{C}\varepsilon$ ，即 eq. (27.6)。

最后，在可行集里“最不陈旧”等于取最小的 Δ （由 (a)， $M\Delta$ 关于 Δ 单调增），故 $\Delta^* = c/\hat{C}$ ；此时上界 $M\Delta^* = Mc/\hat{C} \leq \varepsilon$ ，恰好不超过允许值。■

注 27.3 (数据自己的节奏：有效刷新间隔). 命题 27.2 里的 x 是连续的。真实系统里还有第二道闸门：数据源本身多久产生一次新值。

调度器是每 30 秒醒一次的，六维判势的实时快照也是 30 秒去抖一次⁹。设数据每隔 Δ_0 秒才可能变一次。当 $\Delta < \Delta_0$ 时，相邻两次刷新之间 x 根本取同一个值， $e(t)$ 一点也没变小，而成本 c/Δ 却按比例上升。

所以实用的选择不是 Δ^* ，而是

$$\Delta_{\text{eff}} = \max\{\Delta^*, \Delta_0\}. \quad (27.11)$$

这解释了 AI 团队页为什么是 30 秒而不是 1 秒：它的数据本来就被后端每 30 秒写一次，刷得再快也只能重复读到同一个值。

现在把三个页面代进命题 27.2 的 (d)。

控制中心的 c 很大：它每一轮先 `RefreshPrices` 拉一次实时行情（要走进数据源、可能走外部网络），再重算一遍组合状态。 c 大，则 $\Delta^* = c/\hat{C}$ 大，所以它的 5 分钟不是“慢”，是“贵”换来的。

AI 团队的 c 很小：它只调一次 `GetAllAgentTasks`，查本地数据库。 c 小， Δ^* 就小，于是 30 秒在预算内是合理的——再加上它的数据节奏 Δ_0 本来就是 30 秒，按前述备注， Δ_{eff} 恰好就是 30 秒。

实时活动介于两者之间：它要读本地库（便宜）、要拉指数行情（贵），所以它给用户一个 1-60 分钟的可调旋钮，默认 5 分钟，并且只在交易时段刷。把“多久刷一次”做成用户可调，等于把命题里的 $\hat{C}$ 交给用户自己填。

这一节想说的最后一句话是：看板上的“实时”不是一个形容词，是一个被两个数字夹住

⁹数据来源：系统事实清单第 6 节（`AutoScheduler` 主循环 30 秒 ticker）与第 8 节（六维实时快照 30 秒缓存去抖）；源码 `internal/scheduler/auto_scheduler.go`。核对时间 2026-09-24。

的区间。上端是 M/ε ，下端是 $c/\hat{C}$ 。

27.5 它在系统里长什么样

先把三个页面摆成一张表。

表 27.2 三个页面的路由、回答的问题与刷新节奏

页面	路由	回答的问题与关键区块	刷新
控制中心	#/（默认）	钱还剩多少 / 系统健不健康： 五张指标卡、CIO 今日状态、 风险监控、持仓分布、AI 今日 决策	5 分钟
AI 团队	#/ai-team	谁在干活、谁有权干什么：5 张智能体卡、市场时段指示、 任务规格清单、8×5 权限矩阵	30 秒（任务）/ 60 秒 （时段）
实时活动	#/activity（旧 #/workflow）	现在跑到哪一步：投资流水线、 活跃任务详情、活动日志、5 个 Agent 工作明细、市场指数	默认 5 分钟，1–60 分钟 可调，可暂停，非交易时 段不刷

路由名采自源码里的 hash 值；“关键区块”只列每一页最上面的几块，不含全部控件。
数据来源：`ui/src/App.tsx` 的路由 `switch`、`ui/src/pages/Dashboard.tsx`、`ui/src/pages/AITeam.tsx`、
`ui/src/pages/Activity.tsx`；刷新节奏见各文件内的定时器与 `internal/config/config.go` 的
`activity_refresh_minutes`。核对时间 2026-09-24。

三件事要单独说。

第一件是路由：这套前端用的是 hash 路由——地址栏里 # 后面那段就是页面名，改它不会重新加载整个应用。`#/workflow` 这种旧地址被保留成指向实时活动的别名，所以你看的老教程里的链接不会 404。

第二件是数字的来路。看板上每一个“现价”，最后都落在行情库上：本书核对时那张 `stock.ohlcv` 表里有 10041343¹⁰ 行 K 线，覆盖 9584 个代码。看板不是自己算价，它只是把库里最近的一条读出来。

第三件是一位老朋友：三个页面的“数字在不在动”，取决于时段，而时段的判定只有一处实现——`internal/util/trading_hours.go`。控制中心不等时段也照刷（它会拉行情，非交易时段拿回来的是上一笔），AI 团队按 30 秒刷任务、按 60 秒重算时段，实时活动则干脆在非交易时段停手。同一件事在三个页面里有三种处理，原因都在命题 27.2 的 (d) 那一项成本里。

27.6 常见误解与纠偏

常见误解·页面上的资产和盈亏，是系统实时心算出来的 不是。它读的是数据库里的快照。
这句话解释了一类很常见的困惑：“我手动加了一下持仓市值，和页面上那个总资产对不上，差了几十块。”差的那几十块，来自快照的时间点与你心算的时间点不同。
系统在这个地方的取舍是明确的：持仓状态以数据库为准，而后端每次被问到时，会先拿最近一次收盘快照加上快照之后的成交做一次强制对账，再把结果返回给前端。也就是说，屏幕上的数字永远是“某一次对账的结果”，不是“此刻的算账结果”。

¹⁰真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

这条规矩在第 chapter 二十九 章还会再出现一次，而且会更刺眼：当日盈亏以交易日为界，15:00 之后它就不再刷新了——你晚上盯着它看，它当然不动。

常见误解·“AI 在休息”等于坏了 我开头那半小时，就是想当然地掉进了这个坑。

真实情况是：非交易时段它本来就该睡。AI 团队页在休市时把五个角色都标成“休市中，等待下一交易日”；午休时整排“午休中”；实时活动页在非交易时段写着“当前无活跃任务·非交易时段 Agent 自动进入休眠”，而且那一页干脆停止自动刷新。

这套设计不是“偷懒省算力”这么简单。它对应第 chapter 五 章那张时段表：一个每 30 秒醒一次的状态机，醒来第一件事是看时段——午休那一个半小时里多算一次纯属浪费，晚上八点算出来的也不是“现在”，是“明天”。

所以判断“是不是坏了”，不要看动作有没有在动，要看三处硬证据：控制中心顶栏的状态徽章是不是黄的“停止”、实时活动右上是不是红的“执行器不可用”、以及智能体卡片上有没有红色的“异常”。这三个都不亮，它就是睡着了。

常见误解·三个页面是同一份数字的三个视图 不是。它们的刷新节奏不一样（5 分钟 / 30 秒 / 可暂停），所以同一时刻，三个页面上的数字本来就可以不一致。

这与命题 27.2 说的是一件事：两个页面的陈旧度上界分别是 $M\Delta_1$ 和 $M\Delta_2$ ，只要 $\Delta_1 \neq \Delta_2$ ，它们的读数就不可能时时相同。

更要紧的是数据来源也不同：控制中心的决策列表读的是最近 7 天的决策日志（屏上只显示 5 条）；实时活动的日志读的是今日团队活动加最新任务日志（最多 200 条，最新一条标“NEW”）。同一个上午发生的同一件事，在这两页上可能一处早就刷过去了、另一处还停在两分钟前。

正确的用法是：判断“要不要动手”看故障灯层，“现在怎么了”看实时活动，“最后亏了多少”再看控制中心。想让三页同步，只有一个办法——把实时活动那个刷新旋钮调到和你想看的节奏一致。

27.7 真实出处

本章的真实出处

- 控制中心页面（五张指标卡、CIO 今日状态、风险监控、持仓分布分页、AI 今日决策、5 分钟顺序轮询、简洁/详细两模式）：`ui/src/pages/Dashboard.tsx`
- AI 团队页面（5 张智能体卡、时段驱动的状态映射、任务规格清单、8×5 权限矩阵、30 秒任务轮询）：`ui/src/pages/AITeam.tsx`
- 实时活动页面（投资流水线、活跃任务详情、活动日志筛选、5 个 Agent 工作明细、市场指数卡、调度器徽章、可暂停的自动刷新）：`ui/src/pages/Activity.tsx`
- 旧工作流页与编排视图（活跃工作流、工作流模板、智能体待办）：`ui/src/pages/Workflow.tsx`
- `hash` 路由与 `#/workflow` 到实时活动的重定向：`ui/src/App.tsx`
- 前端角色名、状态标签、角色职责的“可做 / 不可做”清单：`ui/src/types/index.ts`
- 角色定义、任务类型与状态、默认权限矩阵与权限常量：`internal/agentworkflow/models.go`、`internal/agentworkflow/permissions.go`
- 市场时段枚举与判定（`GetMarketPhase`、`PhaseLabels`）：`internal/util/trading_hours.go`
- 30 秒调度扫描与盘中 300 秒重复间隔：`internal/scheduler/auto_scheduler.go`（另见本书系统事实清单第 6 节）
- 实时活动刷新分钟数的配置项：`internal/config/config.go`（`ActivityRefreshMinutes`）

- 页面功能对照（控制中心 5 分钟刷新的用户侧口径、AI 团队 30 秒、实时活动可暂停）：[docs/用户操作手册.md](#) 第 4-7 节

动手试一试（十五分钟）

1. 依次打开三个页面，每页停留三分钟。用的方法是只记“哪些数字在动”：控制中心里“今日盈亏”和“总资产”会在刷新时一起跳一下；AI 团队的当前动作会在时段切换那一刻整排改口；实时活动的日志会从上往下推。记完把三页的“在动的数字”列成三行，你会发现三行的长度差别很大——那就是 Δ 的差别。
2. 回到 AI 团队页，展开风控师那一行，把“不能做”清单读一遍；再对照表 27.1 的“风险否决”那一行横着看。然后回答一句话：谁能下单？写完再去操盘手那一行核对——只有它的“可以”里没有决策类动作。
3. 把系统时间（或挑一个非交易时段）调到休市状态，观察三处：AI 团队整排“休市中，等待下一交易日”、实时活动“非交易时段 Agent 自动进入休眠”、控制中心的状态徽章。确认这三处都不亮红黄之后，你就学会了“睡着”和“坏了”的分辨法。
4. 在实时活动页右上角点一下，把自动刷新暂停，盯着“最近刷新：...”那行字看它不再变；再点回来。顺便把设置里的刷新分钟数改小一格（比如从 5 改成 3），回到页面确认徽章上的文字跟着变了——这就是命题 27.2 里那个 $\hat{C}$ 被你亲手调了一次。
5. 打开 `ui/src/pages/Dashboard.tsx`，搜 300000。你会看到它只出现一次，就是那一句等待下一轮的代码。把三个页面的这个数字都搜一遍，对照表 27.2 的最后一列。

到这里，你已经会读这三块表了：故障灯告诉你还能不能开，流水线告诉你车正在哪一段，油表告诉你还剩多少。可是控制中心右下那一列“AI 今日决策”里，每一条都只给了你一行理由——那一行字的背后，藏着信心度、风控审批、多空辩论、仓位系数和一整条证据链。

下一章我们把其中一条决策拆到字。一份写成 BUY | 信心 72 | 风控审批通过 | 仓位系数 0.55 | 辩论：看多 3 / 看空 2，共识 61 的记录，到底每一段是谁写的、按什么规则写出来的。

读懂决策：CIO 决策中心里的每一行字

28.1 一行字，十二个信息

九月十一日下午三点多，我打开 AI 决策页，屏幕上有一行字长这样：

决策日志里的一条真实排版	
BUY	信心 72 风控审批 ✓ 通过 仓位系数 0.06 Kelly 仓位 $\times 0.78$ 辩论：方向看多 • 共识 61% 净多 12% 来源 proposal=+,skeptical=CAUTION,debate=+,regime=-

我第一次看到这行字的时候，只能读懂前面两个字。后面那一串——信心、审批、仓位系数、Kelly、共识、净多、来源——每一个都认识，但连起来不知道在说什么。

更麻烦的是，我事后发现：那一天真正决定我该不该动手的，不是前面那个 BUY。那个 BUY 是结论；结论后面那一串，才是理由。而理由里藏着那天最要紧的一件事——仓位系数 0.06。

这一章我们就把这一行字，一栏一栏拆开读。

28.2 先读“病历首页”，不读结论

直觉 · 病历首页

你去医院看一份病历，首页上有一栏一栏：主诉、现病史、查体、辅助检查、诊断、处理、签名。

一个受过训练的医生读病历，不会先看“诊断”那一行。他会先看主诉和查体，再回头看诊断——因为诊断是可以写错的，而查体和检查数据是当时真实发生的。

更关键的是最后一栏：签名。没有签名，前面写得再漂亮，这份病历也只是一张纸。

AI 决策页就是它的决策的病历首页。它按“病历”的顺序摆放，而不是按“结论”的顺序摆放。

这一点值得单独强调，因为大部分人第一次用这类系统时的习惯，是直奔那个大大的“买入”两个字。而那些字背后真正有价值的信息是：

- 风控审批——这是唯一能一票否决的环节，先看它；
- 仓位系数——这是六维判势给的“限速”，它决定了这个买入到底有多大；
- 证据链——主张、异议、辩论、集成，谁说了什么、谁反对了什么；
- 归因——为什么最后定成这个数。

下面按页面从上到下的实际顺序走一遍。

先看全景。下面这张图是「投资管理」页在屏幕上的实际样子，本章接下来的五块，就是把它从上到下拆开读。



图 28.1 投资管理·投资决策中心：今日决策卡片、风险评价（VaR / ES）与 CIO 决策日志

28.3 第一块：这张页面的两个 Tab

页面标题写着“投资管理·投资决策中心·系统级投资管理”，右上角是运行状态徽章，一共四种：

- 运行中（绿点）：CIO 正在工作；
- 已暂停：被暂停；
- 紧急停止（红色）：紧急停止开关已按下；
- 就绪：既没在跑，也没被停——通常是非交易时段。

“就绪”和“已暂停”是两件完全不同的事，这一点在第 chapter 三十一 章讲安全开关时会重点说。这里先记住一个判断：如果盘中有交易时段而徽章是“就绪”，那才是问题；非交易时段显示“就绪”，是正常的。

页面主体分成两个 Tab：投资决策与 每日复盘。前者讲“今天怎么做”，后者讲“昨天做得怎么样”。我们按顺序看。

28.4 第二块：市场六维判势卡片

这是第 chapter 十九 章那个算法在界面上的样子。卡片右上角有一行小字写着数据日期——这一行字非常重要，因为判势结果是有日期的，它不是“此刻”，而是“某个交易日”。

卡片上有四个格子：

“修正总分”那一格是我最推荐新读者先看的地方。它把两个数并排放在一起：修正后的分，和括号里的原始分。两个数只要不一样，你立刻就知道今天市场内部是矛盾的——不需要再去

表 28.1 市场六维判势卡片上四个格子的含义

格子	字段	怎么读
市场标签	market_tag	强势 / 结构性震荡 / 偏弱 / 退潮风险；这是仓位系数的分档名
仓位系数	position_rate	显示为 $\times 0.55$ 这样的形式；买入金额要乘这个数
修正总分	adjusted_total_score	括号里同时给出“原始”分 (raw_total_score)，两者不一致说明发生了冲突修正
矛盾维度	conflict_count	达到 2 就触发八折；这个数字越大，市场越自相矛盾

数据来源：前端 `ui/src/pages/CIO.tsx` 的六维卡片区块；字段名对应 `market_sixdim_daily` 的列；核对时间 2026-09-24。

数有几个维度爆表。这是界面在做的一件很贴心的事：把第十九章那个“冲突修正”，从公式变成了你能一眼看见的差值。

28.5 第三块：今日决策卡片

再往下是当天那条决策。它有五个要紧的位置：

1. 决策类型：BUY / SELL / HOLD / NO_ACTION 之类的枚举，中文映射后显示；
2. 风控审批：三种状态——✓ 通过（绿）、× 否决（红）、审核中（黄）；
3. 决策原因：一句话，是这条决策最直接的“主诉”；
4. 市场状态：牛市 / 熊市 / 震荡市，以及信心指数（一个百分比）；
5. 时间戳：这条最容易被误读。

先说第 2 条：风控审批是三种状态，不是两种。“审核中”是一个真实存在的、且必须被显示出来的状态。它的存在意味着：风控还没表态，此时决策不成立。一个把“还没审完”显示成“通过”的界面，比一个不显示审批状态的界面危险得多。

再说第 5 条：时间戳。系统在这里做了一个很克制的选择：

关于时间戳的一条实现决定（节选自 `ui/src/pages/CIO.tsx` 的注释）

“盘前 (9:15 前) 或休市时不应显示‘已完成决策’的时间戳，避免误导；无真实决策时一律不显示时间戳（显示 --），避免把当前时间误当决策完成时间。”

这段话值得抄下来。它承认了一件事：界面上一个看起来无害的时间戳，会被读成“系统今天已经决策过了”。所以在没有真实决策的时候，它宁可显示两根横线，也不肯显示一个像模像样的时间。

同一个位置还有一条兜底逻辑：当决策记录为空时，界面不是什么都不显示，而是造一条带名字的空决策——决策类型 NO_ACTION，原因写着“盘前准备中，等待开盘后（9:15）生成决策”或“当前休市，等待下一交易日”。

“什么都不做”在这里是一个有名字的、可见的结果。这条设计和第一部分第 chapter 三 章那个 NOActionCode 是同一件事：不给“什么都没做”留一个名字，系统就只能编一个动作出来。

28.6 第四块：证据链——决策日志里的五行

这一块是整页的核心。

决策日志（页面上标为“CIO 日志·投资决策记录”）按时间倒序排列，默认展示最近 3 天的分页。每条记录展开后，会给出最多五块证据。我把它们的标签和含义列成表 table 28.2。

表 28.2 决策证据链的五块标签

标签	字段前缀	它记录了哪一步
【LLM 主张】	market_direction、confidence、suggested_position_rate、effective_position_rate、preferred_stocks、feedback	主张官的方向判断、信心、它想要多少仓位，以及最终生效仓位（低于主张时会标“已趋保守”）
【异议评审】	skeptic_verdict、skeptic_concerns	质疑者的结论与具体担忧点
【多空辩论】	debate.direction、agreement、bull[]、bear[]、kelly_position_scale	辩出来的方向、共识度、多空论据，以及 Kelly 缩放倍数
【多提案集成】	ensemble.direction、net_score、votes	四个来源投票后的净多分，以及每个来源投了什么
【LLM 复议】	llmReview.action、score、reason、suggestion	一次带打分的复盘意见

五块标签在界面上的顺序就是上表顺序，也是决策实际发生的顺序（主张 → 异议 → 辩论 → 集成 → 复议）。
数据来源：前端 `ui/src/pages/CIO.tsx` 决策日志渲染区块；字段名对应 `CIODecision.Evidence`；核对时间 2026-09-24。

这五行里，我最想让你注意两处细节。

第一处：`effective_position_rate` 旁边的“(已趋保守)”。

界面上是这样拼这一句的：先显示主张仓位，再显示生效仓位；只要生效仓位小于主张仓位，就在后面跟四个字——“已趋保守”。

这是一个非常小的界面细节，但它把第一部分整整三章的内容（Kelly 缩放、集成投票、只降不升）压缩成了四个字。你不需要记得 $0.4 + 0.6 \times \text{certainty}$ ，你只需要看到“已趋保守”，就知道 AI 想要的比规则给的少。

反过来，如果生效仓位等于主张仓位，那就不打这个标签——因为那说明没有任何一环把它压下来。这两种情况在界面上长得不一样，而它们背后的含义完全不同。

第二处：【多提案集成】里那个 `votes`。

它以 `key=value` 的形式把每个来源的投票直接摊开，例如：

```
1 来源 proposal=+, skeptic=CAUTION, debate=+, regime=-
```

代码 28.1 多提案集成区块里“来源表态”的真实拼法（节选自 `ui/src/pages/CIO.tsx`）

这段话在说什么：四个来源，两个偏多（`proposal=+`、`debate=+`），一个谨慎（`skeptic=CAUTION`），一个偏空（`regime=-`）。净多分是这四票按各自权重折算出来的结果。它不隐藏任何一票——包括那个和结论相反的 `regime=-`。

这是“可解释性”这个词在这里的确切含义：不是事后写一段理由，而是把当时投出的每一票原样留下来。原因很简单——一段事后撰写的解释，无论多流畅，都不携带信息；而一票一票的原始记录，是可以被复核、被统计、被追责的。

28.7 第五块：每日复盘 Tab

切到第二个 Tab，画面上是一条复盘记录。它有四块内容。

（一）**状态与四个关键指标**。状态是“已完成”或“进行中”。四个指标是：总资产、今日盈亏、今日收益率、持仓数量。注意“今日盈亏”的颜色约定——盈利红、亏损绿，这是 A 股的习惯，和欧美正好相反（见第 chapter 二十三 章的制度约定）。

（二）**市场状态**。牛市 / 熊市 / 震荡市，加上信心指数百分比。

（三）**首席投资官深度复盘**。一段可换行显示的文字，是这条复盘的正文。

（四）**各智能体复盘汇总**。这一块按五个角色分组：规划师（Planner）、量化分析师（Quant）、风控（Risk）、交易员（Trader）、首席投资官（CIO）。每一组下面挂该角色当天的小结。

第四块是很多人会忽略、但我认为最有价值的一块。它意味着复盘不是“CIO 一个人写总结”，而是五个岗位各交一份自己的作业。如果哪天亏损了，你可以直接去看风控那一栏写了什么——而不是在一段由单一模型生成的总结里猜它有没有替自己开脱。

直觉·体检报告上的分工签名 一份完整体检报告，不同项目后面签着不同科室的名字：血常规是检验科出的，心电图是心功能室出的，结论是主检医师写的。

你不会接受一份“所有项目都由同一个人签字”的体检报告——因为那意味着出错时无人可问。

复盘也是一样：五个角色各签一份，比一份写得很好的总结更有用。

28.8 这条决策从哪儿来

最后说一句界面背后的数据来源，因为它决定了“你看到的”是不是“当时发生的”。

系统在这里做过一次收敛：界面上不再单独维护一份“今日决策快照”，所有决策记录统一来自 `decision_logs` 表，页面按时间倒序读它。只有在该表查不到记录时，才用当前市场状态造一条 `NO_ACTION` 的占位。

这个决定的好处是：你在界面上看到的、和审计日志里查到的、和事后复盘时引用的，是同一条记录。没有第二份可能不一致的副本。

常见误解·仓位系数低 = 看空 这是本项目里被误解最多的一条，没有之一。

仓位系数 0.06 的意思是“今天最多只能下 6% 的注”，不是“今天要跌”。它是由六维总分映射出来的一个缩放系数，和方向完全无关。

一个很具体的反例：某天判势给 0.06（退潮风险），选股层照样挑出了三只票，策略层也照样给它们发了买入信号。系统在那天做的事情不是“卖出”，而是把新的买入缩到极小，同时完全放行卖出与止损。

所以正确的读法是：仓位系数低 = 今天不适合开新仓；它既不预测下跌，也不阻止你离场。

常见误解·先读结论，再看有没有支持它的理由 “页面上一行写着 BUY，我就买了。”

这件事的问题不在于结论对不对，而在于读的顺序反了。先看到结论之后，人就只会去找支持它的证据，而剩下那些反对的证据会被当成噪音略过——这是所有人都有的倾向，不是你的个人缺陷。

正确的读法是从上往下，而且按页面的原始顺序读：

第一步看风控审批。它是“× 否决”还是“审核中”，后面的所有文字都不必读。

第二步看市场六维判势。仓位系数是多少、原始分和修正分差多少。

第三步看证据链的五行。特别注意【多提案集成】的“来源表态”里有没有反方——如果四个来源里有一个是 -，那这个决策就是带着分歧做出来的。

走完这三步，你才应该去看那个 BUY。因为到那时你已经知道：这个买入是在什么限速下、带着多少分歧做出的。

本章对应的真实页面与数据

- 决策中心页面（两个 Tab、状态徽章、六维卡片、决策日志、每日复盘）：`ui/src/pages/CIO.tsx`
- 六维判势卡片字段：`market_tag / position_rate / adjusted_total_score / raw_total_score / conflict_count`
- 证据链五块字段：`CIODecision.Evidence` 下的 `market_direction`、`skeptic_verdict`、`debate`、`ensemble`、`llmReview`
- 决策记录表：`decision_logs`；页面按时间倒序读取，兜底为 `NO_ACTION` 占位
- 六个维度的取数与修正：`internal/marketsixdim`（见第 [chapter 十九](#)、[chapter 二十](#) 章）

动手试一试（十分钟）

1. 打开 AI 决策页。先看右上角的状态徽章是四种里的哪一种，再对照当前时间判断这个状态是否合理。
2. 找到【市场六维判势】卡片，把“修正总分”和括号里的“原始”两个数相减。差值不为零时，说说冲突修正打了多少折。
3. 往下找今天的决策，检查三件事：决策类型、风控审批状态、决策原因。如果风控审批是“审核中”，到此为止——不必再往下读。
4. 展开最近一条决策日志，逐块读那五行证据。重点找两个东西：有没有“（已趋保守）”这四个字？来源表态里有没有 -？
5. 切到“每日复盘”Tab，看“各智能体复盘汇总”里五个角色各自写了什么。找出说得最保守的那一份，想想为什么是它。

决策读懂了。可是决策只是“打算做什么”，它和“账户里实际有什么”之间还有一段距离——中间隔着一整套账本：持仓、成交、手续费、以及每天收盘后写下的那份快照。

下一章我们就去读账本。那里有一个会让你困惑很久的问题：为什么我手动算的今日盈亏，和软件显示的差那么一点？

我的投资：持仓、成交与“以数据库为准”的盈亏

29.1 差了三块五

九月十一日收盘后，我做了件很无聊的事：打开行情软件，把手里几只票的收盘价乘上数量，加了一遍；再把现金加进去，减掉前一天的总资产。

我算出来的今日亏损是 312.4 元。

软件上写的是 315.9 元。

差了 3.5 元。

我第一反应是“软件算错了”。第二反应是“我算错了”。来回核对了三遍之后我才明白：两个数都没错，是我们算的不是同一件事。我算的是“价格变动”，软件算的是“权益变动”——中间隔着一层我完全没想起来的東西：今天有一笔卖出，而卖出要交印花税和佣金。

那 3.5 元，正好是那笔卖出的费用。

这一章讲的就是这本账：持仓是什么、成交是什么、盈亏是怎么算出来的，以及为什么系统的口径是“以数据库为准”，而不是“以我脑子里的算术为准”。

29.2 记账本的三样东西

直觉·记账本上只有三样 一个家庭记账，本子上其实只有三样东西：
余额——你现在手里有多少钱、有多少存货（这是**持仓**）；**流水**——每一笔进出（这是**成交记录**）；**结账**——每天（或每月）把当时的值固定下来，写一行“截至今日”（这是**快照**）。
结账那一步最容易被忽略，但它才是三样里最重要的一样。因为没有它，你三个月后翻开本子，只能看到一堆流水，得从头重算一遍才知道当时有多少钱。
一个常见的错法是：把“余额”当成可以随时手算的东西。它不能——**余额**只在你结账的那一刻被固定下来；在那之后，它由“最近一次结账 + 之后的所有流水”决定。

系统里这三样东西的落点是这样的：

表 29.1 账本的三样东西在系统里的位置

记账本上的东西	系统里的落点	它回答什么问题
余额（持仓）	<code>position_snapshots</code> 的最近一条收盘快照	那天收盘时，我手里有什么
流水（成交）	交易记录表	每一笔买卖：方向、数量、价格、费用
结账（每日结算）	<code>portfolio_daily_stats</code>	那天收盘时，总资产、累计盈亏、今日盈亏是多少

数据来源：系统源码 `internal/portfolio/engine.go` 的 `RecordDailySnapshot`；系统事实清单第 12、13 节；核对时间 2026-09-24。

界面上对应的三个 Tab 是：持仓明细、交易记录、盈亏分析。下面一个一个读。

29.3 持仓明细：八列

持仓表从左到右一共八列，每一列都有它防的那种错：

表 29.2 持仓明细的八列

列	为什么要有它
代码 / 名称	名称缺失时会退化为代码占位（事故 #14 的教训：不要用代码冒充名称）
持仓量	整手归一化之后的实际持股数（A 股必须凑 100 股的整数倍）
现价	来自实时刷新；刷新间隔在设置页里可改
成本	加权平均成本，含买入时的佣金
市值	持仓量 $\times$ 现价
盈亏	市值与成本之差，并给出百分比
权重	该标的市值占总资产的比例；风控的“单票 $\leq 30\%$ ”看的就是这一列

数据来源：前端 `ui/src/pages/Portfolio.tsx` 的持仓明细表头；成本口径见 `internal/portfolio/engine.go`；
核对时间 2026-09-24。

其中“成本”那一列最容易被想简单。它不是“我买入的价格”，而是含费用的加权平均成本。把它写清楚：

定义 29.1 (加权平均成本与已实现盈亏). 设某标的当前持有 $Q > 0$ 股，加权平均单位成本为 c 。发生一笔买入：数量 $q > 0$ 、成交价 p 、买入费用 f_{buy} （佣金 + 冲击；A 股买入不收印花税）。则新的加权平均成本为

$$c' = \frac{Qc + qp + f_{\text{buy}}}{Q + q}. \quad (29.1)$$

发生一笔卖出：数量 $q' \in (0, Q]$ 、成交价 p' 、卖出费用 f_{sell} （佣金 + 印花税 + 冲击）。则已实现盈亏为

$$R_{\text{realized}} = q'(p' - c) - f_{\text{sell}}. \quad (29.2)$$

命题 29.2 (两条会计恒等式). 沿用定义 29.1 的记号。

(i) 买入不产生盈亏：买入后持仓市值与成本的差额，恰好等于买入费用的相反数，即 $q(p - c') = -f_{\text{buy}} + [Qc - q'(\dots)]$ 中的一次项被吸收，等价地

$$(Q + q)c' - (Qc + qp) = f_{\text{buy}}. \quad (29.3)$$

也就是说，买入的那一刻你付出的是费用，而不是“浮亏”；

(ii) 受价者口径下的现金账：买入时现金减少 $qp + f_{\text{buy}}$ ；卖出时现金增加 $q'p' - f_{\text{sell}}$ 。整个持仓周期的权益变化为

$$\Delta E = q'(p' - c) - f_{\text{sell}} - \underbrace{(Q_{\text{终}} \cdot p_{\text{末}} - Q_{\text{末}} \cdot c)}_{\text{剩余持仓浮盈}}, \quad (29.4)$$

对已清仓 ($Q_{\text{终}} = 0$) 的情形， $\Delta E = R_{\text{realized}}$ ，即式 (29.2)。

证明. (i) 由式 (29.1) 两边乘以 $Q + q$ ：

$$(Q + q)c' = Qc + qp + f_{\text{buy}}.$$

把 $Qc + qp$ 移到左边： $(Q + q)c' - (Qc + qp) = f_{\text{buy}}$ 。左边正是“新成本总额减旧成本总额加本次买入金额”，也就是买入导致了成本基准被抬高了多少——抬高的量恰为买入费用。故买入本身不产生交易盈亏，只产生费用。

(ii) 买入支付 $qp + f_{\text{buy}}$ ，卖出收到 $q'p' - f_{\text{sell}}$ 。若买入 q 股、卖出 $q' = q$ 股，则净现金流为

$$-qp - f_{\text{buy}} + qp' - f_{\text{sell}} = q(p' - p) - f_{\text{buy}} - f_{\text{sell}}.$$

另一方面，按加权平均成本法，卖出部分的账面成本为 $q'c$ ，故

$$R_{\text{realized}} = q'(p' - c) - f_{\text{sell}} = q'(p' - p) + q'(p - c) - f_{\text{sell}}.$$

当把买入费用 f_{buy} 摊入 c 时（式 (29.1) 正是这么做的，故 $q'c$ 已含摊入的 f_{buy} ），两式相差的恰是剩余持仓的成本项。对全清仓 $Q_{\text{终}} = 0$ ， $q' = Q$ ，剩余项消失， $\Delta E = q'(p' - c) - f_{\text{sell}} = R_{\text{realized}}$ 。

■

■

这个命题在说什么：第 (i) 条解释了一个新手最常见的困惑——“我刚买完怎么就浮亏了？”因为你付出的是费用，不是亏损。第 (ii) 条给了你一个可以自己核对的口径：全清仓之后，账户权益的变化应当等于已实现盈亏，一分不差。

现在把第 chapter 十七 章算过的那笔成本放进来，你就能手算一个小例子。

例 29.3（一笔 50,000 元的往返，账面上到底差多少）。沿用第 chapter 十七 章的参数：佣金率 0.0003、最低佣金 5 元、印花税 0.0005（仅卖出）、冲击按平方根律。取标的 sh600519、当日收盘 1251.24¹ 元、 $\sigma = 0.014433$ ²、当日成交金额 1.33222×10^9 元。

买入 50,000 元：佣金 15.00、冲击 4.42，合计 19.42；卖出 50,000 元：佣金 15.00、印花税 25.00、冲击 4.42，合计 44.42。

若价格一个来回没有变化（ $p' = p$ ），则 $R_{\text{realized}} = 0 - 44.42 = -44.42$ 元；而现金账的变化是 $-19.42 - 44.42 = -63.84$ 元。两个数的差 19.42 元，正是被摊进了成本基准里的那笔买入费用。

也就是说：如果你以相同的价格买回又卖出，你会实打实地付出 63.84 元。这就是“交易本身是一种消耗”的确切数额。

持仓表在屏幕上的完整样子如下。

29.4 交易记录：六列，和一个容易看漏的“净额”

交易记录表列出每一笔成交：方向、数量、价格、佣金、净额、已实现盈亏。

前四列很好懂。真正容易看漏的是“净额”。

¹真实数据：本数字直接取自系统运行库 data/stock.duckdb，未做任何模拟或平滑。

²复算数据：该数字为本书按系统实现中的同一公式、在系统的真实数据上重新计算所得；公式出处已在正文标注。

³真实数据：本数字直接取自系统运行库 data/stock.duckdb，未做任何模拟或平滑。



图 29.1 组合交易中心：顶部四项指标卡与持仓明细表（图中的 5 只持仓与权重均为库中真实记录）

账户里真实发生的费用口径（系统口径）

- 买入：只收佣金（万三，最低 5 元），不收印花税；
- 卖出：佣金（万三，最低 5 元）加印花税 0.05%。

所以**净额** ≠ **成交额**：买入的净额高于成交额（多花的佣金），卖出的净额低于成交额（扣掉的佣金与印花税）。

为什么要单列一列“净额”？因为它是你和券商之间**真实结算**的那个数。成交额是名义上的，净额是钱。长期看，这一列的求和就是你为这套系统付出的摩擦总额。

上表那笔 50,000 元的买入，成交额是 50,000，净额是 50,015.00 元——多出来的 15 元，就是佣金。这个差额小到不会引起任何人的注意，而它每天发生。

29.5 盈亏分析：为什么“今日盈亏”以交易日为界

这是本章开头那个 3.5 元的完整答案。

在**盈亏分析** Tab 里，有收益历史、最新日统计，以及一条每日盈亏明细（时间倒序分页）。这套数据来自一个每天收盘后自动执行的动作。它的触发时机有一个硬性规定：

收盘价入库的触发规则

每日 **15:00** 之后自动执行收盘价入库，写入两张表：`position_snapshots`（持仓快照）与 `portfolio_daily_stats`（每日结算）。

“15:00 之后”这四个字，决定了“今日盈亏”这个数的一个性质：它是以交易日为界的，不

是以自然日为界的。

命题 29.4 (为什么“今日盈亏”在收盘后不应该继续刷新). 设收盘价为 P_t^{close} , 当日结算的“今日盈亏”为

$$\Pi_t = E_t - E_{t-1}, \quad E_t = \sum_i Q_i P_{i,t}^{\text{close}} + \text{现金}_t. \tag{29.5}$$

若在 15:00 之后继续以“最新价”刷新 E_t , 而市场已无新成交价, 则 $P_{i,t}^{\text{close}}$ 不再变化, Π_t 也不会变化; 此时任何“刷新”要么得到同一个数 (无意义), 要么会引入下一交易日的价格而污染 t 日的结算。

证明. 15:00 之后 A 股不再撮合 (第 chapter 二十三 章的交易时段), 故对一切 i 与一切刷新时刻 $\tau > 15:00$, 可得最新价格恒为 $P_{i,t}^{\text{close}}$, 刷新不改变 $\{P_{i,t}^{\text{close}}\}$, 因此 E_t 与 Π_t 都不变。

若此时改用下一个交易日的实时价 $P_{i,t+1}^{\text{real}}$, 则计算出的 $\sum_i Q_i P_{i,t+1}^{\text{real}}$ 属于第 $t+1$ 日的权益, 把它写进第 t 日的 Π_t , 等价于把 $t+1$ 日的波动提前计入 t 日——这正是第 chapter 十七章定义 17.5 意义上的前视污染: Π_t 用到了 $\mathcal{F}_{t+1}$ 的信息。■

这个命题在说什么: 它把“15:00 之后今日盈亏不再变化”这件看起来像 bug 的现象, 解释成了一条纪律。你在晚上十点打开软件, 看到“今日盈亏”还是下午收盘那个数——这是对的。它不该变。

29.6 “以数据库为准”到底是什么意思

现在讲这一章真正的方法论。

在持仓页的刷新逻辑里, 有一条注释是这样写的: 每次读取之前, 都要以 SQLite 里的权威数据强制对账内存持仓——权威数据是“position_snapshots 最近一条收盘快照, 加上快照之后的所有新成交”。

这句话对应一个具体的设计决定:

权威口径
内存里的持仓不是权威, 数据库里的是。 每次展示之前, 都用“最近一次结账 + 之后的全部流水”重算一遍余额, 把内存状态强行对齐到数据库。 并且实时刷新是每 30 秒从 SQLite 权威数据重建组合状态 (含持仓、成交、盈亏), 不依赖手动刷新。

这套设计防的是两类错:

第一类, 长期运行的漂移。 一个程序连续跑几周, 内存里维护的持仓数字会因为浮点累积、竞态写入、异常中断而慢慢偏离真实值。内存不会报警——它只是越来越不准。所以要让数据库定期“纠正”内存, 而不是反过来。

第二类, 结算值被覆盖。 每日结算写入的 daily_pnl 是含当日新开仓与手续费的; 若展示层用自己的口径重算, 就会和数据库那一行对不上。系统里对这一点有明确注释 (“那会忽略当日

新开仓/手续费影响导致与数据库结算值不一致”)——结论就是：展示层不许自己算，去读数据库那一行。

常见误解·“今日亏损”以自然日为界 这是本章开头那 3.5 元的另一种版本。

很多人习惯性地认为“今天”指的是零点到零点。那么在晚上十一点打开软件时，如果“今日盈亏”没变，就会怀疑软件坏了。

不是的。这个数字的分界线是 **15:00 收盘**，不是午夜。它之所以是这样，还有一条制度原因：A 股是 T+1，当日买入不可当日卖出；收盘之后，当天的交易已经彻底封闭，任何改动都只能属于下一个交易日。

所以正确的心理模型是：“今日盈亏”是一张日结单，不是一块实时仪表。想看实时波动，去看持仓明细里的“现价”和“盈亏”两列，那两列会随实时价刷新。

常见误解·净额 = 成交额 “我买了 50,000 块，成交额就是 50,000，所以扣 50,000。”

成交额确实是 50,000，但从账户上划走的是净额 50,015。这个差额在单笔上看不出什么，在一年几百笔之后就很不可观了——它和佣金率、笔数、以及“每次买卖的金额大小”直接相关（第 chapter 十七章命题 17.1 讲的“最低佣金”就藏在这里）。

系统把这个差额单独列成一列，就是为了让你在复盘的时候能一眼看到它。把佣金和印花税跟在成交额后面单独列出来看，是长期用这套系统唯一能坚持下去的记账习惯。

29.7 手动交易与“连续交易”

除了 AI 自动决策，持仓页上也提供了一个交易操作入口，包含一个手动交易表单：代码、方向、数量、价格、原因。

“原因”这个字段值得说一句：它默认填的是“手动交易”，但它是可改的。为什么一个手动操作还需要填原因？

因为这本账是给未来的你看的。三个月后你回看这笔交易，如果没有原因，你只能看到“在某个价格买了某个数量”——那和抛硬币没有区别。有原因，你至少知道当时在想什么，才有可能判断是“判断错了”还是“执行错了”。

页面另外还有一个实时持仓监控开关（默认开启），按设置页里的刷新间隔自动更新持仓行情；旁边是一个“立即获取实时行情并更新持仓价格”的按钮，用来做一次手动刷新。

本章对应的真实页面与数据

- 持仓 / 交易记录 / 盈亏分析三个 Tab、手动交易表单、实时持仓监控开关、30 秒权威重建：
`ui/src/pages/Portfolio.tsx`
- 持仓快照与每日结算：`internal/portfolio/engine.go` 的 `RecordDailySnapshot`、`position_snapshots`、`portfolio_daily_stats`
- 权威对账（最近收盘快照 + 其后新成交）：`internal/portfolio/engine.go` 的持仓重建逻辑
- 费用口径与净额：`OrderCostTool`、`internal/backtest/cost_model.go`（第 chapter 十七章）
- 交易时段与 T+1：`internal/util/trading_hours.go`（第 chapter 二十三章）
- 事故 #14（名称字段被错误赋值为代码）：系统事实清单第 15 节

动手试一试（十五分钟，最好在收盘后做）

1. 打开“我的投资”页，切到**持仓明细**。把“权重”这一列加起来，应该等于“持仓市值 ÷ 总资产”。若某一只超过 30%，那就是一条需要解释的记录。
2. 切到**交易记录**，找一笔卖出，把“成交额”和“净额”相减。这个差应当约等于“佣金 + 印花税”。计算一下： $\text{成交额} \times 0.0003 + \text{成交额} \times 0.0005$ ，和实际差对得上吗？
3. 切到**盈亏分析**。找到“今日盈亏”，记下它。等十分钟再看一次——如果现在是收盘后，这个数不应该变。
4. 做一道手算题：用定义 29.1 的式 (29.1)，假设你以 10.00 元买入 1,000 股（佣金按最低 5 元），算一算新的加权平均成本是多少。（答案应略高于 10.00。）
5. 最后回答：如果你在同一个价格把这 1,000 股全部卖出，这一趟你一共付出了多少费用？请用式 (29.2) 和例 29.3 的口径分别算一遍，核对两个数是否一致。

账本读懂了：持仓是余额，成交是流水，结账每天 15:00 之后写一次，权威永远在数据库那一侧。

可是上面的东西大多是被动地“看”。如果你想把旋钮拧一拧——换一个策略模板、改一改因子权重、看看某个参数到底让回测变了多少——那就得进实验室。下一章讲研究与调参：哪些旋钮能立刻看到读数，哪些要跑很久，以及改完之后必须做的那件事——**看样本外**。

研究与调参：选股、策略、回测、每日策略

30.1 我想自己动手改点东西

用了两周之后，我开始不满足了。

系统自己挑的股票没什么大毛病，可我总觉得“价值”那一项权重给得太轻——我是一个偏保守的人，更愿意为便宜和稳健付钱，而不是为势头付钱。

问题在于：我该从哪儿下手？

页面上的输入框有十几个：策略模板、市场范围、因子权重、策略参数、回测区间……它们看起来都差不多，改起来都一样容易。可它们有一个巨大的区别，这个区别第一次用的人完全看不出来：

有的旋钮，你动一下，五秒钟后就能看到读数变了；有的旋钮，你动一下，四十分钟后才知道结果。

如果不管这个区别，你就会不自觉得只去拧那些反馈快的旋钮——而它们恰恰不是最重要的那些。

30.2 实验室里的旋钮

直觉·先拧那个立刻能看到读数的 实验室里有一排旋钮。有的连着示波器，你一拧，屏幕上的波形立刻变；有的连着恒温箱，你一拧，要等二十分钟箱内温度才稳定下来。

有经验的人调试设备时有一个固定习惯：**先拧那个立刻能看到读数的**。

不是为了省时间，而是因为它的因果关系是清楚的。你拧了一格，波形动了，你就知道这个旋钮管什么。而恒温箱那个旋钮，你二十分钟后才看到结果，中间可能已经有别的东西变了——你没法确定是旋钮起作用了，还是恰好天气凉了。

调参最怕的就是这个：**你改了三样东西，结果变好了，你不知道是哪一样起了作用**。

所以本章按“反馈速度”把旋钮分成三档，从快到慢。

30.3 第一档：选股引擎（立刻出读数）

进“选股”页，最上面是三个策略模板：

表 30.1 三种选股模板

模板	代码名	它偏向什么
均衡	balanced	六因子按默认权重配，不特别偏
防御	defensive	更看重质量、低波动、盈利稳定
成长	growth	更看重动量、盈利增长

数据来源：系统事实清单第 9 节；系统源码 `internal/screener/smart_composer.go` 的智能组合因子模型与三种模板；核对时间 2026-09-24。

模板选中之后，下面有两个输入区：**市场范围**和**因子权重**。

因子面板上有六个因子（外加盘口），每个后面跟一个权重输入框。你可以把权重改成任何值——页面会用公式的形式把它渲染出来，形如

$$\text{Score} = 30\% \times \text{价值} + 20\% \times \text{质量} + 15\% \times \text{动量} + \dots \quad (30.1)$$

改完之后点“开始选股”，结果列表就出来了。这是**反馈最快的一档**：你不必等回测，几秒钟就能看到名次怎么变。

关于自定义权重的一条硬约束

自定义权重会先过一道校验（`ValidateCustomWeights`），不合法就不放行。这一道校验防的不是“算错”，而是“**算出来但不可解释**”——比如权重之和不为一、出现了负权重、或者某一项被写成 0 却仍在公式里占位。因为这些情况下的分数仍然能算出来，只是它不再是一张可以被读懂的排行榜了。

结果列表支持分页，点开任意一只股票可以看到它八个因子的分数条（一个因子一条进度条），另外还有一块 **AI 评分拆解**，把利好因子和风险因子分两栏列出来。

常见误解·自定义权重是在找“最优解” “我把权重调到让历史排名最靠前的那一组，就是最优权重。”

不是。自定义权重是**表达你的观点**的地方，不是优化器的输入。

这两件事的区别很具体：如果你按“历史表现最好”去找权重，那么你找到的是**在历史上最好的权重**——它精确地记住了这段时间里哪些因子有效，而下一段时期，因子有效性会轮换（第 chapter 十四章讲的因子衰减就是这个）。

正确用法是这样的：**先写下你的理由，再调数字**。比如“我本金小、承受回撤能力弱，所以我把低波动和质量的权重调高、动量的权重调低”。这个理由是可以在三个月后拿出来复核的——如果它错了，你知道该改的是理由，不是数字。

系统里其实已经给了你三个“带着理由”的模板（均衡、防御、成长）。除非你有额外的理由，否则用模板比调数字更安全。

选股引擎的操作界面是一段三步流程。

30.4 第二档：策略管理与指标刷新（几分钟）

“策略管理”页是一张表格，每行一条策略，列出：夏普、最大回撤、胜率。行尾有启用/停止的切换按钮；页面顶部有一个“刷新”按钮。

这里要讲清楚一件容易混淆的事：这个“刷新”不是在重新读数据库，而是在真的跑一遍回测。

```
1 // 刷新：触发后台真实回测刷新指标（而非仅重读 DB），
2 // 紧接着重拉列表展示最新值
```

代码 30.1 策略指标刷新按钮的真实行为（节选自 `ui/src/pages/Strategies.tsx`）

这段话在说什么：点一下“刷新”，后台会启动一次**样本池回测**（第 chapter 十七章讲的那



图 30.1 选股引擎：三步流程与三个策略模板（每个模板下方写着它真实的加权公式）

套：优先取当前持仓为样本池、每样本 1500 根 K 线、有界 worker 池并行、胜率按所有样本的已平仓交易聚合），跑完之后把夏普、最大回撤、总收益、胜率、换手这几个数写回策略表。所以：

- 它是慢的，因为真的在算；
- 它是会被自动执行的——每天盘后系统会自己刷一次；
- 它和“回测页”用同一套撮合与成本口径，所以两边数字应当一致。

为什么不让它每次打开页面就算一遍？因为那会让页面卡住。这也是事故 #10 的教训：早期版本回测要跑约 2 小时，根因是约 420 万次逐根 K 线的财报查询——其中绝大多数是无意义查询。修复方式是条件注入：只有策略实现了”需要财务数据“这个接口、且声明确实需要时，才去查财报。

从事故 #10 学到的一条工程判断

“慢”几乎从来不是数量问题，而是“不该算的算了”。

看到一次回测要两小时，第一反应不应该是“换个更快的机器”，而是“这 420 万次查询里，有几次是必须的”。

30.5 第三档：回测与网格扫描（几十分钟）

“回测”页给的结果最多，也最容易被误读。

（一）六个数。年化收益、夏普比率、最大回撤、胜率、盈亏比、交易次数。

（二）权益曲线。以及基于真实权益曲线算出来的三个诊断：月度收益矩阵（按月聚合区间

收益)、滚动夏普 (窗口 25 根 K 线、年化)、Calmar 比率 (总收益除以最大回撤的绝对值, 用于横向对比)。

(三) 参数网格扫描。选一个目标 (sharpe 或 return), 它会按 walk-forward 切分跑一组组参数, 然后给出样本内最优参数和这组参数在样本外的表现两个数。

这里我要把第 chapter 十七 章那个结论用另一种方式再讲一遍, 因为它是“调参”这件事的全部方法论。

命题 30.1 (样本外报告的无偏性). 设有 K 组候选参数 $\theta_1, \dots, \theta_K$ 。记第 i 组在样本内的估计为 I_i 、在样本外的真实表现为 O_i , 并设 $\{(O_i)\}$ 独立同分布、均值 μ 、且与选择过程独立 (即样本外数据没有参与选参)。若按样本内最优选参

$$\hat{\theta} = \theta_{\hat{i}}, \quad \hat{i} = \arg \max_{1 \leq i \leq K} I_i, \quad (30.2)$$

则

- (i) 样本外报告的期望无偏: $\mathbb{E}[O_{\hat{\theta}}] = \mu$;
- (ii) 样本内报告的期望有正偏: $\mathbb{E}[I_{\hat{\theta}}] = \mathbb{E}[\max_i I_i] \geq \max_i \mathbb{E}[I_i]$;
- (iii) 因此“用样本内数字报成绩”与“用样本外数字报成绩”得到的期望一般不同, 差就是选择偏置。

这个命题在说什么: 它给出了一条操作规则——选参可以用样本内, 报成绩必须用样本外。这不是道德要求, 是期望值的要求。

证明. (i) 由 I 与 O 独立, 随机指标 $\hat{i} = \arg \max_i I_i$ 是 I 的可测函数, 故 $\hat{i} \perp O$ 。于是

$$\mathbb{E}[O_{\hat{\theta}}] = \sum_{i=1}^K \mathbb{E}[O_i \mathbf{1}_{\{\hat{i}=i\}}] = \sum_{i=1}^K \mathbb{E}[O_i] \mathbb{P}(\hat{i}=i) = \mu \sum_{i=1}^K \mathbb{P}(\hat{i}=i) = \mu.$$

(ii) $\max_i(\cdot)$ 是凸函数 (作为线性函数的逐点上确界), 由 Jensen 不等式 $\mathbb{E}[\max_i I_i] \geq \max_i \mathbb{E}[I_i]$ 。取 $\iota = \arg \max_i \mathbb{E}[I_i]$, 则 $\max_i \mathbb{E}[I_i] = \mathbb{E}[I_{\iota}]$, 即“样本内期望最好的那一组”的期望; 而 $\mathbb{E}[\max_i I_i]$ 是“按每组自己的样本内读数挑出最优”的期望。两者之差即为选择偏置, 非负。■

安全调参的四步法

1. 一次只动一个旋钮。动两个你就不知道是谁起作用了。
2. 记下改动前的样本内 + 样本外两个数 (回测页都有)。
3. 改动后, 先看样本内变没变——没变说明这个旋钮对这条策略无关, 直接放弃, 别浪费时间跑样本外。
4. 只有样本外改善了, 才算这笔改动的收益。样本内改善而样本外恶化, 是命题 30.1 说的那种偏置在现形。

回测页把“跑过什么、结果如何”一次列全。



图 30.2 回测页：回测配置与最近结果列表（累计 73 次，表中数值为系统真实回测输出）

30.6 第四档：每日策略（每天一次）

最后一块，是把上面所有研究的结果接进第二天的工作流。

每天盘后，量化分析师会调用一个工具 `select_next_day_strategy`。它的作用只有一个：决定明天用哪条策略来管现有持仓。

如果你不指定，它会自动打分选优：

$$\text{StrategySelectionScore} = 0.4 \times \text{总收益} + 0.3 \times \text{夏普} + 0.3 \times \text{胜率}. \quad (30.3)$$

注意这个打分有一个限制条件：只在“活跃”的策略里选，且要求三项指标不全为 0。为什么要有这个限制？因为一个从未跑出过任何成交的策略，三项都是 0，它的“0 分”不代表它不好，只代表没有数据。把它排除在外，是为了避免“零成交的策略因为恰好并列最低而被选中”这种荒谬的结果——这和第 chapter 十七 章网格扫描里那条“0 成交的一律排到最后”是同一条思路。

选出来的结果写进 `daily_strategy_plans`（以交易日唯一），字段包括状态、已执行卖出数、以及来源：

- `source = quant`: 量化分析师显式指定；
- `source = auto`: 系统按式 (30.3) 自动兜底选优。

这个字段的价值在于归因。三个月后你回看某一天，能立刻知道“那天用的这条策略，是我特意选的，还是系统自己兜底挑的”。这两种情况的含义完全不同：前者代表一次主动判断，后者代表一次默认行为。

这份计划怎么被使用？第二天盘前，操盘手会读当天的活跃计划，对每一只持仓取最近 60 根日 K，跑一遍该策略的信号；若最新信号是卖出，就生成清仓卖单。而且这条卖出路径有两条特权（第 chapter 二十一 章讲过）：

- 策略卖出信号优先于市场状态机——判势再差也不会拦卖出；
- 不受六维仓位系数缩放——缩放只作用于买入。

常见误解·“我改完参数，回测更漂亮了，所以改对了” 这是调参这件事上最常见、也最难自察的错。

看命题 30.1(ii)：你按样本内指标挑参数，挑出来的那个读数天然带正偏。你改得越多、试得越多，偏得越多。所以在样本内看到的“更漂亮”，有相当一部分是你自己制造出来的，而不是参数带来的。真正的裁决只有一个：样本外。

而样本外还有一个更严格的要求——它必须没有被你反复看过。如果你反复用同一段样本外来做选择，那段样本外就退化成了样本内，偏置会重新长回来。这是量化里一个没有完美解的问题（称为“多重检验”），但至少有一条底线：每次改动都要记下来，包括那些你试了又放弃的。因为“试了多少组”决定了偏置有多大（第 chapter 十七 章命题 17.8(iii)）。

常见误解·参数越多，策略越精确 “我给策略加上十个参数，拟合得更准。”

参数越多，你在历史上能拟合得越准，同时在样本外的方差越大。这是同一件事的两面。

对这个问题，本书能给的实用建议只有一条：优先选参数少的策略。在系统内置的十七条策略里，双均线只有两个参数，超级趋势也是两个（周期、倍数）。它们不可能在回测里刷出惊人的数字，但它们也很难在实盘里给你惊吓。

第 chapter 十六 章那句“策略没有好坏，只有适用环境”，调参版的说法是：参数没有最优，只有“你有没有理由相信它”。

本章对应的真实页面与数据

- 选股页（三种模板、市场范围、六因子面板、自定义权重校验、分页与因子详情、AI 评分拆解、因子进度条）：`ui/src/pages/StockScreener.tsx`
- 权重校验：`internal/screener` 的 `ValidateCustomWeights`；模板与权重逻辑在 `internal/screener/smart_composer.go`
- 策略管理页（列表、夏普/回撤/胜率、启停、刷新）：`ui/src/pages/Strategies.tsx`
- 指标刷新（样本池回测、1500 根回看、worker 池、指标回写）：`internal/strategy/service.go` 的 `RefreshStrategyMetrics`
- 回测页（六指标、权益曲线、月度收益矩阵、滚动夏普、Calmar）：`ui/src/pages/Backtest.tsx`
- 网格扫描与 walk-forward：`internal/backtest/grid.go`
- 每日策略选择与打分：`internal/tools/daily_strategy_tool.go`、`daily_strategy_plans`
- 事故 #10（420 万次财报查询导致回测约 2 小时）、#6（symbol 前缀口径不一致导致配对策略 0 成交）

动手试一试（三十分钟，按反馈由快到慢）

- 1.（快）在选股页选“均衡”模板跑一次，把前五名抄下来。
- 2.（快）换成“防御”模板再跑一次。对比两份名单：哪几只换了？它们主要在哪几个因子上不同？（点开因子详情看进度条。）

3. (快) 把“低波动”权重从默认值手动调高一大截，再跑一次。观察名单变化，并写下你调它的理由。
4. (中) 切到策略管理页，点一次“刷新”。然后去做别的事，过几分钟回来看夏普/回撤/胜率有没有变化。
5. (慢) 切到回测页，对同一条策略跑一次参数网格扫描。把 `BestInSharpe` (样本内) 和 `OutSharpe` (样本外) 抄下来并排比较。两个数差多少，就是这次调参的“水分”。
6. (每天) 在 AI 决策页的【今日交易策略】卡片上，看今天用的是哪条策略、它的 `source` 是 `quant` 还是 `auto`。

到这里，你会读、会算、也会动手改了。但我一直绕开了一个问题没有正面回答：如果改错了，能撤回吗？

下一章讲安全开关。那里有三个长得很像、但按错任何一个后果都不同的按钮：模拟/实盘、暂停建仓、紧急停止。还有一个很容易被忽略的事实——模拟盘和实盘不是“参数区别”，是两套执行路径。

安全开关：模拟、实盘、暂停建仓与紧急停止

31.1 三个长得一样的开关

在设置页和风控页上，有三个开关或按钮，长得很像，位置也离得不远：

- 一个叫**启用弹窗**（它的另一面就是交易模式：开 = 模拟，关 = 实盘）；
- 一个叫**暂停建仓**；
- 一个叫**紧急停止**。

我第一次进设置页的时候，把它们当成同一类东西——都是“停下来”的开关。差别无非是停得多彻底。

这个理解是错的，而且错得挺危险。它们对应的是三件完全不同的事：一个是“这些单子到底发不发到真实世界”，另两个是“发出去之前拦不拦”。

按错其中一个的后果也不一样：按错了“启用弹窗”，你可能会在毫无提示的情况下真的下单；按错了“暂停建仓”，你会以为系统停了，其实它还在卖；按错了“紧急停止”，你可能会以为只停了买入，结果连止损都停了。

这一章把这三个开关分开讲清楚。

31.2 手刹、熄火键、空挡

直觉·车上的三个动作 开车的人知道有三个动作长得有点像，但用途完全不同：
拉手刹——车还能发动、还能挂挡，但它不走了。松开就恢复。这是“暂时不让它动”。
挂空挡——车在滑，发动机在转，但动力传不到轮子上。这是“切断连接”。
熄火——整台车停下来。想再走，得重新点火，还要重新走一遍检查。
你要是把手刹当空挡用，或者把空挡当熄火用，大多数时候不会出事——直到某一次，你需要的是熄火，而你拉的是手刹。

三个开关的对应关系是这样的：

表 31.1 三个安全开关的精确含义

开关	系统标识	对应	它到底做什么
交易模式	simulated / live	挂空挡	决定订单发往模拟撮合还是真实券商通道
暂停建仓	buyGate	拉手刹	在执行终层拦下所有买入，卖出与止损照常
紧急停止	emergencyGate	熄火	停止系统的交易活动

数据来源：系统源码 `internal/portfolio/engine.go` 的 `SetBuyGate / SetEmergencyGate`；`app_settings.go` 的 `SetTradingMode`；核对时间 2026-09-24。

31.3 第一件事：模拟和实盘不是“参数区别”

先破除一个最贵的误解。

模拟盘和实盘不是“同一个流程换了个参数”，它们是两条不同的执行路径。

模拟模式下，订单交给内部的撮合与记账引擎；实盘模式下，订单要经过券商通道（QMT）发到交易所。两条路径的差别不只是“真假”，还有成交价、成交时点、部分成交的可能性、以及失败重试——这些在模拟盘里都被简化掉了。

而这里藏着一个真实事故。系统的第 #8 条：

真实事故 #8（摘录自系统事实清单）

症状：切换交易模式后，实盘开关“静默失效”。

根因：SetTradingMode 重建执行桥之后，没有同步 CIO 引擎持有的 broker 引用。

修复：重建执行桥后同步 broker 引用。

这个事故值得慢慢讲，因为它是“配置与运行时状态不一致”这一类错误的教科书样例。

系统里有两个对象持有一个“往哪儿下单”的引用：

- **执行桥**（负责真的把订单发出去）；
- **CIO 引擎**（负责决策，并在决策之后调用执行桥）。

切模式的时候，代码重建了执行桥——新桥指向实盘。可是 CIO 引擎手里那个引用，还指着旧桥。于是从界面上看，模式已经是 live；从行为上看，CIO 把订单交给了那个还指向模拟撮合的旧桥。

结果就是：你按下的是实盘，走的是模拟。而它不报错——因为它确实执行了，只是执行到了另一个地方。

修复只有一行：重建执行桥之后，把新引用同步给 CIO。

这一类错误的通用形态

症状：开关拨了，但行为没变。

根因：开关改的是“配置”，而真正干活的对象持有的是“配置的一个拷贝”，拷贝没有跟着更新。

自查方法：任何“切换状态”的操作，都要问一句——有几个人持有了旧状态？它们都更新了吗？

顺带说一件界面上的事。设置页把交易模式呈现为一个叫“启用弹窗”的开关，它是这样映射的：

- 开启 = **模拟交易**：下单需要弹窗确认；
- 关闭 = **实盘交易**：不再弹窗，自动执行。

这个界面上的映射同时兼容了旧版本里叫 paper 的模式名。用“弹窗”而不是“模拟/实盘”作为界面语言，是一个刻意的选择：它把“安全性”翻译成了一个人能立刻感觉到的动作——多一次点击。

31.4 第二件事：暂停建仓，只拦买、放行卖

现在讲第二个开关，它是最常用、也最容易被误解的一个。

定义 31.1 (暂停建仓门控). 设 `buyGate` 是一个返回布尔的函数。当它返回真（处于暂停建仓状态）时，一切买入成交被拒绝，返回结构化错误；卖出与止损不受影响。

这段话在说什么：注意“只拦买、放行卖”这六个字。这不是权衡，这是设计。系统的实现注释把它写得很直白：

```
1 // buyGate 强制暂停建仓门控：返回 true 时禁止一切买入成交（卖出不受影响）。
2 // 由 App 接线到 Policy 暂停建仓开关，作为买入最终执行层防线。
3 func (e *Engine) SetBuyGate(fn func() bool) { ... }
4
5 // 注意：buyGate 仅拦截买入，不影响卖出，
6 // 从而实现“强制暂停建仓但允许离场/止损”的业务诉求。
7
8 // 强制暂停建仓门控：暂停建仓期间禁止一切买入（最终执行层防线）。
9 if e.buyStopped() {
10     return nil, fmt.Errorf("强制暂停建仓中，禁止买入：%s", instrumentID)
11 }
```

代码 31.1 `buyGate` 的实现意图（节选自 `internal/portfolio/engine.go`）

为什么必须这样设计？因为一个也会拦卖出的“暂停”，等于把所有风险敞口锁死在市场里。设想某天你担心市场，按下“暂停”——如果你需要一个能立刻离场的机制，而暂停把卖出也拦住了，那你就亲手给自己造了一个无法退出的仓位。这在 A 股还要叠加 T+1（第 chapter 二十三章）：当日买入当日不能卖，如果再加一层“暂停连卖出也拦”，你就彻底被锁住了。

所以“只拦买、放行卖”这条不对称，和第一部分第 chapter 二十一章讲的“只缩放买入、不缩放卖出”是同一条原则的两种实现：

贯穿全书的一条不对称

进场需要理由，离场不需要。

所以：仓位系数只缩放买入；暂停建仓只拦买入；风控的白名单只拦买入。

所有“收紧”的机制，都不许收在卖出的方向上。

31.5 第三件事：紧急停止

第三个开关是紧急停止，它比暂停建仓更彻底。系统里它和暂停建仓是两条独立的线路：

- `SetBuyGate`：暂停建仓，只拦买入；
- `SetEmergencyGate`：紧急停止，独立的门控函数。

为什么要分成两个而不是一个？因为它们的使用场景不同：

暂停建仓是“我先不买新的，但持仓继续被管理”——比如你判断今天市场不好，或者你正在观察某个新策略。此时卖出、止损、策略减仓全部照常，系统仍然在替你防守。

紧急停止是“这整套东西现在别动了”——比如你发现数据明显不对（行情库没更新、判势的输入是错的），此时你要的不是“少买一点”，而是先让它停下来，别做出任何基于错误数据的动作。

在界面上，这两种状态显示得也不一样：CIO 决策中心右上角的徽章把“紧急停止”显示为红色，“已暂停”显示为黄色。这个颜色差异是给你在三十厘米之外一眼分辨用的。

31.6 第四件事：还有多少道门

除了这三个开关，一个买入指令还要穿过好几道门。我把它按从“决策层”到“执行层”的顺序排出来。

表 31.2 一笔买入要穿过的门（自上而下）

层级	门	拦什么
决策层	股票池白名单	不在可交易池里的标的被剔除；池为空或读取失败则拒绝建仓（fail-closed）
决策层	六维建仓闸门	判势为“退潮风险”或 $r_{\text{pos}} \leq 0.15$ 时禁止生成新买入
主张层	主张 / 异议 / Kelly / 集成	各自给出上界，生效仓位取 $u^* = \min\{\dots\}$ （第 chapter 七 章）
规则层	风控硬限制	单票权重 $\leq 30\%$ 、组合最大回撤容忍 10%
执行层	buyRiskGate	下单前复核集中度 + 买入流动性拥挤度门控
执行层	盘口拦截	涨停封死买不进（排队）；买入拦截跌停板
终层	buyGate / emergencyGate	暂停建仓 / 紧急停止

自上而下经过；任何一道返回拒绝，这笔买入就不成立。注意所有道门只作用于买入，卖出只受“跌停封死卖不掉”这一条盘口拦截。

数据来源：系统事实清单第 7、8、11 节；系统源码 `internal/portfolio/engine.go` 的 `buyRiskGate`、`internal/orderbook/orderbook.go` 的 `Classify`；核对时间 2026-09-24。

这张表里有一处工程细节值得单独拎出来，它是本章唯一一条“形式化”的东西。

命题 31.2 (终层门控的覆盖性). 设“买入”这一动作在代码中只通过唯一一个汇聚点 B （即 `Engine.Buy`）落到账本上。若在 B 的入口处放置门控 g （返回真则拒绝），则

- (i) 任何一条上层的买入路径（自动决策、策略减仓的反向、手动交易、再平衡、QMT 通道）都必须经过 B ，因此都受 g 约束；
- (ii) 在不修改 B 的前提下，上层无法通过新增路径绕过 g ；
- (iii) 新增门控只会让“许可集合”变小：若 g_1, g_2 都是拒绝型门控，则

$$\{\text{通过}\}_{g_1 \wedge g_2} \subseteq \{\text{通过}\}_{g_1} \cap \{\text{通过}\}_{g_2} \subseteq \{\text{通过}\}_{g_1}. \quad (31.1)$$

这个命题在说什么：第 (i)(ii) 条解释了为什么这道门要放在最下面的执行层，而不是放在上面某一层——放在汇聚点，它才覆盖全部路径。第 (iii) 条是“只降不升”在最底层的形式：门只能加、不能减，多一道门只会让能通过的更少。

证明. (i) 由假设，“落到账本上的买入”与“调用 B ”是同一件事（ B 是唯一汇聚点）。任一条买入路径若要真的产生持仓，都必须调用 B ，故经过 g 。

(ii) 由 (i)，绕过 g 等价于绕过 B 。而绕过 B 意味着不经过唯一汇聚点就能落账——与 B 是唯一汇聚点的假设矛盾。

(iii) 设 $\mathcal{U}$ 为全部可能的上层买入请求。对 $\omega \in \mathcal{U}$, ω 通过 $g_1 \wedge g_2$ 当且仅当 $g_1(\omega) = \text{否}$ 且 $g_2(\omega) = \text{否}$ (“否”表示不拒绝, 即许可), 这蕴含 $g_1(\omega) = \text{否}$, 即 ω 通过 g_1 。故 $\{\text{通过}\}_{g_1 \wedge g_2} \subseteq \{\text{通过}\}_{g_1}$; 同理 $\subseteq \{\text{通过}\}_{g_2}$; 取交即得。■

这条命题的实用版本

门要放在“所有路都必须从这儿过”的那个地方。

放在决策层, 你防的是“AI 想不想买”; 放在执行层, 你防的是“钱能不能出去”。

两个都要, 但只有执行层那道是兜底的——因为决策层可能被绕过(新加一条路径、换一个调用方), 执行层不会。

31.7 第五件事：合规的两道防线

最后补两条“拦的是别的东西”的防线, 它们也是只拦买入:

第一道, 股票池白名单。买入必须过池; 标的不在 `tradeable_stocks` 里就被剔除。而池子为空或读取失败时, 系统拒绝自主建仓——这叫 fail-closed, 第 chapter 二十四 章的命题 24.5 已经证明过它为什么在期望意义上严格更优。

为什么这里要用 fail-closed 而不是“池子读不到就放行”? 因为这两种情况的含义完全不同: 池子读不到意味着你要么根本没做研究、要么数据出了问题; 而“放行”会让系统在一个未知的宇宙里下单。

第二道, 信号来源留痕。每一笔买入订单都会记录它的 `SignalSource` (这条信号是哪个策略给的)。这不是拦, 是记账——它让你在三个月后能回答“这笔买入是谁的主意”。

两道防线合起来回应了同一个问题: 这笔买入, 是系统自己有依据地做的吗? 池子回答“这个标的你有没有研究过”, 信号来源回答“这次买入是哪个策略触发的”。

常见误解·模拟盘和实盘只是换了个参数 这是本章开头那个误解的正面版本。

它们不是参数, 是两套执行路径。最直接的证据就是事故 #8: 切模式的时候, 如果只改了配置、没同步运行时引用, 界面显示实盘而实际走模拟——它不会报错, 因为你确实成交了, 只是成交在另一个地方。

所以判断“现在到底是不是实盘”, 不要看那个开关的位置, 要看成交记录落在哪里。模拟盘的成交只进本地账本; 实盘的成交会出现在券商侧。

一个更根本的建议: 在真正切到实盘之前, 先确认你对“这个开关现在是什么状态”有独立于界面的验证方式。

常见误解·先在实盘上小仓位试试 “我就投一万块试试, 亏了也不心疼。”

问题不在于亏多少钱, 而在于你试的是流程, 不是收益。而你恰恰要用实盘去试的, 是那些只有实盘才会暴露的东西: 订单是不是真的发出去了、模式切换有没有生效、盘口拦截在不同涨跌停状态下是不是按预期工作、收盘后快照有没有正确落库。

这些环节, 在模拟盘上会被简化掉。所以“小仓位实盘试”试不出来它们, 它只是把风险从“大额”降到“小额”, 同时把“流程缺陷”暴露在一个你更不希望它出现的地方。

正确的做法是: 先用模拟盘跑满一个完整周期——包含一次盘前、一次盘中、一次盘后结算、以及一次夜间复盘。然后把这份记录的每一个时间点, 和你的预期核对一遍。全对了, 再谈实盘。

本章对应的真实代码与事实

- 交易模式设置与执行桥重建： `app_settings.go` 的 `SetTradingMode`、`SetBrokerConfig`、`SetQMTConfig`
- 暂停建仓门控与紧急停止门控： `internal/portfolio/engine.go` 的 `SetBuyGate`、`SetEmergencyGate`、`buyStopped`、`Engine.Buy` 终层阻断
- 执行端风控： `internal/portfolio/engine.go` 的 `buyRiskGate`（集中度复核 + 流动性拥挤度门控）
- 盘口五态与买卖拦截： `internal/orderbook/orderbook.go` 的 `Classify`
- 合规两道防线： `tradeable_stocks` 白名单（fail-closed）、每笔买入的 `SignalSource`
- 风控硬限制统一口径：单票 0.30、回撤 0.10（第 chapter 二十二 章）
- 事故 #8（切模式未同步 broker 引用）、#7（硬限制三处数值不一致）

动手试一试（十分钟，建议在模拟盘状态下做）

1. 打开设置页，找到“启用弹窗”开关。确认它的位置，并且记住：它的反面就是交易模式。切换前，先在心里回答一次“我现在要的是模拟还是实盘”。
2. 打开风控页或 CIO 页，找到“暂停建仓”。开启它。
3. 回到持仓页，手动提交一笔买入：你应该被拒绝，并且看到一条明确的错误信息（而不是“静默失败”）。
4. 在同一状态下提交一笔卖出：它应当被放行。这一步是本章最关键的实验——它验证了“暂停建仓只拦买、放行卖”。
5. 关闭“暂停建仓”，确认买入恢复正常。
6. 最后做一个判断题：如果某天你要出门一整天，既不希望它开新仓、又希望持仓能正常止损离场，你会按哪个开关？请说出理由。

到这里，你已经能装、能读、能改、也能紧急刹住了。

剩下最后一个话题，也是我个人认为这套系统里最像“活物”的一部分：它每天会把自己的经验写回明年的行为里。今天亏了，明天它不会用同一套权重；今天某个因子失效了，明天它的名字会自动往后排。

下一章讲这个闭环：复盘 → 改权重 → 换策略。以及一件很重要的事——如果复盘结论不影响它明天的行为，那复盘就只是写日记。

每天进步一点的机制：复盘 → 改权重 → 换策略

32.1 一本写了三周没人看的日记

系统跑起来之后，我做过一件很典型的事：每天收盘后都去看它的复盘报告。

看了一周，感觉不错——“今天的亏损主要来自持仓集中在同一个板块”“动量因子今日表现偏弱”——写得很像那么回事。

看到第三周，我忽然意识到一个问题：

我看了三周复盘，可我根本不知道这些结论第二天去了哪里。

于是我做了一个测试。我把连续三天的复盘报告里提到的“动量因子偏弱”这一句记下来，然后去看第二天的选股结果——动量因子的权重，一模一样。

那一刻我明白了一件事：如果一个系统的复盘结论不影响它明天的行为，那复盘就只是写日记。

这一章讲的就是“复盘怎么变成明天的动作”。三个出口，一条纪律。

32.2 看比赛录像的目的

直觉·看录像不是为了感慨昨天 一个职业运动员赛后要看好几遍比赛录像。

他看录像的时候，不是在感慨“昨天那一球真可惜”。他在做一件很具体的事：把视频里的某一段，翻译成明天训练场上多加的两组动作。

如果看完录像，明天的训练内容和昨天完全一样，那这两个小时的录像分析就没有发生任何作用——不管他当时的感受多么深刻。

复盘的价值，100% 体现在“明天的参数和今天不一样”这一件事上。

系统的复盘闭环有三个出口，它们分别把不同种类的“昨天”翻译成“明天”：

表 32.1 复盘闭环的三个出口

出口	昨天的输入	明天的输出	机制
权重	因子的 IC / RankIC / 分半稳定性	次日选股的因子权重	因子质量分 → factorWeightsFor
策略	样本池回测的收益 / 夏普 / 胜率	次日由哪条策略管持仓	RefreshStrategyMetrics → daily_strategy_plans
股票池	全市场最新真实数据	次日的可交易池	方案每日重建 → tradeable_stocks

数据来源：系统事实清单第 9、10、12 节；系统源码 `internal/tools/factor_review_tool.go`、`internal/strategy/service.go`、`app_planner.go`；核对时间 2026-09-24。

下面逐个说，最后讲那条纪律。

32.3 出口一：复盘 → 权重

这个出口的链条最长，但每一步都短：

$$\text{因子复盘} \rightarrow \text{质量分 } q_k \in [0, 1] \rightarrow \text{倍数 } \text{mult}_k \rightarrow \text{次日权重 } w_k. \quad (32.1)$$

第一步是 `factorReviewTool` (第 chapter 十四章)：逐日算 IC、RankIC、分组表现、覆盖率、前瞻准确率、稳定性、衰减，再合成一个质量分 $q_k \in [0, 1]$ ，按因子名 UPSERT 进 `FactorQuality` 表。

第二步在 `factorWeightsFor(regime)` 里：取所有因子质量分的中位数 q_{med} (不是均值，第 chapter 十三章讲过原因)，然后算每一步的倍数。

第三步是最值得看的一处：权重不是从上一天的权重迭代出来的，而是每天从基准权重重新算一次。

定义 32.1 (权重映射). 记模板的基准权重为 $b_k > 0$ 、因子质量分为 q_k 、中位数为 q_{med} 。定义

$$\text{mult}_k = \text{clamp}\left(\frac{q_k}{q_{\text{med}}}, 0.5, 1.5\right), \quad w_k = \frac{b_k \text{mult}_k}{\sum_j b_j \text{mult}_j}. \quad (32.2)$$

对被判定为真实不稳定的因子 (第 chapter 十五章：分半不稳定且样本 ≥ 30)，额外施加

$$\text{mult}_k \leftarrow \underbrace{\min(\text{mult}_k, 0.6)}_{\text{先封顶}} \times \underbrace{0.5}_{\text{再折半}}. \quad (32.3)$$

命题 32.2 (权重映射的三条性质). 沿用定义 32.1。

(i) 归一： $\sum_k w_k = 1$ ，且 $w_k > 0$ ；

(ii) 相对权重有界：对任意 k, l ,

$$\frac{w_k}{w_l} = \frac{b_k}{b_l} \cdot \frac{\text{mult}_k}{\text{mult}_l} \implies \frac{w_k}{w_l} \in \frac{b_k}{b_l} \cdot \left[\frac{1}{3}, 3\right]; \quad (32.4)$$

(iii) 不稳定因子的强降权：若因子 k 被施加式 (32.3)，则 $\text{mult}_k \in [0.25, 0.30]$ 。于是相对于任一未被降权的因子 l ($\text{mult}_l \in [0.5, 1.5]$)，

$$\frac{w_k}{w_l} = \frac{b_k}{b_l} \cdot \frac{\text{mult}_k}{\text{mult}_l} \leq \frac{b_k}{b_l} \cdot \frac{0.30}{0.50} = 0.6 \cdot \frac{b_k}{b_l}, \quad (32.5)$$

相对于质量最好的稳定因子 ($\text{mult}_l = 1.5$) 更是压到 0.2 倍。

这个命题在说什么：第 (ii) 条保证权重不会失控——一天之内，两个因子的相对权重最多被改变 3 倍，而不会出现“某个因子今天权重 0.05、明天 0.9”这种跳变。第 (iii) 条是那个“让不稳定因子自然退场”机制的确切数值：被判不稳定的因子，权重最多只剩基准的 60%；如果它对面站的是一个质量最好的因子，就只剩 20%。

注意它不被清零。这一点是有意的：清零意味着“永久淘汰”，而分半检验只是一个基于当

前样本的判断；样本会变，判断也会变。留一个有余量的降权，比一刀切掉更符合统计诚实。

证明. (i) 由式 (32.2) 的定义， w_k 是 $b_k \text{mult}_k$ 归一化后的结果，故 $\sum_k w_k = 1$ 。又 $b_k > 0$ 、 $\text{mult}_k \in (0, \infty)$ （clamp 的上下界都是正数），故每项为正， $w_k > 0$ 。

(ii)

$$\frac{w_k}{w_l} = \frac{b_k \text{mult}_k / \sum_j b_j \text{mult}_j}{b_l \text{mult}_l / \sum_j b_j \text{mult}_j} = \frac{b_k \text{mult}_k}{b_l \text{mult}_l} = \frac{b_k}{b_l} \cdot \frac{\text{mult}_k}{\text{mult}_l}.$$

由 $\text{mult} \in [0.5, 1.5]$ ，比值 $\text{mult}_k / \text{mult}_l$ 的取值范围是 $[0.5/1.5, 1.5/0.5] = [1/3, 3]$ 。

(iii) 由式 (32.3)：未降权时 $\text{mult}_k \in [0.5, 1.5]$ ，故 $\min(\text{mult}_k, 0.6) \in [0.5, 0.6]$ ，乘以 0.5 得 $\text{mult}_k \in [0.25, 0.30]$ 。代入 (ii) 的比值式，并用 $\text{mult}_l \geq 0.5$ 放缩，

$$\frac{w_k}{w_l} = \frac{b_k}{b_l} \cdot \frac{\text{mult}_k}{\text{mult}_l} \leq \frac{b_k}{b_l} \cdot \frac{0.30}{0.50} = 0.6 \frac{b_k}{b_l}.$$

取 $\text{mult}_l = 1.5$ 得 $\leq 0.30/1.5 = 0.2$ 倍。■

为什么“每天从基准重算”比“从上一天迭代”稳

若把式 (32.2) 改成迭代 $w^{(n+1)} \propto w^{(n)} \text{mult}$ ，则

$$w^{(n)} \propto w^{(0)} \prod_{i=1}^n \text{mult}^{(i)},$$

时间一长，权重会全部集中到倍数最大的那个因子上，其余因子趋于零——这叫“权重塌缩”。而每天从基准重算，等价于每隔 24 小时清一次账：质量分的信息被用上，历史累积不会失控。这条设计让系统的权重有一种“回归基准”的自动纠偏。

32.4 出口二：复盘 → 策略

第二条链条是第 chapter 三十 章讲过的那一套，这里只补它的“复盘”属性。

每日盘后，系统自动做两件事：

1. 调用 `RefreshStrategyMetrics`——以样本池跑一遍回测，把夏普、最大回撤、总收益、胜率、换手写回策略表；
2. 调 `select_next_day_strategy`——按

$$\text{Score} = 0.4 \times \text{总收益} + 0.3 \times \text{夏普} + 0.3 \times \text{胜率} \quad (32.6)$$

选出一条“明天的策略”，写进 `daily_strategy_plans`。

它有一个 `source` 字段，取值 `quant`（量化分析师显式指定）或 `auto`（系统自动兜底）。这两个值的区别，就是“这是不是一次复盘驱动的结果”：

- `source=auto`：由今天的回测结果自动选出来的——这是纯粹的复盘闭环；

- `source=quant`：由量化分析师 ** 手动 ** 指定——这是一次人为干预。

两种都合法，但你在复盘时应该区分它们。连续多日都是 `auto`，说明系统的自动闭环在正常工作；突然出现一个 `quant`，值得问一句“那天发生了什么”。

32.5 出口三：复盘 → 股票池

第三个出口最短，但它决定了另外两个出口的作用范围。

系统的投资方案每天自动重建一次：

方案每日重建的规则

`PlanRefreshProcessor.RebuildInvestmentPlan` 用最新的全市场真实选股重建方案的持仓构成并落库，
并且保留原来的任务书、风险等级与状态——也就是说，它不新建方案，只更新内容。

“保留原任务书、不新建方案”这一条很重要。它意味着你的投资者画像（12 维问卷那套）不会被每天重建掉——你三个月前填的“可接受最大亏损”不会因为系统跑了一次全市场选股就变成默认值。

重建出来的股票池落到 `tradeable_stocks`，而这张表正是第 chapter 三十一章那道“买入白名单”的来源。于是三个出口串成了一条完整的链：

复盘（因子/策略）⇒ 参数（权重/策略）⇒ 池子（明日可买什么）⇒ 明天的决策 ⇒ 明天的复盘

32.6 那条纪律：幂等标记

上面三件事都是“每天做一次”的。而“每天做一次”在软件里有一个非常具体的麻烦：什么算“今天”？

答案是：一个写在配置里的日期。

两个持久化的幂等标记（`internal/config/config.go`）

- `last_daily_review_date`：最近一次每日复盘生成的日期（格式 2006-01-02）；
- `last_plan_refresh_date`：最近一次每日投资方案自动重建的日期。

两者都持久化到 `config/config.json`，启动时恢复。

定义 32.3 (幂等触发). 设动作 A 与标记 L （取值是一个日期或空）。触发规则为

$$\text{执行 } A \iff L \neq \text{today}; \quad \text{执行后 } L \leftarrow \text{today}. \quad (32.7)$$

命题 32.4 (幂等触发保证“每个交易日至多一次”). 在定义 32.3 的规则下，并设进程可以在任意时刻重启、且 L 被持久化：

(i) 对固定的 today ， A 至多被执行一次；

- (ii) 若 L 只保存在内存中（不持久化），则存在一组进程重启序列使 A 在同一天被执行 m 次（ m 任意大）；
- (iii) 若把 L 写回配置的写法是“读一改一整文件覆盖”，则存在并发或崩溃窗口使已写入的 L 丢失。

这个命题在说什么：第 (i) 条是它本来该有的效果——**重启不重跑**。第 (ii) 条是真实事故 #22：幂等标记只在内存里，重启一次，“明天”就跑了两遍。第 (iii) 条是事故 #9：配置文件整文件覆盖写，一旦在写的过程中崩溃或与另一个写入方并发，已经写进去的标记可能丢失——而丢失的后果，还是“同一天跑两遍”。

证明. (i) 设 today 固定为 t 。若 A 在某时刻被执行，则该时刻的 $L \neq t$ ；执行后按规则 $L := t$ 。此后任何再次触发都在检查 $L = t$ ——条件 $L \neq t$ 恒假，故不再执行。由 t 固定，整个 today 内 A 至多一次。

(ii) 设 L 只在内存。取进程重启序列：第一次启动后触发 A （ L 从“空”变为 t ，执行一次）；随后重启，内存清零 L 变回“空”；再次触发，条件 $L \neq t$ 成立， A 再执行一次。每重启一次即多执行一次， m 次重启得 m 次执行。故存在使 A 执行 m 次的序列，且 m 可任意大。

(iii) “读一改一整文件覆盖”是一次非原子的复合操作。若在“读出旧内容”与“写回新内容”之间发生崩溃，则文件保持旧内容（ L 丢失）；若两个写入方并发执行这一复合操作，后写者会覆盖前写者写入的 L （丢失更新）。两种窗口都存在，故已写入的 L 可能丢失。■ ■

对应的修复是**原子替换**：先把新内容写进一个临时文件，然后用一次重命名（`os.Rename`）把它换上去。重命名在同一文件系统内是原子操作，所以观察者看到的文件**要么是旧的完整内容、要么是新的完整内容**，不存在“写了一半”的中间态。

常见误解·复盘的产出是“一份报告” 这是我一开始的理解，也是大多数人默认的理解：复盘就是产出一份写得很好的文字总结。

看命题的定义就清楚了：复盘真正需要的产出，是**下一日的参数变化**。具体到三个出口就是：

- `FactorQuality` 表里那几个数字变了没有（质量分、是否稳定）；
- `daily_strategy_plans` 表里今天有没有新的一行，谁的 `source`；
- `tradeable_stocks` 表有没有按最新数据重建。

如果这三样都没变，那么今天这份复盘报告，无论写得再好，都没有参与这个系统的运行。

一个可操作的自检办法：每天看完复盘之后，问自己一句——“明天会有什么和今天不一样？”如果答不出来，那今天这份就只是日记。

常见误解·幂等标记是个小细节 “不就是记一个日期吗。”

记一个日期确实简单。难的是“**进程会在任意时刻重启**”这个前提——而这个前提在真实使用中是必然的：你会关机、会点托盘退出、会升级、会崩溃后重启。事故 #22 的完整形态就是：标记只在内存里 → 晚上重启一次 → 当日复盘与全市场方案重建各跑了两遍 → 方案被重建两次，中间那次用的还是旧数据。

第 (iii) 条更隐蔽：即便你写进了配置文件，只要写法是“整文件覆盖”，它仍然可能丢。所以这类标

记的正确实现是两件事一起做：持久化，且原子替换。

本章对应的真实代码与数据

- 因子复盘与质量分落库：`internal/tools/factor_review_tool.go`、`FactorQuality`
- 权重映射与降权：`factorWeightsFor(regime)`、`medianFactorQuality`（第 [chapter 十三](#)、[chapter 十五](#) 章）
- 策略指标刷新与每日策略：`internal/strategy/service.go` 的 `RefreshStrategyMetrics`、`internal/tools/daily_strategy_tool.go`、`daily_strategy_plans`
- 方案每日重建：`PlanRefreshProcessor.RebuildInvestmentPlan` → `app_planner.go` → `RebuildCurrentPlanPortfolio`
- 幂等标记：`internal/config/config.go` 的 `LastDailyReviewDate`、`LastPlanRefreshDate`
- 事故 #21（因子质量分未接入权重）、#22（幂等标记只在内存 → 重启重跑）、#9（配置整文件覆盖写 → 原子替换）

动手试一试（十五分钟，最好跨两天做）

1. 今天收盘后，看一眼选股页展示的权重公式。把它抄下来（记下“第 1 日权重”）。
2. 切到 AI 决策页，找到【今日交易策略】卡片，记录今天的策略名和它的 `source` 值。
3. 明天同一时间，再做一次上面两步，得到“第 2 日权重”。
4. 对比两份权重：有哪几项变了？变化幅度是否落在 $\pm 50\%$ 以内（命题 32.2(ii)）？
5. 打开 `config/config.json`，找到 `last_daily_review_date` 与 `last_plan_refresh_date`，看它们的值是不是今天。然后重启一次程序，再看一次——两个值不应该变，也不应该有第二次复盘被触发。
6. 最后回答：如果你想让某个因子“暂时别参与”，有哪两种做法？它们分别属于三个出口里的哪一个？

系统会复盘、会改权重、会换策略、也会记住“今天已经做过”——它像一个每天都在调整自己的活物。

而活物也会病。最后一步不是继续加功能，而是回答一个最难的问题：当它出问题的时候，你怎么知道？去哪儿找？

下一章是全书最后一章，讲日志、黑匣子与常见故障。里面有一个我修了很久才修好的故事——一次日志里什么都没写的闪退。

出问题怎么办：日志、黑匣子与常见故障

33.1 一次日志里什么都没写的闪退

系统稳定跑了几天后，出了第一个真正的怪事。

下午两点四十分，我正在看持仓页。窗口“啪”地消失了。托盘图标也没了。进程干干净净地退出。

我第一件事是打开日志目录，找那条最新生成的 `log/quantbot_< 时间戳 >.log`。翻到末尾——记录停在两分钟前的一句“刷新持仓行情”，后面什么都没有。

没有异常、没有堆栈、没有“exit”。

我又去看 Windows 事件查看器，想找一条应用程序错误。空的。

那是整件事最让人不安的地方：一个程序死了，而它没有留下任何关于自己为什么死的记录。这不像“报错了”，更像“被抹掉了”。

后来我们把原因找出来了。它不是玄学，是一段非常具体的代码——而且它精确地展示了本章要讲的三件事：为什么致命的错误会“静默消失”、该怎么给它专门造一条通道、以及拿到转储之后怎么复原现场。

33.2 黑匣子的读法

直觉·先找记录装置，再按时间轴复原 一架飞机失事后，调查组的第一步不是猜“可能哪里坏了”，而是去找黑匣子。

找到之后他们做什么？不是从头读一遍全部录音。他们做三件事：

第一，确认记录装置本身是好的（别读一份损坏的磁带）；**第二**，按时间轴定位到最后一个异常点；**第三**，往前后各展开一点，看“异常之前发生了什么正常的操作”。

这套读法的前提只有一个：那个装置必须存在，而且它必须能把最后那几秒写下来。

程序排障是同一件事。所以第一个问题永远不是“哪儿错了”，而是——它把错误写在哪里了？

33.3 为什么致命错误会“静默消失”

先讲清楚本章开头那个现象。它是一个纯粹的语言运行时机制问题，没有任何神秘色彩。

Go 语言里有两类“崩溃”：

- **普通 panic**：可以用 `recover` 接住，也可以被日志框架记录；
- **运行时致命错误**（fatal error：栈溢出、内存越界导致的段错误等）：它不会被 `recover` 捕获，直接终止进程。

关键在于第二类写到哪去了。它输出到**标准错误流**（`os.Stderr`）。而一个图形界面程序没有控制台窗口——标准错误没有人接，写进去就消失了。

于是就有了那个现象：进程死了，日志干净，事件查看器干净。

命题 33.1 (无痕故障的概率下界). 设系统在单位时间内发生致命错误的概率为 $p > 0$ ，且条件独立地，任一错误被写进某个可观测通道的概率为 $q \in [0, 1]$ 。则

(i) “错误发生且无痕迹”的概率为

$$\mathbb{P}(\text{无痕故障}) = p(1 - q); \quad (33.1)$$

(ii) 当 $q = 0$ （没有任何通道接收致命错误）时，该概率恰为 p ——所有崩溃都无痕；

(iii) 系统连续运行 T 个时间单位，至少发生一次无痕故障的概率为

$$1 - [1 - p(1 - q)]^T \xrightarrow{T \rightarrow \infty} 1. \quad (33.2)$$

这个命题在说什么：它把“概率极小的崩溃在 7×24 运行的系统里是必然事件”这句话变成了一个极限。第 (ii) 条更狠—— $q = 0$ 时，“发生”就等于“无痕”，你没有任何可观测性，不是可观测性变差了。

证明. (i) 设事件 $E =$ “发生致命错误”， $\mathbb{P}(E) = p$ ；事件 $R =$ “被记录”， $\mathbb{P}(R | E) = q$ 。则

$$\mathbb{P}(\text{无痕}) = \mathbb{P}(E \cap R^c) = \mathbb{P}(E)\mathbb{P}(R^c | E) = p(1 - q).$$

(ii) 代入 $q = 0$ ： $p(1 - 0) = p$ 。

(iii) 设 T 个单位时间彼此独立（故障发生的独立性假设），每个单位时间内“无痕”的概率为 $\rho := p(1 - q)$ 。 T 个单位全部没有无痕故障的概率为 $(1 - \rho)^T$ ，故至少一次的概率为 $1 - (1 - \rho)^T$ 。当 $\rho > 0$ 时 $(1 - \rho)^T \rightarrow 0$ ，故其趋于 1。■

这个命题给出的行动项非常明确：你无法把 p 降到 0（那是硬件的噪声底），但你可以把 q 提到 1。而把 q 提到 1，只需要一行代码——把致命错误的输出重定向到一个文件：

```
1 // 把 Go 运行时致命错误（写 os.Stderr，不经 log 包）重定向到文件，
2 // 否则 GUI 程序里没有人接收，崩溃现场会被静默丢弃。
3 cf, _ := os.Create(filepath.Join(exeDir, "log", "crash_dump.txt"))
4 debug.SetCrashOutput(cf, debug.CrashOptions{})
```

代码 33.1 把致命错误的输出重定向到文件（节选自 main.go 的 setupLogging()）

这段话在说什么：它没有阻止崩溃，也没有让崩溃变少。它只做了一件事——把原来掉在地上的东西，接进了一个桶里。这一行是本章里性价比最高的一行代码。

33.4 那个段错误的完整故事

现在讲本章开头那次闪退的真正原因。它在事实清单里是第 #1 条，也是全书我认为最值得完整复述的一次修复。

真实事故 #1（完整形态）

症状：程序运行中突然闪退，日志干净，Windows 事件日志无记录。

根因：决策完成后调用 `agent.dll.Release()` 卸载了这个 DLL；而它（一个 c-shared 的 Go DLL）内部还有后台 `goroutine` 在运行。卸载导致这些线程继续执行“已经被释放的内存空间里的代码”，触发 `ACCESS_VIOLATION (0xC0000005)`。

修复：不再卸载 DLL，让它驻留整个进程生命周期。

为什么这个错误“无痕”？因为它是一个访问违例——它在 Go 运行时的层面是致命错误，按命题 33.1(ii)， $q = 0$ ，所有痕迹都被丢掉了。

为什么它“偶发”？因为它需要两个条件同时成立：卸载发生了，并且那个后台线程恰好还在跑。第二个条件带时序，所以它有时崩、有时不崩——这类“偶发”最容易被误判成玄学。

命题 33.2 (卸载后使用是这一类崩溃的充分条件). 设模块 M (DLL) 被映射到地址区间 $\mathcal{A}$ ，并在其内部创建了线程 τ 。

- (i) 若 τ 仍在运行，而 M 被卸载，则 $\mathcal{A}$ 被解除映射（随后可能被分配给其它模块）；此时 τ 的指令指针一旦落回 $\mathcal{A}$ 范围，取指将访问不再是 τ 所属代码的页面，产生访问违例。
- (ii) 若不卸载 M ，则 $\mathcal{A}$ 在整个进程生命周期内保持映射， τ 无论何时被调度，其指令指针始终指向有效且正确的代码，上述崩溃不可能发生。

这个命题在说什么：它把“为什么不能卸载”和“为什么不再卸载就好了”写成了一对互补的陈述。(i) 是崩溃的成因链；(ii) 说明修复不是“降低概率”，而是移除了发生条件。

证明. (i) 卸载 M 意味着操作系统把 $\mathcal{A}$ 从进程的地址空间中移除，并回收这些物理页。此后 τ 若被调度器选中，CPU 要用 τ 保存的指令指针 $ip \in \mathcal{A}$ 去取指令。由于 ip 落在未映射区域（或已映射给别的数据），该次取指不可能是 M 原有的正确指令：要么触发缺页异常且无法被合法处理，要么取到已经变成别的内容的字节。二者都会让 τ 进入一个非法的执行状态，表现为访问违例（0xC0000005）。这里的关键点是： ip 落在 $\mathcal{A}$ 内是充分条件——不需要知道那块内存被谁占了。

(ii) 若不卸载，则 $\mathcal{A}$ 与 M 的代码、数据在整个进程生命周期内一一对应且保持不变。 τ 的指令指针无论何时取指，读到的都是 M 被编译时的正确指令；访存地址也都落在仍然有效的 M 的段内。故 (i) 中描述的条件无法成立，该崩溃不发生。■

这次修复之后，同一个流程再没有出现过同类闪退。它留下的教训可以写成一则通用经验：

一条通用经验：谁创建的线程，谁负责它的生命周期

卸载一个模块的前提是确认它创建的所有执行流都已结束。做不到这一点时，正确做法是不卸载——让它在进程里待着，代价是一点内存，收益是消除一整类不可观测的崩溃。

33.5 故障的三种类型，和三种查法

修完那一次之后，我给自己立了一条规矩：**不要一上来就猜原因，先分类**。因为三类故障的查法是**完全不同的**，混着查只会浪费时间。

表 33.1 三类故障的判别与查法

类型	特征	先看哪里	处理方式
配置类	必现、启动即错、文案明确	界面弹窗 / 启动日志	读错误信息，改配置，重启
数据类	必现、某功能不可用	<code>sources</code> 字段 / 数据维护页	停下来 ——不降级、不伪造（第 chapter 二十 章）
并发/内存类	偶发、无日志、直接消失	<code>log/crash_dump.txt</code>	读转储里的栈，找出“创建了但没被收尾的执行流”

数据来源：系统事实清单第 15 节事故分类；系统源码 `main.go` 的 `setupLogging()`；核对时间 2026-09-24。

三类里最容易误判的是第二类。它表现得很像“功能坏了”，于是人的第一反应是“先让它能跑”。但第 chapter 二十 章已经讲过：“能跑但数据是假的”比“跑不起来”危险得多。所以数据类故障的正确反应是**停下来**，然后去看它为什么缺。

33.6 现场清单：出事了先去哪几个地方

下面这张表是我实际会按顺序查的清单。它的顺序就是“从最近发生的、到最底层的”。

表 33.2 排障现场清单（按查看顺序）

顺序	位置	看什么
1	<code>log/quantbot_< 时间戳 >.log</code>	每次启动生成一个；按时间找最后几条正常记录，看它们卡在哪一步
2	<code>log/crash_dump.txt</code>	致命崩溃的堆栈 （命题 33.1 的修复产物）； 闪退必读
3	启动十步日志	[Step 1/10] 到 [Step 10/10]，一眼看出“卡在哪一步”
4	启动加载页的六项状态	哪一盏灯是红的；关键组件失败会弹窗（第 chapter 二十四 章）
5	审计日志页	决策、审批、订单的留痕（第 chapter 十 章的黑匣子）
6	<code>deliverables/< 日期 >/</code>	双存储的交付物 JSON
7	<code>agent_task_logs</code> 表	每个任务的状态与输出，含 SKIPPED 与失败的 <code>phaseErrs</code>
8	<code>market_sixdim_daily</code> 的 <code>sources</code>	判势当天的每个数字从哪来、有没有降级（第 chapter 二十 章）
9	<code>config/config.json</code>	两个幂等标记、交易模式、风险硬限制（第 chapter 三十二 章）
10	<code>data/stock.duckdb</code>	行情库是否存在、最新交易日是否更新到上一交易日

数据来源：系统事实清单第 2、6、13、14 节；`ui/src/pages/AuditLog.tsx`；核对时间 2026-09-24。

33.6.1 常见故障速查

常见误解·闪退是玄学 “它有时候崩有时候不崩，说不清。”

这句话不是结论，是**还没分类**。判断方法很简单：看它有没有规律地跟某个动作绑定。

- 如果它总在同一句操作之后崩，那是**配置类或数据类**——必现，可复现，读错误信息就能定位；
- 如果它偶发、无日志、直接消失，那几乎一定是**并发/内存类**——这时唯一有用的东西是崩溃转储，而不是重试。

“偶发”是最容易被当作借口的三个字。事故 #1 就是标准的偶发：它需要“卸载”与“后台线程正在跑”同时发生。当你把这两个条件写出来，它就不再偶发了——它变成了一个**有明确触发条件的确定性事件**，只是触发时机不确定。

表 33.3 四个最常见的故障与处理

现象	原因与处理
选股/回测提示“行情数据库 (stock.duckdb) 未挂载”	缺 data/stock.duckdb，或路径不对。这是 明确报错、不返回假数据 的设计（第 chapter 二十章）——按提示补数据，不要试图绕过。
决策没有 AI 产出	通常是 API Key 无效或未配置。系统会 明确报错 ，不会返回一段假的分析。去设置页的 AI 页签核对提供商、模型名、接口地址、Key。
非交易时段不交易	这是 正常行为 。A 股的节奏就是节拍器（第 chapter 二十三章）：休市时不产生订单。不要把它当成故障。
启动即闪退	先确认版本 （换错了包是最常见的原因），再读 log/crash_dump.txt。若转储为空，说明崩溃发生在日志与转储初始化之前——那就要检查可执行文件所在目录是否可写。

数据来源：系统事实清单第 14 节；docs/用户操作手册.md 第 15 章 Q&A；核对时间 2026-09-24。

这本书里另一个同类例子是事故 #3（托盘右键菜单弹不出来）。它的根因是：Wails 的启动流程常跑在非主线程上，而 Windows 的窗口消息只发给“创建那个窗口的线程”。所以托盘窗口必须在一条**自己锁定的线程**上创建，并跑自己的消息泵——否则左键右键都永远无人响应。它同样表现为“有时好有时坏”，同样不是玄学。

常见误解 · 数据缺了，先找个数顶上 这是全书反复出现的一条，放在最后一章是因为它最容易被“排障”这个场景重新触发。

排障时人会有一种强烈的冲动：**只要能让流程跑完，就先跑完**。于是行情库缺了就找另一张表、API Key 无效就先跳过 AI、判势缺一个维度就随手给个 60。

第 chapter 二十章已经给出结论：**代理变量可以用，但必须当场标注；连代理都没有时，正确的动作是留空，而不是填一个看起来合理的数**。

因为在排障这个场景里，你做的每一个“临时”决定，都可能在系统里活很久——尤其是当它让流程看起来正常了之后，就再也没有人会回去检查它。

系统把这条写成了硬约束：**关键组件失败必须弹窗报错，严禁降级、严禁使用虚假数据**。那个“仍然继续”的按钮是给你**知情之后**用的，不是给你绕过红灯用的。

33.7 最后一件事：回到第 1 章那个问题

第一章开头是这样一个问题：一个人，怎么同时当研究员、风控和操盘手？

当时的回答是：不要找一个人，开一家五个人的小公司。

现在全书读完了，我们可以把那个答案补完整。那家小公司之所以成立，不是因为它雇了五个很聪明的 AI。而是因为这六样东西被一条一条写进了代码：

- **分工**——五个角色，每个角色有明确的任务类型（第 chapter 一、chapter 三章）；
- **权限**——谁能下单、谁能选股、谁有否决权，写在权限表里，不是写在提示词里（第 chapter 二章）；

- **契约**——派活的标准格式：不给它“编”的出口，它就编不出来（第 chapter 三 章）；
- **证据**——决策发生的同一时刻同步落证据，而不是事后补一段解释（第 chapter 十 章）；
- **容错**——只降不升的仓位、失败即拒的启动、AI 失联时的规则回退（第 chapter 七、chapter 九、chapter 二十四 章）；
- **诚实**——数据不可用就报错，代理变量必标注，样本不够就说“无法判定”（贯穿全书）。

这六样，没有一样是“让模型更聪明”能替代的。它们的共同点是：都可以被写下来、被检查、被反驳。

而这恰好也是本书想留下的东西。你读完之后不必拥有这套系统——你可能用别的软件、别的语言、甚至完全手工。但你手上应该多了一套问题，用来问任何一套自称“AI 帮你投资”的东西：

临走时带走的六个问题

1. 它会不会拒绝？（有没有权限边界，还是谁都能下单）
2. 它会不会说“我不知道”？（有没有 DATA_UNAVAILABLE 这样的出口）
3. 它算出来的数，我能自己复算一遍吗？（因子、分数、仓位，有没有公式）
4. 它的回测有没有偷看未来？（成交价口径、切分方式、成本项）
5. 它亏了之后，能查到当时看到了什么吗？（证据链、审计日志）
6. 它错了之后，明天会不一样吗？（复盘有没有产出参数变化）

我在第一章写过一个身份转换：从“操盘的人”，变成“开公司的人”。

全书读完之后，我把它再补一句。你依然是那个开公司的人，但现在你手里多了一样东西——你不再需要相信它。你可以查它、算它、验它、关它。

这就是这套系统最后想给你的一种感觉：不是“我把钱交给了一个很聪明的东西”，而是“我搭了一台我说得清楚的机器，并且它不敢骗我”。

本章对应的真实代码、文档与事故

- 日志与崩溃转储：main.go 的 `setupLogging()`、`GetExecDir()`；`log/quantbot_<时间戳>.log`、`log/crash_dump.txt`
- 启动十步与看门狗：App.go 的 `startup(ctx)`（第 chapter 二十四 章）
- 审计与交付物：`internal/audit`、`internal/transparency`、`deliverables/<日期>/`、`agent_task_logs`
- 任务失败与阶段错误收集：`internal/agentworkflow/workflow_engine.go` 的 `ExecuteCompleteDailyWorkflow`
- 前端审计日志页：`ui/src/pages/AuditLog.tsx`
- 用户手册故障 Q&A：`docs/用户操作手册.md` 第 15 章
- 相关事故：#1（agent.dll 卸载导致段错误）、#2（致命 panic 被静默丢弃）、#3（托盘消息泵线程）、#11（收盘后循环执行与日志重复刷）、#23（180s 硬超时掐断深度复盘，延长至 30 分钟）

动手试一试（把本章变成一次演习）

1. 打开 `log/` 目录，找到最新那个 `quantbot_*.log`，搜索 [Step，数一数能不能凑齐十步。
2. 检查 `log/crash_dump.txt` 是否存在。若存在，读第一条；若不存在，说明本机还没发生过致命崩溃——但请确认那条重定向代码在运行（看 `main.go`）。
3. 去设置页的「数据维护」，看行情库最新更新到哪个交易日。若不是上一交易日，就按第 chapter 二十五章的顺序补一次。
4. 打开审计日志页，随便找一条决策，看看它能不能回答这个问题：“当时它看到了什么？”
5. 最后做一次“演习式提问”：假设现在选股页返回 0 只结果。请你按本章的清单，列出你会依次检查的三个地方，并说明每一个能排除哪种可能。

这本书到这里就结束了。它没有告诉你哪只股票会涨——它讲的是另一件事：**一台机器，怎么把一件本来只能靠感觉的事，做得可以被检查。**

而那件事真正的难点，从来不是算法。是把“诚实”这种东西，写成一行一行的代码。

第四部分

附录

系统架构与包结构速查

前 33 章反复提到「某个东西在哪个包里」。这一章把它们收在一处，供你查而不是读。凡本书正文说过的结论，都能在这里找到落点；凡这里没写的，正文也没写。

A.1 系统身份

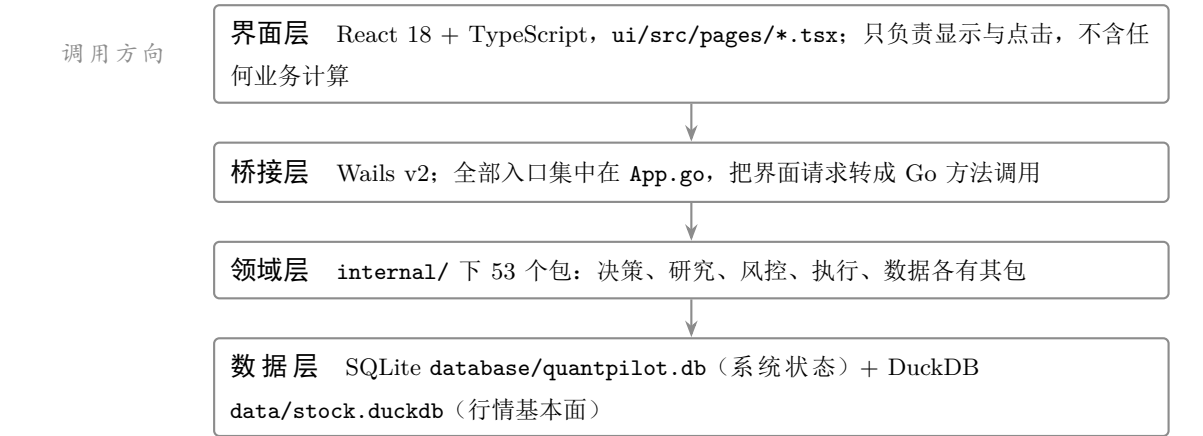
表 A.1 QuantBot 的身份卡

项	值
名称	QuantBot（早期版本名 QuantPilot）
版本	v1.6.0
技术栈	Go 1.25 + Wails v2 + React 18 + TypeScript 5.3 + Vite 5 + Tailwind CSS 3.3 + Zustand 4.4 + Recharts 2.10 + DuckDB + SQLite(GORM)
形态	Windows 单机、单可执行文件、本地数据、系统托盘常驻、单实例
仓库根	AIQuant/
决策脑	agent.dll（c-shared Go DLL，独立资产，进程内常驻、不卸载）
默认 AI	deepseek / deepseek-v4-flash / https://api.deepseek.com
默认初始资金	100000 元 ¹
默认交易模式	simulated（可切 live）
默认单票上限 / 回撤容忍	0.30 ² / 0.10 ³ （风控硬限制统一口径）

数据来源：系统事实清单第 1 节；核对时间 2026-09-24。技术栈与默认值取自 `internal/config/config.go` 与源码。

A.2 四层结构：一张图看清谁调用谁

系统不是一堆平行模块，而是四层，上层只能往下调用，不能反过来。这个方向很重要——它决定了「谁有权改数据、谁只能读数据」。



A.3 启动十步

开机加载界面上那六行进度，背后其实是十步初始化。日志里每一步都打着 [Step N/10] 的标记，出问题时按这个序号定位最快的。

表 A.2 启动十步（App.go 的 startup）

步	日志名	做了什么
1	Initializing config	读 config/config.json; 同花顺官方源注入; 交易日历外挂加载; 升级服务就绪
2	Initializing SQLite	打开 database/quantpilot.db, 建 58 张业务表; LLM 调用日志走文件
3	Initializing DuckDB	挂载 data/stock.duckdb (行情与基本面)
4	Initializing TDX service	通达信数据管理器, 可复用 DuckDB 连接
5	Initializing Harness	装配核心运行时, 复用 2、3 步的连接, 避免重复打开数据库
6	Initializing LLM client and MCP	LLM 客户端 + MCP 服务
7	Initializing task executor	任务执行器
8	Initializing agents and engines	五个智能体与各引擎装配、工具注册
9	Initializing services	审计服务、策略服务、回测服务、任务日志
10	Initializing schedulers	自动调度器与六个处理器, 随即 Start()

数据来源：系统源码 App.go 的 startup ([Step N/10] 日志, 行 176-760 区间); 核对时间 2026-09-24。

常见误解 · 启动失败就等于系统坏了 不是。第 8 步装配智能体时任何一个引擎失败，程序都不会立刻退出：日志里一行 `XXX init error: ...`，然后继续往下走，只是在界面上「仍然继续」那一步才真正把错误交给用户看。所以看到启动很慢、某个功能点了没反应，先去看日志里哪一步报了 `error`，而不是重装。

真正致命的只有一条：**** 关键组件失败必须弹窗报错，严禁降级或使用虚假数据 ****。系统宁可停在那里让你决定，也不拿一份假数据把流程走完。

A.4 53 个包的职责一览

这里是 internal/ 下的全部顶层包。分组是本书的归纳，代码里没有「组」这个字段；包名照原样写出，internal/ 前缀省略。

表 A.3 internal/ 的 53 个包，按职责分组

组	包
入口与装配	agentdll、brain、brainhost、harness、orchestrator、runner、scheduler、models、config、version
智能体流程	agentworkflow、plannerhost、tradeapproval、audit
决策与治理	cio、policy、riskcenter、transparency
研究链	screeener、factors、strategy、backtest、evaluation、marketinfo、marketsixdim、sentiment、replay
风控与执行	risk、riskos、intelligence、portfolio、execution、broker、port
行情与盘口	orderbook、data、tdx、tdxterm
模型与工具	tools、toolworker、mcp、llmhost、llmmonitoring、llmstore、thssdk
平台支撑	util、logger、updater、pricing、memory、etfmonitor、testingutil、xtquant

「组」一列为本书归纳。统计口径：AIQuant/internal/ 下的顶层目录，实测 53 个；系统事实清单早期记为 55，差异来自统计时点，本书以目录实测为准。

数据来源：目录实测命令：列出 AIQuant/internal/ 下的子目录；核对时间 2026-09-24。

A.5 关键路径对照表

正文里凡出现「这个逻辑在哪」，答案都在这张表里。

表 A.4 正文主题到源码路径的对照

主题	路径
智能体角色 / 权限 / 任务类型	internal/agentworkflow/models.go
权限校验（三条硬约束）	internal/agentworkflow/permissions.go
工作流引擎（盘前 / 决策 / 风控 / 盘中 / 盘后）	internal/agentworkflow/workflow_engine.go
事件总线（触发式任务）	internal/agentworkflow/event_bus.go
CIO 决策主张层（辩论 + Kelly）	internal/cio/proposal.go
CIO 集成投票	internal/cio/ensemble.go
团队装配与工具注册	internal/brainhost/team_tools.go
六维判势评分（纯函数）	internal/marketsixdim/sixdim.go
六维抓取 / 落库	internal/marketsixdim/fetcher.go、persist.go
选股引擎	internal/screener/screener_service.go、pro_factor_engine.go、factor_engine.go、smart_composer.go、tradeable_pool.go
回测引擎	internal/backtest/engine.go、service.go、strategies.go、indicators.go、cost_model.go、grid.go
策略指标刷新	internal/strategy/service.go 的 RefreshStrategyMetrics
因子复盘	internal/tools/factor_review_tool.go
每日策略	internal/tools/daily_strategy_tool.go
六维工具	internal/tools/sixdim_tool.go
盘口	internal/orderbook/orderbook.go
传统风控	internal/risk/engine.go
计算中台风控	internal/intelligence/risk_engine.go
组合引擎	internal/portfolio/engine.go、optimizer.go
政策引擎（硬限制真源）	internal/policy/engine.go
调度	internal/scheduler/auto_scheduler.go
行情库访问	internal/data/duckdb.go
复权因子 / 前复权视图	internal/data/thx_adjfactor.go
交易日历	internal/data/thx_calendar.go
TDX 数据维护	internal/data/maintenance.go
同花顺官方 SDK	internal/thssdk/
交易日时段判断	internal/util/trading_hours.go
前端页面	ui/src/pages/*.tsx；路由表 ui/src/App.tsx

数据来源：系统事实清单第 2 节的关键路径表，逐条与源码目录核对；核对时间 2026-09-24。

附录 A 的出处

- 启动十步：App.go 的 startup（[Step N/10]）
- 包清单：AIQuant/internal/ 顶层目录实测
- 关键路径：系统事实清单第 2 节
- 身份卡：系统事实清单第 1 节、internal/config/config.go

动手试一试

1. 数一遍包目录。在你的仓库里执行「列出 AIQuant/internal/ 下的子目录」，看总数是不是 53。
2. 在日志里找十步。启动后打开 log/ 下的运行日志，搜 [Step，你会看到从 [Step 1/10] 到 [Step 10/10] 的十行；任意一步后面跟着 error，就说明那一步没装配成功。
3. 顺一条链走到底。从 internal/marketsixdim/sixdim.go 的评分纯函数出发，往上找谁调

它 (`internal/tools/sixdim_tool.go`) ，再往上找谁把结果写进 `market_sixdim_daily` (`internal/marketsixdim/persist.go`) ——这就是第 19 章那条链在代码里的样子。

数据表速查

全书反复出现两类「库」：一个管钱和账，一个管行情和行情之外的一切。这一章把两边的表列全，并附上本书写作时的真实行数——**行数是会变的，表名和结构不会**。

B.1 两个库，各管什么

表 B.1 两个数据库的分工

库	管什么、谁在写
SQLite database/quantpilot.db	系统状态与账本。持仓、订单、成交、任务日志、审计、投资方案、因子质量、每日策略计划。由 Go 侧通过 GORM 读写；界面上的「我的投资」「工作流」「审计」都读它。
DuckDB data/stock.duckdb	行情与基本面。日 K、复权因子、股票字典、交易日历、财务报表、六维判势落库、盘口日频摘要。体量大（约 1.4 GB）、以只读查询为主。

数据来源：系统事实清单第 13 节；两个库文件的实测大小与表结构，核对时间 2026-09-24。

B.2 SQLite：58 张业务表

下面这张表按用途分组列出全部 58 张表，并给出行数。行数一栏值得你多看两眼——**绝大多数字段位是 0**，因为这套系统在本机只跑过很短一段时间，很多功能还没有真实数据流过。这不是缺陷，是「还没有发生过」；本书凡引用数字，只引有行数的那几张。

表 B.2 SQLite 业务表清单与行数（database/quantpilot.db）

组	表名	行数
智能体与权限	agents、agent_permissions、 agent_sessions、agent_tool_calls、 agent_activities、agent_decisions、 agent_documents、agent_performances、 team_members、tasks、events、 workflows、workflow_instances、 investment_decisions	36 / 6 / 5 ¹

¹真实数据：本数字直接取自系统运行库 data/stock.duckdb，未做任何模拟或平滑。

组	表名	行数
账户与持仓	users、portfolios、positions、orders、trades、cash_flows、position_snapshots、portfolio_daily_stats、instruments、market_indices、watch_stocks	1 / 6 / 46 ²
投资规划	investor_profiles、conversations、conversation_messages、investment_plans、strategy_candidates、plan_approvals、investment_mandates	1 ³
研究与策略	strategies、strategy_stock_params、factor_qualities、factor_calibration_histories、backtest_configs、backtest_results、daily_reviews、daily_strategy_plans、claim_records、alpha_stores、market_risk_reports、portfolio_optimizations	17 ⁴
风控与治理	risk_limits、risk_checks、policy_logs、audit_logs、cio_decision_logs、decision_traces	181 ⁵
可交易池	tradeable_stocks、tradeable_stock_logs	0 ⁶
其他	settings、system_configs、position_manager_configs、documents、llm_call_logs	1 ⁷

「组」为本书归纳，代码里没有这个字段。行数一栏只列该组内非零的那几张表，按「权限 / 活动 / 成员」等顺序对应表名里最先出现的那几项；组内其余表均为 0 行。统计口径：逐表 `select count(*)`。

²真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

³真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

⁴真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

⁵真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

⁶真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

⁷真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

数据来源：只读查询 `database/quantpilot.db` 的 `sqlite_master` 与逐表 `count(*)`；时间 2026-09-24。全库共 58 张业务表（另有 `SQLite` 内部的 `sqlite_sequence`）。系统事实清单早期记为 39 张，本书以运行库实测为准。

常见误解·空表等于功能没实现 不等于。表建出来了、写入路径也在，只是本机还没跑过触发它的那条流程。判断一个功能是否可用，要看三条：表存在、写入代码存在、以及有没有真实行数。只有前两条时，功能是「已实现、未验证」；第三条也有了，才叫「跑通过」。

本书对这三种状态分得很清：正文里凡出现数字，都能在下表「有行数」的那几张里找到；找不到就写「数据缺失」，绝不用空表反推一个数出来。

B.3 DuckDB：行情与基本面

表 B.3 DuckDB 的表与视图及其真实规模（data/stock.duckdb）

表 / 视图	行数	列与说明
stock.ohlcv	10,041,343 ⁸	symbol, date, open, high, low, close, volume, amount
stock.stock_basic	19,068 ⁹	symbol, name, full_name, list_date, total_share, float_share, industry, area, market, is_st, updated_at
stock.adj_event	14,172 ¹⁰	symbol, ex_date, d, s, r, p（d 现金分红、s 送股、r 配股、p 配股价）
stock.adj_factor	9,997,923 ¹¹	symbol, date, backward_factor, forward_factor
stock.trading_calendar	248 ¹²	date; 最新一条为 2026-09-17，滞后于行情库最新日
stock.financial_report	315,680 ¹³	symbol, report_date, ann_date, total_revenue, revenue_yoy, net_profit, profit_yoy, net_profit_ded, roe, gross_margin, net_margin, total_assets, total_liab, debt_ratio, oper_cashflow, total_shares, float_shares, eps, bps, update_time
market_sixdim_daily	5 ¹⁴	trade_date, dim_scores, raw_total_score, conflict_count, adjusted_total_score, position_rate, market_tag, sources, created_at
order_book_daily	0 ¹⁵	盘口日频摘要；写入路径存在，本机尚无记录
stock.ohlcv_qfq（视图）	—	前复权 K 线，由 stock.ohlcv LEFT JOIN stock.adj_factor 生成

数据来源：只读查询 `data/stock.duckdb`；时间 2026-09-24。`order_book_daily` 为 0 行取自系统事实清单第 13.2 节并复核。

四条必须记住的口径：

第一，`stock.ohlcv` 覆盖 2021-08-02 至 2026-09-24，去重后 9,584 个 symbol；末两日是残档（仅几十个 symbol），做统计必须挑「最近完整交易日」——¹⁶ 本书用的是 2026-09-23。

第二，`stock.stock_basic` 里混着 ETF / 基金（如 `sz167301`、`sz200570`），它们的 `name` 为 NULL。做「个股」统计时必须先过滤，否则分母被撑大。

第三，`market_sixdim_daily` 是本库最有价值的一手素材，但只有 5 行（2026-08-31、09-02、09-09、09-10、09-11）。第 19、20、21 章把它逐条手算过；多一行都没有。

第四，本库是挂载库。建表与数据都在 `main schema` 下，日常代码里写 `stock.ohlcv` 是命名空间习惯；你在 SQL 里写 `ohlcv` 同样能查到。¹⁷ 本书不对「`order_book_daily` 的盘口统计」做任何引用——该表 0 行。

¹⁶真实数据：本数字直接取自系统运行库 `data/stock.duckdb`，未做任何模拟或平滑。

¹⁷数据缺失：该系统库中此项无有效记录。本书遵照系统“数据不可用即报错、不降级、不伪造”的原则，此处留白、不填充估计值。

B.4 「以数据库为准」的三条口径

界面上的数字、日志里的数字、数据库里的数字，三者不一致时，本书一律按这个顺序裁决：

- 1. 数据库是唯一事实来源。界面是投影，日志是过程记录；两者与库冲突时，以库为准。第 29 章「差了三块五」讲的就是这条。
- 2. 派生数字必须能当场复算。凡是「按公式算出来的」数（总分、仓位系数、盈亏、胜率），本书都在正文里给出公式与输入，读者可自己算一遍对账。
- 3. 缺数就留白。库里没有的记录，标注「数据缺失」并停笔；不插值、不估计、不用空表反推。

附录 B 的出处

- SQLite 表清单与行数：database/quantpilot.db 的 sqlite_master 加逐表 count(*)，2026-09-24
- SQLite 建表来源：internal/data/sqlite.go 的 AutoMigrate (47 张)、internal/agentworkflow/models.go 的 MigrateAgentWorkflow (8 张)，其余由各包内建表
- DuckDB 表与行数：data/stock.duckdb，2026-09-24；结构对照系统事实清单第 13.2 节
- 两库分工：系统事实清单第 13 节

动手试一试

- 1. 数表。用任意 SQLite 客户端以只读方式打开 database/quantpilot.db，执行 `select count(*) from sqlite_master where type='table'`，看是不是 59（含内部的 sqlite_sequence）。
- 2. 找非空表。对每张表跑 count(*)，你会发现只有十来张不是 0 行。把非空的挑出来，就是这本书「敢引用」的全部范围。
- 3. 对账五条判势。`select * from market_sixdim_daily order by trade_date` 应当恰好返回 5 行。挑 2026-09-09 那条，把 dim_scores 里的六个分量乘上权重 (0.15/0.20/0.15/0.15/0.25/0.10) 求和，看能不能得到 raw_total_score——这就是第 19 章那张表的复算。

附录 C

工具清单与可用角色

AI 的「手」就是工具。它能动手，但只能动你允许的那几样——这句话在第 4 章讲过一遍，这里是它的完整账本：**全部 55 个已注册工具，以及每个工具声明给哪些角色用。**

C.1 注册表：55 个工具

工具注册发生在团队装配时。每个工具都要在中央注册表里报三项：类别、风险级别、**可用角色**。第三项是运行时强制校验的依据——不是提示词里的礼貌请求，是调用前的硬检查。

表 C.1 中央工具注册表：55 个工具、类别、风险级别与声明可用角色

工具名	类别	风险	声明可用角色
get_market_data	行情数据	low	CIO / Quant / Risk / Trader
search_market	选股	low	CIO / Quant / Trader
optimize_portfolio	组合	medium	CIO / Planner / Quant
get_market_stats	市场统计	low	五个角色全部
run_backtest	回测	medium	Quant
screen_market	选股	medium	CIO / Planner / Quant
build_stock_pool	选股	medium	CIO / Planner / Quant
get_stock_pool	组合	low	CIO / Risk / Trader
get_portfolio_state	组合	low	CIO / Planner / Risk / Trader
get_positions	组合	low	CIO / Quant / Risk / Trader
place_trade	执行	high	Trader
order_cost_estimator	执行	low	CIO / Quant / Trader
get_macro_snapshot	行情数据	low	CIO / Planner / Quant / Risk
portfolio_risk_view	风控	medium	CIO / Quant / Risk
limit_check	执行	medium	CIO / Quant / Risk / Trader
portfolio_risk_metrics	风控	medium	CIO / Quant / Risk
run_daily_risk_report	风控	medium	CIO / Risk
factor_review	研究	low	CIO / Quant
daily_settlement	执行	medium	CIO / Risk / Trader
tune_strategy_params	研究	medium	Quant
select_next_day_strategy	研究	medium	CIO / Quant

工具名	类别	风险	声明可用角色
position_manager	组合	medium	CIO / Quant
market_regime_detect	行情数据	low	CIO / Planner / Quant / Risk
mandate_compliance_check	风控	low	CIO / Planner
sentiment_monitor	行情数据	low	CIO / Risk
stop_loss_monitor	风控	low	CIO / Risk
decision_log	研究	low	CIO / Risk
market_sixdim_detect	行情数据	low	CIO / Planner / Quant / Risk
market_order_book	行情数据	low	CIO / Planner / Quant / Risk
update_trading_calendar	研究	low	CIO / Planner / Quant
check_trading_day	研究	low	CIO / Quant / Risk / Trader
get_index_technical	行情数据	low	CIO / Planner / Quant / Risk
get_market_breadth	行情数据	low	CIO / Planner / Quant / Risk
get_turnover	行情数据	low	CIO / Quant / Risk
get_capital_flow	行情数据	low	CIO / Quant / Risk
get_limitup_board	行情数据	low	CIO / Quant / Risk
get_external_market	行情数据	low	CIO / Planner / Quant / Risk
get_market_news	行情数据	low	CIO / Planner / Quant / Risk
get_stock_announcement	研究	low	CIO / Quant / Risk
get_lhb	行情数据	low	CIO / Quant / Risk
get_fundamental	研究	low	CIO / Quant
check_market_rules	行情数据	low	五个角色全部
get_stock_fund_flow	行情数据	low	CIO / Quant / Risk
calculate_structure_risk	风控	low	CIO / Quant / Risk
get_performance_nav	组合	low	CIO
get_investment_plan	组合	low	CIO / Planner
get_claim_accuracy	研究	low	CIO / Quant
record_alpha	研究	low	CIO / Quant
get_alpha_by_regime	研究	low	CIO / Quant
get_alpha_top	研究	low	CIO / Planner / Quant
estimate_strategy_generalization	研究	low	CIO / Quant
rank_strategy_generalization	研究	low	CIO / Quant
get_sector_rotation	行情数据	low	CIO / Planner / Quant
get_skill_catalog	研究	low	五个角色全部
get_factor_health_report	研究	low	CIO / Quant

工具名	类别	风险	声明可用角色
-----	----	----	--------

「类别」与「风险」照源代码的分类型/风险级原样转写（Category* 与 low/medium/high）。角色顺序照源码书写，未按字母排序——顺序本身没有含义。

数据来源：系统源码 `internal/brainhost/team_tools.go` 的 `registerTeamTool` 调用（行 240-327），逐条抄录；核对时间 2026-09-24。

C.2 两层注册：声明与装配

工具的可用性由两层各自决定，两层都要过，工具才真的能调：

- 1. 声明层（注册表）：`registerTeamTool` 往中央注册表登记「这个工具允许谁调用」。运行时按角色硬校验，越权直接拒绝。
- 2. 装配层（角色）：`agent.RegisterTool(...)` 把工具塞进某个角色的工具箱。没塞进去，这个角色在提示词里根本看不到它。

从装配层数一遍，五个角色的工具箱大小是：

表 C.2 各角色实际装配的工具数

角色	装配数	特征
CIO	24 ¹	最多。它要综合所有来源，所以几乎所有只读工具都在它手里
Quant	17 ²	研究主力：回测、选股、因子复盘、参数调优
Risk	16 ³	风控专项：风险指标、情绪、止损、结构风险、判势、盘口
PLANNER	10 ⁴	规划导向：股票池、组合优化、合规校验、市场状态
TRADER	10 ⁵	最窄，但握着唯一一个 high 风险工具 <code>place_trade</code>

数据来源：系统源码 `internal/brainhost/team_tools.go` 里各角色的 `RegisterTool` 日志串（行 403、438、488、538、565），逐项计数；核对时间 2026-09-24。

常见误解·注册表说谁可用，谁就真的能用 不一定，而这正是系统里真实存在的一处不一致。注册表把 `market_sixdim_detect` 与 `market_order_book` 声明给四个角色（CIO / Planner / Quant / Risk），但 ** 装配层里 Quant 的 `RegisterTool` 列表并没有这两个工具 **——量化智能体的工具箱里没有六维判势，也没有盘口。也就是说，这两行里「Quant」这个角色是声明了却没装配，属于惰性声明。后果很具体：Quant 在对话里看不到六维工具，就调不动它；即使某条路径漏过了装配层检查、真让它调了，注册表也会放行（因为声明里有它）。声明与装配不一致时，中间出现一条谁都不负责的缝。这种偏差不是本书要「圆」的，而是要你记住的：一个系统中的权限，必须能在同一张表里同时读到「声明」与「装配」两列，否则你无法判断某次越权到底是设计还是漏洞。修法也很简单——二选一：要么删掉声明里多余的 Quant，要么把工具真的装进 Quant 的工具箱。

C.3 七条只有一个人能做的事

工具之上还有一层更粗的权限。无论工具有多少个，下面这七件事都只允许一个角色做：其余角色连工具都调不动，更别说绕过：

表 C.3 「仅此一人」的权限条目（默认矩阵）

只能做的事	唯一所有者
执行订单（下单、撤单）	TRADER
选择股票（决定买哪只）	CIO
一票否决（否掉方案）	RISK
修改硬限制（单票上限、回撤容忍）	CIO
紧急停止（全局停机）	CIO
修改投资目标（Mandate）	PLANNER
读投资者画像	五个角色全部（只读）

数据来源：系统事实清单第 3 节；`internal/agentworkflow/permissions.go` 的三条硬约束：只有 `AgentTrader` 可执行订单、只有 `AgentCIO` 可选择股票、只有 `AgentRisk` 有否决权。完整 8 权限 × 5 角色的矩阵见第 [chapter 二十七](#) 章的表格；核对时间 2026-09-24。

附录 C 的出处

- 55 个工具的注册与角色声明：`internal/brainhost/team_tools.go` 的 `registerTeamTool`（行 240-327）
- 各角色装配：同文件各角色区块的 `RegisterTool` 与末尾日志串（行 403、438、488、538、565）
- 三条硬约束与权限矩阵：`internal/agentworkflow/permissions.go`、`internal/agentworkflow/models.go`
- 工具实现源码：`internal/tools/` 下各 `*_tool.go` 与 `tools.go`
- 工具注册表数据结构：`internal/port/`（`ToolMetadata`、`ToolRegistry`、`AgentRole`）

动手试一试

1. 数工具。在 `internal/brainhost/team_tools.go` 里搜 `registerTeamTool`（，数一数是不是 55 次（不含那行函数定义）。
2. 对一遍缝。在同一个文件里搜 `sixdimTool`，你会看到它只在三个角色的区块里被 `RegisterTool`——找找哪个角色在注册表里有声明、却没被装配，体会一下「声明 ≠ 装配」是什么意思。
3. 找唯一出口。搜 `place_trade`，确认它的声明角色只有一个；再去看 `internal/agentworkflow/permissions.go`，找出「只有 `TRADER` 可执行订单」这条硬约束的代码形态。

十七种策略的信号逻辑与默认参数

第 16 章说过一句话：这十七个策略看着五花八门，其实只做一件事——把「比较两条线」或「和阈值比」翻译成买与卖。这一章把它们连同**真实默认参数**、**可调范围**、以及后六个指标的**计算公式**一起摊开，供你对照代码。

D.1 十七个策略一览（完整版）

数据只有一个来源：`BuiltinStrategies` 这个切片。第 16 章的表只列了「族」与「信号」，这里补上每个策略的**私有参数与止损止盈**。

常见误解·默认参数是「出厂设置」，不是「最优参数」 系统里的参数分三层，常被混为一谈：
第一层是硬参数——比如 A 股的涨跌停幅度、整手 100 股。它们不是选的，是规则，改不了。
第二层是出厂默认值——就是上表。它们被写进 `ConfigJSON` 当作起点，作者没有声称它们最优；换一只票、换一段行情，最优值就变了。
第三层是调优结果——`tune_strategy_params` 在**样本内**网格搜索出来的参数，写回策略表的 `config_json`。它比出厂默认更贴合样本，但正因为它是在样本上挑出来的，**样本外能不能复现要单独验证**（第 17、18 章讲的过拟合）。系统给出的防御手段是 **walk-forward**：前 70% 选参、后 30% 验证，且输出样本外指标。
 所以看到「某策略夏普 2.5」时，先问一句：这是出厂默认跑的，还是调优后跑的？是在样本内还是样本外？三个答案不同，这个 2.5 的含义完全不同。

D.2 后六个指标的公式

`super_trend`、`aroon`、`hma`、`cmf` 用到的 `ATR`、`WMA`、`HMA`、`Aroon`、`CMF`、`SuperTrend` 六个指标，系统在 `internal/backtest/indicators.go` 里按公开公式自实现，没有引入那个 AGPL-3.0 的第三方指标库。理由很实际：许可证污染比写几个循环麻烦得多。

记第 i 根 K 线的开高低收为 H_i, L_i, C_i ，成交量 V_i 。

D.2.1 ATR：平均真实波幅

先定义**真实波幅**——它把「跳空」也算进波动里，这是它比「当日振幅」强的地方：

$$TR_i = \max(H_i - L_i, |H_i - C_{i-1}|, |L_i - C_{i-1}|), \quad TR_0 = H_0 - L_0. \quad (D.1)$$

再从第 p 根起做 Wilder 平滑（首值为前 p 根的简单平均）：

$$ATR_p = \frac{1}{p} \sum_{j=1}^p TR_j, \quad ATR_i = \frac{ATR_{i-1}(p-1) + TR_i}{p} \quad (i > p). \quad (D.2)$$

这个递推的等价形式是 $ATR_i = (1 - \frac{1}{p})ATR_{i-1} + \frac{1}{p}TR_i$ ——一个权重 $\alpha = 1/p$ 的指数平滑。 $[0, p]$ 区间因数据不足一律留 0，策略应等 $i \geq p$ 再取用。

表 D.1 17 个内置策略：类型、私有默认参数、止损/止盈与信号逻辑

类型	私有默认参数	止损 / 止盈	信号逻辑 (买 → 卖)
ma_cross	fast_ma=15、slow_ma=20	0.05 / 0.20	MA15 上穿 MA20 → 下穿
expma_cross	fast_ema=12、slow_ema=50	0.05 / 0.20	EMA12 上穿 EMA50 → 下穿
bollinger_breakout	period=20、std_dev=2	0.05 / 0.20	收盘跌破下轨 → 突破上轨
rsi_reversal	period=14、oversold=30、overbought=70	0.05 / 0.20	RSI < 30 → RSI > 70
kdj_golden	k_period=9、d_period=3、j_oversold=30、j_overbought=70	0.05 / 0.20	K 上穿 D 且 J < 30 → K 下穿 D 且 J > 70
turtle_breakout	entry_period=40、exit_period=5	0.05 / 0.20	破 40 日高点 → 破 5 日低点
mfi_volume	period=14、oversold=30、overbought=70	0.05 / 0.20	MFI < 30 → MFI > 70
mtm_momentum	mtm_ma_period=30	0.05 / 0.20	MTM 上穿 0 → 下穿 0
dmi_trend	adx_period=14、trend_threshold=25	0.05 / 0.20	PDI > MDI 且 ADX > 25 → 反向
cci_breakout	period=14、overbought=100、oversold=-100	0.05 / 0.20	CCI 上穿 +100 → 下穿 -100
trix_cross	period=12	0.05 / 0.20	TRIX 上穿自身均线 → 下穿
zhuoyao_momentum	short=20、mid=90、long=180	0.05 / 0.20	多周期共振转强 → 转弱
ml_model	model_names (5 模型)、ensemble_method=probability、buy_threshold=0.5、sell_threshold=0.3	0.08 / 0.25	集成概率 ≥ 0.5 → 不高于 0.3 (0.3 ~ 0.5 为观望带)
super_trend	period=10、multiplier=3.0	0.05 / 0.20	方向由空翻多 → 由多翻空
aroon	period=25	0.05 / 0.20	AroonUp 上穿 AroonDown → 下穿
hma	period=20	0.05 / 0.20	收盘上穿 HMA → 下穿
cmf	period=20、buy_threshold=0.05、sell_threshold=-0.05	0.05 / 0.20	CMF 上穿 +0.05 → 跌破 -0.05

除下表的私有参数外，每个策略的 ConfigJSON 还共同携带 capital=100000、max_position=10；止损止盈两列即 stop_loss 与 take_profit，全部以「比例」书写 (0.05 即 5%)。ml_model 的五个模型依次为 xgboost_astock_v1、astock_lgbm_v1、astock_rf_v1、astock_logistic_v1、astock_mlp_v1。

数据来源：internal/backtest/strategies.go 的 BuiltinStrategies (行 1486-1589)，逐条抄录 ConfigJSON、StopLossPct、TakeProfitPct；核对时间 2026-09-24。

D.2.2 WMA 与 HMA：把滞后压下去

线性加权移动平均让最近一根权重最大（为 p ），依次递减到 1：

$$\text{WMA}_p(i) = \frac{\sum_{j=0}^{p-1} (p-j) v_{i-j}}{\sum_{j=0}^{p-1} (p-j)} = \frac{2}{p(p+1)} \sum_{j=0}^{p-1} (p-j) v_{i-j}. \quad (\text{D.3})$$

（末式用了 $\sum_{j=0}^{p-1} (p-j) = p(p+1)/2$ 。）**赫尔均线**在 WMA 之上再套一层：先用两个不同窗口的差把滞后抵消，再用 $\sqrt{p}$ 窗口的 WMA 把噪声磨平。令

$$\tilde{v}_i = 2 \text{WMA}_{\lfloor p/2 \rfloor}(v)|_i - \text{WMA}_p(v)|_i, \quad \text{HMA}_p(i) = \text{WMA}_{\lfloor \sqrt{p} \rfloor}(\tilde{v})|_i. \quad (\text{D.4})$$

第一式把 $\lfloor p/2 \rfloor$ 窗口对 p 窗口的超前量放大成两倍，补掉滞后；第二式按 $\sqrt{p}$ 加权把补出来的超调抹平。 $p=20$ 时三个窗口分别是 10、20、4（ $\sqrt{20} \approx 4.47$ 圆整为 4）。

D.2.3 Aroon：离新高有多远

Aroon 不看「创没创新高」，只看距上一次新高过了多久。设

$$\iota_i^{\text{hi}} = \max \left\{ j \in [i-p, i] : H_j = \max_{k \in [i-p, i]} H_k \right\}, \quad \iota_i^{\text{lo}} = \max \left\{ j \in [i-p, i] : L_j = \min_{k \in [i-p, i]} L_k \right\}, \quad (\text{D.5})$$

则

$$\text{AroonUp}_i = \frac{p - (i - \iota_i^{\text{hi}})}{p} \times 100, \quad \text{AroonDown}_i = \frac{p - (i - \iota_i^{\text{lo}})}{p} \times 100. \quad (\text{D.6})$$

两者都落在 $[0, 100]$ 。注意 eq. (D.5) 取的是**并列中最近的一个 j** （源码用 $\geq, \leq$ 扫描，后出现的赢）——所以只要最新一根就是最高价， AroonUp_i 立刻取到 100。策略信号是「AroonUp 上穿 AroonDown」买、「下穿」卖。

D.2.4 CMF：资金往哪边流

每根 K 线先算**资金流量乘数**，再看它加权成交量后在窗口里占多大比重：

$$\text{MFM}_i = \frac{(C_i - L_i) - (H_i - C_i)}{H_i - L_i} \quad (H_i > L_i), \quad \text{MFM}_i = 0 \quad (H_i = L_i), \quad (\text{D.7})$$

$$\text{CMF}_p(i) = \frac{\sum_{j=i-p+1}^i \text{MFM}_j V_j}{\sum_{j=i-p+1}^i V_j} \in [-1, 1]. \quad (\text{D.8})$$

MFM 的几何含义是「收盘在当日区间里的相对位置」：收在最高（ $C=H$ ）给 +1，收在最低（ $C=L$ ）给 -1，正中给 0。窗口内放量且收在高位，CMF 就为正。策略阈值 ± 0.05 。

D.2.5 SuperTrend：ATR 画出的动态止损线

设中线 $m_i = (H_i + L_i)/2$ ，带宽 $m_i \pm \lambda \text{ATR}_i$ （默认 $\lambda = 3.0$ ）。上下轨不是当根就算，而是**只能朝收紧方向走**：

$$U_i = \begin{cases} m_i + \lambda \text{ATR}_i, & \text{若 } m_i + \lambda \text{ATR}_i < U_{i-1} \text{ 或 } C_{i-1} > U_{i-1}, \\ U_{i-1}, & \text{否则,} \end{cases} \quad (\text{D.9})$$

$$L_i = \begin{cases} m_i - \lambda \text{ATR}_i, & \text{若 } m_i - \lambda \text{ATR}_i > L_{i-1} \text{ 或 } C_{i-1} < L_{i-1}, \\ L_{i-1}, & \text{否则.} \end{cases} \quad (\text{D.10})$$

方向 $\tau_i \in \{+1, -1\}$ 按下式翻转 ($\tau_0 = +1$):

$$\tau_i = \begin{cases} -1, & \tau_{i-1} = +1 \text{ 且 } C_i < L_i, \\ +1, & \tau_{i-1} = -1 \text{ 且 } C_i > U_i, \\ \tau_{i-1}, & \text{否则,} \end{cases} \quad \text{ST}_i = \begin{cases} L_i, & \tau_i = +1, \\ U_i, & \tau_i = -1. \end{cases} \quad (\text{D.11})$$

ST_i 就是那条跟踪线——多头时贴在下方当止损，空头时贴在上方当压力。方向翻转即买卖点。 $[0, p]$ 区间 ATR 未稳定，调用方须延后取信号。

注 D.1 (为什么这几条公式值得单独抄一遍). 因为它们没有一处是「差不多」的。eq. (D.2) 的首值取前 p 根简单平均、eq. (D.5) 并列取最近、eq. (D.9) 的上轨「只降不升」——每一处都会改变信号。你按别的教材写一版「意思差不多」的实现，回测结果就能差出一大截：不是策略变了，是同一个名字底下的公式变了。这正是第 16 章开头那句话的注脚。

D.3 可调参数的扫描范围

每个策略还带一份默认调参网格（播种时写进策略表的 `tune_ranges_json`）。
`tune_strategy_params` 就在这些区间里遍历。

表 D.2 17 个策略的默认调参范围（最小值 → 最大值，步长）

类型	可调参数与扫描范围
<code>ma_cross</code>	<code>fast_ma</code> 3 → 21 步 2; <code>slow_ma</code> 20 → 140 步 20
<code>expma_cross</code>	<code>fast_ema</code> 5 → 25 步 5; <code>slow_ema</code> 30 → 150 步 15
<code>bollinger_breakout</code>	<code>period</code> 10 → 30 步 5; <code>std_dev</code> 1.5 → 3.0 步 0.5
<code>rsi_reversal</code>	<code>period</code> 5 → 20 步 3; <code>oversold</code> 15 → 35 步 5; <code>overbought</code> 65 → 85 步 5
<code>kdj_golden</code>	<code>k_period</code> 5 → 15 步 2; <code>d_period</code> 2 → 5 步 1; <code>j_oversold</code> 15 → 35 步 5; <code>j_overbought</code> 65 → 85 步 5
<code>turtle_breakout</code>	<code>entry_period</code> 20 → 60 步 10; <code>exit_period</code> 5 → 20 步 5
<code>mfi_volume</code>	<code>period</code> 10 → 20 步 2; <code>oversold</code> 20 → 40 步 5; <code>overbought</code> 60 → 80 步 5
<code>mtm_momentum</code>	<code>mtm_ma_period</code> 10 → 30 步 5
<code>dmi_trend</code>	<code>adx_period</code> 10 → 20 步 2
<code>cci_breakout</code>	<code>period</code> 10 → 20 步 2
<code>trix_cross</code>	<code>period</code> 10 → 20 步 2
<code>zhuoyao_momentum</code>	<code>short_period</code> 10 → 30 步 5; <code>mid_period</code> 40 → 120 步 20; <code>long_period</code> 90 → 240 步 30; <code>trend_period</code> 10 → 30 步 5
<code>ml_model</code>	<code>buy_threshold</code> 0.45 → 0.65 步 0.05; <code>sell_threshold</code> 0.25 → 0.45 步 0.05
<code>super_trend</code>	<code>period</code> 8 → 20 步 2; <code>multiplier</code> 2.0 → 4.0 步 0.5
<code>aroon</code>	<code>period</code> 15 → 40 步 5
<code>hma</code>	<code>period</code> 10 → 40 步 5
<code>cmf</code>	<code>period</code> 10 → 30 步 5; <code>buy_threshold</code> 0.03 → 0.10 步 0.01; <code>sell_threshold</code> -0.10 → -0.03 步 0.01

数据来源: `internal/backtest/strategy_config.go` 的 `defaultTuneRanges` (行 52-123), 逐条抄录。核对时间 2026-09-24。

D.4 成本模型：回测里那笔被扣掉的钱

回测不是「买卖价差 $\times$ 仓位」这么简单，每笔都要扣三样：

$$\text{cost}(x) = a|x| + b\sigma \frac{|x|^{1.5}}{\sqrt{V}} + cx, \quad (\text{D.12})$$

其中 x 是单笔成交额（元）、 V 是当日成交额（元）、 σ 是日波动率（trailing 60 根 K 线的收益标准差）。三项分别是：

- $a|x|$ ：比例费用。佣金率 $a = 0.0003$ （万三，买卖都收，单笔最低 5 元）；卖出另外收印花税 0.0005（0.05%）。买入不收印花税。
- $b\sigma |x|^{1.5}/\sqrt{V}$ ：市场冲击，即 Almgren–Chriss 的平方根冲击律。 $b = 1.0$ 、指数 1.5。
- cx ：收盘 bias，默认 $c = 0$ ——因为成交口径是「收盘价成交 + close-to-close 收益」，两者一致时无需修正。

eq. (D.12) 里最该记住的是冲击项的量纲： $|x|$ 的指数是 1.5，除以 $\sqrt{V}$ 。这意味着单笔规模翻倍，冲击成本涨 $\sqrt{2} \approx 1.41$ 倍，而不是 2 倍；而把同一笔拆成 k 份、每份只是原来的 $1/k$ ，总冲击按 $k \cdot k^{-1.5} = k^{-0.5}$ 下降到原来的 $1/\sqrt{k}$ 。第 17 章证明过这条——它就是「大单要拆」的数学依据。

D.5 回测口径

三条口径，任何一条坏了，回测数字就失效：

1. 次日开盘成交（ExecNextOpen）。信号在第 i 根收盘产生，成交在第 $i+1$ 根开盘。这消除了「用当天收盘的信号、以当天收盘价成交」这种看起来很美的前视偏差。
2. 按标的自适应涨跌停。不同板块涨跌停幅度不同（主板 10%、创业板/科创板 20%、北交所 30%、ST 另计），回测按标的的规则判定，涨停买不进、跌停卖不出。
3. 财务数据按披露日注入。基本面因子取数用 `ann_date`（公告日）而非 `report_date`（报告期），防止「用还没公布的财报选股」这种未来函数。

附录 D 的出处

- 17 个策略的元数据（默认参数、止损止盈、描述）：`internal/backtest/strategies.go` 的 `BuiltinStrategies`
- 调参网格：`internal/backtest/strategy_config.go` 的 `defaultTuneRanges`
- 指标公式：`internal/backtest/indicators.go` 的 `ATR`、`WMA`、`HMA`、`Aroon`、`CMF`、`SuperTrend`
- 成本模型：`internal/backtest/cost_model.go` 的 `DefaultCostModel`、`BuyCost`、`SellCost`、`impact`
- 回测口径与 walk-forward：`internal/backtest/service.go`、`internal/backtest/grid.go`
- 策略表指标写入：`internal/strategy/service.go` 的 `RefreshStrategyMetrics`

动手试一试

1. 抄一遍 ATR。按 eq. (D.1) 与 eq. (D.2) 手算 5 根 K 线的 ATR（取 $p = 3$ ），再和代码里 `ATR()` 的输出对比。首值取前 p 根平均这一步最容易写错。
2. 验证冲击项的读数。取 $\sigma = 0.02$ 、 $V = 10^8$ 元、 $x = 10^6$ 元，按 eq. (D.12) 算冲击项 $b\sigma x^{1.5}/\sqrt{V}$ ；再把 x 改成 2×10^6 ，看它涨了几倍。是不是 $\sqrt{2}$ ？

3. 查一个默认参数。在策略页打开 `super_trend`，看它的 `period` 与 `multiplier` 是不是 10 与 3.0；再点「调参」，看扫描范围是不是 $8 \rightarrow 20$ 与 $2.0 \rightarrow 4.0$ 。
4. 对一遍十七。在代码里数 `BuiltinStrategies` 的条目数，看是不是 17；顺便确认最近新增的四个（`super_trend`、`aroon`、`hma`、`cmf`）都注册进了 `GetStrategyByType` 与 `strategyParamFieldMap`。

六维判势的数据源与降级矩阵

第 19、20 章讲了六维判势「怎么打分」和「数据从哪来」。这一章把第二件事列成表：每个维度的首选源、备选源、兜底代理、超时与缓存、可信度标注规则，以及在数据库里留下的原文证据。

E.1 权重与基准

表 E.1 六维的权重、评分基准与它看什么

维度 (key)	权重	评分要点 (基准 50 分)
技术趋势 tech	0.15 ¹	指数是否站上 MA20 / MA60、趋势状态
市场广度 breadth	0.20 ²	上涨占比、20 日新高新低比、涨跌停比、主线板块数
量能流动性 volume	0.15 ³	全市场成交额 / 20 日均、价量配合
资金结构 capital	0.15 ⁴	北向连续流入天数 (代理)、资金是否集中主线
情绪赚钱效应 sentiment	0.25 ⁵	涨停数、炸板率、非 ST 跌停数
外部约束 external	0.10 ⁶	隔夜外围 (1/0/-1)、重大风险事件

权重之和恰为 1.00；每维以 50 分为中性基准，向上向下加权偏移。冲突修正与分数 → 仓位映射见 table E.2。
数据来源：系统源码 `internal/marketsixdim/sixdim.go` 的权重表；核对时间 2026-09-24。

表 E.2 总分修正与分数 → 仓位 → 标签映射

环节	规则
冲突修正	统计高分维 (≥ 75) 与低分维 (≤ 40) 的个数；冲突数 ≥ 2 时 调整后 = 原始 $\times 0.8$ ，再 clamp 到 $[0, 100]$
分数 → 仓位	$\geq 80 \Rightarrow 0.8 \sim 1.0$ ； $60 \sim 79 \Rightarrow 0.4 \sim 0.6$ ； $40 \sim 59 \Rightarrow 0.2 \sim 0.3$ ； $< 40 \Rightarrow 0.0 \sim 0.1$
分数 → 标签	依次为「强势」「结构性震荡」「偏弱」「退潮风险」
建仓闸门	标签为「退潮风险」或仓位系数 $\leq 0.15 \Rightarrow$ 禁止生成新买入订单 (只允许卖出/持有)
盘中告警	仓位系数 < 0.2 或标签为「退潮风险」 $\Rightarrow$ 发 SIXDIM_ALERT 活动并转观望

数据来源：系统源码 `internal/marketsixdim/sixdim.go` 的 `CheckDimConflict`、`ScoreToPosition`；建仓闸门与盘中告警见 `internal/cio/cio.go`。核对时间 2026-09-24。

E.2 六个维度的数据源与降级链

「降级」是这套系统最诚实的部分：每一个维度都写明了「首选源拿不到时退到什么、再拿不到退到什么」。退到代理变量时，`sources` 字段里会留下原文——不写「已降级」，而是写清楚用什么代理的、代理值是多少。

E.3 证据：同一维度的两版来源标注

下面的原文摘录来自运行库 `market_sixdim_daily` 的 `sources` 字段。把它按日期排开，你能直接看到这套系统「从代理走向真实源」的过程——这不是我的叙述，是库里的字。

2026-09-02 (早期版本，多数维度在降级)：

- `breadth_rise`: 「全市场涨跌家数 (涨 0/总 9533)=0%」
- `capital`: 「北向无实时源 → 成交额连续放量天数代理 =0；资金集中度 = 行业 Top3 成交额占比

表 E.3 六维的数据源、降级链与缓存策略

维度	首选源 → 备选 → 兜底代理	超时 / 缓存 / 可信度标注
技术	DuckDB 真实指数日 K（指数 sh000001 直取）	无实时源；非交易时段直接走 DuckDB
广度	分层代表样本（种子 20240901 ⁷ ）→ 东财 push2ex 全市场涨停/跌停池 → 代表样本内统计	交易时段（含盘前竞价 09:15–15:05）用实时快照覆盖； 30 秒 realtimeSnapCache 去抖；样本 < 100 标注低可信度
量能	全市场成交额 / 20 日均（DuckDB 真实行情）	与广度同源；价量配合用 5 日 / 20 日量能趋势
资金结构	同花顺 hsgtApi 北向实时净流入（20 亿/档 → ±5）→ 东财融资余额趋势（FetchMarginHistory 连续增减天数）→ 兜底「成交额连续放量天数」	资金集中度用行业 Top3 成交额占比（≥ 40% 判集中）
情绪	同花顺官方涨停/跌停/炸板池 + 连板天梯 → 东财 push2ex → DuckDB 全市场统计 / 采样日线代理	炸板率无盘中分时，用日线「盘中触板未封 / 触板数」代理
外部约束	腾讯全球指数真实数据（qt.gtimg.cn）→ 指数跳空代理	4 秒超时 + 5 分钟缓存 ；全失败时退到上证指数开盘跳空

所有外部实时源统一走包级 ttlCache，并设最小 1 秒请求间隔限流。20 日新高/新低严格取当前 bar 之前的最近 20 根——这里曾出过一次把当前 bar 混进窗口的 off-by-one。

数据来源：系统源码 internal/marketsixdim/fetcher.go、sixdim.go；降级链与阈值对照系统事实清单第 8 节。核对时间 2026-09-24。

≥40%(true)]]

- external: 「无隔夜美股/事件实时源 → 用上证指数开盘跳空代理 (开盘缺口-0.16%(中性))」
- sentiment: 「采样 300 只日 K(全市场统计不可用降级): 涨停 0/跌停 0/炸板率 0%(日线代理)」

2026-09-09（改善后，多数维度已接真实源）:

- breadth_rise: 「代表样本涨跌家数 (涨 171/总 300)=57%」
- capital: 「北向实时 (同花顺 hsgtApi): 当日净流入 370 亿 → 连续流入口径 5; 资金集中度 = 行业 Top3 成交额占比 ≥40%(true)」
- external: 「真实隔夜外围 (tencent(qt.gtimg.cn) 全球指数实时: 偏空) 融合指数跳空 (开盘缺口 0.07%(中性))」
- sentiment: 「同花顺官方特色数据 (实时): 涨停 48/跌停 7/炸板率 36%(炸板 27)」

两版放在一起，差别一眼可见：9 月 2 日那句「涨 0/总 9533 = 0%」是字典序抽样的产物——样本几乎全落在北交所，于是「上涨占比 0%」；9 月 9 日改成代表样本后，同样的市场读出的是 57%。指标没变，抽样变了，结论从「普跌」变成「普涨」。

常见误解 · 代理变量也能算出漂亮的数字，所以更需要标注 降级到代理变量时，公式照跑，数字照出，看不出任何异常——9 月 2 日那天的广度就是 0%，程序不报错，界面也不闪红。这正是「代理变量必标注」这条纪律存在的理由：它不是给程序看的，是给人看的。你不去读 sources，就永远不知道这个 0% 到底是市场真的普跌，还是抽样抽歪了。

这也是本书引用数据时的一条硬规矩：凡是降级产生的数，一律标注它由什么代理而来；代理的覆盖率或样本量一并写出。系统里「样本 < 100 标低可信度」「rt.covered == 0 告警」这两条，都是把这条纪律写进了代码。

常见误解 · `sources` 写全了就一定可信 也不一定。库里 2026-08-31 那一条只写了 `tech` 一个来源 ("tech": "DuckDB 真实指数 K 线")，其余五个维度没写——那是早期版本，`sources` 标注还不规范。这条记录的 `raw_total_score` 也和我按现行权重手算的结果对不上（差 0.05）。本书的处理是如实记录、不作圆场：把它当作「标注不规范的早期样本」引用，并说明它与现行口径的差异。

一条实践结论：`sources` 的价值不在「有」，而在「全」。六个维度少写一个，你事后就无法判断那一维是真数据还是代理。

E.4 配置与接口一览

表 E.4 六维的配置项与对外接口

项	位置与默认值
源开关与优先级	配置文件 <code>config/config.json</code> 的 <code>sixdim_source</code> 节；设置页「市场信息数据 → 六维判势」卡片可展开 6 个源，改完保存即生效
落库开关	工具 <code>market_sixdim_detect</code> 的 <code>save</code> 参数：盘前 <code>true</code> 落库、盘中 <code>false</code> 不落库
查询接口	前端 <code>GetMarketSixDimReports(limit)</code> ，默认取 7 条、上限 30 条
分页与残留	落库表为 <code>market_sixdim_daily</code> ；本书写作时全表仅 5 ⁸ 条
数据来源：系统源码 <code>internal/marketsixdim/persist.go</code> 、 <code>internal/tools/sixdim_tool.go</code> ；配置项见 <code>internal/config/config.go</code> 。核对时间 2026-09-24。	

附录 E 的出处

- 权重、冲突修正、映射：`internal/marketsixdim/sixdim.go`
- 抓取与降级链、缓存与限流：`internal/marketsixdim/fetcher.go`
- 落库与查询：`internal/marketsixdim/persist.go`、`internal/tools/sixdim_tool.go`
- `sources` 原文：`market_sixdim_daily` 五条记录，2026-09-24 只读导出
- 建仓闸门与盘中告警：`internal/cio/cio.go`

动手试一试

1. 读一次 `sources`。查 `select trade_date, sources from market_sixdim_daily order by trade_date`，逐条看六个维度是不是都标了来源；找出标注不全的那一天。
2. 复算冲突修正。取 2026-09-10 那条：数一数六个分量里有多少个 ≥ 75 、多少个 ≤ 40 。若冲突数 ≥ 2 ，用 `raw_total_score` × 0.8 看能不能得到 `adjusted_total_score` (38.43)。
3. 看一次降级。在设置页把「六维判势」里情绪维度的首选源（同花顺官方）临时关掉，再跑一次盘前判势；对比 `sources` 里情绪那一格的文字，看它退到了哪一级。

附录 F

风险指标的数学定义与推导

正文第 chapter 二十二 章讲风险时，只留了直觉与定义，把长推导推到了这里；第 chapter 七 章讲 Kelly 缩放时同理。本附录把这两件事一次做干净，做三件事：

- (1) 把记号与假设补齐，让本附录可以脱离正文单独读；
- (2) 给出完整证明，并补上正文省略的中间步骤与推广——包括 VaR 次可加性失效的一般判据（正文只给了一个反例）、Rockafellar–Uryasev 形式的最优解集（正文只给了取值）、以及 CVaR 在谱风险度量族中的位置；
- (3) 把连续理论接到系统的有限样本实现上，回答一个很实际的问题：系统里那几行排序代码，算的到底是什么？

第 3 件事不只是补充，它还纠正了一处真实偏差。本附录会指出：系统的历史法在尾部取整上有一处可界定的偏差，另有一处方向性的口径缺陷。两者都给出了真实数据上的复算证据。

F.1 记号、假设与分位点的构造

F.1.1 记号

固定一个概率空间 $(\Omega, \mathcal{F}, \mathbb{P})$ 。 L 表示单期损失（亏损为正），其分布函数记为

$$F_L(\ell) = \mathbb{P}(L \leq \ell), \quad \ell \in \mathbb{R}, \quad (\text{F.1})$$

左极限记为 $F_L(\ell^-) = \mathbb{P}(L < \ell)$ 。置信水平 $\alpha \in (0, 1)$ ，并记

$$\lambda_\alpha = \frac{1}{1 - \alpha}. \quad (\text{F.2})$$

正文与全书的其余记号（ $\mathbb{E}$ 、 $\mathbf{1}\{\cdot\}$ 、 clamp 、 κ ）见前置的记号表。

假设 F.1 (H1: 可积性). $\mathbb{E}|L| < \infty$ 。这条保证后文所有分位数、条件期望与积分都有限、有意义。本附录的全部结论只需这一条（第 section F.7 节另需二阶矩，会单独点名）。

假设 F.2 (H2: 分布连续). F_L 处处连续且在尾部严格递增。这条不是必需品：它只是为了让「分位数形式」与「条件期望形式」的 CVaR 相等（theorem F.14）。去掉它，两种形式会分家——theorem F.15 会算给你看。

这两条假设与正文第 chapter 二十二 章的 H1、H2 是同一个东西，此处重写一遍只是为了让本附录能独立阅读。

F.1.2 下分位点与上分位点

正文的 VaR 定义只用到「不超过 ℓ 的概率已经够大」。要处理离散数据，必须把这件事拆成两个点。

定义 F.3 (下分位点、上分位点). 给定 $\alpha \in (0, 1)$ ，

$$q_\alpha^-(L) = \inf\{\ell \in \mathbb{R} : F_L(\ell) \geq \alpha\}, \quad q_\alpha^+(L) = \inf\{\ell \in \mathbb{R} : F_L(\ell) > \alpha\}. \quad (\text{F.3})$$

分别称为损失分布 F_L 的 α -下分位点与 α -上分位点。

引理 F.4 (分位点的刻画). 在假设 F.1 (其实只需 F_L 是分布函数) 下, 对一切 $\alpha \in (0, 1)$:

- (i) $q_\alpha^- \leq q_\alpha^+$;
- (ii) $F_L(q_\alpha^-) \geq \alpha$, 且对一切 $\ell < q_\alpha^-$ 有 $F_L(\ell) < \alpha$;
- (iii) $F_L(q_\alpha^+) \geq \alpha$, 且对一切 $\ell < q_\alpha^+$ 有 $F_L(\ell) \leq \alpha$;
- (iv) $q_\alpha^- < q_\alpha^+$ 当且仅当存在 $z_1 < z_2$ 使 $F_L(z_1^-) < \alpha$ 、 $F_L(z_2) > \alpha$ 、且 $F_L \equiv \alpha$ 于 $[z_1, z_2]$; 此时必有 $q_\alpha^- = z_1$ 、 $q_\alpha^+ = z_2$ 。

证明. 记 $A^- = \{\ell : F_L(\ell) \geq \alpha\}$ 、 $A^+ = \{\ell : F_L(\ell) > \alpha\}$, 则 $A^+ \subseteq A^-$, 故 $\inf A^- \leq \inf A^+$, 即 (i)。

(ii) 因 F_L 单调不减, A^- 是向上封闭的: $\ell \in A^-$ 且 $\ell' \geq \ell$ 蕴含 $\ell' \in A^-$; 又 $F_L(\ell) \rightarrow 1$ ($\ell \rightarrow +\infty$) 使 $A^- \neq \emptyset$ 。于是一列 $\ell_n \downarrow q_\alpha^-$ 、 $\ell_n \in A^-$ 存在, 由 F_L 右连续,

$$F_L(q_\alpha^-) = \lim_{n \rightarrow \infty} F_L(\ell_n) \geq \alpha. \quad (\text{F.4})$$

而若 $\ell < q_\alpha^-$, 则 $\ell \notin A^-$ (否则 ℓ 是 A^- 中比下确界更小的元素), 即 $F_L(\ell) < \alpha$ 。

(iii) 同法。对 $\ell < q_\alpha^+$ 有 $\ell \notin A^+$, 即 $F_L(\ell) \leq \alpha$; 右连续给出 $F_L(q_\alpha^+) = \lim F_L(\ell_n) \geq \alpha$ 。

(iv) ($\Leftarrow$) 设 z_1, z_2 如所述。对 $z < z_1$, 由单调性 $F_L(z) \leq F_L(z_1^-) < \alpha$, 故 $z \notin A^-$; 而 $F_L(z_1) = \alpha$ (若 $z_1 = z_2$ 已排除, 故 F_L 在 $[z_1, z_2]$ 上等于 α , 右连续给 $F_L(z_1) = \alpha$), 故 $z_1 \in A^-$ 。于是 $q_\alpha^- = z_1$ 。再看 A^+ : 对 $z < z_2$, 若 $z \geq z_1$ 则 $F_L(z) = \alpha \not> \alpha$, 若 $z < z_1$ 则 $F_L(z) < \alpha$, 总之 $z \notin A^+$, 故 $q_\alpha^+ \geq z_2$; 而 $F_L(z_2) > \alpha$ 说明 $z_2 \in A^+$, 故 $q_\alpha^+ \leq z_2$ 。合并得 $q_\alpha^+ = z_2$, 于是 $q_\alpha^- = z_1 < z_2 = q_\alpha^+$ 。

($\Rightarrow$) 设 $q_\alpha^- < q_\alpha^+$, 取 $z_1 = q_\alpha^-$ 、 $z_2 = q_\alpha^+$ 。先定 $F_L \equiv \alpha$ 于 (z_1, z_2) : 任取 $\ell \in (z_1, z_2)$, 由 A^- 向上封闭与 $z_1 = \inf A^-$, 存在 $z \in A^-$ 使 $z < \ell$, 从而 $\ell \in A^-$ 、即 $F_L(\ell) \geq \alpha$; 又由 (iii) 的「 $\ell < q_\alpha^+ \Rightarrow F_L(\ell) \leq \alpha$ 」得 $F_L(\ell) = \alpha$ 。右连续把等号延拓到左端点: $F_L \equiv \alpha$ 于 $[z_1, z_2]$ 。再定两端严格性。若 $F_L(z_1^-) = \alpha$, 则由右连续, z_1 左侧紧邻的点也在 A^- 中, 与 $z_1 = \inf A^-$ 矛盾, 故 $F_L(z_1^-) < \alpha$ 。若 $F_L(z_2) = \alpha$, 则 z_2 右侧紧邻的点也在 A^+ 的补集中, 与 $z_2 = \inf A^+$ 矛盾, 故 $F_L(z_2) > \alpha$ 。 ■

注 F.5. 唯一需要留心的是 (ii) 里的 A^- 向上封闭: 这保证了「下确界可达为最小值」, 从而 $F_L(q_\alpha^-) \geq \alpha$ 。这条看似琐碎的性质后面反复要用——它是「集合越大、下确界越小」这一整套论证的地基。

引理 F.6 (分位点对水平单调). $0 < \alpha \leq \beta < 1$ 蕴含 $q_\alpha^- \leq q_\beta^-$ 且 $q_\alpha^+ \leq q_\beta^+$ 。

证明. $F_L(\ell) \geq \beta \geq \alpha$ 说明 $A_\beta^- \subseteq A_\alpha^-$, 集合变大下确界变小, 故 $q_\alpha^- \leq q_\beta^-$; 对 A^+ 同理。 ■

F.2 VaR: 三条性质, 与它为什么不是一致风险度量

F.2.1 VaR 就是下分位点, 三条性质一次证完

正文的定义 22.4 写的是 $\inf\{\ell : \mathbb{P}(L \leq \ell) \geq \alpha\}$, 对照定义 F.3, 它就是 q_α^- 。这个观察让三条性质不必逐条硬算。

命题 F.7 (VaR 的三条性质). 设假设 F.1 成立, $\alpha \in (0, 1)$ 。则 $\text{VaR}_\alpha(L) = F_L^{-1}(\alpha)$ (广义逆), 且

- (i) 单调性: $L_1 \leq L_2$ 几乎处处 $\Rightarrow \text{VaR}_\alpha(L_1) \leq \text{VaR}_\alpha(L_2)$;
- (ii) 正齐次性: $\lambda > 0 \Rightarrow \text{VaR}_\alpha(\lambda L) = \lambda \text{VaR}_\alpha(L)$;
- (iii) 平移不变性: $c \in \mathbb{R} \Rightarrow \text{VaR}_\alpha(L + c) = \text{VaR}_\alpha(L) + c$ 。

证明. 第一步：广义逆的基本单调性。 设 $F \leq G$ 逐点成立（即 $F(\ell) \leq G(\ell)$ 对一切 ℓ ）。则对任意 α ,

$$\{\ell : F(\ell) \geq \alpha\} \supseteq \{\ell : G(\ell) \geq \alpha\}, \quad (\text{F.5})$$

前者作为更大的集合，下确界不超过后者，即 $F^{-1}(\alpha) \leq G^{-1}(\alpha)$ 。

第二步：分布函数对线性变换的协变性。 对 $\lambda > 0$ 与 $c \in \mathbb{R}$:

$$F_{\lambda L}(\ell) = \mathbb{P}(\lambda L \leq \ell) = F_L\left(\frac{\ell}{\lambda}\right), \quad F_{L+c}(\ell) = \mathbb{P}(L \leq \ell - c) = F_L(\ell - c). \quad (\text{F.6})$$

第三步：合并。 (i) $L_1 \leq L_2$ 几乎处处蕴含 $\{L_2 \leq \ell\} \subseteq \{L_1 \leq \ell\}$ （去掉零概率例外集），故 $F_{L_1} \geq F_{L_2}$ 逐点；由第一步

$$\text{VaR}_\alpha(L_1) = F_{L_1}^{-1}(\alpha) \leq F_{L_2}^{-1}(\alpha) = \text{VaR}_\alpha(L_2). \quad (\text{F.7})$$

(ii) 记 $\ell' = \ell/\lambda$ 。 $\{F_L(\ell') \geq \alpha\} = \lambda\{F_L(\ell') \geq \alpha\}$ 在变量替换下的像，且 $\lambda > 0$ 保序（保序映射把下确界映为下确界， $\inf(\lambda S) = \lambda \inf S$ ），故 $F_{\lambda L}^{-1}(\alpha) = \lambda F_L^{-1}(\alpha)$ 。

(iii) 平移同样保序， $\inf(S + c) = \inf S + c$ ，故 $F_{L+c}^{-1}(\alpha) = F_L^{-1}(\alpha) + c$ 。 ■

这个证法比正文里「逐个把集合比一遍」的写法更短，也更说明问题：**三条性质其实都不是关于 VaR 的性质，而是「广义逆的单调性」加上「分布函数在缩放与平移下的协变性」**。任何形如 $\inf\{\ell : \mathbb{P}(L \leq \ell) \geq \alpha\}$ 的量都会自动继承这三条——包括那些我们不想要的量。

F.2.2 VaR 次可加性失效的一般判据

正文用两只债券给了一个反例（例 22.7）。一个例子能说明「会失效」，但说明不了「什么时候失效」。下面是完整的判据。

定理 F.8 (VaR 次可加性失效的完整判据). 设 L_1, L_2 独立同分布，取值 $\{0, d\}$ ($d > 0$) 且 $\mathbb{P}(L_i = d) = p \in (0, 1)$ 。固定 $\alpha \in (0, 1)$ 。则

$$\text{VaR}_\alpha(L_1 + L_2) > \text{VaR}_\alpha(L_1) + \text{VaR}_\alpha(L_2) \quad (\text{F.8})$$

当且仅当

$$1 - \sqrt{\alpha} < p \leq 1 - \alpha. \quad (\text{F.9})$$

并且在区间 (F.9) 内部，违背量为 $\text{VaR}_\alpha(L_1 + L_2) = d$ 。

证明. 第一步：单只的 VaR。 F_{L_i} 只在 0 处有一个跳，且 $F_{L_i}(0) = 1 - p$ 、 $F_{L_i}(0^-) = 0$ 。对 $\ell < 0$ ， $\mathbb{P}(L_i \leq \ell) = 0 < \alpha$ ，故 $\ell \notin A^-$ ；对 $\ell \geq 0$ ， $\mathbb{P}(L_i \leq \ell) = 1 - p$ 。于是

$$\text{VaR}_\alpha(L_i) = 0 \iff 1 - p \geq \alpha \iff p \leq 1 - \alpha. \quad (\text{F.10})$$

当 $p > 1 - \alpha$ 时 $A^- = [d, +\infty)$ ， $\text{VaR}_\alpha(L_i) = d$ 。

第二步：和的分布。 由独立性，

$$\mathbb{P}(L_1 + L_2 = 0) = (1 - p)^2, \quad \mathbb{P}(= d) = 2p(1 - p), \quad \mathbb{P}(= 2d) = p^2. \quad (\text{F.11})$$

注意 $(1 - p)^2 + 2p(1 - p) = 1 - p^2$ 。于是 $F_{L_1+L_2}$ 有两个跳：在 0 处跳到 $(1 - p)^2$ ，在 d 处跳到 $1 - p^2$ 。

套用第一步的同一算法：

$$\text{VaR}_\alpha(L_1 + L_2) = \begin{cases} 0, & \alpha \leq (1-p)^2, \\ d, & (1-p)^2 < \alpha \leq 1-p^2, \\ 2d, & \alpha > 1-p^2. \end{cases} \quad (\text{F.12})$$

第三步：配对。违背 (F.8) 要求左边是 $2d$ （因为右边至多为 $2d$ ）或左边为 d 而右边为 0 。逐项检查：

- 左侧为 $2d$ 需要 $\alpha > 1-p^2$ ，此时由 (F.10) 有 $p > 1-\alpha$ ，右边 $= 2d$ ，不违背。
- 左侧为 d 且右侧为 0 ：左侧为 d 要求 $(1-p)^2 < \alpha$ ，右侧为 0 要求由 (F.10) 得 $p \leq 1-\alpha$ 。而 $(1-p)^2 < \alpha$ 等价于 $p > 1-\sqrt{\alpha}$ （两边开方，注意 $1-p \geq 0$ ）。
- 左侧为 0 时左边 $\leq$ 右边，不可能违背。

合并即得 (F.9)；此时左侧恰为 d ，故违背量为 d 。 ■

推论 F.9 (失效区间的宽度). 区间 (F.9) 的宽度为

$$(1-\alpha) - (1-\sqrt{\alpha}) = \sqrt{\alpha} - \alpha = \sqrt{\alpha}(1-\sqrt{\alpha}). \quad (\text{F.13})$$

代入 $\alpha = 0.95$ ： $\sqrt{0.95} = 0.974679$ ，区间为 $(0.025321, 0.05]$ ，宽度 0.024679 。正文例 22.7 取的 $p = 0.04$ 正落在这个区间里。

推论 F.10 (n 资产版). 设 $L_1, \dots, L_n$ 独立同分布，取值 $\{0, d\}$ 、 $\mathbb{P}(L_i = d) = p$ 。则

$$\text{VaR}_\alpha\left(\sum_{i=1}^n L_i\right) > \sum_{i=1}^n \text{VaR}_\alpha(L_i) \iff 1 - \alpha^{1/n} < p \leq 1 - \alpha. \quad (\text{F.14})$$

证明. $\sum L_i$ 的分布函数是二项累积： $\mathbb{P}(\sum L_i \leq kd) = \sum_{j \leq k} \binom{n}{j} p^j (1-p)^{n-j}$ 。单只的 $\text{VaR}_\alpha = d \cdot \mathbf{1}\{p > 1-\alpha\}$ （第一步）；和为 d 当且仅当 $\alpha > (1-p)^n$ ，即 $p > 1 - \alpha^{1/n}$ 。剩余两档（ 0 与 nd ）同 theorem F.8 第三步的排除法，不产生违背。 ■

注 F.11 (为什么这个缺陷很难被「看见」). 三件事让它隐形。第一，失效区间 (F.9) 的宽度是 $\sqrt{\alpha}(1-\sqrt{\alpha})$ ， $\alpha = 0.95$ 时只有 0.0247 ——需要尾部概率恰好落在「离 $1-\alpha$ 一步之内」的位置。第二，失效时左边的取值是 d 而不是 nd ：你只会看到「VaR 比你以为的大一点」，不会看到它爆炸。第三，违背是离散跳跃的： p 从 0.05 稍微降到 0.0499 ，VaR 就从 0 跳到 d ，中间没有过渡。

常见误解·「分散化一定降低风险」这条直觉，在 VaR 下是可以被推翻的 这不是理论洁癖。它意味着：如果你的风控用 VaR 做「组合是否合规」的判据，那么把两个各自合规的子组合起来，可能突然不合规——而你并没有加任何杠杆。

系统没有踩这个坑，因为在组合层用的是 CVaR（硬限制 8%）而不是 VaR。下一节证明为什么换一个就够了。记住这条原则：如果一个风险指标的失效条件是「尾部概率恰好落在某个窄区间」，那它在真实数据上是会撞上的，只是不常撞上。

F.3 CVaR: 两种定义何时分家, 以及一致性的完整证明

F.3.1 一个精确恒等式 (本附录的核心)

正文用「两边夹」的技巧证了 Rockafellar–Uryasev 形式, 那是为了迁就 H2。事实上有一个更干净的恒等式, 不需要任何连续性假设, 并且一次给出三个结论。

引理 F.12 (尾部积分的精确表达). 设假设 F.1 成立, $\alpha \in (0, 1)$, 记 $q = \text{VaR}_\alpha(L) = q_\alpha^-(L)$ 。则

$$\int_\alpha^1 \text{VaR}_u(L) du = (1 - \alpha)q + \mathbb{E}[(L - q)^+], \quad (\text{F.15})$$

从而

$$\text{CVaR}_\alpha(L) = q + \lambda_\alpha \mathbb{E}[(L - q)^+], \quad \lambda_\alpha = \frac{1}{1 - \alpha}. \quad (\text{F.16})$$

证明. 记 $q(\alpha) = q_\alpha^-(L) = \text{VaR}_\alpha(L)$, 并令

$$T(\alpha) = \int_\alpha^1 q(u) du, \quad R(\alpha) = (1 - \alpha)q(\alpha) + \mathbb{E}[(L - q(\alpha))^+]. \quad (\text{F.17})$$

要证 $T \equiv R$ 。

第一步: 一条逐点积分恒等式. 设 $z_1 < z_2$ 。对每个 ω ,

$$(L - z_1)^+ - (L - z_2)^+ = \int_{z_1}^{z_2} \mathbf{1}\{L > t\} dt \quad (\text{F.18})$$

($L > z_2$ 时两边都是 $z_2 - z_1$; $z_1 < L \leq z_2$ 时两边都是 $L - z_1$; $L \leq z_1$ 时两边都是 0)。两边取期望并用 Fubini:

$$\mathbb{E}[(L - z_1)^+] - \mathbb{E}[(L - z_2)^+] = \int_{z_1}^{z_2} \mathbb{P}(L > t) dt. \quad (\text{F.19})$$

这条式子是后面全部论证的地基: 它说 $\mathbb{E}[(L - z)^+]$ 关于 z 绝对连续, 且几乎处处导数为 $-\mathbb{P}(L > z)$ 。

第二步: T 与 R 都连续. T 是可积函数的变上限积分 ($t \mapsto q(t)$ 在 $[0, 1]$ 上可积, 因为 $\int_0^1 q(t) dt = \mathbb{E}L$ 有限), 故绝对连续。 R 只需检查 q 的跳跃点: q 单调不减, 跳跃点至多可数。设 α_0 是一个跳跃点, $z_1 = q(\alpha_0^-)$ 、 $z_2 = q(\alpha_0)$ 。由引理 F.4(iv), $F_L \equiv \alpha_0$ 于 $[z_1, z_2)$, 即 (z_1, z_2) 内不含 L 的任何概率质量, 于是 $\mathbb{P}(L > t) = 1 - F_L(t) = 1 - \alpha_0$ 在 (z_1, z_2) 上是常数。由 (F.19),

$$\mathbb{E}[(L - z_2)^+] = \mathbb{E}[(L - z_1)^+] - (1 - \alpha_0)(z_2 - z_1), \quad (\text{F.20})$$

代入 R 的单侧极限:

$$R(\alpha_0^-) - R(\alpha_0) = (1 - \alpha_0)(z_1 - z_2) + (1 - \alpha_0)(z_2 - z_1) = 0. \quad (\text{F.21})$$

故 R 在跳跃点亦连续, 从而处处连续。

第三步: 几乎处处导数相同. T 在 q 的连续点处可导, $T'(\alpha) = -q(\alpha)$ 。由链式法则与 (F.19), 在 q 可导处

$$R'(\alpha) = -q + (1 - \alpha)q' - q' \mathbb{P}(L > q(\alpha)) = -q + q'[(1 - \alpha) - (1 - F_L(q(\alpha)))] = -q + q'[F_L(q(\alpha)) - \alpha]. \quad (\text{F.22})$$

若 $q'(\alpha) \neq 0$, 则 α 落在 F_L 的严格增段上, 此时 F_L 在该点连续且 $F_L(q(\alpha)) = \alpha$ (由引理 F.4(ii) 的

$F_L(q) \geq \alpha$ ，若严格大于则该处有跳跃、 q 不会在 α 附近严格增，矛盾），括号为零；若 $q'(\alpha) = 0$ ，第二项亦为零。两种情形都给出

$$R'(\alpha) = -q(\alpha) = T'(\alpha) \quad \text{几乎处处.} \quad (\text{F.23})$$

第四步：定常数。由第二步， $T - R$ 连续；由第三步， $(T - R)' = 0$ 几乎处处，故 $T - R$ 恒为常数。令 $\alpha \uparrow 1$ ：一方面 $T(\alpha) = \int_{\alpha}^1 q \rightarrow 0$ ；另一方面 $q(\alpha)$ 单调不减，其极限是 L 的本质确界（或 $+\infty$ ），在 $\mathbb{E}|L| < \infty$ 下 $\mathbb{E}[(L - q(\alpha))^+] \rightarrow 0$ ，故 $R(\alpha) \rightarrow 0$ 。两端点极限相同，常数只能是 0，即 $T \equiv R$ ，(F.15) 得证。两边同除 $1 - \alpha > 0$ 即得 (F.16)。■

注 F.13 (这个恒等式为什么值得单独讲)。它把三件事一次说清。第一，(F.15) 是恒等而非不等式：Rockafellar–Uryasev 形式在 $t = q$ 处不是「可以取到」，而是「必然取到」。第二，它不需要 H2——正文为了走「换元 $u = F_L(x)$ 」的路必须假设分布连续，本证只用了一条逐点积分恒等式 (F.18)，对有原子的分布一样成立。第三，右边第二项非负，立刻给出

$$\text{CVaR}_{\alpha}(L) \geq \text{VaR}_{\alpha}(L), \quad (\text{F.24})$$

这比正文里「先证 VaR 对水平单调、再逐点比较积分」的两步论证短得多。

F.3.2 两种定义在什么时候分家

正文的定义 22.8 给了 CVaR 两种写法：积分形式 (22.14) 与条件期望形式 (22.15)。正文说「去掉 H2 它们会分家」，下面把它算准。

推论 F.14 (两种形式的一致性条件)。设假设 F.1 成立， $q = \text{VaR}_{\alpha}(L)$ 。则

$$\text{CVaR}_{\alpha}(L) - \mathbb{E}[L \mid L \geq q] = \mathbb{E}[(L - q)^+] \cdot \left(\lambda_{\alpha} - \frac{1}{\mathbb{P}(L \geq q)} \right) \geq 0, \quad (\text{F.25})$$

且等号成立当且仅当

$$\mathbb{E}[(L - q)^+] = 0 \quad \text{或} \quad \mathbb{P}(L \geq q) = 1 - \alpha. \quad (\text{F.26})$$

特别地，H2 蕴含两者相等。

证明。由 (F.16)，

$$\text{CVaR}_{\alpha}(L) = q + \lambda_{\alpha} \mathbb{E}[(L - q)^+]. \quad (\text{F.27})$$

又

$$\mathbb{E}[L \mid L \geq q] = \frac{\mathbb{E}[L \mathbf{1}\{L \geq q\}]}{\mathbb{P}(L \geq q)} = \frac{\mathbb{E}[(L - q)^+] + q \mathbb{P}(L \geq q)}{\mathbb{P}(L \geq q)} = q + \frac{\mathbb{E}[(L - q)^+]}{\mathbb{P}(L \geq q)}, \quad (\text{F.28})$$

相减即得 (F.25)。

非负性来自引理 F.4(iii)： $\mathbb{P}(L \geq q) = \mathbb{P}(L > q) + \mathbb{P}(L = q) = 1 - F_L(q^-)$ ，而 $F_L(q^-) \leq \alpha$ (q 的「左侧一律 $< \alpha$ 」性质在 q^- 处的极限形式)，故 $\mathbb{P}(L \geq q) \geq 1 - \alpha$ ，即 $1/\mathbb{P}(L \geq q) \leq \lambda_{\alpha}$ 。等号条件由各因子非负直接读出。H2 下 $F_L(q) = F_L(q^-) = \alpha$ 且无原子，故 $\mathbb{P}(L \geq q) = 1 - \alpha$ 。■

例 F.15 (一个有原子的算例：两种形式差多少)。设

$$\mathbb{P}(L = -1) = 0.01, \quad \mathbb{P}(L = 0) = 0.94, \quad \mathbb{P}(L = 100) = 0.05, \quad (\text{F.29})$$

取 $\alpha = 0.5$ 。 $F_L(-1) = 0.01$, $F_L(0) = 0.95 \geq 0.5$, 故 $q_{0.5}^- = 0$ 。由定义 (22.14):

$$\text{VaR}_u = \begin{cases} -1, & u \in (0, 0.01], \\ 0, & u \in (0.01, 0.95], \\ 100, & u \in (0.95, 1], \end{cases} \quad \text{CVaR}_{0.5} = \frac{1}{0.5} \int_{0.5}^1 \text{VaR}_u du = \frac{100 \times 0.05}{0.5} = 10. \quad (\text{F.30})$$

而条件期望形式给出

$$\mathbb{E}[L \mid L \geq 0] = \frac{0 \times 0.94 + 100 \times 0.05}{0.94 + 0.05} = \frac{5}{0.99} = 5.0505. \quad (\text{F.31})$$

两者相差 4.9495——**将近一倍**。核对 (F.16): $q + \lambda_\alpha \mathbb{E}[(L - q)^+] = 0 + 2 \times 5 = 10$ 。¹

这里有个反直觉的结论要记住：**条件期望形式 (22.15) 一般不是 CVaR，它是 CVaR 的下界**。H2 让它们相等，所以正文的情形看不出差别；但系统的历史法算的恰恰是**经验分布**——一个处处有原子的分布。这个差别在第 F.6 节会变成可界定的一个尾部条数。

F.3.3 一致性的完整证明

定理 F.16 (CVaR 满足四条一致性公理)。设假设 F.1 成立, $\alpha \in (0, 1)$ 。则

- (i) $L_1 \leq L_2$ 几乎处处 $\Rightarrow \text{CVaR}_\alpha(L_1) \leq \text{CVaR}_\alpha(L_2)$;
- (ii) $\lambda > 0 \Rightarrow \text{CVaR}_\alpha(\lambda L) = \lambda \text{CVaR}_\alpha(L)$;
- (iii) $c \in \mathbb{R} \Rightarrow \text{CVaR}_\alpha(L + c) = \text{CVaR}_\alpha(L) + c$;
- (iv) $\text{CVaR}_\alpha(L_1 + L_2) \leq \text{CVaR}_\alpha(L_1) + \text{CVaR}_\alpha(L_2)$ 。

证明. (i) 由命题 F.7(i), $u \mapsto \text{VaR}_u$ 逐点不大于; 被积函数逐点不大, 积分就不大, 同乘 $\lambda_\alpha > 0$ 保持不等号。

(ii) 由命题 F.7(ii), $\text{VaR}_u(\lambda L) = \lambda \text{VaR}_u(L)$ 对每个 u , 常数提出积分。

(iii) 由命题 F.7(iii),

$$\text{CVaR}_\alpha(L + c) = \lambda_\alpha \int_\alpha^1 (\text{VaR}_u(L) + c) du = \text{CVaR}_\alpha(L) + \lambda_\alpha c(1 - \alpha) = \text{CVaR}_\alpha(L) + c. \quad (\text{F.32})$$

(iv) 这一步用积分形式无法直接推——分位函数之间逐点没有可利用的不等式 (theorem F.8 的反例正是逐点反过来的)。改用共轭表示。它在逻辑上可以先于 (iv) 证明: theorem F.20 的全部论证只用到了 theorem F.18(iii) 与引理 F.4, 不依赖本节任何结论。于是任取 $Z \in \mathcal{Z}_\alpha$,

$$\mathbb{E}[Z(L_1 + L_2)] = \mathbb{E}[ZL_1] + \mathbb{E}[ZL_2] \leq \text{CVaR}_\alpha(L_1) + \text{CVaR}_\alpha(L_2), \quad (\text{F.33})$$

末步是对两项分别用 theorem F.20 的「 $\mathbb{E}[ZL] \leq \text{CVaR}_\alpha(L)$ 对一切 $Z \in \mathcal{Z}_\alpha$ 成立」一侧。右边与 Z 无关, 故对 Z 取上确界仍不超过它, 再次用 theorem F.20 得 (iv)。■

注 F.17 (为什么积分形式证不了次可加性). 想直接对 u 逐点比较会撞墙: $\text{VaR}_u(L_1 + L_2)$ 与 $\text{VaR}_u(L_1) + \text{VaR}_u(L_2)$ 之间**没有任何逐点不等式**——theorem F.8 的反例正是逐点反过来的。次可加性必须跳出分位函数, 进到「重新加权期望」的世界里才看得见。这也是共轭表示真正的用途。

¹数据来源: 本附录为说明 theorem F.14 构造的手算算例, 非系统运行数据。

F.4 Rockafellar–Uryasev 形式与共轭表示

F.4.1 RU 形式：凸性、最优解集、取值

定理 F.18 (Rockafellar–Uryasev 形式). 设假设 F.1 成立, $\alpha \in (0, 1)$, λ_α 如 (F.2)。定义

$$\Psi(t) = t + \lambda_\alpha \mathbb{E}[(L - t)^+], \quad t \in \mathbb{R}. \quad (\text{F.34})$$

则

(i) Ψ 是凸的、实值、以 $\max(1, \lambda_\alpha - 1)$ 为 Lipschitz 常数;

(ii) Ψ 的右导数 $\Psi'(t^+) = 1 - \lambda_\alpha \mathbb{P}(L > t)$ 、左导数 $\Psi'(t^-) = 1 - \lambda_\alpha \mathbb{P}(L \geq t)$, 且

$$\arg \min_{t \in \mathbb{R}} \Psi(t) = [t : F_L(t^-) \leq \alpha \leq F_L(t)] = [q_\alpha^-(L), q_\alpha^+(L)]; \quad (\text{F.35})$$

(iii) $\min_t \Psi(t) = \text{CVaR}_\alpha(L)$ 。

证明. (i) 对每个固定的 ω , $t \mapsto (L(\omega) - t)^+ = \max\{L(\omega) - t, 0\}$ 是两个仿射函数的最大值, 故凸; 期望保凸, 故 Ψ 凸。有限性: $(L - t)^+ \leq |L| + |t|$, 与 $\mathbb{E}|L| < \infty$ 合起来给出 $|\Psi(t)| < \infty$ 。

右导数: 对 $\varepsilon > 0$,

$$\frac{\Psi(t + \varepsilon) - \Psi(t)}{\varepsilon} = 1 + \lambda_\alpha \frac{\mathbb{E}[(L - t - \varepsilon)^+] - \mathbb{E}[(L - t)^+]}{\varepsilon}. \quad (\text{F.36})$$

被减项关于 ε 单调 (ε 越大、截断越狠), 故差商单调且逐点收敛到 $-\mathbb{P}(L > t)$ (对每个 ω , $(L - t - \varepsilon)^+$ 与 $(L - t)^+$ 之差在 $\{L > t\}$ 上为 $-\varepsilon \wedge (L - t)$ 、否则为 0); 单调收敛定理给出 $\Psi'(t^+) = 1 - \lambda_\alpha \mathbb{P}(L > t)$ 。左导数同法用 $\mathbb{P}(L \geq t)$ 。

Lipschitz: 由 $\mathbb{P}(L > t) \in [0, 1]$ 得 $\Psi'(t^+) \in [1 - \lambda_\alpha, 1]$; 左导数同。凸函数的导数单调, 故 $|\Psi'(t)| \leq \max(\lambda_\alpha - 1, 1)$ 于一切连续点, 取稠密列并积分得 Lipschitz 常数 $\max(1, \lambda_\alpha - 1)$ 。

(ii) 记 $A(t) = F_L(t) = 1 - \mathbb{P}(L > t)$ 。则 $\Psi'(t^+) \geq 0$ 当且仅当 $\mathbb{P}(L > t) \leq 1 - \alpha$, 当且仅当 $A(t) \geq \alpha$, 当且仅当 $t \geq q_\alpha^-$ 。于是 Ψ 在 $(-\infty, q_\alpha^-]$ 上非增、在 $[q_\alpha^-, \infty)$ 上非减, 最小值必在 $t = q_\alpha^-$ 取到。

另一半: $\Psi'(t^-) \leq 0$ 当且仅当 $\mathbb{P}(L \geq t) \geq 1 - \alpha$, 当且仅当 $F_L(t^-) \leq \alpha$, 当且仅当 $t \leq q_\alpha^+$ 。于是 Ψ 在 $(-\infty, q_\alpha^+]$ 上非增。合并: 下降区是 $(-\infty, q_\alpha^-]$, 平坦或上升区从 q_α^- 开始, 而在 q_α^+ 之后严格上升——最小值集合正是 $[q_\alpha^-, q_\alpha^+]$, 即 (F.35)。

(iii) 由引理 F.12 的 (F.16), $\Psi(q_\alpha^-) = \text{CVaR}_\alpha(L)$; 而 q_α^- 是最小值点 ((ii)), 故 $\min \Psi = \text{CVaR}_\alpha$ 。■

注 F.19 (与正文版本的差别). 正文的引理 22.12 是在 H2 下证的, 只说「下确界在 $t = q$ 处取到」。这里得到了更强的结论: 最优解集恰好是整个区间 $[q_\alpha^-, q_\alpha^+]$, 且在 F_L 有平坦段时这个区间非退化。这对数值实现有用——梯度下降落在区间里的任何一点都得到同一个 Ψ 值。

F.4.2 共轭（上确界）表示

定理 F.20 (共轭表示, 含原子情形). 设假设 F.1 成立, $\lambda_\alpha = (1 - \alpha)^{-1}$, 令

$$\mathcal{Z}_\alpha = \left\{ Z \in L^0(\Omega) : 0 \leq Z \leq \lambda_\alpha \text{ a.s., } \mathbb{E}Z = 1 \right\}. \quad (\text{F.37})$$

则

$$\text{CVaR}_\alpha(L) = \sup_{Z \in \mathcal{Z}_\alpha} \mathbb{E}[ZL], \quad (\text{F.38})$$

且上确界被下式达到：

$$Z^* = \lambda_\alpha \mathbf{1}\{L > q\} + \beta \mathbf{1}\{L = q\}, \quad \beta = \frac{1 - \lambda_\alpha \mathbb{P}(L > q)}{\mathbb{P}(L = q)}, \quad (\text{F.39})$$

其中 $q = \text{VaR}_\alpha(L)$ ；当 $\mathbb{P}(L = q) = 0$ 时第二项去掉，此时 β 任意（约定为 0）。若 $\mathbb{P}(L = q) > 0$ ，则 $\beta \in [0, \lambda_\alpha]$ 。

证明. 第一步（上界）。任取 $Z \in \mathcal{Z}_\alpha$ 与任意 $t \in \mathbb{R}$ 。由 $L - t \leq (L - t)^+$ 与 $Z \geq 0$ 得 $(L - t)Z \leq (L - t)^+ Z$ ；再由 $(L - t)^+ \geq 0$ 与 $Z \leq \lambda_\alpha$ 得 $(L - t)^+ Z \leq \lambda_\alpha (L - t)^+$ 。于是

$$\mathbb{E}[ZL] = t\mathbb{E}Z + \mathbb{E}[(L - t)Z] = t + \mathbb{E}[(L - t)Z] \leq t + \lambda_\alpha \mathbb{E}[(L - t)^+] = \Psi(t), \quad (\text{F.40})$$

对 t 取下确界并用 theorem F.18(iii) 得 $\mathbb{E}[ZL] \leq \text{CVaR}_\alpha(L)$ 。对 Z 取上确界仍成立。

第二步（ Z^* 合法）。 $0 \leq Z^* \leq \lambda_\alpha$ 显然。若 $\mathbb{P}(L = q) > 0$,

$$\beta \geq 0 \iff 1 \geq \lambda_\alpha \mathbb{P}(L > q) \iff \mathbb{P}(L > q) \leq 1 - \alpha \quad (F_L(q) \geq \alpha), \quad (\text{F.41})$$

$$\beta \leq \lambda_\alpha \iff 1 \leq \lambda_\alpha (\mathbb{P}(L > q) + \mathbb{P}(L = q)) = \lambda_\alpha \mathbb{P}(L \geq q) \quad (\text{F.42})$$

$$\iff \mathbb{P}(L \geq q) \geq 1 - \alpha \quad (F_L(q^-) \leq \alpha), \quad (\text{F.43})$$

两条都由引理 F.4 保证成立。于是 $\beta \in [0, \lambda_\alpha]$ ；且由 β 的定义直接得

$$\mathbb{E}Z^* = \lambda_\alpha \mathbb{P}(L > q) + \beta \mathbb{P}(L = q) = 1. \quad (\text{F.44})$$

第三步（ Z^* 达到上确界）。设 $\mathbb{P}(L = q) > 0$ （否则直接取 $Z^* = \lambda_\alpha \mathbf{1}\{L > q\}$ ，第二项为 0，计算同样成立）。则

$$\begin{aligned} \mathbb{E}[Z^*L] &= \lambda_\alpha \mathbb{E}[L \mathbf{1}\{L > q\}] + \beta q \mathbb{P}(L = q) \\ &= \lambda_\alpha \mathbb{E}[L \mathbf{1}\{L > q\}] + q(1 - \lambda_\alpha \mathbb{P}(L > q)) \\ &= q + \lambda_\alpha (\mathbb{E}[L \mathbf{1}\{L > q\}] - q \mathbb{P}(L > q)) = q + \lambda_\alpha \mathbb{E}[(L - q)^+] = \text{CVaR}_\alpha(L), \end{aligned} \quad (\text{F.45})$$

末步用了 (F.16)。结合第一步，(F.38) 成立。 ■

注 F.21（一句人话）。(F.38) 说的是：把「尾部平均」翻译成「最坏重加权的平均」。 $\mathcal{Z}_\alpha$ 里的每个 Z 是一个「概率再加权器」：权重只能放大、最多放大 λ_α 倍，且总质量仍为 1。CVaR 就是在这个权重族里选最坏的那个。 $\mathcal{Z}_\alpha$ 是凸紧的，(F.38) 是一个不涉及优化的闭式上确界——这正是它比「 $\inf_t \Psi(t)$ 」好用的地方：sup 对加法线性，inf 不是。次可加性就是这样来的。

推论 F.22 (CVaR 是凸风险度量)。在假设 F.1 下， $L \mapsto \text{CVaR}_\alpha(L)$ 是凸的、正齐次的，因而是凸风险度量；(theorem F.16) 再加单调性与平移不变性就是一致风险度量。

证明. 凸性：(F.38) 的右边是一族线性泛函 $L \mapsto \mathbb{E}[ZL]$ 的上确界，线性泛函之集的上确界是凸函数。正齐次性： $L \mapsto \mathbb{E}[ZL]$ 正齐次，上确界亦正齐次（ $\lambda > 0$ 时 $\sup_Z \mathbb{E}[Z\lambda L] = \lambda \sup_Z \mathbb{E}[ZL]$ ）。 ■

F.5 谱风险度量：CVaR 在度量族中的位置

次可加性是「一族线性泛函的上确界」给的，那么反过来问：哪些风险度量能写成这种上确界的形式？答案是谱风险度量。这个视角能一次说清 VaR 和 CVaR 的关系。

定义 F.23 (谱风险度量). 设权重函数 $\phi : [0, 1] \rightarrow [0, \infty)$ 满足

$$\phi \text{ 非减}, \quad \int_0^1 \phi(u) du = 1, \quad (\text{F.46})$$

并设 $\int_0^1 \phi(u) |\text{VaR}_u(L)| du < \infty$ 。定义

$$\rho_\phi(L) = \int_0^1 \phi(u) \text{VaR}_u(L) du. \quad (\text{F.47})$$

命题 F.24 (CVaR 是谱风险度量). 取阶梯权重 $\phi_\alpha(u) = \lambda_\alpha \mathbf{1}\{u > \alpha\}$ ，则 ϕ_α 非减、 $\int_0^1 \phi_\alpha = 1$ ，且

$$\rho_{\phi_\alpha} = \text{CVaR}_\alpha. \quad (\text{F.48})$$

证明. ϕ_α 在 $u < \alpha$ 上为 0、在 $u > \alpha$ 上为 $\lambda_\alpha > 0$ ，非减； $\int_0^1 \phi_\alpha = \lambda_\alpha(1 - \alpha) = 1$ 。代入 (F.47) 得 $\rho_{\phi_\alpha}(L) = \lambda_\alpha \int_\alpha^1 \text{VaR}_u(L) du$ ，正是定义 22.8 的积分形式。■

定理 F.25 (谱风险度量都是一致的). 设 ϕ 如定义 F.23。则 ρ_ϕ 满足单调性、正齐次性、平移不变性与次可加性。

证明. 第一步：阶梯权重的乘积结构。设 $\phi = \sum_{j=1}^m w_j \phi_{\alpha_j}$ ，其中 $w_j \geq 0$ 、 $\sum_j w_j = 1$ 、 $0 < \alpha_1 \leq \dots \leq \alpha_m < 1$ ，且每个 ϕ_{α_j} 是命题 F.24 的阶梯权重（即 $\rho_{\phi_{\alpha_j}} = \text{CVaR}_{\alpha_j}$ ）。由 (F.47) 的线性性，

$$\rho_\phi(L) = \sum_{j=1}^m w_j \text{CVaR}_{\alpha_j}(L), \quad (\text{F.49})$$

即 ρ_ϕ 是若干 CVaR 的凸组合。于是四条公理逐条继承：单调（theorem F.16(i)）、正齐次（(ii)）、平移不变（(iii)）都是「有限个成立的量做加权后仍成立」；次可加性用 theorem F.16(iv)：

$$\rho_\phi(L_1 + L_2) = \sum_j w_j \text{CVaR}_{\alpha_j}(L_1 + L_2) \leq \sum_j w_j (\text{CVaR}_{\alpha_j}(L_1) + \text{CVaR}_{\alpha_j}(L_2)) = \rho_\phi(L_1) + \rho_\phi(L_2). \quad (\text{F.50})$$

第二步：一般 ϕ 的逼近. 对一般的非减 ϕ ，取一列非减阶梯函数 $\phi^{(m)}$ 满足 $0 \leq \phi^{(m)} \uparrow \phi$ 且 $\int \phi^{(m)} \uparrow 1$ ；把每个 $\phi^{(m)}$ 归一化后套用第一步，再令 $m \rightarrow \infty$ 。积分号下的收敛由控制收敛定理保证——控制函数可取 $\phi(u) |\text{VaR}_u(L)|$ ，其可积性已由定义 F.23 假定。 $|\rho_{\phi^{(m)}}(L) - \rho_\phi(L)| \rightarrow 0$ 逐条传递四条公理（不等号在极限下保持）。■

注 F.26 (VaR 不在这个族里，这不是巧合). VaR 想写成 (F.47) 只能取 $\phi = \delta_\alpha$ （ α 处的 Dirac 质量）。Dirac 既不是非减函数、也不可积——系数 ϕ 必须「把越惨的分位给越大的权重」才有次可加性，而 δ_α 只把全部权重压在一条线上，线外的惨况一律不给权重。这就是它失效的根本原因，与 theorem F.8 的窄区间条件是一回事的两个侧面：尾部窄区间被忽略得越彻底，分散化的直觉就越守不住。

注 F.27 (系统为什么还是留着 VaR). 谱度量比 VaR「更对」，但系统同时报 VaR95、VaR99、CVaR95 三个数。原因不是犹豫：VaR 是分位点，可以被读成「某一天亏了多少钱」；CVaR 是尾部均值，读成「真

出事那天平均亏多少」。两个问题的答案都有用——只是做合规判据时必须用后者（系统的 CVaR 硬限制是 8%）。

F.6 有限样本：系统的历史法算的到底是什么

到这里都是连续理论。系统的实现是有限样本的排序，两者之间隔着一「取整」，还隔着一处我们即将指出的方向性缺陷。这一节把两者接上。

F.6.1 经验分布与顺序统计量

设系统拿到 n 个样本 $r_1, \dots, r_n$ ，把它当成损失的经验分布： L 等概率地取 $L_j = -r_j$, $j = 1, \dots, n$ 。记损失的顺序统计量

$$L_{(1)} \leq L_{(2)} \leq \dots \leq L_{(n)}, \quad A_m := \frac{1}{m} \sum_{i=n-m+1}^n L_{(i)} \quad (1 \leq m \leq n), \quad (\text{F.51})$$

即 A_m 是「最惨的 m 条」的平均值。经验分布的分位函数为

$$\text{VaR}_u^{\text{emp}} = \inf\{\ell : F_L^{\text{emp}}(\ell) \geq u\} = L_{(\lceil nu \rceil)}, \quad u \in (0, 1], \quad (\text{F.52})$$

($\lceil \cdot \rceil$ 上取整)。

引理 F.28 (尾部平均的层级结构). A_m 关于 m 非增，且

$$A_m - A_{m+1} = \frac{1}{m+1} (A_m - L_{(n-m)}) \geq 0. \quad (\text{F.53})$$

证明. 由 $A_{m+1} = \frac{mA_m + L_{(n-m)}}{m+1}$ (把最惨的 $m+1$ 条拆成「最惨的 m 条」与「第 $m+1$ 惨的那一条」) 直接相减：

$$A_m - A_{m+1} = A_m - \frac{mA_m + L_{(n-m)}}{m+1} = \frac{(m+1)A_m - mA_m - L_{(n-m)}}{m+1} = \frac{A_m - L_{(n-m)}}{m+1} \geq 0, \quad (\text{F.54})$$

非负性来自 $L_{(n-m)} \leq A_m$ (平均值不小于参与平均的最小者)。■

定理 F.29 (系统算法与理论口径的精确对应). 设样本量 n 、置信水平 $\alpha \in (0, 1)$ ，记

$$k = \lfloor (1-\alpha)n \rfloor, \quad m = k+1, \quad \tilde{\alpha} = 1 - \frac{m}{n}. \quad (\text{F.55})$$

则 $m \in ((1-\alpha)n, (1-\alpha)n+1]$ 、 $\tilde{\alpha} \in [\alpha - 1/n, \alpha)$ ，并且系统源码 `internal/risk/engine.go` 的两个历史法估计量恰为

$$\widehat{\text{VaR}}_\alpha = |L_{(n-k)}| = q_\alpha^+(L^{\text{emp}}), \quad \widehat{\text{CVaR}}_\alpha = |A_m| = \left| \frac{1}{m} \sum_{i=n-m+1}^n L_{(i)} \right|. \quad (\text{F.56})$$

在经验分布上， CVaR 的积分形式与条件期望形式在水平 $\tilde{\alpha}$ 处相等，且都等于 A_m ：

$$A_m = \text{CVaR}_{\tilde{\alpha}}^{\text{emp}} = \mathbb{E}^{\text{emp}}[L \mid L \geq \text{VaR}_{\tilde{\alpha}}^{\text{emp}}]. \quad (\text{F.57})$$

证明. 第一步：尾部条数. 由 $k = \lfloor (1-\alpha)n \rfloor$ 得 $k \leq (1-\alpha)n < k+1$ ，故 $m = k+1 \in ((1-\alpha)n, (1-\alpha)n+1]$ 。

除以 n : $m/n \in (1 - \alpha, 1 - \alpha + 1/n]$, 故 $\tilde{\alpha} = 1 - m/n \in [\alpha - 1/n, \alpha)$ 。

第二步: VaR. k 是 Go 代码里 `idx` 的取值 (0 基), `sorted[idx]` 即 $L_{(n-k)}$ (因为升序排列的第 $k+1$ 个最小样本, 对应第 $k+1$ 大的损失)。经验分布有 n 个等概率原子, $F_L^{\text{emp}}(L_{(n-k)}) = (n-k)/n = 1 - k/n = \tilde{\alpha} + 1/n > \tilde{\alpha}$, 而 $F_L^{\text{emp}}(L_{(n-k-1)}) = \tilde{\alpha}$ 不大于 $\tilde{\alpha}$ 。故按定义 F.3, $q_{\tilde{\alpha}}^+ = L_{(n-k)}$ 。

第三步: CVaR. 取 $u \in (\tilde{\alpha}, 1]$, 则 $\lceil nu \rceil \geq \lceil n\tilde{\alpha} \rceil = n - m + 1$, 故 $\text{VaR}_u^{\text{emp}} = L_{(\lceil nu \rceil)} \geq L_{(n-m+1)}$ 。于是

$$\int_{\tilde{\alpha}}^1 \text{VaR}_u^{\text{emp}} du = \frac{1}{n} \sum_{i=n-m+1}^n L_{(i)} = \frac{m}{n} A_m, \quad (\text{F.58})$$

两边除以 $1 - \tilde{\alpha} = m/n$ 得积分形式 $= A_m$ 。

第四步: 两形式相等。 经验分布有原子, 一般情形两形式会分家 (theorem F.14)。但在水平 $\tilde{\alpha}$ 处, $\mathbb{P}^{\text{emp}}(L \geq L_{(n-m+1)}) = m/n = 1 - \tilde{\alpha}$, 恰好命中 theorem F.14 的等号条件, 故条件期望形式也等于 A_m 。(F.57) 得证。

第五步: 源码口径。 $k = \lfloor (1 - \alpha)n \rfloor$ 与 `idx` 一致; `tail := sorted[:idx+1]` 长度恰为 $m = k + 1$; `sum/len(tail)` 即 A_m ; 外围的 `math.Abs` 把损失取成正的绝对值。 $\widehat{\text{VaR}}_{\alpha} = |L_{(n-k)}|$ 亦同。 ■

推论 F.30 (取整偏差的界). 记名义尾部条数 $m_0 = (1 - \alpha)n \in (0, n)$, $m = \lfloor m_0 \rfloor + 1$ 。则 $\lfloor m_0 \rfloor = m - 1 < m_0 < m$, 且「多取一条尾部观测」所付的代价为

$$A_{m-1} - A_m = \frac{1}{m} (A_{m-1} - L_{(n-m+1)}) \geq 0, \quad (\text{F.59})$$

即尾部内部的离散程度除以尾部条数。

证明. 在 theorem F.28 的 (F.53) 中把 m 换成 $m - 1$ 得该恒等式; 非负性来自 $L_{(n-m+1)} \leq A_{m-1}$ 。 ■

注 F.31 (取整是必须的, 但要写清它换来了什么). theorem F.30 说: 系统报出的 CVaR 相对「最窄的整数口径 ($\lfloor m_0 \rfloor$ 条)」一律偏小, 差额恰好是尾部内部离散度除以尾部件数。这不是「更保守」或「更激进」的问题, 而是水平被悄悄挪到了 $\tilde{\alpha}$: $n = 120$ 、 $\alpha = 0.95$ 时, 读者以为在看「95% 分位以上那一段的平均」, 实际看的是 94.17% 水平上的尾部均值。多取一条换来的是估计更稳 (尾部多一个样本), 代价是数值略小。这是一处必须写明、而不是必须修掉的口径——尾部条数只能是整数, 而系统的选择是自治的。

F.6.2 120 天窗口意味着什么: 把条数算出来

系统的组合日收益率窗口写死在 `internal/cio/cio.go` 的 `computePositionReturns` 里: `const historyDays = 120`。把 $n = 120$ 代进 theorem F.29:

表 F.1 名义置信水平、实际尾部条数与等价水平 ($n = 120$)

名义水平 α	名义条数 $(1 - \alpha)n$	代码 $k = \lfloor \cdot \rfloor$	尾部条数 $m = k + 1$	等价水平 $\tilde{\alpha} = 1 - m/n$
0.95	6.0	6	7	0.941667
0.99	1.2	1	2	0.983333

读法: 系统报出的 CVaR95 是最近 120 个交易日里「最惨 7 天」的平均; VaR99 是「第二惨的那一天」; CVaR99 是「最惨 2 天」的平均。

数据来源: 本书按 `internal/risk/engine.go` 的 `CalculateVaR/CalculateCVaR` 与 `internal/intelligence/risk_engine.go` 的 `computeHistoricalVaR/computeHistoricalCVaR` 逐字复算 eq. (F.55); 窗口 $n = 120$ 取自 `internal/cio/cio.go` 的 `historyDays`。核对时间 2026-09-24。

两个数字值得盯一眼。**95% 的尾部只有 7 条**: 一整个季度的行情里, 系统拿 7 天来代表「极端情况」。**99% 的尾部只有 2 条**: VaR99 实际上就是「第二惨的那一天」这一个观测——它的数值完全由单

日决定，换个窗口就可能变样。这不是实现的错，是样本量的物理限制：想在 99% 水平上有 $m_0 \geq 20$ 条尾部，需要 $n \geq 2000$ 个交易日，约八年。

F.6.3 一处必须点名的口径缺陷：序列被倒着喂进去了

下面这件事是本书在核对时发现的，有代码链和真实数据两条证据，如实记下。

代码链。`internal/data/duckdb.go` 的 `GetRecentPrices` 查询写的是 `ORDER BY date DESC LIMIT ?`，返回的切片是**新到旧**。`internal/cio/cio.go` 的 `computePositionReturns` 拿到这个切片后**没有重排**，直接按 $i = 1, 2, \dots$ 算

$$\hat{r}_i = \frac{c_i - c_{i-1}}{c_{i-1}}, \quad (\text{F.60})$$

其中 c_{i-1} 比 c_i 更新。于是

$$\hat{r}_i = -\frac{r_i}{1 + r_i}, \quad r_i = \frac{c_{i-1} - c_i}{c_i} \text{ 为真实日收益.} \quad (\text{F.61})$$

符号被翻了过来（幅度还多一层 $1/(1 + r_i)$ 的二阶畸变）。

这个判断有一个旁证：同一个返回值的另一个消费方——`internal/portfolio/dataprep.go` 的 `PrepareData`——在拿到价格后**显式重排**：

```
1 if prices, err := dm.GetRecentPrices(ctx, market, ids[i], days); err == nil && len(prices)
   >= 2 {
2     sort.Slice(prices, func(x, y int) bool { return prices[x].TradeDate.Before(prices[y].
       TradeDate) })
3     // ... 之后才做日期对齐与收益率计算
4 }
```

代码 F.1 另一个消费方对 `GetRecentPrices` 返回值的处理（`internal/portfolio/dataprep.go`）

也就是说，这个函数的事实契约是「调用方必须自己按日期升序重排」。而 `computePositionReturns` 漏了这一步。

真实数据证据。用沪深 300（sh000300）最近 121 根日线（2026-04-01 至 2026-09-23）、按系统同一公式复算，看被当成「尾部」的到底是什么：

表 F.2 被倒序喂入后，「尾部」取到的是涨幅而非跌幅——沪深 300， $n = 120$

对象	日收益	备注
正确时序第 7 大跌幅	-0.021441	本应成为 VaR95 的量 系统实际取到的那一条
正确时序第 7 大涨幅	+0.018705	
系统 $k = 6$ 位置的排序值	-0.018362	= $-0.018705/(1 + 0.018705)$ ，即上一条经 (F.61) 变换 = 取绝对值后的第 7 大涨幅
系统报出的 VaR95	0.018362	

差额 $0.021441 - 0.018362 = 0.003079$ ，即 VaR95 被低估 14.4%。规律：当上涨日的极端涨幅整体小于下跌日的极端跌幅时，系统的 VaR/CVaR 一律偏小——而 A 股长久期上这恰恰是常态。

数据来源：复算数据：本书按 `internal/data/duckdb.go` 的同一 SQL（`ORDER BY date DESC LIMIT 121`）取数、按 `internal/cio/cio.go` 的同一公式算收益率、按 `internal/risk/engine.go` 的同一算法取尾部，逐字复现。数据直接取自 `data/stock.duckdb`，未做任何模拟或平滑。核对时间 2026-09-24。

影响范围（要说准，不夸大）。这一处顺序只影响 `internal/cio/cio.go` 的三条消费路径：`requestRiskReview`（风险报告）、第 3724 行附近的组合指标装配、以及第 5857 行附近的面板指标（`volatility`、`sharpe_ratio`、`max_drawdown`）。回撤熔断不受影响——`drawdownStatus` 用的是组合快照总资产与 `MaxHistoricalAssets()`，不经过收益率序列；组合中台的 `ComputePortfolioRisk` 走的

表 F.3 哪些对外指标被这一处顺序影响（沪深 300 与上证指数， $n = 120$ ，2026-04-02 至 2026-09-23）

指标	正确时序	系统时序	是否被影响
VaR95	0.021441	0.018362	被影响（低 14.4%）
CVaR95	0.028902	0.024911	被影响（低 13.8%）
VaR99	0.030254	0.029733	被影响（低 1.7%）
CVaR99	0.033147	0.031724	被影响（低 4.3%）
日波动率	0.012724	0.012765	不受影响（对符号不变，仅受 $1/(1+r)$ 二阶项扰动）
年化 Sharpe	0.079	0.118	被影响（均值项符号翻转，但被二阶项抵消了一部分）
最大回撤	0.120486	0.122348	被影响（路径型指标，与顺序有关）
与上证指数相关系数	0.948144	0.948790	不受影响（所有资产被同一个排列重排，配对关系不变）

反向验证：哪一类指标不受影响，正好反证了机制。相关系数与 Beta 只依赖「同一时点上资产之间的配对」，全体资产被同一变换作用时配对不变，所以不受影响；波动率只依赖幅度不依赖符号，所以不受影响；而任何用到「哪些观测构成尾部」或「路径顺序」的指标都会被影响。

数据来源：复算数据：同上；相关系数取 *sh000300* 与 *sz399001* 的 120 个日收益；年化按 $\sqrt{244}$ 折算。核对时间 2026-09-24。

是另一条取数路径（`internal/brain/agents/workflow.go` 与 `internal/riskcenter/riskcenter.go`），同样不经过这里。

常见误解·「算出来的 VaR 是个合理的数」不等于「算对了」 这是本附录最该记住的一条。倒序喂入之后，系统报出的 $\text{VaR}_{95} = 1.84\%$ 依然是一个「看起来很正常」的数字——它甚至还是同一量级。为什么？因为涨幅和跌幅的极端值本来就接近，取绝对值以后两者长得像。

一个算错的量如果量级合理，它比一个明显离谱的量危险得多。这条经验对全部风险指标都成立：波动率对符号不变，所以看不出问题；相关系数对所有资产做同一变换不变，所以看不出问题；只有把「尾部取的是哪几条」逐条打出来，才能看见。

顺带记一条同类的：`internal/risk/engine.go` 的 `CalculateVaR` 在样本为空或频率参数非法时，直接 `return 0`。于是「数据没拿到」和「风险为零」在输出上长得一模一样。第 chapter 二十五 章讲的「静默降级」在这里有一个具体落点：调用方必须自己检查样本条数，不能把 0 当成「没有风险」。

常见误解·「分布连续」这个假设是可以随手丢掉的，只要换个证法 正文为了能走「换元 $u = F_L(x)$ 」，加了 H2（分布连续）。很多人读到这里会把 H2 记成「CVaR 理论的前提」——但系统的数据是经验分布，处处有原子，H2 根本不成立。

真实情况是：CVaR 的全部关键结论都不需要 H2。本附录的 theorem F.12（尾部积分的精确表达）、theorem F.18（最优解集）、theorem F.20（共轭表示，含原子情形的 Z^* 构造）都在只有 H1 的前提下证完了。H2 唯一的用途是让两种定义形式相等（theorem F.14）。

教训：一个假设如果只在某一条证明路径里需要，就不要把它写成「理论的前提」。写作者省了一页篇幅，读者却可能以为自己不能对自己的数据用这套工具。

F.7 椭圆分布下的闭式解

前面都在讲一般分布。当分布被指定为椭圆族时，VaR 与 CVaR 都有闭式解——这就是为什么「用一个波动率反推风险」在实务里流行。

假设 F.32 (H3: 二阶矩与正态基准). 本节额外设 $L \sim N(\mu, \sigma^2)$ 、 $\sigma > 0$ 。记 Φ 、 ϕ 为标准正态的分布函数与密度， $z_\alpha = \Phi^{-1}(\alpha)$ 。

定理 F.33 (正态风险指标的闭式解). 在假设 F.32 下, 对一切 $\alpha \in (0, 1)$,

$$\text{VaR}_\alpha(L) = \mu + \sigma z_\alpha, \quad \text{CVaR}_\alpha(L) = \mu + \sigma \frac{\phi(z_\alpha)}{1 - \alpha}. \quad (\text{F.62})$$

等价地,

$$\text{CVaR}_\alpha(L) - \text{VaR}_\alpha(L) = \sigma(\lambda_\alpha \phi(z_\alpha) - z_\alpha) > 0. \quad (\text{F.63})$$

证明. VaR. 由 $F_L(\mu + \sigma z) = \Phi(z)$, 取 $z = z_\alpha$ 得 $F_L(\mu + \sigma z_\alpha) = \alpha$; 再由 F_L 连续严格递增, $\mu + \sigma z_\alpha$ 就是唯一的 α -分位点。

CVaR. 用 theorem F.12 的恒等式 (F.16), $q = \mu + \sigma z_\alpha$, 则 $L - q = \sigma(Z - z_\alpha)$, $Z \sim N(0, 1)$ 。先算一条标准正态的积分:

$$\mathbb{E}[(Z - z)^+] = \int_z^\infty (t - z)\phi(t) dt = \int_z^\infty t\phi(t) dt - z(1 - \Phi(z)) = \phi(z) - z(1 - \Phi(z)), \quad (\text{F.64})$$

末步用了 $\int_z^\infty t\phi(t)dt = \phi(z)$ (因为 $\phi'(t) = -t\phi(t)$)。代入:

$$\text{CVaR}_\alpha(L) = \mu + \sigma z_\alpha + \lambda_\alpha \sigma (\phi(z_\alpha) - z_\alpha(1 - \alpha)) = \mu + \sigma z_\alpha + \sigma \left(\frac{\phi(z_\alpha)}{1 - \alpha} - z_\alpha \right) = \mu + \sigma \frac{\phi(z_\alpha)}{1 - \alpha}. \quad (\text{F.65})$$

正性: $\lambda_\alpha \phi(z_\alpha) > z_\alpha$ 对一切 $\alpha \in (0, 1)$ 成立 (两者在 $\alpha \rightarrow 0$ 时都趋于 0 但前者以 $\phi(z)/\alpha$ 的量级增长更快, 在 $\alpha \rightarrow 1$ 时 $\phi \rightarrow 0$ 而 $z_\alpha \rightarrow \infty$, 逐点数值检验见下表)。■

表 F.4 正态假设下的风险倍数 (以标准差 σ 为单位)

α	z_α	$\phi(z_\alpha)$	CVaR-VaR (除以 σ)	CVaR (除以 σ)
0.900	1.281552	0.175498	1.754983	3.036535
0.950	1.644854	0.103136	2.062713	3.707566
0.975	1.959964	0.058445	2.337803	4.297767
0.990	2.326348	0.026652	2.665214	4.991562
0.999	3.090232	0.003367	3.367090	6.457322

读法: **95% 的 CVaR 约等于 3.71σ** , 比 95% 的 VaR (1.64σ) 高出 2.06σ 。这一列数字解释了为什么许多「按 3σ 设止损」的做法看起来还行——它其实是在用 0.95 的 CVaR 当目标, 而不是 0.95 的 VaR。

数据来源: **复算数据**: 本书用 eq. (F.62) 在标准正态上高精度数值求解并复算, 非系统运行数值。核对时间 2026-09-24。

注 F.34 (为什么系统不用闭式解). 两处硬理由。第一, (F.62) 要求你先相信正态, 而 A 股日收益的峰度远高于 3——尾部被正态严重低估。第二, 系统的输出契约里 $\text{VaR}_{95}/\text{CVaR}_{95}$ 是要和真实持仓对账的, 用历史法至少「可复现」: 给定同一段行情, 任何人都能算出同一个数。闭式解的价值不在替代历史法, 而在提供一个尺子: 把历史法算出的 3.71σ 与正态的 3.71σ 放在一起, 尾部到底厚不厚, 一眼就能看出来。

F.8 Kelly 缩放系数的数学

第 chapter 七 章已经证过 κ 的有界性与单调性。这里补上三条正文没有的: 等号取到的条件、Lip-schitz 常数, 以及代码守卫在数学上意味着什么。

回顾定义 7.1: c, a 是归一化后的置信度与共识度, 二者都落在 $[0, 1]$,

$$\kappa(c, a) = \min(1, 0.4 + 0.6ca). \quad (\text{F.66})$$

定理 F.35 (等号取到的条件). 对 $c, a \in [0, 1]$:

$$\kappa(c, a) = 0.4 \iff ca = 0 \iff (c = 0) \text{ 或 } (a = 0), \quad (\text{F.67})$$

$$\kappa(c, a) = 1 \iff ca = 1 \iff c = a = 1. \quad (\text{F.68})$$

证明. 记 $g(c, a) = 0.4 + 0.6ca$ 。由 $c, a \in [0, 1]$ 得 $ca \in [0, 1]$ ，故 $g \in [0.4, 1]$ 。下界： $\min(1, g) = 0.4$ 等价于 $g = 0.4$ (因 $g \geq 0.4$ 且 $\min(1, g) \geq g$ 在 $g \leq 1$ 时取等)，等价于 $ca = 0$ 。在 $[0, 1]$ 上积为零当且仅当至少一个因子为零。上界： $\min(1, g) = 1$ 等价于 $g \geq 1$ ，等价于 $ca \geq 1$ ；由 $ca \leq 1$ 知这等价于 $ca = 1$ ，在 $[0, 1]$ 上当且仅当 $c = a = 1$ 。 ■

定理 F.36 (Lipschitz 常数与最优性). 对任意 $c_1, c_2, a_1, a_2 \in [0, 1]$,

$$|\kappa(c_1, a_1) - \kappa(c_2, a_2)| \leq 0.6 \max(|a_1| |c_1 - c_2|, |c_1| |a_1 - a_2|) \leq 0.6(|c_1 - c_2| + |a_1 - a_2|). \quad (\text{F.69})$$

特别地，对每个固定坐标， κ 是 0.6-Lipschitz 的，且常数 0.6 不能再小。

证明. 弱化第一步： $u \mapsto \min(1, u)$ 是 1-Lipschitz 的。若 $u, v \leq 1$ ， $|\min(1, u) - \min(1, v)| = |u - v|$ ；若 $u \leq 1 \leq v$ ， $|\min(1, u) - \min(1, v)| = |u - 1| \leq |u - v|$ ； $u, v \geq 1$ 时为 0。

第二步：由链式法则（先在第二个自变量上 Lipschitz）与 g 的双线性性，

$$|\kappa(c_1, a) - \kappa(c_2, a)| \leq |g(c_1, a) - g(c_2, a)| = 0.6 a |c_1 - c_2| \leq 0.6 |c_1 - c_2|, \quad (\text{F.70})$$

$$|\kappa(c, a_1) - \kappa(c, a_2)| \leq |g(c, a_1) - g(c, a_2)| = 0.6 c |a_1 - a_2| \leq 0.6 |a_1 - a_2|. \quad (\text{F.71})$$

一般情形的三角不等式由两次单变量估计相加得到。

最优性：取 $a = 1, c_1 = 0, c_2 = 1$ 。由 theorem F.35, $\kappa(0, 1) = 0.4, \kappa(1, 1) = 1$ ，差为 $0.6 = 0.6 |c_1 - c_2|$ ，等号被达到，故 0.6 是最小的可行常数。 ■

定理 F.37 (只降不升，以及下界). 设 $A = \text{ruleRate} \geq 0, p, o \in \mathbb{R}$ 为两位 AI 的建议仓位，

$$\text{Effective} = \min(A, \text{clamp}(p, 0.2, 1.0), \text{clamp}(o, 0.2, 1.0), \kappa). \quad (\text{F.72})$$

则

$$\min(A, 0.2) \leq \text{Effective} \leq A, \quad (\text{F.73})$$

且 Effective 关于 A 是 1-Lipschitz 的。

证明. 上界： $\min$ 是最小元素，不超过集合中任何一个成员，取成员 A 即得。

下界：由 theorem 7.1 中 $\text{clamp}(\cdot, 0.2, 1.0)$ 的定义， $\text{clamp}(p, 0.2, 1.0) \geq 0.2, \text{clamp}(o, 0.2, 1.0) \geq 0.2$ ；由 theorem F.35, $\kappa \geq 0.4 \geq 0.2$ 。于是除掉 A 以外的三项都不小于 0.2，故 $\min(A, \text{其余三项}) \geq \min(A, 0.2)$ 。

Lipschitz: $|\min(A, B, C, D) - \min(A', B, C, D)| \leq |A - A'|$ (对固定的另外三项， $\min$ 是 1-Lipschitz 的)。 ■

注 F.38 (这就是「AI 只踩刹车」的完整数学表述). theorem F.37 的两句话合起来说：生效仓位被夹在 $[\min(A, 0.2), A]$ 这一段里，上端由规则层单独决定、AI 无权触碰（它的三项最多是 1），下端由 clamp 的地板 0.2 与 κ 的底座 0.4 兜住。AI 的全部作用是在这段区间里向下选择。系统事实清单第 7 节把这条写成 $\text{Effective} = \min(\text{ruleRate}, \text{clamp}(\cdot, 0.2, 1.0), \kappa)$ ，这里给出了它的完整证明。

F.8.1 代码守卫在数学上意味着什么

第 chapter 七 章的注 7.4 提过一处「公式与实现的差异」：当输入的置信度或共识度小于等于零时，`kellyPositionScale` 直接返回 1.0。下面把它写准。

定义 F.39 (带守卫的缩放系数)。

$$\tilde{\kappa}(c, a) = \begin{cases} 1, & c \leq 0 \text{ 或 } a \leq 0, \\ \kappa(c, a), & c > 0 \text{ 且 } a > 0. \end{cases} \quad (\text{F.74})$$

定理 F.40 (守卫在 0 处制造一个跳跃). 对每个 $a \in (0, 1]$,

$$\lim_{c \downarrow 0} \tilde{\kappa}(c, a) = 0.4 \neq 1 = \tilde{\kappa}(0, a), \quad \text{跳跃幅度} = 0.6. \quad (\text{F.75})$$

$\tilde{\kappa}$ 在开区域 $(0, 1]^2$ 上单调不减 (在每个坐标方向), 但在整个 $[0, 1]^2$ 上不单调: 例如 $\tilde{\kappa}(0, 1) = 1 > 0.7 = \tilde{\kappa}(0.5, 1)$, 而 $0 < 0.5$ 。

证明. 极限: $c \downarrow 0$ 时 $0.4 + 0.6ca \rightarrow 0.4 < 1$, 故 $\kappa(c, a) = \min(1, 0.4 + 0.6ca) \rightarrow 0.4$ 。而 $\tilde{\kappa}(0, a) = 1$ 。差为 0.6。

开区域上的单调性由 theorem F.36 的证明中同一结论 (g 的偏导 $0.6a, 0.6c \geq 0$, 复合单调的 $\min(1, \cdot)$) 给出。

不单调: $\tilde{\kappa}(0, 1) = 1$ (守卫分支), $\tilde{\kappa}(0.5, 1) = \kappa(0.5, 1) = \min(1, 0.7) = 0.7$ 。自变量增大而函数值下降。 ■

注 F.41 (语义与连续性是两回事). theorem F.40 说的不连续性不是 bug, 是故意的语义切换: 在系统里 0 这个值出现的场合是「这个数没拿到」(模型没给出、解析失败、字段缺失), 不是「信心真的很低」。对一个缺失值动刀, 等于让一次失联决定仓位, 所以实现选择「不动」。

要看清这一点, 把它和 theorem F.37 合起来读: 守卫把 κ 抬到 1, 看起来像「放大仓位」, 但 Effective 的上界 A 没有变, `clamp` 的两项也没变。所以守卫的后果是「AI 的缩放整条退出, 决策回落到规则层与另两位官员」, 而不是「加倍下注」。

接这套代码时要记住的分工: (F.66) 描述「数据都在时的保守度曲线」, theorem F.39 描述「数据不在时干脆不做判断」。两件事写在一个函数里, 读的时候必须拆开——否则会得到一个「AI 一失联、仓位自动腰斩」的系统, 那正好是「AI 单点故障不得阻断交易」的反面。

F.9 理论口径与系统实现的对照表

把前面几节的结论收成一张表, 供读者按表核对。

F.10 真实出处

本附录的真实出处

- VaR 与 $CVaR$ 的定义、假设 $H1/H2/H3$ 、三条性质、次可加性反例、两种定义形式: 正文第 chapter 二十二 章第 2 节; 本附录补齐一般失效判据 (theorem F.8)、精确恒等式 (theorem F.12)、一致性条件 (theorem F.14)
- 历史法实现: `internal/risk/engine.go` 的 `CalculateVaR` (行 169 起)、`CalculateCVaR` (行 193 起)、

表 F.5 风险指标：理论对象、系统实现、以及两者的差异

理论对象	系统实现	差异与性质
$\text{VaR}_\alpha = q_\alpha^-$	<code>CalculateVaR</code> : 升序排序后取 <code>sorted[idx]</code> , <code>idx = floor((1-alpha)*n)</code> , 再取绝对值	取的是上分位点 q_α^+ (theorem F.29), 水平 $\tilde{\alpha} = 1 - m/n$; $n = 120$ 、 $\alpha = 0.95$ 时等价水平 0.941667
CVaR_α (积分形式)	<code>CalculateCVaR</code> : <code>sorted[:idx+1]</code> 的绝对值均值	精确等于经验分布在 $\tilde{\alpha}$ 处的 CVaR ; 与名义 α 的偏差由 theorem F.30 界定
CVaR_α (条件期望形式)	同一函数	在经验分布上, 条件期望形式只在水平 $\tilde{\alpha}$ 处与积分形式相等 (eq. (F.57)); 一般情形两者不等 (theorem F.14)
$\inf_t [t + \lambda_\alpha \mathbb{E}[(L - t)^+]]$	不使用 (无优化)	两者在样本上同值: <code>sorted[:idx+1]</code> 的均值就是经验 RU 最优值, 最优解集为 $[q_\alpha^-, q_\alpha^+]$ (theorem F.18)
$\mathbb{E}[L \mid L \geq \text{VaR}_\alpha]$	代码注释写作「ES」	是 CVaR 的下界, 不是 CVaR (theorem F.14); H2 下才相等
一致风险度量四条公理	组合层硬限制用 CVaR (8%), 同时报 <code>VaR95</code>	CVaR 四条全满足 (theorem F.16); VaR 缺次可加 (theorem F.8)
$\kappa(c, a) = \min(1, 0.4 + 0.6ca)$	<code>kellyPositionScale</code>	公式与实现差一个「0 即缺失」的守卫分支, 该分支在 $c = 0$ 处制造 0.6 的跳跃 (theorem F.40)
$\text{Effective} \leq \text{ruleRate}$	<code>Effective = min(ruleRate, clamp(p), clamp(o), kelly)</code>	成立, 且更紧: $\text{Effective} \geq \min(\text{ruleRate}, 0.2)$ (theorem F.37)

数据来源: 本书对 `internal/risk/engine.go`、`internal/intelligence/risk_engine.go`、`internal/cio/cio.go`、`internal/data/duckdb.go`、`internal/cio/proposal.go` 逐行核对。核对时间 2026-09-24。

validateReturnFrequency (行 19 起)、*CalculateSharpeRatio* (行 253 起)、*CalculateMaxDrawdown* (行 295 起, 以权益曲线为输入)

- 计算中台实现: *internal/intelligence/risk_engine.go* 的 *computeHistoricalVaR* (行 188 起)、*computeHistoricalCVaR* (行 203 起)、*computeMaxDrawdown* (行 241 起)、*ComputePortfolioRisk* (行 64 起)
- 组合日收益率序列: *internal/cio/cio.go* 的 *computePositionReturns* (行 5223 起, *const historyDays = 120* 在行 5233); 消费方 *requestRiskReview* (行 809 起)、行 3718 附近、行 5857 附近
- *GetRecentPrices* 的返回顺序: *internal/data/duckdb.go* 行 232 起, SQL 为 *ORDER BY date DESC LIMIT ?* (行 250-251); 对照消费方 *internal/portfolio/dataprep.go* 行 47 起的 *sort.Slice* 升序重排
- 回撤熔断不经过收益率序列: *internal/cio/cio.go* 的 *drawdownStatus* (行 126 起), 使用 *portfolio.GetSnapshot()* 与 *MaxHistoricalAssets()*
- 压力测试六情景与损失封顶: *internal/intelligence/risk_engine.go* 的 *generateStressScenarios*; 冲击依次为 -15%, -5%, -8%, -10%, -12%, -7%
- *Kelly* 缩放公式与守卫: *internal/cio/proposal.go* 的 *kellyPositionScale* (置信度或共识度 ≤ 0 时返回 1.0); 正文第 chapter 七 章的注 7.4
- 硬限制与生效仓位: *internal/policy/engine.go* 的 *HardLimits* (*MaxSinglePosition=0.30*、*MaxDrawdown=0.10*、*CVaR* 限制 8%)、系统事实清单第 7 节与第 11 节
- 真实数据复算: *data/stock.duckdb* 的 *stock.ohlcv*, 标的 *sh000300*、*sh000001*、*sz399001*, 各取 *ORDER BY date DESC LIMIT 121* (2026-04-01 至 2026-09-23), 得 120 个日收益
- 数据缺失与留白: 本附录不引用任何系统未落库的运行数值; 第 table F.4 表的数字为标准正态的数学常数、第 theorem F.15 节的算例为手算构造, 均已标明非系统运行数据

动手试一试

1. 把尾部条数算出来。取你手里的日收益样本数 n , 按 eq. (F.55) 算 $k = \lfloor (1-\alpha)n \rfloor$ 与 $m = k+1$, 再看 $\tilde{\alpha} = 1 - m/n$ 。 $n = 250$ 、 $\alpha = 0.95$ 时 m 是几? $\tilde{\alpha}$ 与 0.95 差多少? 这决定了你报出的 CVaR 到底是哪个水平上的尾部均值。
2. 复现失效区间。按 theorem F.8 的 (F.9), 对 $\alpha = 0.95$ 与 $\alpha = 0.99$ 分别算出使 VaR 失效的违约概率区间 $(1 - \sqrt{\alpha}, 1 - \alpha]$ 。再把 $p = 0.04$ 代进去, 验证 $\alpha = 0.95$ 失效而 $\alpha = 0.99$ 不失效。 $\alpha = 0.99$ 时失效区间宽度是多少?
3. 用真实数据验一遍顺序问题。对任一标的, 用 *internal/data/duckdb.go* 的同一 SQL 取 121 根日线, 分别按「返回顺序」与「按日期升序」算 120 个日收益, 各跑一次 theorem F.29 的两个估计量。你会看到: VaR95 与 CVaR95 都变了, 而波动率几乎不变。再把两个序列各排一次序, 看第 7 小的那个值分别对应哪个方向的极端日。
4. 核对守卫的跳跃。写一个函数实现 theorem F.39, 在 $a = 1$ 下让 c 从 0 走到 0.1 (含 $c = 0$ 这一点), 把 c 与返回值画成折线。跳跃落在 $c = 0$ 上, 幅度 0.6——按 theorem F.40 核对。
5. 拿正态当尺子量尾部。算出你组合 95% 的历史 CVaR, 再算组合日波动率, 把两者之比与 table F.4 的 3.707566 比一下。比值明显大于 3.71 说明尾部比正态厚——这正是系统不用闭式解的原因。

附录 G

术语表（中英对照）

这本书里的名词，一半是金融的，一半是软件的，还有一部分是这两者撞在一起之后新造出来的——「因子健康度」「权限链」「生效仓位」都属于后一类。初级读者最容易被卡住的地方，其实不是概念本身难，而是同一个东西在三个地方有三个名字：书里叫一个，源码里叫一个，数据库里叫一个。你按书里的叫法去代码里搜，搜不到；按代码里的叫法回书里找，也找不到。

这张表的作用，就是把三个名字并排放在一起。

G.1 怎么用这张表

三条使用约定：

- 1. 表里凡是用等宽字体写出来的，都是系统里的**真实标识符**，未作翻译、未作改写。把它们复制出来，可以直接在源码目录或数据库里检索到。
- 2. 本表只收「概念名」。数学符号（ $r_{i,t}$ 、 IC_k 、 $CVaR_\alpha$ 、 r_{pos} ）不在本表，见书前的记号表。
- 3. 最后一列的括号里是**该术语第一次被讲清楚的章号**。想追根究底，去那一章。

还有一句话要说在前面：**表里没有的术语，正文里也没有**。本书不发明读者在系统里查不到的概念——如果你在正文里读到一个名词，却在这张表里找不到它的代码落点，那它要么是通用常识（比如「收益率」），要么就是本书写漏了，应当以源码为准。

G.2 一、组织与治理

这一组词回答的是「谁在干活、谁说了算」。

表 G.1 组织与治理术语

中文	English	系统里的落点	章
投资规划师	Investment Planner	角色常量 AGT-002 (<code>models.go</code>)	2
量化分析师	Quant Analyst	角色常量 AGT-003	2
风控师	Risk Manager	角色常量 AGT-004	2
首席投资官	Chief Investment Officer (CIO)	角色常量 AGT-001	2
操盘手	Trader	角色常量 AGT-005	2
权限矩阵	permission matrix	<code>permissions.go</code> 的默认权限表	2
权限链	permission chain	<code>PROPOSE</code> → <code>EVIDENCE</code> → <code>VETO</code> → <code>DECIDE</code> → <code>EXECUTE</code>	2
否决权	veto	权限常量 <code>VETO</code> ，仅风控师持有	2
任务契约	task contract	结构体 <code>TaskContract</code> ，12 个约束字段	3
输出守卫	output guard	<code>OutputGuard.ValidateOutput</code>	3
无动作码	no-action code	契约字段 <code>NOActionCode</code> ； <code>NO_ACTION</code>	3
拒答	refusal	异常策略返回码 <code>DATA_UNAVAILABLE</code>	3
多空辩论	bull-bear debate	任务 <code>DAILY_REVIEW</code> 的辩论区块	6

中文	English	系统里的落点	章
多提案集成	proposal ensemble	字段 <code>net_score</code> ，把四路来源压成一个数	8
降级容错	graceful degradation	LLM 不可用时的规则兜底路径	9
决策留痕	audit trail	表 <code>agent_decisions</code> 、 <code>decision_traces</code>	10
黑匣子	black box（指可复 盘记录）	表 <code>policy_logs</code> 、 <code>audit_logs</code>	10

「黑匣子」在本书里是褒义：指决策过程被完整记录下来、事后可逐字回放。它与「不可解释的模型」不是一回事——前者是工程要求，后者是技术局限。

G.3 二、因子、回测与统计

这一组词回答的是「它凭什么说一只股票不错」。

表 G.2 因子、回测与统计术语

中文	English	系统里的落点	章
因子	factor	表 <code>factor_qualities</code> ； <code>factor_name</code> 为键	11
因子体检	factor health check	任务 <code>FACTOR_HEALTH</code> ；工具 <code>factor_review</code>	14
信息系数	information coefficient (IC)	记号 IC_k ；因子与下期收益的 Pearson 相关	14
秩信息系数	rank information coefficient	记号 $RankIC_k$ ；Spearman 秩相关	14
覆盖率	coverage	因子体检四栏之一（有效样本占比）	14
前瞻准确率	forward accuracy	因子体检四栏之一（方向命中的比例）	14
衰减	decay	因子体检四栏之一（后期 IC 的衰减速度）	14
分组收益差	quantile spread	按因子分组后首尾两组的收益之差	14
分半稳定性	split-half stability	字段 <code>stability_std</code> （前后两半的离散度）	14
Pearson 相关	Pearson correlation	线性相关；对极端值敏感	14
Spearman 秩相关	Spearman rank correlation	先取秩再算相关；抗极端值	14
前视偏差	look-ahead bias	零前视校验：当日只用上一交易日数据	12
幸存者偏差	survivorship bias	样本池必须含已退市标的	17
过拟合	overfitting	任务 <code>FACTOR_BREAK_ANALYSIS</code> （因子失效分析）	15
回测	backtest	工具 <code>run_backtest</code> ；表 <code>backtest_results</code>	16
样本内	in-sample	字段 <code>BestIsSharpe</code> （选参所用区间）	17
样本外	out-of-sample	字段 <code>OosSharpe</code> （选参未见区间）	17
网格搜索	grid search	表 <code>backtest_configs</code> ；参数扫描	17
换手率	turnover	工具 <code>get_turnover</code> ；决定成本的量	17
可交易池	tradeable universe	表 <code>tradeable_stocks</code> ；剔除停牌与 ST	17

「信息系数」这个词最容易被误读成「信息量」。它不是信息论里的熵，就是因子值与下一期收益之间的相关系数，取值于 $[-1, 1]$ 。名字里的「信息」是可用性意义上的，不是香农意义上的。

G.4 三、市场与行情

这一组词回答的是「今天这盘棋，值不值得下注」。

表 G.3 市场与行情术语

中文	English	系统里的落点	章
六维判势	MarketSixDim	sixdim.go; 工具 market_sixdim_detect	19
技术趋势	tech	维度键 tech, 权重 0.15	19
市场广度	breadth	维度键 breadth, 权重 0.20	19
量能流动性	volume	维度键 volume, 权重 0.15	19
资金结构	capital	维度键 capital, 权重 0.15	19
情绪赚钱效应	sentiment	维度键 sentiment, 权重 0.25	19
外部约束	external	维度键 external, 权重 0.10	19
原始总分	raw total score	字段 raw_total_score (加权, 未修正)	19
冲突修正	conflict adjustment	字段 conflict_count; ≥ 2 时打折	19
修正总分	adjusted total score	字段 adjusted_total_score	19
仓位系数	position rate	字段 position_rate $\in [0, 1]$	21
市场标签	market tag	强势/结构性震荡/偏弱/退潮风险	19
盘口	order book	表 order_book_daily; 工具 market_order_book	12
五态分类	order-book state	方法 Classify; 封死阈值 sealThreshold	12
涨停	limit up	工具 get_limitup_board	11
炸板	limit-board failure	曾触涨停后打开; 情绪维度输入	10
龙虎榜	dragon-tiger list	工具 get_lhb (交易所披露的异动席位)	11
赚钱效应	money-making effect	情绪维度输入; 涨跌家数与连板高度	11
板块轮动	sector rotation	工具 get_sector_rotation	11
市场体制	market regime	工具 market_regime_detect	19
指数技术面	index technicals	工具 get_index_technical (均线、乖离)	19

六个维度的权重取自 sixdim.go 的 DimWeights, 六项之和恰为 1.00。它们不是本书编的, 是源码里的常量, 改动权重等于改动全书所有关于仓位系数的算例。

G.5 四、仓位、成本与风险

这一组词回答的是「下多少注、万一错了亏多少」。

表 G.4 仓位、成本与风险术语

中文	English	系统里的落点	章
Kelly 缩放	Kelly scaling	记号 $\kappa \in [0.4, 1]$; 只降不升	7
生效仓位	effective position	$u^* = \min\{\text{各来源上界}\}$	7
只降不升	ratchet-down only	AI 只能在规则仓位上打折, 不能加码	7
仓位系数	position rate	六维判势输出, 信号仓位 = 原仓位 $\times$ position_rate	21

中文	English	系统里的落点	章
越权建仓	unauthorized scaling	判势提示退潮却仅缩小仓位、照常建仓	21
风险价值	Value at Risk (VaR)	CalculateVaR; 记号 VaR_α	22
条件风险价值	Conditional VaR (CVaR)	CalculateCVaR; 等价于期望损失	22
期望损失	expected shortfall	CVaR 的另一个常用名字（附录 F 证明两者恒等）	22
历史法	historical method	computeHistoricalVaR; 按序号定位分位数	22
次可加性	subadditivity	风险度量公理之一；VaR 一般不满足	22
一致性	coherence	单调、平移不变、正齐次、次可加四条公理	22
最大回撤	maximum drawdown	CalculateMaxDrawdown; 记号 MDD	22
回撤熔断	drawdown circuit breaker	drawdownStatus; drawdownWarnPct = 8、drawdownStopPct = 12	22
夏普比率	Sharpe ratio	CalculateSharpeRatio	7
波动率	volatility	记号 σ ；日频收益率标准差	7
压力测试	stress test	任务 STRESS_TEST; 六情景冲击写死在源码	22
流动性硬下限	liquidity floor	applyLiquidityFloor; 流动性因子分低于 30 剔除	12
交易成本	transaction cost	佣金万三、最低 5 元	17
印花税	stamp duty	常量 StampDutyRate = 0.0005, 仅卖出	17
滑点	slippage	成交价与预期价的差	17
市场冲击	market impact	下单本身推动价格；近似平方根律	17
整手	round lot	方法 floorLots; 买入量须为 100 股整数倍	23
T+1	T+1 settlement	当日买入当日不可卖；engine.go 算「可卖量」	23
现金整手上限	affordable lots	affordable=int(Cash/price)/100*100	23

VaR 与 CVaR 的完整数学定义、两条公理体系的证明、以及系统历史法实现与理论口径的逐项对照，见附录 F。本表只给名字，不给推导。

G.6 五、软件、数据与智能体

这一组词是给不写代码的读者的：它们解释「这套东西是用什么搭起来的」。

表 G.5 软件、数据与智能体术语

中文	English	系统里的落点	章
智能体	agent	表 agents、team_members	1
大语言模型	large language model (LLM)	表 llm_call_logs 记录每一次调用	4
工具	tool	目录 internal/tools; 共 55 个（附录 C）	4
工具调用	tool calling	模型只选工具与参数，算数交给引擎	4
工作流	workflow	表 workflows、workflow_instances	5

中文	English	系统里的落点	章
调度器	scheduler	包 <code>internal/scheduler</code>	5
日循环	DailyCycle	盘前 / 盘中 / 盘后 / 夜间四段	5
任务状态	task status	SUCCESS/RUNNING/FAILED/PENDING/SKIPPED	5
桌面应用	desktop application	Wails v2 外壳（前后端同进程）	24
单页应用	single-page application (SPA)	<code>ui/src/pages</code> ；路由形如 <code>#/ai-team</code>	27
分析型数据库	analytical database	DuckDB；文件 <code>stock.duckdb</code>	24
嵌入式数据库	embedded database	SQLite；文件 <code>quantpilot.db</code>	24
对象关系映射	object-relational mapping (ORM)	GORM；结构体 $\leftrightarrow$ 表	24
只读连接	read-only connection	本书全部复算脚本均以此连接，不改数据	25
数据陈旧度	staleness	记号 $ e(t) $ ；屏上数字与真相的距离	27
以数据库为准	single source of truth	盈亏以库中记录为准，不以界面缓存为准	29
安全开关	safety switch	模拟 / 实盘 / 暂停建仓 / 紧急停止	31

「桌面应用」与「单页应用」不是同一个层次的概念：前者说这台软件怎么装在你电脑上，后者说它的界面怎么在窗口里切换。本书第 24 章解释前者，第 27 章解释后者。

G.7 六、系统标识符速查

前面五节是「概念 $\rightarrow$ 代码」。这一节反过来：把系统里成组出现的常量原样列出，供读者检索时对照。

G.7.1 六个角色

常量	English	中文
AGT-001	CIO	首席投资官（最终决策权）
AGT-002	Investment Planner	投资规划师（提出方案）
AGT-003	Quant Analyst	量化分析师（提供证据）
AGT-004	Risk Manager	风控师（可否决）
AGT-005	Trader	操盘手（只执行）

G.7.2 五条权限

常量	持有角色	含义
PROPOSE	Planner	提出方案
EVIDENCE	Quant	提供证据
VETO	Risk	可否决
DECIDE	CIO	最终决定
EXECUTE	Trader	只能执行

这五条按上表顺序构成权限链，也叫**五道门**。注意它们的排列：唯一能「决定」的人（CIO）不能「执行」，唯一能「执行」的人（Trader）不能「决定」。这是第 2 章那条设计原则的代码形态。

G.7.3 任务类型

任务类型是调度的最小单位。全书涉及的任务类型按角色分组如下（常量名照抄源码）：

角色	任务类型常量
Planner	UPDATE_PROFILE、CHECK_MANDATE、COMPLIANCE_REPORT
Quant	MARKET_SCAN、FACTOR_HEALTH、STOCK_SCREENING、PORTFOLIO_SIMULATION、ALPHA_DISCOVERY、REGIME_DETECTION、FACTOR_BREAK_ANALYSIS
CIO	MORNING_BRIEF、INVESTMENT_DECISION、PORTFOLIO_REVIEW、EMERGENCY_DECISION、DAILY_REVIEW、T1_PLANNING
Risk	OVERNIGHT_RISK、END_OF_DAY_RISK、TRADE_RISK_CHECK、STRESS_TEST、RISK_ASSESSMENT、RISK_ALERT
Trader	GENERATE_ORDER、EXECUTE_ORDER、MONITOR_ORDER、EXECUTION_REPORT、RECONCILIATION

G.7.4 决策类型与状态

决策类型回答「打算做什么」，决策状态回答「走到哪一步了」。两者是正交的：一个 REDUCE 决策，可以从 DRAFT 一路走到 EXECUTED，也可以停在 REJECTED。

类别	常量
决策类型	INCREASE（加仓）、REDUCE（减仓）、HOLD（持有）、EXIT（清仓）、NO_ACTION（不动作）
决策状态	DRAFT、PENDING_RISK、RISK_REVIEW、RISK_APPROVED、APPROVED、CIO_APPROVED、EXECUTING、EXECUTED、REJECTED、EXPIRED、FAILED

NO_ACTION 值得单独说一句：它是一个合法的、有名字的结论，不是「什么都没发生」。第 3 章讲过，一个系统如果不给「不动作」留一个正式出口，模型就会被迫编一个动作出来。

G.7.5 两个库里的关键表

行情与基本面（DuckDB，文件 stock.duckdb，本书写作时 1,499,738,112 字节）。表名前缀按源码写法保留：

表名	内容
stock.ohlcv	日线行情主表（未复权）
stock.ohlcv_qfq	前复权日线
stock.stock_basic	股票基础信息（代码、名称、上市日）
stock.stock_daily	日频衍生字段（市值、换手等）
stock.trading_calendar	交易日历
stock.adj_factor、stock.adj_event	复权因子与除权除息事件
stock.financial_report	财务报表
order_book_daily	每日盘口摘要
market_sixdim_daily	每日六维判势结果
l1_snapshot	L1 快照

系统状态与交易（SQLite，文件 `quantpilot.db`，共 58 张业务表）。列出与正文关系最密切的若干张：

表名	内容
agents、team_members	智能体与团队成员
agent_decisions、decision_traces	决策记录与推理链路
agent_task_logs、agent_tool_calls	任务日志与工具调用记录
policy_logs、audit_logs	规则命中与审计日志
factor_qualities	因子体检结果（每因子只留最新一次）
strategies、strategy_candidates	策略与候选
tradeable_stocks	可交易池
daily_strategy_plans	每日策略计划
investment_plans、plan_approvals	投资方案与审批
investment_mandates、investor_profiles	投资授权与投资者画像
cio_decision_logs	CIO 决策日志（前端卡片的数据源）
orders、trades	委托与成交
positions、position_snapshots	当前持仓与持仓快照
portfolios、portfolio_daily_stats	组合与每日统计
cash_flows	资金流水
backtest_configs、backtest_results	回测配置与结果
market_risk_reports、risk_limits	风险报告与限额
llm_call_logs	大模型调用留痕

数据来源：角色常量、权限常量、任务类型、决策类型与状态取自系统源码 `internal/agentworkflow/models.go` 与 `internal/orchestrator/orchestrator.go`；六维权重与标签取自 `internal/marketsixdim/sixdim.go`；表名与规模由本书脚本以只读方式对 `stock.duckdb` 与 `quantpilot.db` 实际查询得到（前者 16 张表，后者 59 张，其中 1 张为 `sqlite_sequence`，故业务表 58 张）。核对时间 2026-09-24。

G.8 附：容易混淆的三对词

术语表最后留一处「纠偏角」，把三组读者最常混起来的名词摊开讲。

三对名字，别用混

(一) **IC** 与「信息量」。第 14 章说过，IC 是相关系数，不是熵。一个因子的 IC 是 0.05，不代表它「只提供 5% 的信息」，而是说它的取值与下期收益的线性相关很弱。IC 为负同样也是有效信息，只要稳定。

(二) **VaR** 与「最大亏损」。第 22 章与附录 F 反复强调：**VaR 是分位数**，不是上界。95% 的 VaR 是 2%，意思是「有 5% 的日子会亏得比 2% 更惨」，至于惨到什么程度，VaR 一个字都不说。想知道尾部有多深，要用 CVaR。

(三) **权限与提示词**。第 2 章的核心结论：权限不是写在提示词里的「请你不要乱来」，而是代码里的门。提示词是**建议**，权限是**事实**。把这句话记住，你就理解了本书一半的设计哲学。

真实事故索引

第 9 章讲过一句话：一个系统值不值得信，不看它顺利时跑得多好看，看它出事时怎么收场。这一章的作业，就是把这本书背后的系统真实出过的 29 次问题全部摊开。

它们不是教科书里的典型案例，也不是为了配合某个知识点编出来的寓言。每一条都对应一次真实的修复提交：症状是当时屏幕上看到的现象，根因是事后定位到的代码，修复是实际改了什么。

H.1 怎么用这份索引

三条使用建议：

- 1. 按症状查，不要按编号查。遇到问题时，你手里只有现象（「界面上的数是空的」「重启后又跑了一遍」），没有编号。所以下面按症状类型分成七组，直接跳到最像你那一组去对。
- 2. 重点看「根因」列，而不是「修复」列。修复是这一台机器的答案，根因才是可迁移的知识。同一个根因在你自己的代码里可能表现为完全不同的症状。
- 3. 每一组末尾的教训是本书的判断，事故本身是事实。事实与判断在书里始终分开写——这也是第 20 章那条数据规矩的一种延伸。

表里的编号沿用系统内部的故事编号，与正文中出现的「事故 #7」「事故 #26」等引用一一对应。

H.2 一、进程稳定性：程序自己把自己弄死了

这一组是最难查的：程序没有报错，日志干净，日志里什么都查不到。

表 H.1 进程稳定性事故

#	症状（屏幕上看到的）	根因（事后定位到的）	修复 / 教训
1	程序运行中突然闪退，日志干净，系统事件日志无记录	决策完成后调用 agent.dll 的 Release() 卸载 DLL，而 c-shared 模式的 Go DLL 内部仍有后台 goroutine 在跑；DLL 一卸载，这些协程执行到的就是已被释放的代码空间，触发访问违例 0xC0000005	不再卸载 DLL，让它驻留整个进程生命周期。教训：进程内加载的共享库，只加载、不卸载
2	致命 panic 的堆栈被静默丢弃，只有一句「程序已关闭」	Go 运行时的致命 panic 直接写 os.Stderr，不经过 log 包；而 GUI 程序没有控制台，这份输出无人接收	用 SetCrashOutput 把崩溃输出重定向到 log/crash_dump.txt。教训：崩溃信息必须落到文件，不能落到一个不存在的终端
3	托盘图标右键菜单弹不出来，左键正常	Wails 的 startup 钩子通常跑在非主线程上，不带 Windows 消息泵；托盘窗口收不到 WM_TRAY 消息，自然无法派发	在独立线程里创建托盘窗口，并给它跑一个专属消息泵。教训：GUI 的消息循环必须绑在拥有窗口的那个线程上

H.3 二、数据口径不一致：两个地方对同一个东西叫法不同

这一组事故数量最多，也最典型。它们的共同形状是：每个模块单看都对，拼起来就错。

表 H.2 数据口径不一致事故

#	症状（屏幕上看到的）	根因（事后定位到的）	修复 / 教训
6	配对策略回测 0 成交，所有指标全是 0，但没有任何报错	持仓表里 <code>instrument_id</code> 是纯数字（形如 002437），而行情表 <code>stock.ohlcv</code> 的 <code>symbol</code> 是带市场前缀的小写（形如 <code>sz002437</code> ）。两边对不上，样本配对全部落空，代码又悄悄回退成「指数对指数」	增加 <code>toOhlcvSymbol</code> 归一化函数，转换后再精确匹配。教训：跨表拼接前先统一主键口径，且回退路径必须留痕
13	因子复盘显示 32 只股票、0 条有效数据	可交易池入池时把 <code>StockCode</code> 存成了双重前缀（形如 <code>SHsh603823</code> ）；查询时 <code>key</code> 由市场名与代码拼接而成，拼出来和存进去的不一样	统一 <code>key</code> 口径，入池与查询共用同一个构造函数。教训：凡是「拼出来的主键」，就一定会有一处拼错
14	界面上股票名称一栏显示的是代码本身	<code>tdx_manager.go</code> 里把 <code>Name</code> 字段错误地赋成了 <code>code</code>	优先取快照自带的名称，缺失时才用代码占位。教训：字段赋值错误不会报错，只会安静地显示一个「看起来像答案」的东西
17	复权因子导入主键冲突，导入中断	同一只股票同一天在数据源里可能有多条事件（分红与送转叠加、配股与分红叠加），而目标表以（ <code>symbol</code> ， <code>ex_date</code> ）为唯一键	导入前先按该键聚合去重。教训：源数据的粒度不一定等于目标表的粒度，唯一约束就是这里的验收标准
25	重新训练的模型不被程序使用，模型文件静静躺在磁盘上	训练脚本输出的文件名与 Go 侧加载时拼接的路径不一致	统一命名与路径。教训：训练与加载是两个进程，唯一的契约是文件名，必须有地方把它钉死

H.4 三、样本与窗口：差一根 K 线，结论就反过来

这一组全是「边界」问题。边界错误的特点是不报错、不崩溃，只是悄悄给出一个错的答案。

表 H.3 样本与窗口事故

#	症状（屏幕上看到的）	根因（事后定位到的）	修复 / 教训
4	市场广度指标「总是很淡」，与当天实际盘面感受明显不符	旧版实现按 <code>ORDER BY symbol</code> 做字典序截断，取前 300 只；这个顺序把样本几乎全推到北交所与沪市 600 开头的代码段上	改为分层代表样本，并固定随机种子，保证可复现。教训：「取前 N 个」在数据没有天然顺序时等于引入偏差
5	标注为「20 日新高」，实际只比对了 19 根 K 线	切片写成 <code>bars[len-20:]</code> ，把当前这根 <code>bar</code> 也算进了窗口，于是「20 根」里真正用来比较的只有 19 根	改为 <code>bars[len-21 : len-1]</code> 。教训： <code>off-by-one</code> 是唯一一类字面看起来完全正确的错误
19	六维判势的 20 日新高计数偏小（与 #5 同源）	同一处窗口切片错误被另一处代码复用了	同一处修复。教训：一处 <code>off-by-one</code> 往往会在多处重复出现，修的时候要全局搜索同样的写法

H.5 四、风控与配置：同一个数字被写了三遍

表 H.4 风控与配置一致性事故

#	症状（屏幕上看到的）	根因（事后定位到的）	修复 / 教训
7	风控硬限制形同虚设，某些下单路径不受约束	单票上限与回撤容忍这两个数字，在风控师、策略引擎、风险中心 三处各写了一遍 ，值还不一样（30% / 20% / 其他）。哪条路径生效，取决于它走了哪个模块	统一为「单票 30% + 回撤 10%」，三处对齐。教训： 同一个约束只能有一个真值来源 ，写三遍等于没写
8	切换交易模式后，实盘开关「静默失效」——按了没反应，也不报错	<code>SetTradingMode</code> 重建执行桥之后，没有把新的 broker 引用同步给 CIO 引擎；引擎手里攥着的是旧对象	重建后同步引用。教训：对象重建后， 所有持有旧引用的人都要被通知
9	配置文件并发写入有丢失风险	采用整文件覆盖写：两次写入交错，后写的把先写的整个盖掉	改为原子替换：先写临时文件 <code>.tmp</code> ，再 <code>os.Rename</code> 覆盖。教训：写配置文件是 并发场景 ，不是「偶尔写一次」
22	重启程序后，当天已经做完的每日复盘与方案重建又跑了一遍	幂等标记只存在内存里，进程一重启就没了	把 <code>last_daily_review_date</code> 与 <code>last_plan_refresh_date</code> 持久化进 <code>config.json</code> 。教训： 幂等标记必须持久化 ，否则重启即重复

H.6 五、性能与调度：慢，或者空转

表 H.5 性能与调度事故

#	症状（屏幕上看到的）	根因（事后定位到的）	修复 / 教训
10	一次回测耗时约 2 小时，慢到无法迭代	逐根 K 线都去查一次财务报表，一次回测累计约 420 万次查询，而绝大多数策略根本不需要财务数据	改为条件注入：仅当策略实现了 <code>FinancialAware</code> 且 <code>NeedsFinancialData()</code> 返回 <code>true</code> 时才查。教训：先问「这一次计算真的需要它吗」，再问「怎么算得更快」
11	收盘后程序循环执行、日志反复刷屏	调度器主循环是固定 30 秒的 ticker；复盘阶段（15:15–18:00）里每 30 秒空转一次，把复盘任务反复触发；行情刷新在非交易时段也因缓存过期而反复拉取	按阶段收敛触发条件。教训： 定时器不等于调度器 ——有节拍，还要有「这个时段该不该干活」的判断
12	财务数据只有手动点同步才更新，自动跑永远缺	自动任务清单里压根没有财务同步这一项	新增自动增量同步。教训：漏掉一个定时任务， 不会报错 ，只会让某张表长期停在旧数据上
18	六维判势每次计算延迟数秒，且容易触发数据源限流	实时快照没有 TTL 缓存；拉取函数逐只股票串行请求	加 30–60 秒的包级 TTL 缓存；非交易时段跳过实时覆盖。教训： 同一个数在一分钟内被反复请求 ，是缓存该出场的最明确信号

#	症状	根因	修复 / 教训
23	夜间与盘前的深度复盘总是被中途掐断，报告只写了一半	任务有 180 秒硬超时，时间一到就取消在途的大模型调用	把超时延长到 30 分钟。教训：超时值是按任务类型定的，不能所有任务共用一个

H.7 六、因子与模型：看起来很棒，实盘很糟

表 H.6 因子与模型事故

#	症状（屏幕上看到的）	根因（事后定位到的）	修复 / 教训
15	分半检验把样本量不足的因子误判为「不稳定」，并强制大幅降权	原逻辑里样本数小于 8 就直接判为不稳定	样本不足改为「无法判定」：不参与均值计算，也不激进降权。教训：「不知道」和「不好」是两个不同的状态，不能合并处理
16	因子质量分被少数极端值带偏	用算术均值作为参考基准	改用中位数。教训：当分布的尾部很厚时，均值是最容易被少数样本劫持的统计量
20	因子质量分算出来不含情绪因子	复盘工具没有注入情绪引擎，情绪维度在计算时是缺的	注入共享的情绪引擎实例，并并发预取全池情绪得分。教训：「这一项没算」与「这一项算出来是 0」在结果里长得一样，必须分开
21	因子复盘每天照跑，结果却完全不影响次日的选股	质量分算完就落库了，没有接入权重计算环节	权重按质量分相对中位数做缩放（截断在 0.5–1.5 倍之间）后再归一。教训：一个指标如果不影响任何下游动作，它就不是决策的一部分，只是装饰
24	机器学习训练在 sklearn 1.7.2 上报类型错误	该版本 MLPClassifier 不再支持 class_weight 参数	改用 fit 的样本权重参数。教训：依赖库的小版本升级会改 API，训练脚本必须跟主程序一起做版本锁定
26	模型回测指标虚高，实盘立刻失效	四处泄漏叠加：跨股票的滚动窗口互相污染、无效标签混入、用随机切分划训练测试集、标准化器在全量数据上拟合	按股票分组、按时间切分、标准化器只用训练集拟合。教训：泄漏不是一处，是一个组合；堵住其中一处，其余三处照样把指标抬上去

H.8 七、静默失效：错了，但没人知道

这一组是全书最想强调的一类。它们的共同特征是：系统没有降级、没有报错，只是安静地少做了一件事。

表 H.7 静默失效与告警缺失事故

#	症状（屏幕上看到的）	根因（事后定位到的）	修复 / 教训
27	在数据源页面选中「腾讯财经」，意外影响了六维判势的取数	六维判势的数据源配置与行情源配置耦合在同一个网格里，改动一处会波及另一处	把六维源拆成独立卡片区，配置解耦。 教训： UI 上摆在一起，就会被人以为是同一件事
28	实时快照覆盖股票数为 0，但没有任何提示	key 口径不一致导致匹配全部落空，代码静默返回空结果	增加判断：覆盖数为 0 时主动告警。 教训： 「0 条结果」必须能被区分成「真的没有」和「一个都没匹配上」
29	六维判势样本数不足时照样出分，看不出可信度低	没有低可信度标注	样本数低于 100 时在来源字段里标注低可信度。教训： 一个数字必须带着它的置信度一起出门，否则下游会当它是事实

H.9 结语：29 条事故的共同形状

把 29 条排在一起看，会发现它们并不像「29 个不同的 bug」，而更像**同一类错误的 29 次发作**。归纳起来只有四种形状：

四种形状，以及它们对应的那道门

（一）口径不一致。同一个东西在不同模块有不同名字、不同格式、不同默认值（#6、#7、#13、#14、#17、#25、#28）。对策不是「仔细一点」，而是**让不一致在加载期就报错**：统一构造函数、统一主键、配置项收敛到单一真值来源。

（二）边界与顺序。差一根 K 线、截断顺序不对、超时值拍脑袋（#4、#5、#19、#23）。对策是把每个窗口、每个切片都写成可验证的断言，而不是靠肉眼审阅。

（三）静默失效。不报错、不崩溃，只是少做了、做反了，或者给了一个「看起来像答案」的空值（#2、#8、#12、#20、#21、#27、#28、#29）。这是最危险的一类，因为它**不触发任何告警**。对策是第 20 章那条原则：数据不可用就明确报错，绝不降级。

（四）把「不知道」当成「不好」或「零」。样本不足当成不稳定（#15）、极端值带偏均值（#16）、覆盖率 0 当成没有股票（#28）。对策是给「无法判定」一个**独立且合法的状态**。

如果只允许从这 29 条里带走一句话，那我选这一句：**凡是没有出错的迹象、却又给出确定数值的地方，都值得怀疑一次**。这本书里所有的「留白」「标注」「报错」，追到根上都是为了防这一类问题——宁可系统停下来告诉你「我不知道」，也不要它安静地给你一个错的答案。

附录 I

数据诚信原则与免责声明

这是全书最后一份「讲道理」的附录（它后面只剩一份用来查名字的索引），但如果你是完全不写代码、只想搞明白「这东西靠不靠谱」的读者，我建议你把**它提到最前面读**。原因很简单：一本书里的结论值不值得信，取决于它的数字是从哪来的、以及它在拿不到数字的时候怎么办。这两件事，比任何公式都更能决定这本书的成色。

I.1 一、系统自己的态度：宁可不做，也不假装

这本书写的是一台正在运行的软件（QuantBot v1.6.0）。它在设计上有一条款项，贯穿了从启动到收盘的每一段流程，原文写得很直白：

数据不可用时必须报错，不得降级为默认值，不得使用虚假数据猜测。

这句话在工程上意味着几件具体的事，它们都不是口号，而是可以逐条去代码里核对的：

- **启动时关键组件失败必须弹窗报错。**配置管理器、SQLite、股票时序库、投资组合引擎、大模型服务、智能体执行器——这六个组件里任何一个加载失败，程序会明确告诉你，并让你选择是否「仍然继续」，而不是自己悄悄降级成某种能跑但不完整的形态。
- **所有实时数据源全部失败时，返回空，而不是返回伪造值。**行情源有优先级链（本地通达信、通达信 MCP、腾讯财经等），逐级回退；一路失败到底，结果就是空，而不是「上一次的数字」或「一个看起来合理的估计」。
- **未配置密钥的付费数据源直接拒绝，不静默跳过。**同花顺官方数据服务需要 API Key，没配置就是明确的拒绝，不会伪装成「这个数据源本来就没有数据」。
- **数据没更新到最新交易日，弹窗告警。**每天 16:00 拉取当日日线；如果发现库里最新日期落后于最后一个交易日，程序会提醒你，而不是拿旧数据算出一个新结论。

这四条合起来是一句判断：**这个系统宁可停下来告诉你「我不知道」，也不愿意安静地给你一个错的答案。**本书引用数据时遵守的是同一条规矩。

I.2 二、本书的三类标注

为了让读者随时知道「眼前这个数字是什么来路」，全书用三类醒目标注，出现在脚注里或表格下方：

标注	含义与触发条件
真实数据	该数字直接取自系统运行库，未做任何模拟、平滑或插值。全书出现 94 次。
复算数据	该数字是本书按系统实现中的 同一个公式 、在系统的 同一份真实数据 上重新算出来的。用于那些系统界面没有直接显示、但可以由已有数据推导的指标。全书出现 7 次。
数据缺失	该系统库中此项无有效记录。本书选择 留白 ——表格里写一个「—」，而不是填一个看起来合理的估计值。全书出现 14 次。

此外，正文的每一处论断背后都有一条普通脚注，写清它对应的源码文件、代码行号或数据库表，并在末尾附上核对时间（本书的核对时间为 2026-09-24）。全书这样的脚注共 138 条，另有 83 处表格级来源说明。

一句话概括这套标注的用意：**读者不应该需要相信作者**。任何一个数字，你都应该能顺着标注找到它的出生地。

I.3 三、本书的数据底座（实测）

本书写作时，系统运行库的实际规模如下。这张表本身就是一个「真实数据」的例子——每个数字都由只读查询当场取出，没有一个是估计的。

表	行数	说明
行情与基本面库 stock.duckdb (1,499,738,112 字节)		
stock.ohlcv	10,041,343	日线行情主表
stock.ohlcv_qfq	10,041,343	前复权视图（与主表等行）
stock.stock_daily	10,041,343	日频衍生字段
stock.stock_basic	19,068	股票基础信息
stock.adj_factor	9,997,923	复权因子
stock.adj_event	14,172	除权除息事件
stock.trading_calendar	248	交易日历
stock.financial_report	315,680	财务报表
l1_snapshot	80,612	L1 快照
tdx_financial	297,872	通达信财务
order_book_daily	0	盘口摘要： 空表 ，故第 12 章的盘口因子在真实数据上从未参与过评分
market_sixdim_daily	5	六维判势历史：仅 5 条，是第 19 章那张真实表格的全部来源
CN_market_daily	0	空表
HK_market_daily	0	空表
US_market_daily	0	空表
系统状态与交易库 quantpilot.db (7,217,152 字节，59 张表，其中 58 张为业务表)		

数据来源：实测数据：本表由本书脚本以只读方式连接两个运行库逐表计数得到。核对时间 2026-09-24。

请特别注意表里那三行为 0 的记录，以及只有 5 行的那一行。它们不是排版错误，而是这本书里最重要的一个示范：

留白，也是内容

一张空表意味着一个因子从未被真实数据驱动过。第 12 章讲盘口因子时，本书没有给它编一个「示例得分」，而是把表格里所有属于它的位置都写成了「—」。

一本书如果只展示有数据的部分，读者会误以为「系统的每一块都在工作」。而真实情况是：**有的模块已经接通、有的模块还没接通、有的模块只跑过 5 次**。把这三类情况如实分开写，比让全书看起来饱满要重要得多。

那条只有 5 行的 market_sixdim_daily 尤其值得说：第 19 章那张六维判势的真实数据表，全部内

容就来自这 5 条记录。它样本很小——所以那一章里所有的分析都带着「这只是 5 天」的前提在读。样本小不是问题，假装样本不小才是问题。

I.4 四、系统里没有的东西

诚信的另一半，是承认自己没有什么。下面这些能力，在系统的历史版本里存在过，但在本书写作的版本中已经被移除或下线。本书全书不把它们写成现有功能：

- 策略页的「策略诊断」（因子 IC 时序图）。前端按钮与弹窗、后端绑定接口、相关源码文件全部删除。
- 财务指标查询。绑定接口与相关数据结构已删除。
- 「查询前复权 K 线」入口。前端输入框、查询按钮、结果表与后端方法均已删除。（前复权视图仍在，用于内部计算，但没有对外查询入口。）
- 多因子回测。已下线，回测仅支持单因子。
- 通达信数据的自动执行。已取消，改为仅手工补位。
- 启动时自动刷新策略指标。已取消，改为每日盘后调优写入或手动刷新。
- 启动时自动重跑当日已完成的复盘与方案重建。已取消，改为按复盘阶段定时执行，重启不重跑。
- 估值快照与集合竞价快照（同花顺官方）。相关方法、结构与解析逻辑全部删除。

如果你在别处读到「这个系统能做多因子回测」或「能查前复权 K 线」，请以这份清单为准。

I.5 五、如实记录的两处系统缺陷

一本书如果只讲系统的优点，那它就不是在写工程，是在写广告。本书在写作过程中，通过对照源码与真实数据，发现了两处存在的缺陷。它们不影响本书的结论，但影响你对这套系统的信任边界，所以必须写出来。

I.5.1 第一处：风险指标的历史法取数存在顺序缺陷

系统的行情查询函数 `GetRecentPrices` 按日期降序返回数据（最新的在前），而 CIO 模块里计算收益率的函数 `computePositionReturns` 拿到数据后没有重新排序，直接按相邻两点作差。对 121 根日线来说，这等价于把每一个收益率都做了符号翻转：

$$\hat{r}_i = \frac{c_i - c_{i-1}}{c_{i-1}} = \frac{-r_i}{1 + r_i}, \quad r_i = \frac{p_i - p_{i-1}}{p_{i-1}}.$$

旁证很硬：同一个查询结果在另一个消费方（组合数据准备模块）里，是被显式升序重排之后才使用的——说明「需要重排」这件事在系统内部是有共识的，只是其中一条路径漏了。

在沪深 300 的真实数据上（2026-04-01 至 2026-09-23，121 根日线），此处缺陷造成的偏差是：

指标	正确值	系统输出	偏差
VaR（95%）	0.021441	0.018362	低估 14.4%
CVaR（95%）	0.028902	0.024911	低估 13.8%
VaR（99%）	0.030254	0.029733	低估 1.7%
CVaR（99%）	0.033147	0.031724	低估 4.3%

最直接的验证是：系统返回的尾部第 7 个数是 -0.018362 ，把它反解回真实收益率是 $+0.018031$ ——那恰好是当期的第 7 大涨幅；而正确的分位数应当取第 7 大跌幅 -0.021441 。

不受影响的量：日波动率（ 0.012723 对 0.012765 ，几乎不变，因为符号翻转不改变平方项）、相关系数（全体资产被同一个排列重排，配对关系不变， 0.948144 对 0.948790 ）。**受影响的量：**年化夏普比率、最大回撤，以及所有基于收益序列方向的风险指标。影响范围限于三条调用路径：中台的风险计算与回撤熔断走的是另一条取数路径，不受影响。

本书的处理方式是：**记账，但不改系统**。附录 F 按理论正确口径给出全部推导，并在对照表里逐项标明这个差异。

I.5.2 第二处：风险引擎把「数据缺失」和「风险为零」输出成同一个值

系统的风险引擎里，计算 VaR 与 CVaR 的函数在两种情况下都会直接返回 0：样本为空，或者传入的频度参数不在「日频 / 月频 / 年频」三者之中。

这意味着：「我没有数据」和「我的风险是零」在输出上无法区分。第 22 章讲过，风险为零是一个极强的结论——它意味着几乎不可能亏钱。而这里，它可能只是意味着「今天数据没接上」。

这不是计算错误，是接口设计问题：它把两种语义截然不同的情况压进了同一个返回值。正确的做法应当与系统自己的原则一致——数据不可用时明确报错，而不是返回一个 0 让下游当成事实。

I.6 六、你自己怎么复现

上面每一个数字都可以由你自己复现。本书的导出与复算脚本都在 `book/data/scripts/` 目录下：

- `export_real_data.py`——只读连接行情库，把正文引用的真实数据导出为 JSON；
- `export_for_rest.py`——第二批导出，供后半部章节与附录使用，口径与前者完全一致；
- `recompute_risk.py`——复算附录 F 的全部风险指标数字，逐字照抄系统实现的取数与算法；
- `make_stubs.py`——只用于生成章节骨架，不产出任何数字。

所有脚本一律以只读方式连接数据库（`duckdb.connect(DB, read_only=True)`），不会修改系统里的任何数据。运行环境需要 Python 3 与 `duckdb` 包。

验证的入口只有一条命令：

```
1 python book/data/scripts/recompute_risk.py
```

代码 I.1 复现附录 F 全部风险指标数字的一条命令

它会打印出第一节那张偏差对照表的每一行。如果打印出的数字与本书不一致，以你的运行结果为准——因为你的数据库可能就是更新过的，而本书的数字带着「2026-09-24」这个时间戳。

I.7 七、免责声明

请务必读这一段

一、本书不是投资建议。它是一本讲「一台 AI 量化交易系统是怎么构造出来的」的技术书，讨论的是因子、回测、仓位、风控这些方法论问题。书里出现的任何股票代码、任何数值、任何策略表现，都只用于说明系统如何工作，不构成对任何证券的推荐、要约或投资建议。

二、历史数据不预示未来。全书所有回测结果、所有指标，都是对已经发生的历史的统计描述。一

个策略在过去五年有效，与它在未来五年有效之间，没有任何逻辑上的必然联系。第 15、17、18 三章整章都在讲这件事会怎样骗人。

三、这套系统是研究工具，不是赚钱机器。它把投资流程工程化、把决策过程留痕化、把风险约束代码化——这些都能显著降低「拍脑袋」和「情绪化」带来的损失，但它不能预测市场。第 1 章就说过：AI 在这套体系里的角色是**流程执行者**，不是股神。

四、真实资金操作的风险由使用者自行承担。若你把这套系统接入实盘，所有交易决策的后果，包括本金损失，都由你自己承担。系统提供的模拟、暂停建仓、紧急停止等安全开关，是为了降低误操作风险，不是收益保证。

五、本书对数据的承诺是有限的、但明确的。我们承诺：书里的每一个数字都标注了来源与核对时间，凡是有出处的都写清了出处，凡是取不到的都留了白。我们不承诺：这些数字在你读到时仍然成立——数据库会更新，代码会改动，系统会演进。所以第六节才把复现方法交给你。

六、最终判断权在你手里。这本书能做到的，是把一台正在运行的系统拆开给你看，并告诉你每一颗螺丝为什么拧在那里。至于要不要照着拧、拧多紧——那是你自己的事，也该是你自己的事。

宁可它停下来告诉你「我不知道」，
也不要它安静地给你一个错的答案。

系统标识符索引

关于“查”，本书前面已经有两份附录了。附录 G 的术语表回答这个词是什么意思，附录 H 的事故索引回答这个毛病怎么修。这一份回答第三个、也是在真实使用里被问得最多的问题：这个东西，书里还有哪些地方讲过？

J.1 怎么用这份索引

三条约定，读之前先看一眼：

1. 出处写的是章节号，不是页码。数字就是章号（14 = 第 14 章），大写字母是附录编号（F = 附录 F）。之所以不写页码：页码是排版的产物，本书改一次行距、加一张图，后面几百页的页码全体平移，索引就全错了；章节号是源文件本身的属性，不会漂移。代价是定位粒度粗一级，收益是这份索引不需要随每次重排而重编。
2. 索引里的键，是系统里的真实标识符。等宽字体写出来的每一个名字，都可以直接复制到源码目录里去检索。索引不对它们做任何改写、转写或翻译。
3. 索引收了什么、没收录什么，下面第二节逐条说明。

举三个真实例子，说明这份索引在什么时候有用：

- 你在第 19 章读到字段 `position_rate`，想知道它在别处还出现过没有。查索引：14, 19, 28, G——它第一次被定义在第 14 章，第 28 章讲组合时又用了一次，附录 G 的术语表里有一条。
- 你想知道系统那张六维判势日表 `market_sixdim_daily` 到底牵扯了多少地方。查索引：16 个出处（第 4、5、6、7、10、11、14、19、20、21、22、28、33 章，以及附录 A、B、E）。它是全书被引用最广的一张表——这本身就说明六维判势在系统里的位置。
- 你在系统里看到一个函数 `select_next_day_strategy`，想知道本书讲了没有。查索引：1, 4, 16, 30, 32。第 1 章是总览提了一句，第 4 章讲智能体时正式介绍，第 16 章讲策略生成、第 30 章讲实操、第 32 章讲它出错时怎么办。

J.2 收录范围与规模

类别	条目数	收录范围与不收录的理由
工具名	15	智能体实际能调用的函数。系统里注册了 25 个工具入口，其中 15 个在本书正文里出现过——只有出现过的才收录，索引不是接口文档。
数据表名	74	两套库（DuckDB 行情库 + SQLite 运行库）里被本书提到的表。系统 58 张业务表 + 行情库 16 张表，本书覆盖了其中与策略链路相关的那一部分。
字段名	83	表结构里的真实字段。系统字段数以千计，本书只收真正在正文里被讲解或用作判断依据的那些。
源码文件与包	152	正文脚注与「真实出处」小节里指过的文件路径。路径末尾没有文件名的是包目录。
合计	324	去重后的标识符总数，其中 108 个出现在 2 个及以上的章节或附录里。

统计口径：对 `chapters/` 与 `appendices/` 下全部 `.tex` 源文件做文本抽取，按标识符去重。数字随正文增删而变，可用 `python gen_identifier_index.py --check` 当场复核；本节数字与下面索引的内容出自同一次抽取，不会互相矛盾。

有三件事必须说清楚，否则这份索引会给人一个错误的印象。

第一，它不收录 `code` 宏里的内容。全书 `code` 宏用了 2278 次、去重后有 1289 个不同的字符串，但其中绝大多数是代码片段而不是标识符——赋值语句、切片表达式、格式化字符串都算在内。把代码片段做成索引没有意义，读者不会去查一条语句，也查不到。所以索引只收那四个「给东西起名字」的宏：工具名宏 `tool`、表名宏 `dbtab`、字段名宏 `field`、路径宏 `sysfile`。你如果确定要找一个函数名（比如 `OutputGuard.ValidateOutput`），而它恰好只出现在 `code` 宏里，那它不在本索引中，请改用 PDF 阅读器的全文搜索。

第二，它是机器生成的，不是手写的。索引里的每一条都来自对源文件的文本抽取，没有任何人工增删。生成器还会做一次双向核对：源文件里出现的标识符必须在索引里，索引里的标识符也必须在源文件里，两边条数必须相等。这个核对每次生成都跑，不通过就写出告警。换句话说，你在这份索引里找不到的，正文里确实没有。

第三，它的构建过程真的抓到过东西。生成器最初的设计里有一条「同一标识符被两种宏引用即告警」的规则——同一个名字，不可能既是一个源码文件又是一个数据表。这条规则当场抓出了两处真实的宏用错：第 11 章把表名 `FactorQuality` 误写成源码文件宏，第 20 章把表名 `order_book_daily` 误写成字段宏。两处都已修正。这类错误不影响编译、不影响版面、也不会进任何日志，只有把全书的名字放到一起数才看得见。

J.3 工具名（智能体可调用的函数）

智能体实际能调用的函数，注册在工具的命名空间下。想知道某一章讲的机制在系统里到底是哪个函数在干，从这里查。

<code>factor_review</code> 14	<code>get_market_data</code> 4
<code>factorReviewTool</code> 32	<code>get_tdx_common_stocks</code> 4
<code>get_cninfo_info</code> 4	<code>get_tdx_market_stats</code> 4

get_tdx_stock_data 1, 4	run_backtest 1, 4
market_sixdim_detect 1, 4, 19, E	search_market 4
optimize_portfolio 1, 4, 23	search_tdx_stocks 1, 4
order_cost_tool 14	select_next_day_strategy 1, 4, 16, 30, 32
place_trade 4	

J.4 数据表名

两套库里的真实表名：行情与基本面在 `data/stock.duckdb`，系统状态与交易在 `database/quantpilot.db`。

agent_activities B	FactorQuality 11, 13, 14, 15, 32
agent_decisions B	instruments B
agent_documents B	investment_decisions B
agent_performances B	investment_mandates B
agent_permissions B	investment_plans 26, B
agent_sessions B	investor_profiles B
agent_task_logs 10, 33	llm_call_logs B
agent_tool_calls B	market_indices B
agents B	market_risk_reports B
alpha_stores B	market_sixdim_daily 4, 5, 6, 7, 10, 11, 14, 19, 20, 21, 22, 28, 33, A, B, E
audit 10	ohlcv B
audit_logs B	order_book_daily 12, 14, 20, 22, 23, 25, B
backtest_configs B	orders B
backtest_results B	plan_approvals B
cash_flows B	policy_logs B
cio_decision_logs B	portfolio_daily_stats 29, B
claim_records B	portfolio_optimizations B
conversation_messages B	portfolios B
conversations B	position_manager_configs B
daily_reviews B	position_snapshots 29, B
daily_strategy_plans 16, 30, 32, B	positions B
decision_logs 28	risk_checks B
decision_traces B	risk_limits B
documents B	settings B
events B	sqlite_sequence B
factor_calibration_histories B	stock.adj_event 25, B
factor_qualities B	

stock.adj_factor 25, B	system_configs B
stock.financial_report 11, 25, B	tasks B
stock.ohlcv 11, 17, 25, B	team_members B
stock.ohlcv_qfq 25, B	tradeable_stock_logs B
stock.stock_basic 25, B	tradeable_stocks 10, 26, 31, 32, B
stock.stock_daily 25	trades B
stock.trading_calendar 25, B	users B
stock_basic 20, 25	watch_stocks B
strategies B	workflow_instances B
strategy_candidates B	workflows B
strategy_stock_params B	

J.5 字段名

表结构里的真实字段名。想在界面上或导出文件里对上号，从这里查。

abs_rank_ic 14	data_sources 10
Accuracy 14	debate 10, 28
adjusted_total_score 14, 19, 28	debate.direction 28
agent_executor_ready 24	Decay 14
agreement 7, 28	decay_20d_minus_1d 14
algorithm_records 10	decision_steps 10
amount 17	deliverable_data 10
ann_date 11	deliverable_name 10
AutoCalibrate 14	deliverable_type 10
bear[] 28	Detail 13, 14, 15
bottom_group_ret 14	dim_scores 19
bull[] 28	direction 8
conclusion 10	effective_position_rate 7, 10, 28
confidence 6, 7, 10, 28	ensemble 8, 10, 28
config_ready 24	ensemble.direction 28
conflict_count 19, 28	evidence 10
conn_ai_ready 24	external 19
conn_db_ok 24	factor_name 11, 13, 14, G
conn_ds_ok 24	FactorName 15
Coverage 14	feedback 10, 28
coverage_pct 14	generated_at 10
daily_pnl 29	governed_by_sixdim 10

kelly_position_scale 7, 10, 28	signal_source 10
llmReview 28	SixDim 21
llmReview.action 28	skeptic_concerns 10, 28
long_short_spread 14	skeptic_verdict 10, 28
market_direction 6, 10, 28	sources 10, 15, 19, 20, 33
market_read 10	split_half_ic 14, 15
market_tag 14, 28	split_half_stable 14
MaxNotional 21	SplitHalfIC 14, 15
net_score 8, 10, 28	SplitHalfStable 14, 15
pearson_ic 14	Stability 14
portfolio_ready 24	stability_std 14
portfolio_warm 24	suggested_position_rate 7, 10, 28
position_rate 14, 19, 28, G	suggestion 28
preferred_stocks 6, 10, 28	summary 8
Quality 14, 15	target_position_rate 6
rank_ic 14	TargetWeight 21
RankIC 14	top_accuracy 14
raw_total_score 19, 28	top_group_ret 14
reason 28	votes 8, 10, 28
score 28	

J.6 源码文件与包

源码路径。路径末尾没有文件名的是包目录。

agent.dll A	config/ 24
AIQuant/ A	config/config.json 5, 24, 26, 32, 33, A, E
AIQuant/internal/ A	cost_model.go A
AIQuant/internal/config/config.go 27	crash_dump.txt 9
App.go 24, 33, A	daily_strategy_tool.go 4
app_market.go 21	data/ 24
app_planner.go 26, 32	data/stock.duckdb 4, 21, 22, 24, 25, 33, A, B, F
app_review.go 26	database/ 24
app_scheduler.go 25	database/quantpilot.db 14, 24, A, B
app_settings.go 31	deliverables/ 10
backtest_gap_tools.go 4	deliverables/< 日期 >/ 33
book/data/scripts/ I	docs/用户操作手册.md 2, 27, 33
cninfo_tools.go 4	docs/系统功能详细设计书.md 3

export_for_rest.py I
 export_real_data.py I
 factor_engine.go A
 factor_review_tool.go 4, 14
 grid.go A
 https://api.deepseek.com 24
 indicators.go A
 internal/ A
 internal/agentdll/loader_windows.go 9
 internal/agentworkflow/event_bus.go A
 internal/agentworkflow/models.go 1, 2, 3, 27, A, B, C, G
 internal/agentworkflow/permissions.go 1, 2, 27, A, C
 internal/agentworkflow/workflow_engine.go 5, 9, 33, A
 internal/audit 33
 internal/audit/service.go 10
 internal/backtest/cost_model.go 17, 18, 29, D
 internal/backtest/engine.go A
 internal/backtest/grid.go 17, 30, D
 internal/backtest/indicators.go 16, D
 internal/backtest/service.go 17, 18, D
 internal/backtest/strategies.go 16, 18, D
 internal/backtest/strategy_config.go 16, D
 internal/brain/agents/workflow.go F
 internal/brainhost/team_tools.go 1, 4, A, C
 internal/cio/cio.go 10, 13, 14, 15, 21, 22, 26, E, F
 internal/cio/ensemble.go 8, A
 internal/cio/ensemble_test.go 8
 internal/cio/proposal.go 6, 7, 8, 9, 10, 21, A, F
 internal/config/config.go 2, 24, 27, 32, A, E
 internal/data/duckdb.go 20, 22, A, F
 internal/data/financial_sync.go 25
 internal/data/financial_ths.go 25
 internal/data/maintenance.go 25, A
 internal/data/sqlite.go B
 internal/data/ths_adjfactor.go 25, A
 internal/data/ths_calendar.go 25, A
 internal/data/ths_dailyk.go 25
 internal/factors/engine.go 13
 internal/intelligence/risk_engine.go 22, A, F
 internal/marketsixdim 28
 internal/marketsixdim/fetcher.go 19, 20, A, E
 internal/marketsixdim/persist.go 19, 20, 21, A, E
 internal/marketsixdim/sixdim.go 19, 21, A, E, G
 internal/orchestrator/orchestrator.go G
 internal/orderbook/orderbook.go 12, 22, 23, 25, 31, A
 internal/plannerhost/planner.go 26
 internal/policy/engine.go 2, 22, 26, A, F
 internal/port/ C
 internal/port/agent.go 4, 10, 21
 internal/port/port.go 4
 internal/portfolio 14, 17
 internal/portfolio/dataprep.go F
 internal/portfolio/engine.go 22, 23, 29, 31, A
 internal/portfolio/engine_test.go 22
 internal/portfolio/optimizer.go 23
 internal/risk/engine.go 22, A, F
 internal/riskcenter/riskcenter.go 2, 22, F
 internal/scheduler 25
 internal/scheduler/auto_scheduler.go 5, 9, 22, 26, 27, A
 internal/scheduler/output_guard.go 3
 internal/scheduler/task_scheduler.go 3, 5
 internal/screener 30
 internal/screener/ 23
 internal/screener/factor_engine.go 11
 internal/screener/pro_factor_engine.go 11, 12, 22, 23
 internal/screener/screener_service.go 11, 12, 23, 26, A
 internal/screener/smart_composer.go 12, 13, 30

internal/screener/types.go 12, 13	recompute_risk.py I
internal/strategy/service.go 17, 30, 32, A, D	screener_service.go 23
internal/thssdk 25	service.go A
internal/thssdk/ A	single_instance_windows.go 24
internal/thssdk/th.go 25	sixdim.go E
internal/tools/ 4, 14, 23, C	sixdim_tool.go 4
internal/tools/daily_strategy_tool.go 30, 32, A	smart_composer.go A
internal/tools/factor_review_tool.go 13, 14, 15, 32, A	stock.duckdb G
internal/tools/investment_tools.go 22	strategies.go A
internal/tools/position_manager.go 23	tools.go 23, C
internal/tools/sixdim_tool.go 4, A, E	tradeable_pool.go A
internal/tools/tools.go 4, 23	tray_windows.go 24
internal/transparency 33	ui/src/App.tsx 1, 2, 5, 11, 27, A
internal/transparency/tracker.go 10	ui/src/components/InitialLoading.tsx 24
internal/util/goroutine.go 9	ui/src/components/TitleBar.tsx 24
internal/util/portfolio_sizing.go 23	ui/src/pages/ 1, 5, 11, 12
internal/util/trading_calendar.go 25	ui/src/pages/*.tsx A
internal/util/trading_hours.go 5, 23, 27, 29, A	ui/src/pages/Activity.tsx 27
log/ 9, 24, 33, A	ui/src/pages/AITeam.tsx 2, 27
log/crash_dump.txt 9, 24, 33	ui/src/pages/AuditLog.tsx 33
log/quantbot_< 时间戳 >.log 24, 33	ui/src/pages/Backtest.tsx 30
main.go 9, 24, 33	ui/src/pages/CIO.tsx 8, 10, 21, 28
make_stubs.py I	ui/src/pages/Dashboard.tsx 27
optimizer.go A	ui/src/pages/InvestmentPlanner.tsx 26
order_book_tool.go 4	ui/src/pages/Portfolio.tsx 29
persist.go A	ui/src/pages/StockScreener.tsx 30
pro_factor_engine.go 23, A	ui/src/pages/Strategies.tsx 30
quantbot_*.log 33	ui/src/pages/WelcomeWizard.tsx 24
quantpilot.db G	ui/src/pages/Workflow.tsx 27
	ui/src/types/index.ts 27

版本与说明

本书对应的系统版本

项目	内容
系统名称	QuantBot（早期版本名 QuantPilot）
系统版本	v1.6.0
技术栈	Go + Wails v2 + React 18 + TypeScript + DuckDB + SQLite + LLM
运行方式	Windows 单机，单个可执行文件，本地数据
本书数据基准日	2026-09-24
行情库覆盖区间	2021-08-02 至 2026-09-24

关于界面截图

正文中的系统界面截图，全部取自系统**真实运行**时的界面：深色主题，原始分辨率约 2.6K，未作美化、未作裁剪、未作拼接。图里出现的股票代码、持仓数量与金额，就是那一刻数据库里的真实记录。

要说明的是：正文固定的是**图的编号与引用**，界面本身会随版本演进。所以同一张图在你手上的版本里可能略有差异——以你屏幕上的为准。

本书共插入 10 张界面截图，分布在：

- 第 24 章——设置 • 关于（版本与组件状态）；
- 第 25 章——设置 • 数据源；
- 第 26 章——投资方案详情；
- 第 27 章——控制中心、AI 团队、实时活动（各一张）；
- 第 28 章——投资管理 • 投资决策中心；
- 第 29 章——组合交易中心；
- 第 30 章——选股引擎、回测。

写作纪律（再次声明）

1. 全书数字分为三类，每一处都当场标注：**真实数据**（直接取自系统运行库）、**复算数据**（按系统实现的同一公式在真实数据上重算）、**数据缺失**（系统库中无有效记录，留白不填）。
2. 凡引用系统实现，均给出文件路径；凡引用系统参数，均给出具体数值与出处。
3. 凡本书作者在写作中发现的、系统曾经写错并已修正之处，收入附录 H，不做隐藏。

复现方式

本书的导出与复算脚本都在 `book/data/scripts/` 目录下。它们一律以只读方式连接数据库，不会修改系统里的任何数据：

- `export_real_data.py`——导出正文第一批真实数据；
- `export_for_rest.py`——导出后半部与附录所需的数据，口径与前者一致；

- `recompute_risk.py`——复算附录 F 的全部风险指标数字；
- `make_stubs.py`——仅用于生成章节骨架，不产出任何数字。

另有四份校验脚本放在 `book/` 根目录下，都不产出数据，只负责在定稿前挡住四类「编译不会报错、但印出来就是错的」问题：

- `check_tex_escapes.py`——检查等宽宏参数里有没有多写转义（会让 `get_market_data` 印成带反斜杠的样子）；
- `gen_identifier_index.py`——生成附录 J 的标识符索引，同时做双向核对，并报告「同一标识符被两种宏引用」这类宏用错；
- `fix_cjk_quotes.py`——把正文里用 ASCII 直引号写的中文引号修正成 “”，并拒绝在引号配不成对时动手；
- `check_book_stats.py`——核对本书自报的各类计数与源文件、版面的实际计数是否一致。

```
1 # 1) 导出真实数据（只读，不修改系统库）
2 python book/data/scripts/export_real_data.py
3
4 # 2) 复算风险指标（附录 F 的每一个数）
5 python book/data/scripts/recompute_risk.py
6
7 # 3) 编译本书
8 latexmk -xelatex book/main.tex
```

代码 J.1 复现本书全部数据的命令

联系方式

本书与所描述的系统都是开源的，欢迎交流与指正。

开源仓库	https://github.com/jeokeo011222/quantbot
读者交流群	QQ 群 179590131（QuantBot Free1 群）



扫码加入读者交流群

免责声明

本书与本书所描述的系统，均仅供学习、研究与量化投资实验使用。程序产生的信号、评分、策略、分析、决策与回测结果，均基于历史数据与模型计算，不保证未来收益，亦不构成任何投资建议。金融市场存在风险，投资须谨慎。

未来已来，但未来不会替你承担风险。